CSE 309 (Compilers)

January 2026 Term

Part II

Khaled Mahmud Shahriar

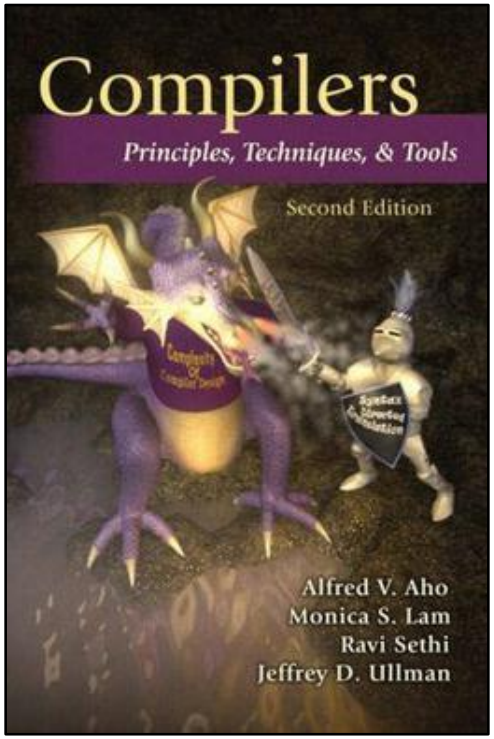
Assistant Professor
Department of CSE, BUET

Text Book

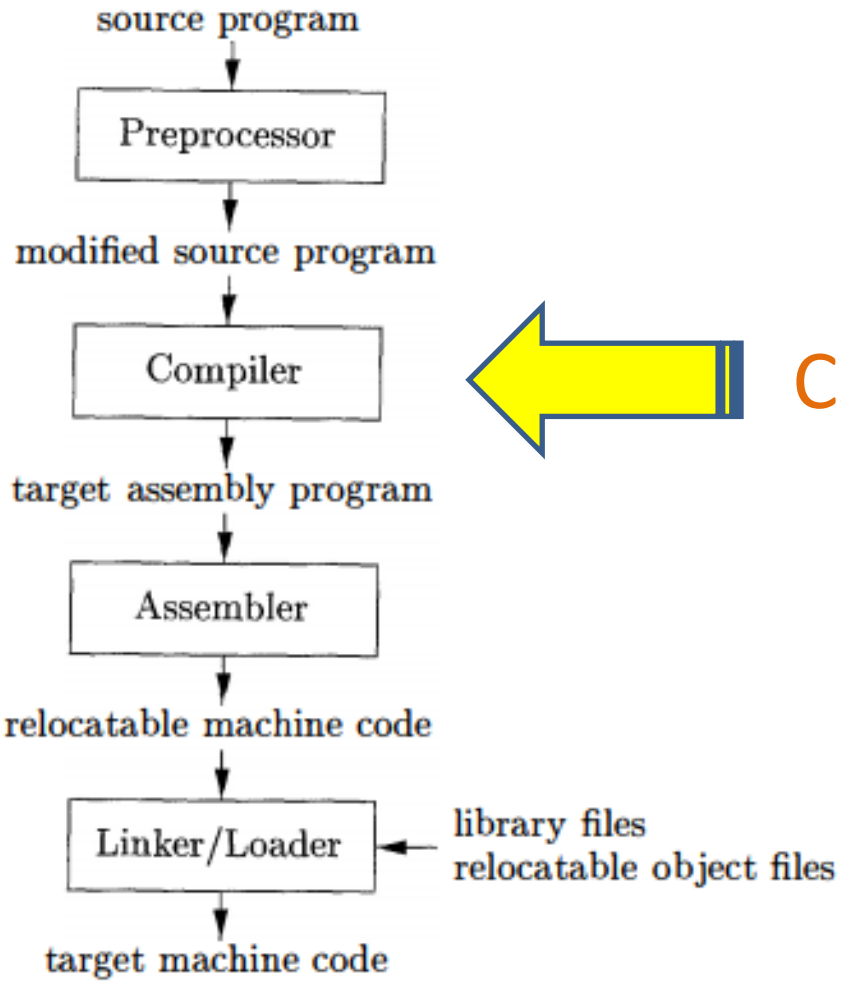
Compilers: Principles, Techniques, and Tools (2nd Edition)

by Alfred Aho, Monica Lam, Ravi Sethi, Jeffrey Ullman

Pearson/Addison Wesley, 2007

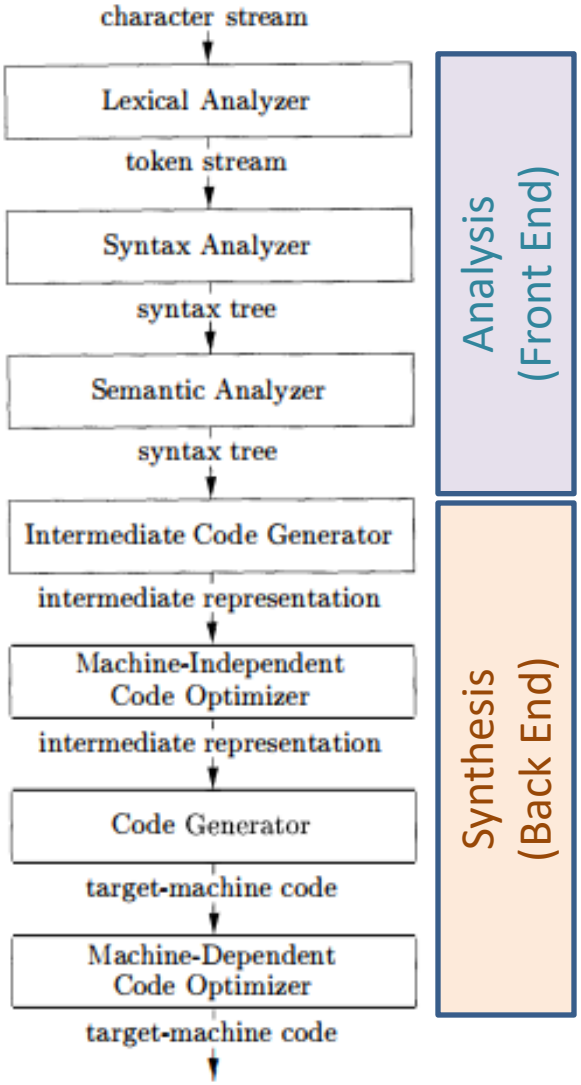


A Language Processing System



CSE 309

Phases of A Compiler



Analysis (Front End)

Synthesis (Back End)

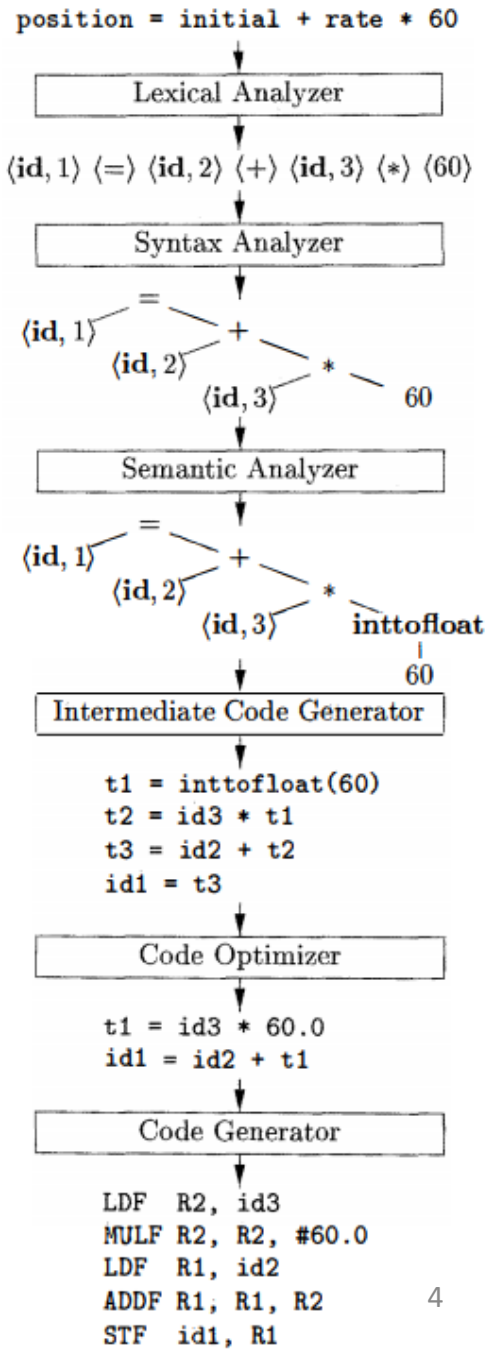
Symbol Table

1	position	···
2	initial	···
3	rate	···

SYMBOL TABLE

Part I (by AMR)

Part II (by KMS)



Topics To be Covered

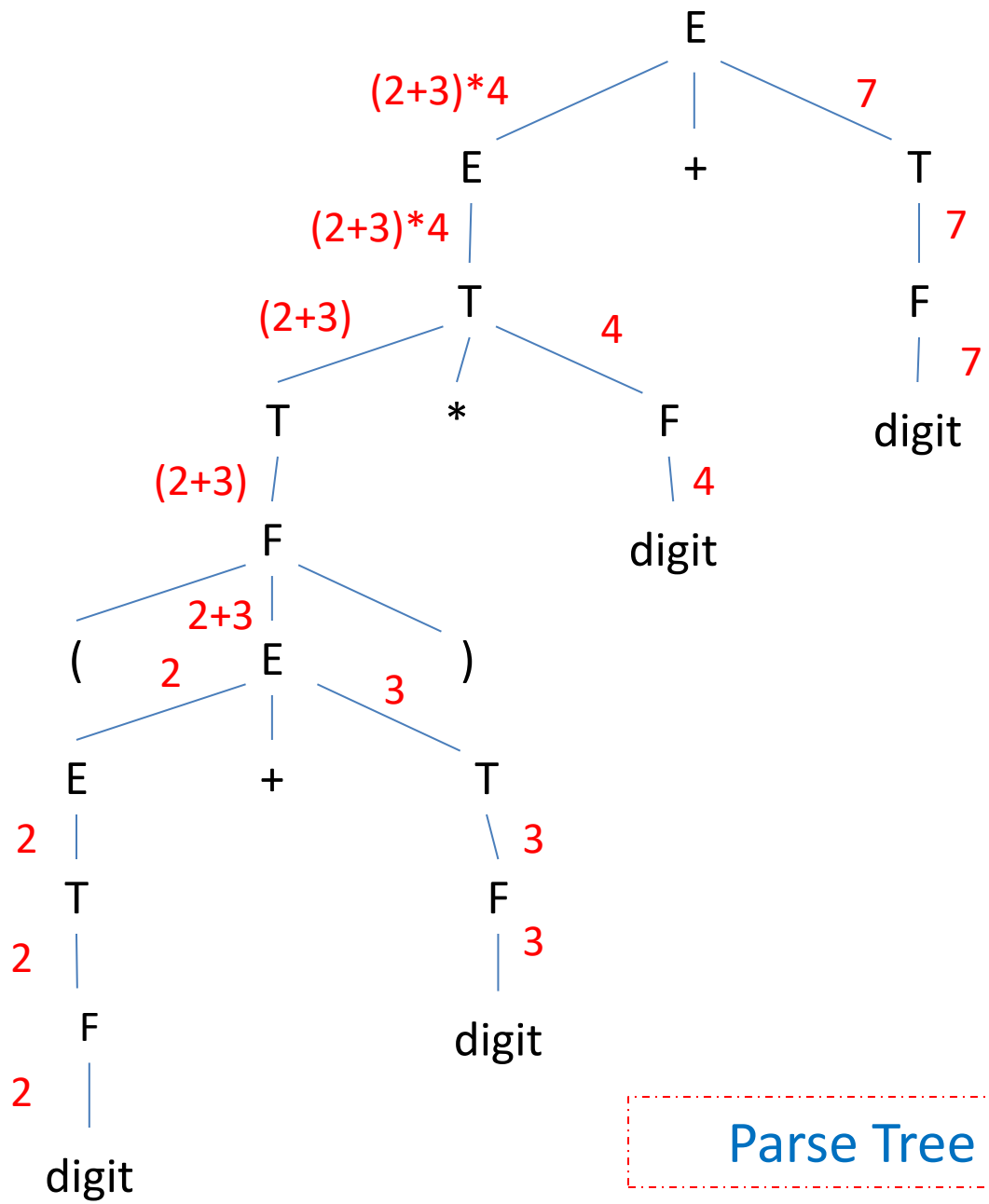
- Syntax-Directed Translation (Chapter 5)
- Intermediate-Code Generation (Chapter 6)
- Runtime Environments (Chapter 7)
- Code Generation (Chapter 8)
- Machine-Independent Optimizations (Chapter 9)

Chapter 5

Syntax Directed Translations

Evaluate: (2+3)*4+7

Production
E → E + T
E → T
T → T * F
T → F
F → (E)
F → digit



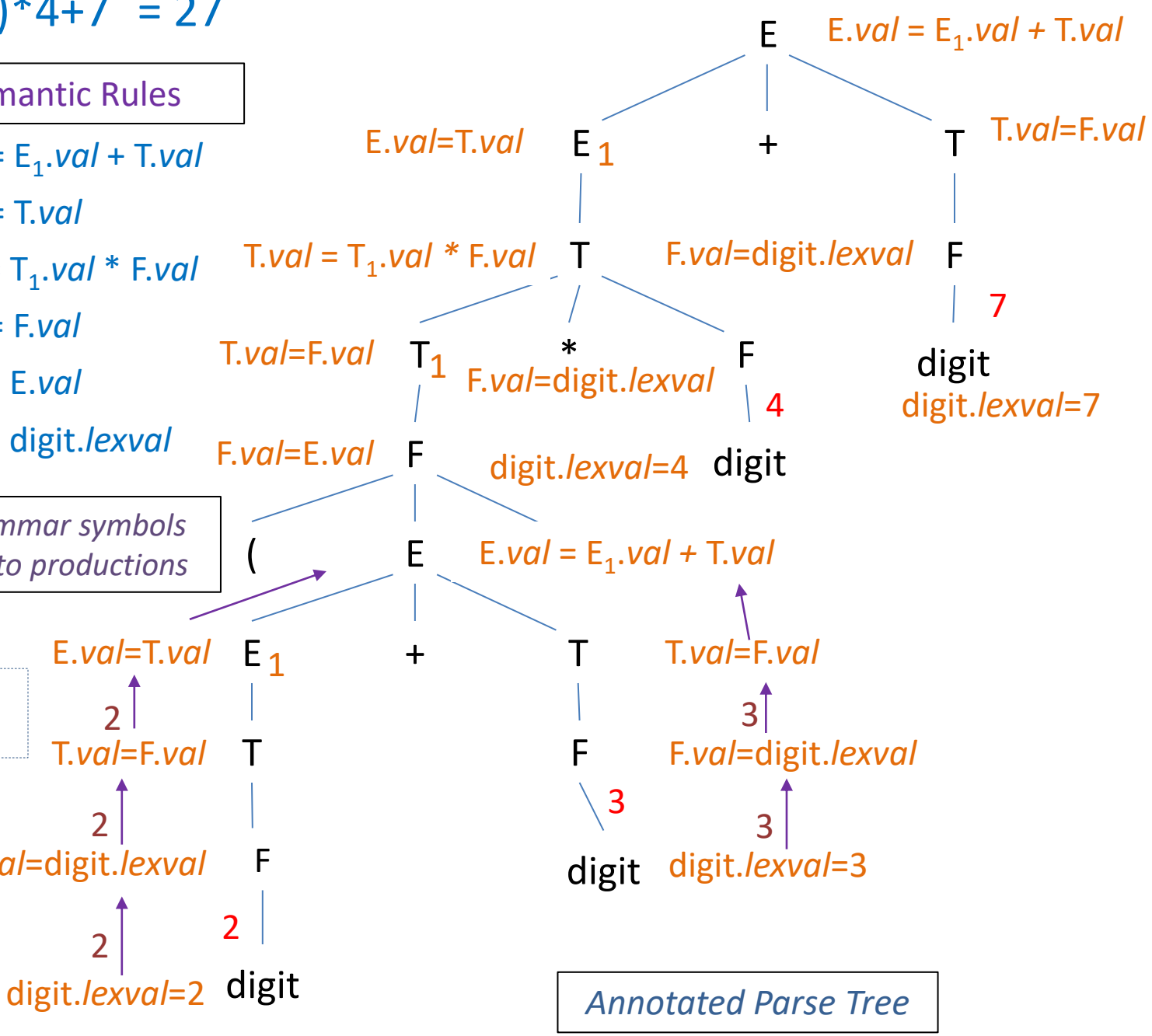
Parse Tree

Evaluate: (2+3)*4+7 = 27

Production	Semantic Rules
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val * F.val$
$T \rightarrow F$	$T.Val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$

Add attributes to grammar symbols
Add calculation rules to productions

Syntax Directed Definition (SDD)



Syntax-Directed Definitions

- A context free grammar together with attributes and rules
  - Attributes associated with grammar symbols
  - Rules associated with productions
    - Rules calculate attributes

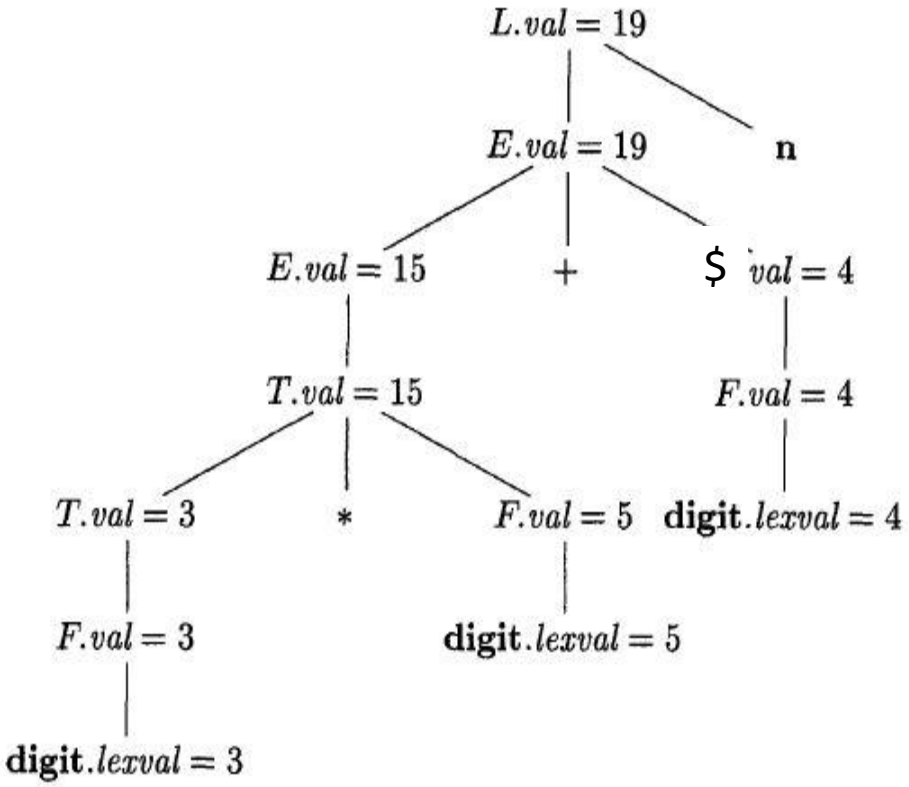
- If *a* is an attribute of grammar symbol *X*, then *X.a* denotes the value of *a* at a particular parse tree node labeled as *X*.

Example: SDD for desk calculator

	PRODUCTION	SEMANTIC RULE
1.	$L \to E$ \$	$L.val = E.val$
2.	$E \to E_1 + T$	$E.val = E_1.val + T.val$
3.	$E \to T$	$E.val = T.val$
4.	$T \to T_1 * F$	$T.val = T_1.val \times F.val$
5.	$T \to F$	$T.val = F.val$
6.	$F \to (E)$	$F.val = E.val$
7.	$F \to$ **digit**	$F.val =$ **digit**$.lexval$

What are the attributes?

- *val* and *lexval*



Annotated parse tree for 3 * 5 + 4 $

Types of Attributes

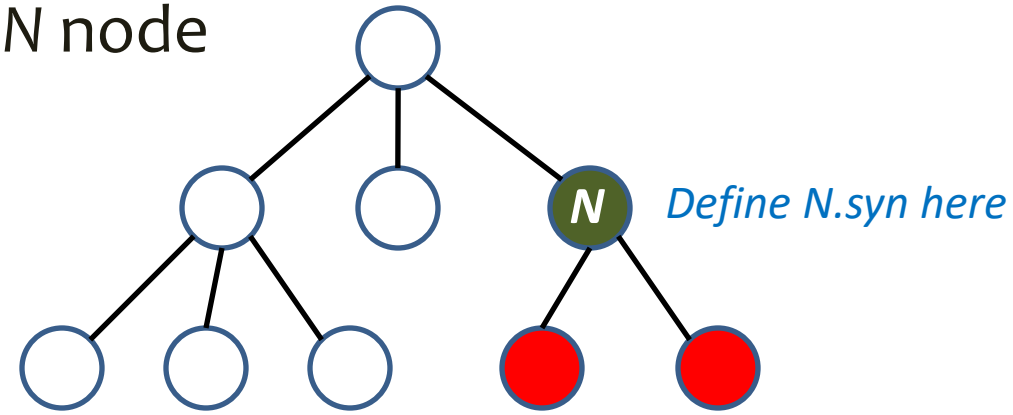
- Need to deal with two kinds of attributes for nonterminals
  - Synthesized Attributes
  - Inherited Attributes

Synthesized Attributes

- Synthesized attributes at node *N* are defined only in terms of the attribute value of the children of *N*, and *N* itself

$N \rightarrow \ldots N_i \ldots$	$N.syn = N_i.syn + N.inh$

  - Defined at rule of *N* node

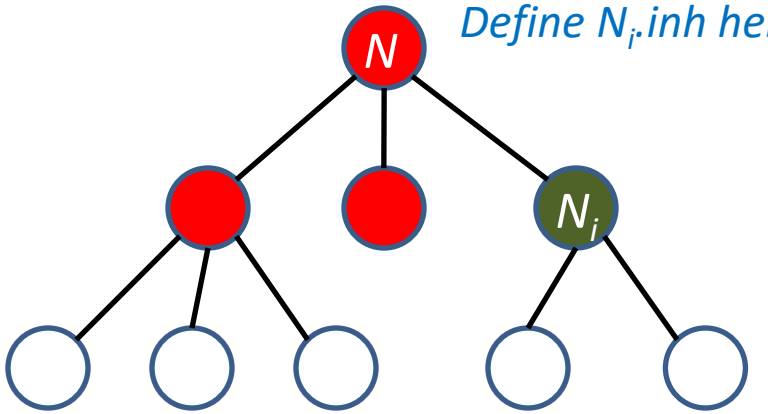


Inherited Attributes

- Inherited attributes at node *N* are defined only in terms of the attribute value *N*'s parent, *N* itself and *N*'s siblings

$N \rightarrow N_1 \ldots N_i \ldots$	$N_i.inh = N.inh + N_1.syn$

  - Defined at rule of parent node of *N* node



Synthesized and Inherited Attributes

- Synthesized attribute at node *N* can be defined in terms of inherited values of *N*
- Inherited attributes at node *N* cannot be defined in terms of attribute values of children on *N* node
- Terminals can have synthesized attributes only
  - Supplied by Scanner

Synthesized and Inherited Attributes

- Are the attributes synthesized or inherited?

	PRODUCTION	SEMANTIC RULE
1.	$L \rightarrow E$ \$	$L.val = E.val$
2.	$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3.	$E \rightarrow T$	$E.val = T.val$
4.	$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5.	$T \rightarrow F$	$T.val = F.val$
6.	$F \rightarrow (E)$	$F.val = E.val$
7.	$F \rightarrow$ **digit**	$F.val =$ **digit**$.lexval$

- *val* and *lexval* are both synthesized

Some Definitions

- Attribute Grammar:
  - SDD with no side effects i.e. no print, symbol table insertion
  - Define value of an attribute purely on other attributes and constants
- S-attributed SDD:
  - All attributes are synthesized
  - Each rule computes attribute for head in terms of attributes of nonterminals from body

Some Definitions

	PRODUCTION	SEMANTIC RULE
1.	$L \rightarrow E$ \$	$L.val = E.val$
2.	$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3.	$E \rightarrow T$	$E.val = T.val$
4.	$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5.	$T \rightarrow F$	$T.val = F.val$
6.	$F \rightarrow (E)$	$F.val = E.val$
7.	$F \rightarrow$ **digit**	$F.val =$ **digit**$.lexval$

- This can be also called Attribute Grammar
- And S-attributed

Evaluating an SDD

- Suppose we have constructed a parse tree first
  - Attributes will be evaluated at each node
  - A parse tree may not be needed to be built in practice
- In what order do we evaluate attributes?
  - For an S-attributed SDD we must evaluate in bottom-up order (like post-order traversal)
  - But what if there are inherited attributes also

Evaluate: 2*3*4

Production
T → T * F
T → F
F → digit

Eliminating Left Recursion

Syntax Directed Definition (SDD)

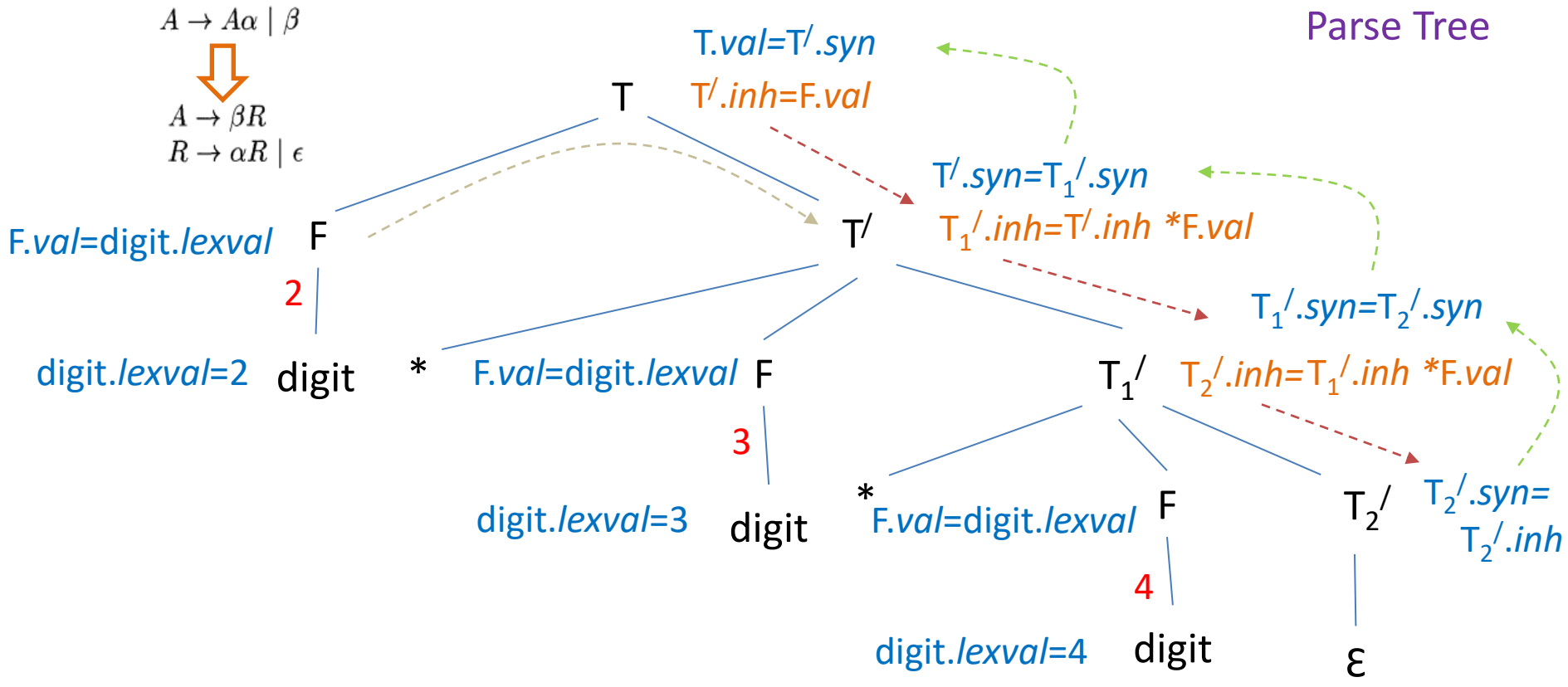
Production	Semantic Rules
T → F T′	T′.*inh* = F.*val*, T.*val*=T′.*syn*
T′ → * F T_1′	T_1′.*inh* = T′.*inh* * F.*val*, T′.*syn* = T_1′.*syn*
T′ → ε	T′.*syn* = T′.*inh*
F → digit	F.*val*=digit.*lexval*

$A \rightarrow A\alpha \mid \beta$

$A \rightarrow \beta R$

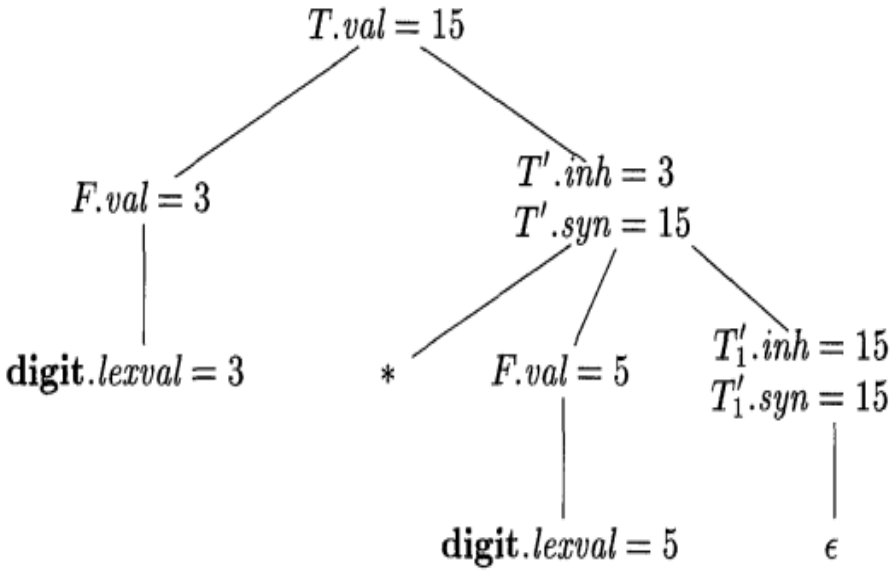
$R \rightarrow \alpha R \mid \epsilon$

Parse Tree



Evaluating Inherited Attributes

	PRODUCTION	SEMANTIC RULES
1)	$T \rightarrow F\ T'$	$T'.inh = F.val$ $T.val = T'.syn$
2)	$T' \rightarrow *\ F\ T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3)	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4)	$F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.lexval$

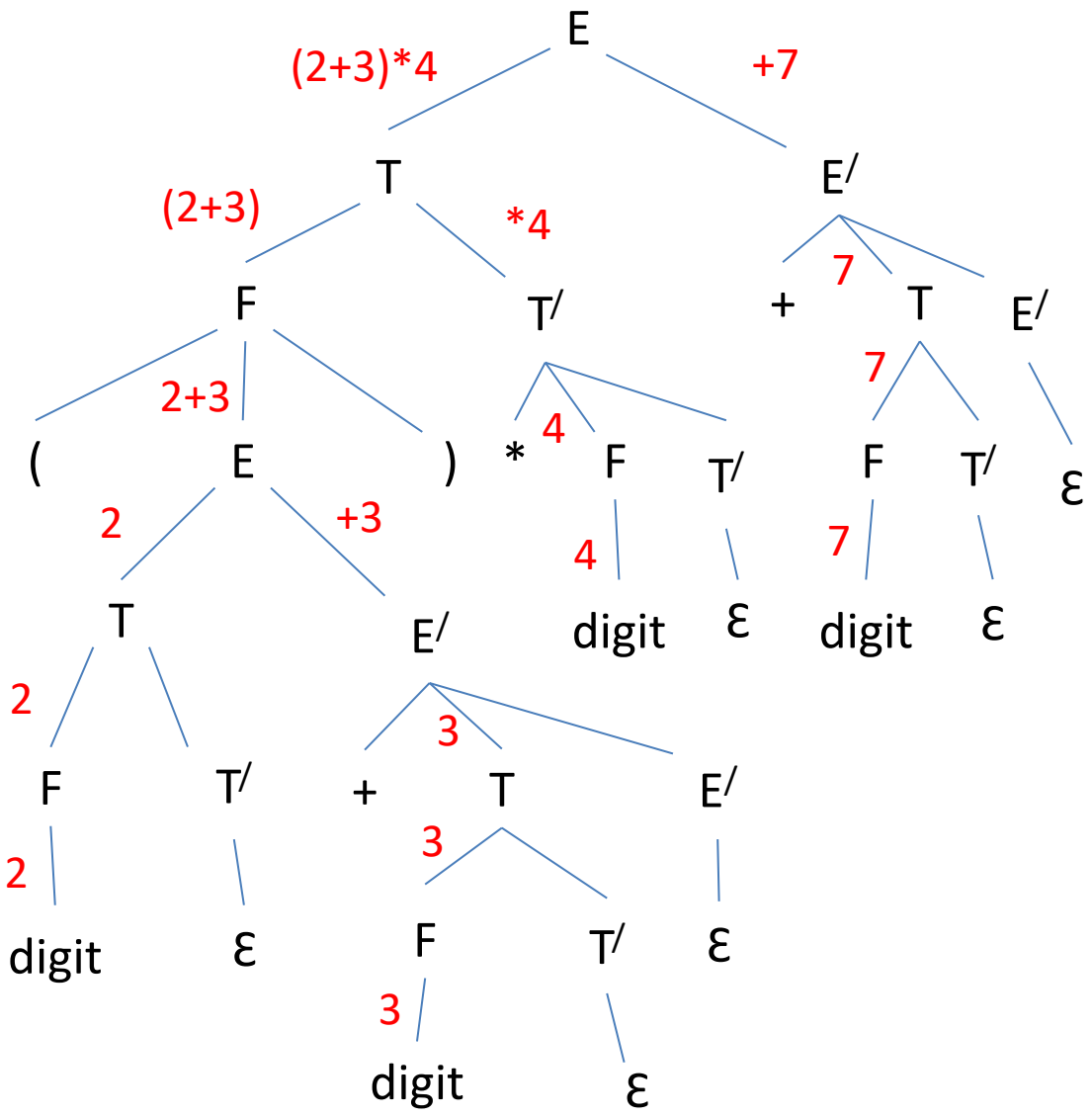


Annotated parse tree for 3 * 5

Evaluate: (2+3)*4+7

Production
E → T E/
E/ → + T E/
E/ → ε
T → F T/
T/ → * F T/
T/ → ε
F → (E)
F → digit

Parse Tree



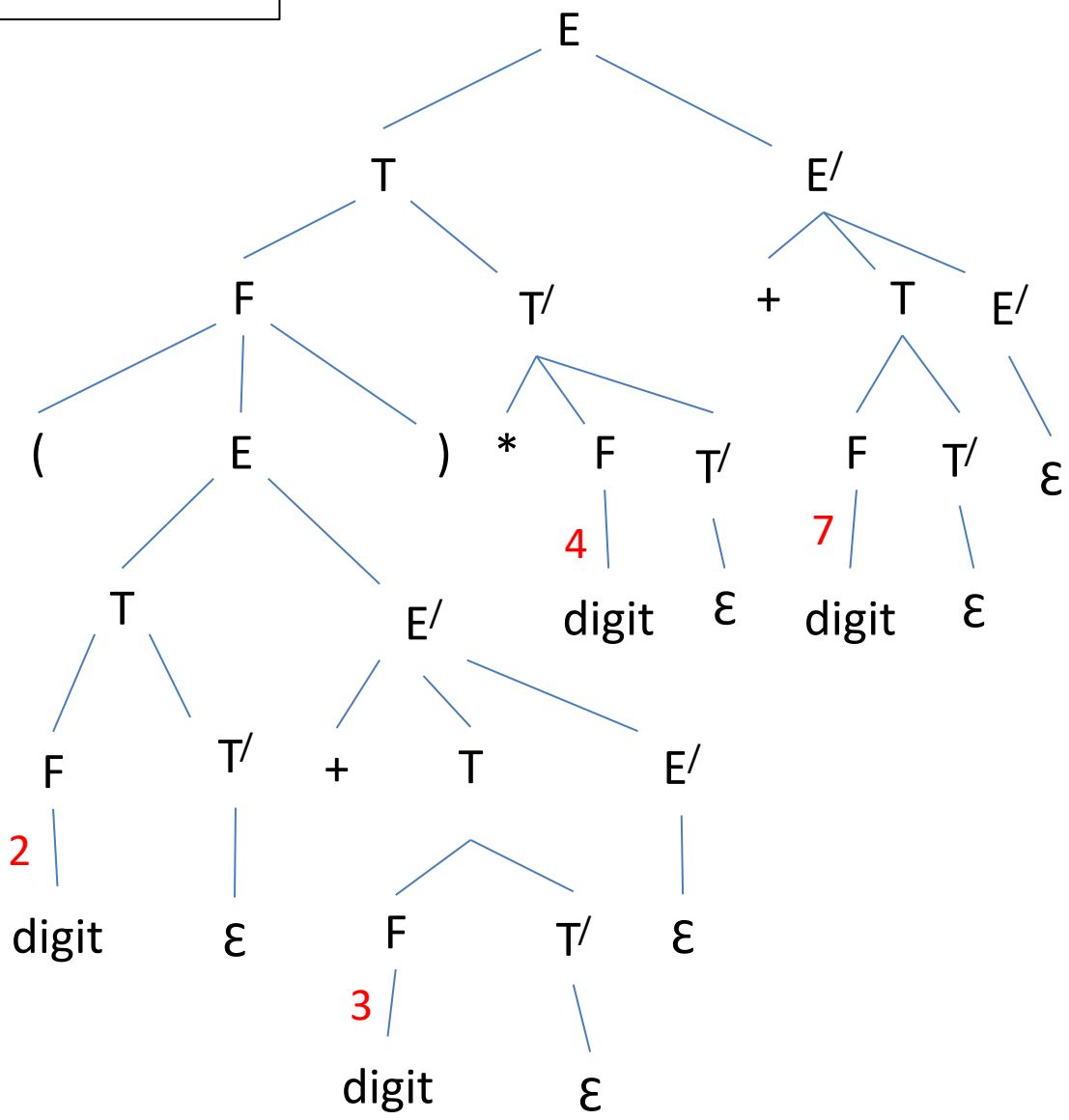
For automated parse tree generation, see:
https://www.cs.princeton.edu/courses/archive/spring20/cos320/LL1/

Evaluate: (2+3)*4+7

Annotated Parse Tree

Production	Semantic Rules
E → T E/	
E/ → + T E/	
E/ → ε	
T → F T/	
T/ → * F T/	
T/ → ε	
F → (E)	
F → digit	

?



Evaluation Order of SDD

- What is the evaluation order for the attribute instance in a given parse tree?

- Useful tool:
  - Dependency Graph

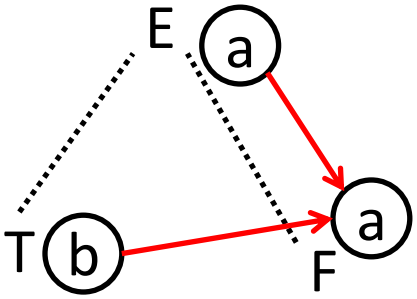
Dependency Graph

- Depicts flow of information among attribute instances in a particular parse tree
- Nodes represent attribute instances
- Nodes for all attributes at a given parse tree nodes
- An edge from one attribute to another means that the first attribute is needed to compute the second
- Edge represent constraint implied by semantic rule

Dependency Graph

Example 1:

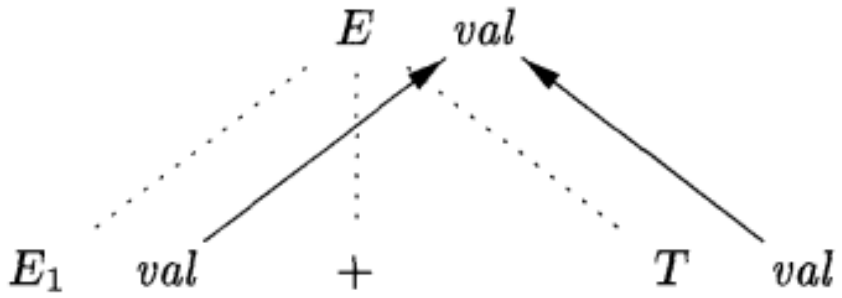
Production	**Semantic Rule**
E -> TF	F.a=E.a+T.b



Dependency Graph

Example 2:

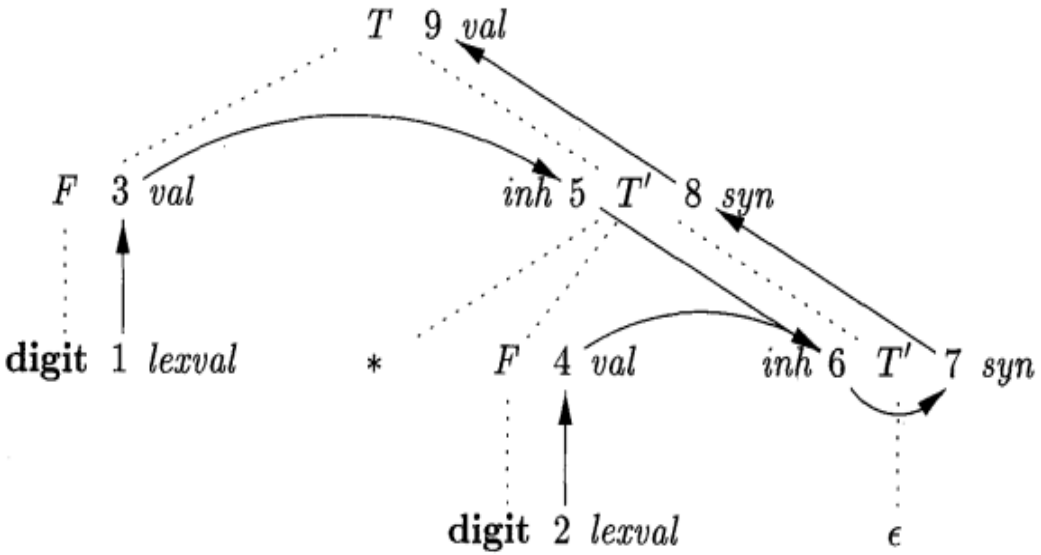
PRODUCTION	SEMANTIC RULE
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$



Dependency Graph

Example 3:

	PRODUCTION	SEMANTIC RULES
1)	$T \rightarrow F\,T'$	$T'.inh = F.val$ $T.val = T'.syn$
2)	$T' \rightarrow *\,F\,T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3)	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4)	$F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.lexval$



Dependency Graph for 1*2

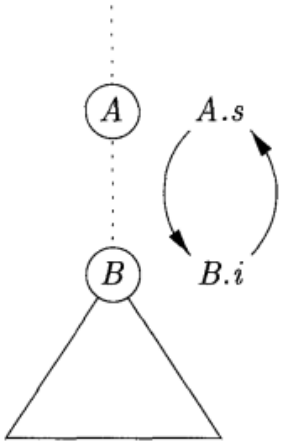
Dependency Graph

- Allowable Order : $N_1, N_2, N_3, \ldots, N_k$ such that, if there is an edge from N_i to N_j then $i<j$
- Called Topological sort
- If a directed graph is acyclic, then there is always at least one topological sort

Evaluating an SDD

- Not every time we will find an evaluation order
  - Circular dependency

PRODUCTION	SEMANTIC RULES
$A \rightarrow B$	$A.s = B.i;$
	$B.i = A.s + 1$



Well Behaved SDD classes

- Difficult to tell whether there exist any dependency graph with cycle

- Safe way: Use classes of SDD that guarantees evaluation order
  - S-attributed SDD
  - L-attributed SDD

S-attributed SDD

- We already know!
  - All attributes are synthesized
  - Can be calculated in bottom up order
    - Post Order Traversal on Parse Tree
  - Can be implemented during bottom-up parsing, without creating tree nodes explicitly

L-attributed SDD

- For $A \rightarrow X_1 X_2 X_3 \ldots X_n$, $X_i.a$ is an inherited attribute computed by a rule associated with this production
  - The rule may use
  i. Inherited attribute associated with head

$X_i.a = f(A.inh, \ldots)$, where *inh* is an inherited attribute of *A*

L-attributed SDD

- Between attributes of a production body, dependency graph edges can go from left to right

- Attributes can be
  1. Synthesized
  2. Inherited with some restrictions

L-attributed SDD

- For $A \rightarrow X_1 X_2 X_3 \ldots X_n$, $X_i.a$ is an inherited attribute computed by a rule associated with this production
  - The rule may use
  ii. Either inherited or synthesized attribute associated with occurrence of $X_1, X_2, X_3, \ldots, X_{i-1}$

$$X_5.a = f(X_2.a, X_3.b, \ldots)$$

L-attributed SDD

- For $A \rightarrow X_1 X_2 X_3 \ldots X_n$, $X_i.a$ is an inherited attribute computed by a rule associated with this production
  - The rule may use
  iii. Either inherited or synthesized attribute associated with this occurrence of X_i, without creating cycle

$$X_5.a = f(X_5.b, \ldots)$$

L-attributed SDD

- Example of L-attributed SDD

	PRODUCTION	SEMANTIC RULES
1)	$T \rightarrow F\, T'$	$T'.inh = F.val$ $T.val = T'.syn$
2)	$T' \rightarrow * F\, T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3)	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4)	$F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.lexval$

L-attributed SDD

- *val* (of both *T* and *F*), *syn* and *lexval* are synthesized

	PRODUCTION	SEMANTIC RULES
1)	$T \rightarrow F\ T'$	$T'.inh = F.val$ $T.val = T'.syn$
2)	$T' \rightarrow *\ F\ T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3)	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4)	$F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.lexval$

L-attributed SDD

- *inh* is inherited

	PRODUCTION	SEMANTIC RULES
1)	$T \rightarrow F\,T'$	$T'.inh = F.val$ $T.val = T'.syn$
2)	$T' \rightarrow *\,F\,T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3)	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4)	$F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.lexval$

L-attributed SDD

- *inh* is inherited

	PRODUCTION	SEMANTIC RULES
1)	$T \rightarrow F\,T'$	$T'.inh = F.val$
		$T.val = T'.syn$
2)	$T' \rightarrow *\,F\,T'_1$	$T'_1.inh = T'.inh \times F.val$
		$T'.syn = T'_1.syn$
3)	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4)	$F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.lexval$

Rule 2

Rule 1 & 2

L-attributed SDD

- Is the following SDD S-attributed or L-attributed?

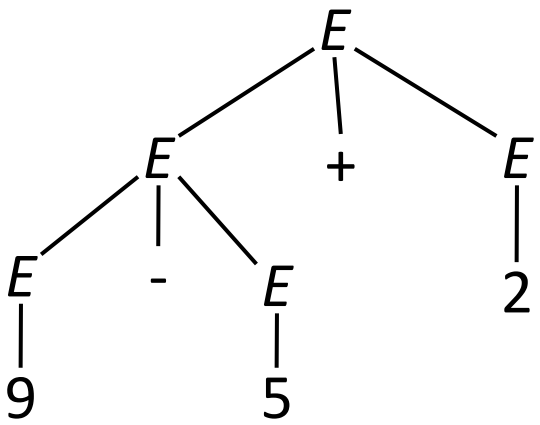
PRODUCTION	SEMANTIC RULES
$A \rightarrow BC$	$A.s = B.b$ $B.i = F(C.c, A.s)$

- NO! Why?

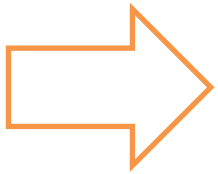
SDD Application: Construction of AST

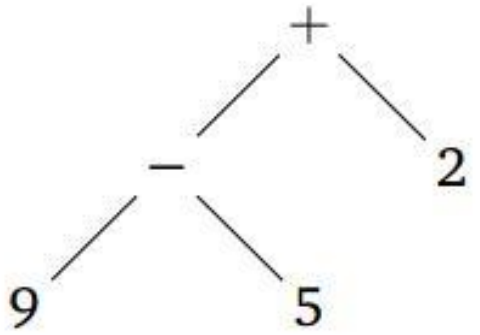
Tree	Internal Nodes
Syntax Tree	Language Construct
Parse Tree	Non-terminals

$$E \rightarrow E + E \mid E - E \mid \textbf{digit}$$



Parse tree for 9-5+2





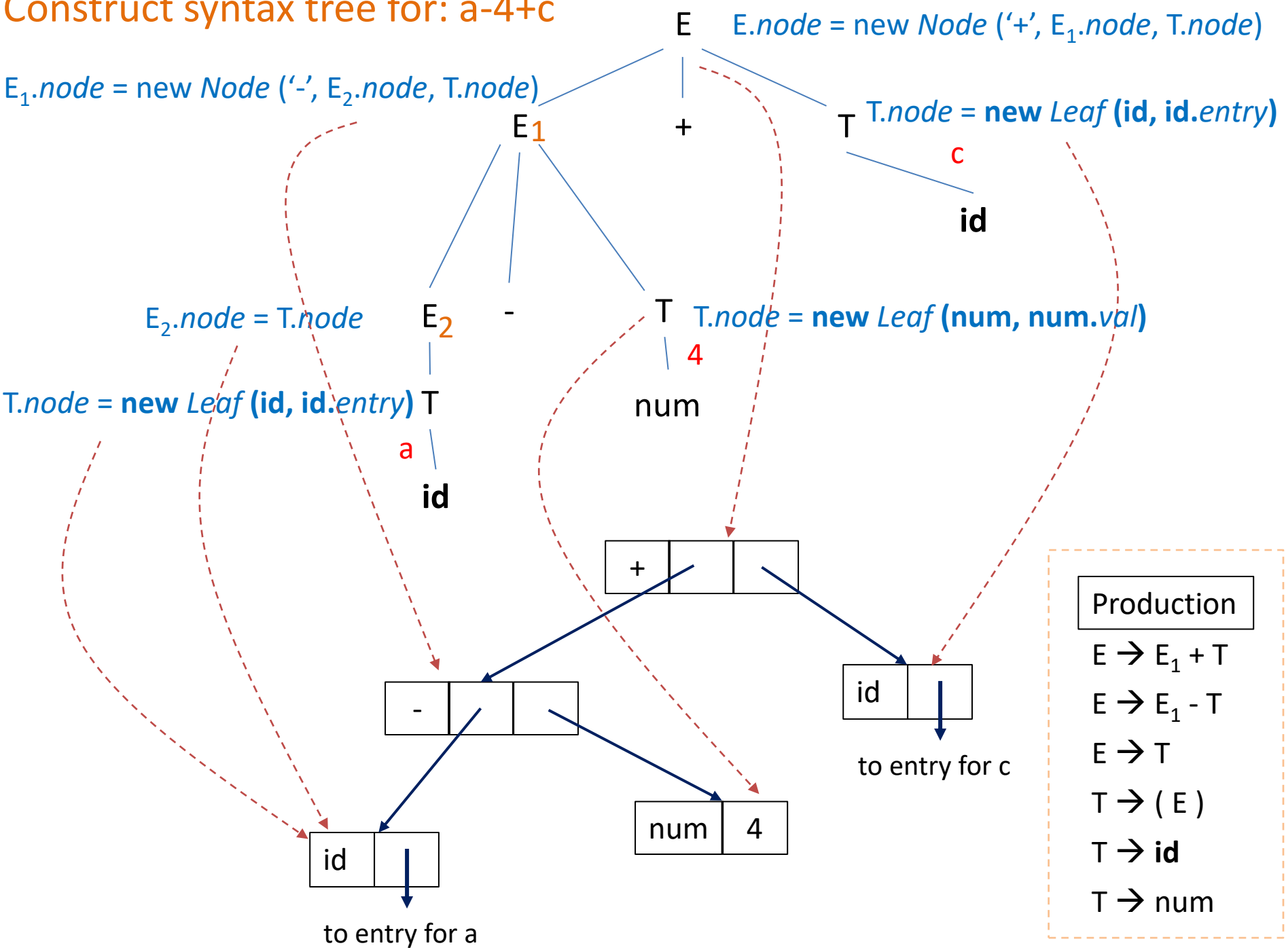
Abstract Syntax tree

Construction of AST

Example 1: AST construction in S-attributed SDD

	PRODUCTION	SEMANTIC RULE
1.	$E \rightarrow E_1 + T$	$E.node = \textbf{new}\ Node('+', E_1.node, T.node)$
2.	$E \rightarrow E_1 - T$	$E.node = \textbf{new}\ Node('-', E_1.node, T.node)$
3.	$E \rightarrow T$	$E.node = T.node$
4.	$T \rightarrow (E)$	$T.node = E.node$
5.	$T \rightarrow \textbf{id}$	$T.node = \textbf{new}\ Leaf(\textbf{id}, \textbf{id}.entry)$
6.	$T \rightarrow \textbf{num}$	$T.node = \textbf{new}\ Leaf(\textbf{num}, \textbf{num}.entry)$

Construct syntax tree for: a-4+c



Construction of AST

Example 1: AST construction in S-attributed SDD

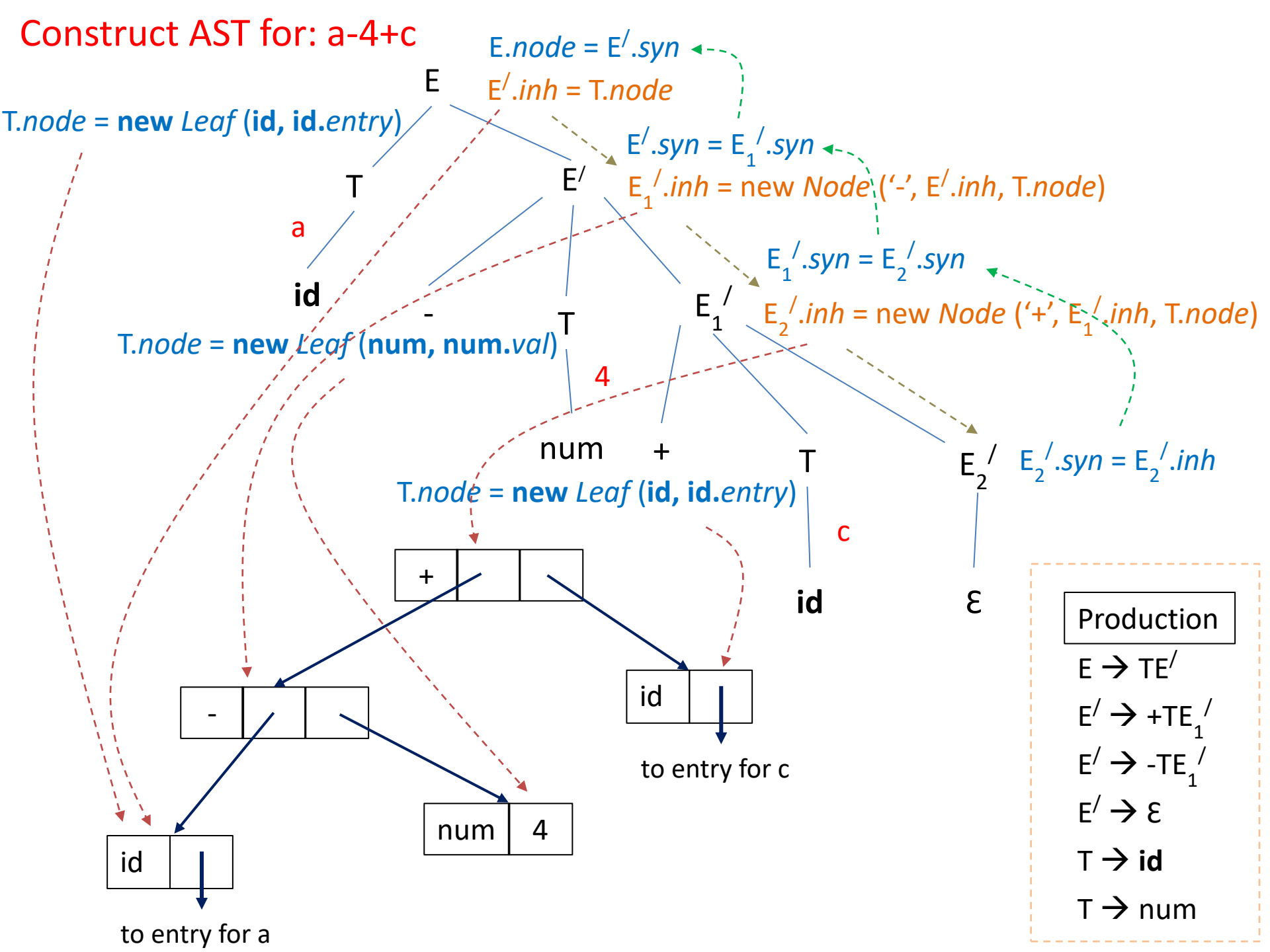
	PRODUCTION	SEMANTIC RULE
1.	$E \rightarrow E_1 + T$	$E.node = \textbf{new}\ Node('+', E_1.node, T.node)$
2.	$E \rightarrow E_1 - T$	$E.node = \textbf{new}\ Node('-', E_1.node, T.node)$
3.	$E \rightarrow T$	$E.node = T.node$
4.	$T \rightarrow (E)$	$T.node = E.node$
5.	$T \rightarrow \textbf{id}$	$T.node = \textbf{new}\ Leaf(\textbf{id}, \textbf{id}.entry)$
6.	$T \rightarrow \textbf{num}$	$T.node = \textbf{new}\ Leaf(\textbf{num}, \textbf{num}.entry)$

Construction of AST

Example 2: AST construction in L-attributed SDD

	PRODUCTION	SEMANTIC RULE
1.	$E \rightarrow T\,E'$	$E.node = E'.syn;\ E'.inh = T.node$
2.	$E' \rightarrow +\,T\,E'_1$	$E'_1.inh = \textbf{new}\ Node('+', E'.inh, T.node)$ $E'.syn = E'_1.syn$
3.	$E' \rightarrow -\,T\,E'_1$	$E'_1.inh = \textbf{new}\ Node('-', E'.inh, T.node)$ $E'.syn = E'_1.syn$
4.	$E' \rightarrow \epsilon$	$E'.syn = E'.inh$
5.	$T \rightarrow (\,E\,)$	$T.node = E.node$
6.	$T \rightarrow \textbf{id}$	$T.node = \textbf{new}\ Leaf(\textbf{id}, \textbf{id}.entry)$
7.	$T \rightarrow \textbf{num}$	$T.node = \textbf{new}\ Leaf(\textbf{num}, \textbf{num}.entry)$

Construct AST for: a-4+c



Production
E → TE/
E/ → +TE1/
E/ → -TE1/
E/ → ε
T → **id**
T → num

Construction of AST

Example 2: AST construction in L-attributed SDD

	PRODUCTION	SEMANTIC RULE
1.	$E \to T\,E'$	$E.node = E'.syn;\ E'.inh = T.node$
2.	$E' \to +\,T\,E'_1$	$E'_1.inh = \textbf{new}\ Node('+', E'.inh, T.node)$
		$E'.syn = E'_1.syn$
3.	$E' \to -\,T\,E'_1$	$E'_1.inh = \textbf{new}\ Node('-', E'.inh, T.node)$
		$E'.syn = E'_1.syn$
4.	$E' \to \epsilon$	$E'.syn = E'.inh$
5.	$T \to (\,E\,)$	$T.node = E.node$
6.	$T \to \textbf{id}$	$T.node = \textbf{new}\ Leaf(\textbf{id}, \textbf{id}.entry)$
7.	$T \to \textbf{num}$	$T.node = \textbf{new}\ Leaf(\textbf{num}, \textbf{num}.entry)$

Just like we calculated value previously!

Syntax-Directed Translation Schemes (SDT)

	PRODUCTION	SEMANTIC RULES
1)	$L \rightarrow E\ \mathbf{n}$	$L.val = E.val$
2)	$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3)	$E \rightarrow T$	$E.val = T.val$
4)	$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5)	$T \rightarrow F$	$T.val = F.val$
6)	$F \rightarrow (E)$	$F.val = E.val$
7)	$F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.\text{lexval}$

SDD

Semantic Action

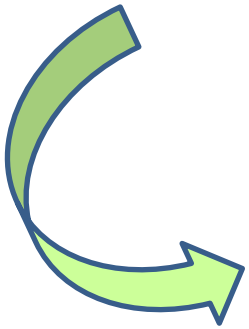
SDT

L	$\rightarrow$	$E\ \mathbf{n}$	$\{\ \text{print}(E.val);\ \}$
E	$\rightarrow$	$E_1 + T$	$\{\ E.val = E_1.val + T.val;\ \}$
E	$\rightarrow$	T	$\{\ E.val = T.val;\ \}$
T	$\rightarrow$	$T_1 * F$	$\{\ T.val = T_1.val \times F.val;\ \}$
T	$\rightarrow$	F	$\{\ T.val = F.val;\ \}$
F	$\rightarrow$	(E)	$\{\ F.val = E.val;\ \}$
F	$\rightarrow$	$\mathbf{digit}$	$\{\ F.val = \mathbf{digit}.lexval;\ \}$

Syntax-Directed Translation Schemes (SDT)

Prod.	Semantic Actions
$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *FT_1'$	$T_1'.inh = T'.inh \times F.val$ $T'.syn = T_1'.syn$
$T' \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow$ **digit**	$F.val =$ **digit**$.lexval$

SDD



SDT

$T \rightarrow F \{T'.inh = F.val\}\ T' \{T.val = T'.syn\}$
$T' \rightarrow *F \{T_1'.inh = T'.inh \times F.val\}\ T_1' \{T'.syn = T_1'.syn\}$
$T' \rightarrow \varepsilon \{T'.syn = T_1'.syn\}$
$F \rightarrow$ **digit** $\{F.val =$ **digit**$.lexval\}$

Syntax-Directed Translation Schemes

- SDT is complementary notation of SDD
  - SDD easier to read
  - SDT easier to implement
  - All SDD we encountered so far can be implemented using SDT
- SDT is a context free grammar with program fragments (Semantic Actions) embedded in production body
- Semantic Action can appear any place of production body

Another Example

- Consider the following SDD

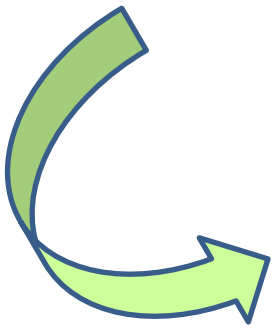
Production	Semantic Rules
$E \rightarrow E + T$	$print('+')$
$E \rightarrow T$	
$T \rightarrow T * F$	$print('*')$
$T \rightarrow F$	
$F \rightarrow \mathbf{num}$	$print(num.lexval)$

- What this SDD do?

Another Example

SDD for infix to postfix conversion

Production	Semantic Rules
$E \rightarrow E + T$	$print('+')$
$E \rightarrow T$	
$T \rightarrow T * F$	$print('*')$
$T \rightarrow F$	
$F \rightarrow \textbf{num}$	$print(num.lexval)$



SDT for infix to postfix conversion

$E \rightarrow E + T \quad \{print('+')\}$

$E \rightarrow T$

$T \rightarrow T * F \quad \{print('*')\}$

$T \rightarrow F$

$F \rightarrow \textbf{num} \quad \{print(num.lexval)\}$

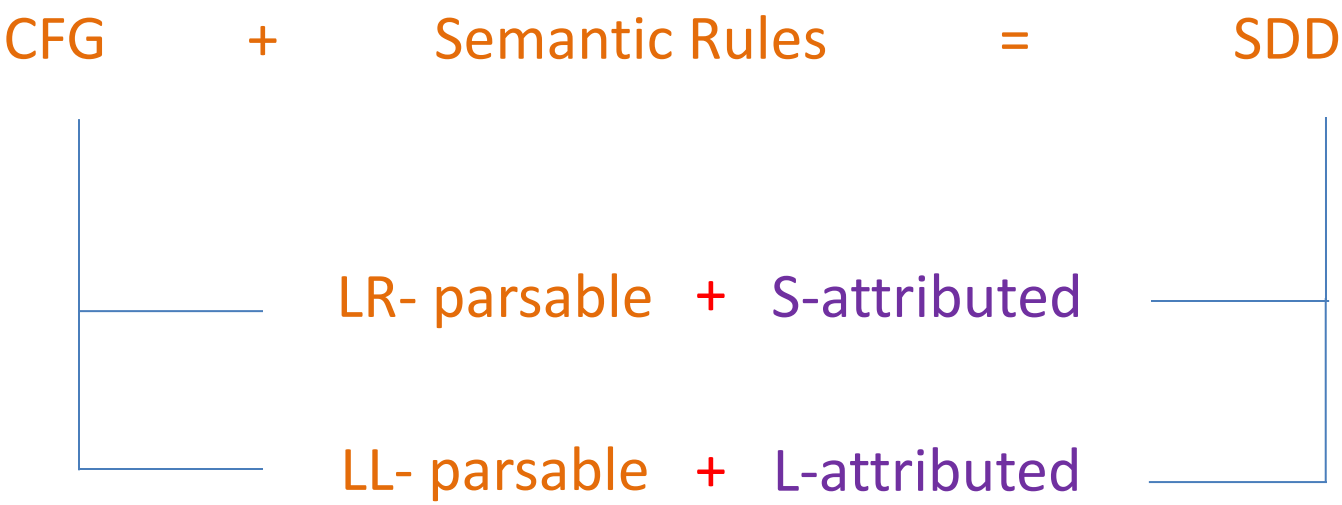
An ANTLR Example

```
// Arithmetic.g4
grammar Arithmetic;

// Parser rules
expr: expr '+' term    { $$.value = $expr.value + $term.value; }
    | expr '-' term    { $$.value = $expr.value - $term.value; }
    | term;

term: term '*' factor { $$.value = $term.value * $factor.value; }
    | term '/' factor { $$.value = $term.value / $factor.value; }
    | factor;

factor: INT           { $$.value = Integer.parseInt($INT.text); }
      | '(' expr ')'  { $$.value = $expr.value; };

// Lexer rules
INT : [0-9]+;
WS  : [ \t\r\n]+ -> skip;
```

Semantic Actions

Implementation of SDT

- Any SDT can be implemented by building a parse tree and then performing the actions in left-right depth first order
- However, typically, SDTs are implemented during parsing without explicitly constructing the parse tree
- Notably, there are SDT's that are not implementable during parsing.
- Hence, we focus on two classes



Postfix SDT

- SDTs with all the actions at the right ends of the production body

Example 1

L	$\rightarrow$	E **n**	$\{$ print($E.val$); $\}$
E	$\rightarrow$	$E_1 + T$	$\{$ $E.val = E_1.val + T.val$; $\}$
E	$\rightarrow$	T	$\{$ $E.val = T.val$; $\}$
T	$\rightarrow$	$T_1 * F$	$\{$ $T.val = T_1.val \times F.val$; $\}$
T	$\rightarrow$	F	$\{$ $T.val = F.val$; $\}$
F	$\rightarrow$	(E)	$\{$ $F.val = E.val$; $\}$
F	$\rightarrow$	**digit**	$\{$ $F.val =$ **digit**$.lexval$; $\}$

Example 2

$E \rightarrow E + T \quad \{print('+')\}$

$E \rightarrow T$

$T \rightarrow T * F \quad \{print('*')\}$

$T \rightarrow F$

$F \rightarrow$ **num** $\quad \{print(num.lexval)\}$

- Postfix SDT can be constructed if the grammar can be parsed bottom up and the SDD is S-Attributed
- Each action is executed along with the reduction of the body to the head of the production

Parser-Stack Implementation of Postfix SDT

- The attribute(s) of each grammar symbol can be put on the stack in a place where they can be found during reduction
- The best plan is to place the attributes along with the grammar symbol (or the LR states that represent these symbols) in records on the stack itself
- In example below, if we reduce by a production such as **A->XYZ**, then all the attributes are available at known positions of the stack
- After the action, A and its attributes are at the top of the stack, in the position of record X.

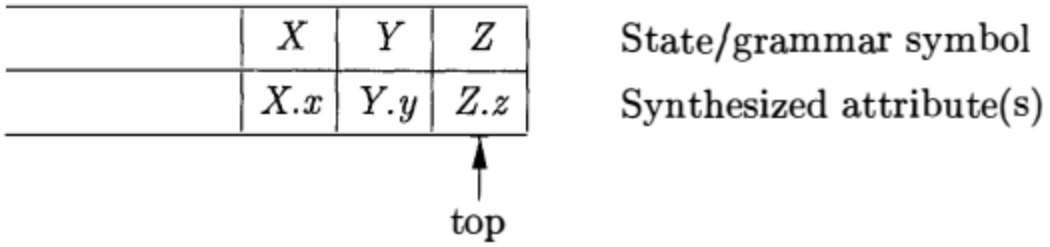


Figure 5.19: Parser stack with a field for synthesized attributes

Parser-Stack Implementation of Postfix SDT

PRODUCTION	ACTIONS
$L \rightarrow E\ \mathbf{n}$	{ print($stack[top-1].val$); $top = top - 1$; }
$E \rightarrow E_1 + T$	{ $stack[top-2].val = stack[top-2].val + stack[top].val$; $top = top - 2$; }
$E \rightarrow T$	*No actions, why?*
$T \rightarrow T_1 * F$	{ $stack[top-2].val = stack[top-2].val \times stack[top].val$; $top = top - 2$; }
$T \rightarrow F$	*No actions, why?*
$F \rightarrow (\ E\ )$	{ $stack[top-2].val = stack[top-1].val$; $top = top - 2$; }
$F \rightarrow \mathbf{digit}$	

Figure 5.20: Implementing the desk calculator on a bottom-up parsing stack

SDTs with Action Inside Productions

- An action can be placed at any position of the production body
- Performed immediately after all symbols to its left are processed
- For *B* -> *X* { *a* } *Y*, action ***a*** is performed after recognizing X or all terminals derived from X
  - Bottom up parsing: As soon as X appears in the stack
  - Top down parsing: Before expanding Y or checking Y in input

SDTs Non Implementable during Parsing

- Example: Conversion to Prefix form

1)	L	$\rightarrow$	E **n**
2)	E	$\rightarrow$	{ print('+'); } $E_1 + T$
3)	E	$\rightarrow$	T
4)	T	$\rightarrow$	{ print('*'); } $T_1 * F$
5)	T	$\rightarrow$	F
6)	F	$\rightarrow$	(E)
7)	F	$\rightarrow$	**digit** { print(**digit**.*lexval*); }

- This example cannot be implemented during top-down or bottom up parsing
- Doing critical task before knowing input

SDTs Non Implementable during Parsing

- So, what to do with the previous SDT? Not implementable at all?
- Any SDT can be implemented using:
  - Produce parse tree ignoring action
  - For each internal node, insert additional children at appropriate place representing actions if any
  - Perform preorder traversal and perform action at nodes corresponding to action

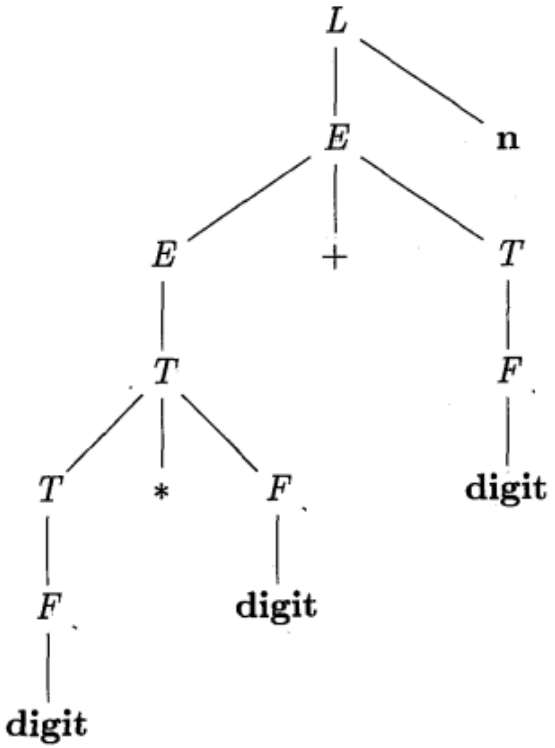
SDT Implementation using Parse Tree

Convert from infix to prefix: 3*5+4

1) $L \rightarrow E\ \mathbf{n}$
2) $E \rightarrow \{\ \text{print}('+');\ \}\ E_1 + T$
3) $E \rightarrow T$
4) $T \rightarrow \{\ \text{print}('*');\ \}\ T_1 * F$
5) $T \rightarrow F$
6) $F \rightarrow (\ E\ )$
7) $F \rightarrow \mathbf{digit}\ \{\ \text{print}(\mathbf{digit}.\mathit{lexval});\ \}$

1. Produce parse tree ignoring action
2. For each internal node, insert additional children at appropriate place representing actions if any
3. Perform preorder traversal and perform action at nodes corresponding to action

Parse tree with actions embedded



Eliminating Left Recursion

We know

Production
$T \rightarrow T * F$
$T \rightarrow F$
$F \rightarrow$ digit

Eliminating Left Recursion

Production
$T \rightarrow F\ T'$
$T' \rightarrow * F\ T_1'$
$T' \rightarrow \varepsilon$
$F \rightarrow$ digit

What about this?

T	$\rightarrow$	$T_1 * F$	$\{ T.val = T_1.val \times F.val; \}$
T	$\rightarrow$	F	$\{ T.val = F.val; \}$
F	$\rightarrow$	(E)	$\{ F.val = E.val; \}$
F	$\rightarrow$	**digit**	$\{ F.val =$ **digit**$.lexval; \}$

$T \rightarrow F \{T'.inh = F.val\}\ T' \{T.val = T'.syn\}$
$T' \rightarrow *F \{T_1'.inh = T'.inh \times F.val\}\ T_1' \{T'.syn = T_1'.syn\}$
$T' \rightarrow \varepsilon \{T'.syn = T_1'.syn\}$
$F \rightarrow$ **digit** $\{F.val =$ **digit**$.lexval\}$

Eliminating Left Recursion

- Consider the following postfix SDT

$$\begin{array}{lcll} E & \rightarrow & E_1 + T & \{\ \text{print}('+');\ \} \\ E & \rightarrow & T & \end{array}$$

- It converts infix expression to prefix
- Good with bottom-up parsing
- Is it also good for top-down parsing?
- No! Need to remove left recursion
- With action in production body, it becomes bit tricky
  - Need to preserve order

Eliminating Left Recursion

- Recap:
  - Grammar with left recursion

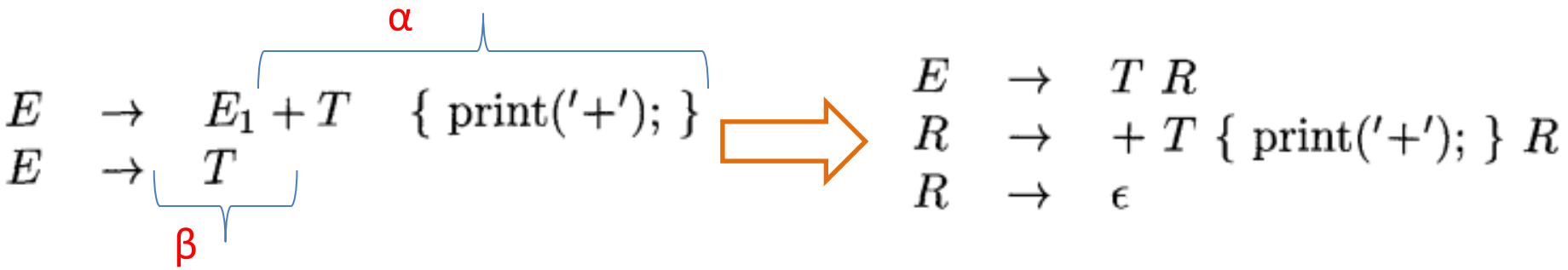
$$A \rightarrow A\alpha \mid \beta$$

  - After eliminating left recursion

$$A \rightarrow \beta R$$
$$R \rightarrow \alpha R \mid \epsilon$$

Eliminating Left Recursion

- Case 1: when we only care about the order in which actions in an SDT are performed
  - For example, printing string
- Treat the actions as if they are terminal symbols
  - Grammar transformation preserves the order of the terminals in the generated string
  - Therefore, the actions are executed in the same order in any left-to-right parse, top-down or bottom-up



SDT with left recursion **SDT without left recursion**

Eliminating Left Recursion

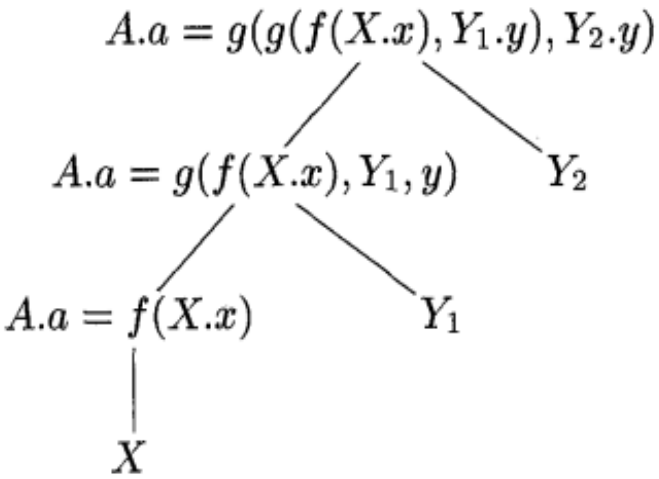
Case 2: Actions Involving Attribute Calculation

$$A \rightarrow A_1\ Y\ \{A.a = g(A_1.a, Y.y)\}$$
$$A \rightarrow X\ \{A.a = f(X.x)\}$$

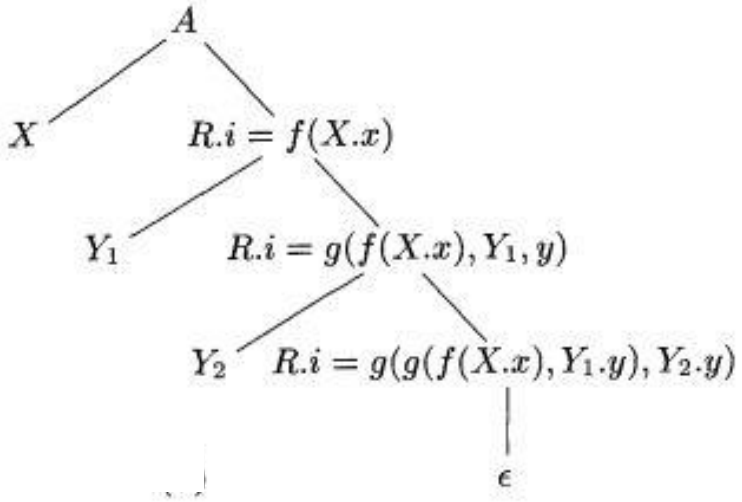
SDT with left recursion

$$A \rightarrow X\ R$$
$$R \rightarrow Y\ R \mid \epsilon$$

CFG without left recursion



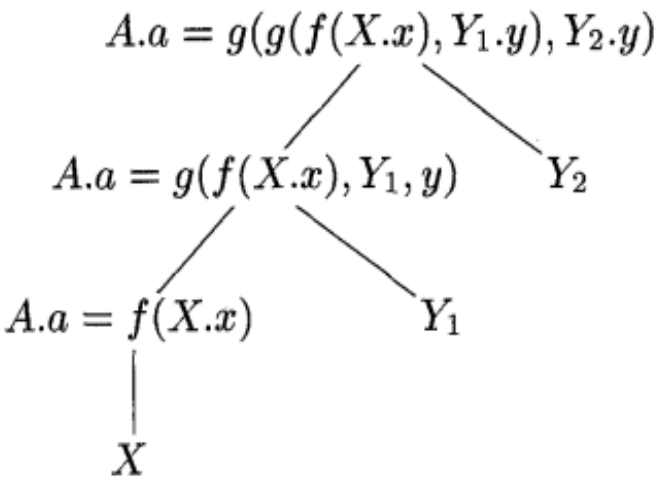
Annotated parse tree



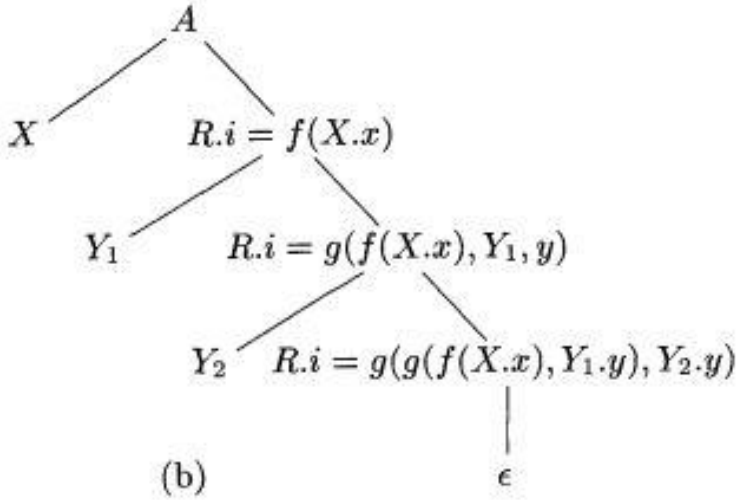
Annotated parse tree

Eliminating Left Recursion

Case 2: Actions Involving Attribute Calculation



Annotated parse tree



Annotated parse tree

$$A \rightarrow A_1\ Y\ \{A.a = g(A_1.a, Y.y)\}$$
$$A \rightarrow X\ \{A.a = f(X.x)\}$$

SDT with left recursion

$$A \rightarrow X\ \{R.i = f(X.x)\}\ R\ \{A.a = R.s\}$$
$$R \rightarrow Y\ \{R_1.i = g(R.i, Y.y)\}\ R_1\ \{R.s = R_1.s\}$$
$$R \rightarrow \epsilon\ \{R.s = R.i\}$$

SDT without left recursion

Illustrative Example

Find an SDT computes the value of a string of 0's and 1's interpreted as a positive, binary integer

$$
\begin{array}{lcl}
B & \rightarrow & B_1\ 0\ \{B.val = 2 \times B_1.val\} \\
 & | & B_1\ 1\ \{B.val = 2 \times B_1.val + 1\} \\
 & | & 1\ \{B.val = 1\}
\end{array}
$$

Illustrative Example

Rewrite the previous SDT so that the underlying grammar is not left recursive, and yet the same value of B.*val* is computed for the entire input string.

$$
\begin{array}{lcl}
B & \rightarrow & B_1\ 0\ \{B.val = 2 \times B_1.val\} \\
 & | & B_1\ 1\ \{B.val = 2 \times B_1.val + 1\} \\
 & | & 1\ \{B.val = 1\}
\end{array}
$$

$B \rightarrow 1\ \{ R.i = 1 \}\ R\ \{ B.val = R.s \}$

$R \rightarrow 1\ \{ R_1.i = 2 * R.i + 1 \}\ R_1\ \{ R.s = R_1.s \}$

$R \rightarrow 0\ \{ R_1.i = 2 * R.i \}\ R_1\ \{ R.s = R_1.s \}$

$R \rightarrow \varepsilon\ \{ R.s = R.i \}$

$$
\begin{array}{lcl}
A & \rightarrow & A_1\ Y\ \{A.a = g(A_1.a, Y.y)\} \\
A & \rightarrow & X\ \{A.a = f(X.x)\}
\end{array}
$$

$$
\begin{array}{lcl}
A & \rightarrow & X\ \{R.i = f(X.x)\}\ R\ \{A.a = R.s\} \\
R & \rightarrow & Y\ \{R_1.i = g(R.i, Y.y\}\ R_1\ \{R.s = R_1.s\} \\
R & \rightarrow & \epsilon\ \{R.s = R.i\}
\end{array}
$$

SDT for L-Attributed SDD

- Rules for conversion:
  - Embed the action that computes the inherited attributes for a nonterminal A immediately before that occurrence of A in the body of the production.
    - Arrange calculation of inherited attributes of A so that dependency is maintained
  - Place the actions that compute a synthesized attribute for the head of a production at the end of the body of that production.

SDT for L-Attributed SDD

- Example 1:

Prod.	Semantic Actions
$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *FT_1'$	$T_1'.inh = T'.inh \times F.val$ $T'.syn = T_1'.syn$
$T' \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow$ **digit**	$F.val =$ **digit**$.lexval$

L-attributed SDD

SDT
$T \rightarrow F \{T'.inh = F.val\}\ T'\{T.val = T'.syn\}$
$T' \rightarrow *F \{T_1'.inh = T'.inh \times F.val\}\ T_1'\{T'.syn = T_1'.syn\}$
$T' \rightarrow \varepsilon \{T'.syn = T_1'.syn\}$
$F \rightarrow$ **digit** $\{F.val =$ **digit**$.lexval\}$

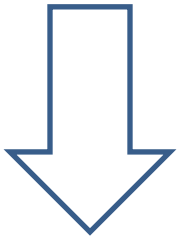
SDT for the L-attributed SDD

SDT for L-Attributed SDD

- Example 2: SDD for while-statements

$S \rightarrow \textbf{while}\ (\ C\ )\ S_1$	$L1 = new();$ $L2 = new();$ $S_1.next = L1;$ $C.false = S.next;$ $C.true = L2;$ $S.code = \textbf{label} \parallel L1 \parallel C.code \parallel \textbf{label} \parallel L2 \parallel S_1.code$

SDT



$S \rightarrow \textbf{while}\ ($	$\{\ L1 = new();\ L2 = new();\ C.false = S.next;\ C.true = L2;\ \}$
$C\ )$	$\{\ S_1.next = L1;\ \}$
S_1	$\{\ S.code = \textbf{label} \parallel L1 \parallel C.code \parallel \textbf{label} \parallel L2 \parallel S_1.code;\ \}$

Implementing L-Attributed SDD

- Using parse tree
  - Build the parse tree and annotate
  - Convert to SDT → build the parse tree → add actions → execute the actions in preorder
- Translation during parsing
  - Using a recursive descent parser
  - Generate code on the fly
  - Implement SDT in conjunction with an *LL*-parser
  - Implement SDT in conjunction with an *LR*-parser (counter intuitive!)

Translation During Recursive-Descent Parsing

$S \rightarrow$ **while** ($\{ L1 = new(); L2 = new(); C.false = S.next; C.true = L2; \}$
C) $\{ S_1.next = L1; \}$
S_1 $\{ S.code =$ **label** $\| L1 \| C.code \|$ **label** $\| L2 \| S_1.code; \}$

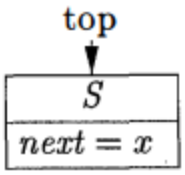
```
string S(label next) {
        string Scode, Ccode; /* local variables holding code fragments */
        label L1, L2; /* the local labels */
        if ( current input == token while ) {
                advance input;
                check '(' is next on the input, and advance;
                L1 = new();
                L2 = new();
                Ccode = C(next, L2);
                check ')' is next on the input, and advance;
                Scode = S(L1);
                return("label" || L1 || Ccode || "label" || L2 || Scode);
        }
        else /* other statement types */
}
```

On-The-Fly Code Generation

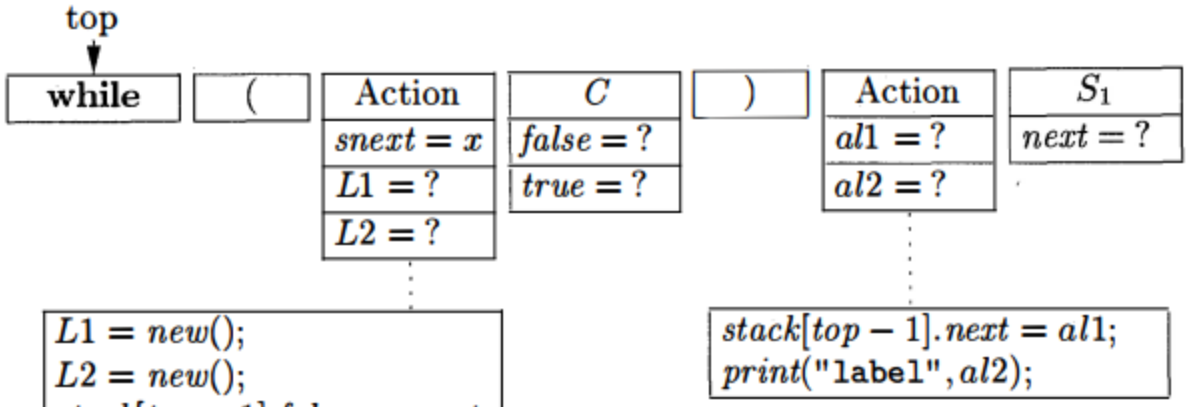
$$
\begin{array}{lll}
S & \rightarrow & \textbf{while} \; (\quad \{ \; L1 = new(); \; L2 = new(); \; C.false = S.next; \\
 & & \qquad C.true = L2; \; print(\texttt{"label"}, L1); \; \} \\
 & & C \;) \qquad \{ \; S_1.next = L1; \; print(\texttt{"label"}, L2); \; \} \\
 & & S_1
\end{array}
$$

```
void S(label next) {
        label L1, L2; /* the local labels */
        if ( current input == token while ) {
                advance input;
                check '(' is next on the input, and advance;
                L1 = new();
                L2 = new();
                print("label", L1);
                C(next, L2);
                check ')' is next on the input, and advance;
                print("label", L2);
                S(L1);
        }
        else /* other statement types */
}
```

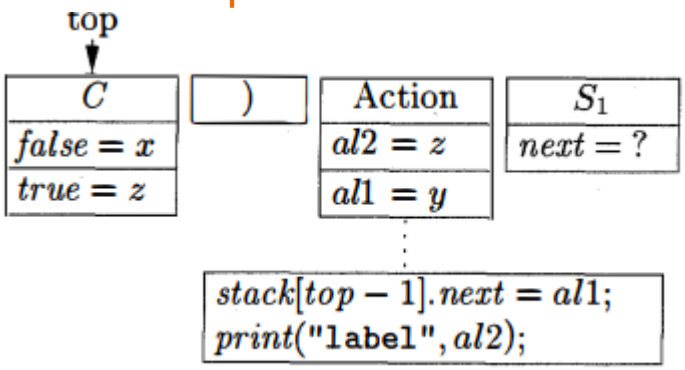
L-Attributed SDD's and LL Parsing



Expansion of S according to the while-statement production



After the action above C is performed



On-The-Fly generation

$$
\begin{array}{lcll}
S & \rightarrow & \textbf{while} \; (& \{ \; L1 = new(); \; L2 = new(); \; C.false = S.next; \\
 & & & \quad C.true = L2; \; print(\texttt{"label"}, L1); \; \} \\
 & & C \;) & \{ \; S_1.next = L1; \; print(\texttt{"label"}, L2); \; \} \\
 & & S_1 &
\end{array}
$$

L-Attributed SDD's and LL Parsing

$S \rightarrow$ **while** (	{ $L1 = new(); L2 = new(); C.false = S.next; C.true = L2;$ }
C)	{ $S_1.next = L1;$ }
S_1	{ $S.code =$ **label** $\|\|$ $L1$ $\|\|$ $C.code$ $\|\|$ **label** $\|\|$ $L2$ $\|\|$ $S_1.code;$ }

Code as a synthesized attribute

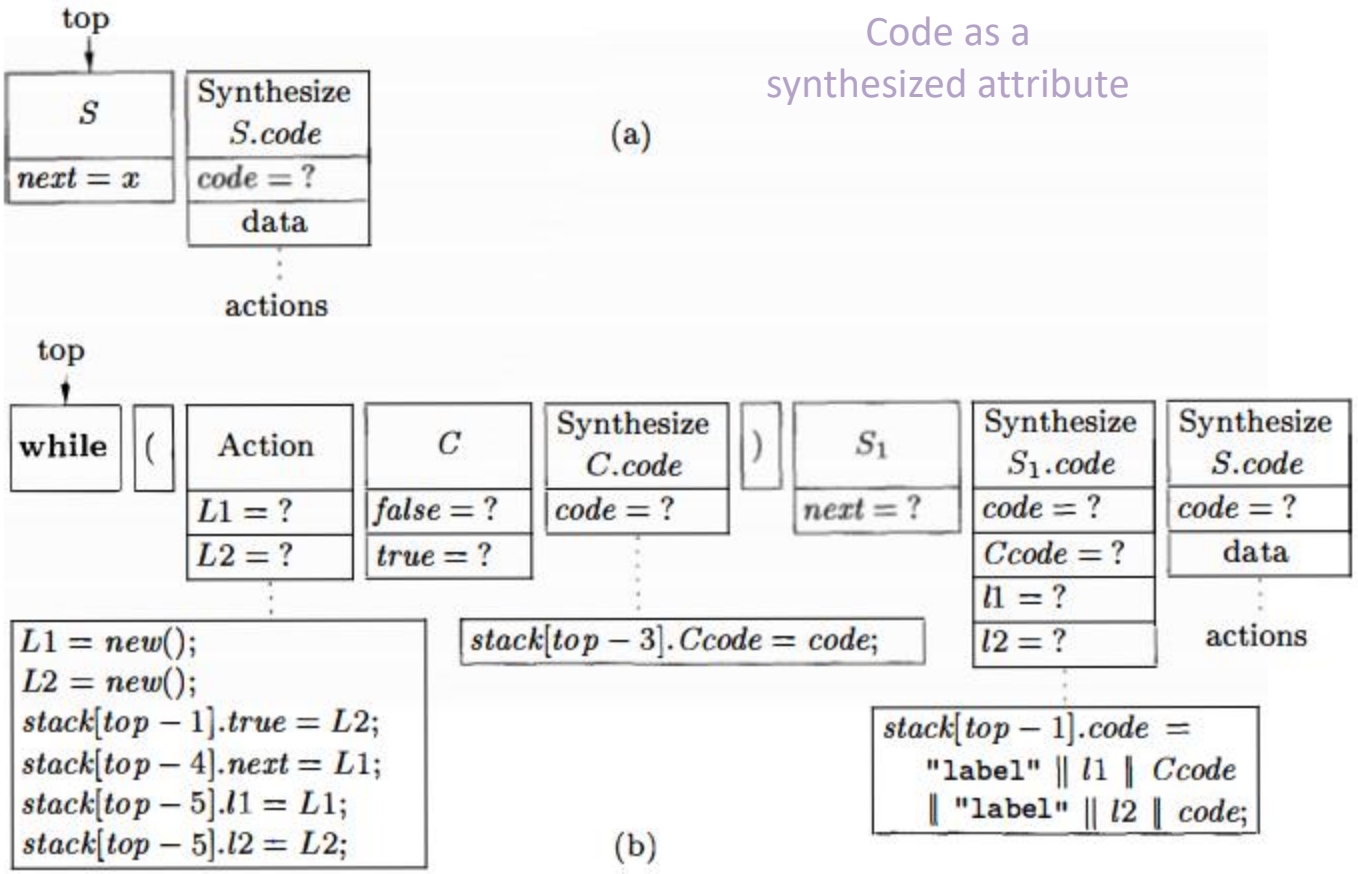
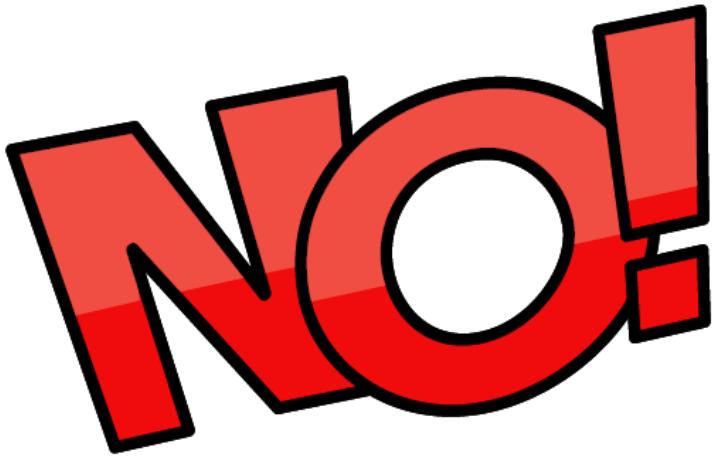


Figure 5.35: Expansion of S with synthesized attribute constructed on the stack

L-Attributed SDD's on LR Grammars

- Every S-attributed SDD on an LR grammar can be implemented during a bottom-up parse.
- Every L-attributed SDD on an LL grammar can be parsed top-down.
- LL grammars are a proper subset of the LR grammars
- S-attributed SDD's are a proper subset of the L-attributed SDD's

Can we handle every LR grammar and L-attributed SDD bottom-up?



L-Attributed SDD's on LR Grammars

- Consider the production A → B C in an LR-grammar
- There is an inherited attribute *B.i* that depends on inherited attributes of A.
- When we reduce to B,
  - we still have not seen the input that C generates, so we cannot be sure that we have a body of production A →B C.
  - Thus, we cannot compute *B.i* yet, since we are unsure whether to use the rule associated with this production.
  - Perhaps we could wait until we have reduced to C, and know that we must reduce B C to A.
  - However, even then, we do not know the inherited attributes of A, because even after reduction, we may not be sure of the production body that contains this A.
  - We could reason that this decision, too, should be deferred, and therefore further defer the computation of *B.i*.

L-Attributed SDD's on LR Grammars

- If we keep reasoning this way, we soon realize that we cannot make any decisions until the entire input is parsed.
- Essentially, we have reached the strategy of "build the parse tree first and then perform the translation."

However, we can do bottom up parsing of an L-attributed SDD on LL grammar

Reference

- Compilers: Principles, Techniques, & Tools, by

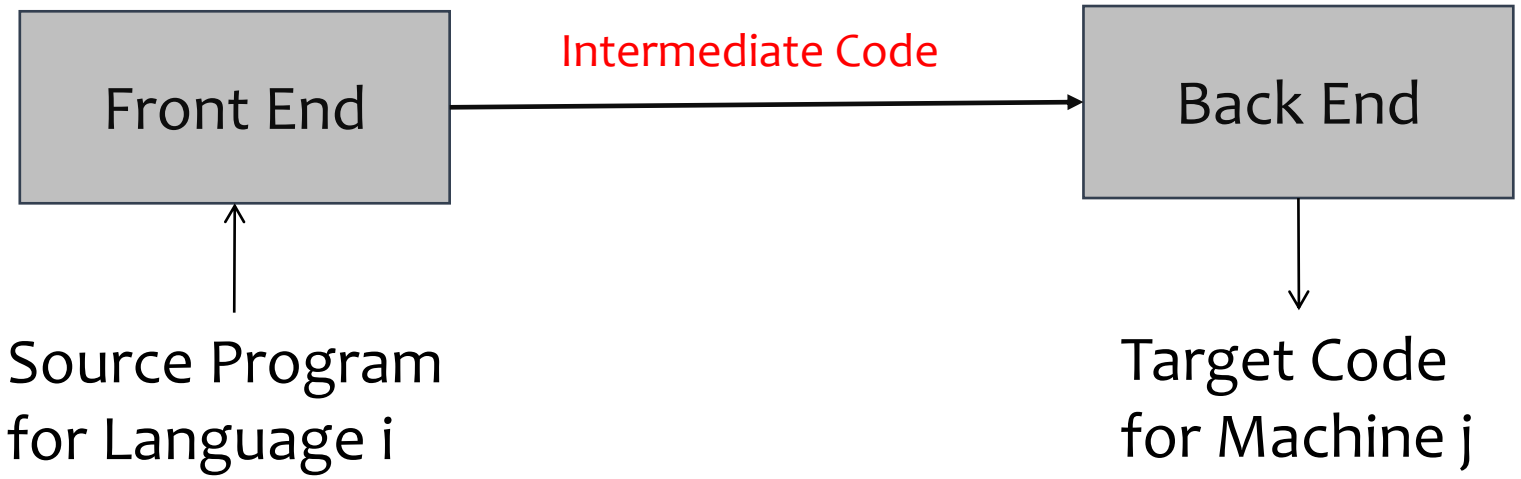
  Alfred B. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman

  - Chapter 5
    - 5.1
    - 5.2
    - 5.3
    - 5.4
    - 5.5 (5.5.1-5.5.3)

Chapter 6

Intermediate Code Generation

Intermediate Representation



Intermediate Representation

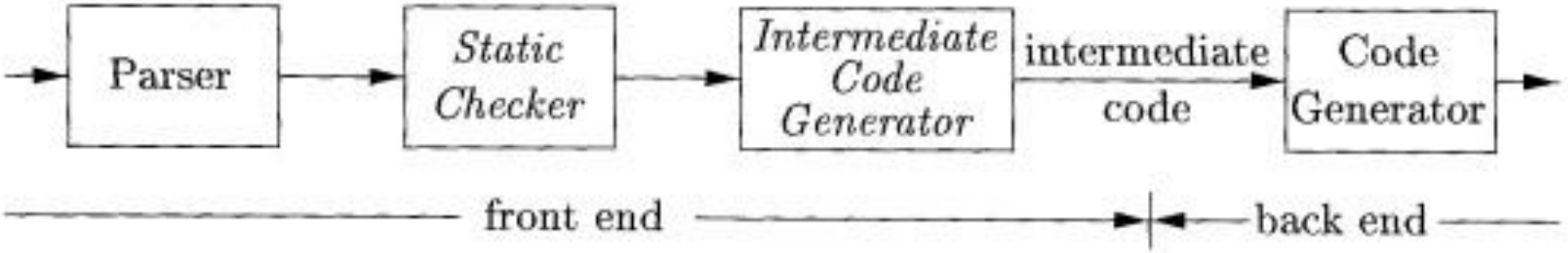


Figure 6.1: Logical structure of a compiler front end

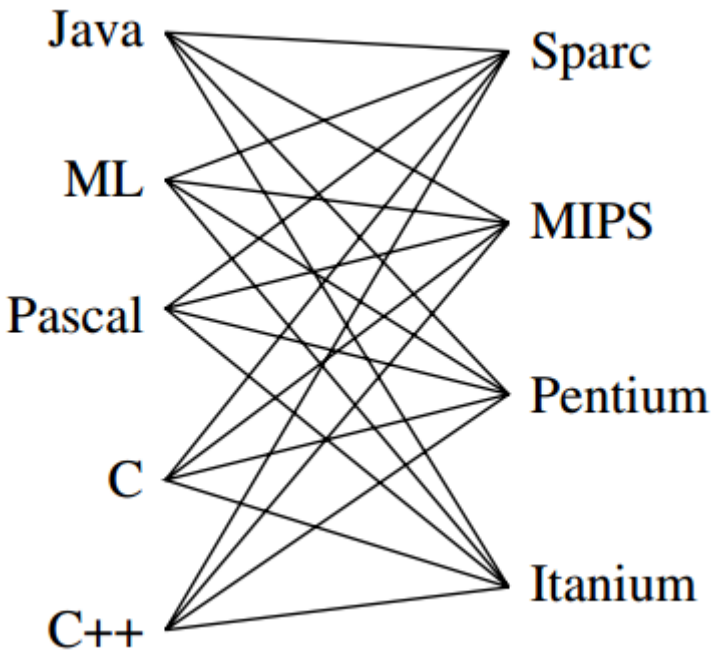
Intermediate Representation

- What is an Intermediate Representation (IR)?
  - ➢ Abstract Machine Language
  - ➢ Machine and Language independent version of source program
  - ➢ Can express target machine operation without committing too much machine specific details

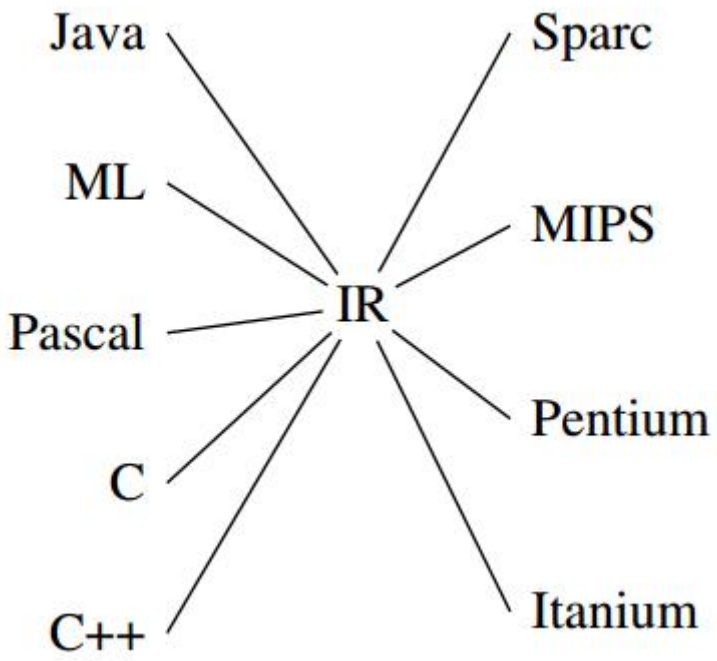
Why Need IR?

- Modularity:
  - Front end is not complicated with machine specific details
  - Back end is not bothered with information specific to the source code
- Portability & Cross-Compilation:
  - For n source language and m machines, need only n front ends and m back ends
  - Without IR, a total of $n \times m$ compilers are required
- Simplify Certain Optimization:
  - Easier and clearer optimization tasks

Why Need IR?



Without an IR

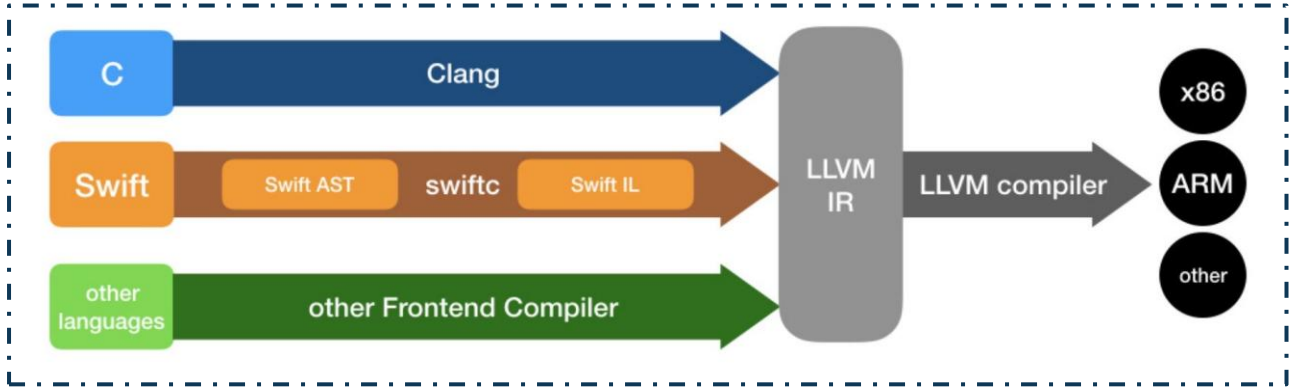


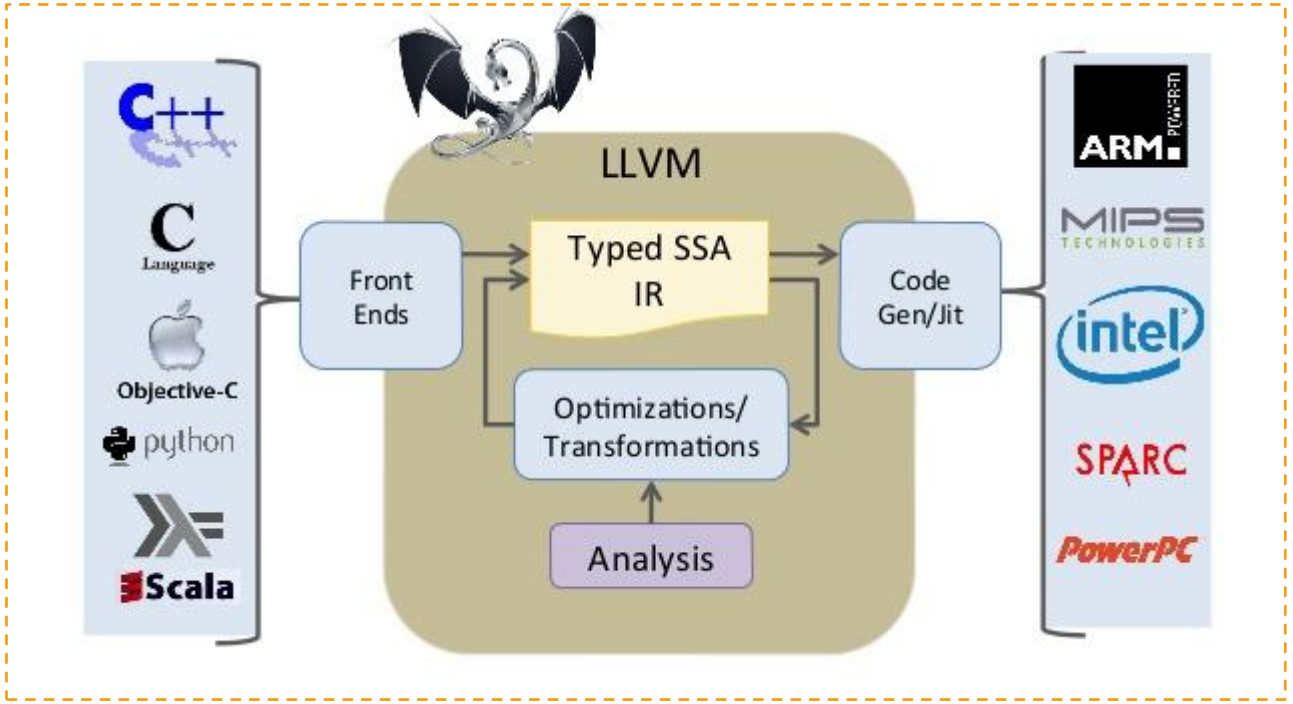
With an IR

Figure Source: Modern Compiler Implementation in Java by Andrew Appel

The LLVM Project

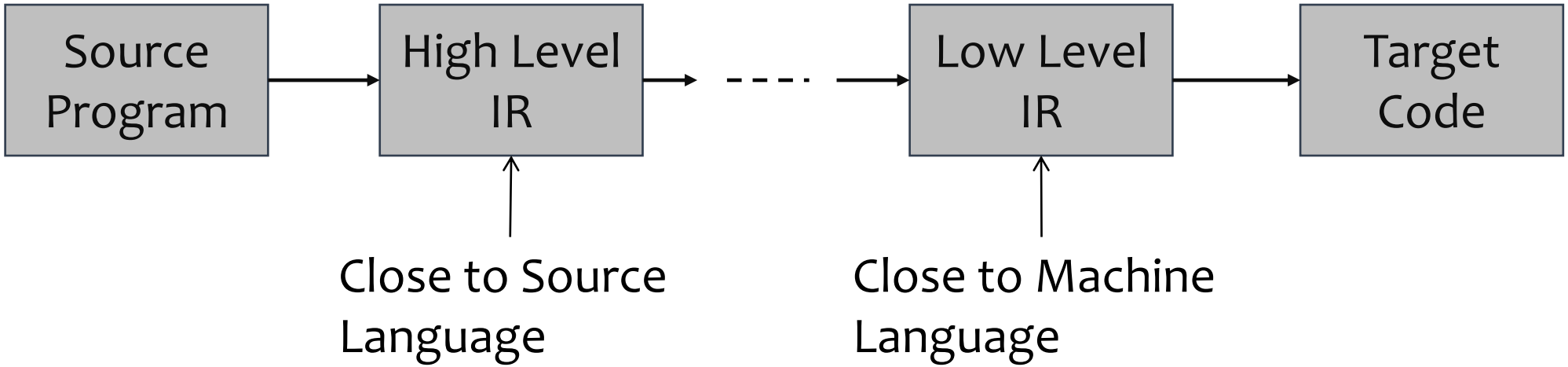
https://www.llvm.org/





Designing Good IR?

- Balance between high-level source language and low level machine language
- Often there are multiple IRs in a single compiler



Choice of an IR

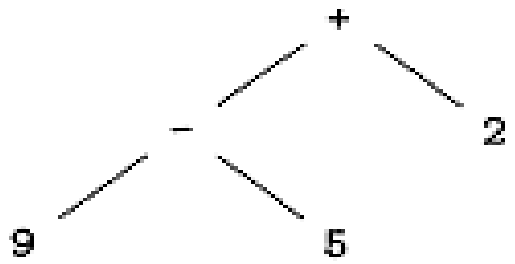
- The choice or design of an intermediate representation varies from compiler to compiler.
- An intermediate representation may either be
  - an actual language or
  - it may consist of internal data structures that are shared by phases of the compiler.
- C is a programming language, yet it is often used as an intermediate form because
  - it is flexible,
  - compiles into efficient machine code, and
  - its compilers are widely available.
- The original C + + compiler consisted of a front end that generated C, treating a C compiler as a back end.
- Another example: python compiler using C++ as an IR (https://github.com/Omyyyy/pycom)

High and Low Level IRs

- High Level IRs:
  - Syntax Tree
  - DAG
  - Three Address Code with high level operators
- Low Level IRs:
  - Three Address Code with low level operators

Syntax Trees

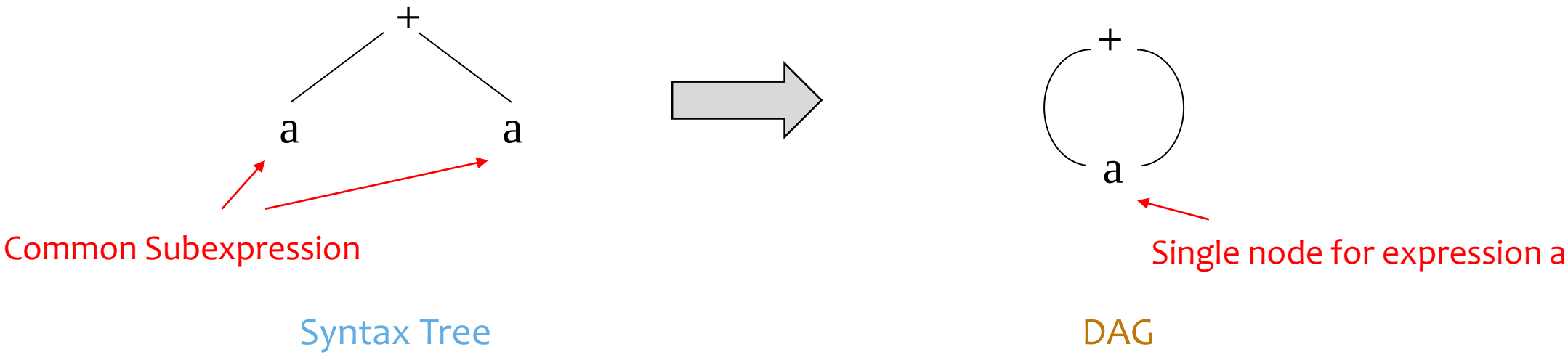
- Syntax Tree:
  - Each node represents a construct of source program
  - Each children represent a meaningful component
  - Leaves correspond to atomic operands
  - Interior nodes correspond to operators



Syntax Tree for expression 9-5+2

Directed Acyclic Graphs (DAG)

- Detect Common Subexpression
  - Produce only one subtree for each subexpression
  - Consider an expression: $a + a$

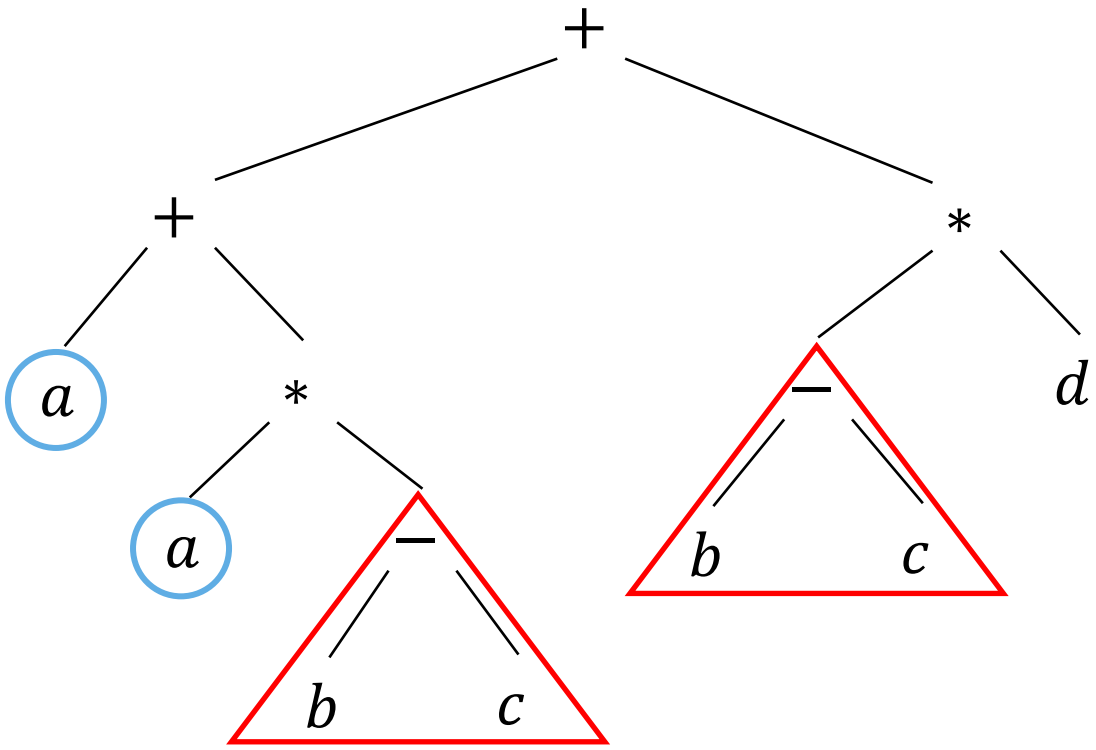


Directed Acyclic Graphs (DAG)

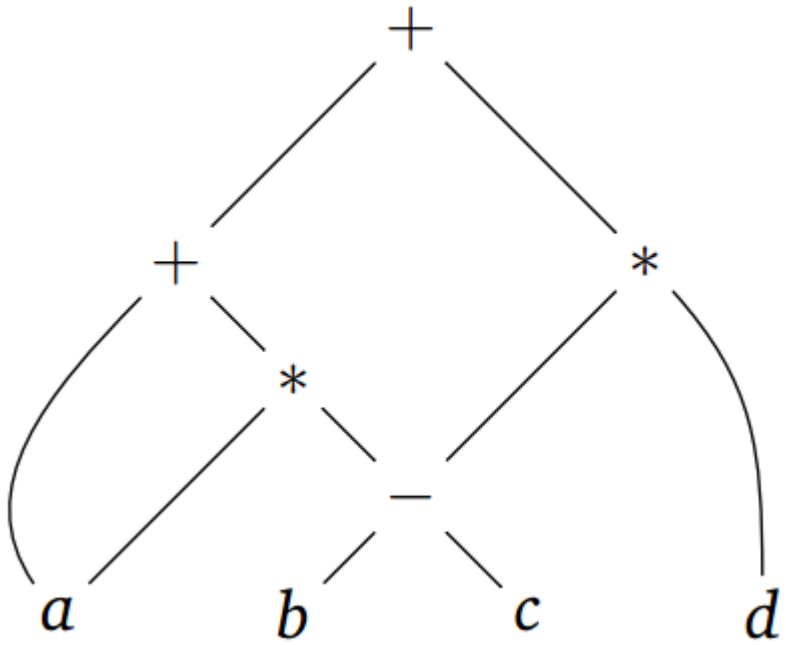
- A Node N can have more than one parent
  - If it represent a common subexpression
- Succinct Representation of Syntax Tree
- Provide clues for generating efficient code

Directed Acyclic Graphs (DAG)

- An example: $a + a * (b - c) + (b - c) * d$



Syntax Tree



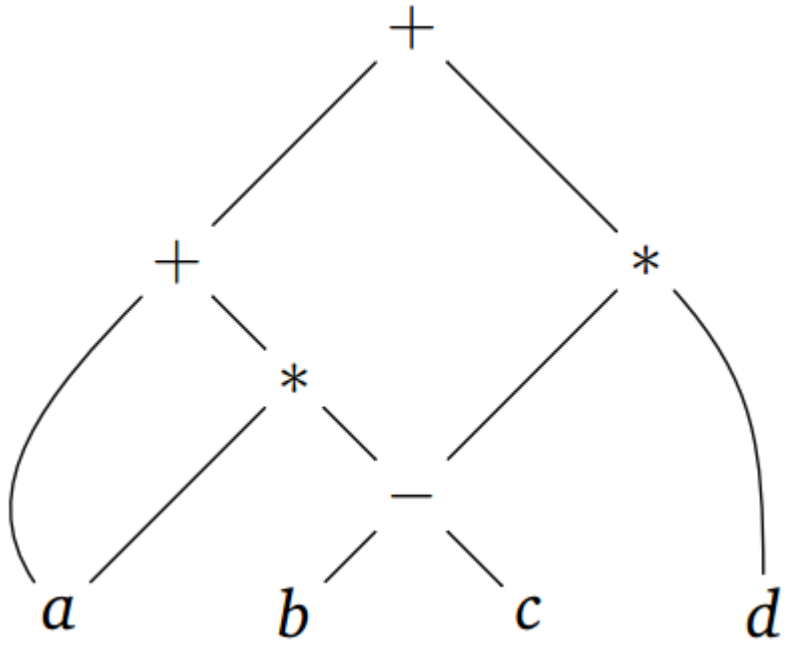
DAG

SDD for Constructing DAG

	PRODUCTION	SEMANTIC RULES
1)	$E \rightarrow E_1 + T$	$E.node = \textbf{new}\ Node('+', E_1.node, T.node)$
2)	$E \rightarrow E_1 - T$	$E.node = \textbf{new}\ Node('-', E_1.node, T.node)$
3)	$E \rightarrow T$	$E.node = T.node$
4)	$T \rightarrow (E)$	$T.node = E.node$
5)	$T \rightarrow \textbf{id}$	$T.node = \textbf{new}\ Leaf(\textbf{id}, \textbf{id}.entry)$
6)	$T \rightarrow \textbf{num}$	$T.node = \textbf{new}\ Leaf(\textbf{num}, \textbf{num}.val)$

- Wait!! What?? This SDD constructs Syntax Tree!
- The Leaf and Node functions will return a previously created identical node if exist, otherwise return a newly created one
- Identical Node means the subtree rooted at that node is same

SDD for constructing DAG

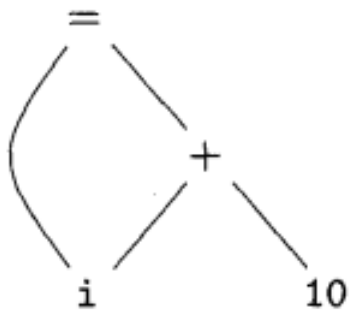


DAG for expression: $a + a * (b - c) + (b - c) * d$

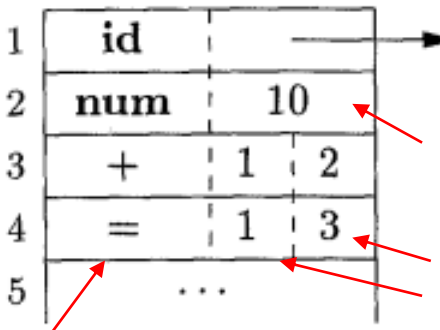
1) $p_1 = Leaf(\mathbf{id}, entry\text{-}a)$
2) $p_2 = Leaf(\mathbf{id}, entry\text{-}a) = p_1$
3) $p_3 = Leaf(\mathbf{id}, entry\text{-}b)$
4) $p_4 = Leaf(\mathbf{id}, entry\text{-}c)$
5) $p_5 = Node('-', p_3, p_4)$
6) $p_6 = Node('*', p_1, p_5)$
7) $p_7 = Node('+', p_1, p_6)$
8) $p_8 = Leaf(\mathbf{id}, entry\text{-}b) = p_3$
9) $p_9 = Leaf(\mathbf{id}, entry\text{-}c) = p_4$
10) $p_{10} = Node('-', p_3, p_4) = p_5$
11) $p_{11} = Leaf(\mathbf{id}, entry\text{-}d)$
12) $p_{12} = Node('*', p_5, p_{11})$
13) $p_{13} = Node('+', p_7, p_{12})$

Steps of constructing the left DAG

Value Number Method for Constructing DAG



(a) DAG



(b) Array.

to entry for i

Second Field of leaf node is lexical value

Second and Third Field of interior node contains index number of left and right child

First Field is an **operation code** (label of the node)

- Nodes of AST or DAG can be stored in array or records (structures)
- Each row represent one record (one node)
- Reference to a node is the index of the node in the array
  - The integer index is called the value number of the node
- The value number of + node is 3

Value Number Method

- Algorithm for constructing Nodes of DAG (Node and Leaf function)
  - Input: Label op, Node l and Node r (For leaf node $r = 0$)
  - Output: The value number of a node in the array with signature $< op, l, r >$
  - Method:
    - Search array for a node M with label op, left child l and right child r
    - If found return value number of M
    - Otherwise create in the array a new node N with label op, left child l and right child r
    - Return value number of N

Value Number Method

- Searching the entire array every time to check whether a node exists is expensive
- Use Hash Table
- Hash using $< op, l, r >$ as hash key

Three Address Code

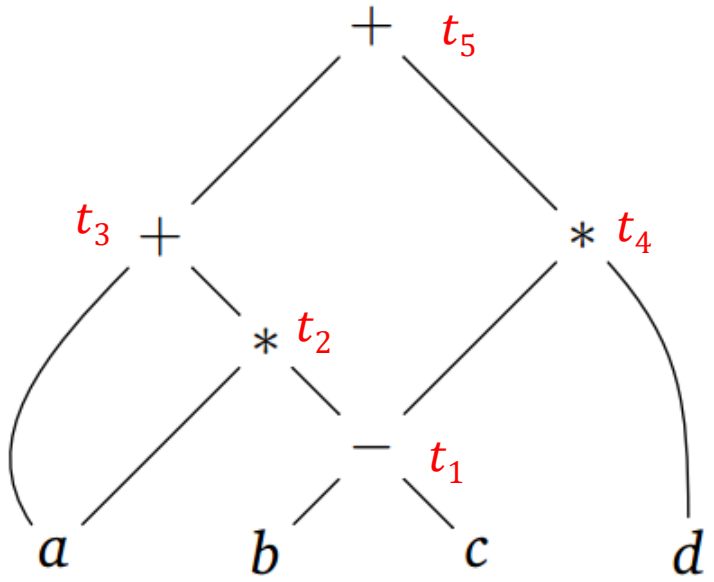
- At most one operator at the right side of an instruction
- For expression $x * y + z$ three address code is

$$t_1 = x * y$$
$$t_2 = z + t_1$$

- t_1 and t_2 are compiler generated names

Three Address Code

- Linearized representation for DAG
  - ➢Explicit name corresponds to the interior nodes
  - ➢Each instruction in the sequence has a label



DAG for expression: $a + a * (b - c) + (b - c) * d$

$$t_1 = b - c$$
$$t_2 = a * t_1$$
$$t_3 = a + t_2$$
$$t_4 = t_1 * d$$
$$t_5 = t_3 + t_4$$

Three Address Code

Three Address Code

- Based on Two concepts:
  - Address
    - What address can be used in the code
  - Instruction
    - What kind of instruction sets can be applied

Address

- Name
  - Name from source program
  - Example: x, y, z
  - May be replaced by pointer to symbol table entry in implementation
- Constant
  - Example: $2.3, 1$
- Compiler generated temporary
  - Example: t_1, t_2

Instructions

- Assignment instructions
  - ***x = y op z*** where x, y, z are addresses and op is a binary operator
  - ***x = op y*** where op is a unary operation such as unary minus, logical negation and conversion operator
- Copy instruction: ***x = y***
  - x is assigned the value of y
- An unconditional jump : *goto L*
  - Instruction with label L is executed next
- Conditional jumps:
  - ***if x goto L***
  - ***ifFalse x goto L***
  - ***if x relop y goto L***

Instructions

- Procedure calls and returns:
  - ***param x*** for parameters
  - ***call p, n*** or ***y = call p, n*** for procedure/function calls
  - ***return y*** represent returning a value
  - For procedure $proc(x_1, x_2, \dots, x_n)$

    param x_1

    param x_2

    …

    paramx_n

    call proc,n

Instructions

- Indexed copy instructions:
  - ***x = y[i]*** sets x to the value in the location *i* memory units beyond location *y*.
  - ***x[i] = y*** sets the contents of the location *i* units beyond x to the value of *y*

- Address and Pointers assignments:
  - x = & y
  - x = * y
  - * x = y

Assigning Labels

- Example: do i = i +1; while(a[i] < v);

$$
\begin{aligned}
L: \quad & t_1 = i + 1 \\
& i = t_1 \\
& t_2 = i * 8 \\
& t_3 = a[t_2] \\
& if\ t_3 < v\ goto\ L
\end{aligned}
$$

Symbolic Labels

$$
\begin{aligned}
100: \quad & t_1 = i + 1 \\
101: \quad & i = t_1 \\
102: \quad & t_2 = i * 8 \\
103: \quad & t_3 = a[t_2] \\
104: \quad & if\ t_3 < v\ goto\ 100
\end{aligned}
$$

Position Numbers

Choice of Operators

- The choice of allowable operators is an important issue
  - The operator set clearly must be rich enough to implement the operations in the source language.
  - Operators that are close to machine instructions make it easier to implement the intermediate form on a target machine.
  - However, if the front end must generate long sequences of instructions for some source-language operations, then the optimizer and code generator may have to work harder to rediscover the structure and generate good code for these operations.

Representing Three Address Code

- What is the suitable data structure to represent three address code?
- Can be represented using objects or record with operators and operands as fields
- Three representations:
  - Quadruples
  - Triples
  - Indirect Triples

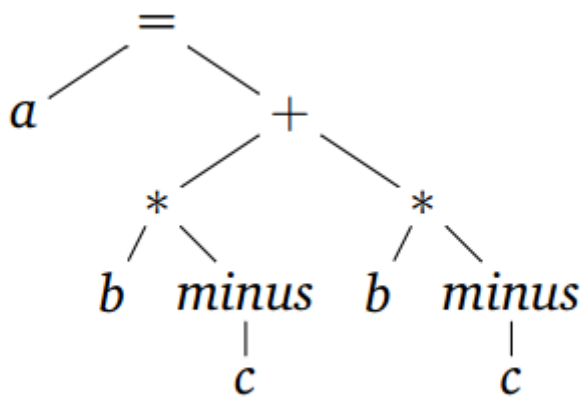
Quadruples

- Four Fields:
  - *op* – represent operator
  - *arg1*, *arg2* – holding operands
  - *result* – hold result of the operation

- Exceptions:
  - Unary Operators: no *arg2*
  - param operator: no *arg2*, no *result*
  - (Un)conditional jump: target label is the *result*

Quadruples

- Example: $a = b * -c + b * -c;$



Syntax Tree

$$t_1 = minus\ c$$
$$t_2 = b * t_1$$
$$t_3 = minus\ c$$
$$t_4 = b * t_3$$
$$t_5 = t_2 + t_4$$
$$a = t_5$$

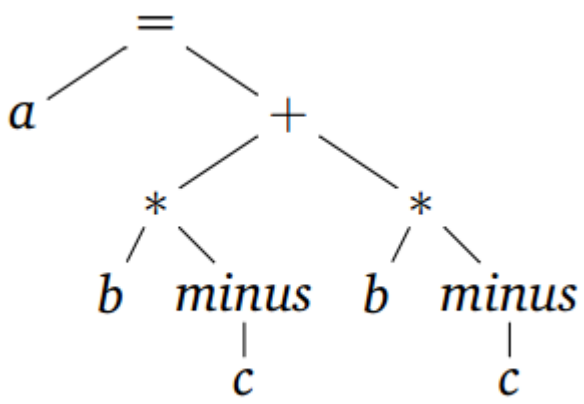
Three Address Code

	OP	ARG1	ARG2	RESULT
0	*minus*	c		t_1
1	$*$	b	t_1	t_2
2	*minus*	c		t_3
3	$*$	b	t_3	t_4
4	$+$	t_2	t_4	t_5
5	$=$	t_5		a

Quadruples

Triples

- Three fields:
  - *op*, *arg1*, *arg2*
  - No *result field*
  - *result* referred by their position
- *Example:* $a = b * -c + b * -c;$



Syntax Tree

$$t_1 = minus\ c$$
$$t_2 = b * t_1$$
$$t_3 = minus\ c$$
$$t_4 = b * t_3$$
$$t_5 = t_2 + t_4$$
$$a = t_5$$

Three Address Code

	OP	ARG1	ARG2
0	*minus*	*c*	
1	*	*b*	(0)
2	*minus*	*c*	
3	*	*b*	(2)
4	+	(1)	(3)
5	=	*a*	(4)

Triples

Triples

- Optimization by moving instructions is difficult in Triples, but easy in Quadruples
- Moving instructions forces to adjust the arguments where that position appear

	OP	ARG1	ARG2
0	*minus*	c	
1	$*$	b	(0)
2	*minus*	c	
3	$*$	b	(2)
4	$+$	(1)	(3)
5	$=$	a	(4)

Triples

Indirect Triples

- Listing pointers to the triples instead of triples
- Reorder the pointers instead of the tuples itself during code motion

	OP	ARG1	ARG2
0	*minus*	*c*	
1	$*$	*b*	(0)
2	*minus*	*c*	
3	$*$	*b*	(2)
4	$+$	(1)	(3)
5	$=$	*a*	(4)

Triples

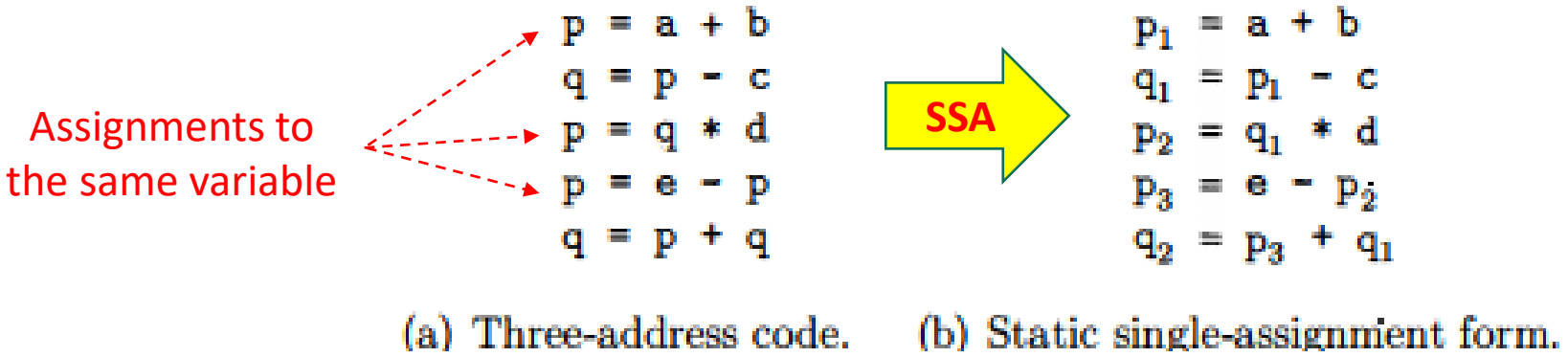
	INSTRUCTION
35	(0)
36	(1)
37	(2)
38	(3)
39	(4)
40	(5)

	OP	ARG1	ARG2
0	*minus*	*c*	
1	$*$	*b*	(0)
2	*minus*	*c*	
3	$*$	*b*	(2)
4	$+$	(1)	(3)
5	$=$	*a*	(4)

Indirect Triples

Static Single Assignment Form (SSA)

- Another form of IR
- All assignments are to variables with distinct names



- Facilitates code optimizations

Static Single Assignment Form (SSA)

- Converting ordinary code into SSA form
  - Replace the target of each assignment with a new variable
  - Replacing each use of a variable with the "version" of the variable reaching that point.
- What to do when branching is involved?

```
if (flag) x = -1
else x = 1
y = x*a
```

```
if (flag) x1 = -1
else x2 = 1
y = x?*a
```

- Φ functions to combine two definitions of the same variable x

```
if (flag) x = -1; else x = 1
y = x*a
```

```
if (flag) x1 = -1; else x2 = 1
x3 = φ(x1, x2)
y = x3*a
```

For further details see: https://en.wikipedia.org/wiki/Static_single-assignment_form

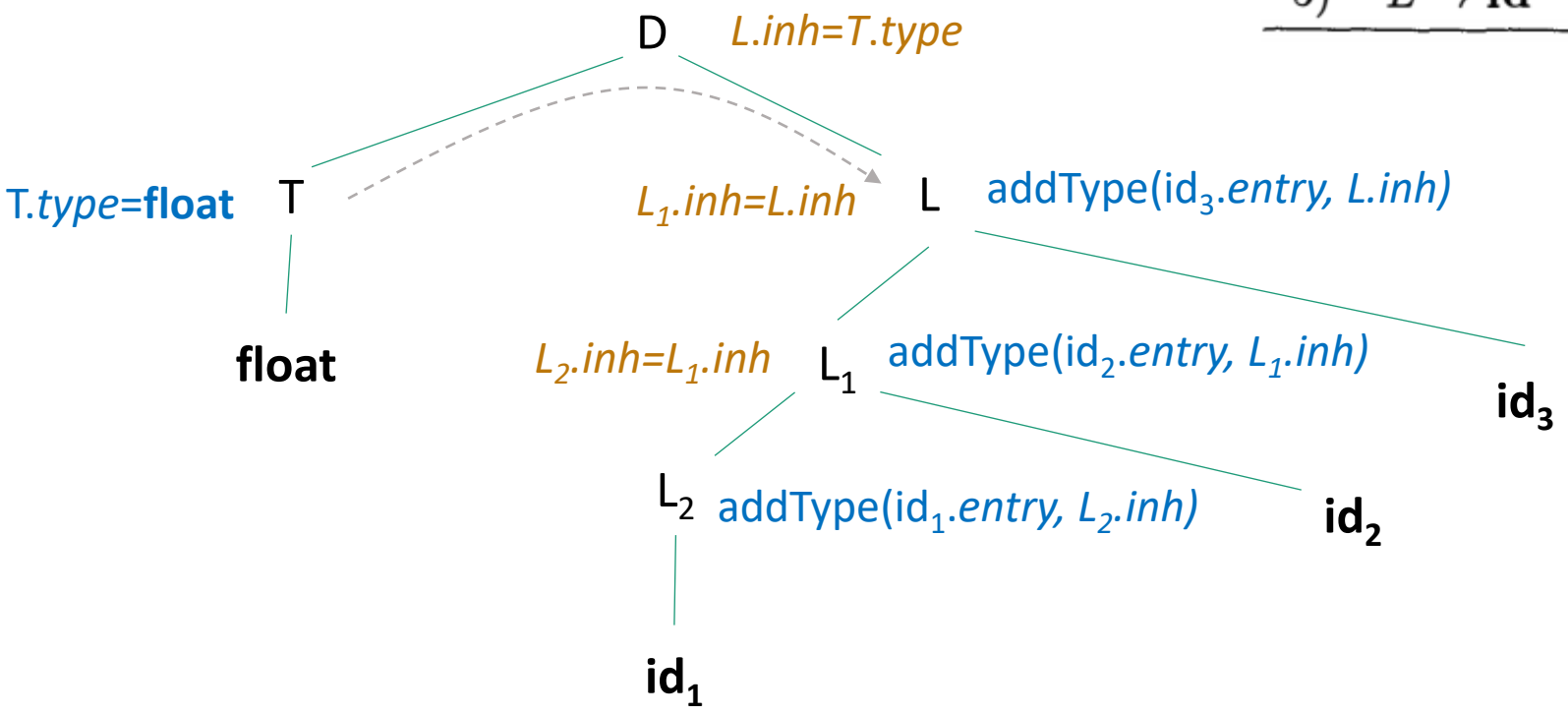
Types and Declarations

- Application of types
  - Type Checking
    - Uses logical rules to reason about the behavior of a program at run time
    - Ensures that the types of the operands match the type expected by an operator
    - **Example:** the && operator in java expects it two operands to be booleans; the result type is also boolean
  - Translation Applications
    - From the type of a name, a compiler can determine its required storage at runtime
    - Type information is also needed
      - to calculate the address denoted by an array reference
      - to insert explicit type conversion
      - to choose the right version of an arithmetic operator

Computing Types

- Let us compute an SDD to support declarations like
  - int id1, id2, id3
  - float id1, id2, id3

	PRODUCTION	SEMANTIC RULES
1)	$D \rightarrow T\ L$	$L.inh = T.type$
2)	$T \rightarrow \mathbf{int}$	$T.type = \text{integer}$
3)	$T \rightarrow \mathbf{float}$	$T.type = \text{float}$
4)	$L \rightarrow L_1\ ,\ \mathbf{id}$	$L_1.inh = L.inh$ $addType(\mathbf{id}.entry, L.inh)$
5)	$L \rightarrow \mathbf{id}$	$addType(\mathbf{id}.entry, L.inh)$



Parse tree for the declaration:
float id1, id2, id3

Type Expressions

- Type expressions are used to represent structures of types
- A type expression is either a basic type or is formed by applying an operator called a *type constructor* to a *type expression*
- The sets of basic types and constructors depend on the language to be checked

Example: The array type `int[2][3]` can be

- read as "array of two arrays of 3 integers each"
- written as a type expression array(2,array(3,integer))
  - the operator array takes two parameters, a number and a type

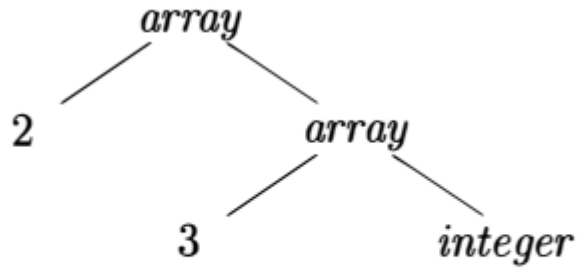


Figure 6.14: Type expression for `int[2][3]`

- Type expressions can be represented using a graph and the value-number method can used to store type expressions.

Definition of Type Expressions

- A basic type is a type expression such as *boolean, float, char, integer and void*
- A type name is a type expression such as name of a record, structure or class
- A type expression can be formed by applying the array type constructor to a number and a type expression
- A record is a data structure with named fields. A type name can be formed by applying the record type constructor to the field names and their types
- A type expression can be formed by using the type constructor → for function types.
  - s→t denotes "function from type s to type t
- If s and t are type expressions, then their Cartesian product s x t is a type expression.
  - Is used to represent a list or tuple of tuples (e.g., for function parameters)
- Type expressions may contain variables whose values are type expressions

Type Equivalence

- Many type-checking rules have the form, "if two type expressions are equal then return a certain type else error"
- When type expressions are represented by graphs, two types are structurally equivalent if and only if one of the following conditions hold
  - They are the same basic type.
  - They are formed by applying the same constructor to structurally equivalent types
  - One is a type name that denotes the other

Storage Layout for Local Names

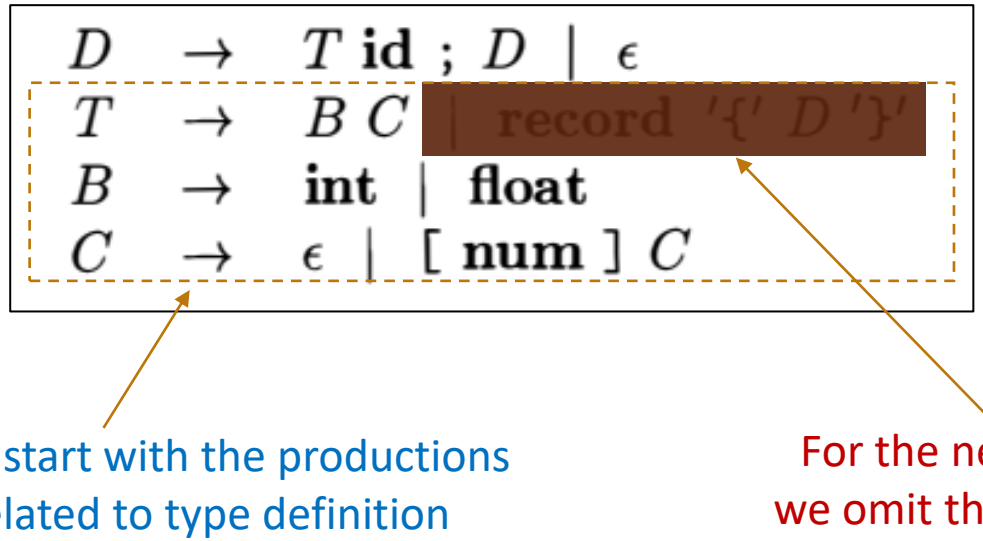
- From the type of a name, the amount of storage required for it during runtime can be determined
- At compile time, these amounts can be used to assign each name a relative address
- The type and relative address are saved in the symbol-table entry for the name
- Data of varying length, such as strings, dynamic arrays is handled by reserving a known fixed amount of storage for a pointer to the data

Storage Layout for Local Names (2)

- The width of a type is the number of storage units needed for objects of that type
  - we consider a byte as the smallest unit of addressable memory
- A basic type, such as a character, integer, or float, requires an integral number of bytes
- For easy access, storage for aggregates such as arrays and classes is allocated in one contiguous block of bytes

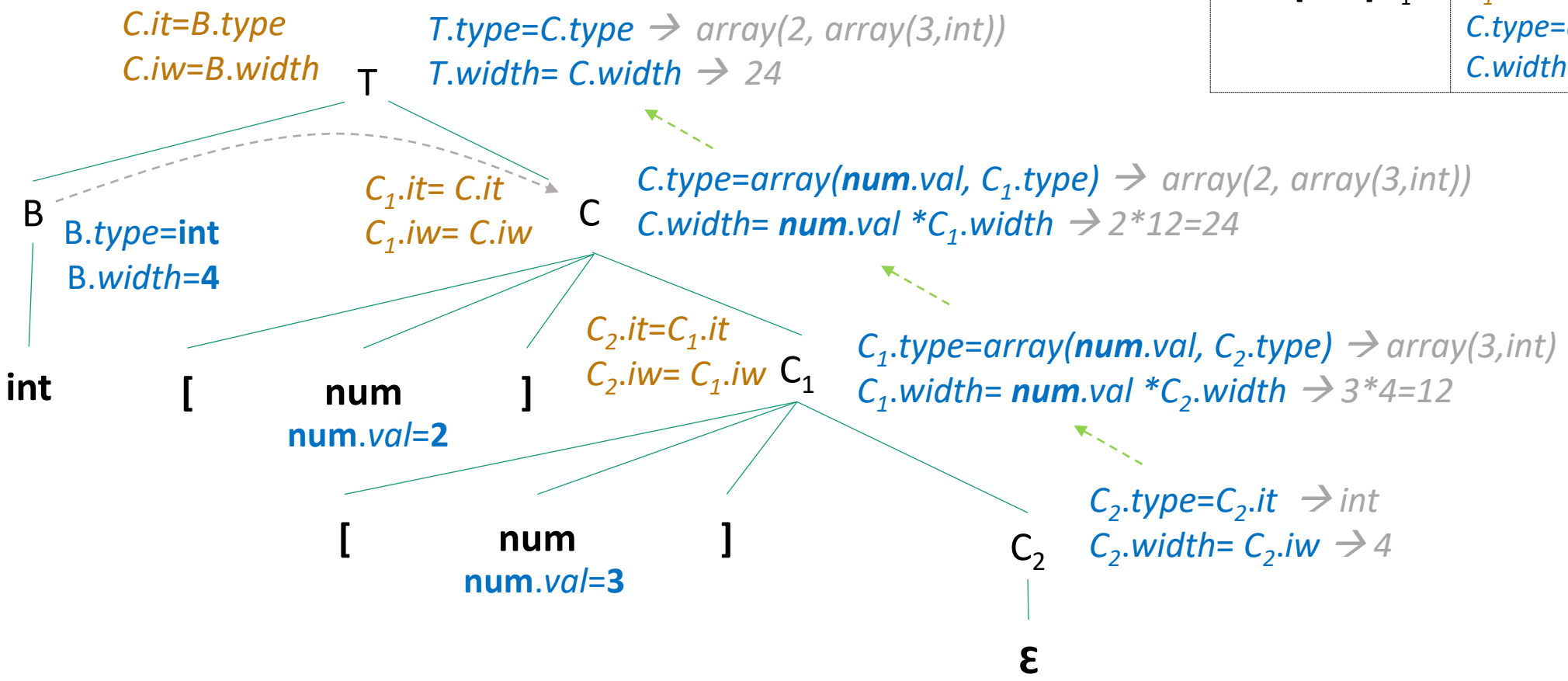
Computing Types and Their Widths

- A modified grammar
- Supports declarations like
  - int id; float id;
  - int[2][3][3] id;
  - record { int id; int[2][3] id2} id



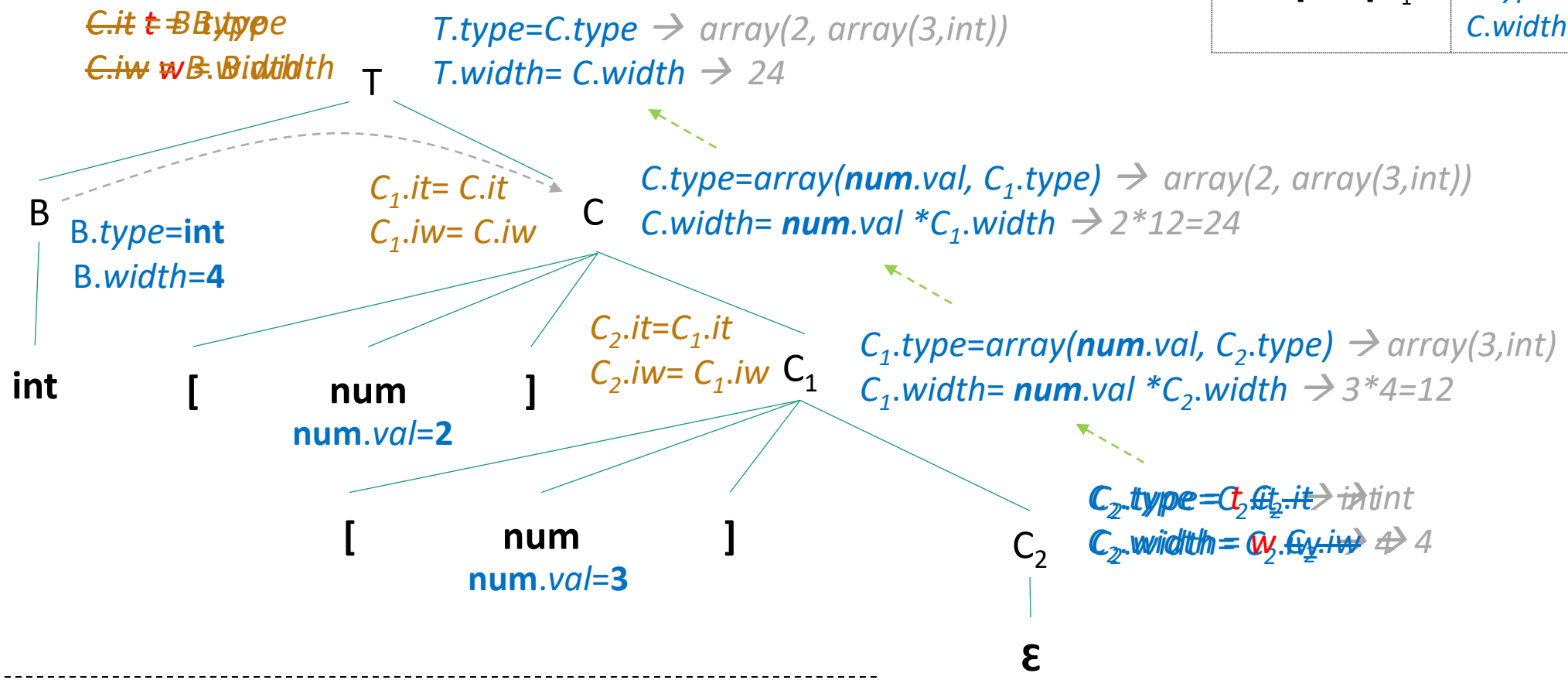
Prod.	Semantic Rules
T → B C	*C.it=B.type, C.iw=B.width* *T.type=C.type T.width= C.width*
B → **int**	B.*type*=**float,** B.*width*=**4**
B → **float**	B.*type*=**int,** B.*width*=**8**
C → ε	*C.type=C.it, C.width= C.iw*
C → [num] C_1	*C_1.it= C.it, C_1.iw= C.iw* *C.type=array(**num**.val, C_1.type)* *C.width= **num**.val *C_1.width*

Annotated Parse tree for **int [2][3]**



Prod.	Semantic Rules
T → B C	*t=B.type, w=B.width* *T.type=C.type T.width= C.width*
B → **int**	B.*type*=**float,** B.*width*=**4**
B → **float**	B.*type*=**int,** B.*width*=**8**
C → ε	*C.type = t, C.width = w*
C → [num] C_1	*C.type=array(**num**.val, C_1.type)* *C.width= **num**.val *C_1.width*

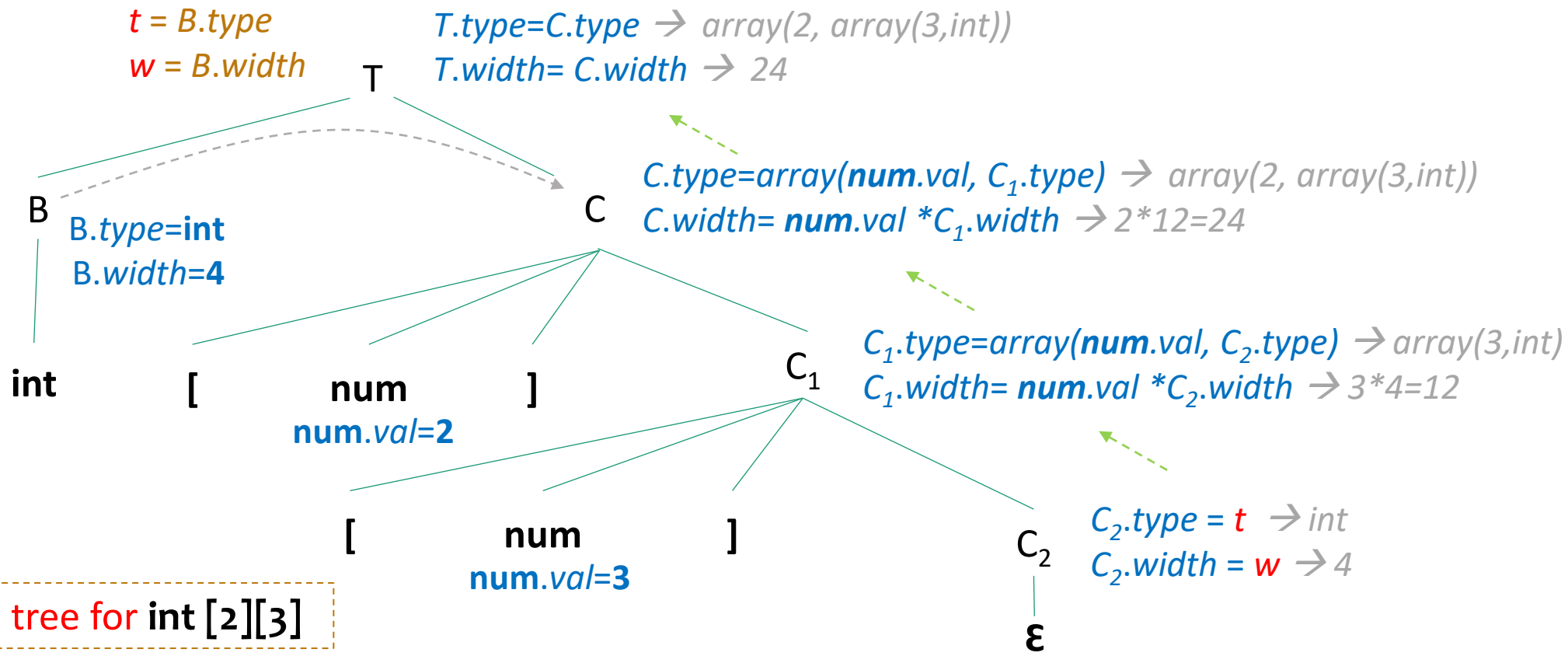
Annotated Parse tree for **int [2][3]**



Use of global variables instead of inherited attributes

SDT for Computing Type and Width	
T → B	{ *t* =*B.type*, *w*=*B.width* }
C	{ *T.type*=*C.type T.width*= *C.width* }
B → **int**	{ B.*type*=**float,** B.*width*=**4** }
B → **float**	{ B.*type*=**int,** B.*width*=**8** }
C → ε	{ *C.type* = *t*, *C.width* = *w* }
C → [num] C_1	{ *C.type*=*array*(***num***.*val*, C_1.*type*) *C.width*= ***num***.*val* *C_1.*width* }

Prod.	Semantic Rules
T → B C	*t*=*B.type*, *w*=*B.width* *T.type*=*C.type T.width*= *C.width*
B → **int**	B.*type*=**float,** B.*width*=**4**
B → **float**	B.*type*=**int,** B.*width*=**8**
C → ε	*C.type* = *t*, *C.width* = *w*
C → [num] C_1	*C.type*=*array*(***num***.*val*, C_1.*type*) *C.width*= ***num***.*val* *C_1.*width*



Annotated Parse tree for **int [2][3]**

Computing Relative Addresses

```cpp
int main(){
    int a,b,c;
    a=1;
    b=2;
    c=1;
    return 0;
}
```

CPP

```asm
main:
        push    rbp
        mov     rbp, rsp
        mov     DWORD PTR [rbp-4], 1
        mov     DWORD PTR [rbp-8], 2
        mov     DWORD PTR [rbp-12], 1
        mov     eax, 0
        pop     rbp
        ret
```

ASM (x86-64 gcc 11.1)

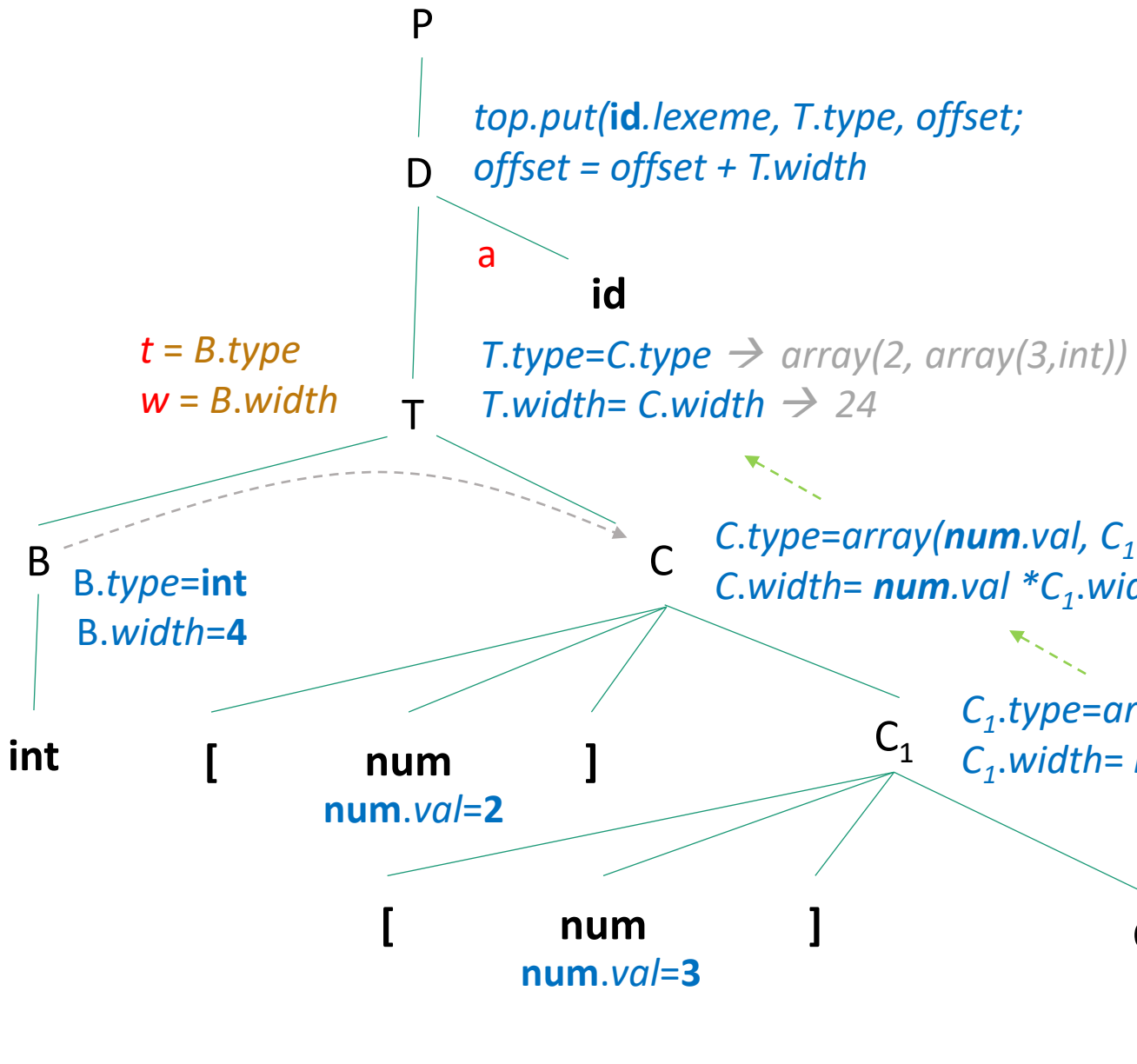
$$P \rightarrow \{\ offset = 0;\ \}\ D$$

$$D \rightarrow T\ \mathbf{id}\ ;\quad \{\ top.put(\mathbf{id}.lexeme,\ T.type,\ offset);\ offset = offset + T.width;\ \}\ D_1$$

$$D \rightarrow \epsilon$$

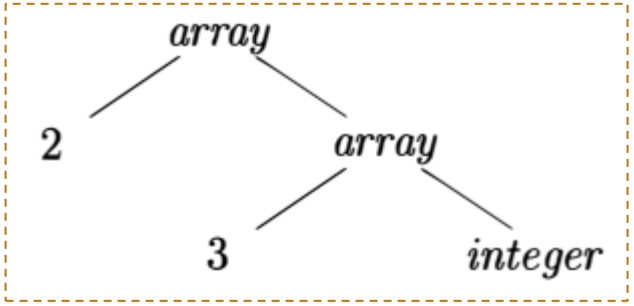
SDT for computing relative addresses

Annotated Parse tree for int [2][3] a

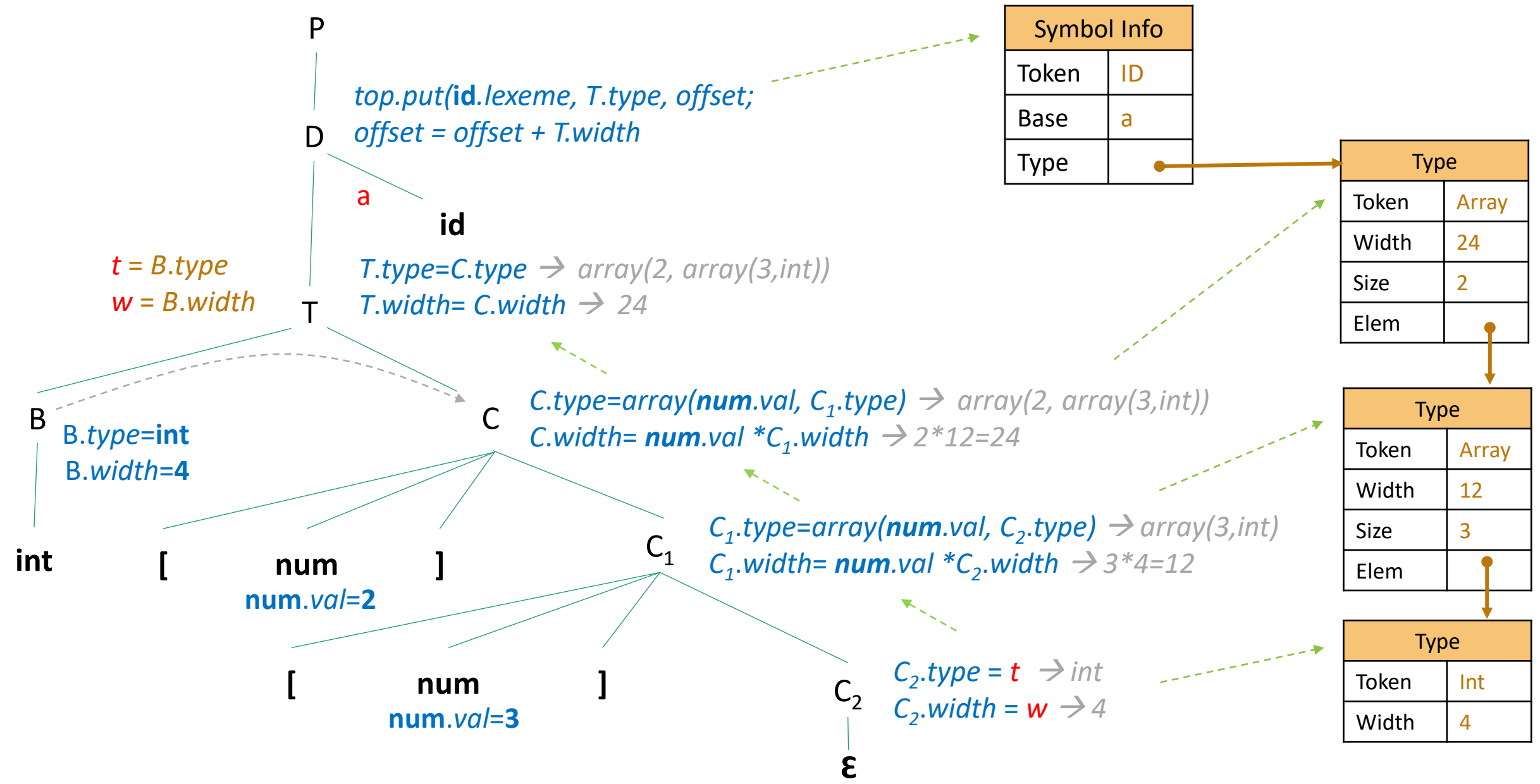


SDT for Computing Type and Width	
P → D	*{offset = 0;}*
D → T **id**	{ *top.put(**id**.lexeme, T.type, offset; offset = offset + T.width*}
T → B C	{ *t =B.type, w=B.width* } { *T.type=C.type T.width= C.width* }
B → **int**	{ B.*type*=**float,** B.*width*=**4** }
B → **float**	{ B.*type*=**int,** B.*width*=**8** }
C → ε	{ *C.type = t, C.width = w* }
C → [num] C_1	{ *C.type=array(**num**.val, C_1.type) C.width= **num**.val *C_1.width* }

Type expression for int[2][3]



Annotated Parse tree for **int [2][3] a**



Fields in Records and Classes

```cpp
struct Info {
    int x, y, z;
};
int main(){
    struct Info a;
    struct Info b;
    int x;
    x=a.x+b.x;
    return 0;
}
```

CPP

```asm
main:
        push    rbp
        mov     rbp, rsp
        mov     edx, DWORD PTR [rbp-16]
        mov     eax, DWORD PTR [rbp-28]
        add     eax, edx
        mov     DWORD PTR [rbp-4], eax
        mov     eax, 0
        pop     rbp
        ret
```

ASM (x86-64 gcc 11.1)

$$T \rightarrow \textbf{record}\ '\{'\ D\ '\}'$$

$$T \rightarrow \textbf{record}\ '\{'\quad \{\ Env.push(top);\ top = \textbf{new}\ Env();\ Stack.push(offset);\ offset = 0;\ \}$$

$$D\ '\}'\quad \{\ T.type = record(top);\ T.width = offset;\ top = Env.pop();\ offset = Stack.pop();\ \}$$

type constructor

symbol table object

Handling of field names in records

Three Address Code Generation

An Example

```
1) {                        // File test
2)     int i; int j; float v; float x; float[100] a;
3)     while( true ) {
4)         do i = i+1; while( a[i] < v);
5)         do j = j-1; while( a[j] > v);
6)         if( i >= j ) break;
7)         x = a[i]; a[i] = a[j]; a[j] = x;
8)     }
9) }
```

```
 1) L1:L3:   i = i + 1
 2) L5:      t1 = i * 8
 3)          t2 = a [ t1 ]
 4)          if t2 < v goto L3
 5) L4:      j = j - 1
 6) L7:      t3 = j * 8
 7)          t4 = a [ t3 ]
 8)          if t4 > v goto L4
 9) L6:      iffalse i >= j goto L8
10) L9:     goto L2
11) L8:     t5 = i * 8
12)         x = a [ t5 ]
13) L10:    t6 = i * 8
14)         t7 = j * 8
15)         t8 = a [ t7 ]
16)         a [ t6 ] = t8
17) L11:    t9 = j * 8
18)         a [ t9 ] = x
19)         goto L1
20) L2:
```

Translation of Expressions

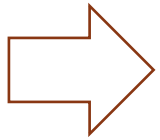
- Need to convert an expression with more than one operator in source program to Three Address Codes with only one operator
  - $a * b + c$
- An array reference will expand in sequence of Three Address Codes that calculate the address for the reference
  - $A[i][j]$

Translation of Expressions

```
int initial = 32;
float rate = .8;
float position = initial + rate * 60 ;
```

E

Source code



```
int t1 = 32
int initial = t1
float t2 = .8
float rate = t2
int t3 = initial
float t4 = rate
int t5 = 60
float t6 = (float) t3
float t7 = t4 * t5
float t8 = t6 + t7
```

E.code

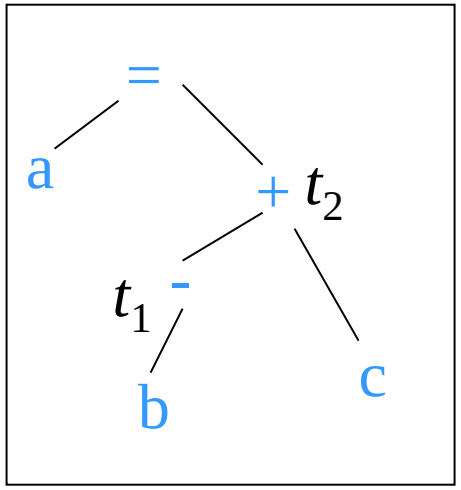
↓ E.addr

```
float position = t8
```

Corresponding Three Address Code

Translation of Expressions

An Example Expression: $a = -(b) + c$

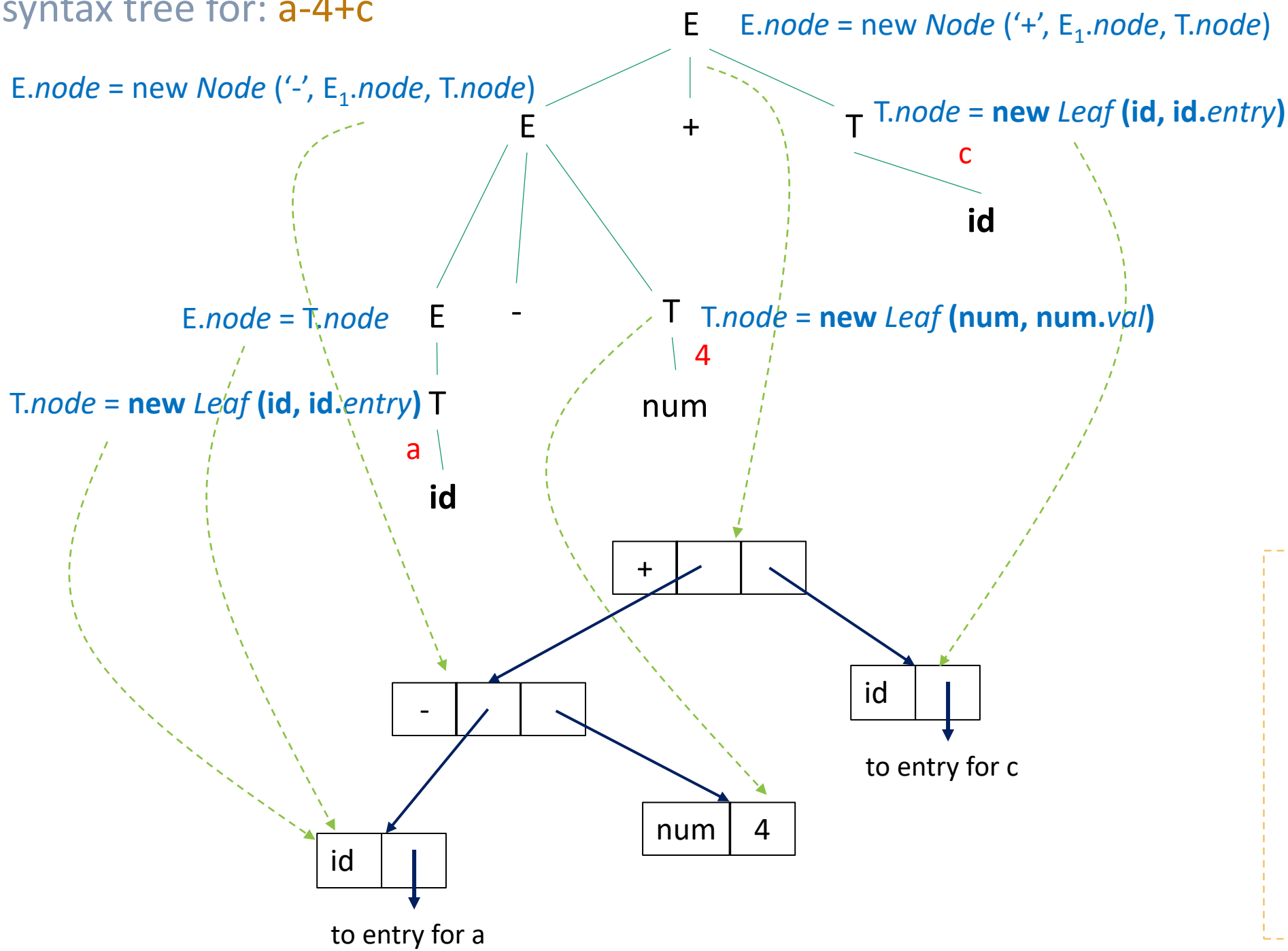


Syntax Tree

$$t_1 = \textbf{minus}\ b$$
$$t_2 = t_1 + c$$
$$a = t_2$$

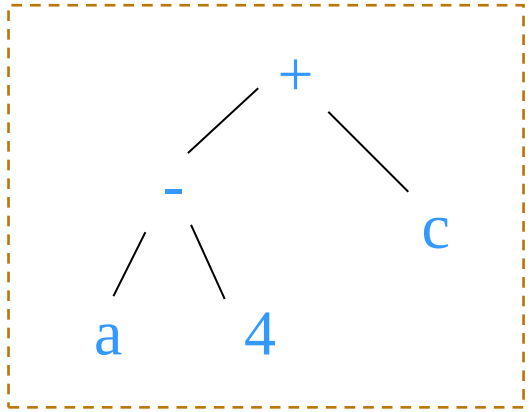
Three Address Code

Construct syntax tree for: a-4+c

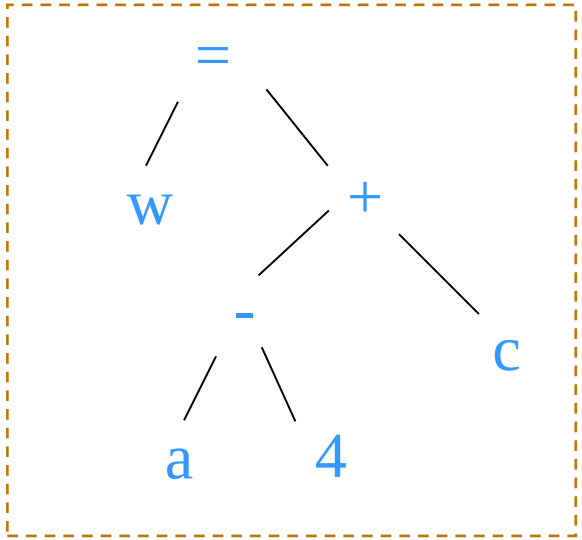


Production
$E \rightarrow E_1 + T$
$E \rightarrow E_1 - T$
$E \rightarrow T$
$T \rightarrow (E)$
$T \rightarrow$ **id**
$T \rightarrow$ num

AST for a-4+c



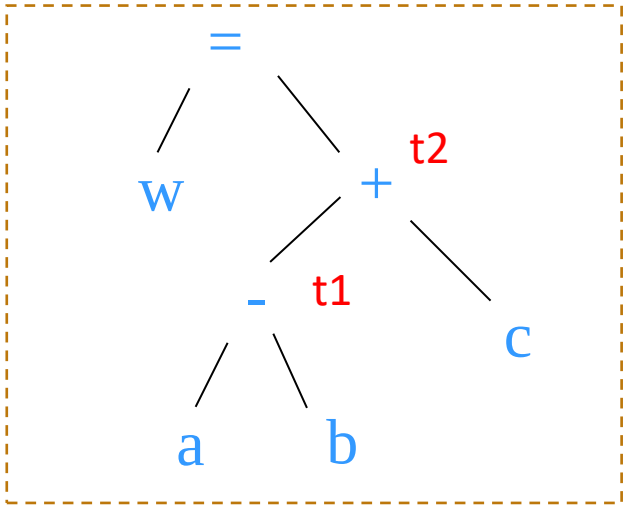
AST for w=a-4+c



Prod.	Semantic Rules
E → E_1 + T	E.*node* = new *Node* ('+', E_1.*node*, T.*node*)
E → E_1 – T	E.*node* = new *Node* ('-', E_1.*node*, T.*node*)
E → T	E.*node* = T.*node*
T → (*E*)	E.*node* = E.*node*
T → **id**	T.*node* = **new** *Leaf* **(id, id.***entry***)**
T → **num**	T.*node* = **new** *Leaf* **(num, num.***val***)**

Prod.	Semantic Rules
S → **id** = E	S.*node* = new *Node* ('=', new Leaf (id, id.entry), E.*node*)
E → E_1 + T	E.*node* = new *Node* ('+', E_1.*node*, T.*node*)
E → E_1 – T	E.*node* = new *Node* ('-', E_1.*node*, T.*node*)
E → T	E.*node* = T.*node*
T → (*E*)	E.*node* = E.*node*
T → **id**	T.*node* = **new** *Leaf* **(id, id.***entry***)**
T → **num**	T.*node* = **new** *Leaf* **(num, num.***val***)**

AST for w=a-b+c



Three address code for w=a-4+c

```
t1 = a - b
t2 = t1 + c
w = t2
```

Prod.	Semantic Rules for AST
S → **id** = E	S.*node* = new *Node* (‘=‘, new Leaf (id, id.entry), E.*node*)
E → E_1 + T	E.*node* = new *Node* (‘+’, E_1.*node*, T.*node*)
E → E_1 – T	E.*node* = new *Node* (‘-’, E_1.*node*, T.*node*)
E → T	E.*node* = T.*node*
T → (*E*)	T.*node* = E.*node*
T → **id**	T.*node* = **new** *Leaf* **(id, id.***entry***)**

Prod.	Semantic Rules Three Address Code
S → **id** = E	*S.code = top.get*(**id**.lexeme) || ‘=‘ || E.addr
E → E_1 + T	E.addr = new Temp() E.*code* = E.addr || ‘=‘ || E_1.addr || ‘+’ || T.addr
E → E_1 – T	E.addr = new Temp() E.*code* = E.addr || ‘=‘ || E_1.addr || ‘-’ || T.addr
E → T	*E.code =‘’, E.addr = T.addr*
T → (*E*)	T.code =‘’, T.addr = *E.addr*
T → **id**	T.code =‘’, T.addr = *top.get*(**id**.lexeme)

Returns a pointer to symbol table of id

Three address code for w=a-b+c

Prod.	Semantic Rules Three Address Code
S → **id** = E	*S.code = top.get*(**id**.lexeme) || '=' || E.addr
E → E_1 + T	E.addr = new Temp() E.*code* = E.addr || '=' || E_1.addr || '+' || T.addr
E → E_1 – T	E.addr = new Temp() E.*code* = E.addr || '=' || E_1.addr || '-' || T.addr
E → T	*E.code =", E.addr = T.addr*
T → (*E*)	T.code =", T.addr = *E.addr*
T → **id**	T.code =", T.addr = *top.get*(**id**.lexeme)

But S.code produces a single line!

S.*code* = *top.get*(**id**.lexeme) || '=' || E.addr → *w= t2*

E.addr = new Temp() → *t2*
E.*code* = E.addr || '=' ||
E_1.addr || '+' || T.addr
→ *t2= t1 + c*

E_1.addr = new Temp() → *t1*
E_1.*code* = E_1.addr || '=' ||
E_2.addr || '-' || T.addr
→ *t1= a -b*

T.code ="
T.addr = *top.get*(**id**.lexeme) → *c*

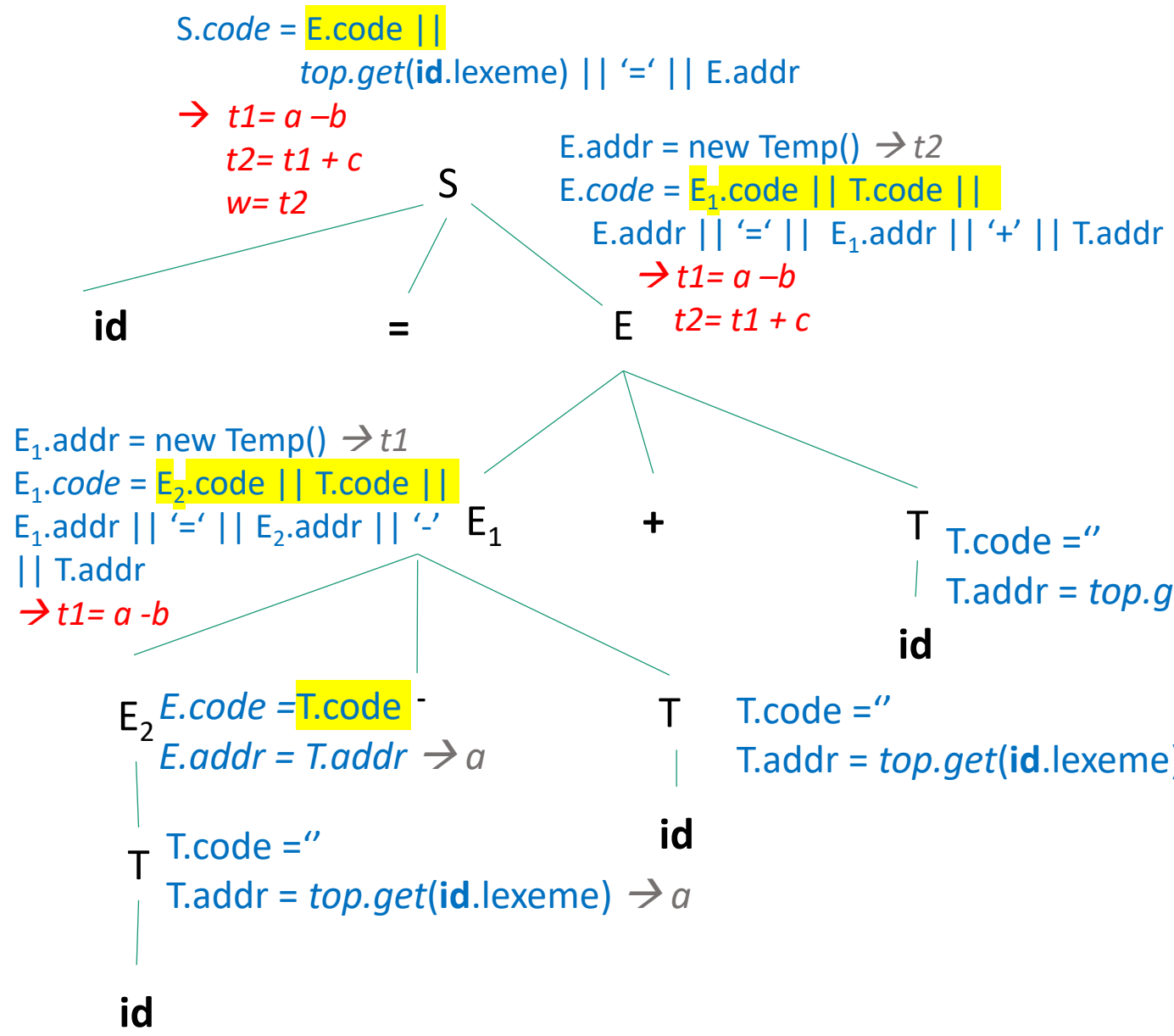
E_2 *E.code ="*
E.addr = T.addr → *a*

T.code ="
T.addr = *top.get*(**id**.lexeme) → *a*

T.code ="
T.addr = *top.get*(**id**.lexeme) → *b*

w=a-b+c ⇒
t1 = a - b
t2 = t1 + c
w = t2

Three address code for w=a-b+c



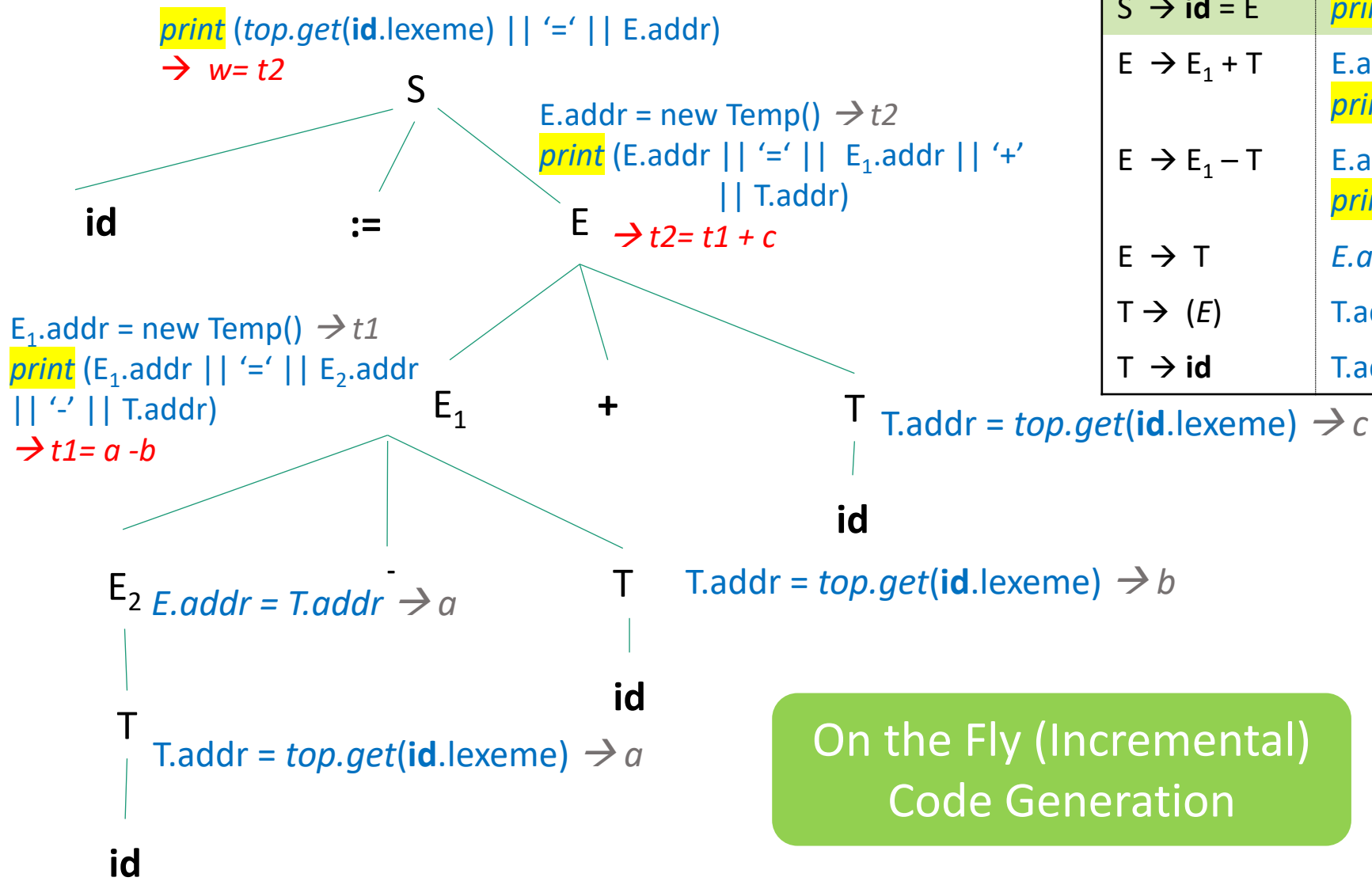
Prod.	Semantic Rules Three Address Code
S → **id** = E	*S.code* = *E*.code || *top.get*(**id**.lexeme) || '=' || E.addr
E → E_1 + T	E.addr = new Temp() E.*code* = E_1.code || T.code || E.addr || '=' || E_1.addr || '+' || T.addr
E → E_1 – T	E.addr = new Temp() E.*code* = E_1.code || T.code || E.addr || '=' || E_1.addr || '-' || T.addr
E → T	*E.code* =T.code, *E.addr* = *T.addr*
T → (*E*)	T.code =E.code, T.addr = *E.addr*
T → **id**	T.code ='', T.addr = *top.get*(**id**.lexeme)

w=a-b+c ⇨

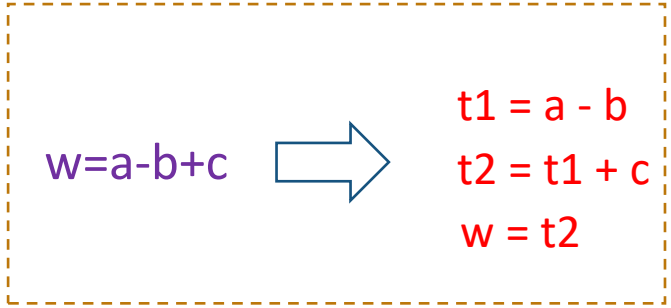
t1 = a - b
t2 = t1 + c
w = t2

Three address code for w:=a-b+c

Prod.	Semantic Rules Three Address Code
S → **id** = E	*print* (*top.get*(**id**.lexeme) || '=' || E.addr)
E → E_1 + T	E.addr = new Temp() *print* (E.addr || '=' || E_1.addr || '+' || T.addr)
E → E_1 – T	E.addr = new Temp() *print* (E.addr || '=' || E_1.addr || '-' || T.addr)
E → T	*E.addr = T.addr*
T → (*E*)	T.addr = *E.addr*
T → **id**	T.addr = *top.get*(**id**.lexeme)



On the Fly (Incremental) Code Generation



Translation of Expressions

CFG and SDD from your text

PRODUCTION	SEMANTIC RULES
$S \rightarrow \mathbf{id} = E\ ;$	$S.code = E.code \mid\mid gen(top.get(\mathbf{id}.lexeme)\ '='\ E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid E_2.code \mid\mid gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$
$\mid\ - E_1$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid gen(E.addr\ '='\ '\mathbf{minus}'\ E_1.addr)$
$\mid\ (\ E_1\ )$	$E.addr = E_1.addr$ $E.code = E_1.code$
$\mid\ \mathbf{id}$	$E.addr = top.get(\mathbf{id}.lexeme)$ $E.code = ''$

gen(…) builds an instruction and returns it

Incremental Translation

- Code attributes can be long strings, so they are usually generated incrementally
- Instead of building up E.code, generate only the new three address instructions
- The function gen(…), in addition to constructing new instruction, append to the stream

PRODUCTION	SEMANTIC RULES
$S \rightarrow \mathbf{id} = E\ ;$	$S.code = E.code \,\|\|$ $gen(top.get(\mathbf{id}.lexeme)\ '='\ E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \,\|\| E_2.code \,\|\|$ $gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$
$\mid\ - E_1$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \,\|\|$ $gen(E.addr\ '='\ '\mathbf{minus}'\ E_1.addr)$
$\mid\ (\ E_1\ )$	$E.addr = E_1.addr$ $E.code = E_1.code$
$\mid\ \mathbf{id}$	$E.addr = top.get(\mathbf{id}.lexeme)$ $E.code = ''$

$$
\begin{array}{lll}
S \rightarrow \mathbf{id} = E\ ; & \{\ gen(\ top.get(\mathbf{id}.lexeme)\ '='\ E.addr);\ \} \\
E \rightarrow E_1 + E_2 & \{\ E.addr = \mathbf{new}\ Temp();\ gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr);\ \} \\
\mid\ - E_1 & \{\ E.addr = \mathbf{new}\ Temp();\ gen(E.addr\ '='\ '\mathbf{minus}'\ E_1.addr);\ \} \\
\mid\ (\ E_1\ ) & \{\ E.addr = E_1.addr;\ \} \\
\mid\ \mathbf{id} & \{\ E.addr = top.get(\mathbf{id}.lexeme);\ \}
\end{array}
$$

Generating three-address code for expressions incrementally

Addressing Array Element

- In C and Java, array elements are numbered *0,1,…,n-1,* for an array of *n* elements
- If the width of each array element is *w*, then the *i*-th element of array A begins in location

$$base + i \times w$$

Here base is the relative address of A[0]

- The above formula generalized to *k*-dimensions as

$$base + i_1 \times w_1 + i_2 \times w_2 + \ldots + i_k \times w_k$$

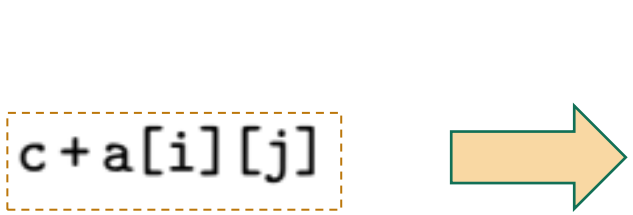
- For the array, int A[5][7][8], location of A[2][3][4] will be

$$base + 2 \times 224 + 3 \times 32 + 4 \times 4$$

Here size of integer is assumed as 4 byte

Translation of Array References

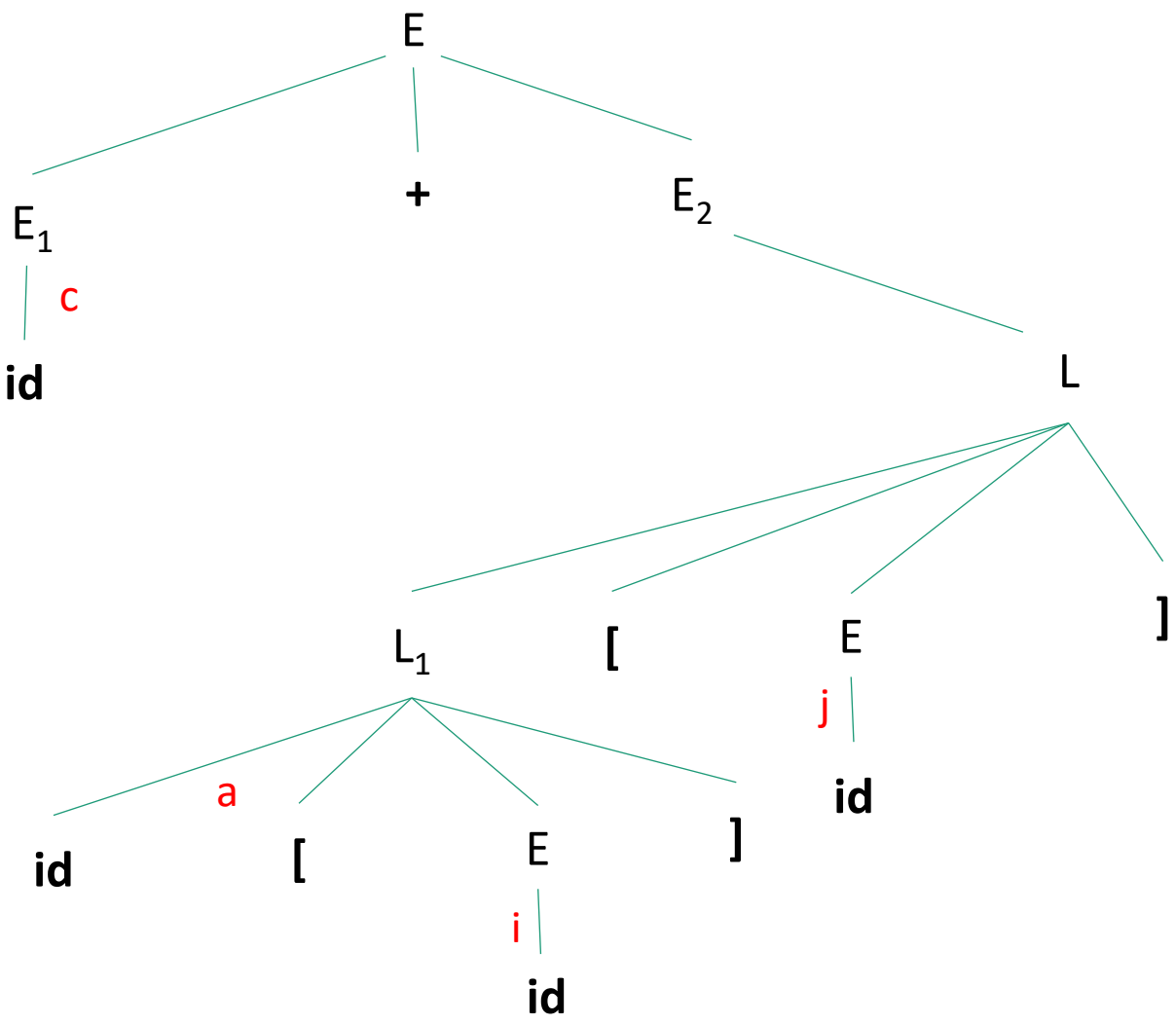
- The chief problem in generating code for array references is to related the address calculation formulas to a grammar for array references
- For the array *a* declared as int[2][3], three-address translation of the expression –



- We will use the following grammar production rule where nonterminal *L* generates an array name followed by a sequence of index expression.

$$L \rightarrow L\ [\ E\ ] \mid \mathbf{id}\ [\ E\ ]$$

Translation of array references for c+a[i][j]

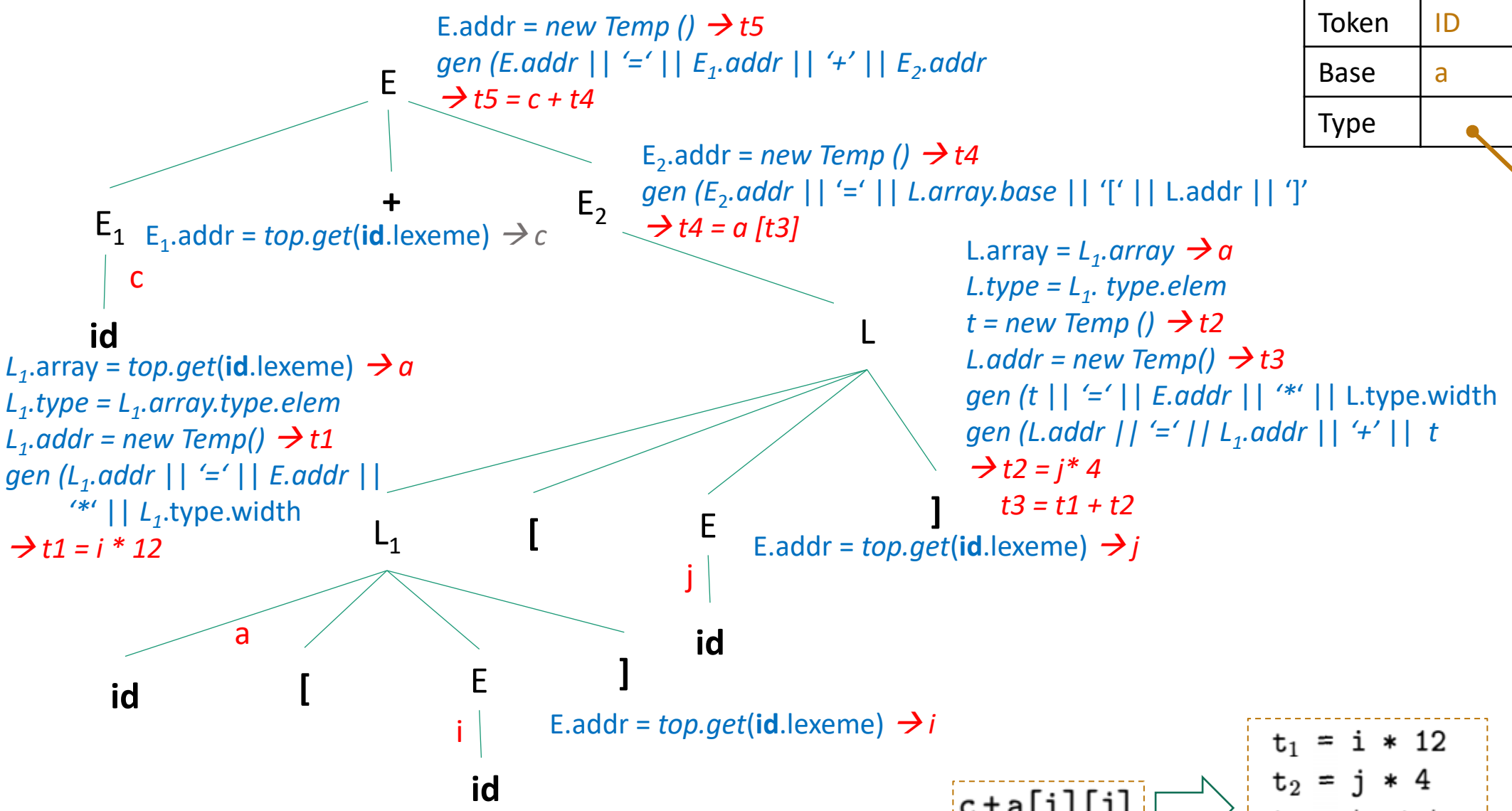


Prod.
S → **id** = E
S → L = E
E → E_1 + E_2
E → **id**
E → L
L → **id** [E]
L → L_1 [E]

c+a[i][j]

t_1 = i * 12
t_2 = j * 4
t_3 = t_1 + t_2
t_4 = a [t_3]
t_5 = c + t_4

Translation of array references for c+a[i][j]



Symbol Info	
Token	ID
Base	a
Type	

Type	
Token	Array
Width	24
Size	2
Elem	

Type	
Token	Array
Width	12
Size	3
Elem	

Type	
Token	Int
Width	4

c+a[i][j]

$t_1 = i * 12$
$t_2 = j * 4$
$t_3 = t_1 + t_2$
$t_4 = a [t_3]$
$t_5 = c + t_4$

Translation of Array References (2)

Attributes of *L*

L.addr - a temporary that holds the sum of the terms $i_k \times w_k$

L.array - a pointer to the symbol-table entry for the array name

L.type - type of the subarray generated by *L*

$S \rightarrow$	$\mathbf{id} = E\ ;$	$\{\ gen(\ top.get(\mathbf{id}.lexeme)\ '='\ E.addr);\ \}$
$\mid$	$L = E\ ;$	$\{\ gen(L.addr.base\ '['\ L.addr\ ']'\ '='\ E.addr);\ \}$
$E \rightarrow$	$E_1 + E_2$	$\{\ E.addr = \mathbf{new}\ Temp\,();$ $gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr);\ \}$
$\mid$	$\mathbf{id}$	$\{\ E.addr = top.get(\mathbf{id}.lexeme);\ \}$
$\mid$	L	$\{\ E.addr = \mathbf{new}\ Temp\,();$ $gen(E.addr\ '='\ L.array.base\ '['\ L.addr\ ']');\ \}$
$L \rightarrow$	$\mathbf{id}\ [\ E\ ]$	$\{\ L.array = top.get(\mathbf{id}.lexeme);$ $L.type = L.array.type.elem;$ $L.addr = \mathbf{new}\ Temp\,();$ $gen(L.addr\ '='\ E.addr\ '*'\ L.type.width);\ \}$
$\mid$	$L_1\ [\ E\ ]$	$\{\ L.array = L_1.array;$ $L.type = L_1.type.elem;$ $t = \mathbf{new}\ Temp\,();$ $L.addr = \mathbf{new}\ Temp\,();$ $gen(t\ '='\ E.addr\ '*'\ L.type.width);\ \}$ $gen(L.addr\ '='\ L_1.addr\ '+'\ t);\ \}$

Translation of Array References (2)

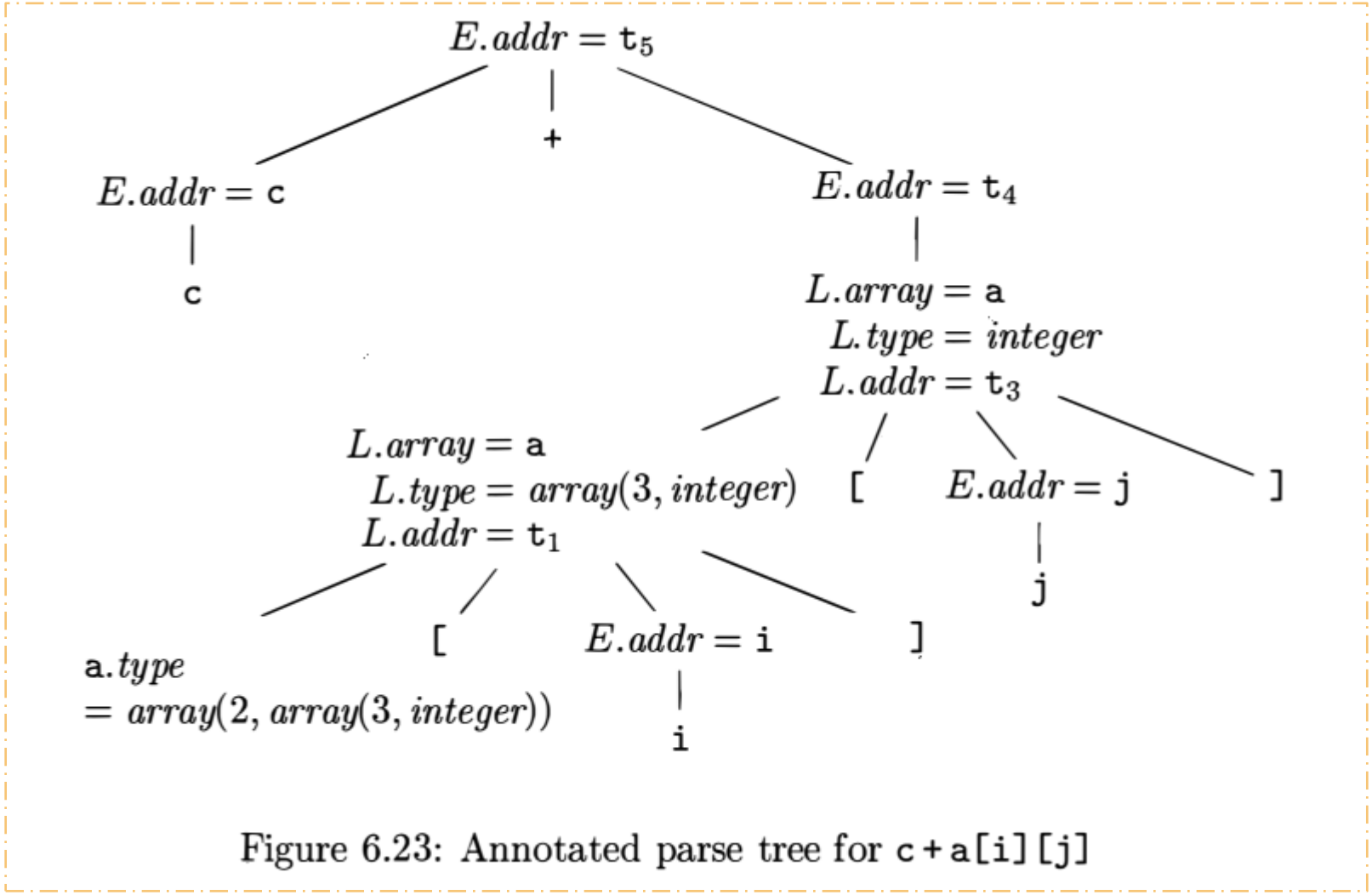


Figure 6.23: Annotated parse tree for c + a[i][j]

Control Flow

- We will translate control flow statements such as *if-else*, *while*
- Translation of control flow statements are tied with Boolean expressions
- Boolean expressions are composed of Boolean operators
- Boolean operators are applied to
  - Boolean variables
  - Relational expression with relational operator

Boolean Expression

- We consider Boolean expressions generated by following grammar

$$B \rightarrow B \,||\, B \mid B \,\&\&\, B \mid !\, B \mid (\, B \,) \mid E \,\mathbf{rel}\, E \mid \mathbf{true} \mid \mathbf{false}$$

- **rel** is relational operator which is any one of < , > , <= , >= , ==, !=

- We assume &&, || are left associative and precedence order is !, && then ||

Boolean Expression

- Boolean expressions are used to
  - Alter flow of control
    - If (B) then S: B must be true to reach S

```
if ( x < 100 || x > 200 && x != y )
      x = 0;
```

  - To compute logical values
    - True or False as value

```
p = x<a && (x>b || y<a);
```

Short Circuit Code

- Also called jumping code
- &&, || and ! are translated into jumps
- The operators don't appear in the code
- The value of a boolean expression is represented by a position in the code

Short Circuit Code

- Example:

A statement with Boolean expressions

```
if ( x < 100 || x > 200 && x != y )
     x = 0;
```

Short Circuit/Jumping Code

```
    if x < 100 goto L2
    ifFalse x > 200 goto L1
    ifFalse x != y goto L1
L2: x = 0
L1:
```

Flow of Control Statements

- We will consider flow of control statements generated by following grammar

$$S \rightarrow if\ (B)\ S_1$$

$$S \rightarrow if\ (B)\ S_1\ else\ S_2$$

$$S \rightarrow while\ (B)\ S_1$$

- B represent Boolean expression
- S represent Statements

Flow of Control Statements

- The Grammar

Productions	Semantic Rules
$P \rightarrow S$	
$S \rightarrow S_1 S_2$	
$S \rightarrow$ **assign**	
$S \rightarrow if\ (B)\ S_1$	
$S \rightarrow if\ (B)\ S_1\ else\ S_2$	
$S \rightarrow while\ (B) S_1$	

assign is a Placeholder for an assignment statement

We need to find out the Semantic Rules

Flow of Control Statements

- Semantic Rules for the production

$$S \rightarrow if\ (B)\ S_1$$

B

S

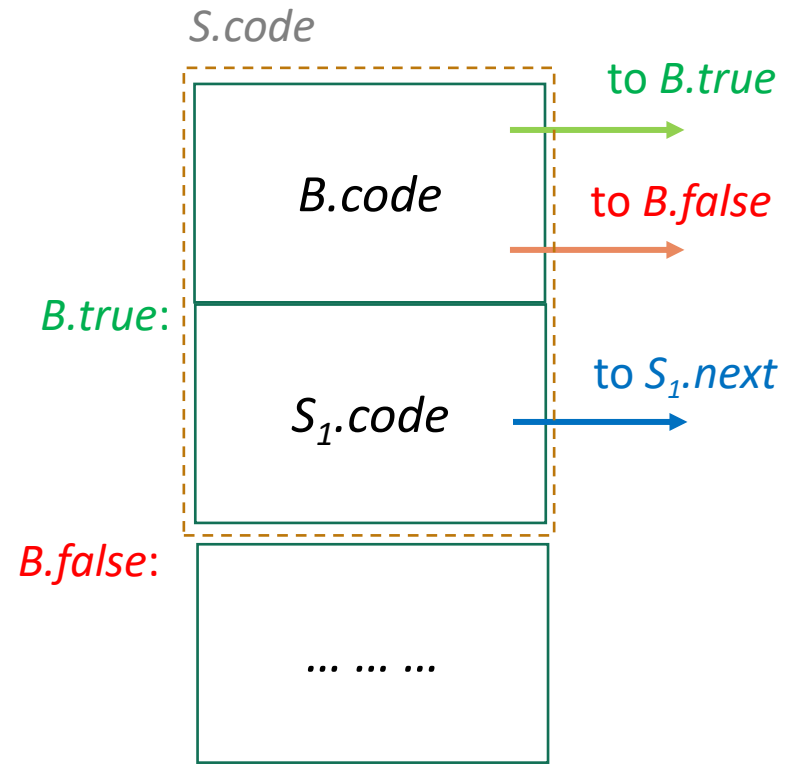
```
if ( x < 100 || x > 200 && x != y )
{
        x = 0;
}
y = x;
```

S_1

newlabel() creates new label each time it is called

Productions	Semantic Rules
$S \rightarrow if\ (B)\ S_1$	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \mid\mid label(B.true) \mid\mid S_1.code$

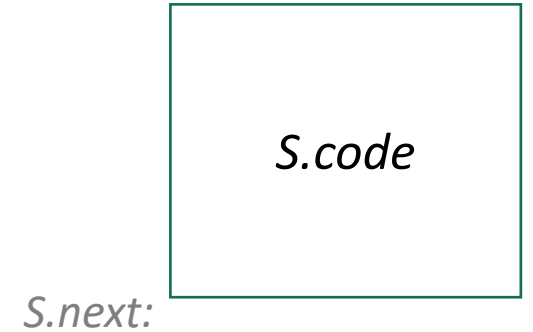
label(*L*) attaches label L to the next instruction to be generated



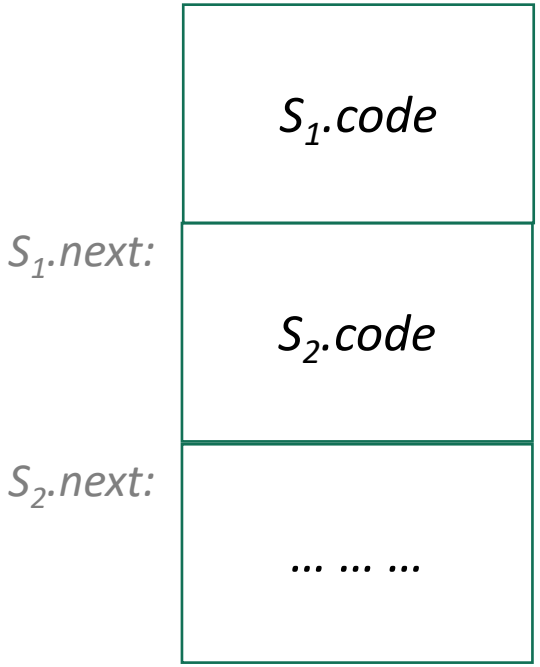
Flow of Control Statements

- Semantic Rules for the first two productions

Productions	Semantic Rules
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \mid\mid label(S.next)$



$S \rightarrow S_1 S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code$ $= S_1.code \mid\mid label(S_1.next) \mid\mid S_2.code$



Flow of Control Statements

- Attributes of SDD:
  - Codes are managed using
    - B.code : contains codes of B
    - S.code : contains codes of S

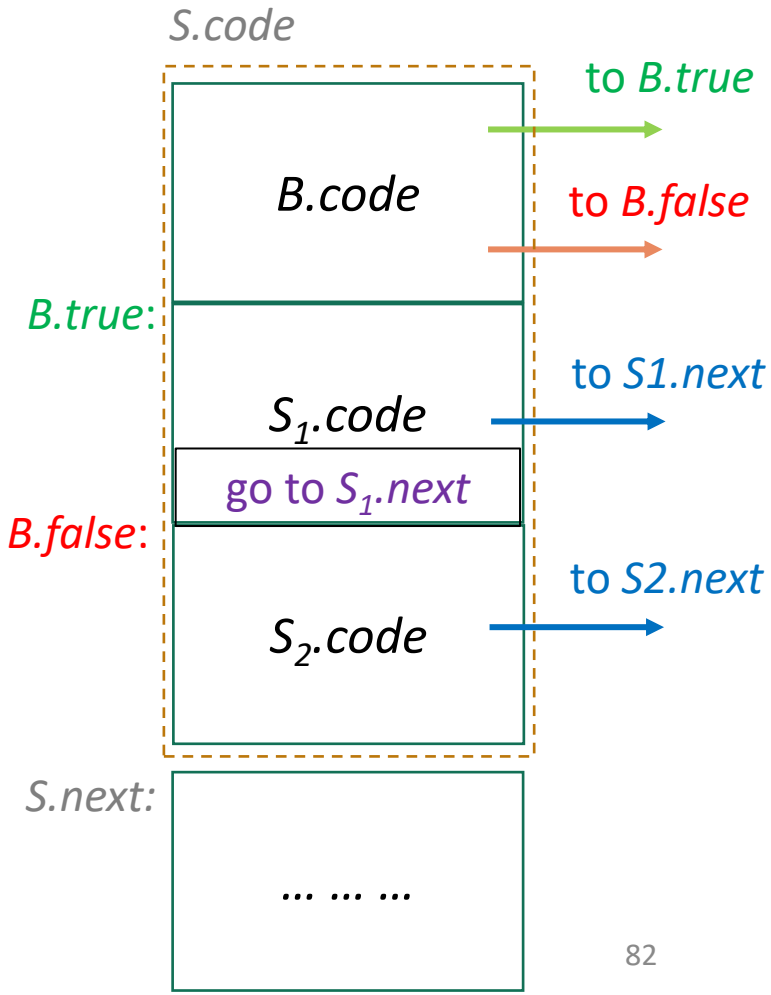
    } Synthesized
  - Jumping Labels are managed using
    - B.true : label to which control flows if B is true
    - B.false : label to which control flows if B is false
    - S.next : label of an instruction immediately after S

    } Inherited

Flow of Control Statements

- Semantic Rules for the production $S \rightarrow if\ (B)\ S_1\ else\ S_2$

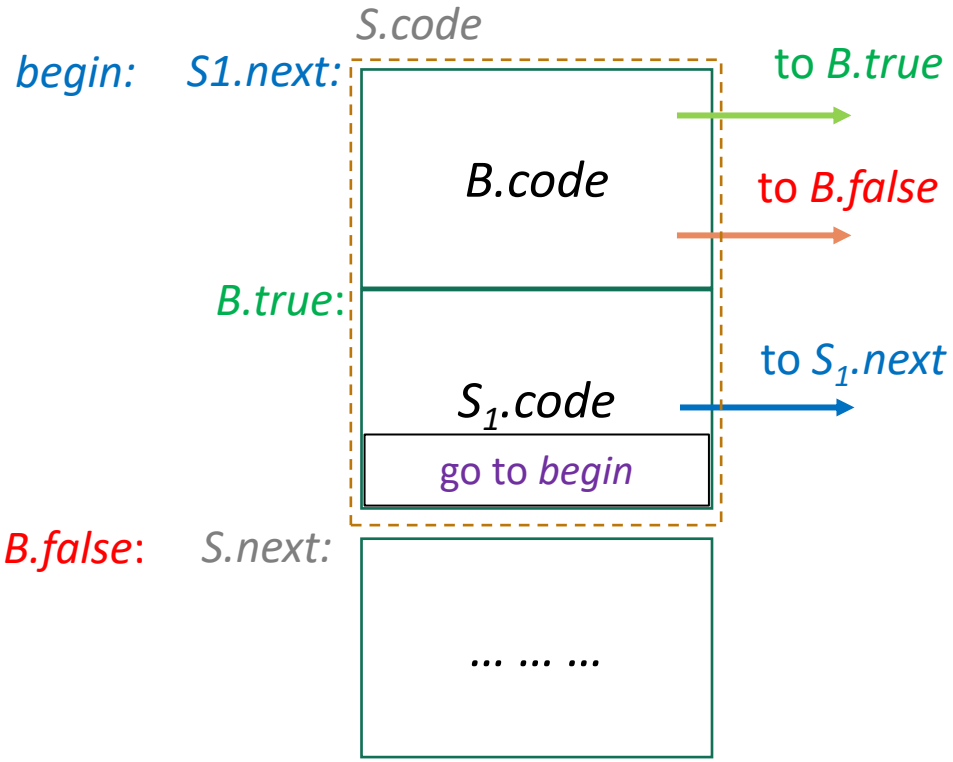
Productions	Semantic Rules
$S \rightarrow if\ (B)\ S_1\ else\ S_2$	$B.true = newlabel()$ $B.false = newlabel()$ $S_2.next = S_1.next = S.next$ $S.code = B.code$ $\|\| label(B.true) \|\| S_1.code$ $\|\| gen('goto'\ S.next)$ $\|\| label(B.false) \|\| S_2.code$



Flow of Control Statements

- Semantic Rules for the production $S \rightarrow while\ (B)\ S_1$

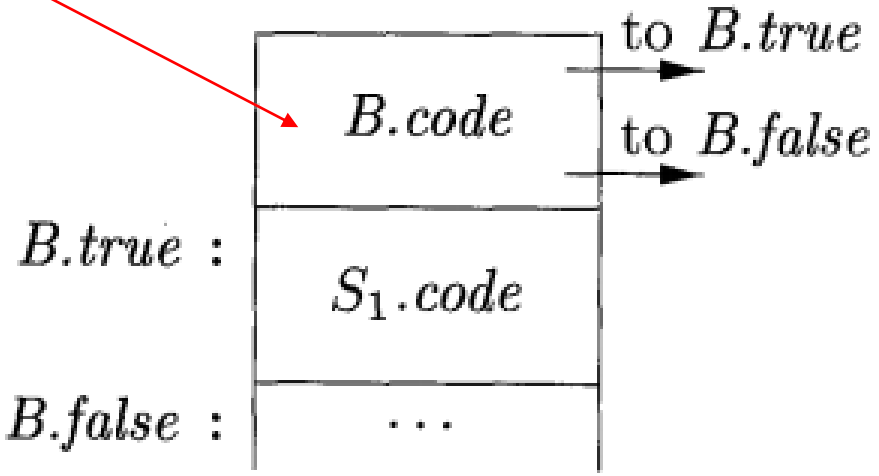
Productions	Semantic Rules
$S \rightarrow while\ (B) S_1$	$begin = newlabel(\ )$ $B.true = newlabel(\ )$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin)$ $\|\|B.code\|\|label(B.true)$ $\|\|S_1.code$



Translation of Boolean Expression

- Now we will consider productions containing B in their head
  - We want to know how we produce $B.code$

How to produce B.code?



Translation of Boolean Expression

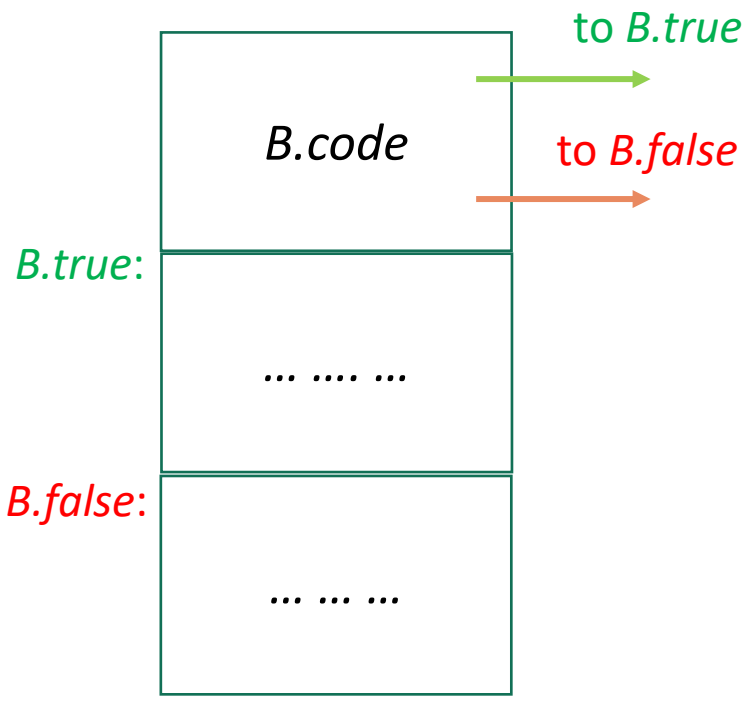
- The Grammar

Productions	**Semantic Rules**
$B \rightarrow B_1 \mid\mid B_2$	
$B \rightarrow B_1 \,\&\&\, B_2$	
$B \rightarrow !\, B_1$	
$B \rightarrow E_1 \ \mathbf{rel}\ E_2$	
$B \rightarrow \mathbf{true}$	
$B \rightarrow \mathbf{false}$	

Translation of Boolean Expression

- Semantic Rules for the production $B \rightarrow B_1 \,||\, B_2$

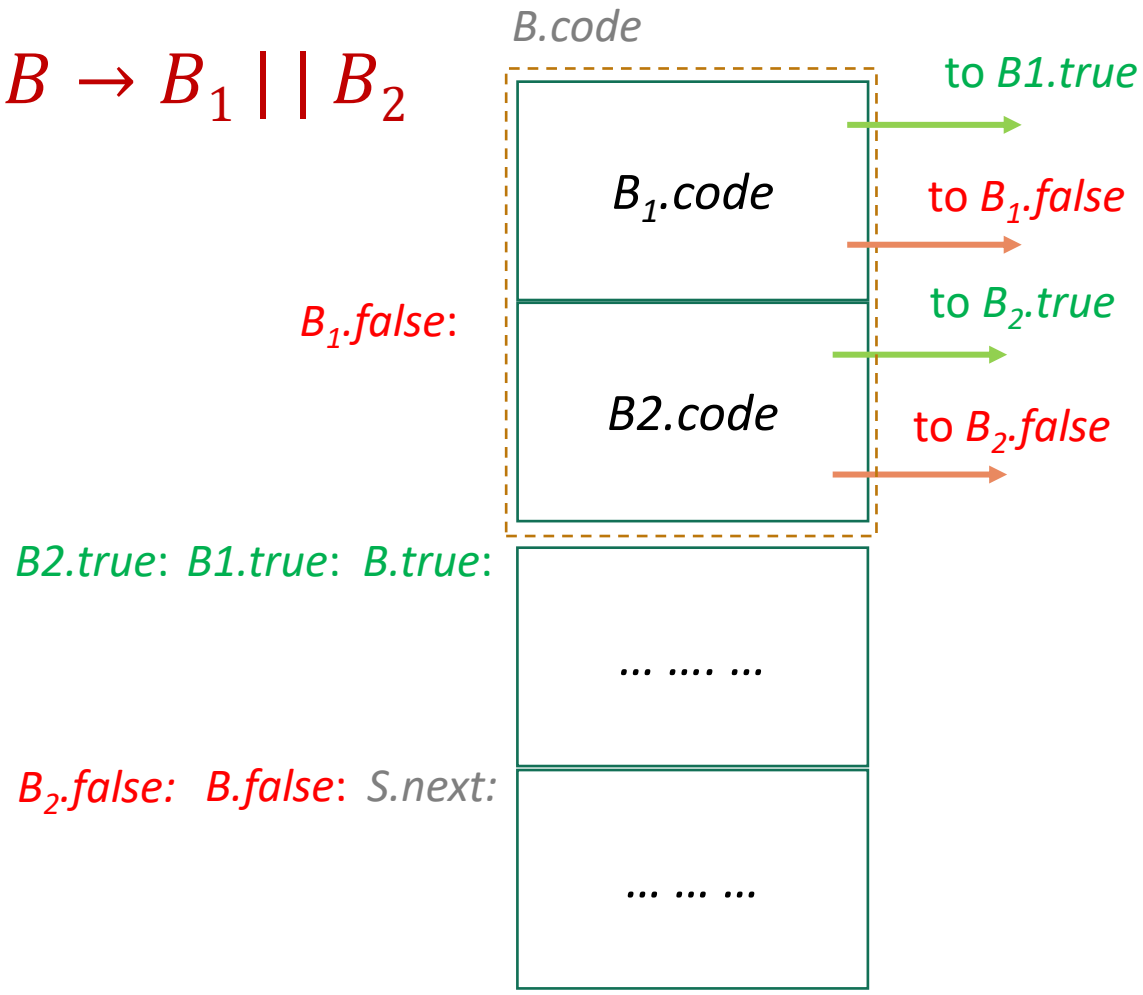
PRODUCTION	SEMANTIC RULE
$B \rightarrow B_1 \,\|\|\, B_2$	$B_1.true = B.true$ $B_1.false = \text{newLabel}()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \,\|\|\, label(B_1.false) \,\|\|\, B_2.code$



Translation of Boolean Expression

- Semantic Rules for the production $B \rightarrow B_1 \mid\mid B_2$

PRODUCTION	SEMANTIC RULE
$B \rightarrow B_1 \mid\mid B_2$	$B_1.true = B.true$ $B_1.false = \text{newLabel}()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \mid\mid label(B_1.false) \mid\mid B_2.code$



Translation of Boolean Expression

- Consider $B \rightarrow B_1 \mid\mid B_2$
- If B_1 is true then B is also true. So label of $B_1.true$ is same as $B.true$
- If B_1 is false, then we need to evaluate B_2. So label of $B_1.false$ is the label of first instruction of B_2
- $B_2.true$ is same as $B.true$
- $B_2.false$ is same as $B.false$

Translation of Boolean Expression

- Semantic Rules for the production $B \rightarrow B_1 \,\&\&\, B_2$

PRODUCTION	SEMANTIC RULE
$B \rightarrow B_1 \,\&\&\, B_2$	$B_1.true = \text{newLabel}()$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \,\|\|\, label(B_1.true) \,\|\|\, B_2.code$

Translation of Boolean Expression

- Semantic Rules for the production $B \rightarrow !\, B_1$

PRODUCTION	SEMANTIC RULE
$B \rightarrow !\, B_1$	$B_1.true = B.false$ $B_1.false = B.true$ $B.code = B_1.code$

Translation of Boolean Expression

- Semantic Rules for the production $B \rightarrow E_1 \ \mathbf{rel} \ E_2$

PRODUCTION	SEMANTIC RULE
$B \rightarrow E_1 \ \mathbf{rel} \ E_2$	$B.code = E_1.code \ \|\| \ E_2.code$ $\|\| \ gen('if' \ E_1.addr \ \mathbf{rel}.op \ E_2.addr \ 'goto' \ B.true)$ $\|\| \ gen('goto' \ B.false)$

Translation of Boolean Expression

- Semantic Rules for the productions $B \rightarrow \mathbf{true}$ and $B \rightarrow \mathbf{false}$

PRODUCTION	SEMANTIC RULE
$B \rightarrow \mathbf{true}$	$B.code = gen('goto'\ B.true)$
$B \rightarrow \mathbf{false}$	$B.code = gen('goto'\ B.false)$

SDD for Generating Three Address Code for Flow-of-Control Statements

PRODUCTION	SEMANTIC RULES
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \mid\mid label(S.next)$
$S \rightarrow$ **assign**	$S.code =$ **assign**$.code$
$S \rightarrow$ **if** (B) S_1	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \mid\mid label(B.true) \mid\mid S_1.code$
$S \rightarrow$ **if** (B) S_1 **else** S_2	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}'\ S.next)$ $\mid\mid label(B.false) \mid\mid S_2.code$
$S \rightarrow$ **while** (B) S_1	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \mid\mid B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}'\ begin)$
$S \rightarrow S_1\ S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \mid\mid label(S_1.next) \mid\mid S_2.code$

SDD for Generating Three Address Code for Booleans

PRODUCTION	SEMANTIC RULES
$B \rightarrow B_1 \mid\mid B_2$	$B_1.true = B.true$ $B_1.false = newlabel()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \mid\mid label(B_1.false) \mid\mid B_2.code$
$B \rightarrow B_1 \,\&\&\, B_2$	$B_1.true = newlabel()$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \mid\mid label(B_1.true) \mid\mid B_2.code$
$B \rightarrow ! B_1$	$B_1.true = B.false$ $B_1.false = B.true$ $B.code = B_1.code$
$B \rightarrow E_1 \textbf{ rel } E_2$	$B.code = E_1.code \mid\mid E_2.code$ $\mid\mid gen('\texttt{if}'\ E_1.addr\ \textbf{rel}.op\ E_2.addr\ '\texttt{goto}'\ B.true)$ $\mid\mid gen('\texttt{goto}'\ B.false)$
$B \rightarrow \textbf{true}$	$B.code = gen('\texttt{goto}'\ B.true)$
$B \rightarrow \textbf{false}$	$B.code = gen('\texttt{goto}'\ B.false)$

Code Generation

SDD for Flow-of-Control and Booleans (*partial*)

Production	Semantic Action
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \mid\mid label(S.next)$
$S \rightarrow S_1 S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \mid\mid label(S_1.next) \mid\mid S_2.code$
$S \rightarrow$ **if** $(B)\ S_1$	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \mid\mid label(B.true) \mid\mid S_1.code$
$B \rightarrow B_1 \mid\mid B_2$	$B_1.true = B.true$ $B_1.false = newlabel()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \mid\mid label(B_1.false) \mid\mid B_2.code$
$B \rightarrow E_1$ **rel** E_2	$B.code = E_1.code \mid\mid E_2.code$ $\mid\mid gen(\text{'if'}\ E_1.addr\ \textbf{rel}\ E_2.addr\ \text{'goto}\ B.true\text{'})$ $\mid\mid gen(\text{'goto}\ B.false\text{'})$

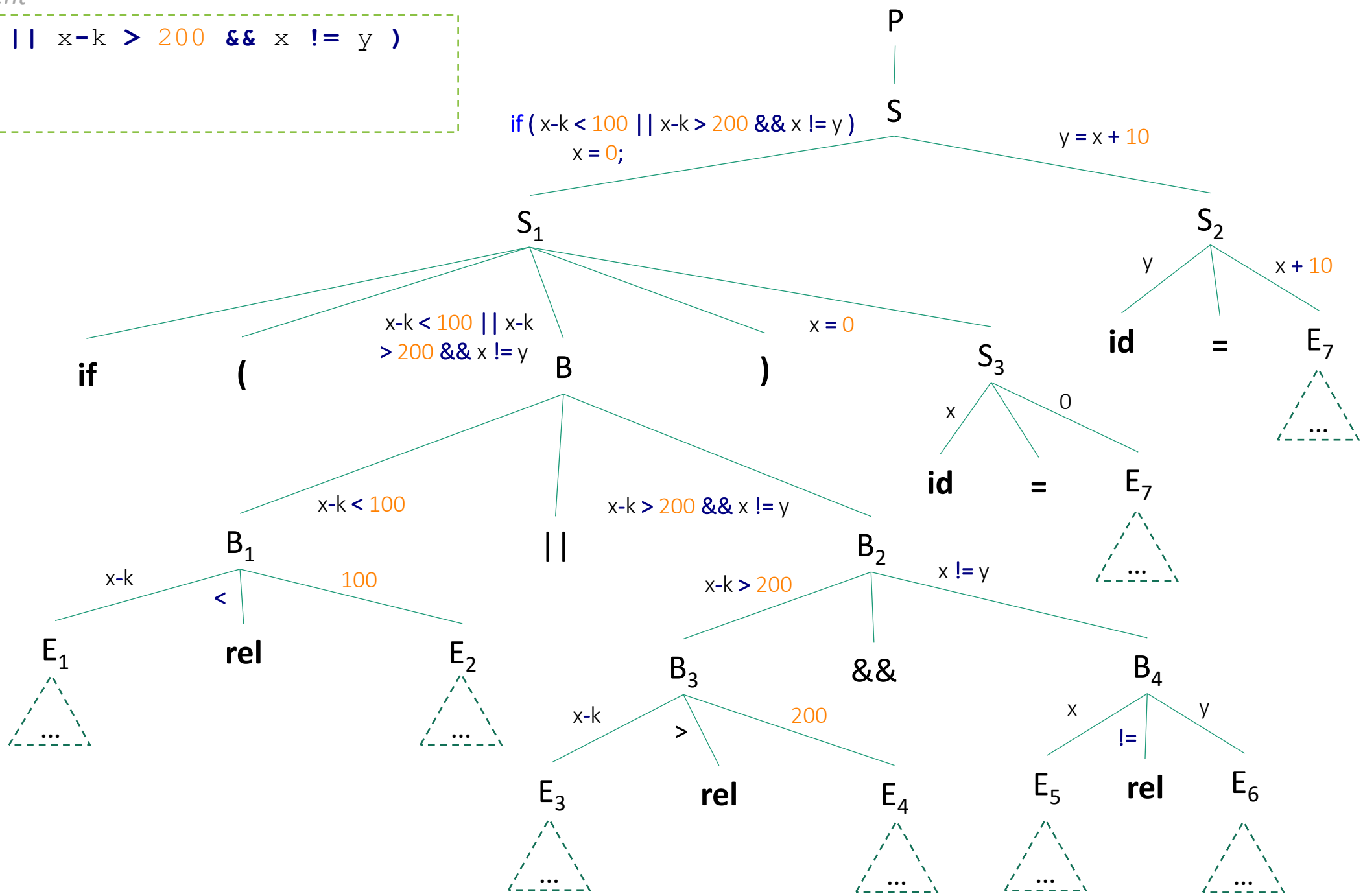
SDT for Flow-of-Control and Booleans (on the fly)

Productions with Embedded Actions
$P \rightarrow \{ S.next = newlabel(); \}\ S\ \{ label(S.next) \}$
$S \rightarrow \{ S_1.next = newlabel() \}\ S_1\ \{ S_2.next = S.next; label(S_1.next) \}\ S_2$
$S \rightarrow$ **if** $(\ \{ B.true = newlabel(); B.false = S_1.next = S.next; \}\ B\ )\ \{ label(B.true); \}\ S_1$
$B \rightarrow \{ B_1.true = B.true; B_1.false = newlabel(); \}\ B_1 \mid\mid \{ B_2.true = B.true; B_2.false = B.false; label(B_1.false); \}\ B_2$
$B \rightarrow E_1$ **rel** $E_2\ \{ gen(\text{'if'}\ E_1.addr\ \textbf{rel}\ E_2.addr\ \text{'goto}\ B.true\text{'});\ gen(\text{'goto}\ B.false\text{'}); \}$

Translation of statement

```
if ( x-k < 100 || x-k > 200 && x != y )
     x = 0;
y = x + 10;
```

Prod.
P → S
S → **id** = E;
S → S_1 S_2
S → **if** (B) S_1
B → B_1 **||** B_2
B → B_1 **&&** B_2
B → E_1 **rel** E_2
E → …

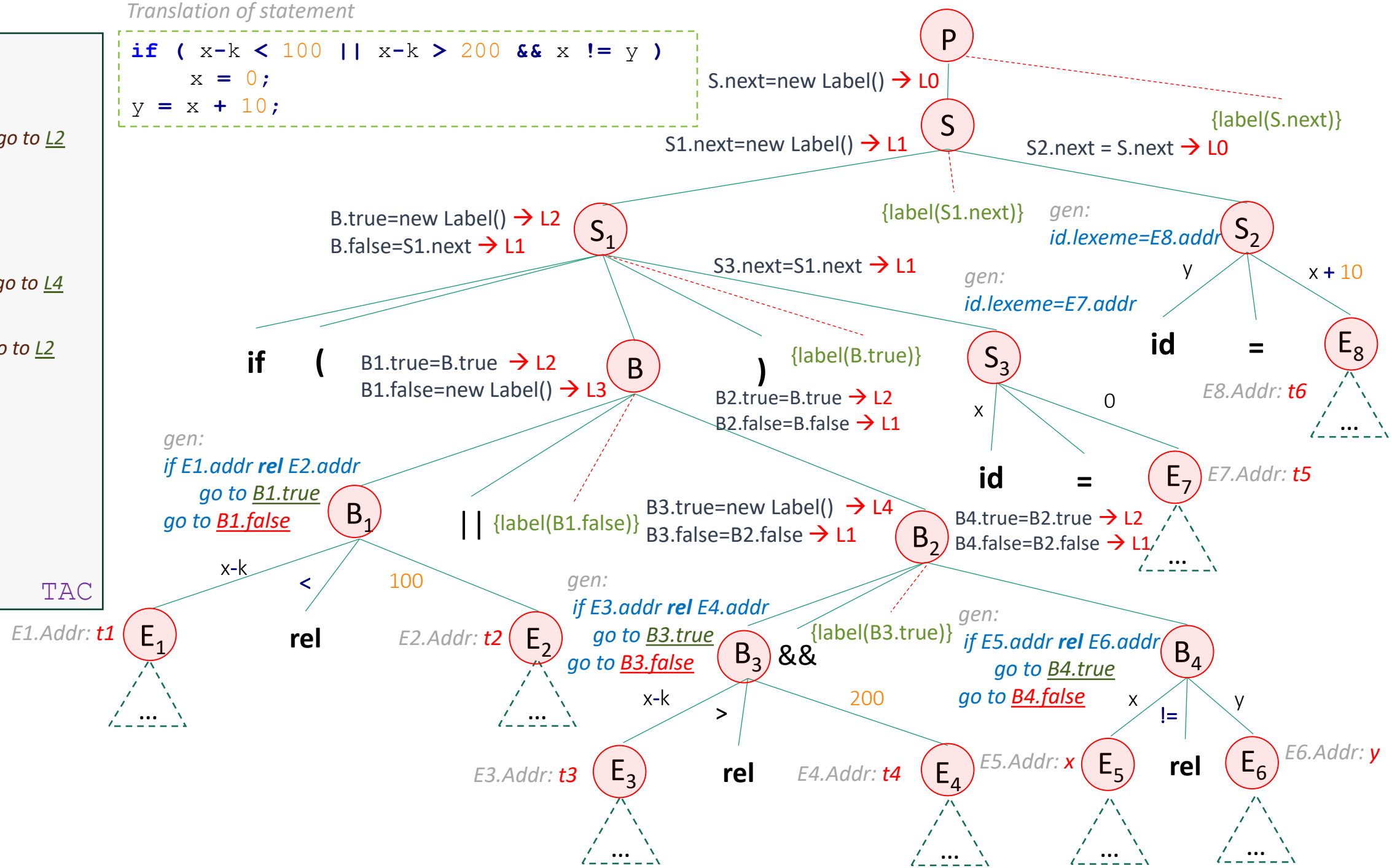


Translation of statement

```
if ( x-k < 100 || x-k > 200 && x != y )
     x = 0;
y = x + 10;
```

```
     t1 = x-k
     t2=100
     if t1 < t2  go to L2
     go to L3
L3:  t3 = x-k
     t4=200
     if t3 > t4  go to L4
     go to L1
L4:  if x != y  go to L2
     go to L1
L2:  t5=0
     x = t5
L1:  t6 = x+10
     y = t6
L0:
```

TAC

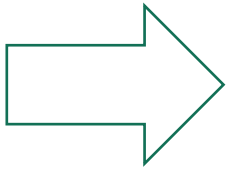


Avoiding Redundant Gotos

```
if( x < 100 || x > 200 && x != y ) x = 0;
```

```
        if x < 100 goto L2
Redundant  goto L3
L3:     if x > 200 goto L4
        goto L1
L4:     if x != y goto L2
        goto L1
L2:     x = 0
L1:
```

Code with redundant goto



```
        if x < 100 goto L2
        ifFalse x > 200 goto L1
        ifFalse x != y goto L1
L2:     x = 0
L1:
```

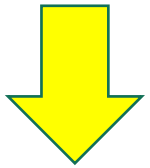
Code without redundant goto

Self Study (6.6.5)

Avoiding Redundant Gotos

$$S \rightarrow \textbf{if}\ (\ B\ )\ S_1$$

$$B.true = newlabel()$$
$$B.false = S_1.next = S.next$$
$$S.code = B.code \,||\, label(B.true) \,||\, S_1.code$$



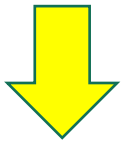
$$B.true = fall$$
$$B.false = S_1.next = S.next$$
$$S.code = B.code \,||\, S_1.code$$

Self Study (6.6.5)

Avoiding Redundant Gotos

$$B \rightarrow B_1 \;||\; B_2$$

$$B_1.true = B.true$$
$$B_1.false = newlabel()$$
$$B_2.true = B.true$$
$$B_2.false = B.false$$
$$B.code = B_1.code \;||\; label(B_1.false) \;||\; B_2.code$$



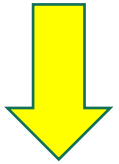
$$B_1.true = \textbf{if } B.true \neq fall \textbf{ then } B.true \textbf{ else } newlabel()$$
$$B_1.false = fall$$
$$B_2.true = B.true$$
$$B_2.false = B.false$$
$$B.code = \textbf{if } B.true \neq fall \textbf{ then } B_1.code \;||\; B_2.code$$
$$\textbf{else } B_1.code \;||\; B_2.code \;||\; label(B_1.true)$$

Self Study (6.6.5)

Avoiding Redundant Gotos

$$B \rightarrow E_1 \ \mathbf{rel} \ E_2$$

$$\begin{aligned} B.code = {} & E_1.code \ || \ E_2.code \\ & || \ gen(\texttt{'if'} \ E_1.addr \ \mathbf{rel}.op \ E_2.addr \ \texttt{'goto'} \ B.true) \\ & || \ gen(\texttt{'goto'} \ B.false) \end{aligned}$$



$$test = E_1.addr \ \mathbf{rel}.op \ E_2.addr$$

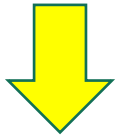
$$\begin{aligned} s = {} & \mathbf{if} \ B.true \neq fall \ \mathbf{and} \ B.false \neq fall \ \mathbf{then} \\ & \quad gen(\texttt{'if'} \ test \ \texttt{'goto'} \ B.true) \ || \ gen(\texttt{'goto'} \ B.false) \\ & \mathbf{else \ if} \ B.true \neq fall \ \mathbf{then} \ gen(\texttt{'if'} \ test \ \texttt{'goto'} \ B.true) \\ & \mathbf{else \ if} \ B.false \neq fall \ \mathbf{then} \ gen(\texttt{'ifFalse'} \ test \ \texttt{'goto'} \ B.false) \\ & \mathbf{else} \ '' \end{aligned}$$

$$B.code = E_1.code \ || \ E_2.code \ || \ s$$

Boolean Values and Jumping Code

- So far, we have used boolean expressions to alter the flow of control in statements
- A boolean expression may also be evaluated for its value such as

```
x = a < b && c < d
```



```
        ifFalse a < b goto L1
        ifFalse c > d goto L1
        t = true
        goto L2
L1:     t = false
L2:     x = t
```

```
if ( a < b && c < d )
    x=true;
else x=false;
```

Self Study (6.6.6)

Boolean Values and Jumping Code

- A clean way of handling both roles of boolean expressions is to first build a syntax tree for expressions, using either of the following approaches
  - Use two passes
    - Construct a complete syntax tree for the input, and
    - then traverse in depth-first order computing the translations specified by the semantic rule
  - Use one pass for statements, but two passes for expressions
    - In the approach, we would first translate *E* in ***while*** *(E) S1* before *S1* is examined.
    - The translation of *E*, however would be done by building its syntax tree and then walking the tree
- Use separate code generation functions for expressions
  - One for generating jumping code
  - The other for computing the value

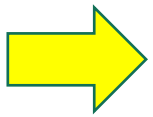
Backpatching

- A key problem when generating code for boolean expression and flow-of-control statement is that of matching a jump instruction with the target of the jump
- The SDD we have used earlier generates symbolic labels
- A separate pass is needed to bind labels to address

```
if( x < 100 || x > 200 && x != y ) x = 0;
```

1st Pass

```
        if x < 100 goto L2
        goto L3
L3:     if x > 200 goto L4
        goto L1
L4:     if x != y goto L2
        goto L1
L2:     x = 0
L1:
```



2nd Pass

```
100:  if x < 100 goto 106
101:  goto 102
102:  if x > 200 goto 104
103:  goto 107
104:  if x != y goto 106
105:  goto 107
106:  x = 0
107:
```

Backpatching

- Backpatching is used to generate code for boolean expressions and flow-of-control statements in one pass
- In Backpatching, list of jumps are passed as synthesized attributes
- Specifically, when a jump is generated, the target of the jump label is left unspecified
- Each such jump is put on a list of jumps whose labels are to be filled in when proper label can be determined
- All of the jumps on a list have the same target label

Code Generation Using Backpatching

- Two synthesized attributes for boolean expression *B*
  - *B. truelist* – a list of jumps or conditional jumps into which we must insert the label to which control goes if *B* is true
  - *B. falselist* – a list of jumps or conditional jumps into which we must insert the label to which control goes if *B* is false
- A synthesized attribute for statement *S*
  - *S. nextlist* – a list of jumps to the instruction immediately following the code for *S*
- Three functions
  - *makelist(i)* – creates a new list containing only *i*, an index into the array of instructions. Returns a pointer to the newly created array
  - *merge(p1, p2)* – concatenates the lists pointed to by *p1* and *p2*, and returns a pointer to the concatenated list
  - *backpatch(p, i)* – inserts *i* as the target label for each of the instructions on the list pointed to by *p*

Code Generation Using Backpatching

SDT for Flow-of-Control and Booleans (on the fly)		SDT for Backpatching	
$L \rightarrow$ L_1 S	{ $L_1.next = newlabel()$; } { $S.next = L.next$; $label(L_1.next)$; }	$L \rightarrow L_1\ M\ S$	{ $Backpatch(L_1.nextlist, M.instr)$; } $L.nextlist = S.nextlist$; }
$L \rightarrow$ S	{ $S.next = L.next$; }	$L \rightarrow S$	{ $L.nextlist = S.nextlist$; }
$S \rightarrow$ **if** (B) S_1	{ $B.true = newlabel()$; $B.false = S_1.next = S.next$; } { $label(B.true)$;}	$S \rightarrow$ **if** $(B)\ M\ S_1$	{ $S.nextlist = merge(B.falselist, S_1.nextlist)$; $Backpatch(B.truelist, M.instr)$; }
$B \rightarrow$ B_1 || B_2	{ $B_1.true = B.true$; $B_1.false = newlabel()$; } { $B_2.true = B.true$; $B_2.false = B.false$; $label(B_1.false)$; }	$B \rightarrow B_1$ || $M\ B_2$	{ $B.truelist = merge(B_1.truelist, B_2.truelist)$; $B.falselist = B_2.falselist$; $Backpatch(B_1.falselist, M.instr)$; }
$B \rightarrow E_1$ **rel** E_2	{ gen('if' $E_1.addr$ **rel** $E_2.addr$ ' goto $B.true$'); gen(' goto $B.false$'); }	$B \rightarrow E_1$ **rel** E_2	{ $B.truelist = makelist(nextinstr)$; $B.falselist = makelist(nextinstr+1)$; gen('if' $E_1.addr$ **rel** $E_2.addr$ ' goto __'); gen(' goto __'); }
		$M \rightarrow \varepsilon$	{ $M.instr = nextinstr$; }

Backpatching for Boolean Expression

1)	$B \rightarrow B_1$ **||** M B_2	{ $backpatch(B_1.falselist, M.instr)$; $B.truelist = merge(B_1.truelist, B_2.truelist)$; $B.falselist = B_2.falselist$; }
2)	$B \rightarrow B_1$ **&&** M B_2	{ $backpatch(B_1.truelist, M.instr)$; $B.truelist = B_2.truelist$; $B.falselist = merge(B_1.falselist, B_2.falselist)$; }
3)	$B \rightarrow$ **!** B_1	{ $B.truelist = B_1.falselist$; $B.falselist = B_1.truelist$; }
4)	$B \rightarrow$ (B_1)	{ $B.truelist = B_1.truelist$; $B.falselist = B_1.falselist$; }
5)	$B \rightarrow E_1$ **rel** E_2	{ $B.truelist = makelist(nextinstr)$; $B.falselist = makelist(nextinstr + 1)$; $emit('\texttt{if}'\ E_1.addr\ \textbf{rel}.op\ E_2.addr\ '\texttt{goto}\ _')$; $emit('\texttt{goto}\ _')$; }
6)	$B \rightarrow$ **true**	{ $B.truelist = makelist(nextinstr)$; $emit('\texttt{goto}\ _')$; }
7)	$B \rightarrow$ **false**	{ $B.falselist = makelist(nextinstr)$; $emit('\texttt{goto}\ _')$; }
8)	$M \rightarrow \epsilon$	{ $M.instr = nextinstr$; }

Backpatching for Flow-of-Control Statements

1) $S \rightarrow \textbf{if}\ (\ B\ )\ M\ S_1$ { $backpatch(B.truelist,\ M.instr)$;
$S.nextlist\ =\ merge(B.falselist,\ S_1.nextlist)$; }

2) $S \rightarrow \textbf{if}\ (\ B\ )\ M_1\ S_1\ N\ \textbf{else}\ M_2\ S_2$
{ $backpatch(B.truelist,\ M_1.instr)$;
$backpatch(B.falselist,\ M_2.instr)$;
$temp\ =\ merge(S_1.nextlist,\ N.nextlist)$;
$S.nextlist\ =\ merge(temp,\ S_2.nextlist)$; }

3) $S \rightarrow \textbf{while}\ M_1\ (\ B\ )\ M_2\ S_1$
{ $backpatch(S_1.nextlist,\ M_1.instr)$;
$backpatch(B.truelist,\ M_2.instr)$;
$S.nextlist\ =\ B.falselist$;
$emit(\text{'goto'}\ M_1.instr)$; }

6) $M \rightarrow \epsilon$ { $M.instr\ =\ nextinstr$; }

7) $N \rightarrow \epsilon$ { $N.nextlist\ =\ makelist(nextinstr)$;
$emit(\text{'goto\ _'})$; }

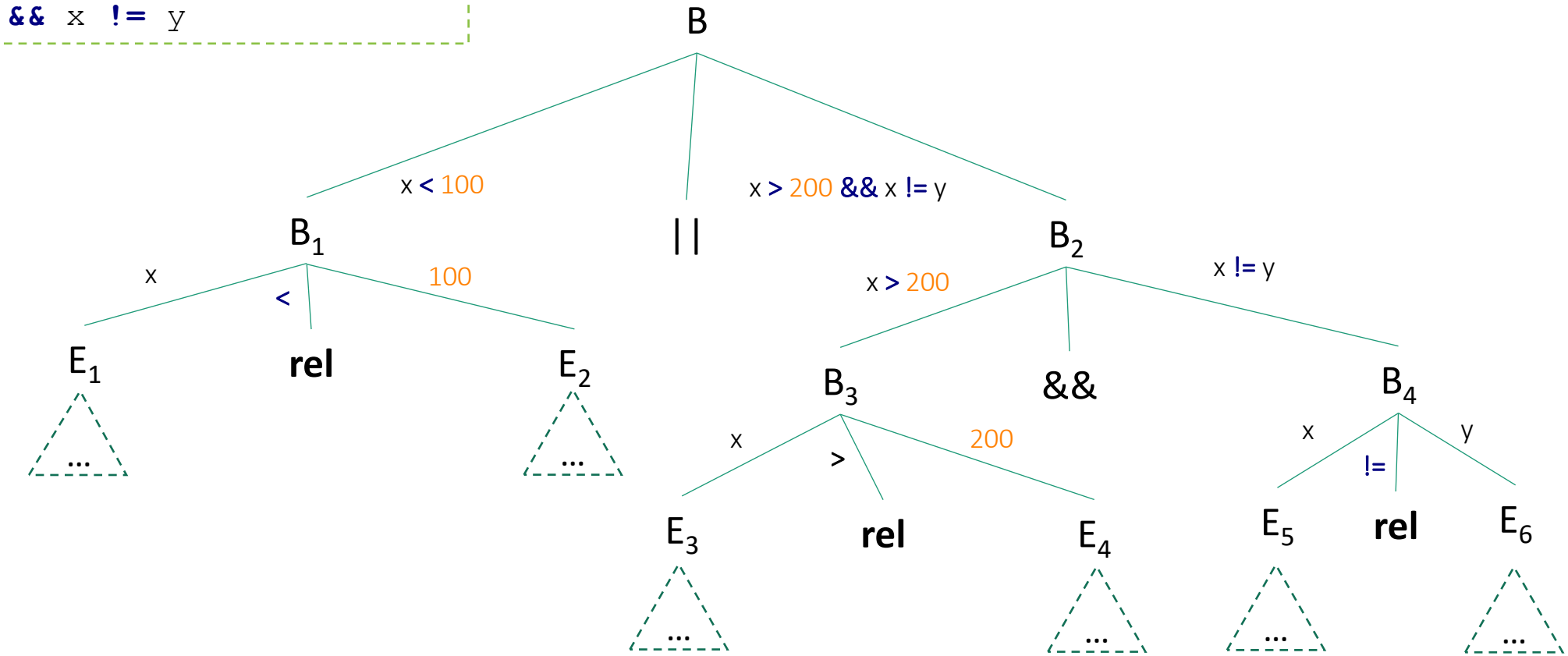
Backpatching for Flow-of-Control Statements

4) $S \rightarrow \{\ L\ \}$	$\{\ S.nextlist\ =\ L.nextlist;\ \}$
5) $S \rightarrow A\ ;$	$\{\ S.nextlist\ =\ \textbf{null};\ \}$
8) $L \rightarrow L_1\ M\ S$	$\{\ backpatch(L_1.nextlist,\ M.instr);$ $L.nextlist\ =\ S.nextlist;\ \}$
9) $L \rightarrow S$	$\{\ L.nextlist\ =\ S.nextlist;\ \}$

Translation of a boolean expression using backpatching

```
x < 100 || x > 200 && x != y
```

Prod
$B \rightarrow B_1$ && B_2
$B \rightarrow B_1$ || B_2
$B \rightarrow E_1$ **rel** E_2



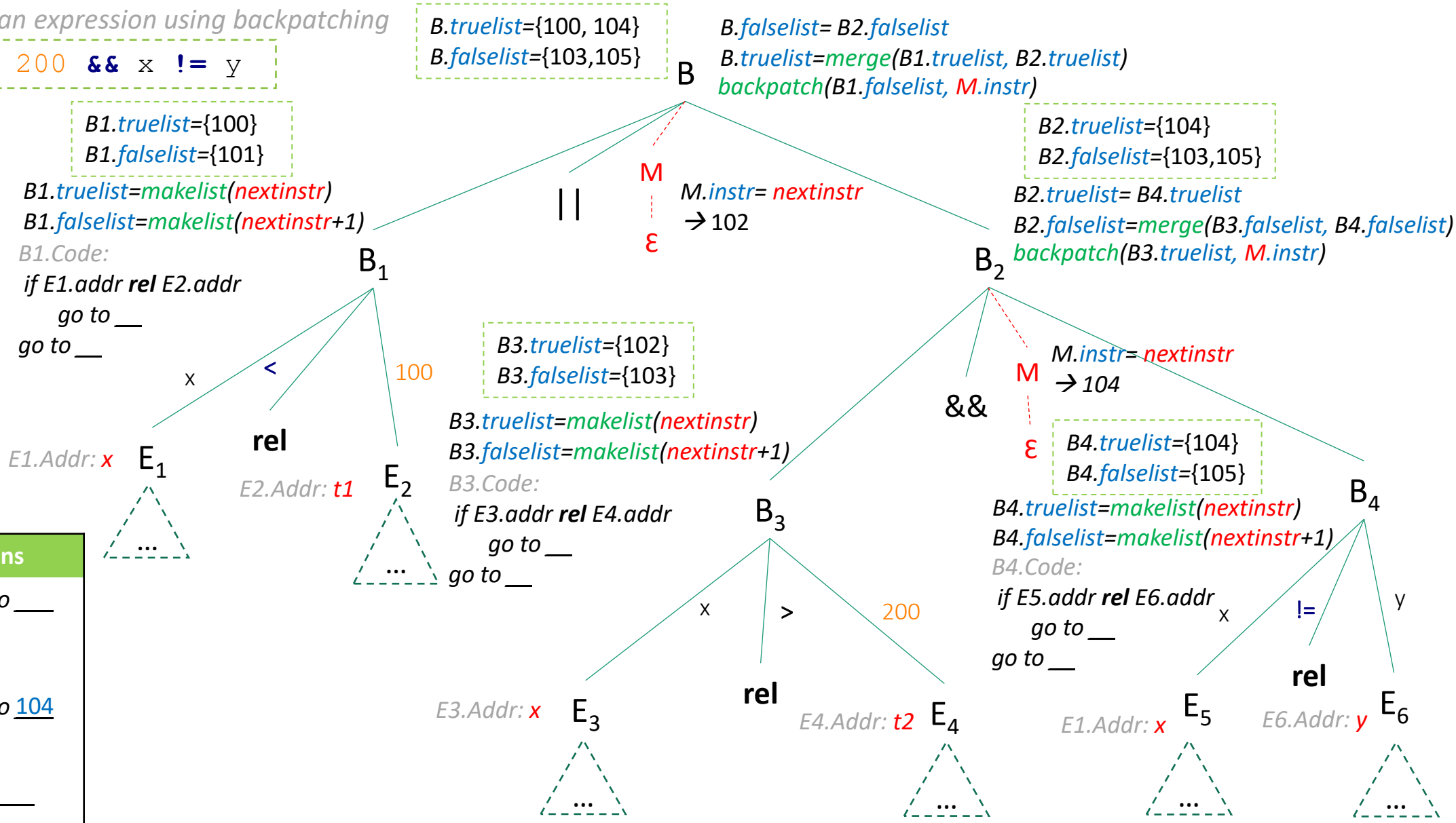
Translation of a boolean expression using backpatching

`x < 100 || x > 200 && x != y`

Prod
$B \rightarrow B_1$ && M B_2
$B \rightarrow B_1$ || M B_2
$B \rightarrow E_1$ **rel** E_2
$M \rightarrow \varepsilon$

nextinstr = 106

Instr. #	Instructions
100	*if x < 100 go to ___*
101	*go to* 102
102	*if x > 200 go to* 104
103	*go to ___*
104	*if x != y go to ___*
105	*go to ___*



Translation of Flow-of-Control statements using backpatching

```
if ( x < 100 || x > 200 && x != y )
    x = 0;
y = x + 10;
```

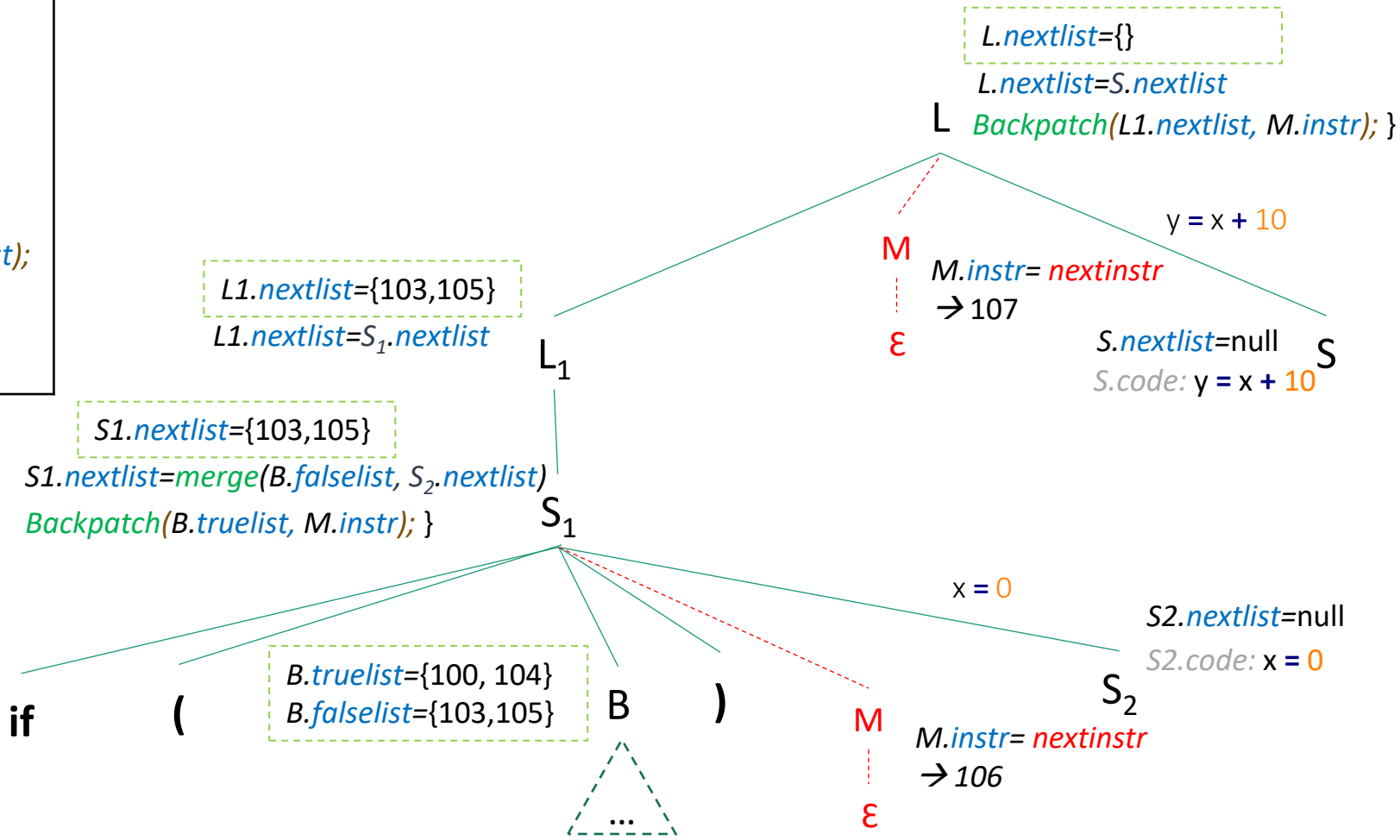
$L \rightarrow L_1\ M\ S$	{ *Backpatch*(L_1 *nextlist*, *M.instr*); } *L.nextlist=S nextlist*; }
$L \rightarrow S$	{ *L.nextlist=S.nextlist*; }
$S \rightarrow A$	{ *S.nextlist=null*}
$S \rightarrow$ ***if*** *(B)* $M\ S_1$	{ *S.nextlist=merge*(*B.falselist*, S_1*.nextlist*); *Backpatch*(*B.truelist*, *M.instr*); }
$M \rightarrow \varepsilon$	{ *M.instr=nextinstr*; }

nextinstr = 108

Instr. #	Instructions
100	*if x < 100 go to* 106
101	*go to* 102
102	*if x > 200 go to* 104
103	*go to* 107
104	*if x != y go to* 106
105	*go to* 107
106	*x = 0*
107	*y = x + 10*

Translation of boolean expression using backpatching

```
x < 100 || x > 200 && x != y
```



Self Study (6.7.4)

Break- and Continue- Statements

```
1)  for ( ; ; readch() ) {
2)      if( peek == ' ' || peek == '\t' ) continue;
3)      else if( peek == '\n' ) line = line + 1;
4)      else break;
5)  }
```

- If *S* is the enclosing loop construct, then a break-statement is a jump to the first instruction after the code for *S*
- Code for break can be generated by
  (1) Keeping track of the enclosing loop statement *S*
  (2) generating an unfilled jump for the break-statement, and
  (3) putting this unfilled jump on *S.nextlist*
- In a two-pass front end that builds syntax trees, *S.nextlist* can be implemented as a field in the node for S
- We can keep track of S by using the symbol table to map a special identifier break to the node for the enclosing loop statement

Self Study (6.7.4)

Break-, Continue-, and Goto-Statements

- Alternatively, a pointer to the *S.nextlist* can be put in the symbol table.
- When a break-statement is reached,
  - a unfilled jump is generated,
  - the *nextlist* is looked up in the symbol table,
  - the jump added to the list, and
  - finally, the list is backpatched
- Continue-statement can be handled in a manner analogous to break-statement.
  - However, the target of the generated jump is different.

Self Study (6.8)

Switch- Statements

- The ‘switch’ or ‘case’ statement is available in a variety of languages
- Syntax:

Switch Syntax

```
switch ( E ) {
        case V1: S1
        case V2: S2
                 ...
        case Vn−1: Sn−1
        default: Sn
}
```

E: Selector Expression

V1, V2, … , Vn-1: constant values

default: any value other than V1, V2, … , Vn-1

S1, S2, …. , Sn: Statements

- Translation of Switch-Statements
  1. Evaluate the expression of E
  2. Find the value of Vj in the list of cases that is the same as the value of the expression. If not match found, match to the default values
  3. Execute the statement Sj associated with the value found

Self Study (6.8)

Translation of Switch- Statements

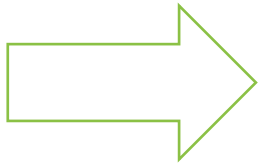
- Step (2) is a n-way branch that can be implemented in several ways
- If the number of cases is small, use a sequence of conditional jumps, each of which tests for an individual value and transfer to the code for the corresponding statement.
- A more compact way is to create a table of pairs, each pair consisting of a value and a label for the corresponding statement's code
  - The value for the expression E is paired with the label for the default statement and placed at the last of the table
  - A loop generated by the compiler compares the value of the expression to the values in the table
  - If the number of values is high, use a hash table
  - If the values all lies in some small range and the number of different values used is a reasonable fraction of the number of values in the range, then use 'buckets'

Self Study (6.8)

SDT for Translation of Switch- Statements

Switch Syntax

```
switch ( E ) {
        case V1: S1
        case V2: S2
                 ...
        case Vn-1: Sn-1
        default: Sn
}
```



```
                code to evaluate E into t
                if t != V1 goto L1
                code for S1
                goto next
L1:             if t != V2 goto L2
                code for S2
                goto next
L2:
                ...
Ln-2:           if t != Vn-1 goto Ln-1
                code for Sn-1
                goto next
Ln-1:           code for Sn
next:
```

Three Address Code (V.1)

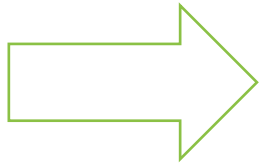
- Requires the compiler to do extensive analysis to find the most efficient implementation
- It is inconvenient in a one-pass compiler to place the branching statements at the beginning, because the compiler could not then emit code for each of statements *Si* as it saw them

Self Study (6.8)

SDT for Translation of Switch- Statements

Switch Syntax

```
switch ( E ) {
        case V1: S1
        case V2: S2
                 ...
        case Vn−1: Sn−1
        default: Sn
}
```



```
        code to evaluate E into t
        goto test
L1:     code for S1
        goto next
L2:     code for S2
        goto next
        ...
Ln−1:   code for Sn−1
        goto next
Ln:     code for Sn
        goto next
test:   if t = V1 goto L1
        if t = V2 goto L2
        ...
        if t = Vn−1 goto Ln−1
        goto Ln
next:
```

1. When the keyword 'switch' is found, generate two labels test and next and a new temporary t
2. Then while E is parsed, generate code to evaluate E into t
3. After processing of E, generate the jump goto test
4. Then for each case keyword, create a new label *Li* and enter in a queue in the symbol table value-label pair <*Vi*, *Li*>
5. Process each statement case *Vi*:*Si* by emitting the label *Li* attached to the code of *Si*, followed by the jump to goto next
6. When the end of the switch is found, generate the code for the n-way branch by reading from the queue of value-label pairs in the symbol table

Reference

- Compilers: Principles, Techniques, & Tools, by
  Alfred B. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman

  - Chapter 6
    - 6.1
    - 6.2
    - 6.3
    - 6.4
    - 6.6
    - 6.7
    - 6.8 (Self study)

Thank You!

Chapter 7

Run-Time Environments

Run-Time Environments

- Programming Languages contain high level structures
  - Functions
  - Objects
  - Exceptions
  - Scopes
  - And more..
- The compiler must co-operate with the operating system and other systems software to support these abstractions on the target machine.
- To do so, the compiler creates and manages a *run-time environment*

Run-Time Environments

- A *run-time environment*(RTE) is a set of data structures maintained at runtime to implement high-level structures of source language

- Compiler assumes its target programs are being executed in RTE
  - Compiler creates and manages it.

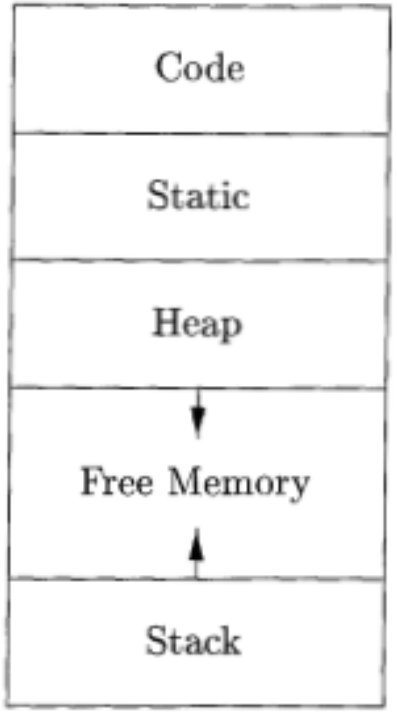
- IR generators depends on the set up of RTE

Run-Time Environments

- Run-Time Environment deals with different issues
  - The layout and allocation of storage location for the names in the source program
  - Mechanism to access variables
  - The linkage between procedures
  - The mechanisms for passing parameters

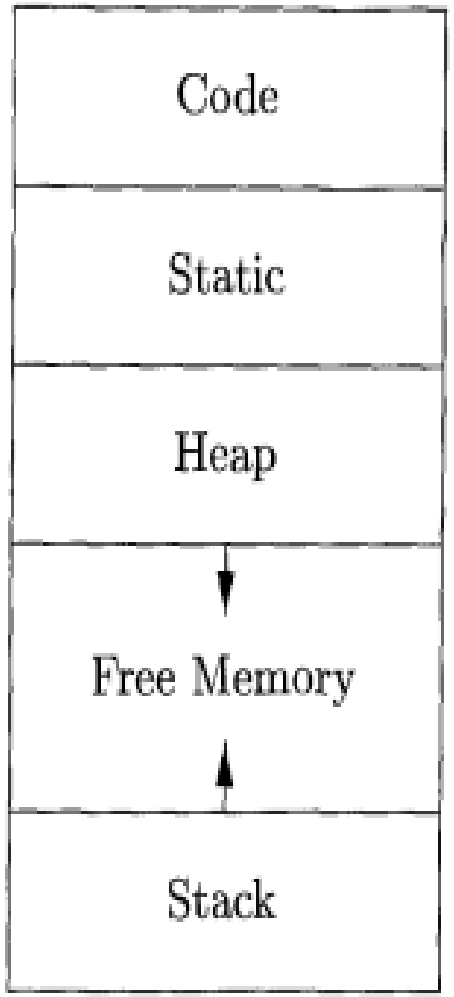
Storage Organization

- The executing target program runs in its own logical address space
- The operating system maps this to physical address space



Typical Subdivision of memory into code and data areas

Storage Organization



Generated executable code, size is fixed

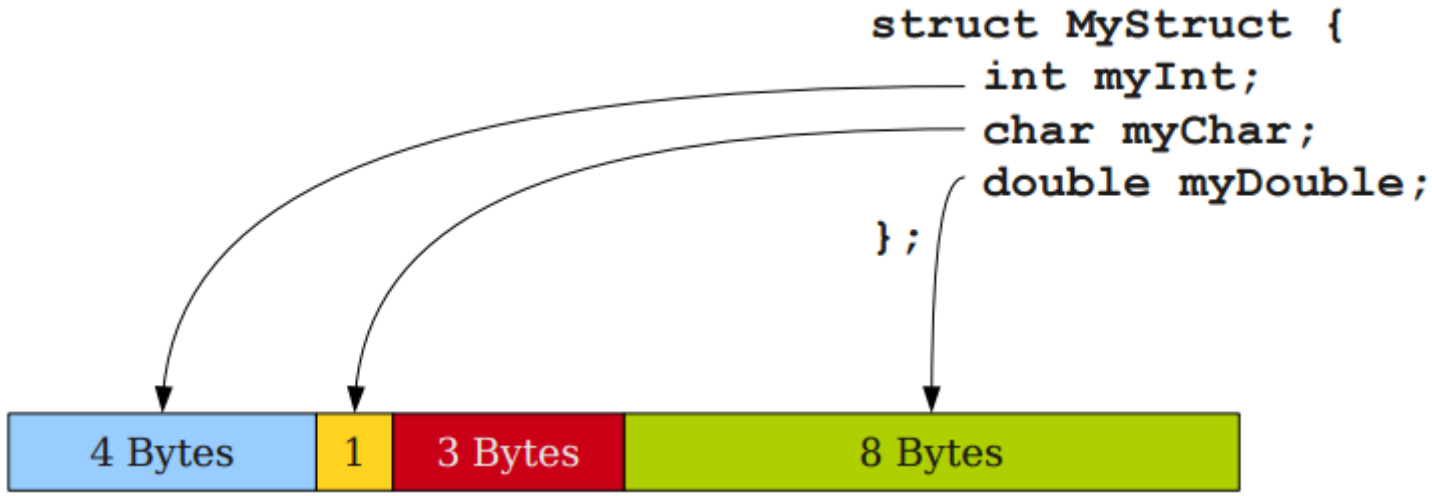
Program data objects, sizes are known at compile time

The heap is used to manage long-lived data, generated by the programmer by allocating and deallocating memory

The stack is used to store data structures called activation records that get generated during procedure calls.

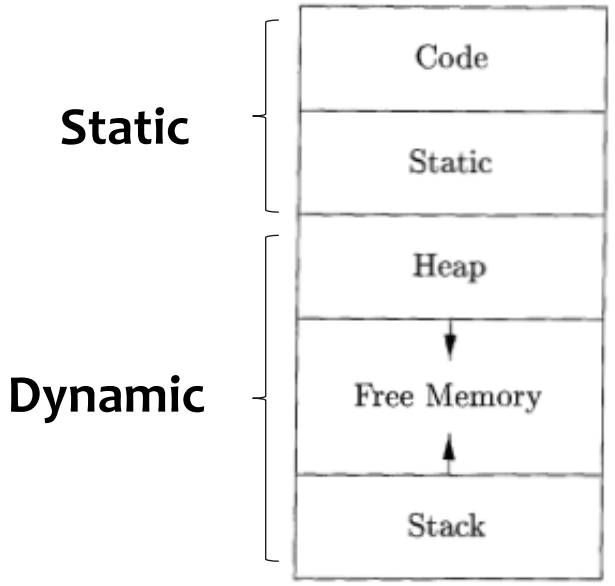
Run-Time Environments

- Storage layout is influenced by addressing constraints of target machine
  - Elementary data types can be stored in an integral number of bytes
  - Aggregate data types should take space large enough to hold all components
  - Padding: Unused place due to alignment



Static vs Dynamic Storage Allocation

- Static: Can be determined at compile time
  - Code segment
  - Static Data segment
- Dynamic : Can be decided only while the program is running
  - Stack Storage
  - Heap Storage



Stack Allocation

- Stack is used for procedures, functions, methods as part of their run time memory
- Each time a procedure is called –
  - Space for its local variable is pushed onto stack
  - At return time that space is popped off.
- Allows space to be shared by procedure calls whose duration do not overlap in time
- Allows to compile code for a procedure in such a way that the relative addresses of its non-local variables are always the same, regardless of the sequence of procedure calls

Stack allocation would not be feasible if procedure calls, or *activation* of procedures, did not nest in time

Nesting of Procedure

- If an activation of procedure p calls procedure q then
  - The activation of q must end before activation of p can end
- There are three common cases:
  - Activation of q terminates normally
  - Activation of q aborts
  - Activation of q terminates because of exceptions that q cannot handle

```
p( )
{
        …..
        q( );
        …..
}
```

Nested Procedure Call

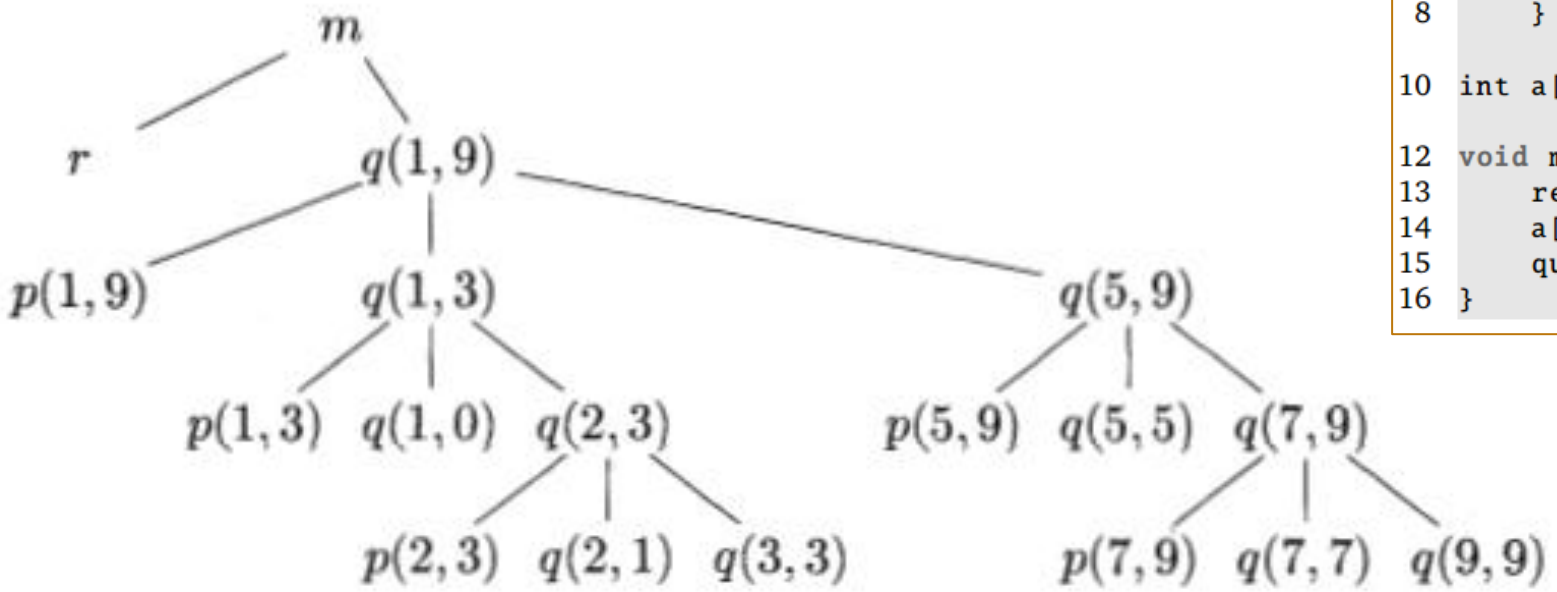
- Procedure Call inside of another procedure
  - Like nested loop

```
1  ...
2  void quicksort(int m, int n){
3      int i;
4      if (n > m){
5          i = partition(m,n); /* picks "quicksort separator value" */
6          quicksort(m, i-1);
7          quicksort(i+1, n);
8      }

10 int a[11];

12 void main() {
13     readArray();  /* reads 9 integers into a[1],..,a[9] */
14     a[0] = -9999; a[10] = 9999;  /* sentinels */
15     quicksort(1,9);
16 }
```

Representing Procedure Call

- Activation Tree:
  - Representation of procedures during the running of an entire program by a tree
- Each Node(p):
  - Corresponds to one activation of a procedure
  - Root: activation of the main procedure
- Each Children(q):
  - Corresponds to activations of the procedures called by this activation of the node
  - q is called from p
  - Order from left to right

Activation Tree



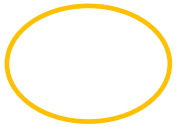

```
1  ...
2  void quicksort(int m, int n){
3      int i;
4      if (n > m){
5          i = partition(m,n); /* picks "quicksort
6          quicksort(m, i-1);
7          quicksort(i+1, n);
8      }

10 int a[11];

12 void main() {
13     readArray();  /* reads 9 integers into a[1],.
14     a[0] = -9999; a[10] = 9999;  /* sentinels */
15     quicksort(1,9);
16 }
```

Activation tree of quicksort during an execution of quicksort

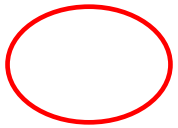
Activation Tree

- Several useful relationships between activation tree and behavior of the program
  - Sequence of procedure calls corresponds to preorder traversal of the activation tree
  - Return sequence corresponds to post order traversal
  - At any node N the live activations are those that corresponds to N and its ancestor

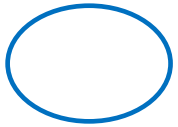
Activation Tree



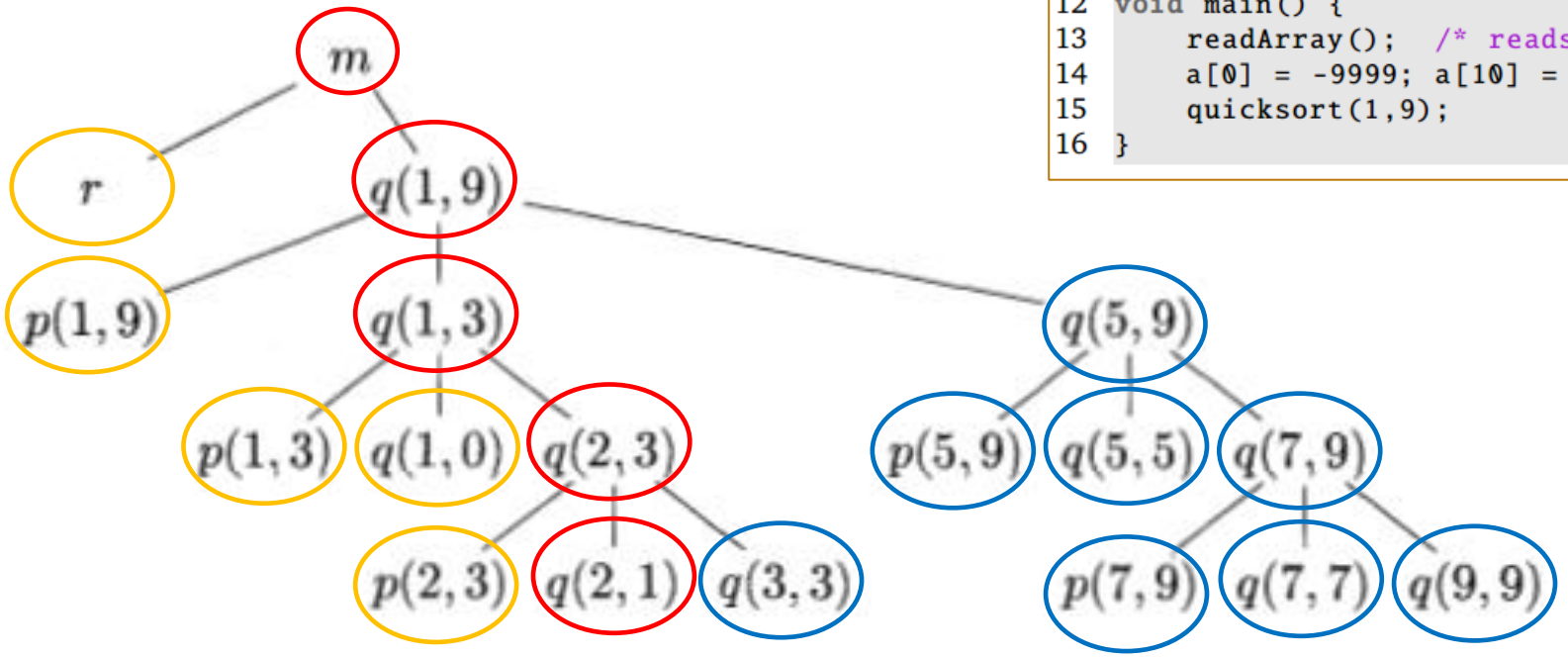
Already Processed



Alive



Yet to call

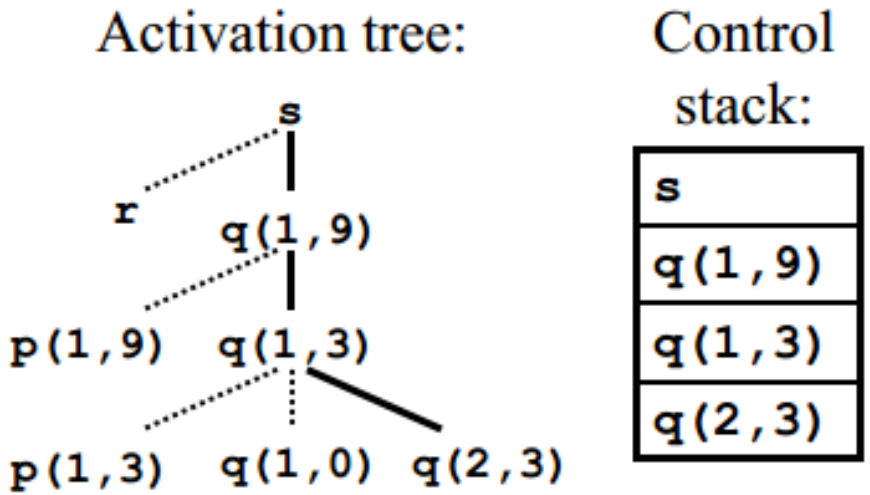



```
1  ...
2  void quicksort(int m, int n){
3      int i;
4      if (n > m){
5          i = partition(m,n); /* picks "quicksort
6          quicksort(m, i-1);
7          quicksort(i+1, n);
8      }

10 int a[11];

12 void main() {
13     readArray();  /* reads 9 integers into a[1],.
14     a[0] = -9999; a[10] = 9999;  /* sentinels */
15     quicksort(1,9);
16 }
```

Activation tree of quicksort during an execution of quicksort

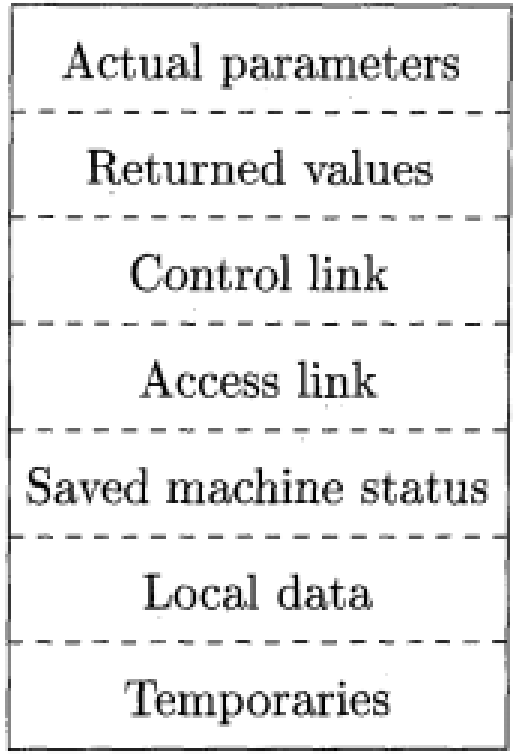
Control Stack

- ***Control Stack:*** Procedure calls and returns are managed by a run time stack
- Each live activation has an *activation record* (**AR**) or *stack frame* on the *control stack*
- Root of the activation tree (main proc) at the bottom
- Sequence of AR in the stack corresponds to path in the activation tree



Activation Records (AR)

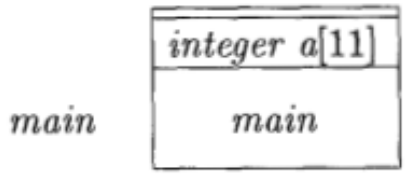
- Contents of AR depends on language.
- Contents may be
  - ***Temporary*:** to evaluate expressions
  - ***Local Data*:** belonging to the procedures
  - ***Saved machine status*:**
  - ***Access link*:** to locate data found in another activation
  - ***Control link*:** pointing to AR of the caller
  - ***Returned Value*:**
  - ***Actual Parameters*:** used by the calling procedure



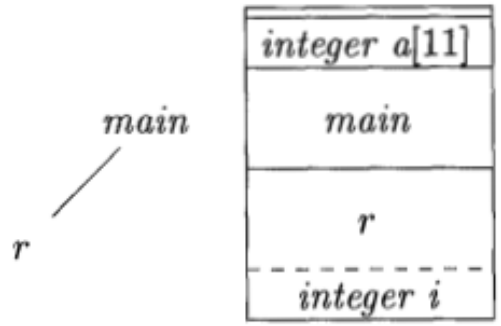
A general Activation Record

Activation Records (AR)

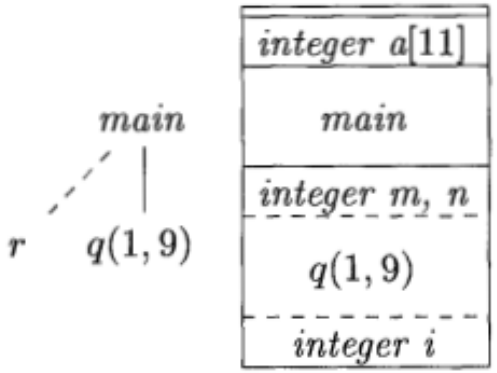
```
1  ...
2  void quicksort(int m, int n){
3      int i;
4      if (n > m){
5          i = partition(m,n); /*
6          quicksort(m, i-1);
7          quicksort(i+1, n);
8      }

10 int a[11];

12 void main() {
13     readArray();  /* reads 9 int
14     a[0] = -9999; a[10] = 9999;
15     quicksort(1,9);
16 }
```

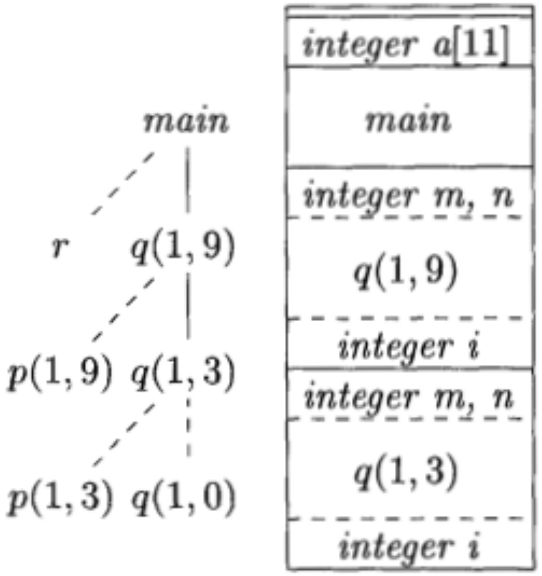


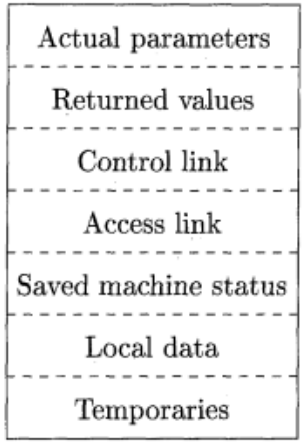
(a) Frame for *main*



(b) r is activated







Calling and Return Sequence

- Calling Sequence: Code that
  - allocates activation record on the stack
  - enters information in the fields
- Return Sequence: Code that
  - restores the state of the machine before calling
  - allows the calling procedure to continue execution after the call
- Calling sequences and the layout of activation records may differ greatly, even among implementations of the same language.
- The code in a calling sequence is often divided between the calling procedure (the "caller") and the procedure it calls (the "callee").
  - No exact division of run-time tasks between caller and callee;
  - the source language, the target machine, and the operating system impose requirements that may favor one solution over another.

Calling Sequence

- If a procedure is called from *n* different points,
  - The portion of the calling sequence assigned to the caller is generated n times.
  - However, the portion assigned to the callee is generated only once.
- Hence, it is desirable to put as much of the calling sequence into the callee as possible — whatever the callee can be relied upon to know.
- However, that the callee cannot know everything.

Principles of Designing Calling Sequences

- Values communicated between caller and callee are generally placed at the beginning of the callee's activation record,
  - As such, placed as close as possible to the caller's activation record.
- Motivation
  - The caller can compute the values of the actual parameters of the call and place them on top of its own activation record,
  - No need to create the entire activation record of the callee, or even to know the layout of that record.
  - Allows for the use of procedures that do not always take the same number or type of arguments, such as C's *printf* function.
  - The callee knows where to place the return value, relative to its own activation record, while however many arguments are present will appear sequentially below that place on the stack.

Principles of Designing Calling Sequences

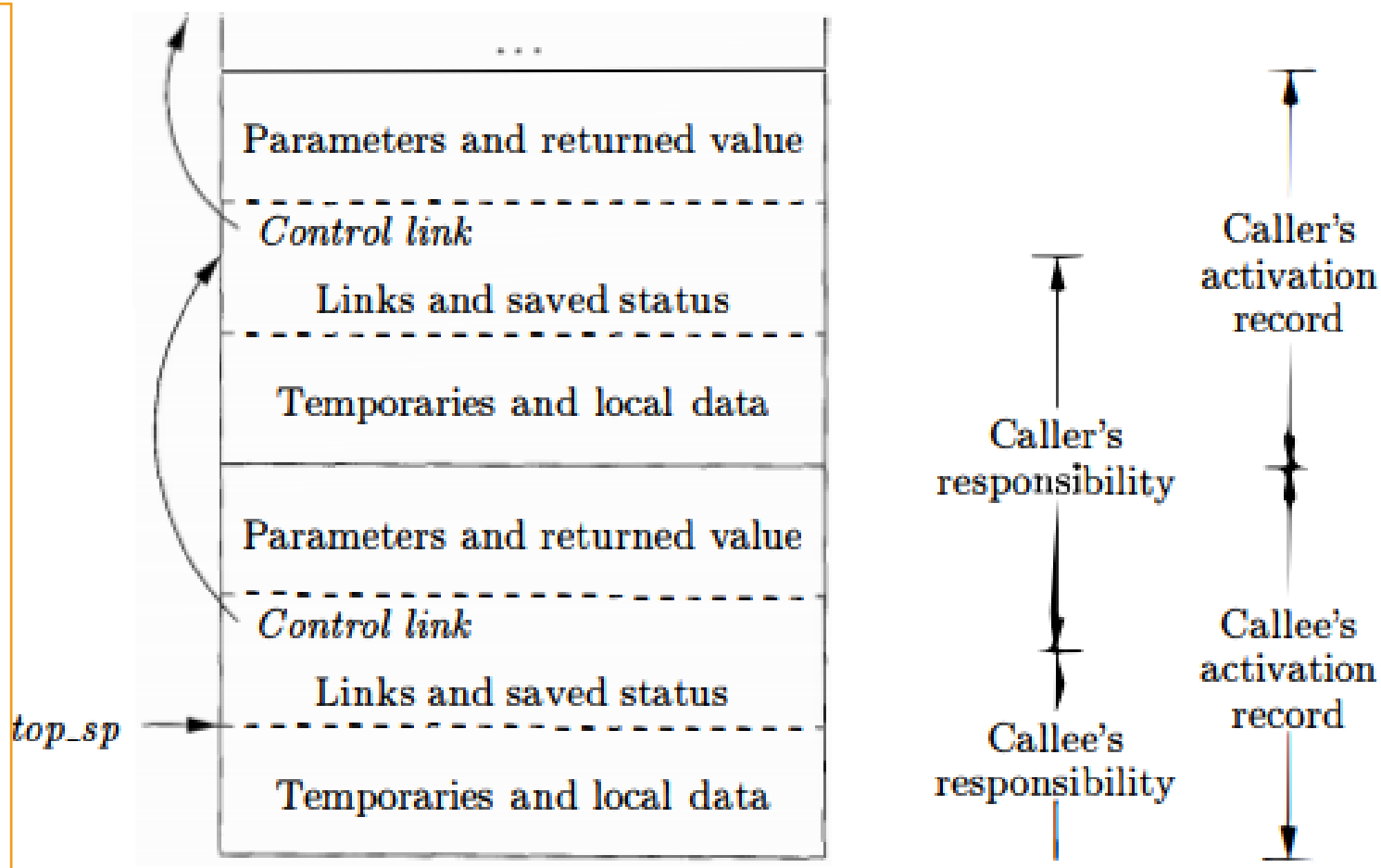
- Fixed-length items are generally placed in the middle.
  - Example: the control link, the access link, and the machine status fields.
- If exactly the same components of the machine status are saved for each call, then the same code can do the saving and restoring for each.
- Moreover, if we standardize the machine's status information, then programs such as debuggers will have an easier time deciphering the stack contents if an error occurs.
- Items whose size may not be known early enough are placed at the end of the activation record.
  - Such as dynamically sized array (however, usually dynamic arrays are handles using heap)
  - Temporary variables

Principles of Designing Calling Sequences

- Locating the top-of-stack pointer
  - A common approach is to have it point to the end of the fixed-length fields in the activation record.
- Fixed-length data can then be accessed by fixed offsets, known to the intermediate-code generator, relative to the top-of-stack pointer.
  - A consequence of this approach is that variable-length fields in the activation records are actually "above" the top-of-stack.
  - Their offsets need to be calculated at run time, but they too can be accessed from the top-of-stack pointer, by using a positive offset.

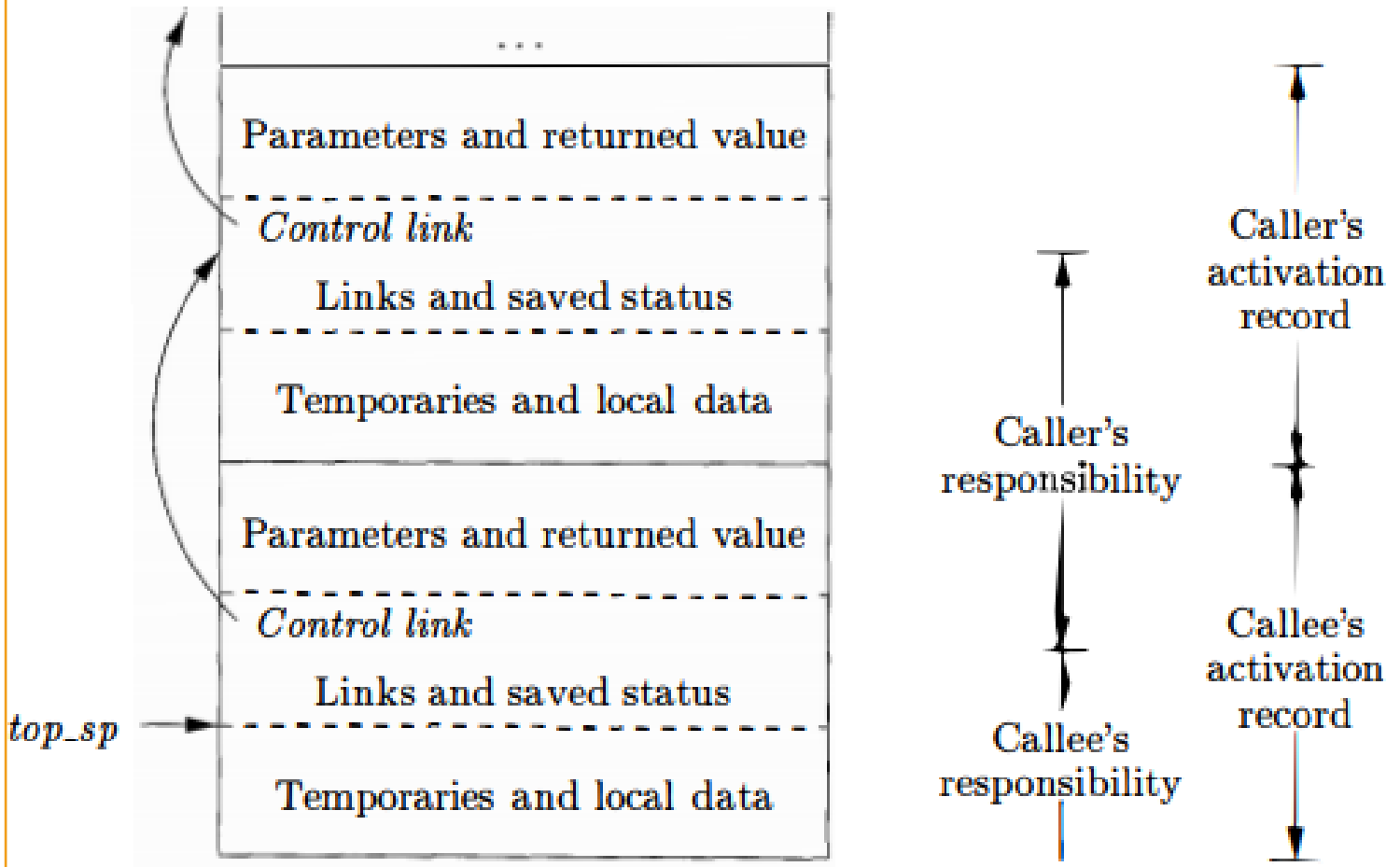
Example of Task Division Between Caller and Callee

- A register *top_sp* points to the end of the machine-status field in the current top activation record.
  - This position within the callee's activation record is known to the caller,
  - the caller can be made responsible for setting top_sp before control is passed to the callee.



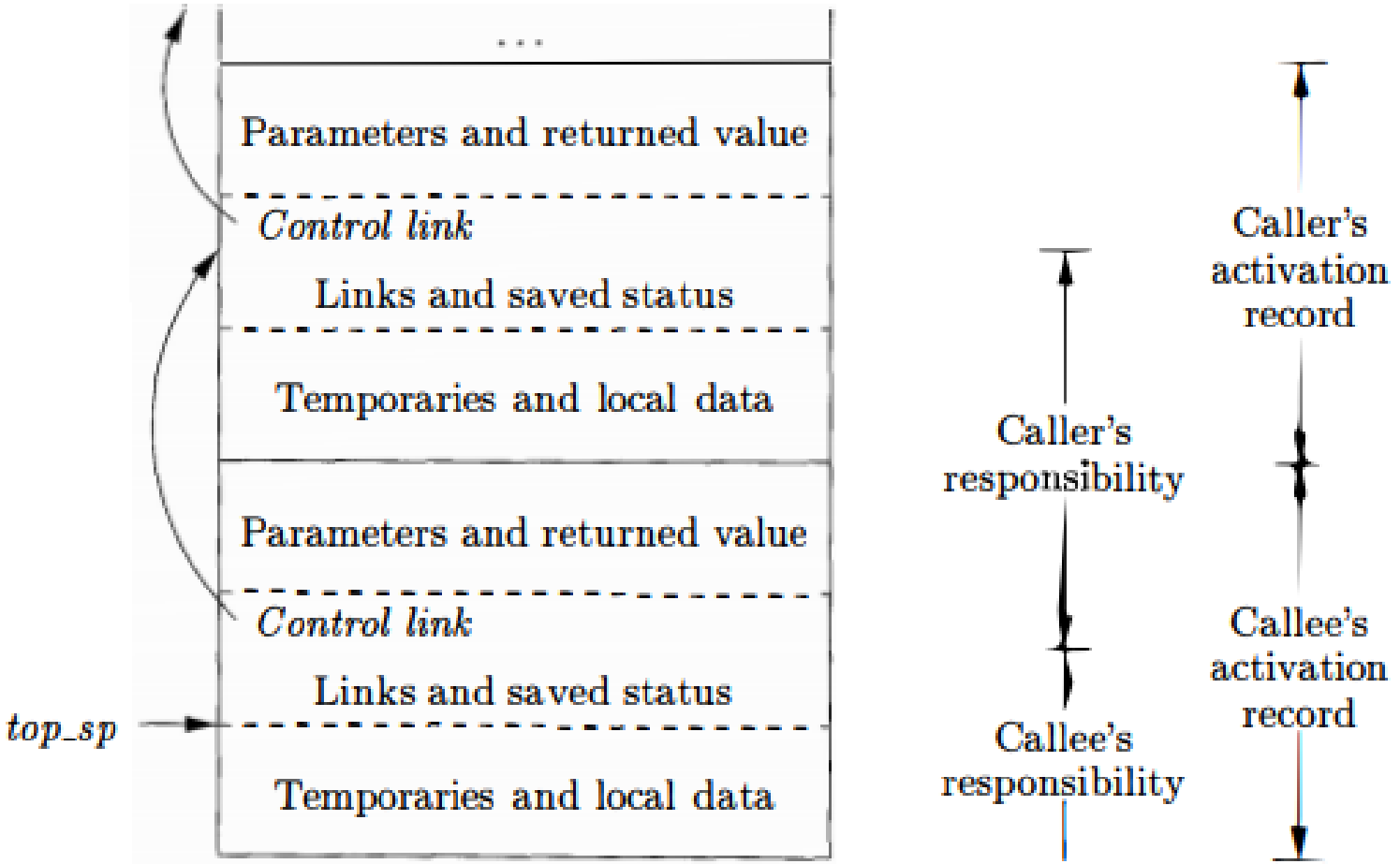
Example of Task Division Between Caller and Callee

- The caller evaluates the actual parameters.
- The caller stores a return address and the old value of *top_sp* into the callee's activation record.
- The caller then increments *top_sp* to the position.
  - *top_sp* is moved past the caller's local data and temporaries and the callee's parameters and status fields.
- The callee saves the register values and other status information.
- The callee initializes its local data and begins execution.



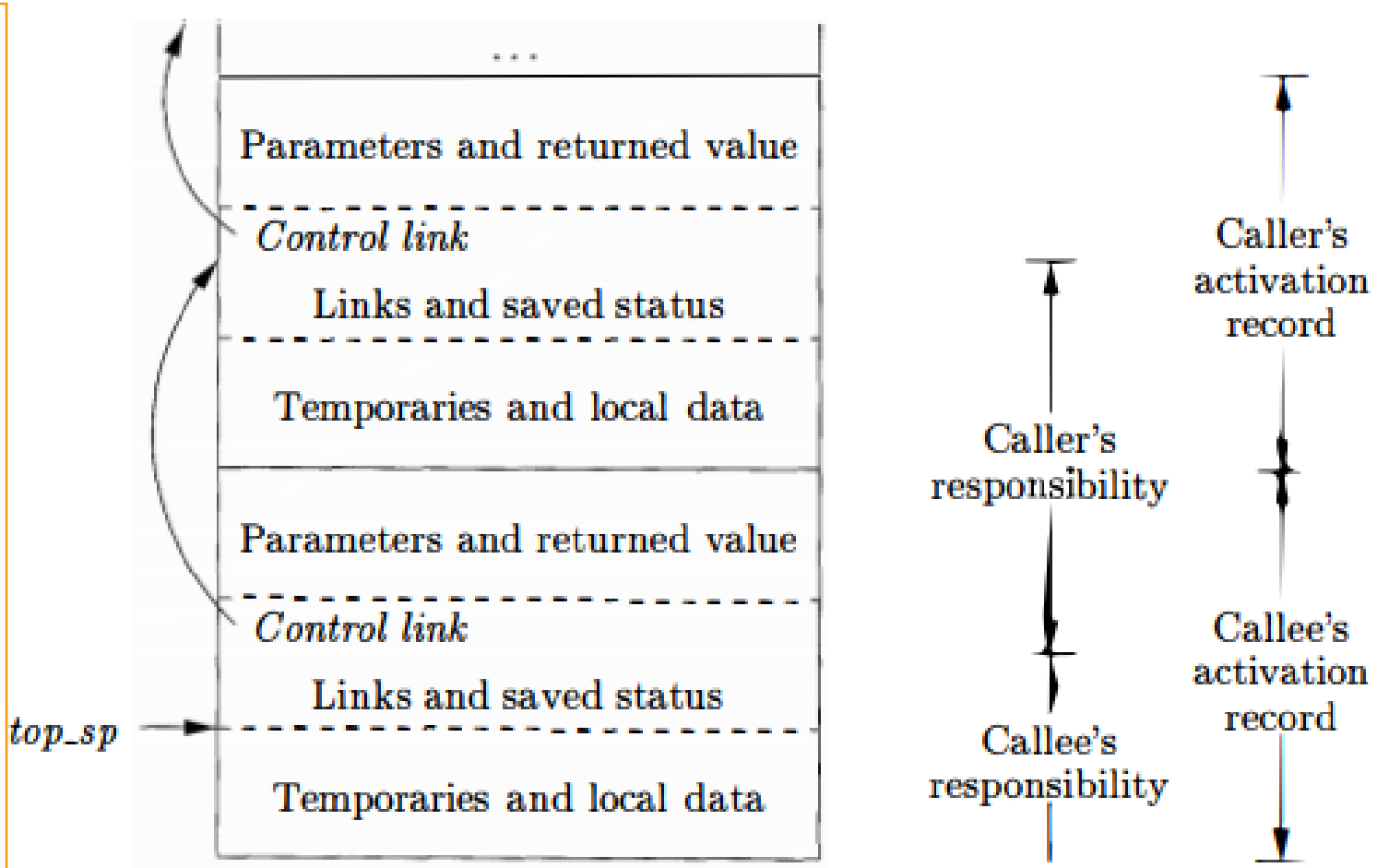
Example of Return Sequence

- The callee places the return value next to the parameters
- Using information in the machine-status field, the callee restores *top_sp* and other registers, and then branches to the return address that the caller placed in the status field.
- Although *top_sp* has been decremented, the caller knows where the return value is, relative to the current value of *top_sp*; the caller therefore may use that value.



Example of Return Sequence

- The described calling and return sequences allow the number of arguments of the called procedure to vary from call to call (e.g., as in C's p r i n t f function).
- At compile time, the target code of the caller knows the number and types of arguments it is supplying to the callee. Hence the caller knows the size of the parameter area.
- The target code of the callee, then examines the parameter field. Using the organization on the figure, information describing the parameters must be placed next to the status field, so the callee can find it.
- For example, in the printf function of C, the first argument describes the remaining arguments, so once the first argument has been located, the caller can find whatever other arguments there are.



A Sample From the Assignment

```
void foo(int a, int b)
{
    int c;
    c=a+b;
}

int main()
{
    foo(2, 3);
    return 0;
}
```

```
foo PROC
    PUSH BP
    MOV BP, SP
    SUB SP, 2
L1:
    MOV AX, [BP+4] ; Line 4
    MOV DX, AX
    MOV AX, [BP+6] ; Line 4
    ADD AX, DX
    MOV [BP-2], AX
    ADD SP, 2
    POP BP
    RET 4
foo ENDP
```

```
main PROC
    MOV AX, @DATA
    MOV DS, AX
    PUSH BP
    MOV BP, SP
    MOV AX, 2       ; Line 9
    PUSH AX
    MOV AX, 3       ; Line 9
    PUSH AX
    CALL foo
L1:
    MOV AX, 0       ; Line 10
    JMP L6
L2:
    POP BP
    MOV AX,4CH
    INT 21H
main ENDP
END main
```

Procedures Not Nesting in Time

- Closure
  - Allows access to outer function's scope from an inner function
  - A python example –

```
def createCounter():
    counter=0
    def innerFunction():
        nonlocal counter
        counter=counter+1
        print (counter)
    return innerFunction

if __name__ == '__main__':
    f = createCounter()
    f()
    f()
```

```
1
2
```

- Coroutine
  - A function that can be suspended and resumed
  - A python example –

```
def downFrom(n):
    while n > 0:
        yield n
        n = n-1
if __name__ == '__main__':
    for i in downFrom(3):
        print(i)
```

```
3
2
1
```

Runtime Memory Management

- Most constructs in a programming language need memory.
  - Local variables, Objects, Arrays, Strings etc.
- Some memory is preallocated and persists throughout the program:
  - Global variables, executable code, etc.
- Some memory is allocated on the runtime stack:
  - Local variables, parameters, temporaries.
- Some memory is allocated in the heap:
  - Arrays, objects.
- Memory management for the first two is trivial.
- How do we manage heap-allocated memory?

Heap Memory Management

- Heap is used for allocating space for objects created at run time
  - For example: nodes of dynamic data structures such as linked lists and trees
- Almost all programming languages support dynamic memory allocation and deallocation
  - *malloc()* and *free()* in C
  - *new()* and *delete()* in C++
  - *new()* and garbage collection in Java
- Allocation and deallocation may be
  - *completely manual* (C/C++),
  - *semi-automatic* (Java), or
  - *fully automatic* (Python)

Memory Manager

- Memory Manager is the subsystem that allocates and deallocates space in the heap.
- Keep track of all the free space in the heap storage at all the time
- Allocation
  - When a program requests memory for a variable or object, the memory manager produces a chunk of contiguous heap memory of the requested size.
  - If possible, it satisfies an allocation request using free space in the heap;
  - if no chunk of the needed size is available, it seeks to increase the heap storage space by getting consecutive bytes of virtual memory from the operating system.
  - If space is exhausted, the memory manager passes that information back to the application program.

Memory Manager (2)

- Deallocation
  - The memory manager returns deallocated space to the pool of free space, so it can reuse the space to satisfy other allocation requests.
  - Memory managers typically do not return memory to the operating system, even if the program's heap usage drops.

Memory Manager (3)

- The memory manager must be prepared to service, in any order, allocation and deallocation requests of any size, ranging from one byte to as large as the program's entire address space.
- Goals
  - Space efficiency: minimize *fragmentation*
  - Program efficiency: take advantage of locality of objects (both temporal and spatial) in memory and make the program run faster
  - Low overhead: allocation and deallocation must be efficient

Locality in Programs

- Most programs exhibit a high degree of locality
  - Temporal locality: the memory locations currently being accessed are likely to be accessed again within a short period of time
  - Spatial locality: the memory locations closed to the locations currently being accessed are likely to be accessed again within a short period of time
- One effective technique to improve the spatiality locality of instructions is to place the basic blocks that are likely to follow each other contiguously -one the same page
  - For example: instructions belonging to the same loop or same function
- Temporal and spatial locality of data accesses can be improved by changing the data layout or the order of computations.

Heap Fragmentation

- In the beginning, the heap is one large and contiguous block of memory
- As allocation requests are satisfied, chunks are cut off from this block and given to the program
- As deallocations are made, chunks are returned to the heap and are free to be allocated again (holes)
- After a number of allocations and deallocations, memory is fragmented and not contiguous

Heap Fragmentation

- To begin with the whole heap is a single chunk of size 500K byte
- After a few allocations and deallocations, there are holes
- In the above picture, it is not possible to allocate 100K or 150K even though total free memory is 150K

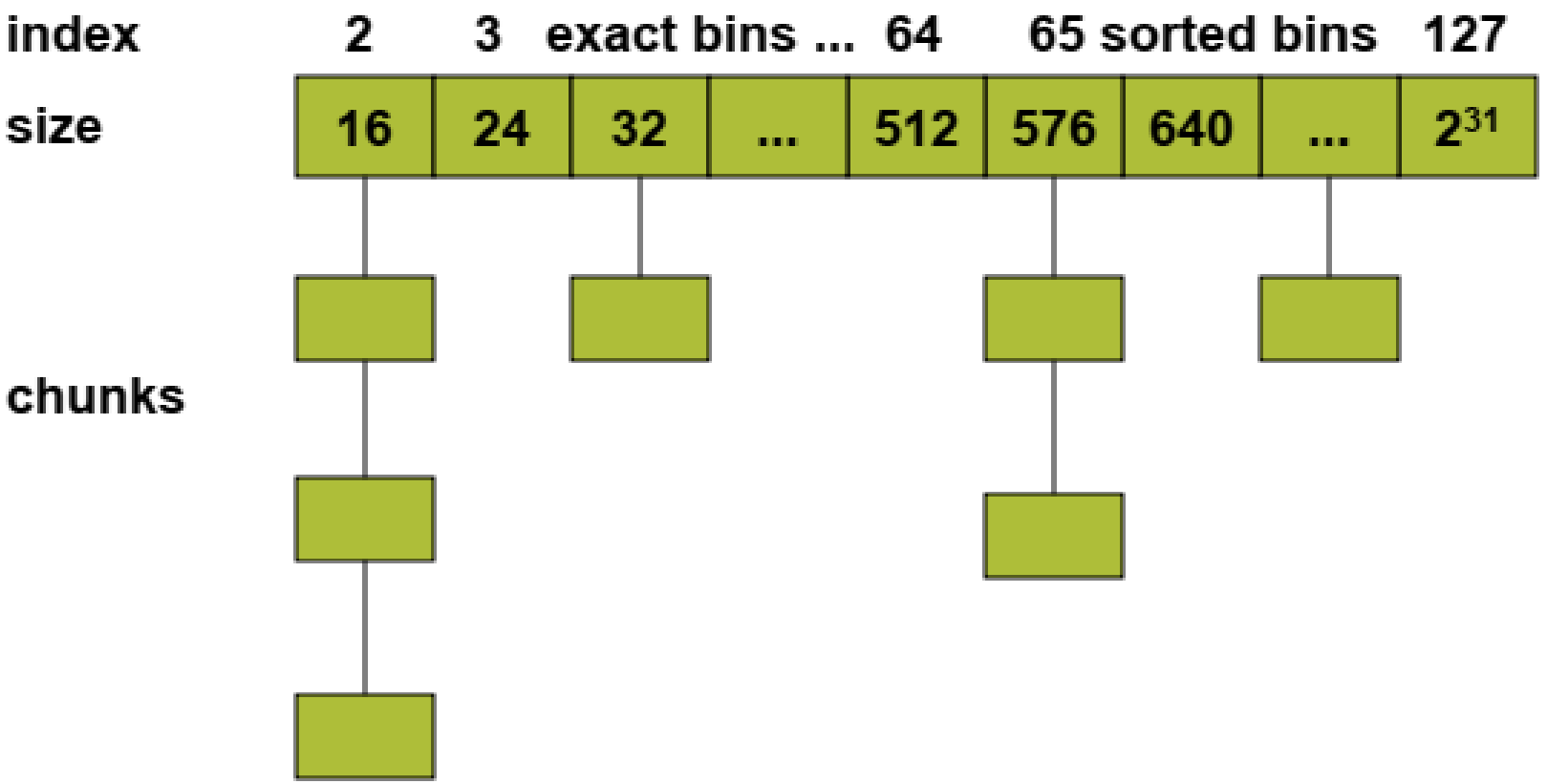
Heap Allocation Strategies

- The *first-fit* strategy picks the **first** available chunk that satisfies the allocation request
- The *best-fit* strategy searches and picks the smallest (**best**) possible chunk that satisfies the allocation request
- Both of them chop off a block of the required size from the chosen chunk, and return it to the program
- The rest of the chosen chunk remains in the heap
- *Best-fit* strategy has been shown to reduce fragmentation in practice, better than first-fit strategy

Bin-based Heap

- To implement best-fit placement more efficiently, free space chunks are organized into bins according to their sizes (Lea Memory Manager in GCC)
  - Many more bins for smaller sizes, because there are many more small objects
  - A bin for every multiple of 8-byte chunks from 16 bytes to 512 bytes
  - Then approximately logarithmically (double previous size)
  - Within each “small size bin”, chunks are all of the same size
  - In others, they are ordered by size
  - The last chunk in the last bin is the *wilderness chunk*, which gets us a chunk by going to the operating system

Bin-based Heap



Ref: http://gee.cs.oswego.edu/dl/html/malloc.html

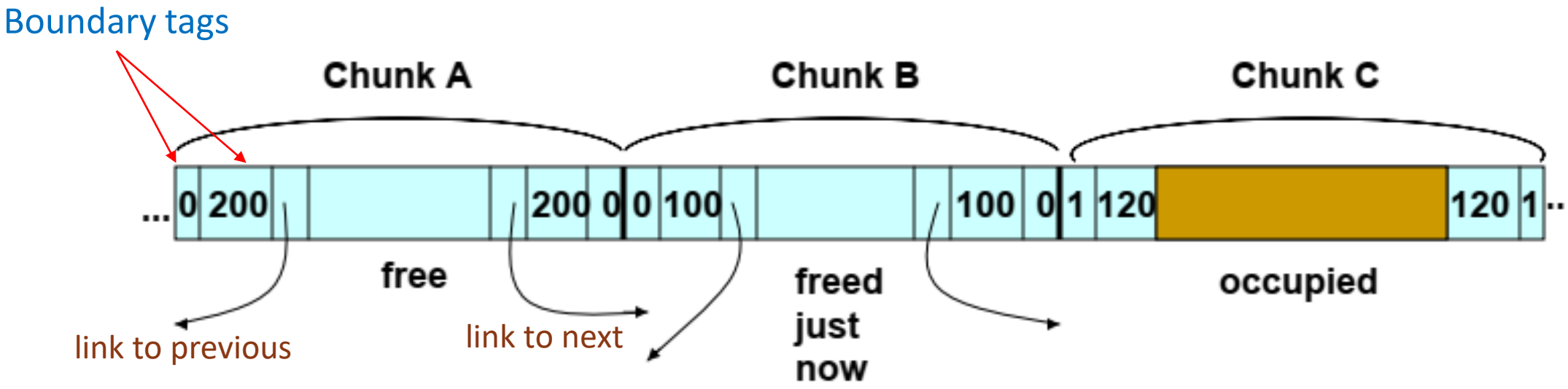
Heap Allocation Strategies

- While best-fit placement tends to improve space utilization, it may not be the best in terms of spatial locality.
- Chunks allocated at about the same time by a program tend to have similar reference patterns and to have similar lifetimes.
- Placing them close together thus improves the program's spatial locality.
- Next-fit strategy tries to allocate the object in the chunk that has last been split
  - Tends to improve speed of allocation
  - Tends to improve spatial locality since objects allocated at about the same time tend to have similar reference patterns and life times (cache behavior may be better)

Managing and Coalescing Free Space

- When an object is deallocated, it is possible to combine (coalesce) the freed chunk with adjacent chunks to form a larger chunk
- Coalescing reduces fragmentation
- Data structures to support coalescing
  - *Boundary tags* (free/used bit and chunk size) at each end of a chunk (for both used and free chunks)
  - A *doubly linked embedded list* of free chunks

Boundary Tags and Doubly Linked List



3 adjacent chunks. Chunk B has just been deallocated and returned to the free list. Chunks A and B can be merged, and this is done just before inserting it into the linked list. The merged chunk AB may have to be placed in a different bin.

Manual Memory Management

- Have the programmer handle allocation and deallocation of dynamic memory.
  - Approach used in C, C++.
- Easily leads to troublesome bugs:
  - Memory leaks
    - Failing to delete data that cannot be referenced
    - Important in long running or non-stop programs
  - Dangling pointer dereferencing
    - Referencing deleted data
    - Creates a program error that is hard to debug

Automatic Memory Management

- Have the runtime environment automatically reclaim memory.
  - Approach used in Java, Perl, ML, Prolog etc.
- Objects that won't be used again are called garbage.
- Reclaiming garbage objects automatically is called garbage collection.

Preliminaries

- Garbage collection is the reclamation of chunks of storage holding objects that can no longer be accessed by a program.
- Assumptions
  - Objects have a type that can be determined by the garbage collector at run time.
  - Type information allows to determine
    - how large the object is, and
    - which components of the object contain references (pointers) to other objects.
  - References to objects are always to the address of the beginning of the object, never pointers to places within the object.
    - All references to an object have the same value and can be identified easily

Preliminaries (2)

- A user program, hereafter referred to as the mutator, modifies the collection of objects in the heap.
- The mutator creates objects by acquiring space from the memory manager
  - May introduce and drop references to existing objects.
- Objects become garbage when the mutator program cannot "reach" them
- The garbage collector finds these unreachable objects and reclaims their space by handing them to the memory manager, which keeps track of the free space.

A Basic Requirement: Type Safety

- For a garbage collector to work, it must be able to tell whether any given data element or component of a data element is, or could be used as, a pointer to a chunk of allocated memory space.
- A language in which the type of any data component can be determined is said to be type safe.
  - There are type-safe languages like ML, for which we can determine types at compile time.
  - There are other type-safe languages, 'like Java, whose types cannot be determined at compile time, but can be determined at run time. The latter are called dynamically typed languages.

A Basic Requirement: Type Safety

- If a language is neither statically nor dynamically type safe, then it is said to be unsafe. .
  - Unsafe languages, such as C and C++, are bad candidates for automatic garbage collection.
- In unsafe languages, memory addresses can be manipulated arbitrarily:
  - arbitrary arithmetic operations can be applied to pointers to create new pointers, and arbitrary integers can be cast as pointers.
  - A program theoretically could refer to any location in memory at any time.
  - Consequently, no memory location can be considered to be inaccessible, and no storage can ever be reclaimed safely.

GC Performance Metrics

- Overall Execution Time
  - Garbage collection can be very slow. It is important that it not significantly increase the total run time of an application.
  - Performance is determined greatly by how it leverages the memory subsystem.
- Space Usage
  - Need to avoid fragmentation and make the best use of the available memory.

Performance Metrics

- Pause Time
  - Simple garbage collectors are notorious for causing programs - the mutators - to pause suddenly for an extremely long time, as garbage collection kicks in without warning.
  - In addition to minimizing the overall execution time, it is desirable that the maximum pause time be minimized.
  - Special case of real-time applications -
    - Requires certain computations to be completed within a time limit.
    - Garbage collection must be suppressed while performing real-time tasks, or restrict maximum pause time.
    - In practice, garbage collection is seldom used in real-time applications.

Performance Metrics

- Program Locality
  - The garbage collector controls the placement of data and thus influences the data locality of the mutator program.
  - It can improve a mutator's temporal locality by freeing up space and reusing it
  - It can improve the mutator's spatial locality by relocating data used together in the same cache or pages

Reachability

- Root set: all the data that can be accessed directly by a program, without having to dereference any pointer
- For example, in Java the root set of a program consists of
  - all the static field members and all the variable on its stack.
- A program obviously can reach any member of its root set at any time.
- Recursively, any object with a reference that is stored in the field members or array elements of any reachable object is itself reachable.

Reachability (2)

- The set of reachable objects changes as a program executes.
- It grows as new objects get created and shrinks as objects become unreachable.
- It is important to remember that once an object becomes unreachable, it cannot become reachable again.

Reachability Changing Operations

- There are four basic operations that a mutator performs to change the set of reachable objects.
- Object Allocations
  - Performed by the memory manager, which returns a reference to each newly allocated chunk of memory.
  - This operation adds members to the set of reachable objects.
- Parameter Passing and Return Values
  - References to objects are passed from the actual input parameter to the corresponding formal parameter, and from the returned result back to the callee.
  - Objects pointed to by these references remain reachable.

Reachability Changing Operations (2)

- Reference Assignments
  - Assignments of the form u = v, where u and v are references, have two effects.
  - First, u is now a reference to the object referred to by v . As long as u is reachable, the object it refers to is surely reachable.
  - Second, the original reference in u is lost. If this reference is the last to some reachable object, then that object becomes unreachable.
  - Any time an object becomes unreachable, all objects that are reachable only through references contained in that object also become unreachable.
- Procedure Returns
  - As a procedure exits, the frame holding its local variables is popped off the stack.
  - If the frame holds the only reachable reference to any object, that object becomes unreachable.
  - Again, if the now unreachable objects hold the only references to other objects, they too become unreachable, and so on.

Finding Unreachable Objects

- There are two basic ways to find unreachable objects.
  1. Catch the transitions as reachable objects turn unreachable,
  2. Periodically locate all the reachable objects and then infer that all the other objects are unreachable.
- Reference counting, is a well-known approximation to the first approach.
  - A count of the references to an object is maintained, as the mutator performs actions that may change the reachability set.
  - When the count goes to zero, the object becomes unreachable.

Finding Unreachable Objects (2)

- The second approach computes reachability by tracing all the references transitively.
- A trace-based garbage collector
  - starts by labeling ("marking") all objects in the root set as "reachable,"
  - examines iteratively all the references in reachable objects to find more reachable objects, and labels them as such.
- This approach must trace all the references before it can determine any object to be unreachable.
- But once the reachable set is computed, it can find many unreachable objects all at once and locate a good deal of free storage at the same time.
- Because all the references must be analyzed at the same time, it is possible to relocate the reachable objects and thereby reduce fragmentation.

Reference Counting Garbage Collectors

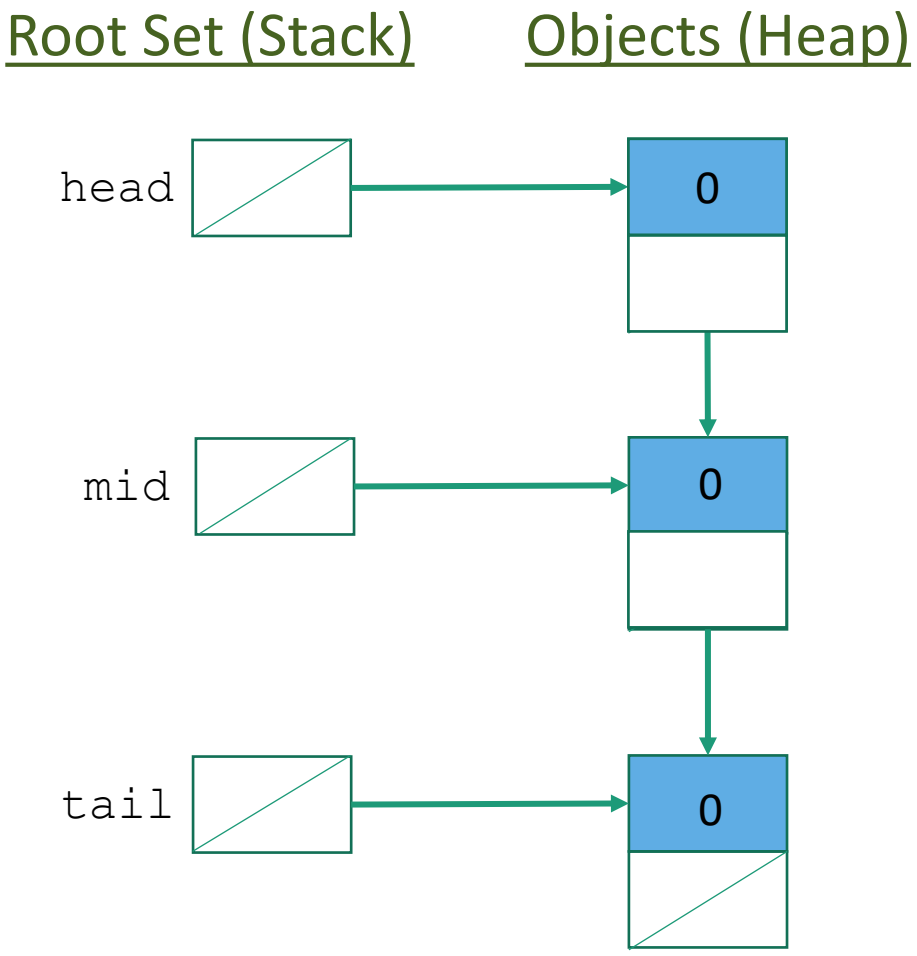
- A simple framework for garbage collection.
  - Though it has several serious weaknesses!
- Idea: Store in each object a **reference count** (**refcount**) tracking how many references exist to the object.

Tracking Reference Count

- Reference counts can be maintained as follows:
  - Object Allocation. The reference count of the new object is set to 1.
  - Parameter Passing. The reference count of each object passed into a procedure is incremented.
  - Reference Assignments. For statement u = v, where u and v are references, the reference count of the object referred to by v goes up by one, and the count for the old object referred to by u goes down by one.
  - Procedure Returns. As a procedure exits, all the references held by the local variables of that procedure activation record must also be decremented.
    - If several local variables hold references to the same object, that object's count must be decremented once for each such reference.
  - Transitive Loss of Reachability. Whenever the reference count of an object becomes zero, we must also decrement the count of each object pointed to by a reference within the object.

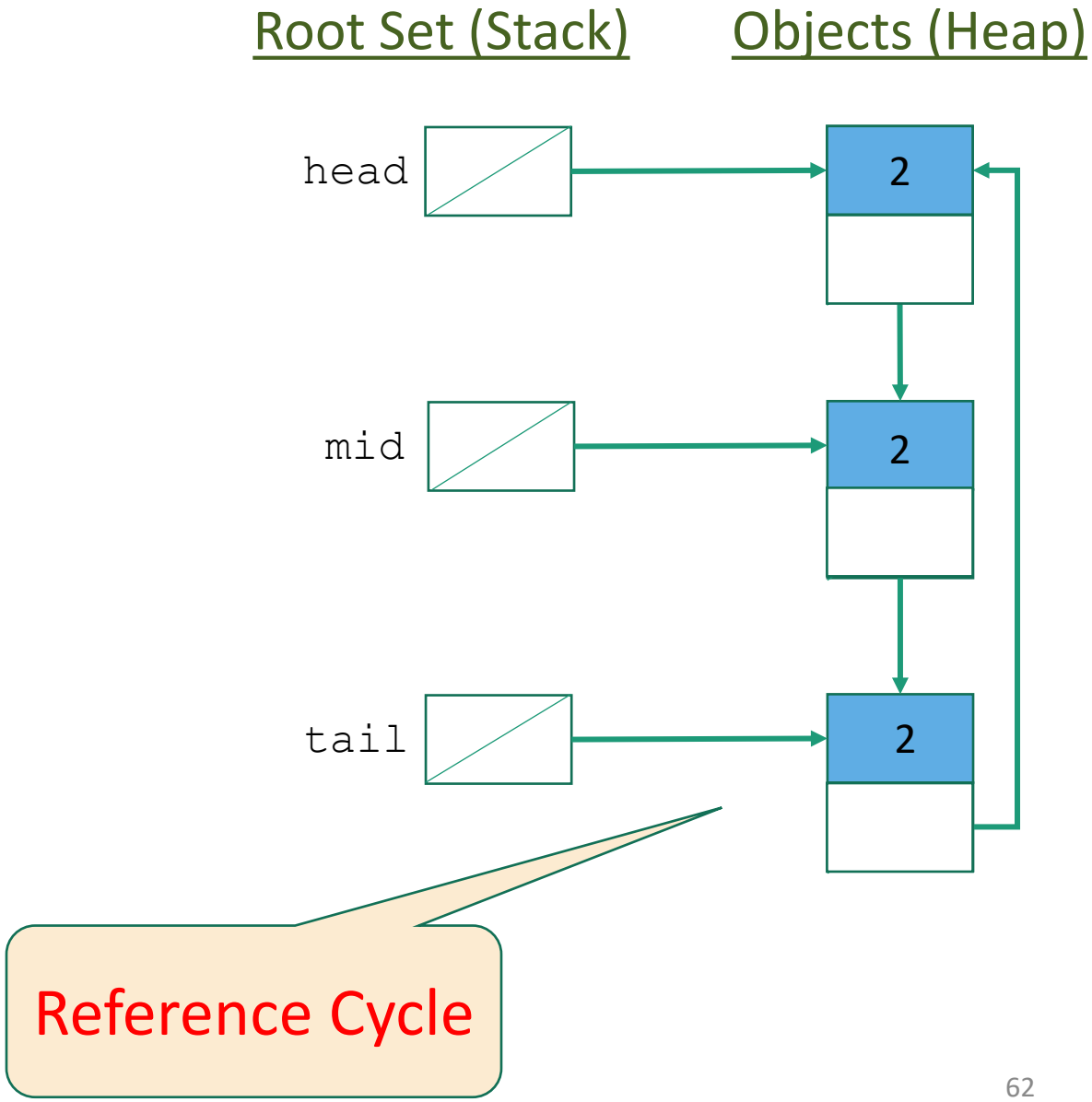
Reference Counting in Action

```
class LinkedList {
    LinkedList next;
}
int main() {
    LinkedList head = new LinkedList;
    LinkedList mid = new LinkedList;
    LinkedList tail = new LinkedList;

    head.next = mid;
    mid.next = tail;

    mid = tail = null;

    head.next.next = null;
    head = null;
}
```



Problem with Reference Counting

```
class LinkedList {
    LinkedList next;
}
int main() {
    LinkedList head = new LinkedList;
    LinkedList mid = new LinkedList;
    LinkedList tail = new LinkedList;

    head.next = mid;
    mid.next = tail;
    tail.next = head;

    head = null;
    mid  = null;
    tail = null;
}
```



Reference Cycles

- A reference cycle is a set of objects that cyclically refer to one another.
- Because all the objects are referenced, all have nonzero refcounts and are never reclaimed.
- Issue: Refcount tracks number of references, not number of reachable references.
- Major problems in languages/systems that use reference counting e.g. Perl.

Analysis of Reference Counting

- Disadvantages:
  - Cannot collect unreachable, cyclic data structures
  - The overhead is high. Proportional to the amount of computation in the program, and not just to the number of objects in the system.
- Advantages:
  - Can be performed in an incremental fashion.
  - Even though the total overhead can be large, the operations are spread throughout the mutator's computation.
  - The operation of recursively modifying reference counts can be deferred and performed piecemeal across time.
  - Suitable when timing deadlines must be met, as well as for interactive applications where long, sudden pauses are unacceptable.
  - Garbage is collected immediately, keeping space usage low.

Trace-Based Collection

- Instead of collecting garbage as it is created, trace-based collectors run periodically to find unreachable objects and reclaim their space.
- Typically, we run the trace-based collector whenever the free space is exhausted or its amount drops below some threshold.
- Trace-based algorithms
  - Mark-and-Sweep Garbage Collectors: stop-the world algorithms
    - Basic algorithm
    - Optimization with Baker's algorithm
  - Mark-and-Compact Garbage Collectors
  - Copying Garbage Collectors

Basic Abstraction

- All trace-based algorithms compute the set of reachable objects and then take the complement of this set.
- Memory is therefore recycled as follows:
  1. The program or mutator runs and makes allocation requests.
  2. The garbage collector discovers reachability by tracing.
  3. The garbage collector reclaims the storage for unreachable objects.
- A memory chunk can be in four states in a GC cycle
  - Free, Unreached, Unscanned, and Scanned.
- The state of a chunk might be stored
  - in the chunk itself, or
  - it might be implicit in the data structures used by the garbage-collection algorithm

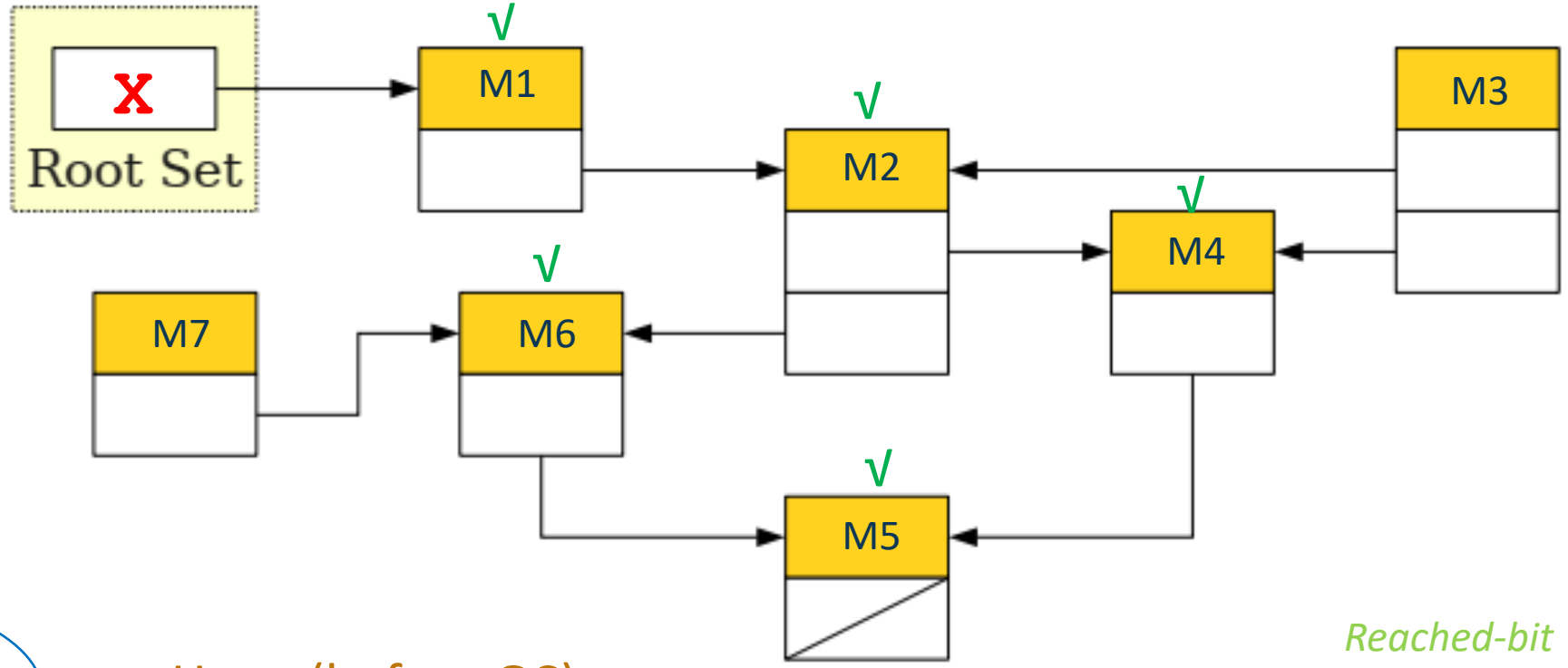
Basic Mark-and-Sweep Collector

- Mark-and-sweep runs in two phases
- Marking phase:
  - Visits and “marks” all the reachable objects
- Sweeping phase:
  - “sweeps’’ the entire heap to free up unreachable object

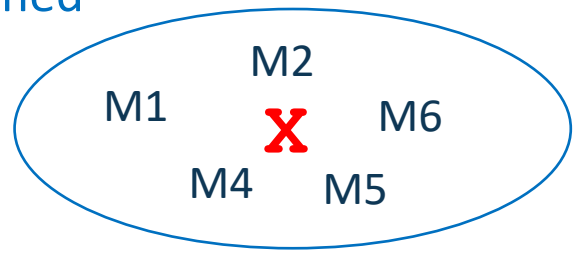
Mark-and-Sweep in Action

Marking Phase

Sweeping Phase



Unscanned



Reached-bit

Heap (before GC)

1	M1	1	M2	0	M3	1	M4	1	M5	1	M6	0	M7

Sweep →

Free

Heap (after GC)

0	M1	0	M2	0	free	0	M4	0	M5	0	M6	0	free

Mark-and-Sweep Garbage Collector

- Algorithm: Mark-and-sweep garbage collection
- Input:
  - A root set of objects,
  - a heap,
  - a list of unallocated chunks of heap, *free*
  - all chunks are marked with boundary tags
- Output:
  - A modified *free* list after all the garbage has been removed
- Data Structures:
  - List *free,*
  - List *unscanned*
  - Reached bits in all objects

```
    /* marking phase */
 1) set the reached-bit to 1 and add to list Unscanned each object
        referenced by the root set;
 2) while (Unscanned ≠ ∅) {
 3)     remove some object o from Unscanned;
 4)     for (each object o′ referenced in o) {
 5)         if (o′ is unreached; i.e., its reached-bit is 0) {
 6)             set the reached-bit of o′ to 1;
 7)             put o′ in Unscanned;
            }
        }
    }
    /* sweeping phase */
 8) Free = ∅;
 9) for (each chunk of memory o in the heap) {
10)     if (o is unreached, i.e., its reached-bit is 0) add o to Free;
11)     else set the reached-bit of o to 0;
    }
```

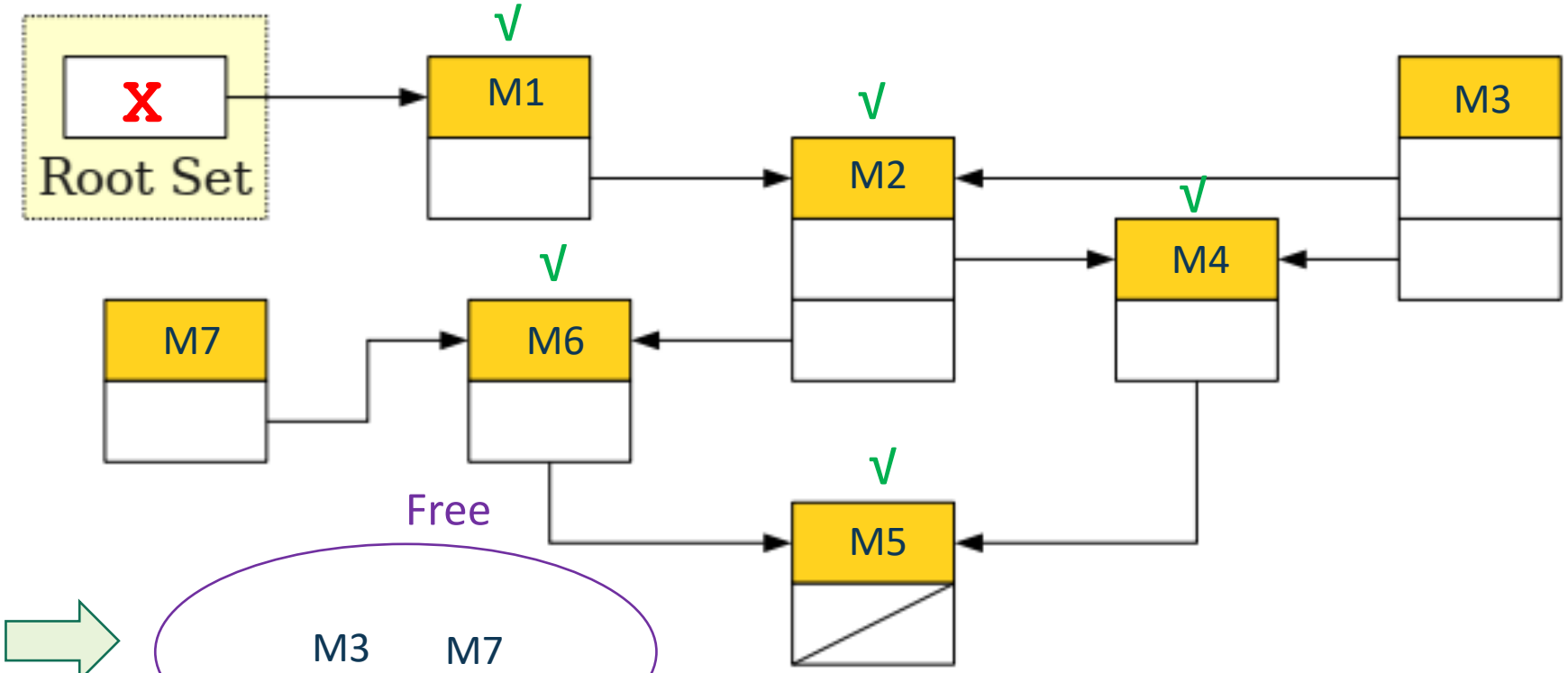
Optimizing Mark-and-Sweep Collector

- The final step of the basic mark-and-sweep is expensive
- Need to examine the entire heap
- An improved algorithm due to Baker
  - Keep a list of allocated objects
  - Take the set difference of the allocated objects and reached objects to determine the set of unreachable objects

Baker's Mark-and-Sweep in Action

Marking Phase

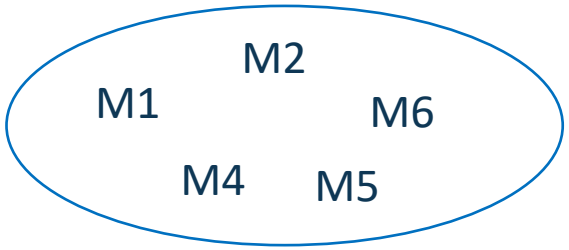
Sweeping Phase



Unreached

Free

Unscanned



Scanned

Heap (before GC)

M1	M2	M3	M4	M5	M6	M7

Heap (after GC)

M1	M2	free	M4	M5	M6	free

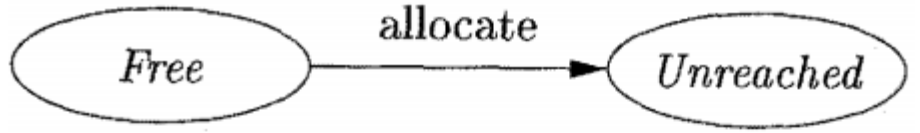
Baker's Mark-and-Sweep Collector

- Algorithm: Baker's Mark-and-Sweep Collector
- Input:
  - A root set of objects,
  - a heap,
  - a free list *free*
  - a list of allocated objects, *unreached*
- Output:
  - Modified lists: *free*, *unreached* holding allocated objects
- Data Structures:
  - Four lists: *free, unreached, unscanned, scanned*

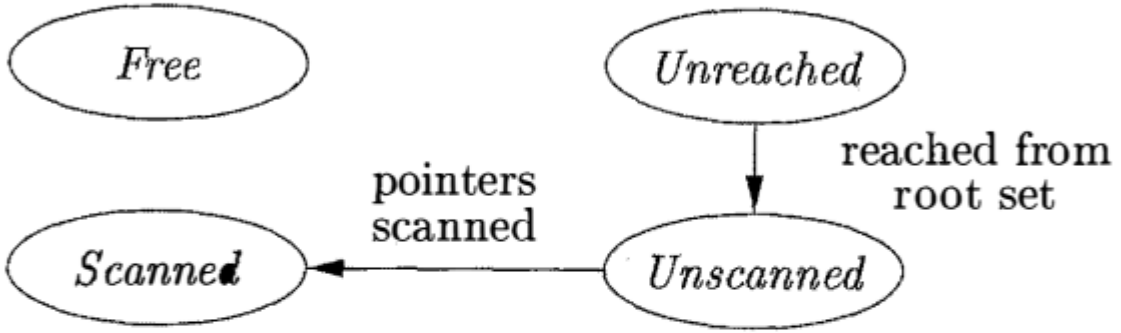
```
1)  Scanned = ∅;
2)  Unscanned = set of objects referenced in the root set;
3)  while (Unscanned ≠ ∅) {
4)          move object o from Unscanned to Scanned;
5)          for (each object o' referenced in o) {
6)                  if (o' is in Unreached)
7)                          move o' from Unreached to Unscanned;
            }
    }
8)  Free = Free ∪ Unreached;
9)  Unreached = Scanned;
```

Memory States in a Garbage Collection Cycle

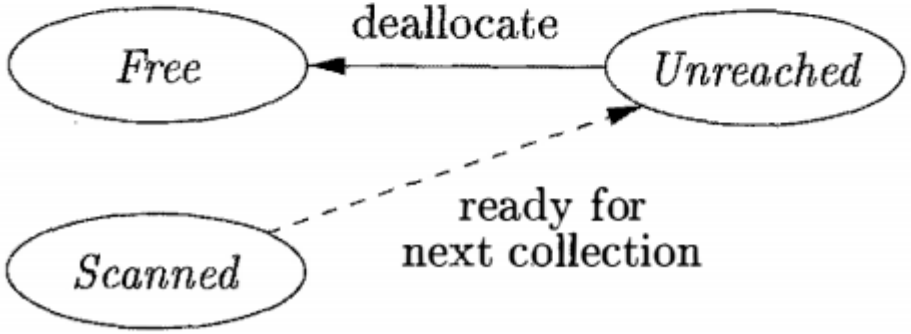
- Free
  - A chunk is in the free state if it is ready to be allocated.
  - A free chunk must not hold a reachable object.



(a) Before tracing: action of mutator



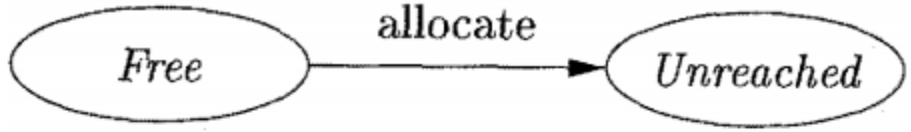
(b) Discovering reachability by tracing



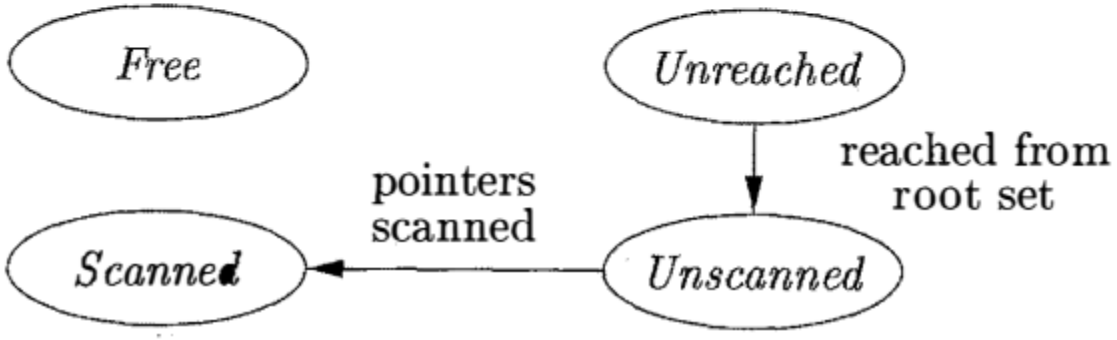
(c) Reclaiming storage

Memory States in a Garbage Collection Cycle (2)

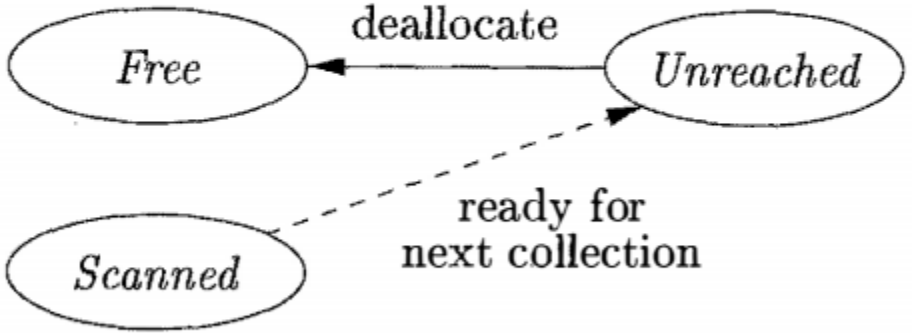
- Unreached
  - Chunks are presumed unreachable, unless proven reachable by tracing.
  - A chunk is in the unreached state at any point during garbage collection if its reach ability has not yet been established.
  - Whenever a chunk is allocated by the memory manager, its state is set to unreached.
  - Also, after a round of garbage collection, the state of a reachable object is reset to Unreached to get ready for the next round;



(a) Before tracing: action of mutator



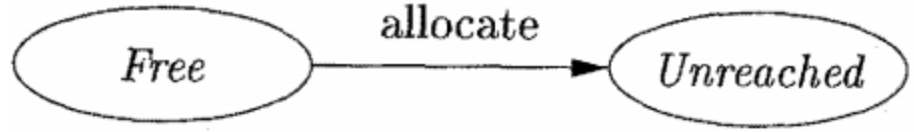
(b) Discovering reachability by tracing



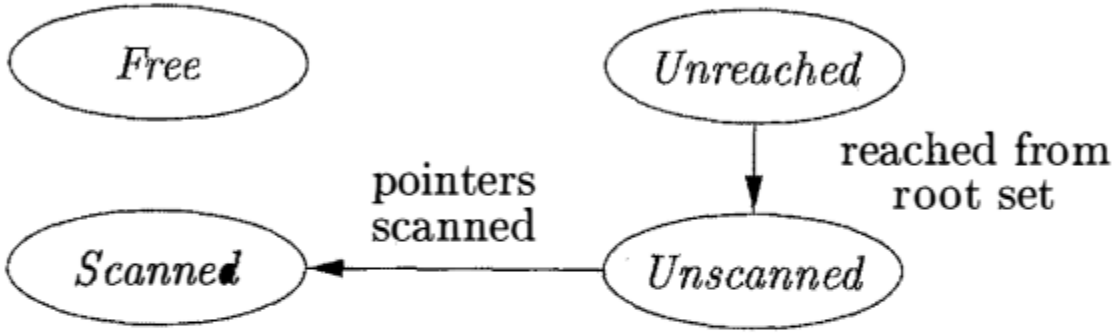
(c) Reclaiming storage

Memory States in a Garbage Collection Cycle (3)

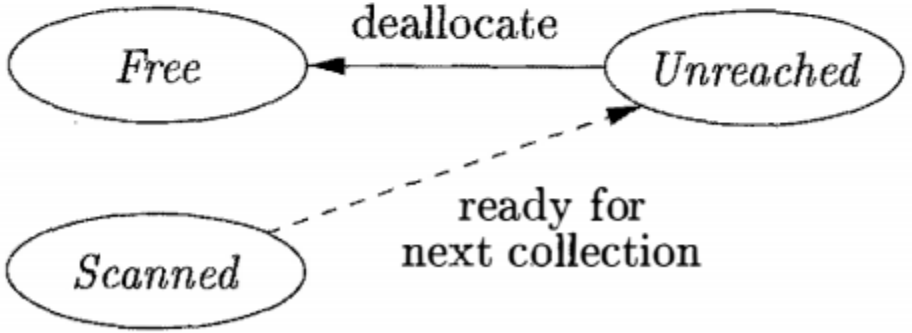
- Unscanned
  - Chunks that are known to be reachable are either in state unscanned or state scanned.
  - A chunk is in the unscanned state if it is known to be reachable, but its pointers have not yet been scanned.
  - The transition to unscanned from unreached occurs when we discover that a chunk is reachable.



(a) Before tracing: action of mutator



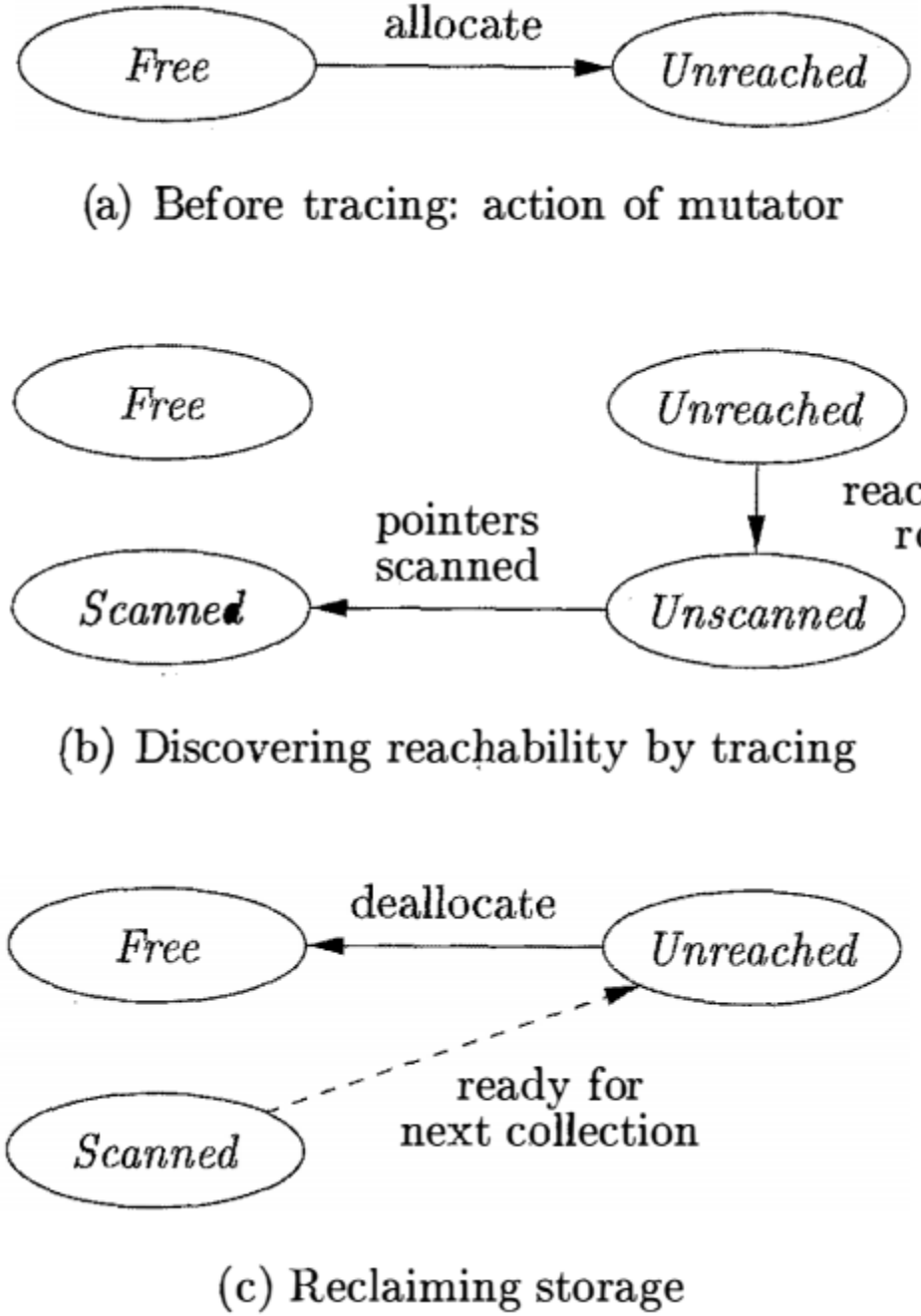
(b) Discovering reachability by tracing



(c) Reclaiming storage

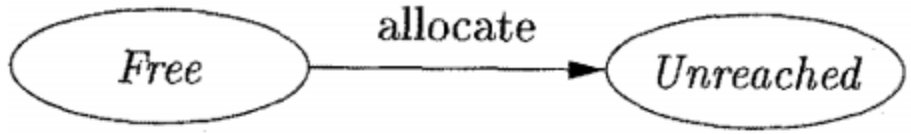
Memory States in a Garbage Collection Cycle (4)

- Scanned
  - Every unscanned object will eventually be scanned and transition to the scanned state.
  - To scan an object, we examine each of the pointers within it and follow those pointers to the objects to which they refer.
  - If a reference is to an unreached object, then that object is put in the unscanned state.
  - When the scan of an object is completed, that object is placed in the scanned state.
  - A scanned object can only contain references to other scanned or unscanned objects, and never to unreached objects.

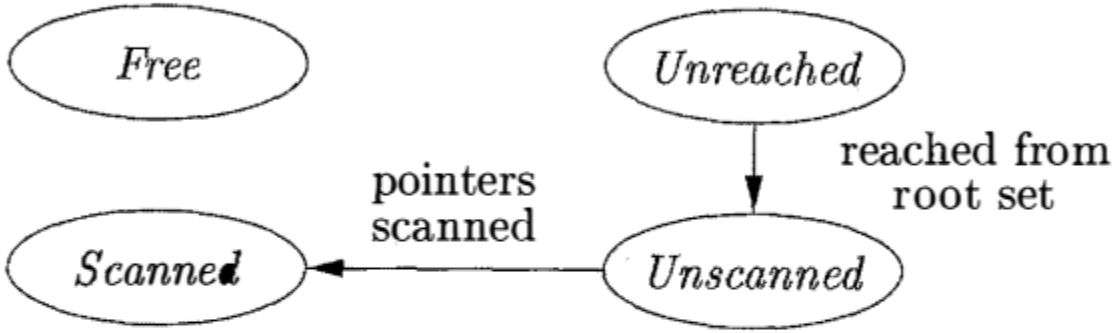


Memory States in a Garbage Collection Cycle (5)

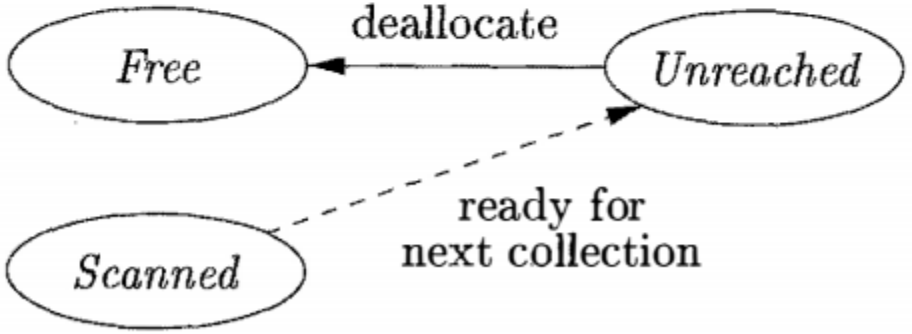
- When no objects are left in the unscanned state, the computation of reachability is complete.
- Objects left in the unreached state at the end are truly unreachable.
- The garbage collector reclaims the space they occupy and places the chunks in the free state
- To get ready for the next cycle of garbage collection, objects in the scanned state are returned to the unreached state
  - we shall want to start all objects out in this state when garbage collection next begins, by which time any of the currently reachable objects may indeed have been rendered unreachable.



(a) Before tracing: action of mutator



(b) Discovering reachability by tracing



(c) Reclaiming storage

Relocating Garbage Collectors

- Relocating collectors relocate all the reachable objects into one end of the heap, leaving the entire rest of the heap as one free chunk.
- Reduces fragmentation of the memory space
- Helps allocation of large objects
- Improves a program's temporal and spatial locality
- Allows prefetching if objects are used together.
- Data structure for maintaining free space is simplified
  - instead of a free list, all we need is a pointer free to the beginning of the one free block.

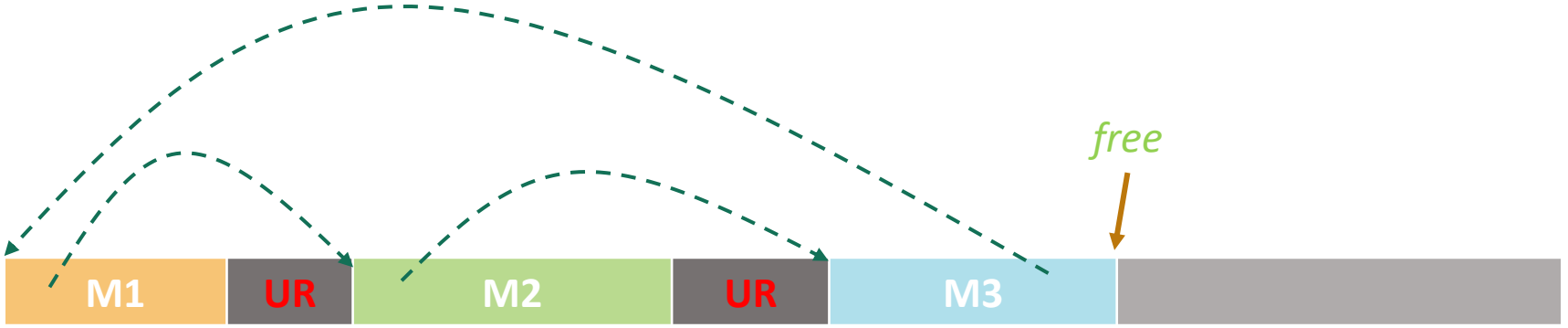
Relocating Garbage Collectors

- Relocating collectors vary in whether they relocated in place or reserve space ahead of time for relocation
- Mark-and-compact collector
  - Compacts objects in place.
  - Relocating in place reduces memory usage.
- Copying collector
  - Moves objects from one region of memory to another.
  - Reserve extra space for relocation.
  - Allows reachable objects to be moved as they are discovered.

Mark-and-Compact Garbage Collectors

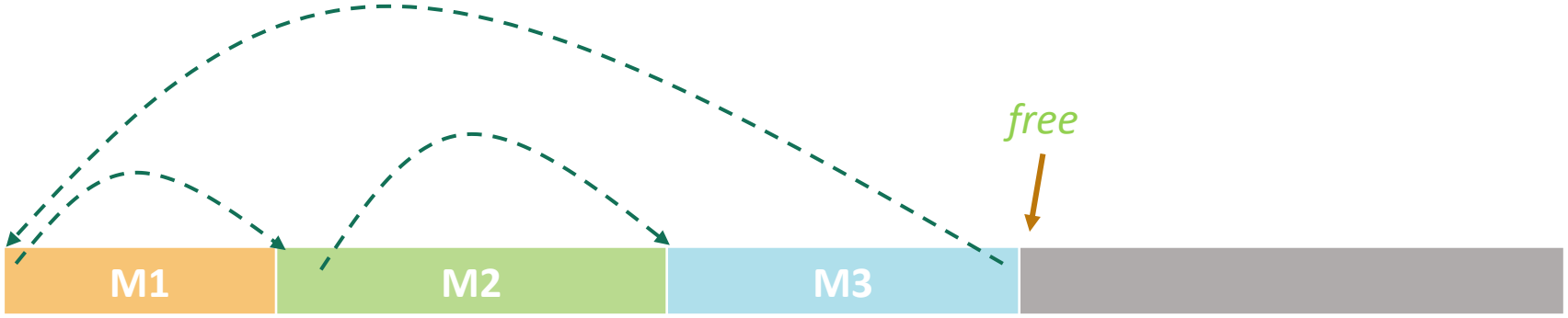
- The mark-and-compact collector works in three phases.
- Marking phase
  - similar to that of the mark-and-sweep algorithm
- New address calculation phase
  - Scans the allocated section of the heap and computes a new address for each of the reachable objects.
  - New addresses are assigned from the low end of the heap, so there are no holes between reachable objects.
  - The new address for each object is recorded in a structure called *NewLocation*.
- Copying phase
  - Copies objects to their new locations
  - Update all references in the objects to point to the corresponding new locations using *NewLocation*.

Mark-and-Compact Garbage Collectors



Heap (before GC)

Moving reached objects to the front of the heap, while preserving internal pointers



Heap (after GC)

A Mark-and-Compact Garbage Collector

- Algorithm: A mark-and-compact garbage collector
- Input
  - A root set of objects,
  - a heap,
  - a pointer *free* marking the start of free space.
- Output
  - The new value of pointer *free*
- Data Structures used
  - An *unscanned* list
  - Reached bits in all objects
  - The table *NewLocation* to keep mappings of objects to their locations

1. Marking

```
     /* mark */
1)   Unscanned = set of objects referenced by the root set;
2)   while (Unscanned ≠ ∅) {
3)         remove object o from Unscanned;
4)         for (each object o′ referenced in o) {
5)               if (o′ is unreached) {
6)                     mark o′ as reached;
7)                     put o′ on list Unscanned;
                 }
           }
     }
```

2. New Address Calculation

```
     /* compute new locations */
 8)  free = starting location of heap storage;
 9)  for (each chunk of memory o in the heap, from the low end) {
10)        if (o is reached {
11)              NewLocation(o) = free;
12)              free = free + sizeof(o);
           }
     }
```

A Mark-and-Compact Garbage Collector

- Algorithm: A mark-and-compact garbage collector
- Input
  - A root set of objects,
  - a heap,
  - a pointer *free* marking the start of free space.
- Output
  - The new value of pointer *free*
- Data Structures used
  - An *Unscanned* list
  - Reached bits in all objects
  - The table *NewLocation* to keep mappings of objects to their locations

3. Copying

```
      /* retarget references and move reached objects */
13)   for (each chunk of memory o in the heap, from the low end) {
14)       if (o is reached) {
15)           for (each reference o.r in o)
16)               o.r = NewLocation(o.r);
17)           copy o to NewLocation(o);
          }
      }
18)   for (each reference r in the root set)
19)       r = NewLocation(r);
```

Copying Garbage Collectors

- A copying collector reserves space to which the objects can move
- Breaks the dependency between tracing and finding free space
- The memory space is partitioned into two semispaces.
- The mutator allocates memory in one semispace, until it fills up
- At that point the mutator is stopped and the garbage collector copies the reachable objects to the other space.
- When garbage collection completes, the roles of the semispaces are reversed

Copying Garbage Collectors in Action

At the very beginning when any object is yet to be allocated

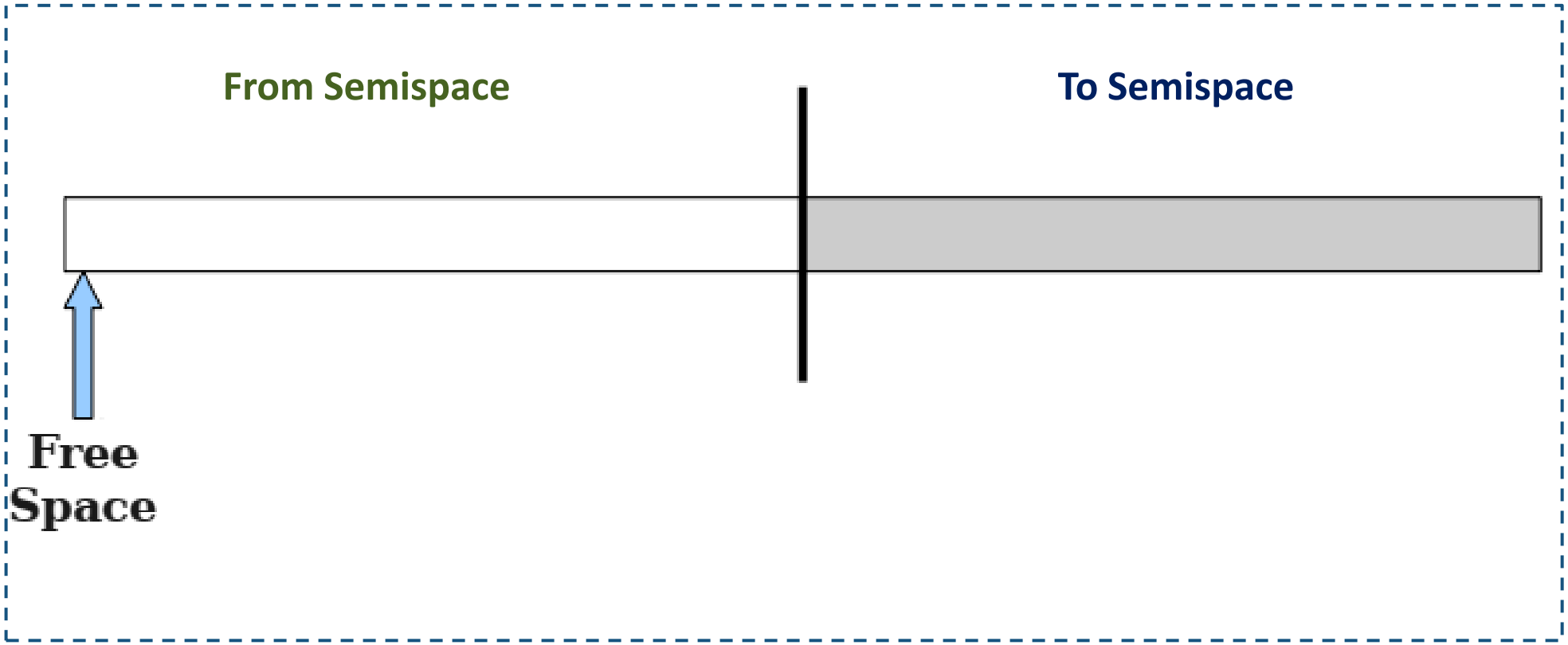


Image ref: https://web.stanford.edu/class/archive/cs/cs143/cs143.1128/lectures/18/Slides18.pdf

Copying Garbage Collectors in Action

From Semispace is full to a level that no more objects can be allocated.

Need to run Garbage Collector

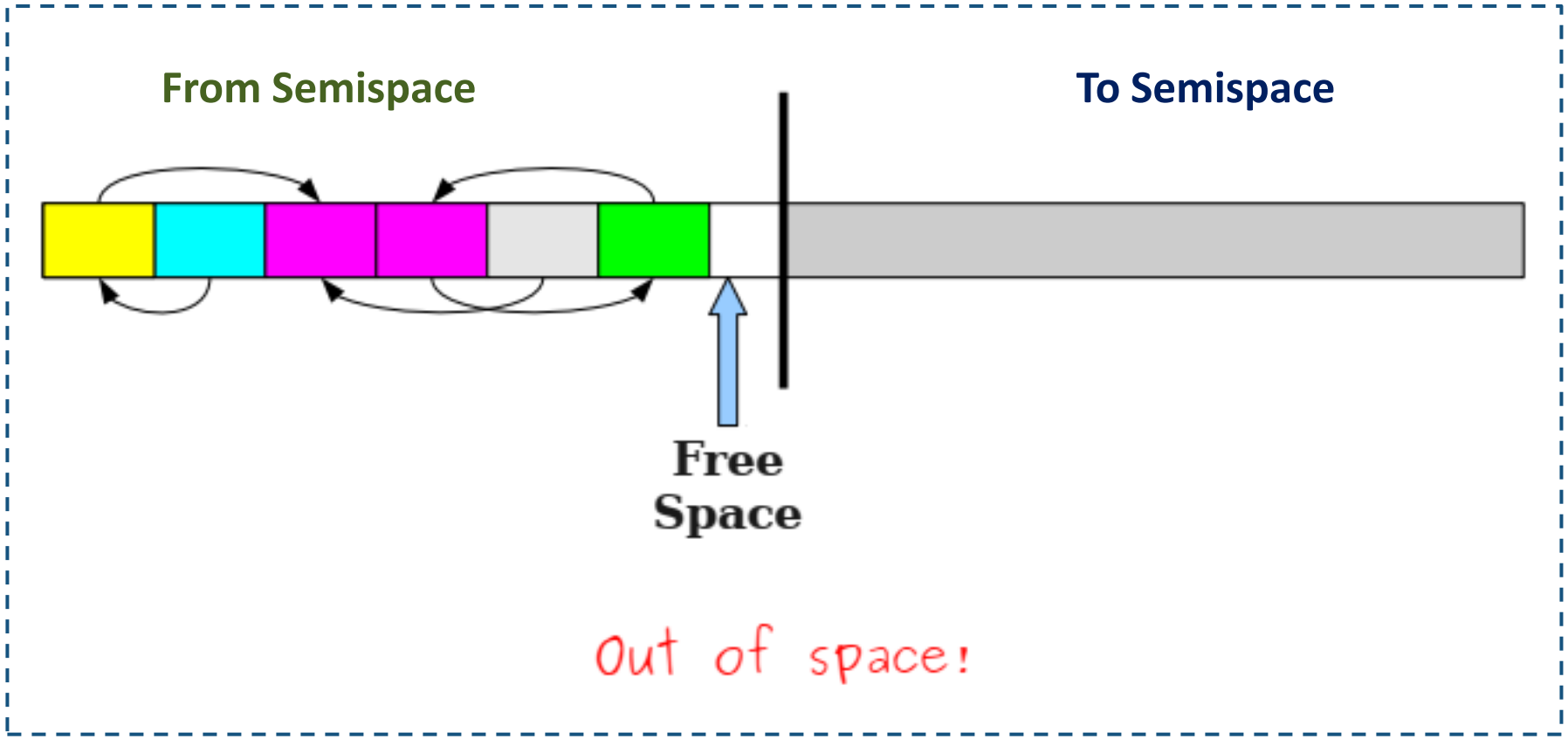


Image ref: https://web.stanford.edu/class/archive/cs/cs143/cs143.1128/lectures/18/Slides18.pdf

Copying Garbage Collectors in Action

Garbage Collector initiated. Start of mark phase

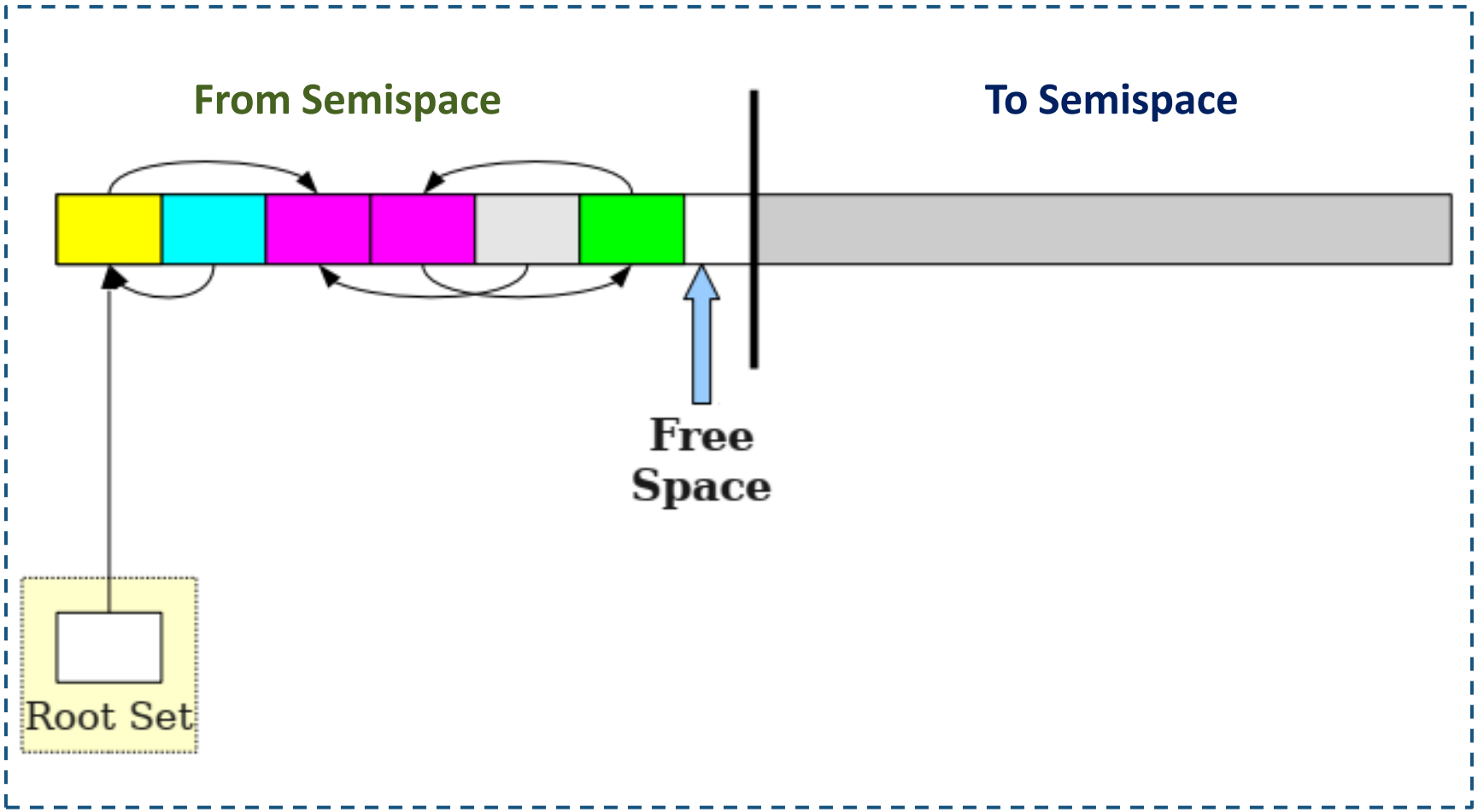


Image ref: https://web.stanford.edu/class/archive/cs/cs143/cs143.1128/lectures/18/Slides18.pdf

Copying Garbage Collectors in Action

GC running: reachable objects copied from 'From' semispace to 'To' semispace and all the relevant links are updated

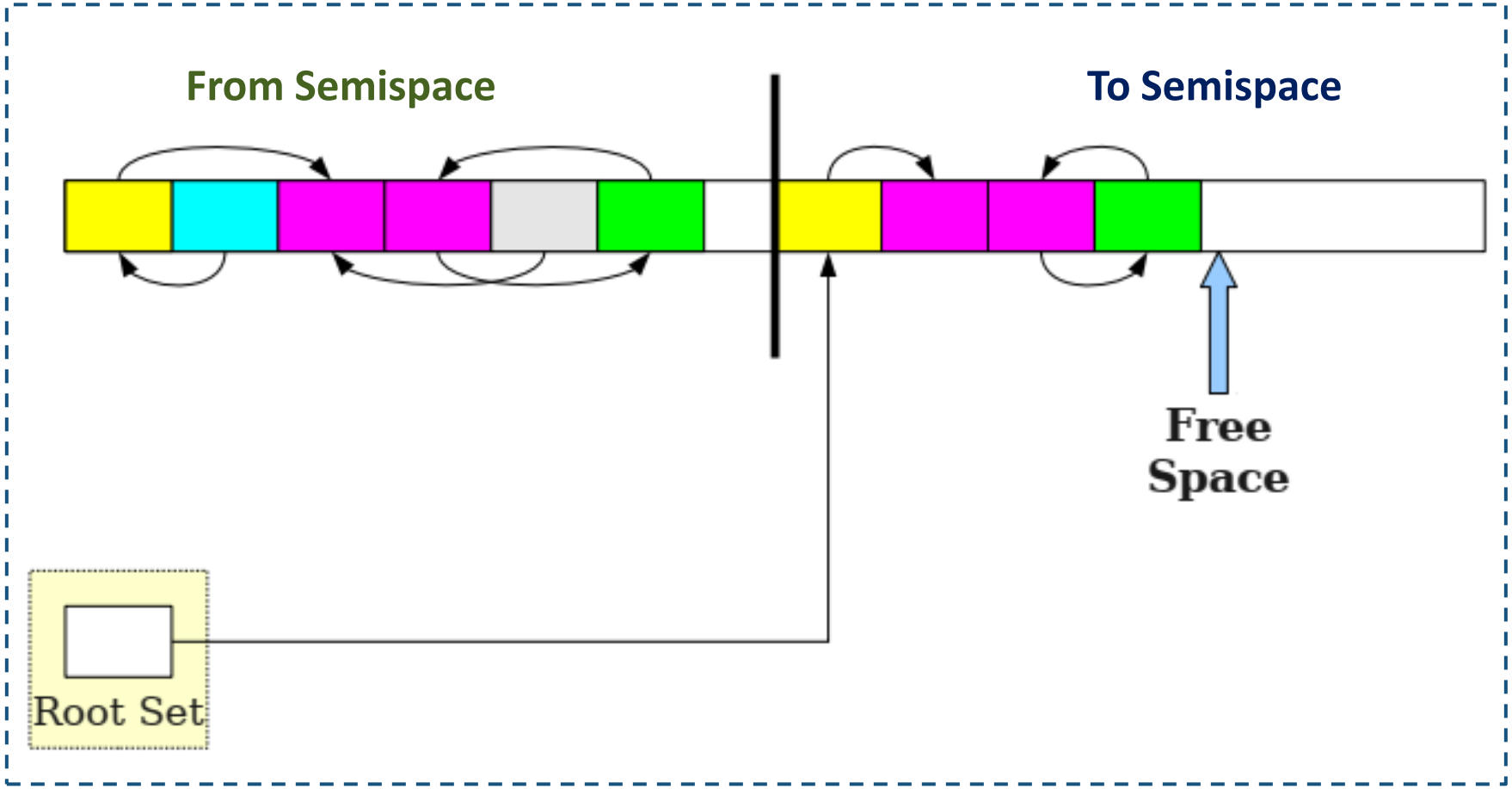


Image ref: https://web.stanford.edu/class/archive/cs/cs143/cs143.1128/lectures/18/Slides18.pdf

Copying Garbage Collectors in Action

One run of GC complete. Roles of 'From' and 'To' semispaces are reversed

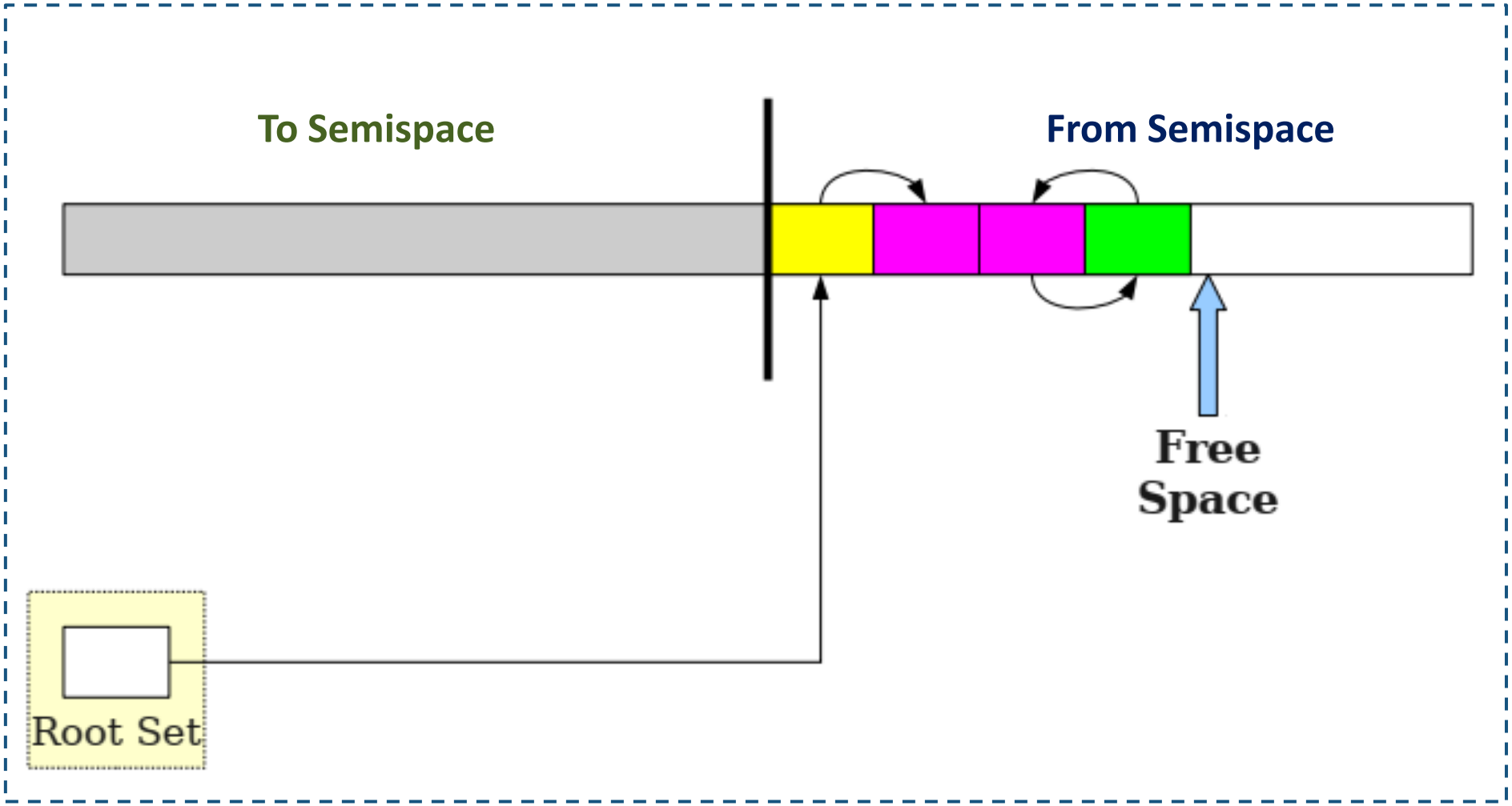


Image ref: https://web.stanford.edu/class/archive/cs/cs143/cs143.1128/lectures/18/Slides18.pdf

Cheney's Copying Collector

- Algorithm: Cheney's Copying Collector
- Input
  - A root set of objects,
  - a heap consisting of two semispaces
    - *From* semispace containing allocated objects
    - *To* semispace which is free
- Output
  - *To* semispace holds all the allocated objects
  - A free pointer pointing to the start of free space in the *To* semispace
  - *From* semispace is completely free

```
1)   CopyingCollector () {
2)         for (all objects o in From space) NewLocation(o) =NULL;
3)         unscanned = free = starting address of To space;
4)         for (each reference r in the root set)
5)               replace r with LookupNewLocation(r);
6)         while (unscanned ≠ free) {
7)               o = object at location unscanned;
8)               for (each reference o.r within o)
9)                     o.r = LookupNewLocation(o.r);
10)              unscanned = unscanned + sizeof(o);
           }
     }

     /* Look up the new location for object if it has been moved. */
     /* Place object in Unscanned state otherwise. */
11)  LookupNewLocation(o) {
12)        if (NewLocation(o) = NULL) {
13)              NewLocation(o) = free;
14)              free = free + sizeof(o);
15)              copy o to NewLocation(o);
           }
16)        return NewLocation(o);
     }
```

Cheney's Copying Collector

- Advantage
  - Does not touch any of the unreachable objects.
- Disadvantage
  - Moving the contents of all the reachable objects expensive especially for
    - large objects and
    - long-lived objects that survive multiple rounds of garbage collection.

Comparing Costs

- Basic Mark-and-Sweep: Proportional to the number of chunks in the heap.
- Baker's Mark-and-Sweep: Proportional to the number of reached objects.
- Basic Mark-and-Compact: Proportional to the number of chunks in the heap plus the total size of the reached objects
- Cheney's Copying Collector: Proportional to the total size of the reached objects

Reference

- Compilers: Principles, Techniques, & Tools, by
  Alfred B. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman

  - Chapter 7
    - 7.1
    - 7.2.1-7.2.3
    - 7.4
    - 7.5
    - 7.6

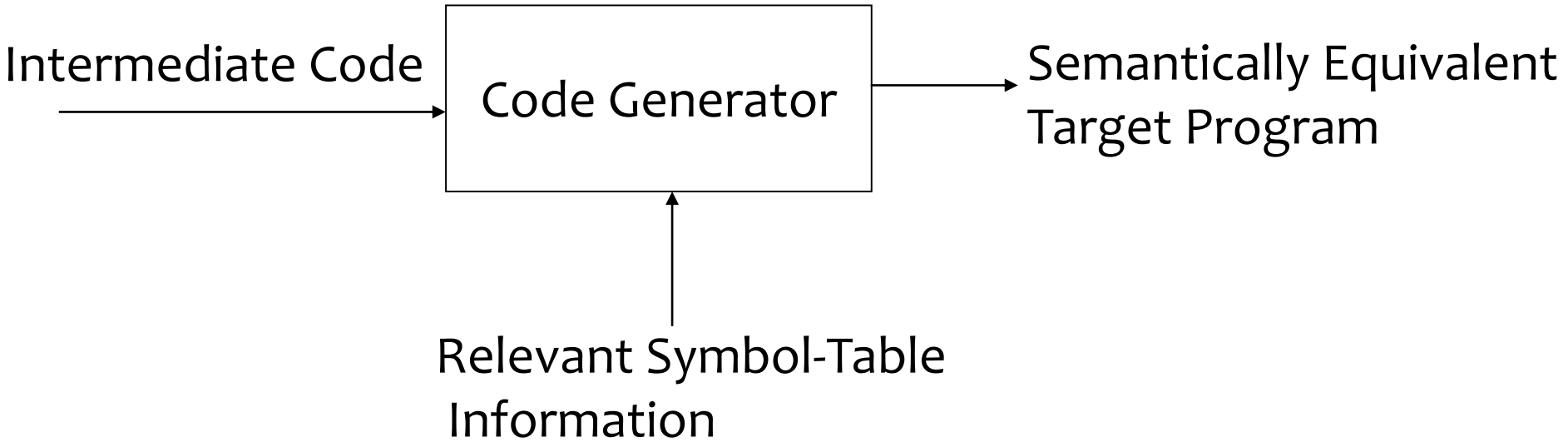
Thank You!

Chapter 8

Code Generation

Code Generation

- Final phase of our compiler model

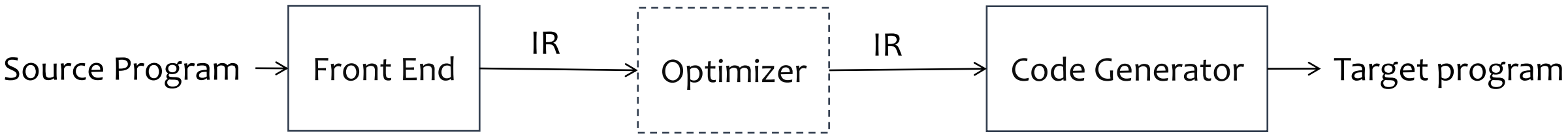


Important Issues of Code Generation

- Should produce correct target program
  - Preserving semantic meaning of source program
- Should produce high quality target program
  - Efficient use of available resources of target program
- Should run efficiently

Code Generation & Optimization

- To produce optimized target program, compiler may include an optimization phase prior to the generation
- Optimizer maps the IR into IR from which efficient code can be generated



Position of a Code Generator

Issue 1: Input to Code Generator

- The intermediate representation (IR) of the source program.

  ➢ Graphical representation
    - Syntax tree and DAG

  ➢ Three address representation
    - Quadruples
    - Triples
    - Indirect triples

  ➢ Virtual machine representation
    - Byte codes and stack-machine code

- We consider Three address representation.

Issue 1: Input to Code Generator

- Code generator assumes that
  - Front end has scanned, parsed, translated the source program into a relatively low level IR
  - All syntactic and static semantic errors have been detected
  - The necessary type checking has taken place
  - Type conversion operators have been inserted

Issue 2: Target Machine and Program

- Instruction set architecture of the target machine has significant impact on code generation
- The most common machine architectures are:
  - RISC
    - Many Registers
    - Three address instructions
    - Simple addressing modes
    - Relatively simple instruction – set architecture
  - CISC
    - Few registers
    - Two address instructions
    - Variety of addressing modes
    - Variable length instructions
  - Stack Based
    - For example, JVM

Issue 2: Target Machine and Program (2)

- Absolute machine code
  - It can be placed in a fixed location in memory
  - Can immediately be executed
- Relocatable machine – language program
  - Allows subprograms to be compiled separately
  - Linker : links a set of Relocatable object modules together
  - Loader: load them in memory
- Assembly language
  - Easier to generate using of macro and symbolic instruction
  - However, requires assembly step after code generation

Issue 3: Instruction Selection

- The code generator must map the IR program into a code sequence that can be executed by the machine
- The complexity of performing the mapping is determined by factors such as
  - the level of the IR
  - the nature of the ISA (instruction-set architecture)
  - the desired quality of the generated code
- From a set of instructions, we need to select instructions such that
  - The generated code is correct and efficient
- Instruction selection is easy if we don't care efficiency
  - For each three address statement, we can design a code skeleton that defines the target code for that construct

Issue 3: Instruction Selection (2)

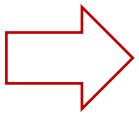
- Instruction selection for three address statement of form x = y + z can be:

```
LD  R0, y       // R0 = y        (load y into register R0)
ADD R0, R0, z   // R0 = R0 + z   (add z to R0)
ST  x, R0       // x = R0        (store R0 into x)
```

- But can produce redundant loads and stores

```
a = b + c
d = a + e
```

Three address statements



```
LD  R0, b       // R0 = b
ADD R0, R0, c   // R0 = R0 + c
ST  a, R0       // a = R0
LD  R0, a       // R0 = a
ADD R0, R0, e   // R0 = R0 + e
ST  d, R0       // d = R0
```

Translated Code

Issue 3: Instruction Selection (3)

- On most machines, a given IR can be translated in many different ways
  - Some may be efficient, some not
- Example statement: a = a + 1
  - If the target machine has an INC instruction, then we have two ways to implement for that machine

```
LD   R0, a        // R0 = a
ADD  R0, R0, #1   // R0 = R0 + 1
ST   a, R0        // a = R0
```

(i)

```
INC a
```

(ii)

Issue 4: Register Allocation

- Registers are the fastest computational unit but we do not have enough of them
- Values not held in register need to reside in memory
- Instruction involving register operands are shorter and faster
- Use of registers is subdivided into two subproblems –
  - Register allocation: select the set of variables that will reside in registers at each point in the program
  - Register assignment: we pick a register that a variable will reside in.
- Finding an optimal assignment of registers to variables is difficult
  - NP-complete

Issue 5: Evaluation Order

- Evaluation order can affect the efficiency of the target code
  - Some ordering requires fewer registers to hold intermediate results
- Finding the best order is difficult
  - NP-complete

A Simple Target Machine Model

- Target Language
  - Assembly code for a simple computer representative of many register machines
- Our target computer models –
  - ➢A three - address machine
    - load/store operations
    - computational operations
    - jump/conditional jump operations
  - ➢A byte addressable machine
    - Having n general purpose register
    - $R0, R1, \ldots, Rn-1$
  - ➢All operands are integer

A Simple Target Machine Model(2)

- Available Instructions:
  - ➢Load operations:
    - **LD $dst, addr$**
    - Loads the value in location $addr$ into location dst
    - It denotes the assignment $dst = addr$
    - Examples –
      - **LD r, x** // loads value in location x into register r
      - **LD r_1, r_2** // register – to – register copy
  - ➢Store operations:
    - **ST x, r**
    - Stores the value in register *r* into the location x
    - It denotes the assignment $x = r$

A Simple Target Machine Model

- Available Instructions:

  - Computational operations:
    - **OP** $\boldsymbol{dst}, \boldsymbol{src_1}, \boldsymbol{src_2}$
    - OP is like ADD, SUB
    - SUB r_1, r_2, r_3 denotes $r_1 = r_2 - r_3$

  - Unconditional jumps:
    - **BR** $\boldsymbol{L}$
    - Causes control to branch to the machine instruction with label L

A Simple Target Machine Model

- Available Instructions:
  - ➢ Conditional jumps:
    - **B*cond* r, L**
    - Here r is a register
    - ***cond*** stands for a common tests on value in register r
    - Example:
      - **BLTZ r, L**
      - Jump to lable L if the value of r is less than 0

Addressing Modes

- Addressing Modes: from which location you can find the value for an operand
  - ➢Absolute Mode
    - A location can be a variable name referring to the memory
    - **LD** $\boldsymbol{r_1, x}$
  - ➢Register mode
    - A location can be a register
    - **ST** $\boldsymbol{x, r_0}$

Addressing Modes

- Addressing Modes: from which location you can find the value for an operand

  - Indexed Mode
    - A location can be an indexed address
    - $\boldsymbol{a(r)}$: a is a variable and r is a register.
      - Location is computed by adding the l – value of a to the value in register in r.
      - $\mathbf{LD\ R1, a(R2)}$
      - $\mathrm{R1} = contents\ (\mathrm{a} + \mathrm{contents}\ (\mathrm{R2}))$
    - $\boldsymbol{c(r)}$: here c is a constant and r is a register
      - $\mathbf{LD\ R1, 100(R2)}$
      - $\mathrm{R1} = contents(100 + contents(\mathrm{R2}))$

Addressing Modes

- Addressing Modes: from which location you can find the value for an operand
  - Indirect modes:
    - $*r$
      - $\mathrm{LD\ R1, *R2}$
        - $\mathrm{R1} = contents\ (contents\ (\mathrm{R2}))$
    - $*a(r)$
      - $\mathrm{LD\ R0, *x(R1)}$
        - $\mathrm{R0} = contents\ (contents\ (\mathrm{x} + contents(\mathrm{R1})))$
    - $*c(r)$
      - $\mathrm{LD\ R0, *100(R1)}$
        - $\mathrm{R0} = contents\ (contents\ (100 + contents\ (\mathrm{R1})))$

Addressing Modes

- Addressing Modes: from which location you can find the value for an operand

  - Immediate Constant Addressing Modes:

    - Allow source to be a constant
    - Constant prefixed by #
    - LD R1, #100
      - R1 = 100
    - ADD R1, R1, #100
      - R1 = R1 + 100

Statement-wise Code Generation

IR (TAC)	Target Code
`x = y - z`	`LD R1, y` // R1 = y `LD R2, z` // R2 = z `SUB R1, R1, R2` // R1 = R1 - R2 `ST x, R1` // x = R1
`b = a[i]`	`LD R1, i` // R1 = i `MUL R1, R1, 8` // R1 = R1 * 8 `LD R2, a(R1)` // R2 = contents(a + contents(R1)) `ST b, R2` // b = R2
`a[j] = c`	`LD R1, c` // R1 = c `LD R2, j` // R2 = j `MUL R2, R2, 8` // R2 = R2 * 8 `ST a(R2), R1` // contents(a + contents(R2)) = R1

Statement-wise Code Generation

IR (TAC)	Target Code
`x = *p`	`LD R1, p` // R1 = p `LD R2, 0(R1)` // R2 = contents(0 + contents(R1)) `ST x, R2` // x = R2
`*p = y`	`LD R1, p` // R1 = p `LD R2, y` // R2 = y `ST 0(R1), R2` // contents(0 + contents(R1)) = R2
`if x < y goto L`	`LD R1, x` // R1 = x `LD R2, y` // R2 = y `SUB R1, R1, R2` // R1 = R1 - R2 `BLTZ R1, M` // if R1 < 0 jump to M

Program Cost

- A cost is often associated with compiling and running program
- Common cost measure may be-
  - Compilation time
  - Size of the target program
  - Running-time of the target program
  - Power consumption of the target program
- We assume, the cost of a target-language program on a given input is the sum of the costs of the individual instructions executed when the program is run on that input.

Instruction Cost

- Simple model for instruction cost
  - Cost of an instruction = One plus cost for the addressing modes.

- Addressing modes –
  - involving registers have zero additional cost
  - involving a memory location / constant have an additional cost
    - Since such operands have to be stored in the words following the instruction

Instruction Cost

- Examples:
  1. **LD R0, R1**
     - Meaning: $\mathrm{R0} = contents(\mathrm{R1})$
     - Cost = $1 + 0 = 1$
  2. **LD R0, M**
     - Meaning: $\mathrm{R0} = contents(\mathrm{M})$
     - Cost = $1 + 1 = 1$
  3. **LD R1, *100(R2)**
     - Meaning: $\mathrm{R1} = contents(contents(100 + contents(\mathrm{R2})))$
     - Cost = $1 + 2 = 3$
  4. **ADD R1, #3, #4**
     - Meaning: $\mathrm{R1} = 3 + 4$
     - Cost = $1 + 2 = 3$

Basic Blocks & Flow Graphs

- *Flow graph* is a graphical representation of Intermediate code (IR)
  - Helps in Code Generation
  - Better job in Register Allocation
  - Better job in Instruction Selection

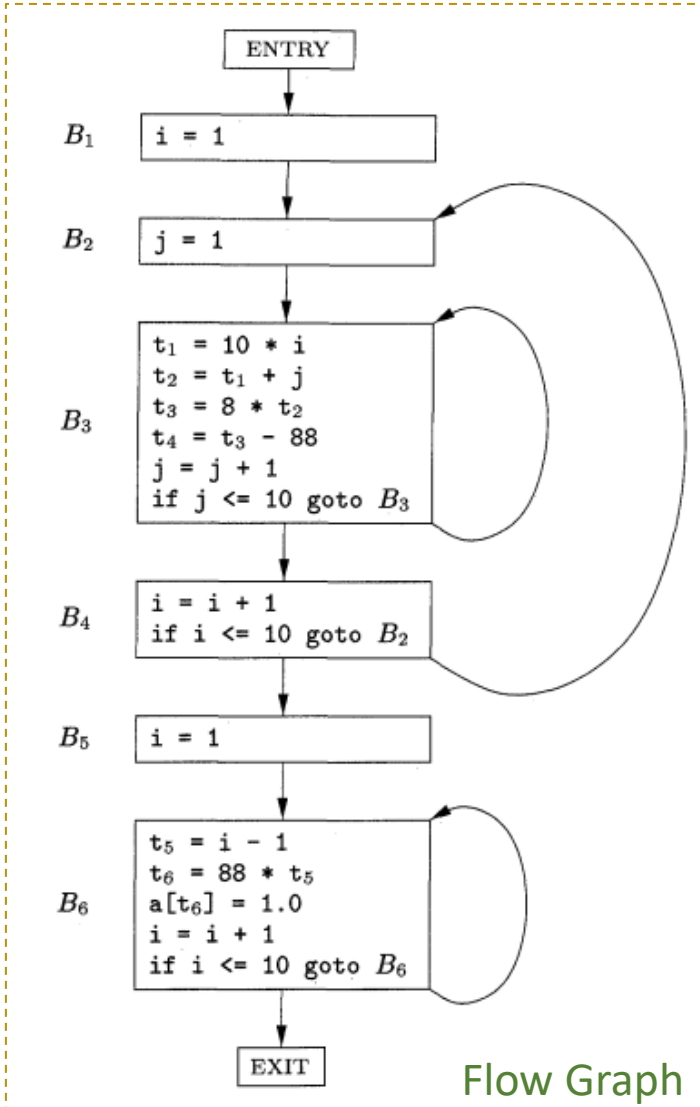
Basic Blocks & Flow Graphs

```
for i from 1 to 10 do
        for j from 1 to 10 do
                a[i, j] = 0.0;
for i from 1 to 10 do
        a[i, i] = 1.0;
```

Source Program

```
 1)   i = 1
 2)   j = 1
 3)   t1 = 10 * i
 4)   t2 = t1 + j
 5)   t3 = 8 * t2
 6)   t4 = t3 - 88
 7)   a[t4] = 0.0
 8)   j = j + 1
 9)   if j <= 10 goto (3)
10)   i = i + 1
11)   if i <= 10 goto (2)
12)   i = 1
13)   t5 = i - 1
14)   t6 = 88 * t5
15)   a[t6] = 1.0
16)   i = i + 1
17)   if i <= 10 goto (13)
```

Three Address Code



Flow Graph

Constructing Flow Graphs

- Partition the intermediate code into *basic blocks*, which are maximal sequences of consecutive Three-Address instructions with the properties –
  - The flow of control can only enter the basic block through the first instruction in the block
    - There are no jumps into the middle of the bock
  - Control will leave the block without halting or branching
    - Except possibly at the last instruction in the block
- The basic blocks become the nodes of a *flow graph*
  - Edges indicate which block can follow which other blocks

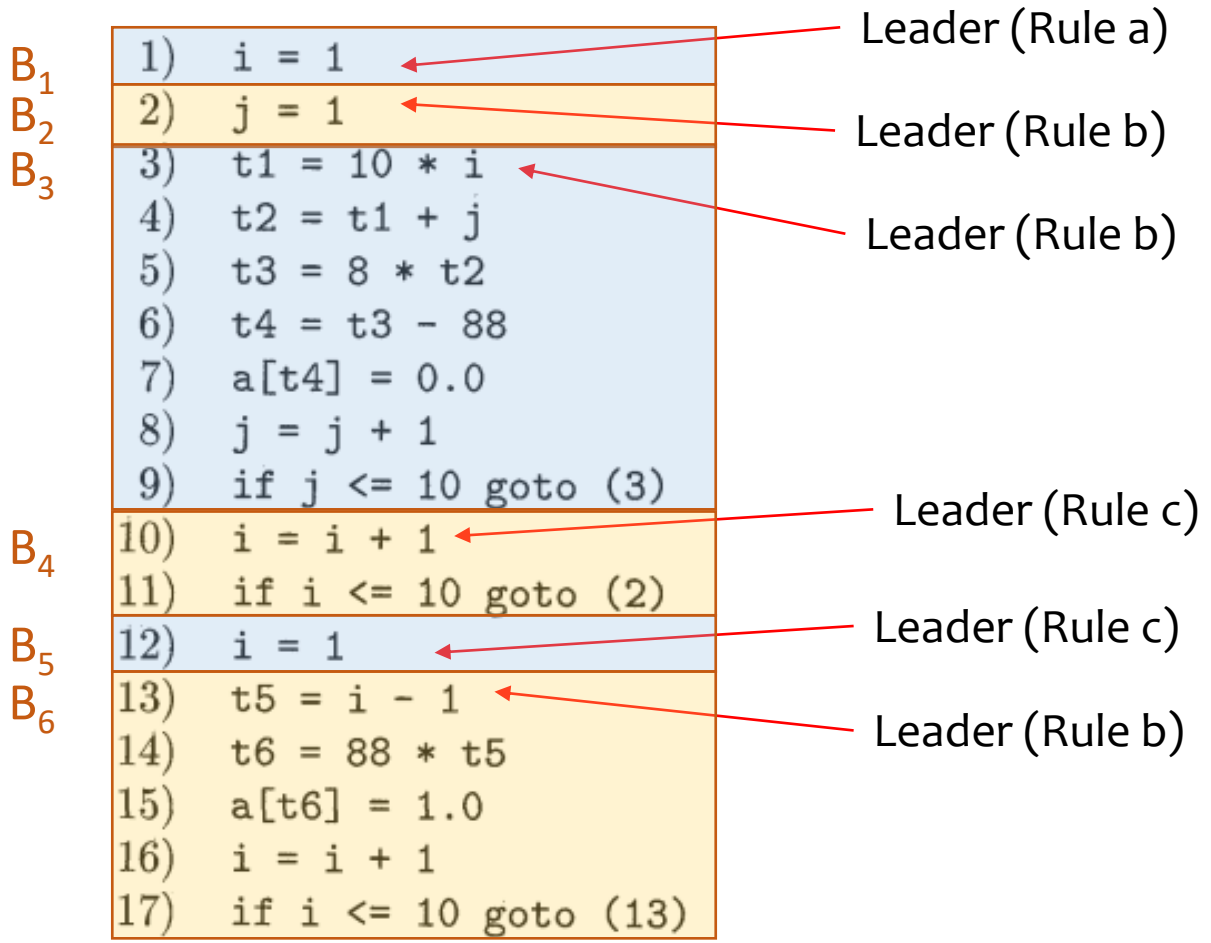
Constructing Basic Blocks

- **Algorithm:** Partitioning Three-Address Code into Basic Blocks
- **Input:** A sequence of Three-Address Code
- **Output:** A list of basic blocks where each instruction is assigned to exactly one block
- **Method:**
  1. Determine those instructions in the intermediate code that are the first instructions in some basic blocks – leaders
     a. The first instruction in a sequence is a leader
     b. Any instruction that is the target of a branch is a leader
     c. Any instruction that follows a branch is a leader
  2. For each leader, basic block consists codes upto next leader or end

Constructing Basic Blocks

Method:

1. Determine those instructions in the intermediate code that are the first instructions in some basic blocks – leaders
   a. The first instruction in a sequence is a leader
   b. Any instruction that is the target of a branch is a leader
   c. Any instruction that follows a branch is a leader
2. For each leader, basic block consists codes upto next leader or end

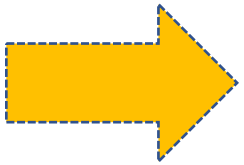


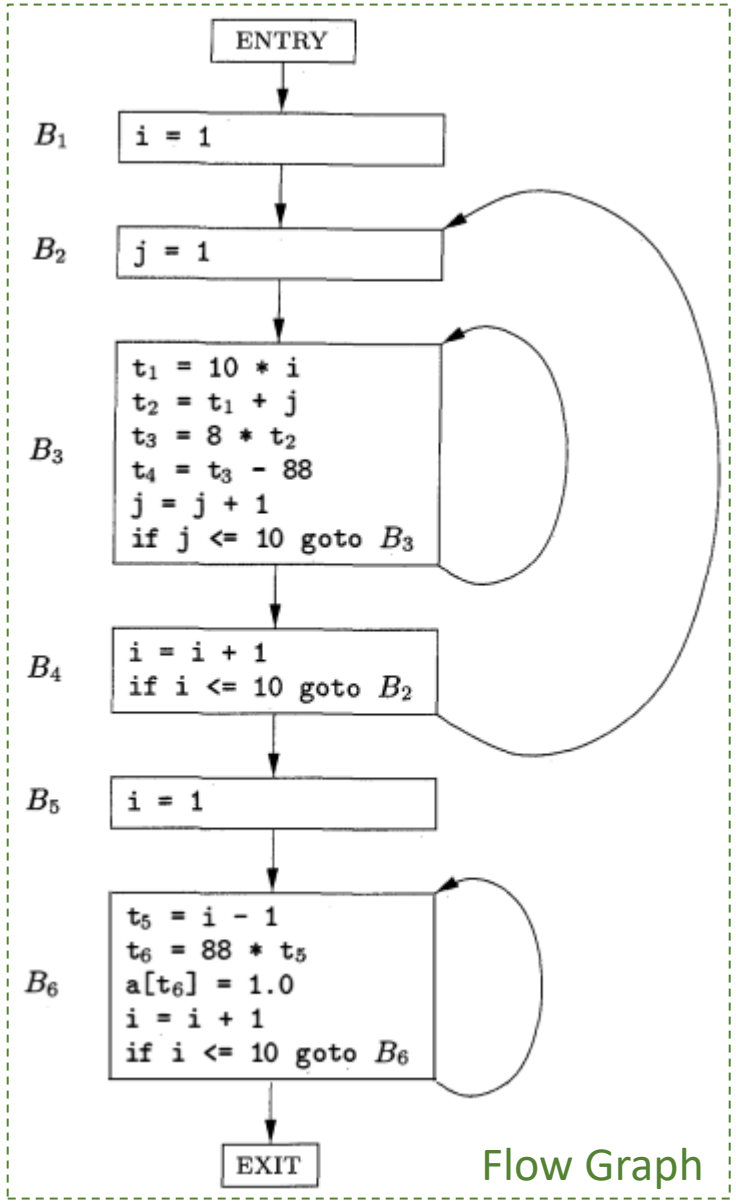
Flow Graphs

- There is an edge from a block B to block C *iff* –
  - It is possible for the first instruction in block C to immediately follow the last instruction in block B
- There are two ways to construct such an edge -
  - There is a conditional or unconditional jump from the end of B to the beginning of C
  - C immediately follows B in the original order of the three – address instructions and B does not end an unconditional jump
- B is a predecessor of C and C is a successor of B

Flow Graphs

B_1	1)	i = 1
B_2	2)	j = 1
B_3	3)	t1 = 10 * i
	4)	t2 = t1 + j
	5)	t3 = 8 * t2
	6)	t4 = t3 - 88
	7)	a[t4] = 0.0
	8)	j = j + 1
	9)	if j <= 10 goto (3)
B_4	10)	i = i + 1
	11)	if i <= 10 goto (2)
B_5	12)	i = 1
B_6	13)	t5 = i - 1
	14)	t6 = 88 * t5
	15)	a[t6] = 1.0
	16)	i = i + 1
	17)	if i <= 10 goto (13)





Flow Graph

Representation of Flow Graphs

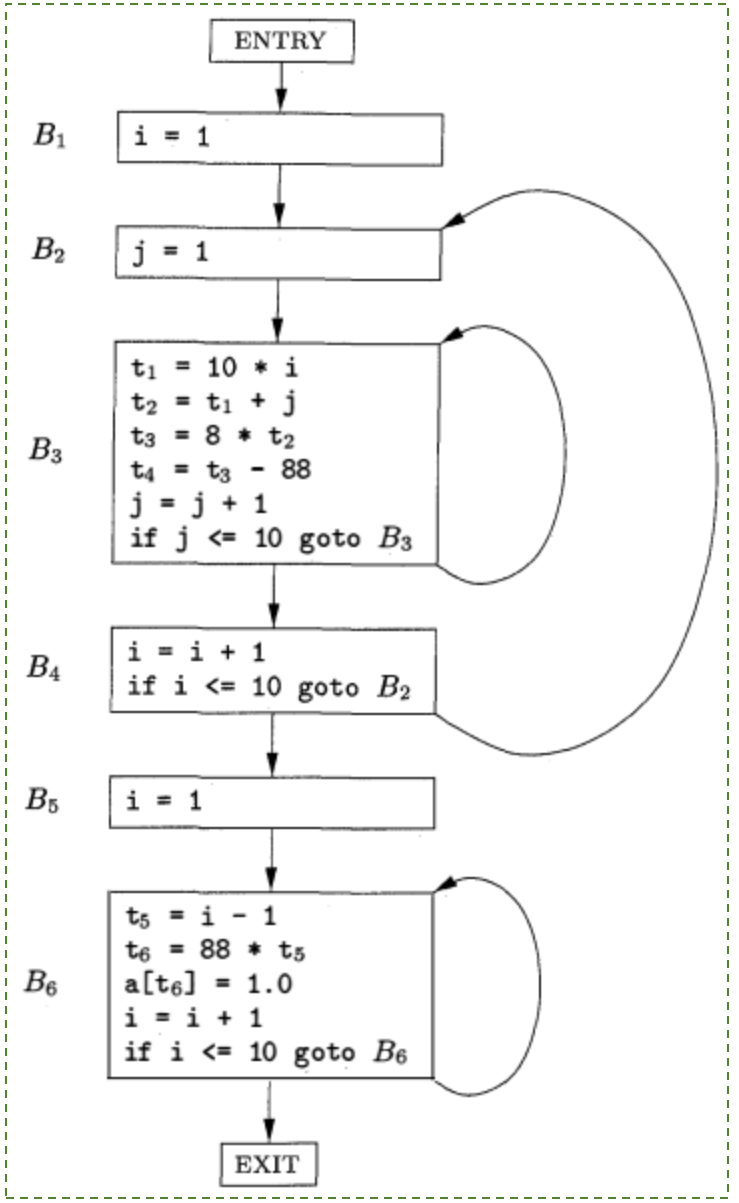
- Jumps to instruction numbers or labels are replaced by jumps to basic blocks
  - Every conditional or unconditional jump is to the leader of some basic block, it is to this block this jump will now refer
- After constructing the flow graph it is common to make substantial changes to the instructions in the various basic blocks
  - If jumps were to instructions, it would cause to fix the targets with each change
- Flow graphs can be represented by any of the data structures used for graph
- The content of nodes can be represented by
  - pointer to the leader in the array of three address instructions together with count of the number of instructions or a second pointer to the last instruction.
  - It is better to maintain a linked list as the number of instructions in a basic block are changed frequently.

Loops

- Programming-language constructs like while-statements and for-statements naturally gives rise to loops in programs.
- Virtually every program spends most of its time in executing its loops
  - It is important for a compiler to generate good code for loops
- Many code transformations depend upon the identification of “loops” in a flow graph

Loops

- A set of nodes *L* in the flow graph is a loop if *L* contains a node *e* called the loop entry, such that
  1. *e* is not ENTRY, the entry of the entire flow graph
  2. No node in *L* besides *e* has a predecessor outside *L*, *i.e.*, every path from ENTRY to any node in *L* goes through *e*
  3. Every node in *L* has a nonempty path, completely withing *L*, to *e*
- Example: Loops in the flow graph on the right are -
  1. B_3 by itself
  2. B_6 by itself
  3. $\{B_2, B_3, B_4\}$



Liveliness & Next-Use Information

- Knowledge of next *use* of a variable is helpful in good code generation
- If the value of a variable that is currently in a register will never be referenced subsequently, then that register can be assigned to another variable
- Definition of *use*

  $i : x = 5$ // x is live at statement i

  ..... // no use of x

  $j : y = x$ // statement j uses the value of x $computed\ at\ i$

- For each statement $x = y + z$, we want to find next use of x, y and z

Liveliness & Next-Use Information

- **Algorithm:** Determining liveliness & next-use information for each statement of a basic block
- **Input:** A basic block *B* of three-address statements. It is assumed that the symbol table initially shows all nontemporary variables in *B* as being live on exit
- **Output:** At each statement i: $x = y + z$, liveliness and next-use information of x, y and z will be attached with i
- **Method:** From last statement of B, scan backwards
  1. Attach current liveliness & Next-use of x, y and z found in symbol table to statement i
  2. In the symbol table, set x to not live and no next use
  3. In the symbol table, set y, z to live and next use at statement i

Liveliness & Next-Use Information

From the last statement of *B*, scan backwards. At each statement i: $x=y+z$ -

1. Attach current liveliness & Next-use of x, y and z found in symbol table to statement i
2. In the symbol table, set x to not live and no next use
3. In the symbol table, set y, z to live and next use at statement i

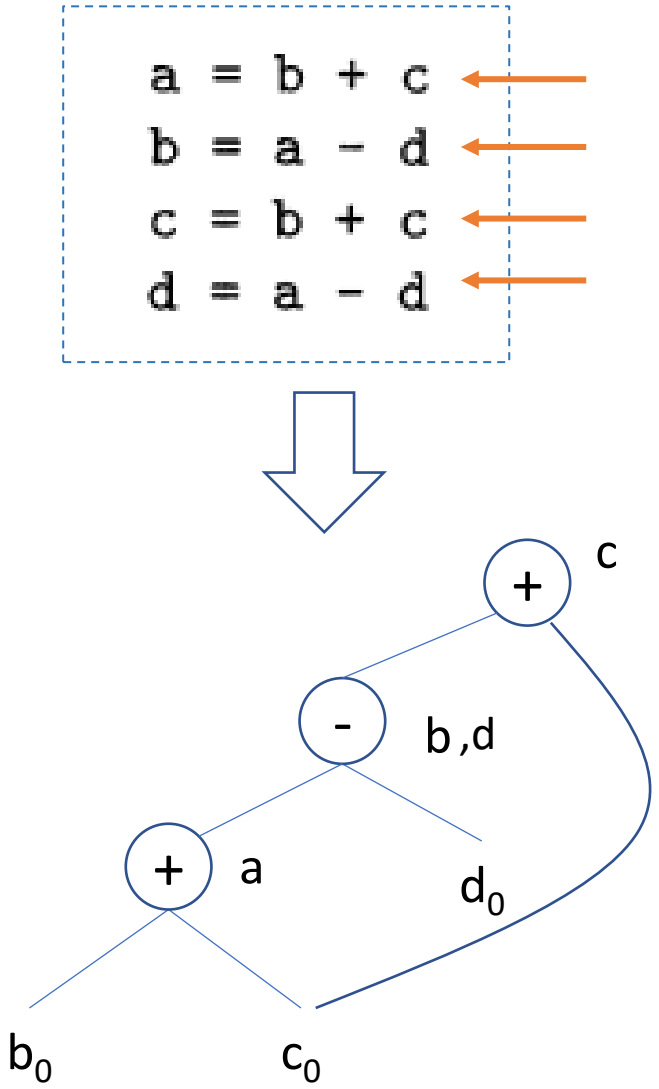
Basic Block	Symbol Table						
	a	b	c	d	t	u	v
	L1	L1	L2	L4	N	N	N
1. t (L3) = a (L2) - b (L6)	L2	L6	L2	L4	L3	N	N
2. u (L3) = a (N) - c (L6)	N	L6	L6	L4	L3	L3	N
3. v (L5) = t (N) + u (L5)	N	L6	L6	L4	N	L5	L5
4. a (L6) = d (N)	L6	L6	L6	N	N	L5	L5
5. d (L6) = v (N) + u (N)	L6	L6	L6	L6	N	N	N

Optimization of Basic Blocks

- Optimization within each basic block is also called local optimization
- Some code improving transformations are -
  - Common Subexpression Elimination
  - Dead Code Elimination
  - Reordering of statements to reduce the time of a temporary variable in a register
  - Simplify Algebraic Computation

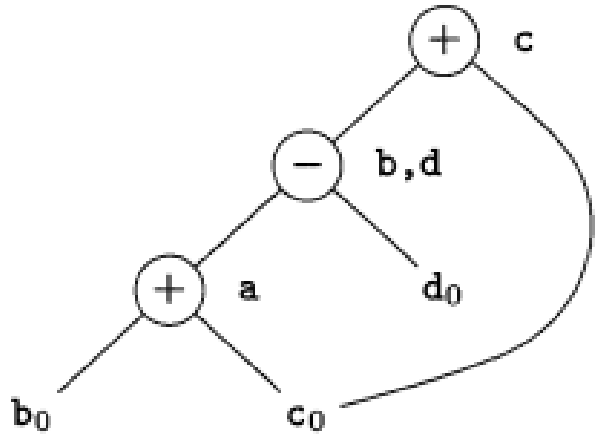
DAG representation of Basic Blocks

- Many important techniques for local optimization begin by transforming a basic block into a DAG
- We can construct DAG for a basic block as follows
  - There is a node in the DAG for each of the initial values of the variables appearing in the basic block
  - There is a node *N* associated with each statement *s* within the block. The children of N are those nodes corresponding to statements that are the last definitions, prior to *s*, of the operands used by *s*
  - Node *N* is labeled by the operator applied at *s*, and also attached to *N* is the list of variables for which it is the last definition within the block
  - Certain nodes are designated *output nodes*. These are the nodes whose variables that are *live on exit* from the block



Eliminate Local Common Subexpressions

```
a = b + c
b = a - d
c = b + c
d = a - d
```



b is not live on exit

```
a = b + c
d = a - d
c = d + c
```

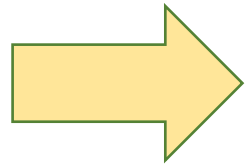
b is live on exit

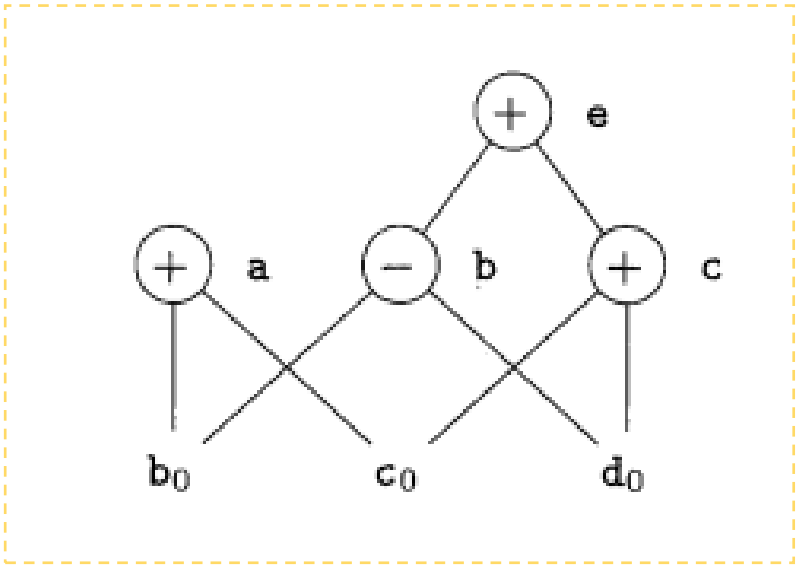
```
a = b + c
d = a - d
b = d
c = d + c
```

Finding Local Common Subexpressions

- Another Example:

```
a = b + c;
b = b - d
c = c + d
e = b + c
```



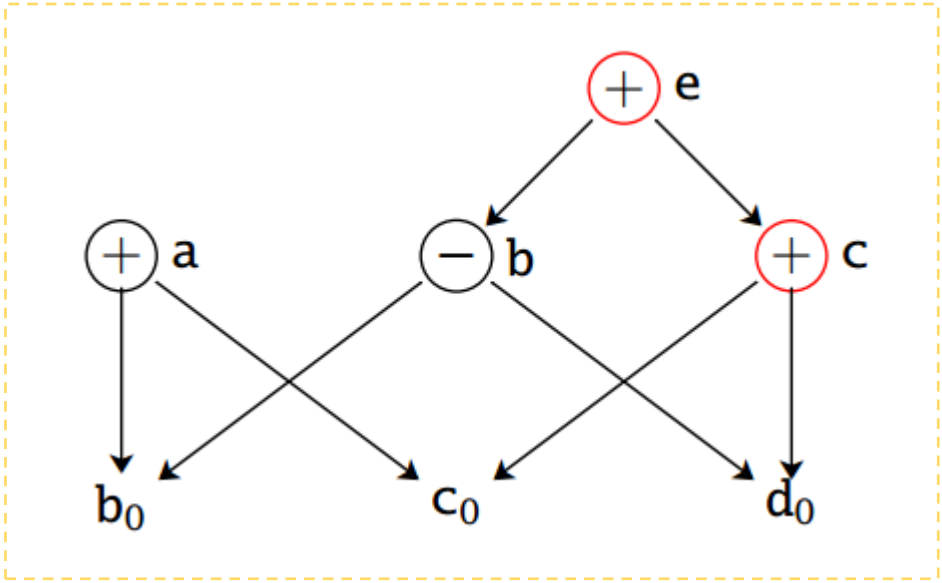


- What will happen in this case?
  - DAG method will fail to recognize detect common subexpression of 1st and 4th instruction

Dead Code Elimination

- Eliminate instructions that compute values that are never used
- Delete the nodes with no ancestor and no attached live variables from the DAG

```
a = b + c
b = b - c
c = c + d
e = b + c
```



Assuming e and c are not live on exit

```
a = b + c
b = b - c
```

Use of Algebraic Identities

- Apply algebraic identities to eliminate computation

$$x + 0 = 0 + x = x \qquad x - 0 = x$$
$$x \times 1 = 1 \times x = x \qquad x/1 = x$$

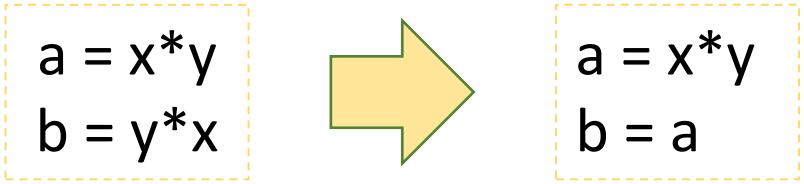
- Local reduction in strength

EXPENSIVE		CHEAPER
x^2	=	$x \times x$
$2 \times x$	=	$x + x$
$x/2$	=	$x \times 0.5$

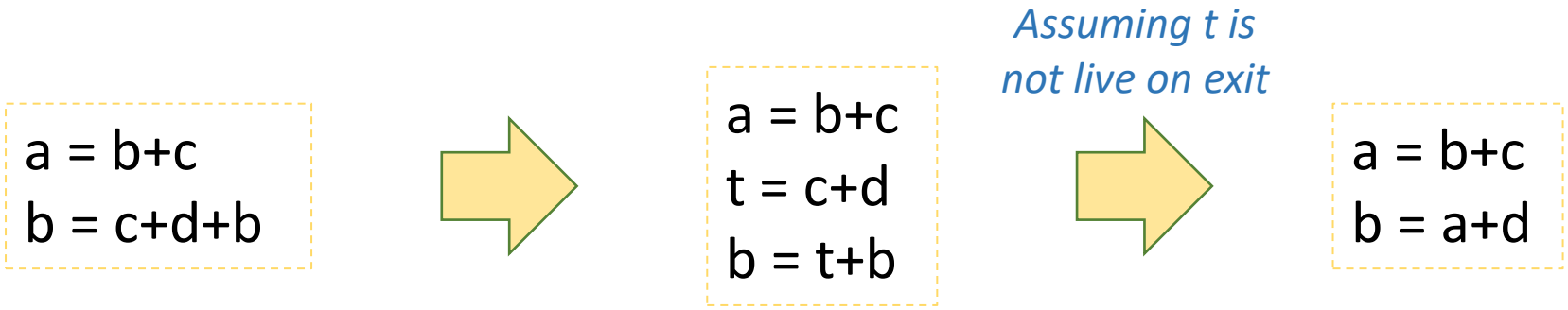
- Constant Folding by evaluating constant expressions at compile time
  - Example: replace 2*3.14 by 6.28

Use of Algebraic Identities

- Commutativity with local common subexpression



- Associativity with composite expression



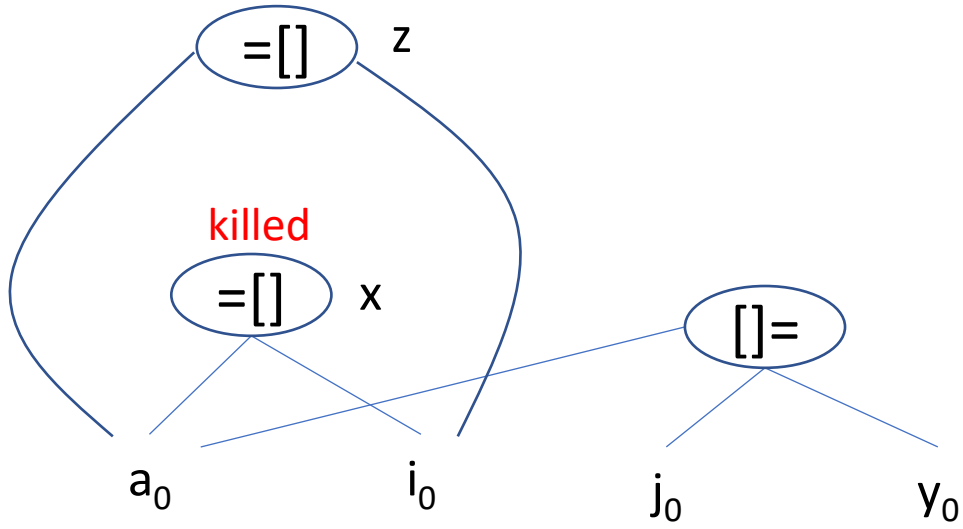
Representation of Array References

1. An assignment from an array, like `x = a[i]`, is represented by creating a node with operator =[] and two children representing the initial value of the array, $\mathtt{a}_0$ in this case, and the index `i`. Variable `x` becomes a label of this new node.

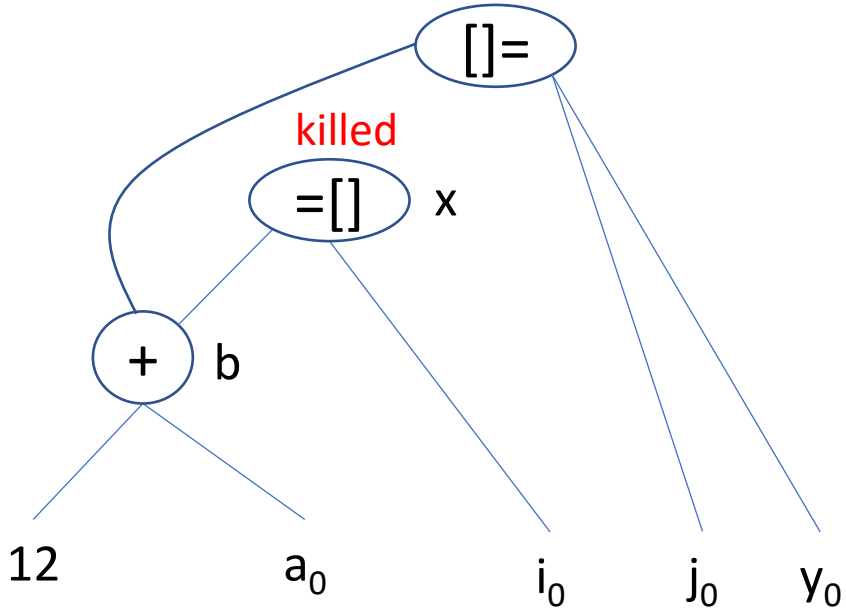
2. An assignment to an array, like `a[j] = y`, is represented by a new node with operator []= and three children representing $\mathtt{a}_0$, `j` and `y`. There is no variable labeling this node. What is different is that the creation of this node *kills* all currently constructed nodes whose value depends on $\mathtt{a}_0$. A node that has been killed cannot receive any more labels; that is, it cannot become a common subexpression.

Representation of Array References

```
x = a[i]
a[j] = y
z = a[i]
```




```
b = 12 + a
x = b[i]
b[j] = y
```



Reassembling Basic Blocks From DAGs

- After performing optimizations on DAG, we need to reconstitute the three-address code for the basic block from which the DAG was built
- For each node, that has one or more attached variables, a three-address statement is constructed that computes the value of one of those variables
  - Select the variable that is live on exit from the block
  - However, if global-live variable information is not available, all variables except the temporaries are to be computed
- If a node has more than one live variable attached, then introduce copy statement to give the correct values to each of these variables.

Reassembling Basic Blocks From DAGs(2)

- While reconstructing a basic block from the DAG, the order of instructions should also be correctly maintained conforming to the following rules -
  - A node's values cannot be computed until the value for each of its children has been computed
  - Assignments to an array must follow all previous assignments to, or evaluations from, the same array, according to the order of these instructions in the original basic block
  - Evaluations of array elements must follow any previous (according to the original block) assignments to the same array. The only permutation allowed is that two evaluations from the same array may be done in either order, as long as neither crosses over an assignment to that array
  - Any use of variable must follow all previous (according to the original block) procedure calls or indirect assignments through a pointer
  - Any procedure call or indirect assignments through a pointer must follow all previous (according to the original block) evaluation of any variable

A Simple Code-Generator

- Generates code for a single basic block
- Assumes -
  - the basic block has already been optimized
  - some set of registers are available to hold values within blocks
  - there is exactly one machine instruction for each operator that accepts the operands from registers and stores the result in a registers
- Consider each three-address instruction in turn
- Keep track of what values are in what registers
  - Avoid generating unnecessary loads and stores

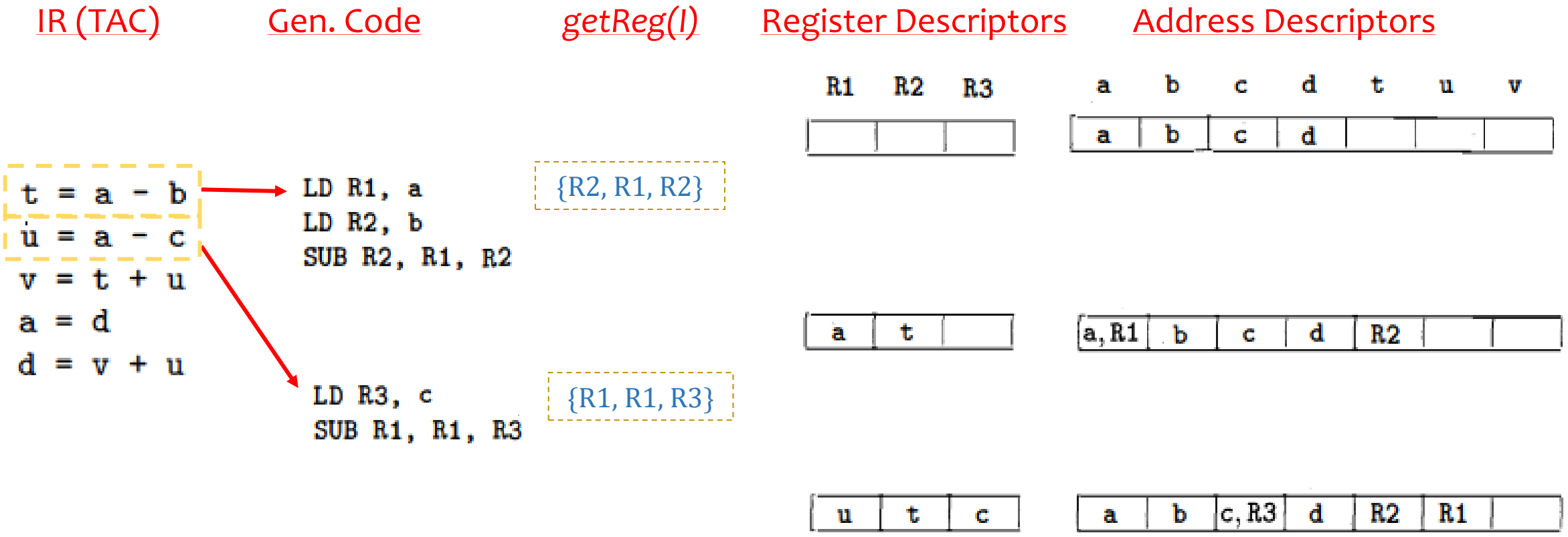
Uses of Registers

- One of the primary issues during code generation is deciding how to use registers to best advantage
- There are four principal uses of registers
  - In most machine architectures, some or all of the operands of an operation must be in registers
  - Registers make good temporaries –
    - Hold the result of a subexpression while evaluating a larger expression
    - Hold a variable that is used only within a single basic block
  - Registers are used to hold (*global*) values that are computed in one basic block and used in other blocks, for example a loop index
  - Registers are often used to help with run-time storage management, for example maintaining stack pointers and top of the stack

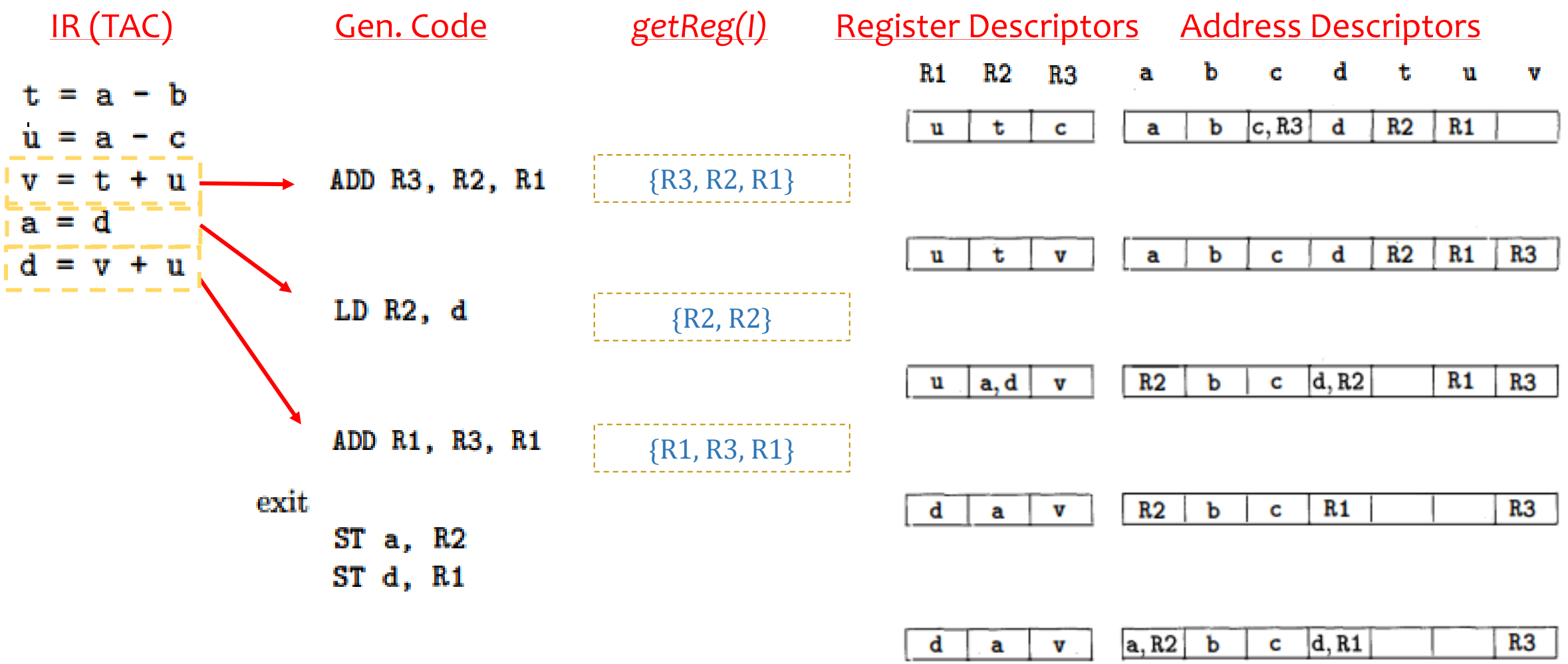
Register and Address Descriptors

- Register Descriptor
  - One for each register
  - Keeps track of the variable names whose current value is in the register
  - Initially all are set to empty
- Address Descriptor
  - One for each program variable
  - Keeps track of the location or locations where the current value of that variable can be found
    - Locations can be a register, a memory address, a stack location
    - Can be a set of more than one of the above
    - Can be stored in the symbol table entry of the variable

An Example of Code-Generation



An Example of Code-Generation



The Code-Generation Algorithm

- Machine instructions for operations

For a three-address instruction such as $x = y + z$, do the following:

1. Use $getReg(x = y + z)$ to select registers for x, y, and z. Call these R_x, R_y, and R_z.

2. If y is not in R_y (according to the register descriptor for R_y), then issue an instruction `LD` R_y, y', where y' is one of the memory locations for y (according to the address descriptor for y).

3. Similarly, if z is not in R_z, issue and instruction `LD` R_z, z', where z' is a location for z.

4. Issue the instruction `ADD` R_x, R_y, R_z.

The Code-Generation Algorithm (2)

- Machine instructions for Copy Statements (of the form *x=y)*
  - Assumes `getReg(I)` will always generate the same register for *x* and *y*
  - If y is not already in *Ry*, then generate instruction LD *Ry, y*
  - Else do nothing

- Ending the basic block
  - Variables used in a block may wind up with their only location being a register
  - For temporary variables within a block, do nothing
  - For each variable *x* that is live on exit from the block, generate instruction
    ST x, R where R is the register that stores x's value

Managing Register and Address Descriptors

1. For the instruction LD R, x

   (a) Change the register descriptor for register R so it holds only x.

   (b) Change the address descriptor for x by adding register R as an additional location.

2. For the instruction ST x, R, change the address descriptor for x to include its own memory location.

Managing Register and Address Descriptors (2)

3. For an operation such as `ADD` R_x, R_y, R_z implementing a three-address instruction $x = y + z$

   (a) Change the register descriptor for R_x so that it holds only x.

   (b) Change the address descriptor for x so that its only location is R_x. Note that the memory location for x is *not* now in the address descriptor for x.

   (c) Remove R_x from the address descriptor of any variable other than x.

4. When we process a copy statement $x = y$, after generating the load for y into register R_y, if needed, and after managing descriptors as for all load statements (per rule 1):

   (a) Add x to the register descriptor for R_y.

   (b) Change the address descriptor for x so that its only location is R_y.

Design of Function getReg(*I*)

- For variable y (and z as well) in the expression `x=y+z`
  - If y is currently in a register, pick that register as R_y
  - Else if y is not in a register and there are empty registers, pick an empty register as R_y
  - Else if y is not in a register and there are no empty registers, then
    - for each register R
      - For each variable v that are in R
        - If v is somewhere besides R (check from address descriptor), pick R
        - Else if v is x, and x is not one of the other operands (z), pick R
        - Else if v is not used later, pick R
        - Else generate a store instruction ST v, R; called a *spill*
      - Since R may hold several variables at the moment, calculate R's score as total number of spills required for all its variables
    - pick the register R with lowest score as R_y

Design of Function getReg(*I*)

- For variable *x* in the expression *x=y+z*

  - The issues and options are almost as for y, except for the following differences

  - A register that holds only *x* is always an acceptable choice for R_x
    - holds true even if *x* is one of *y* and *z*

  - If *y* is not used after instruction *I* and R_y holds only *y* after being loaded, if necessary then pick R_y as R_x
    - similar option holds regarding *z* and R_z

  - If *I* is a copy instruction (*x*=*y*), always choose R_y as R_x

Global Register Allocation

(See the pdf titled Global Register Allocation)

Related References

- https://web.eecs.umich.edu/~mahlke/courses/583f12/reading/chaitin82.pdf
- http://web.cs.ucla.edu/~palsberg/course/cs132/linearscan.pdf
- https://dash.harvard.edu/bitstream/handle/1/34325454/tr-21-97.pdf
- https://en.wikipedia.org/wiki/Register_allocation

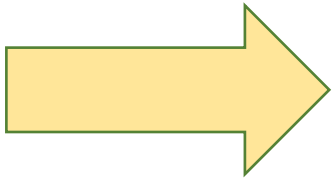
Peephole Optimization

- Peephole
  - a small sliding window of target instructions
- Replace instructions in peephole with shorter and faster sequence
- Can be applied on Intermediate code or assembly code
- Repeated passes result in better optimization

Peephole Optimization

- Redundant-instruction elimination

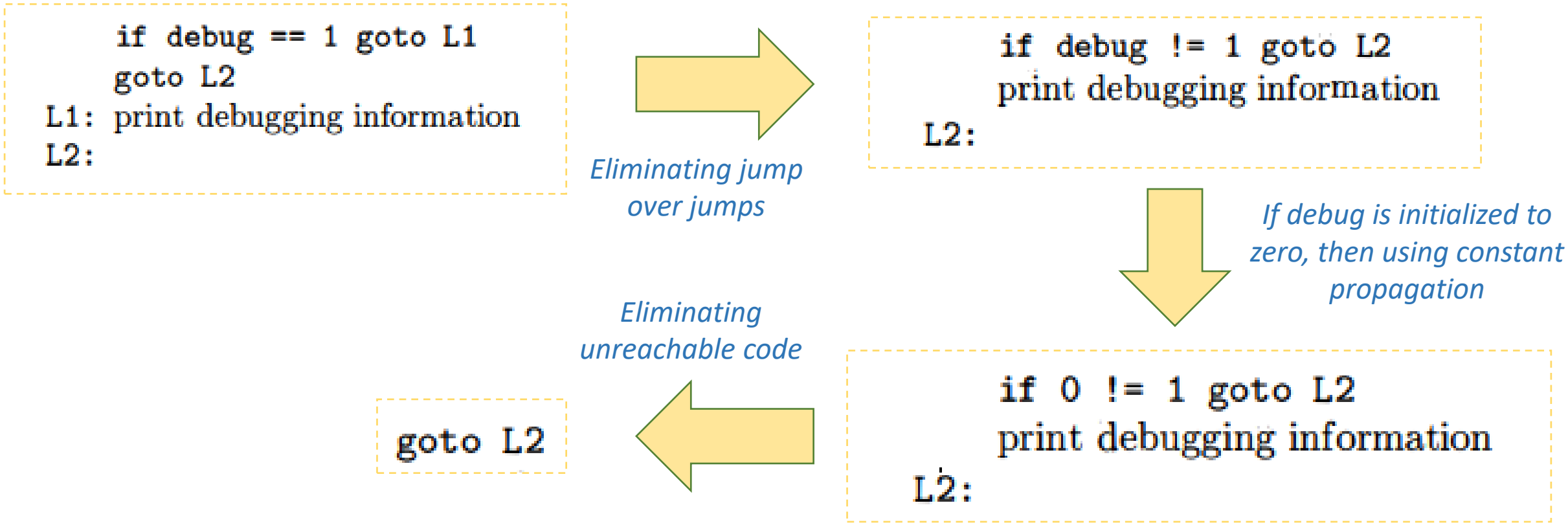
```
LD a, R0
ST R0, a
```



```
LD a, R0
```

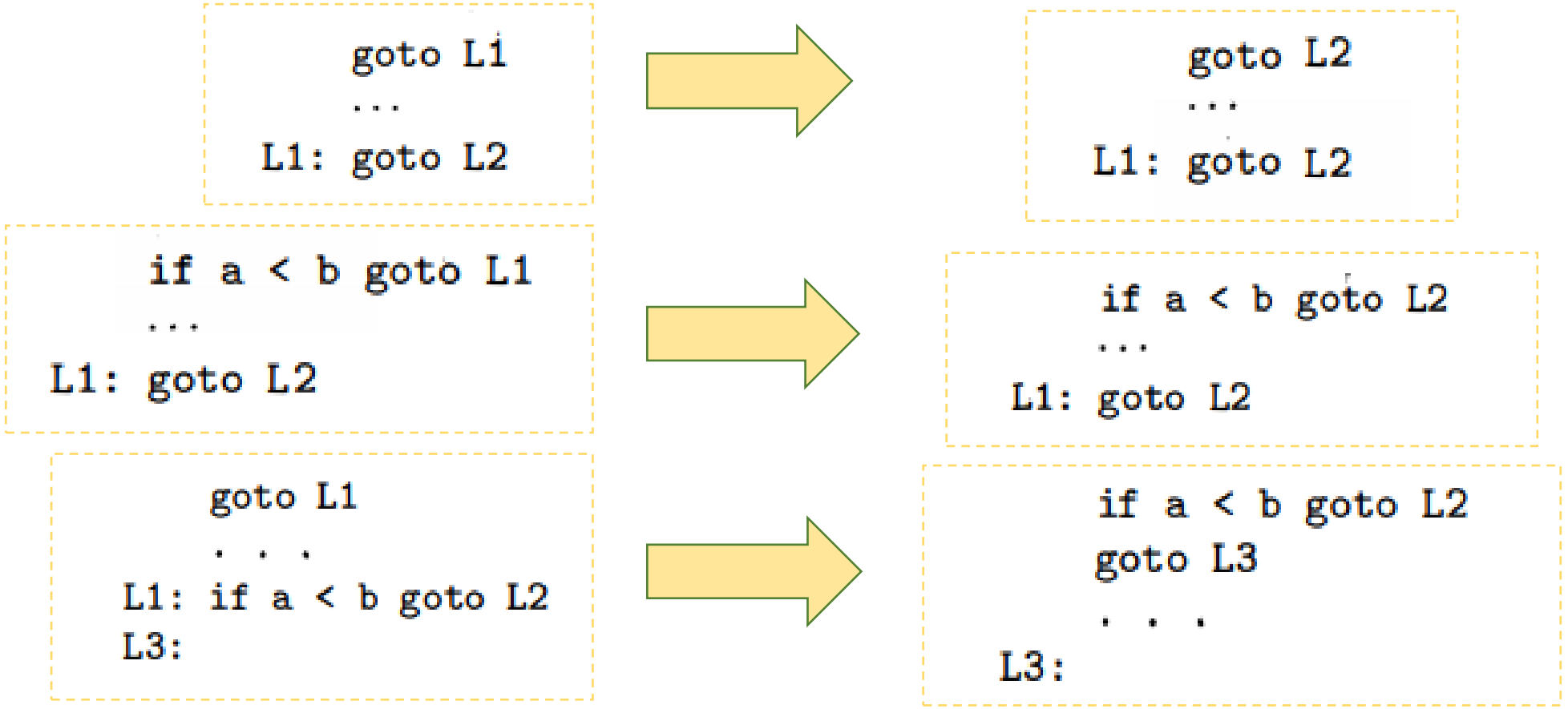
Peephole Optimization

- Eliminating unreachable code



Peephole Optimization

- Flow-of-control optimization



Peephole Optimization

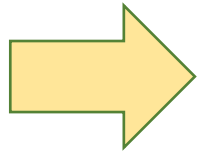
- Algebraic Simplification

$$x + 0 = 0 + x = x \qquad x - 0 = x$$
$$x \times 1 = 1 \times x = x \qquad x/1 = x$$

- Reduction in strength

EXPENSIVE		CHEAPER
x^2	=	$x \times x$
$2 \times x$	=	$x + x$
$x/2$	=	$x \times 0.5$

- Use of machine idioms

x= x + 1  x++

Reference

- Compilers: Principles, Techniques, & Tools, by
  Alfred B. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman

  - Chapter 8
    - 8.1
    - 8.2
    - 8.3
    - 8.4
    - 8.5
    - 8.6
    - 8.7

Thank You!

Live Ranges and Live Intervals

- Recall: A variable is **live** at a particular program point if its value may be read later before it is written.
  - Can find this using global liveness analysis.
- The **live range** for a variable is the set of program points at which that variable is live.
- The **live interval** for a variable is the smallest subrange of the IR code containing all a variable's live ranges.
  - A property of the IR code, **not** the CFG.
  - Less precise than live ranges, but simpler to work with.

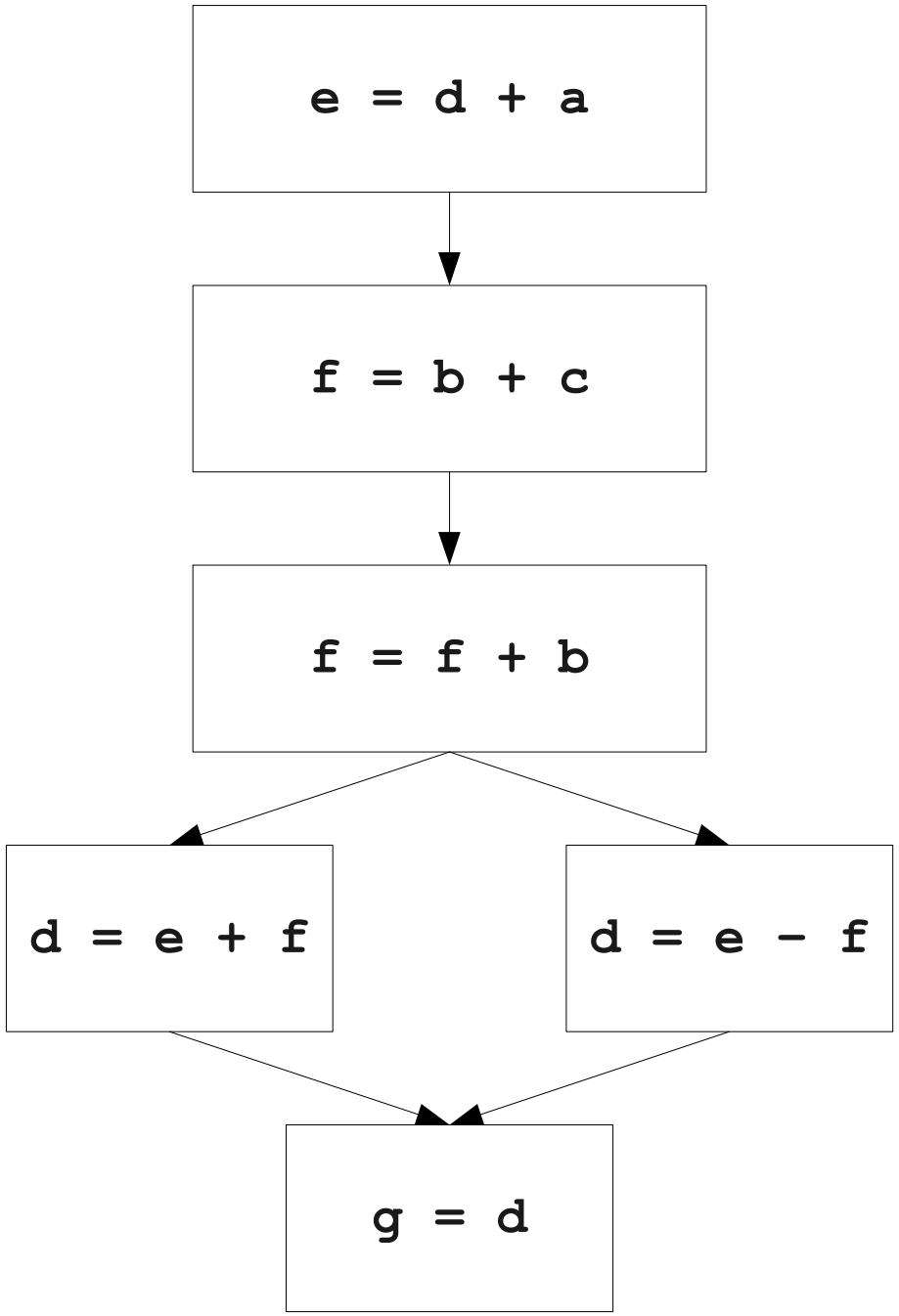
Live Ranges and Live Intervals

Live Ranges and Live Intervals

```
    e = d + a
    f = b + c
    f = f + b
    IfZ e Goto _L0
    d = e + f
    Goto _L1;
_L0:
    d = e - f
_L1:
    g = d
```

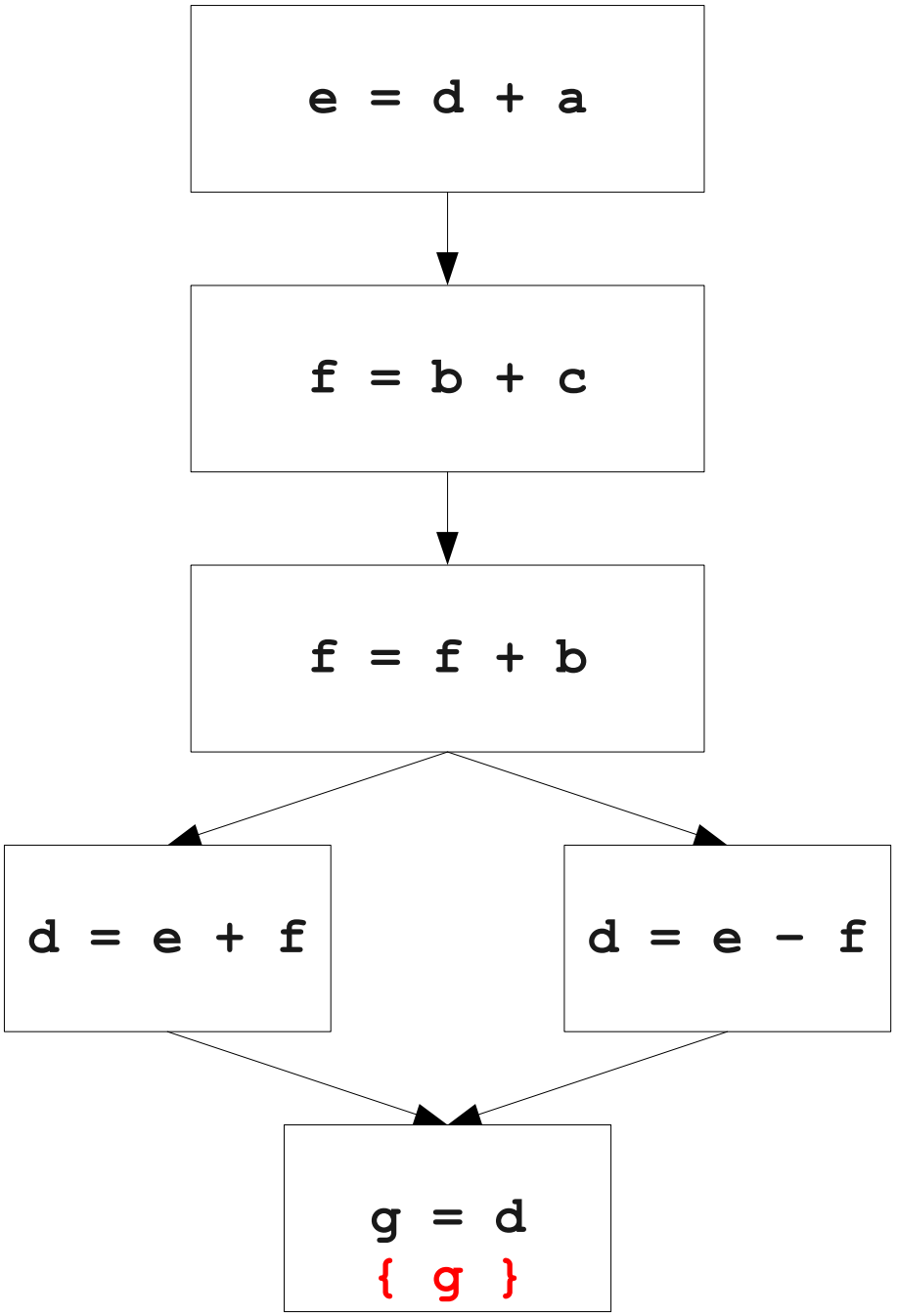
Live Ranges and Live Intervals

```
    e = d + a
    f = b + c
    f = f + b
    IfZ e Goto _L0
    d = e + f
    Goto _L1;
_L0:
    d = e - f
_L1:
    g = d
```



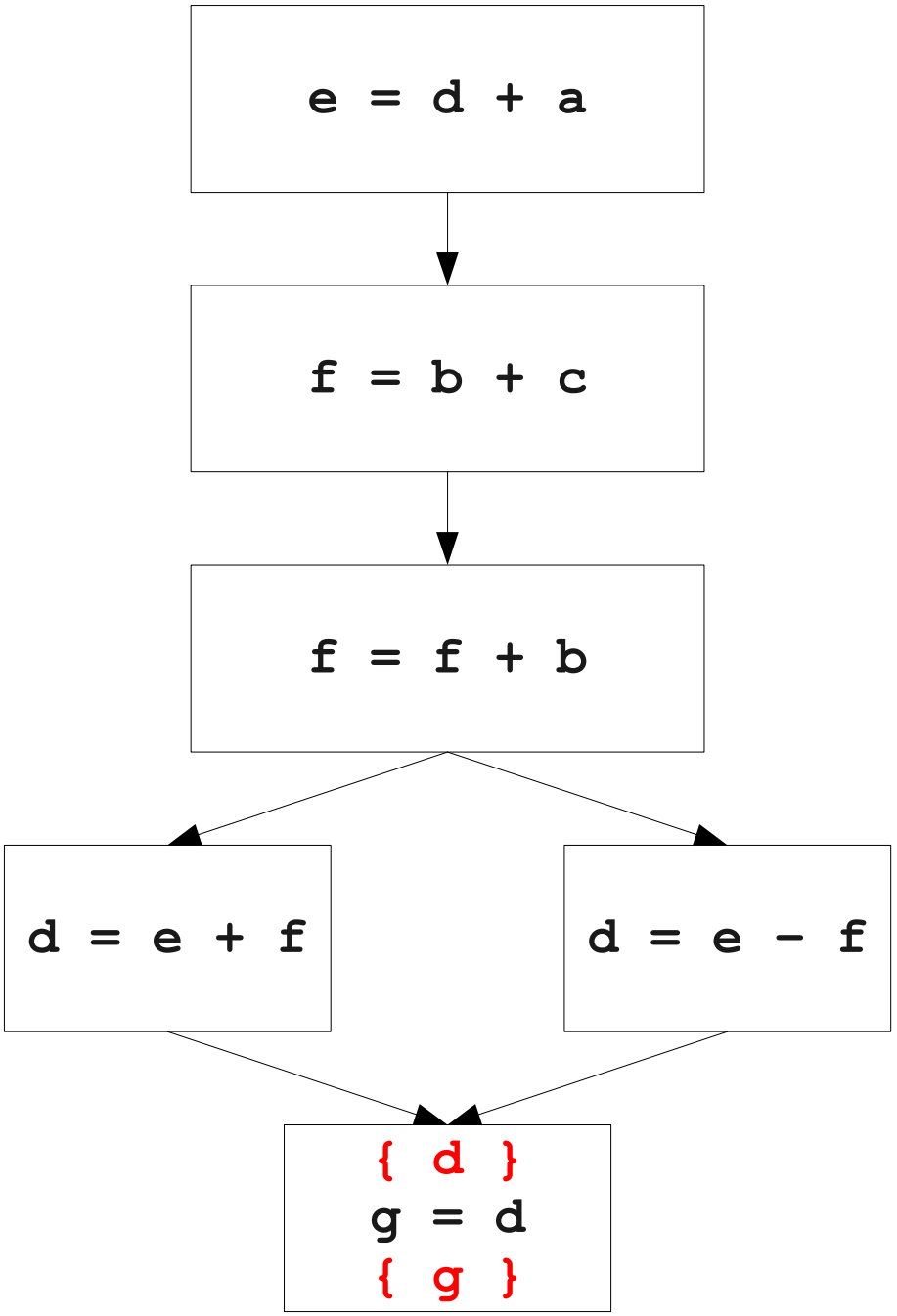
Live Ranges and Live Intervals

```
    e = d + a
    f = b + c
    f = f + b
    IfZ e Goto _L0
    d = e + f
    Goto _L1;
_L0:
    d = e - f
_L1:
    g = d
```



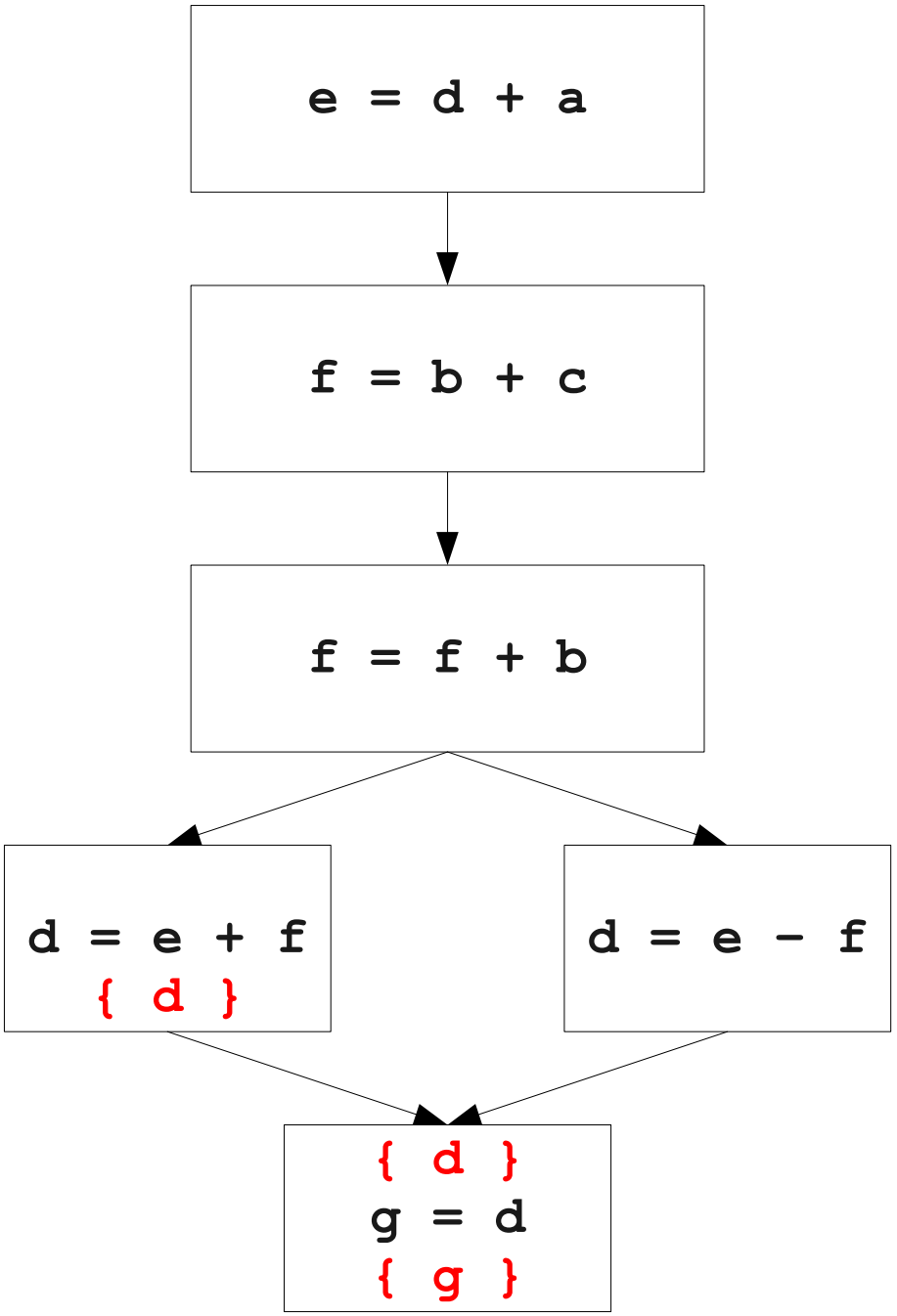
Live Ranges and Live Intervals

```
    e = d + a
    f = b + c
    f = f + b
    IfZ e Goto _L0
    d = e + f
    Goto _L1;
_L0:
    d = e - f
_L1:
    g = d
```



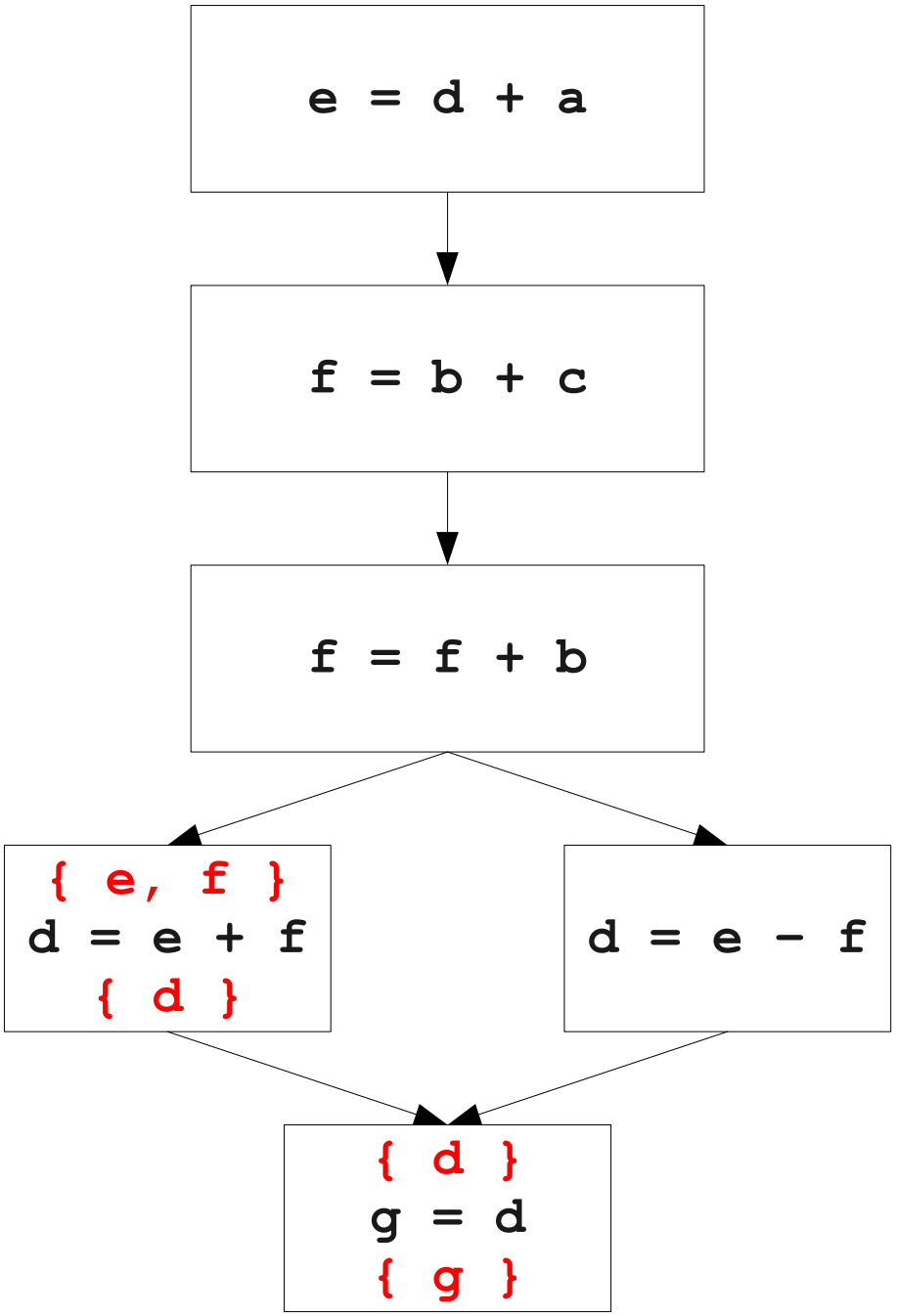
Live Ranges and Live Intervals

```
    e = d + a
    f = b + c
    f = f + b
    IfZ e Goto _L0
    d = e + f
    Goto _L1;
_L0:
    d = e - f
_L1:
    g = d
```



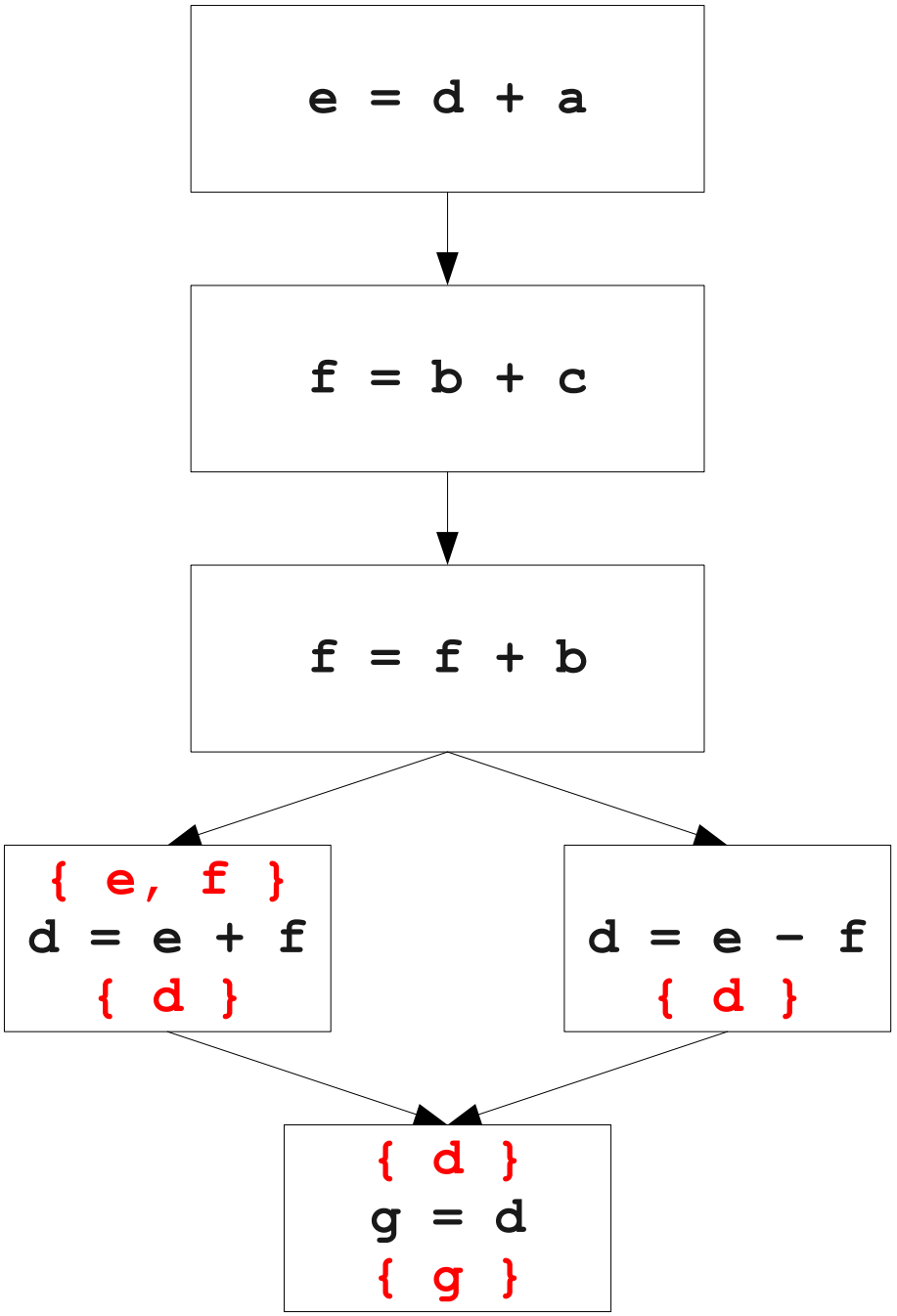
Live Ranges and Live Intervals

```
    e = d + a
    f = b + c
    f = f + b
    IfZ e Goto _L0
    d = e + f
    Goto _L1;
_L0:
    d = e - f
_L1:
    g = d
```



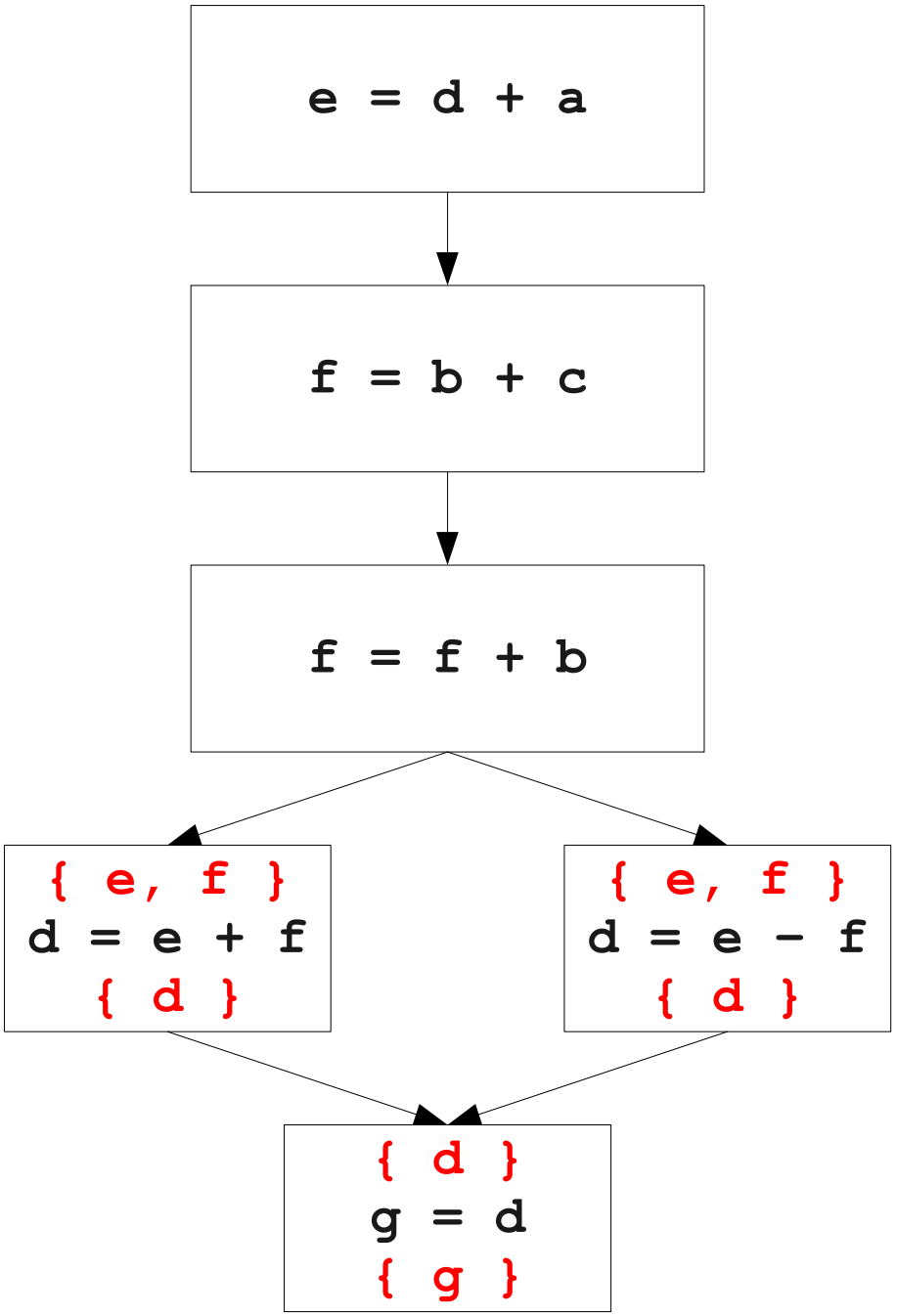
Live Ranges and Live Intervals

```
    e = d + a
    f = b + c
    f = f + b
    IfZ e Goto _L0
    d = e + f
    Goto _L1;
_L0:
    d = e - f
_L1:
    g = d
```



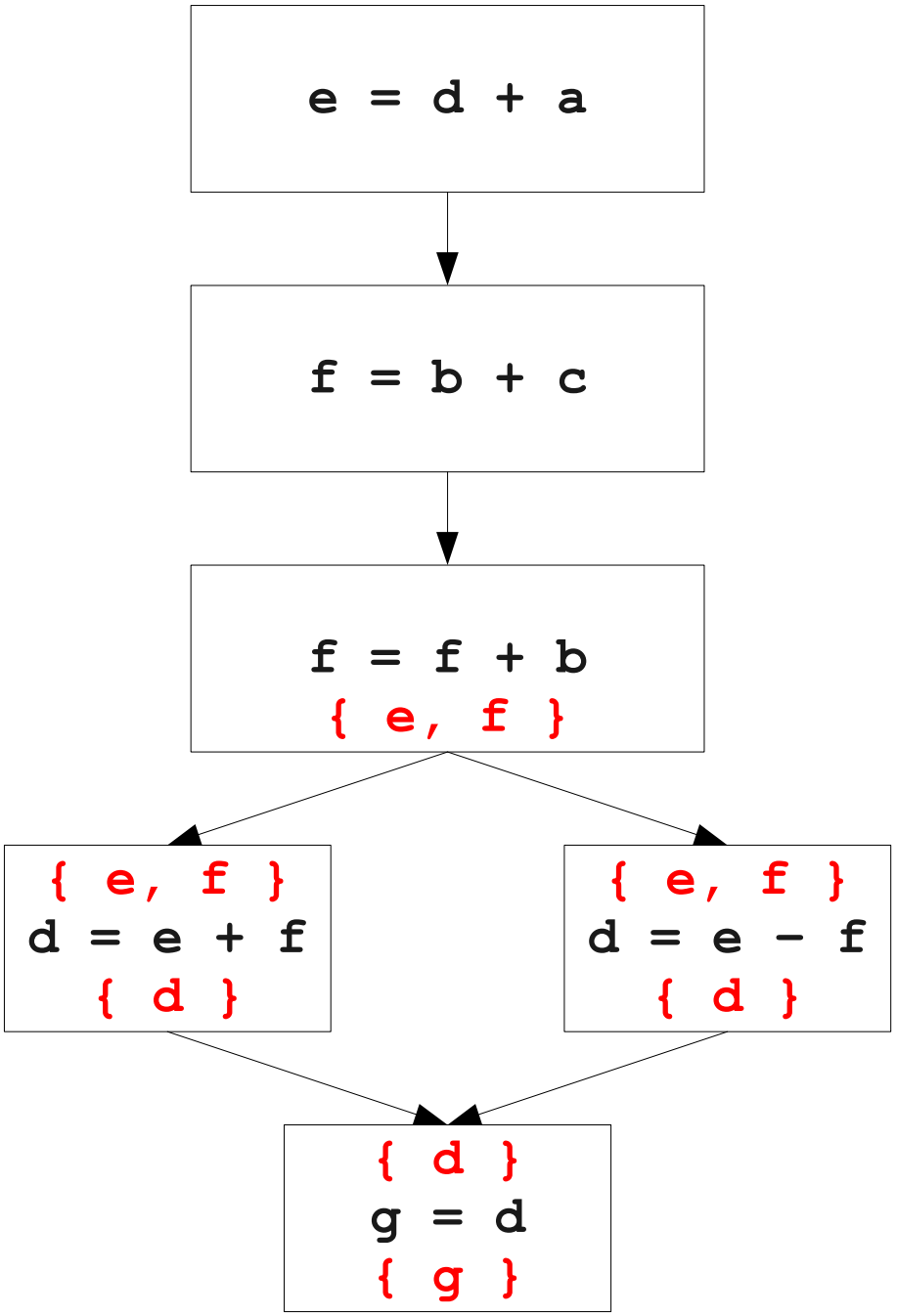
Live Ranges and Live Intervals

```
    e = d + a
    f = b + c
    f = f + b
    IfZ e Goto _L0
    d = e + f
    Goto _L1;
_L0:
    d = e - f
_L1:
    g = d
```



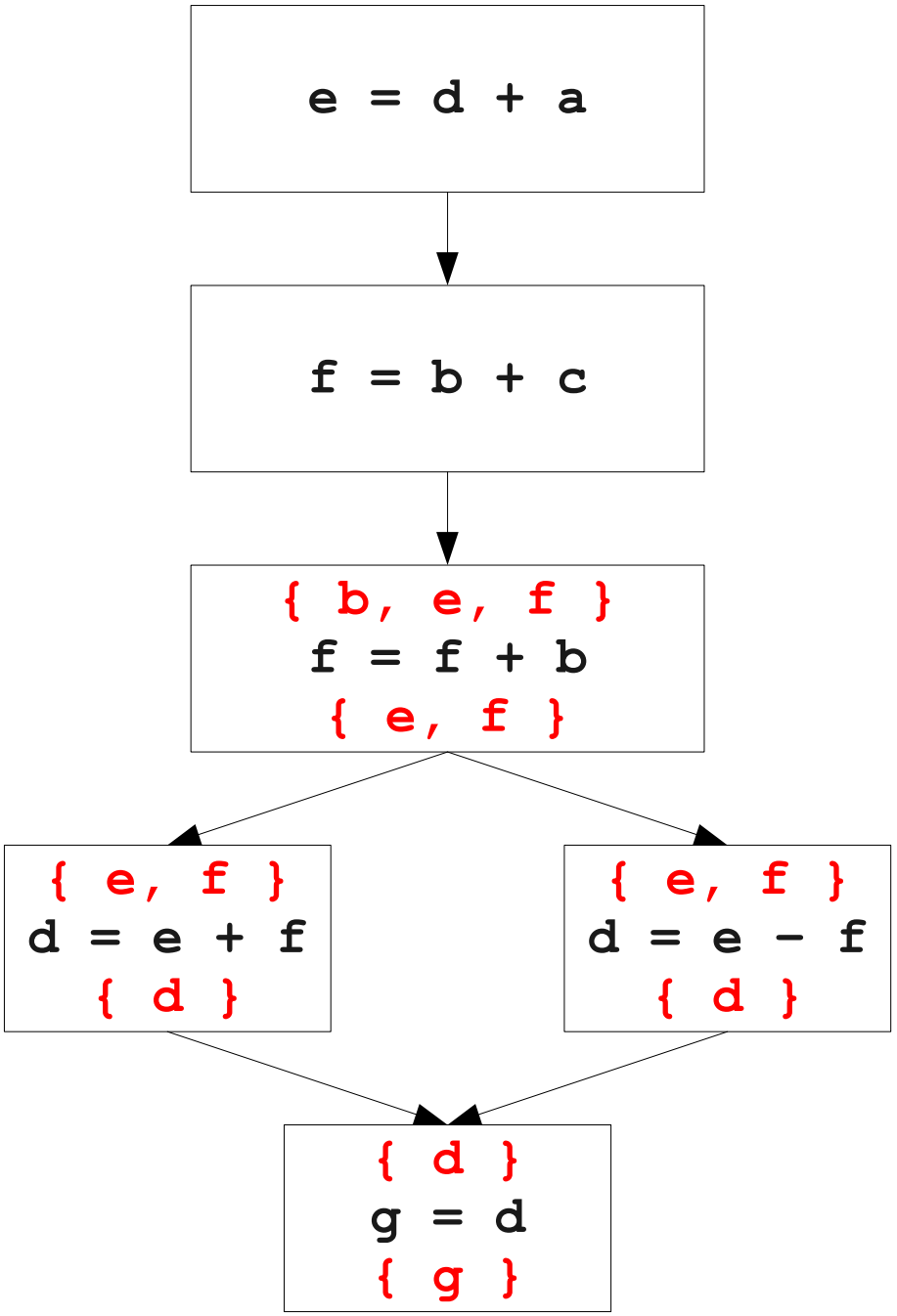
Live Ranges and Live Intervals

```
    e = d + a
    f = b + c
    f = f + b
    IfZ e Goto _L0
    d = e + f
    Goto _L1;
_L0:
    d = e - f
_L1:
    g = d
```



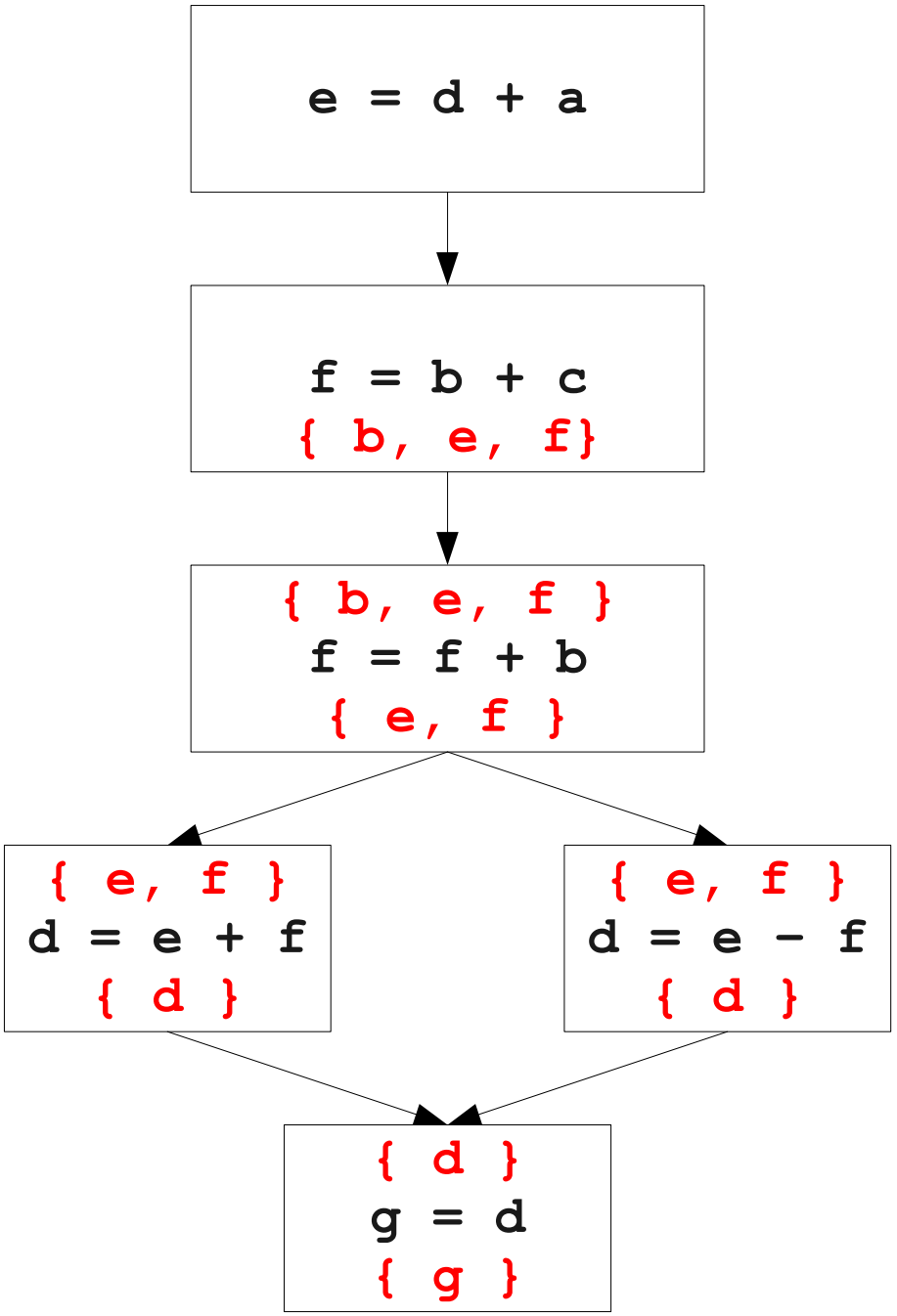
Live Ranges and Live Intervals

```
    e = d + a
    f = b + c
    f = f + b
    IfZ e Goto _L0
    d = e + f
    Goto _L1;
_L0:
    d = e - f
_L1:
    g = d
```



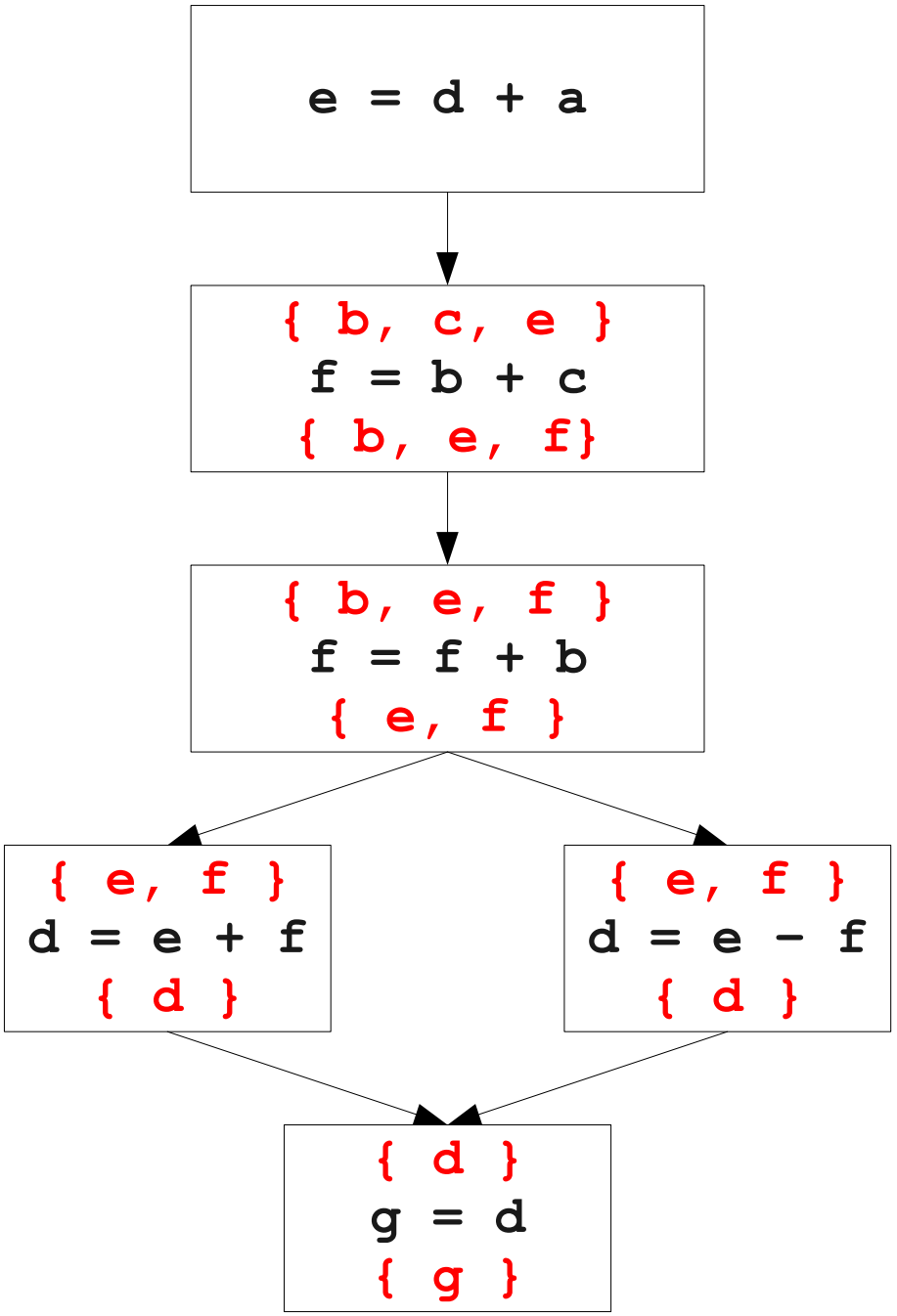
Live Ranges and Live Intervals

```
    e = d + a
    f = b + c
    f = f + b
    IfZ e Goto _L0
    d = e + f
    Goto _L1;
_L0:
    d = e - f
_L1:
    g = d
```



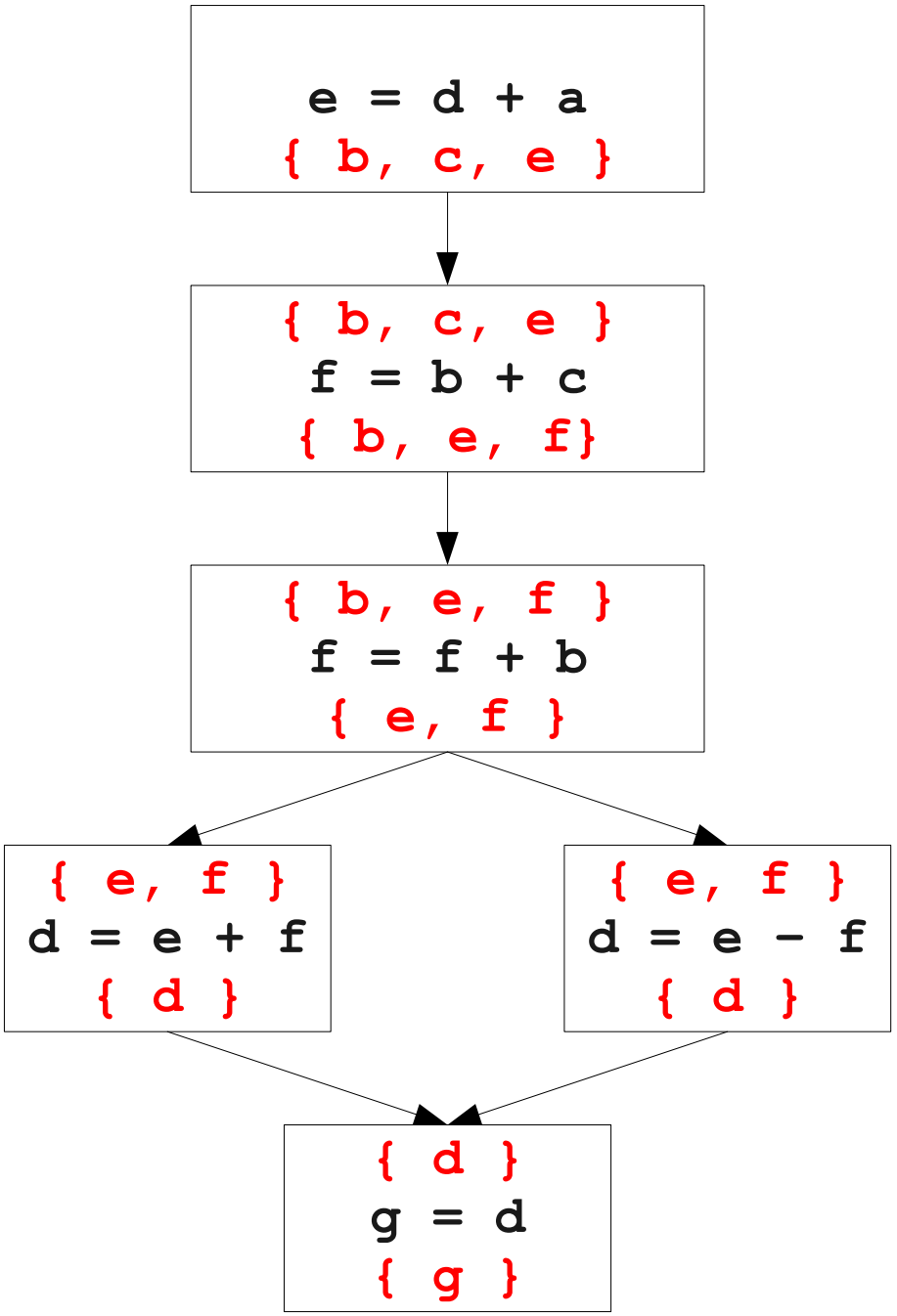
Live Ranges and Live Intervals

```
    e = d + a
    f = b + c
    f = f + b
    IfZ e Goto _L0
    d = e + f
    Goto _L1;
_L0:
    d = e - f
_L1:
    g = d
```



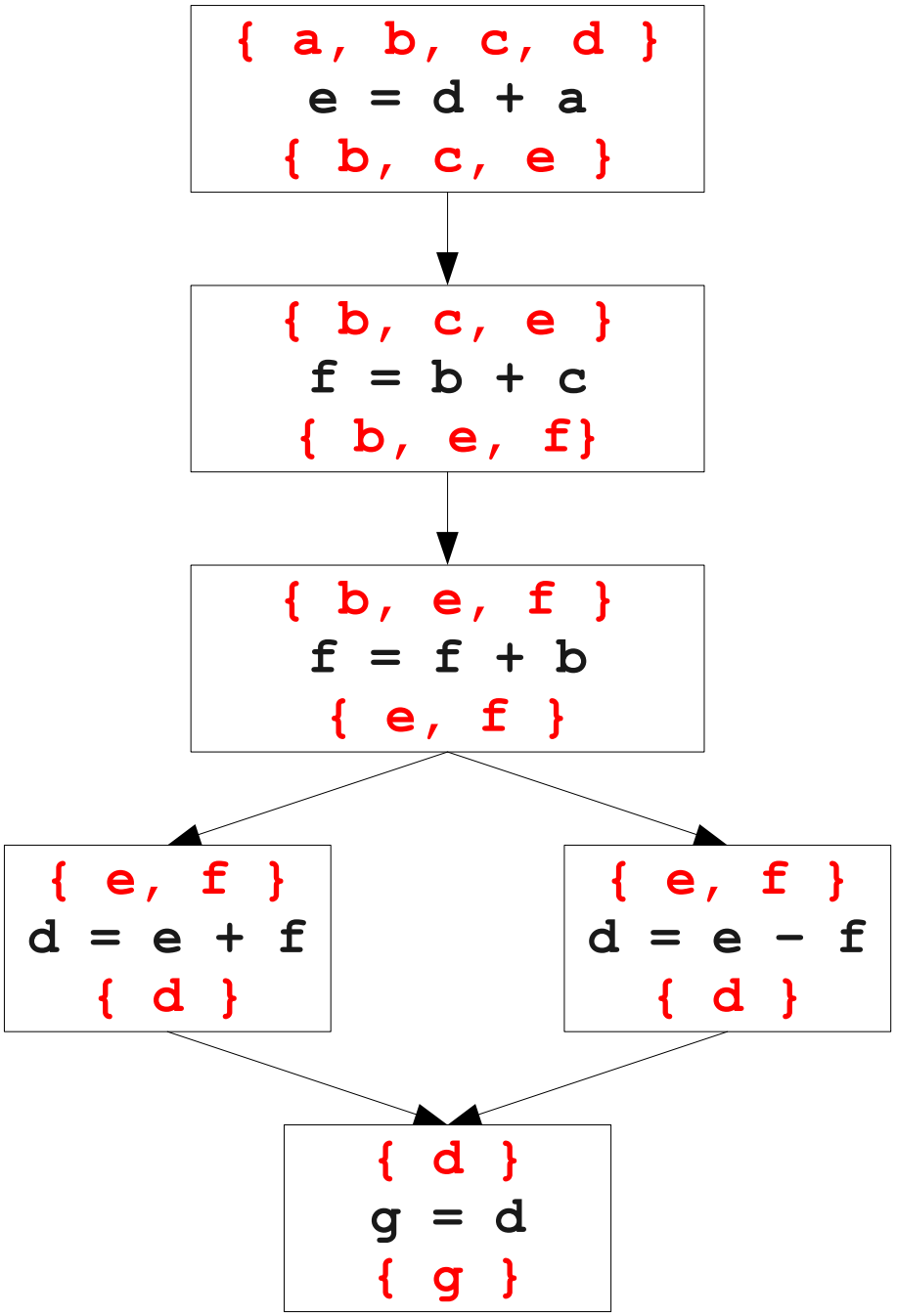
Live Ranges and Live Intervals

```
    e = d + a
    f = b + c
    f = f + b
    IfZ e Goto _L0
    d = e + f
    Goto _L1;
_L0:
    d = e - f
_L1:
    g = d
```



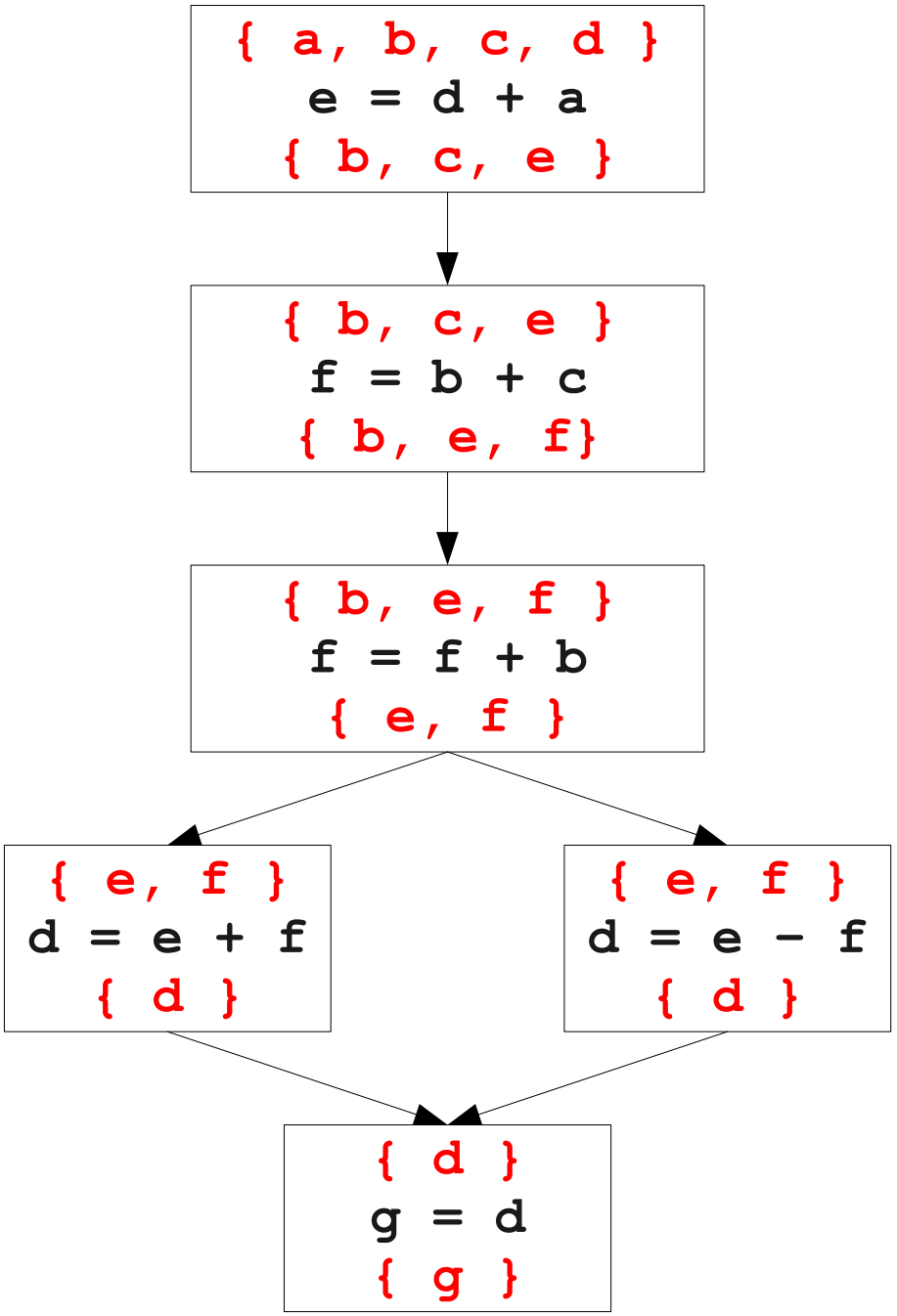
Live Ranges and Live Intervals

```
    e = d + a
    f = b + c
    f = f + b
    IfZ e Goto _L0
    d = e + f
    Goto _L1;
_L0:
    d = e - f
_L1:
    g = d
```

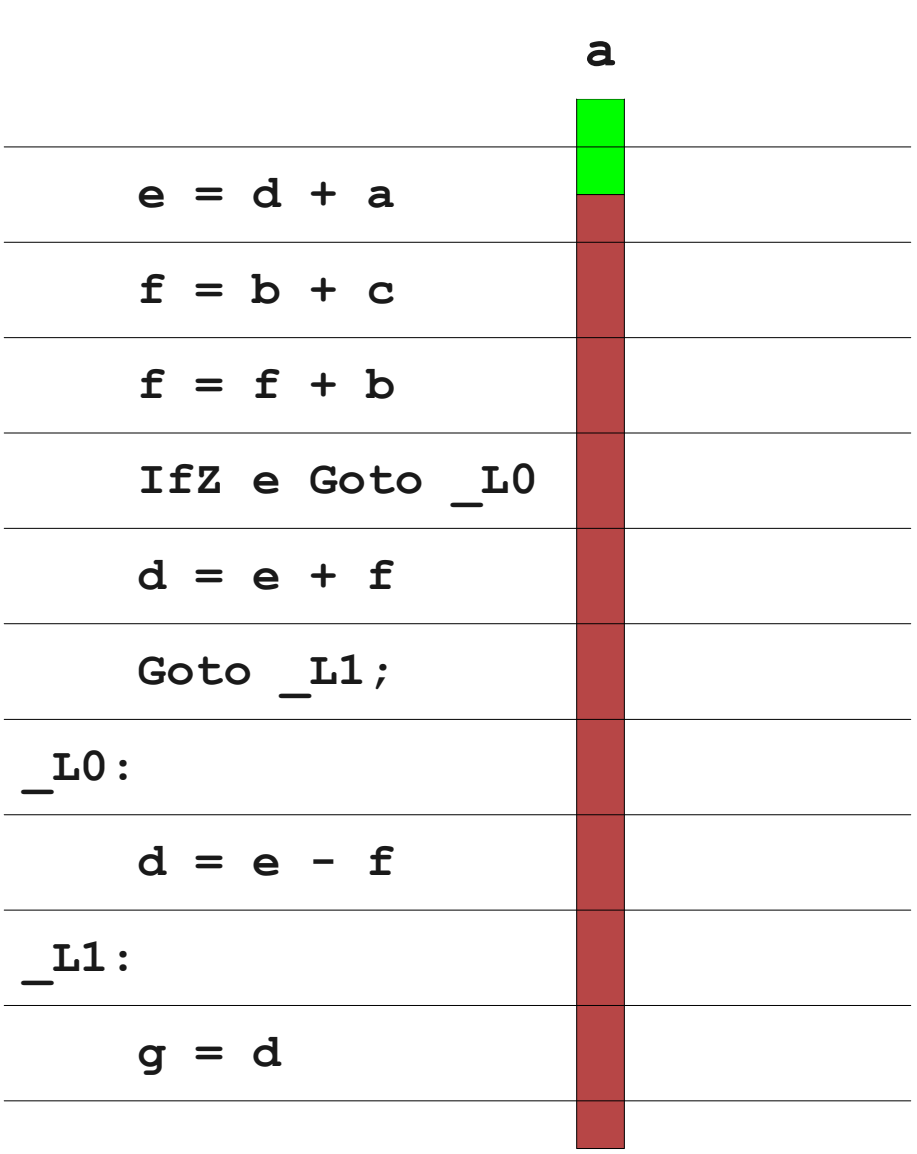


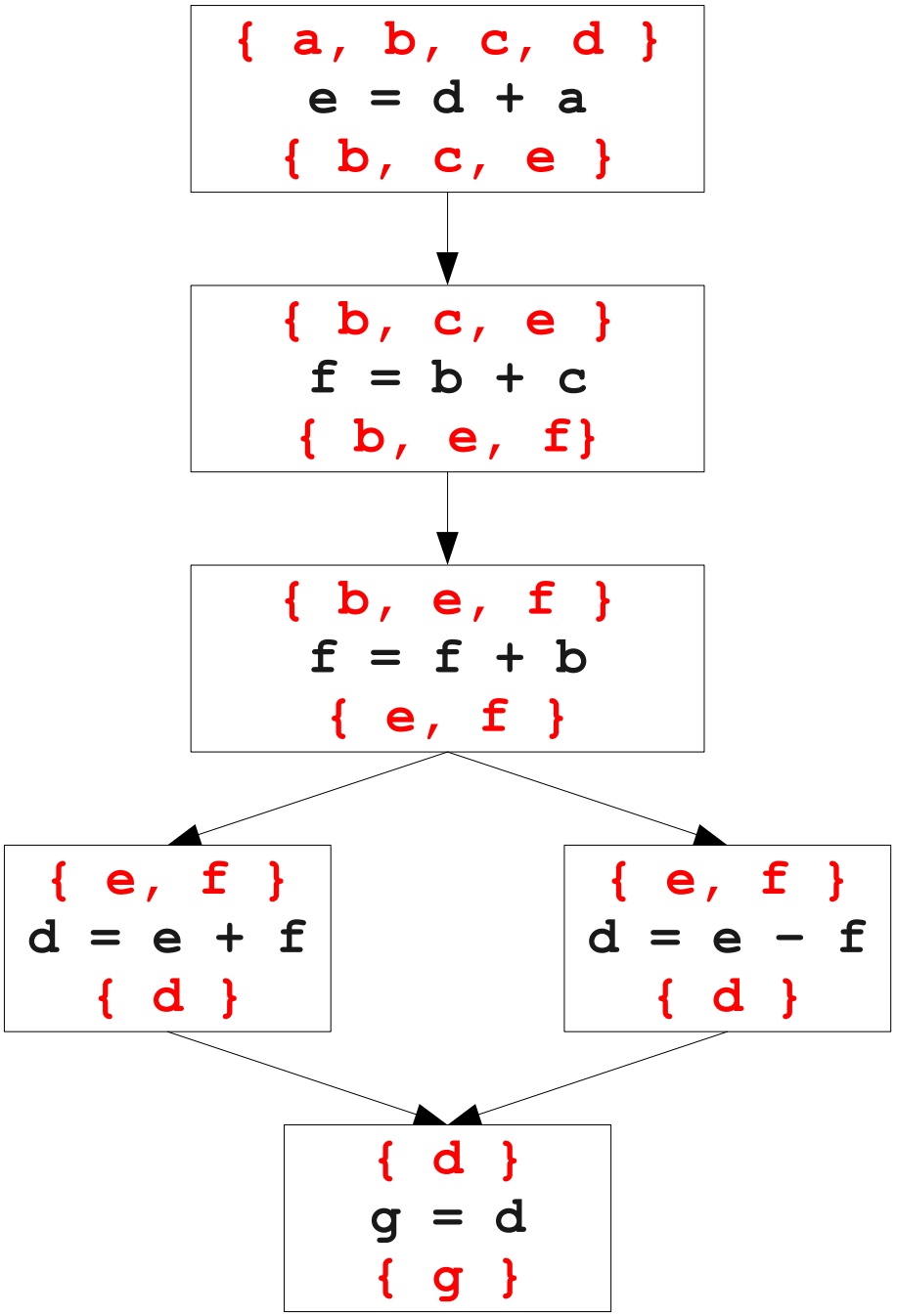
Live Ranges and Live Intervals

```
    e = d + a
    f = b + c
    f = f + b
    IfZ e Goto _L0
    d = e + f
    Goto _L1;
_L0:
    d = e - f
_L1:
    g = d
```

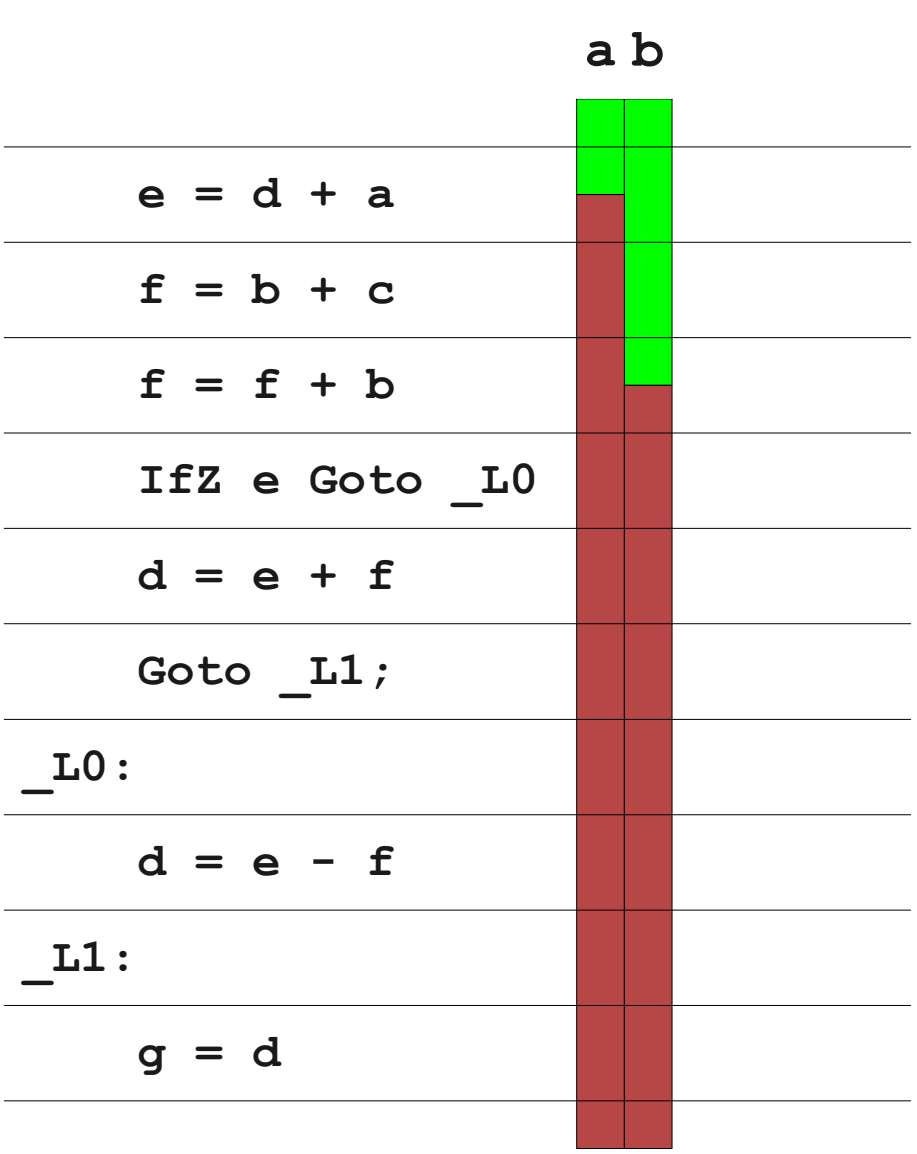


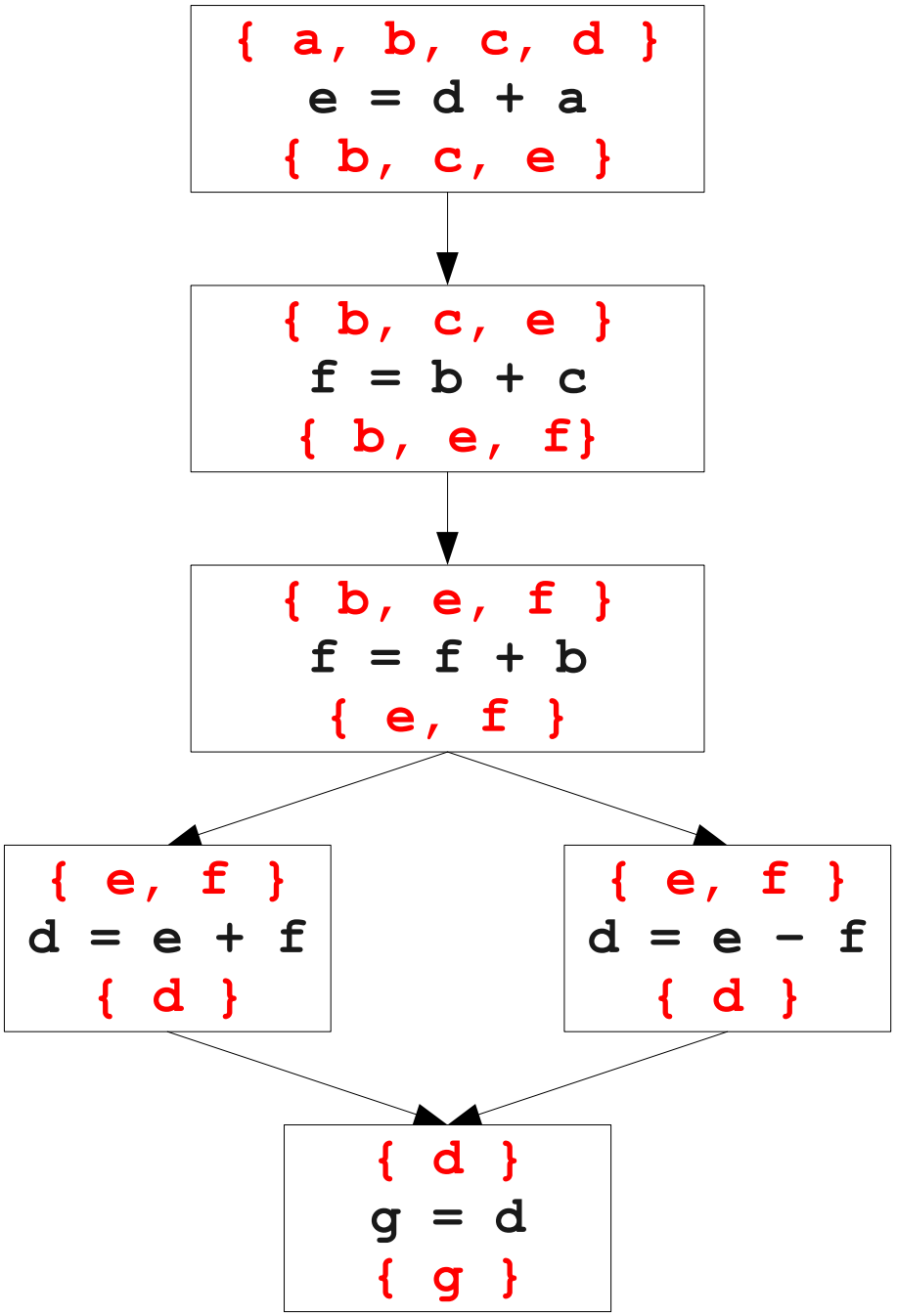
Live Ranges and Live Intervals



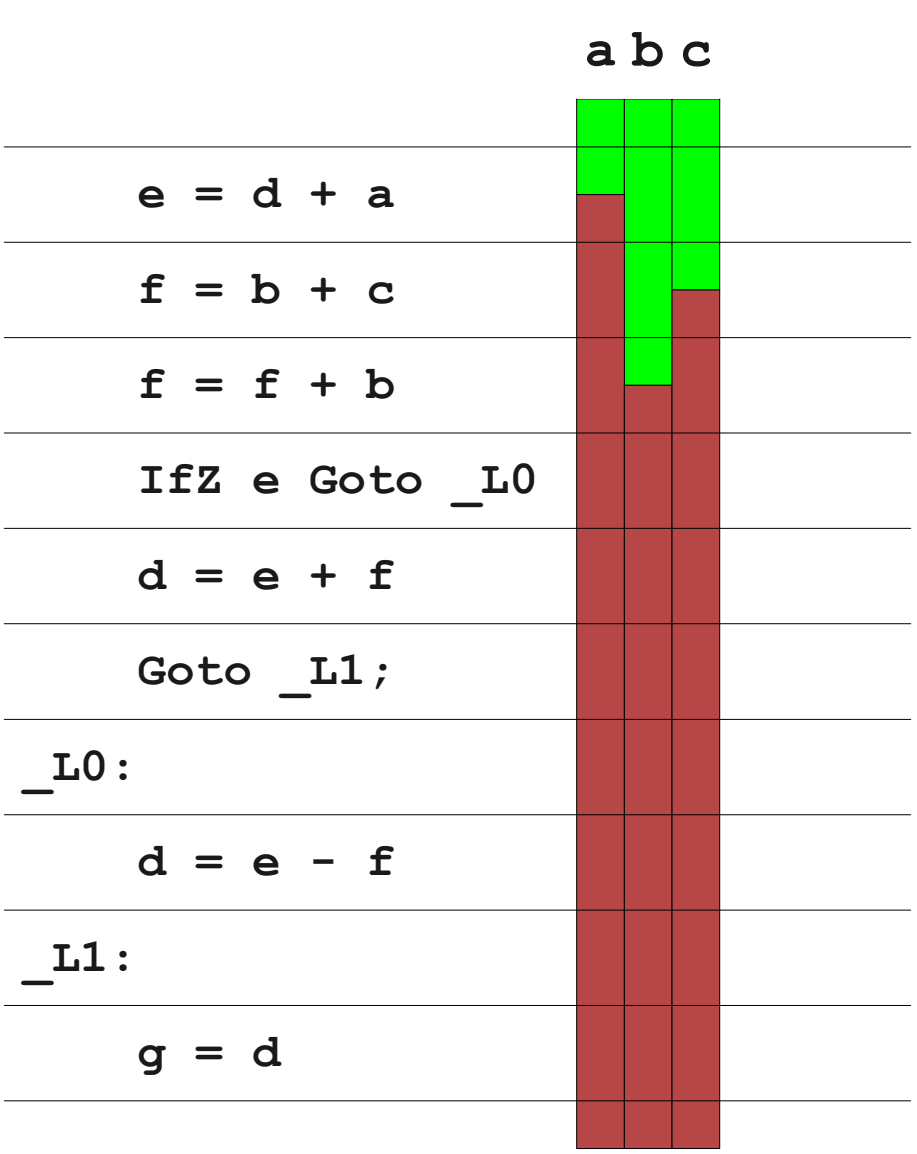


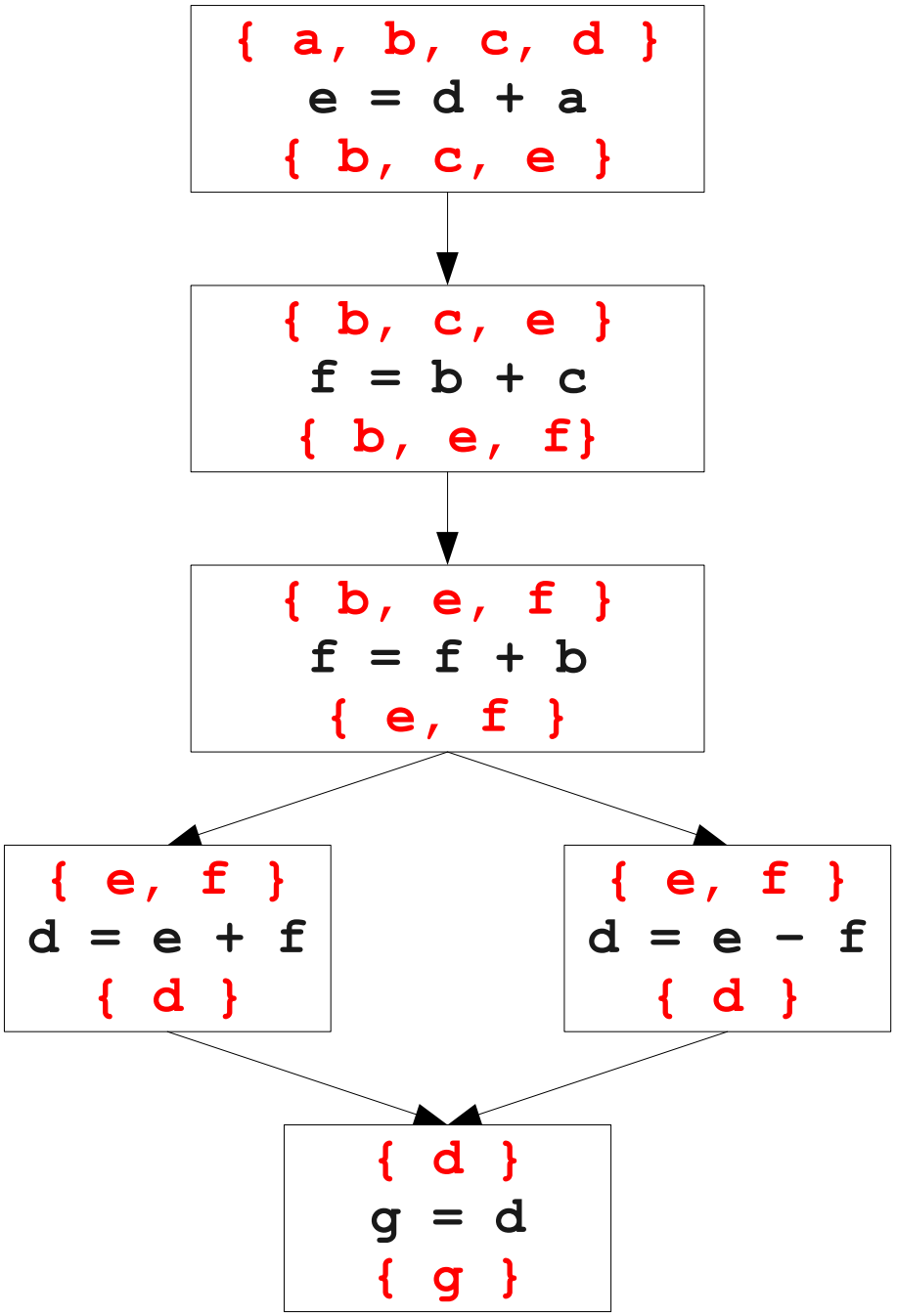
Live Ranges and Live Intervals



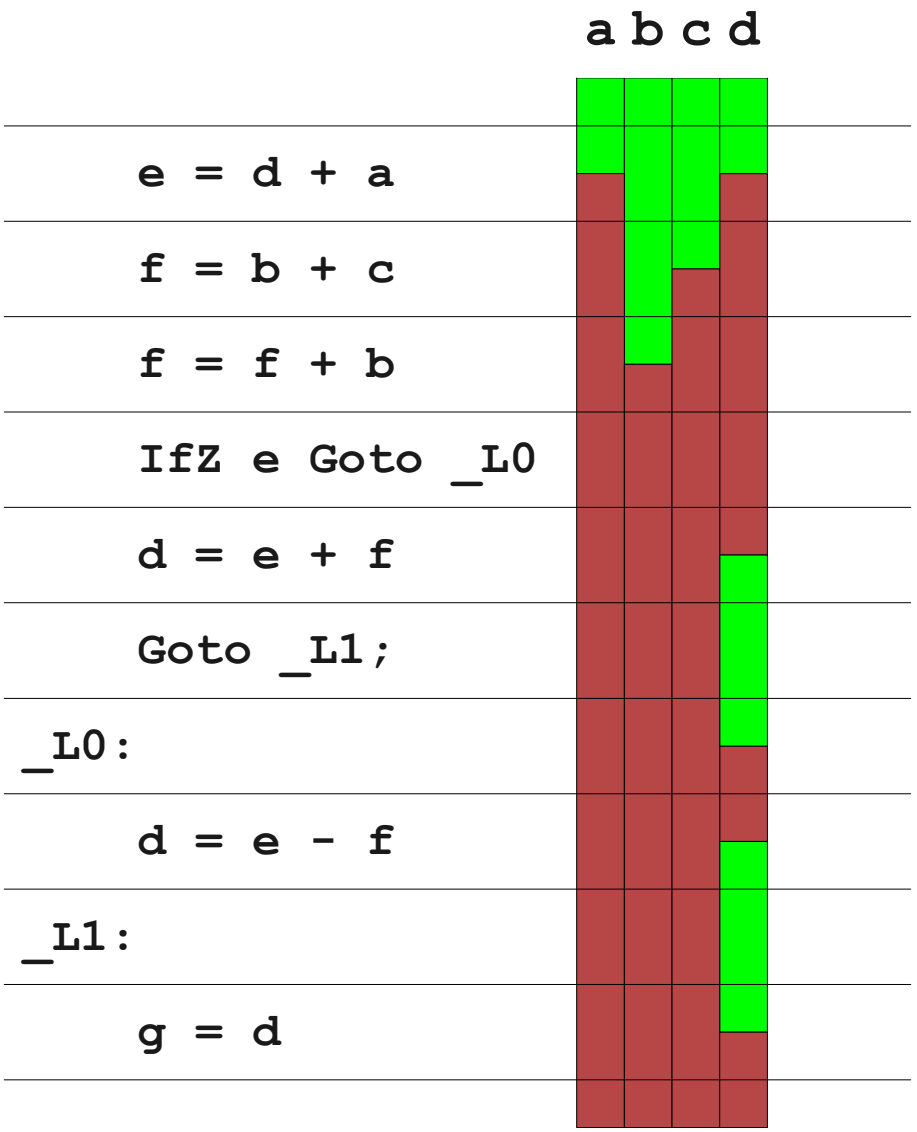


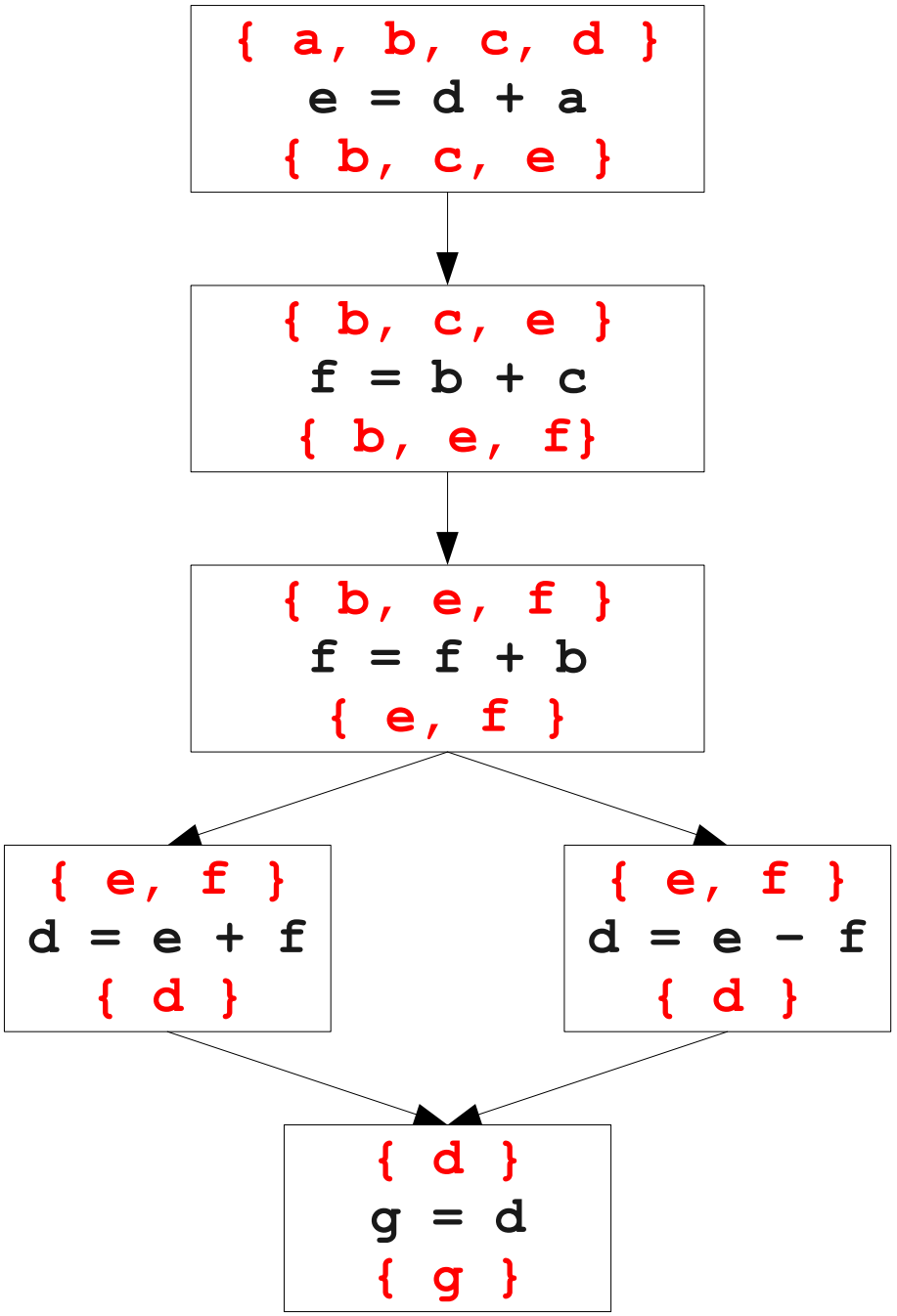
Live Ranges and Live Intervals



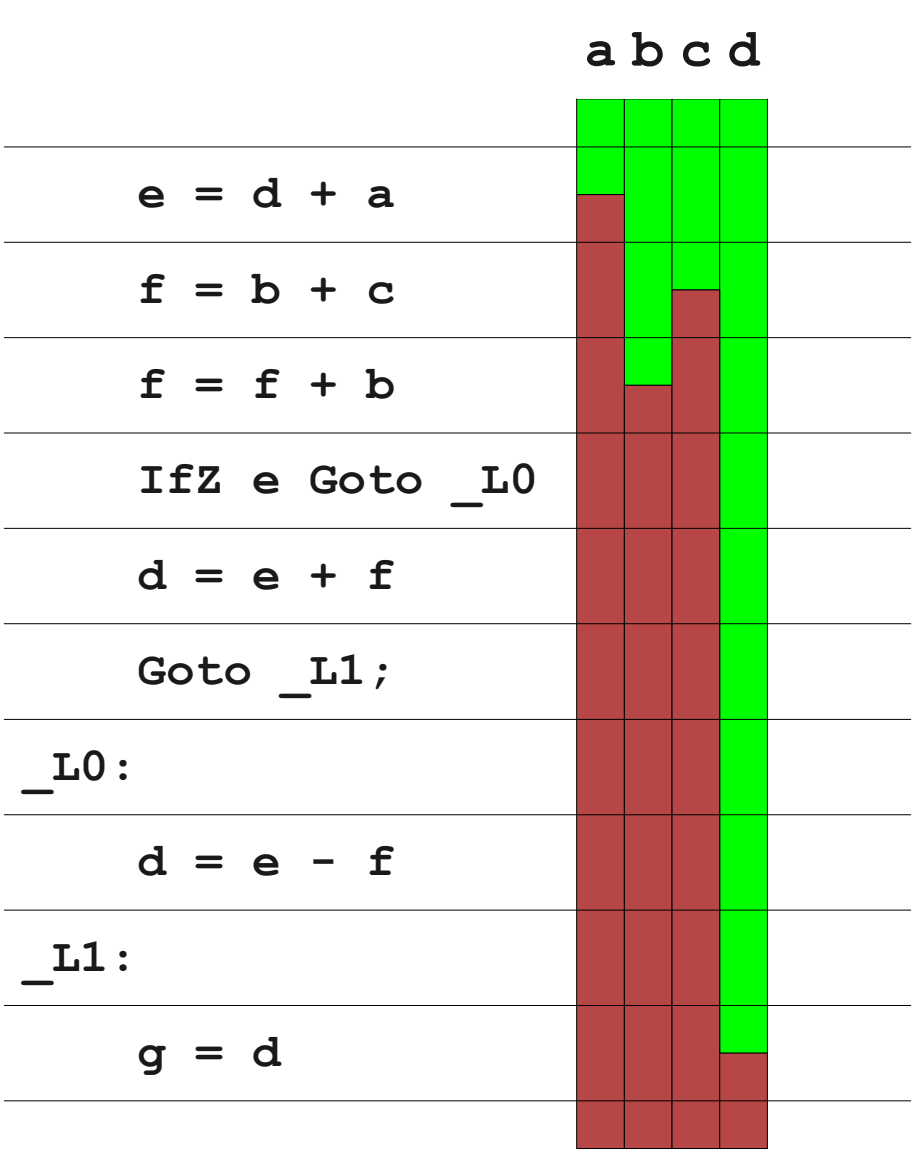


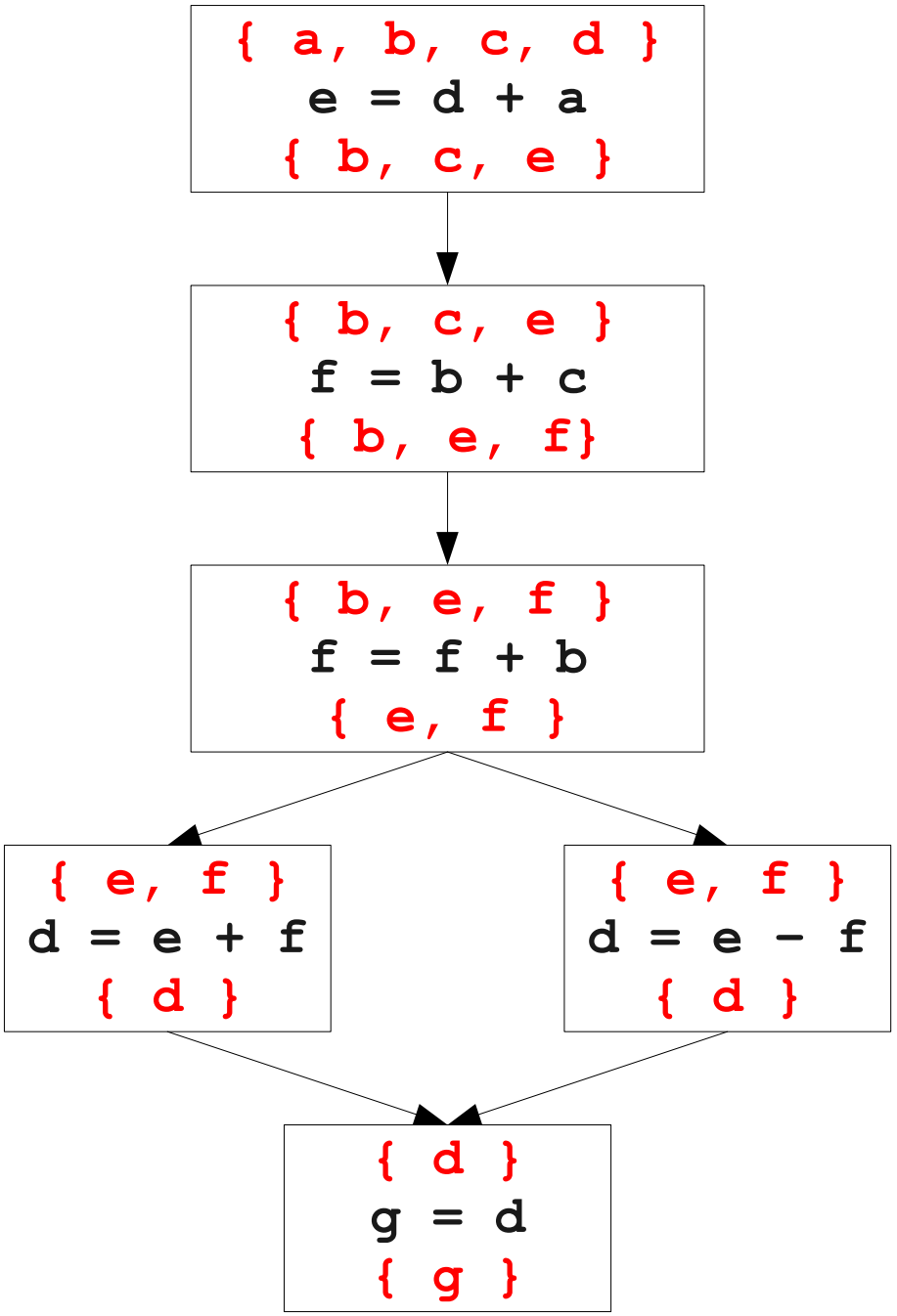
Live Ranges and Live Intervals



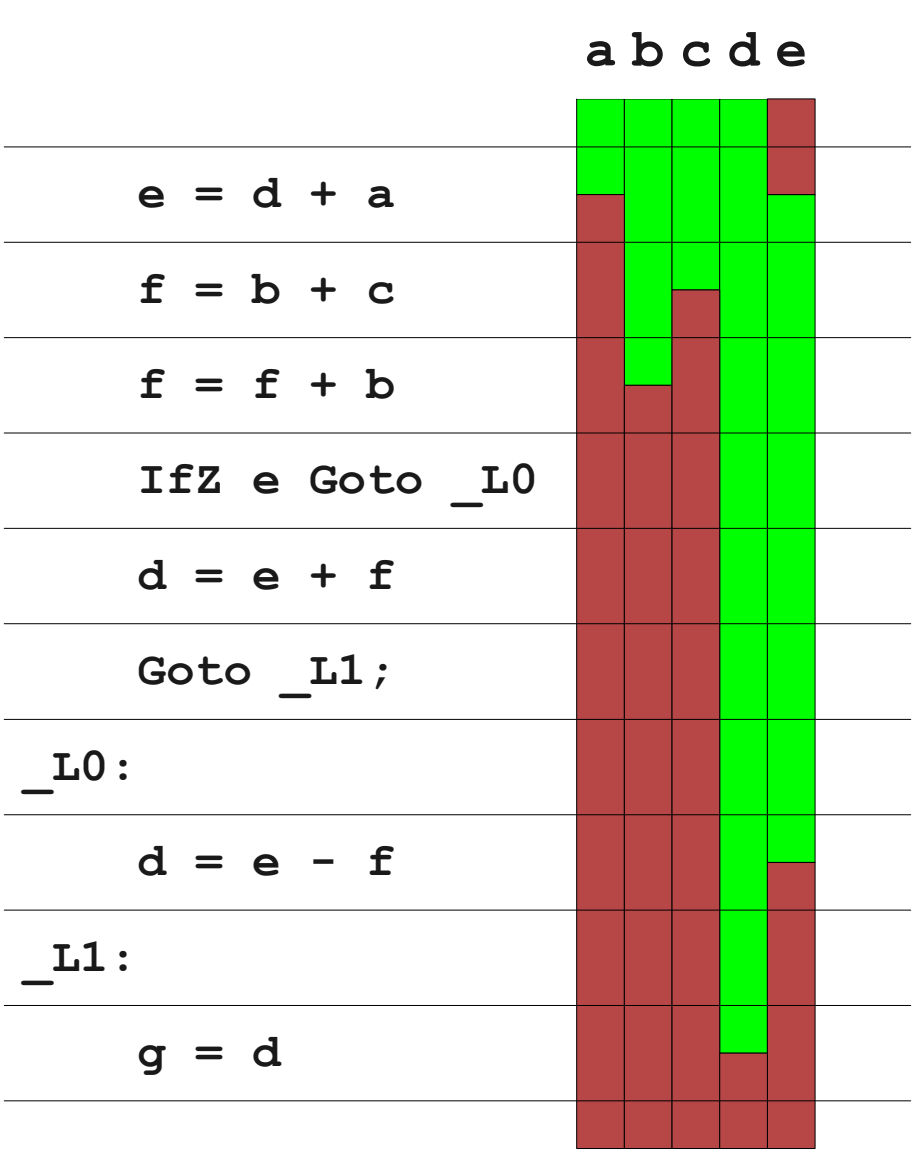


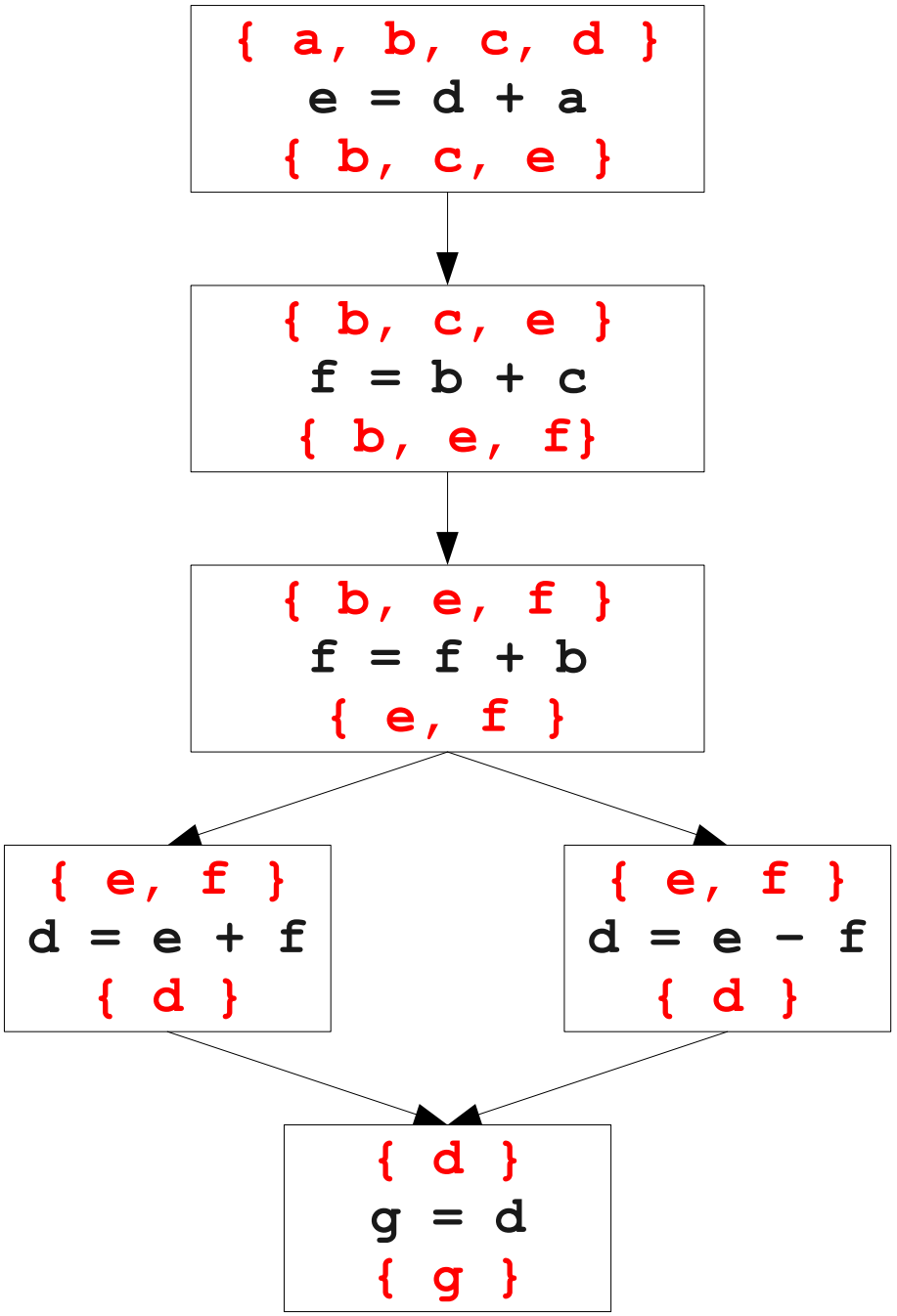
Live Ranges and Live Intervals



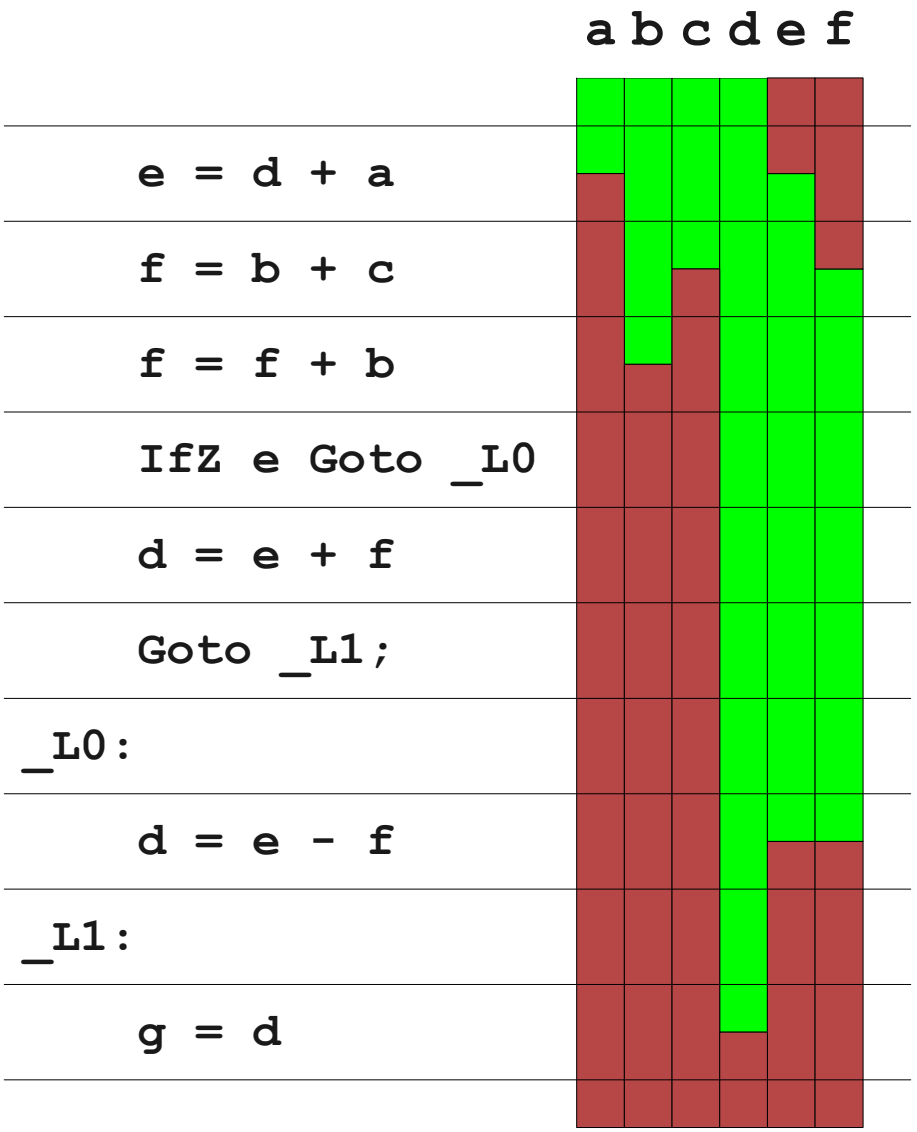


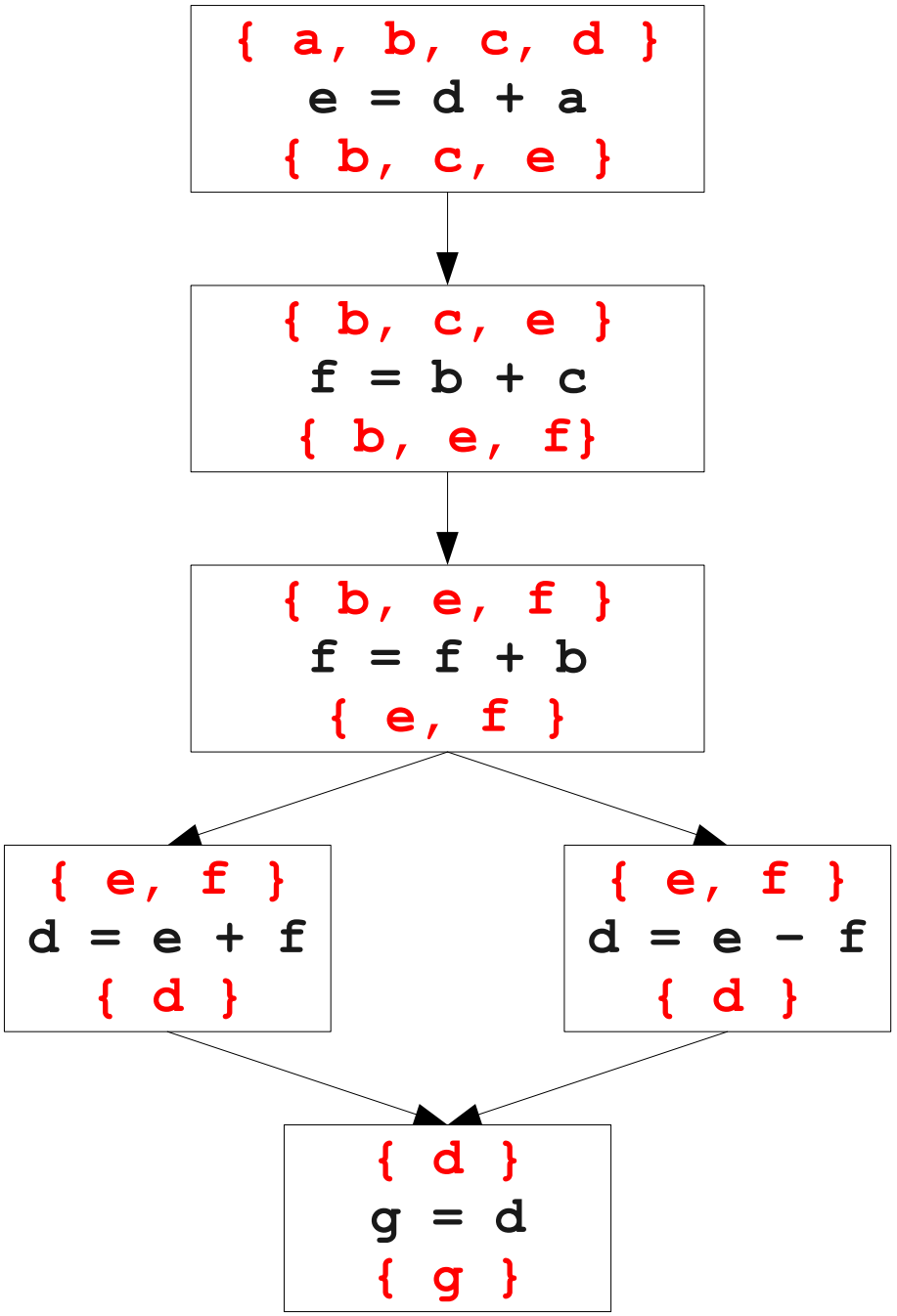
Live Ranges and Live Intervals



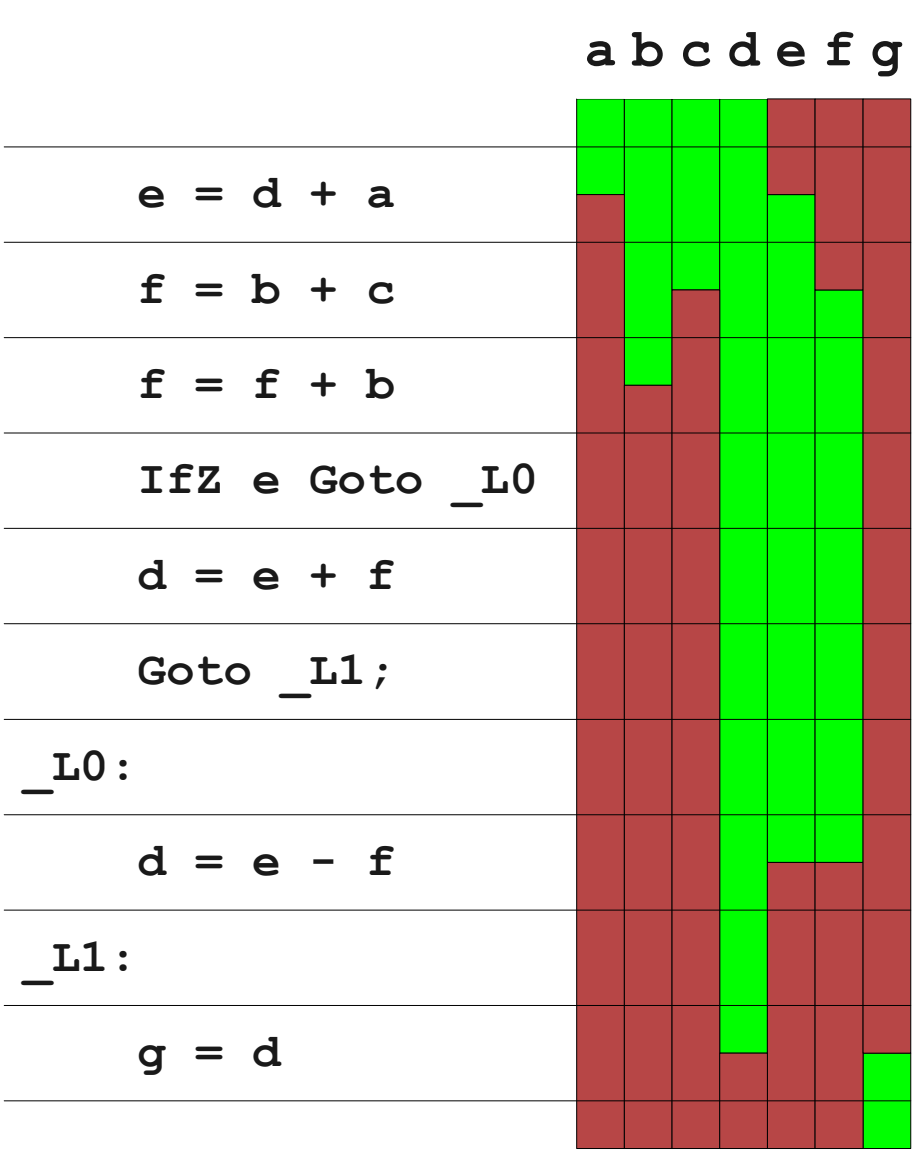


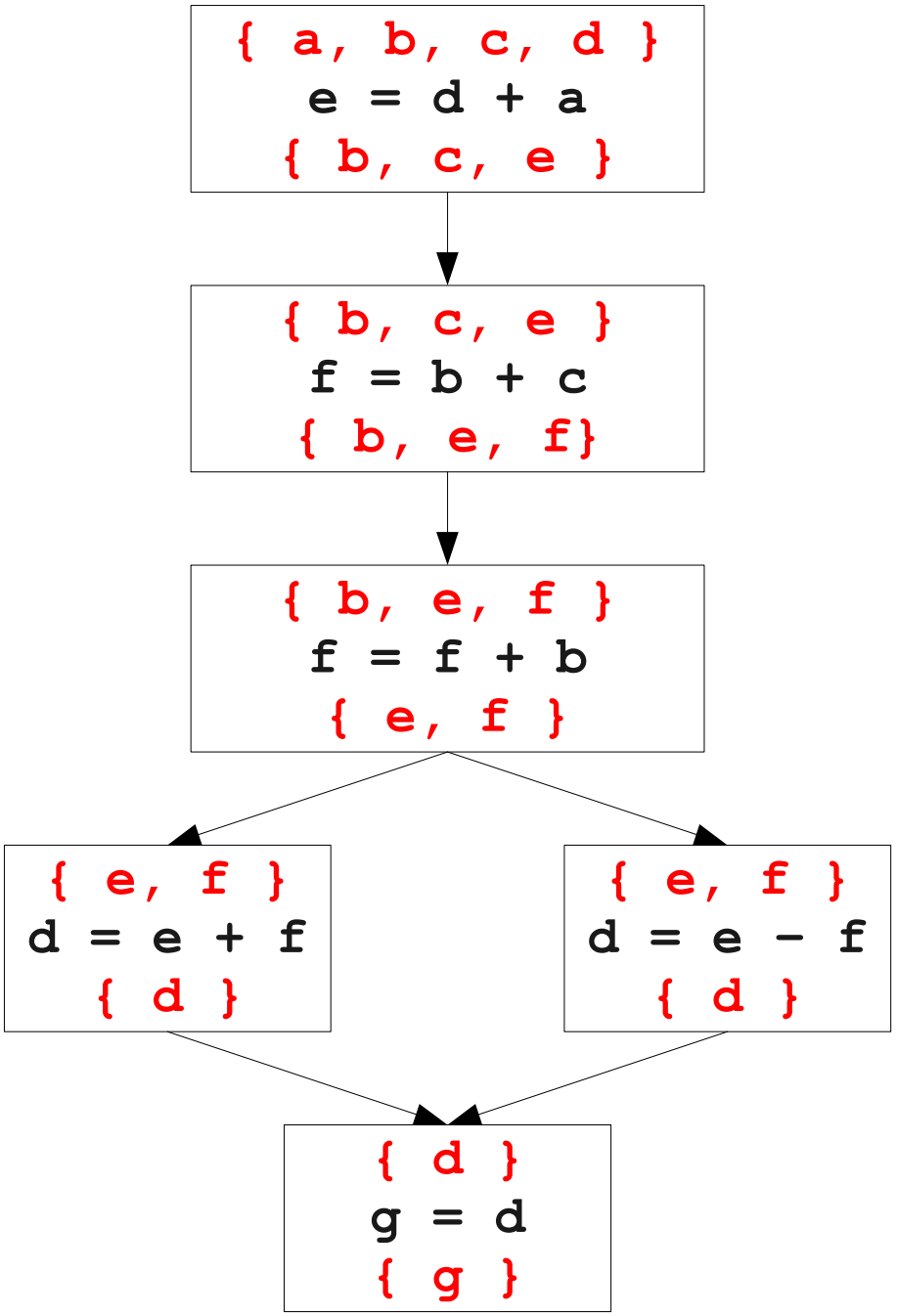
Live Ranges and Live Intervals





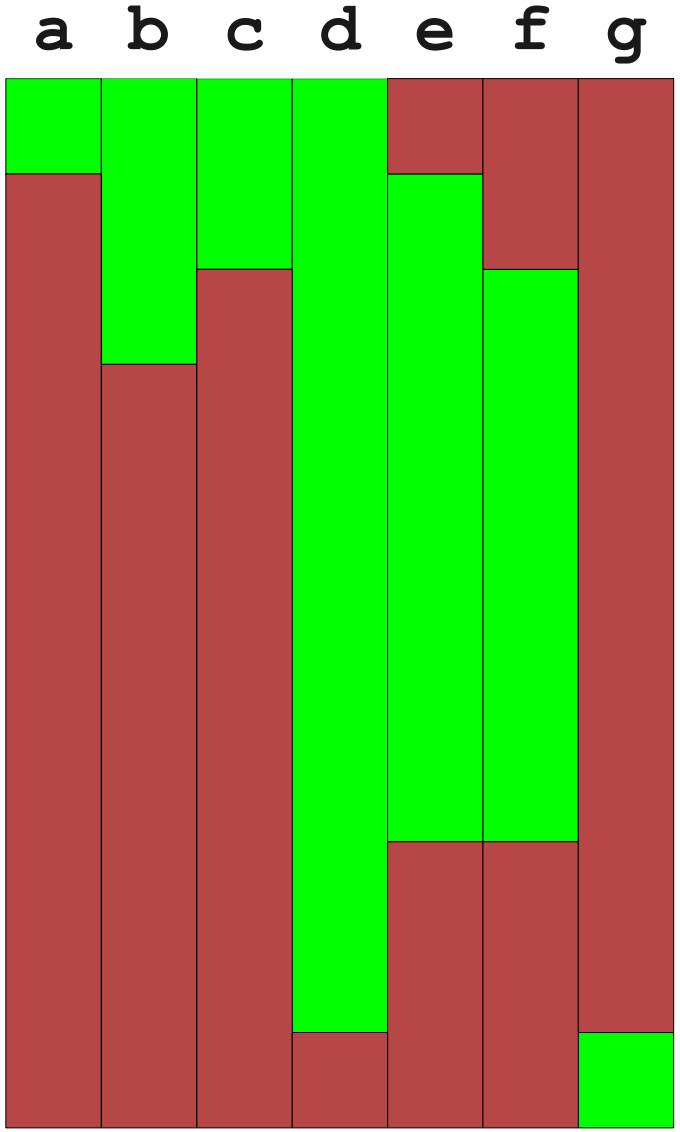
Live Ranges and Live Intervals



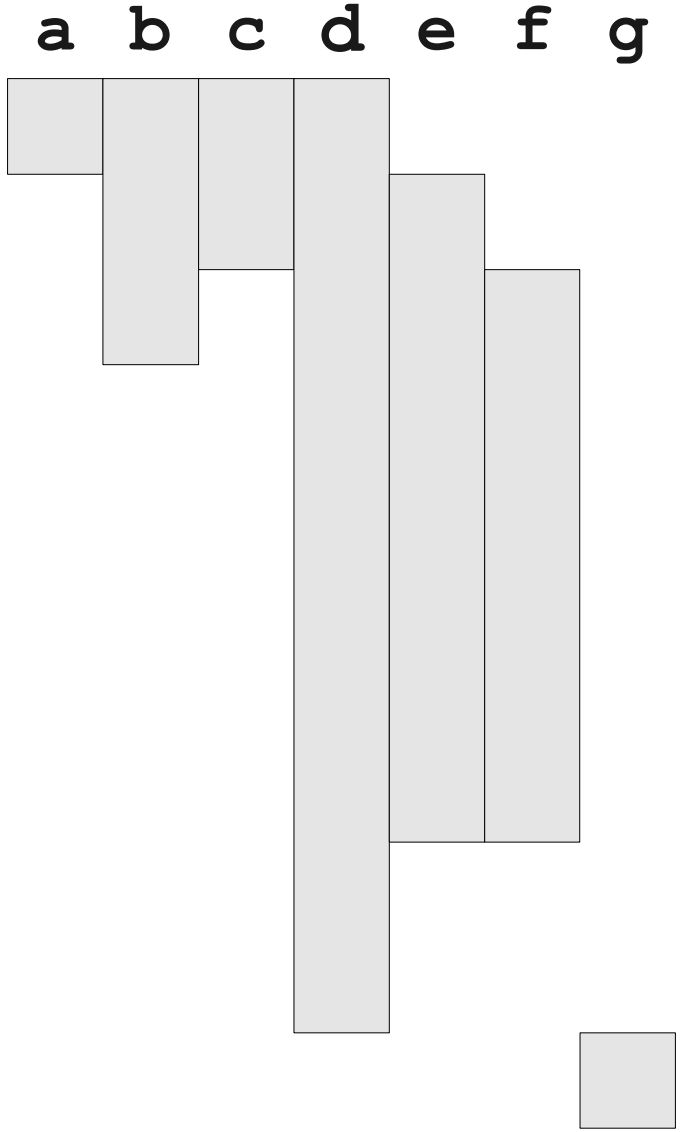


Register Allocation with Live Intervals

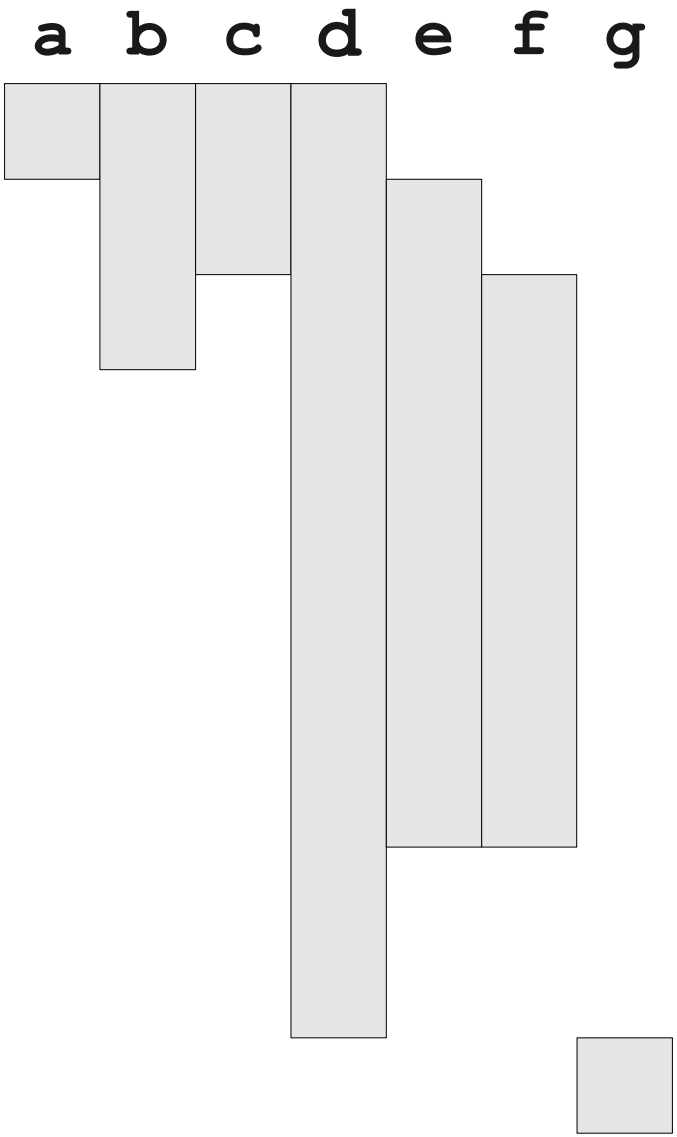
- Given the live intervals for all the variables in the program, we can allocate registers using a simple greedy algorithm.
- Idea: Track which registers are free at each point.
- When a live interval begins, give that variable a free register.
- When a live interval ends, the register is once again free.
- We can't always fit everything into a register; we'll see what do to in a minute.



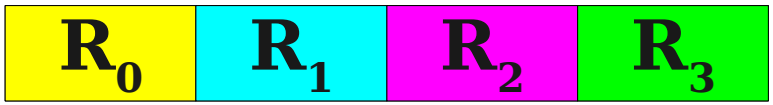
Register Allocation with Live Intervals



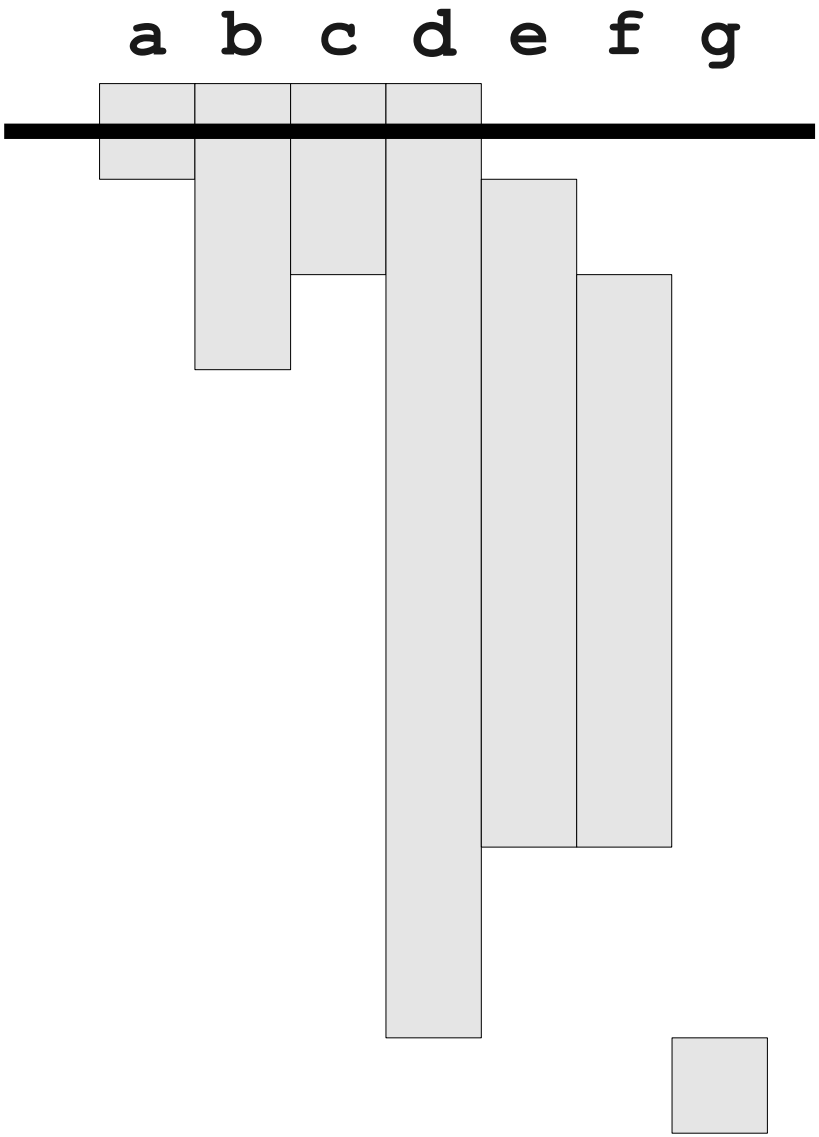
Register Allocation with Live Intervals



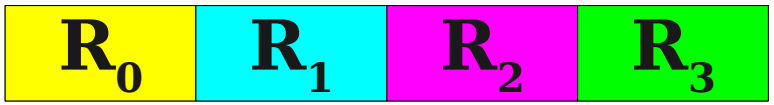
Free Registers



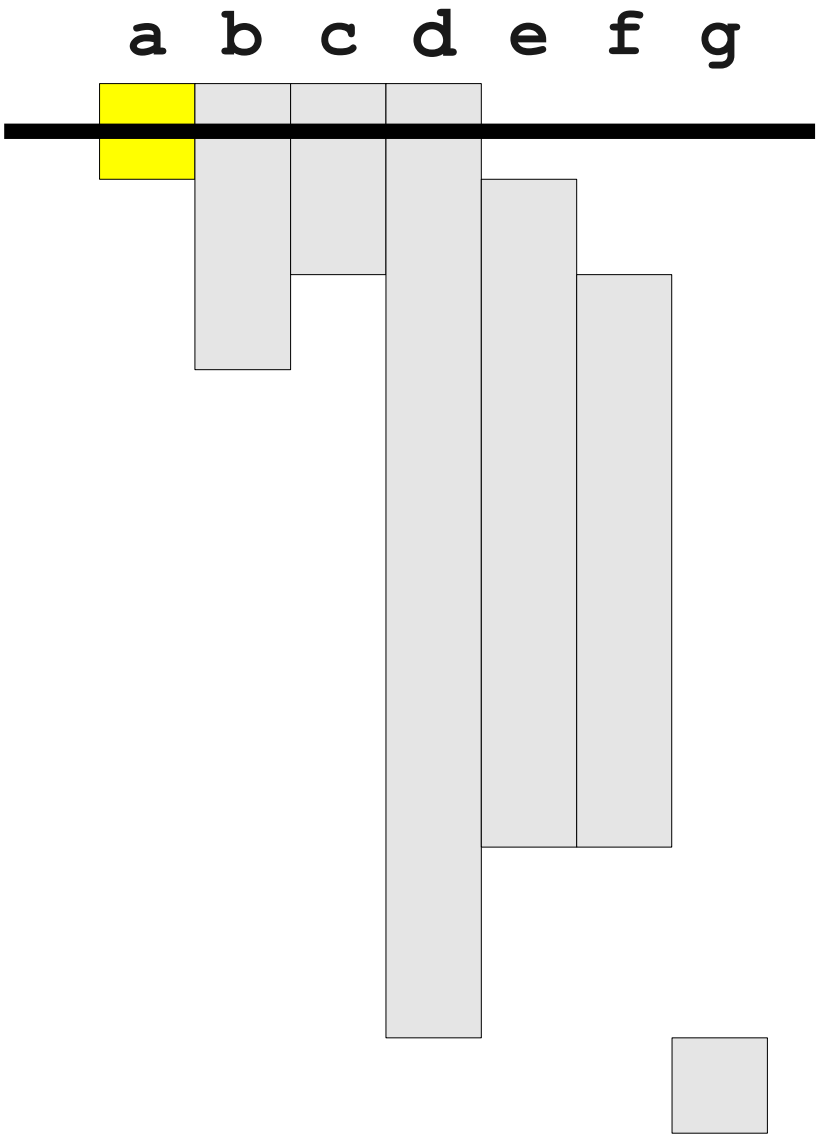
Register Allocation with Live Intervals



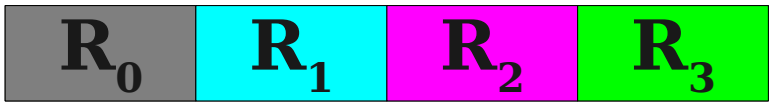
Free Registers



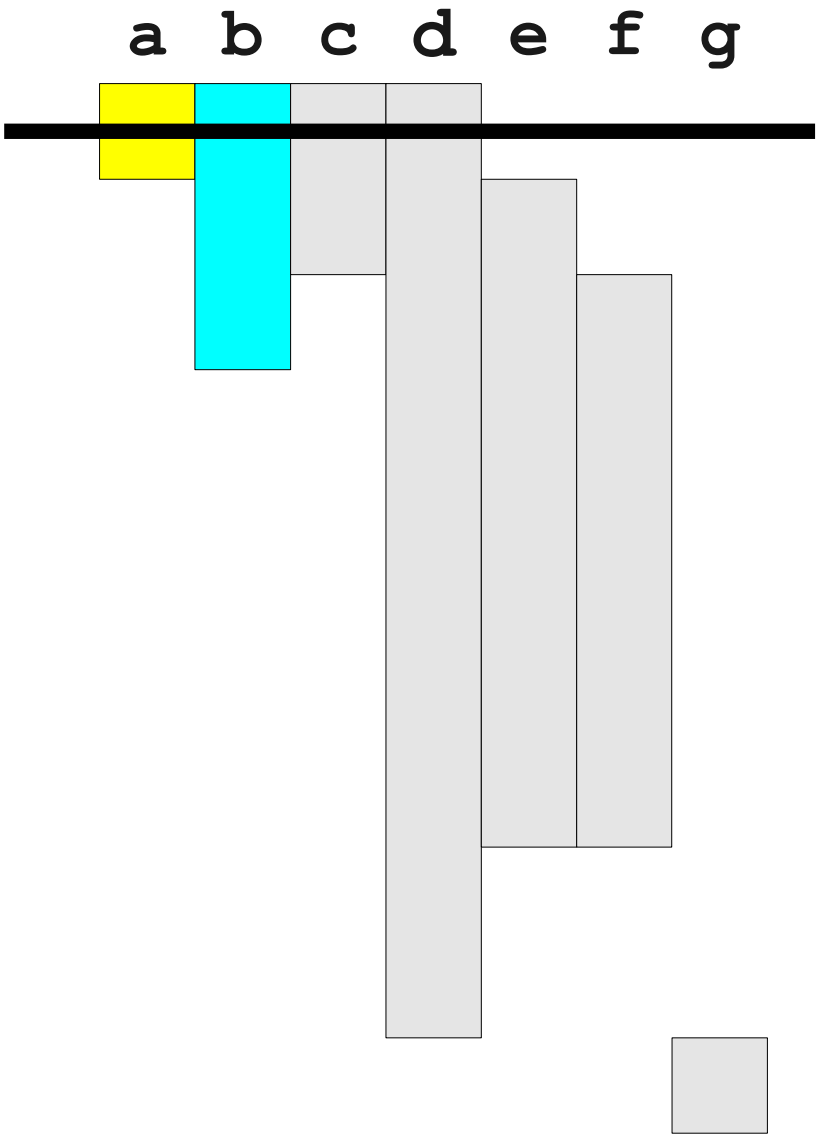
Register Allocation with Live Intervals



Free Registers

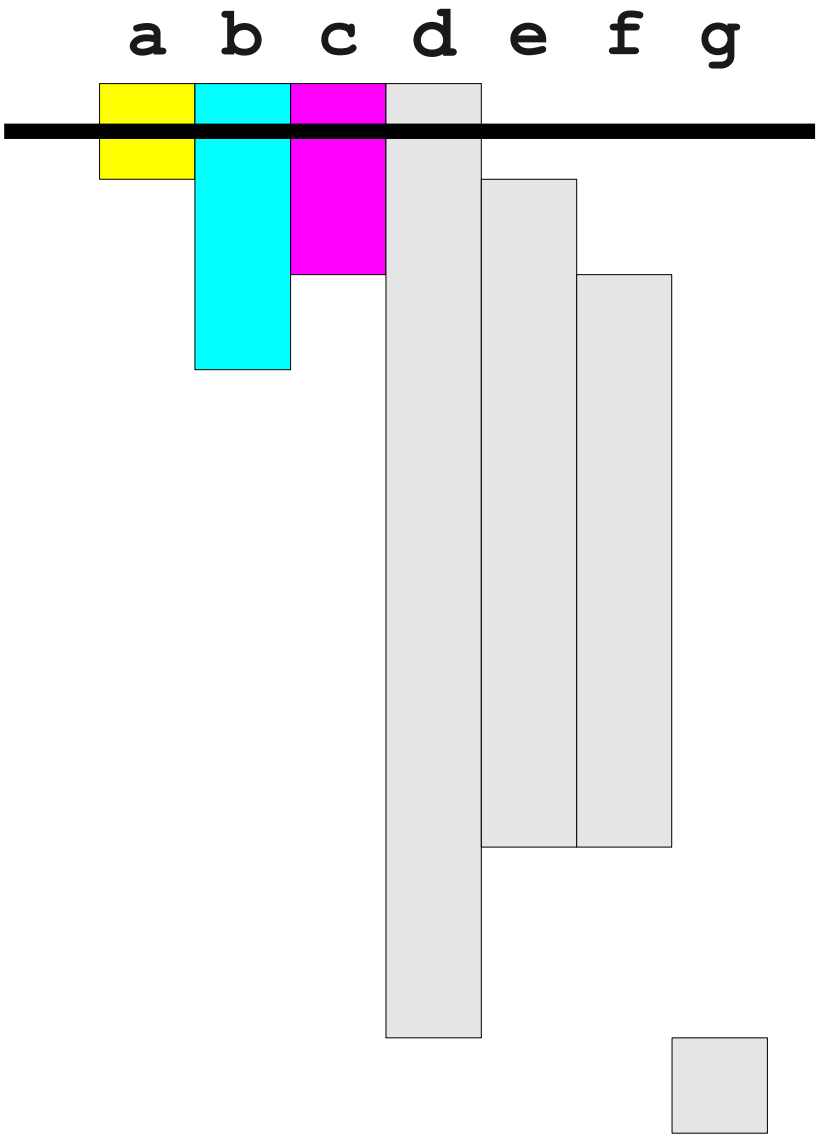


Register Allocation with Live Intervals



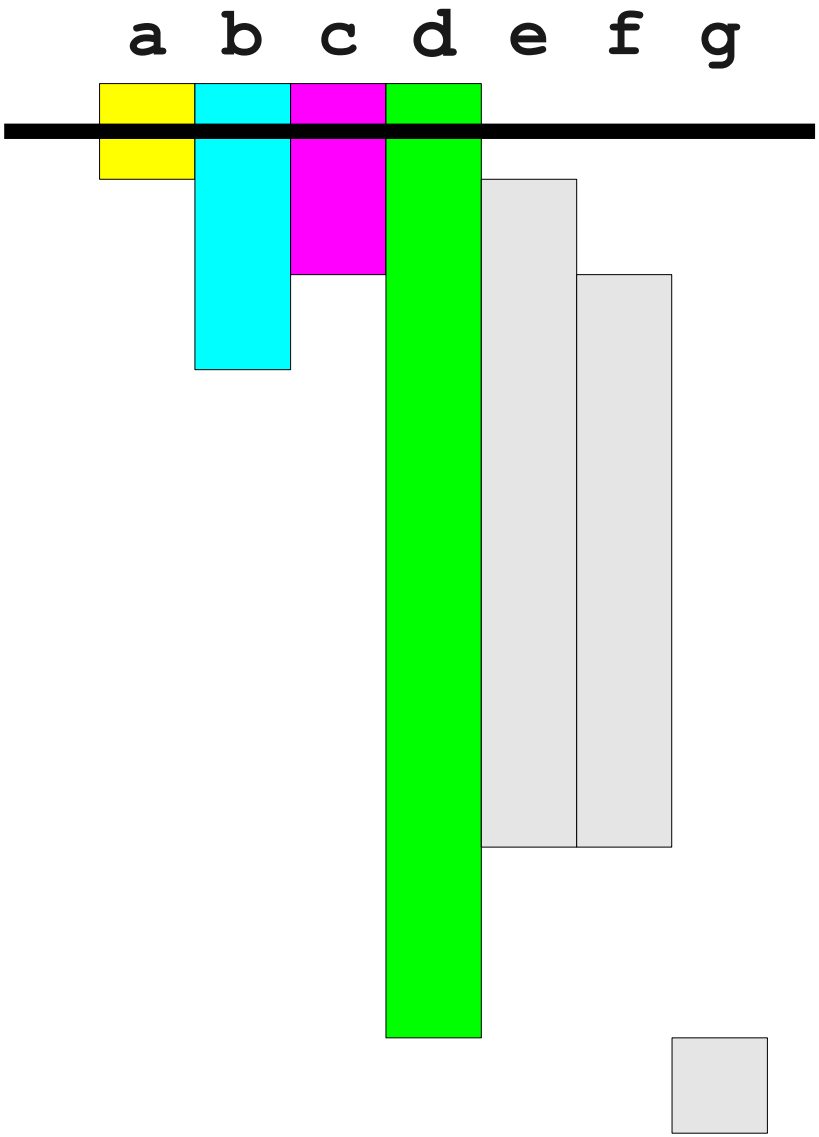
Free Registers

Register Allocation with Live Intervals



Free Registers

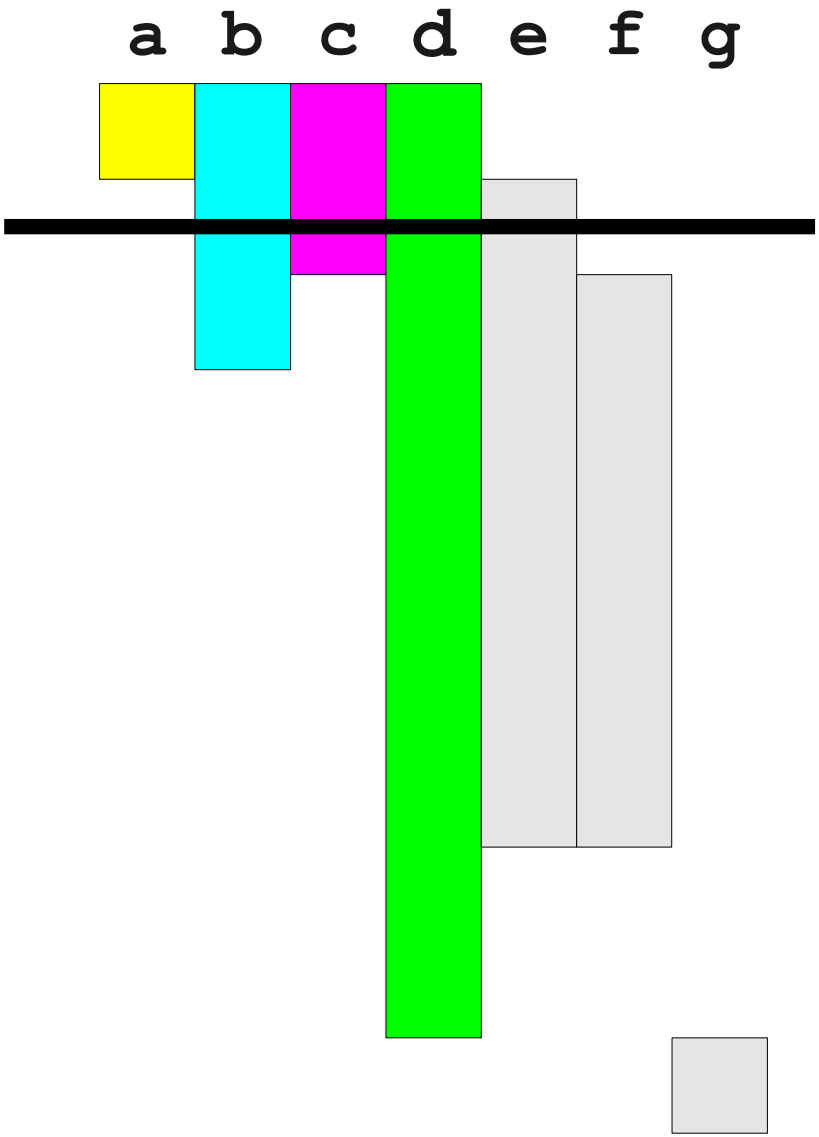
Register Allocation with Live Intervals



Free Registers

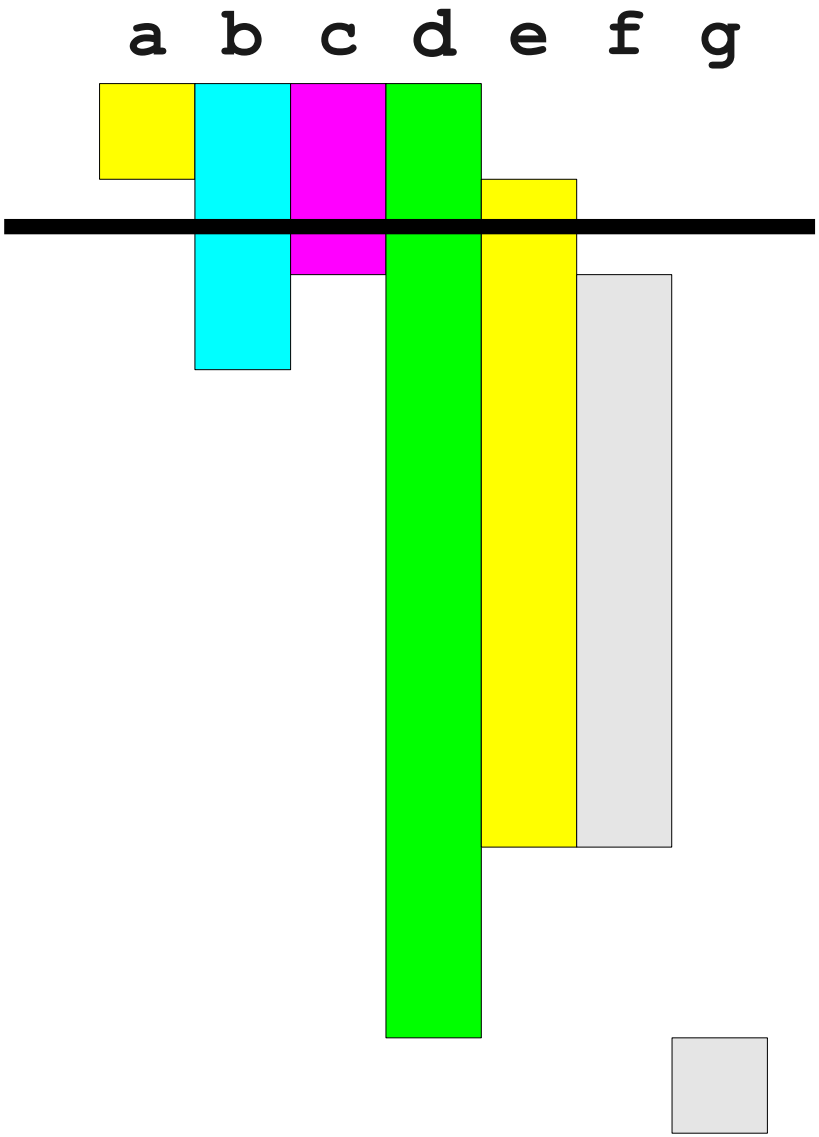
R_0	R_1	R_2	R_2

Register Allocation with Live Intervals



Free Registers

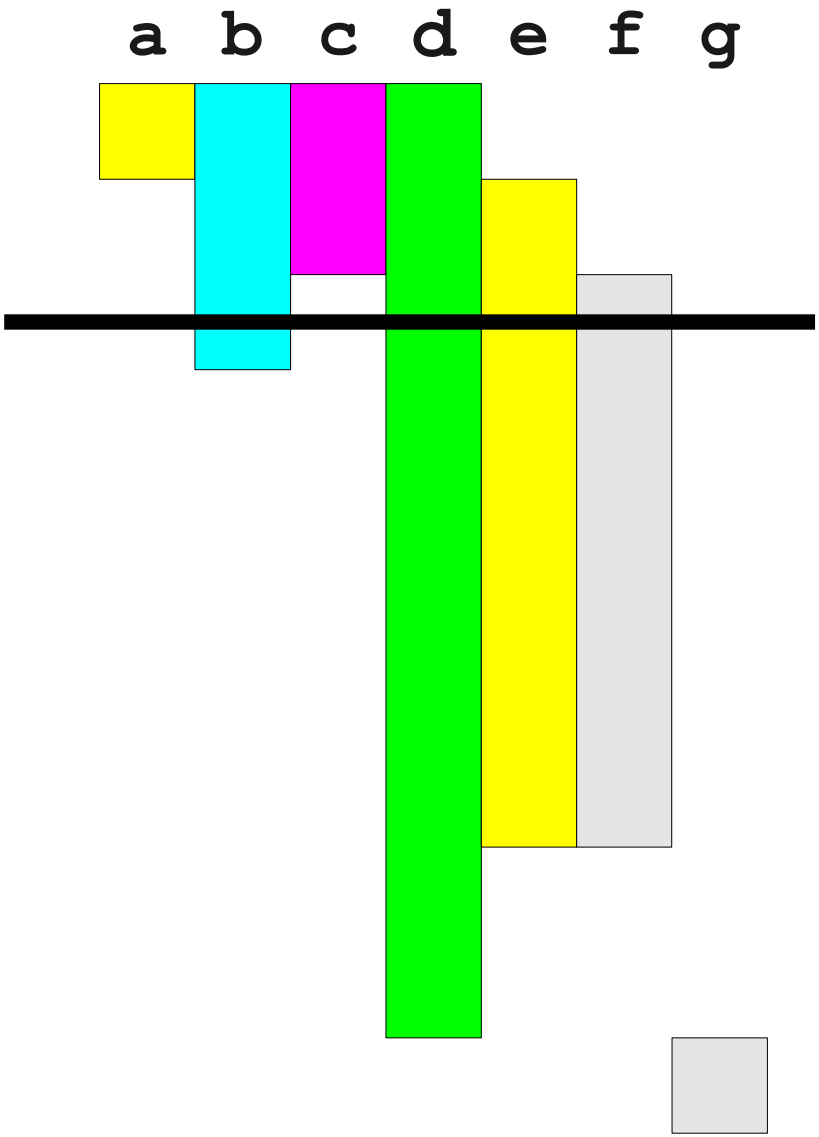
Register Allocation with Live Intervals



Free Registers

R_0	R_1	R_2	R_2

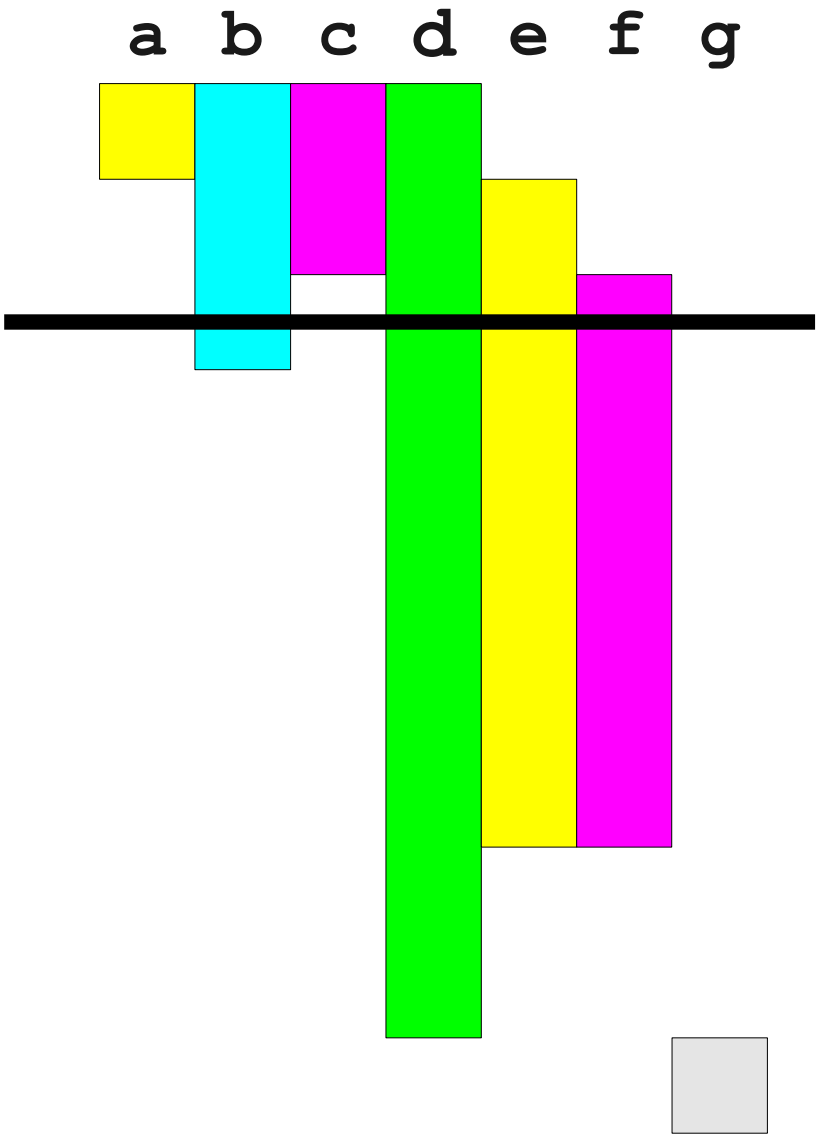
Register Allocation with Live Intervals



Free Registers

R_0	R_1	R_2	R_2

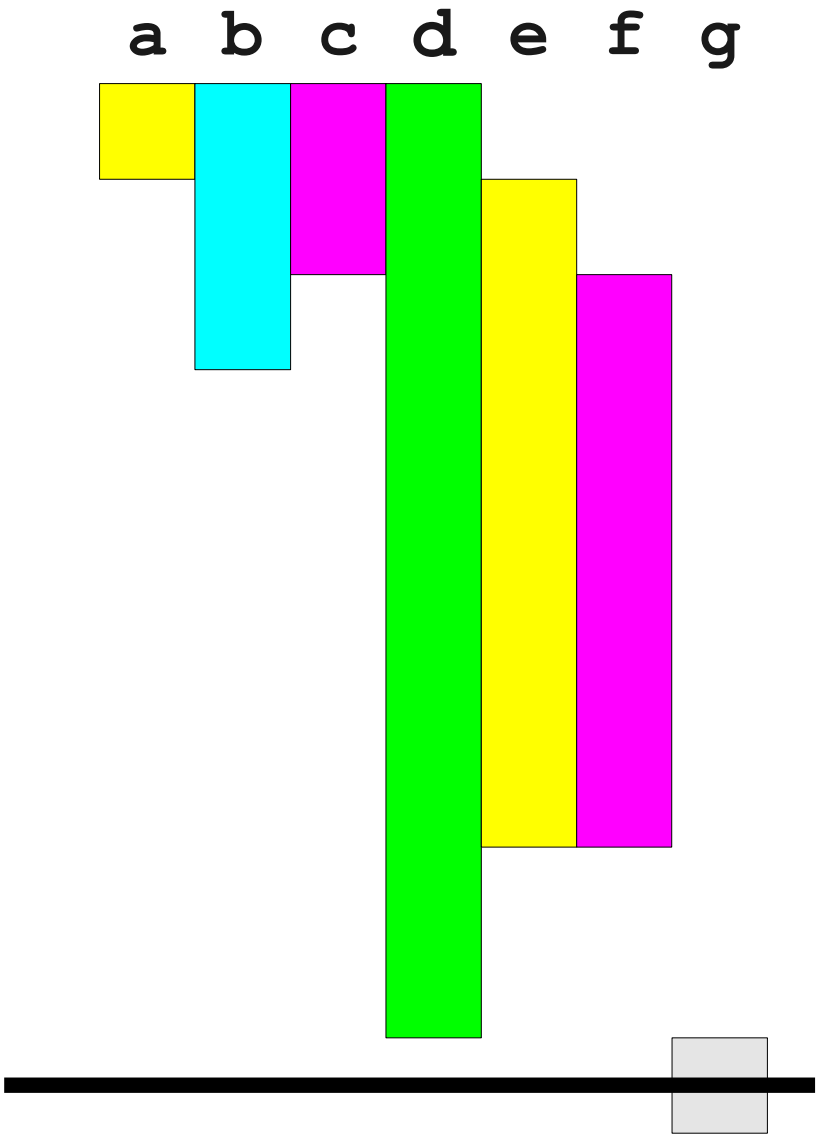
Register Allocation with Live Intervals



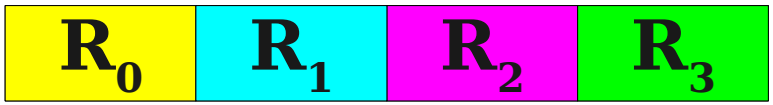
Free Registers

R_0	R_1	R_2	R_2

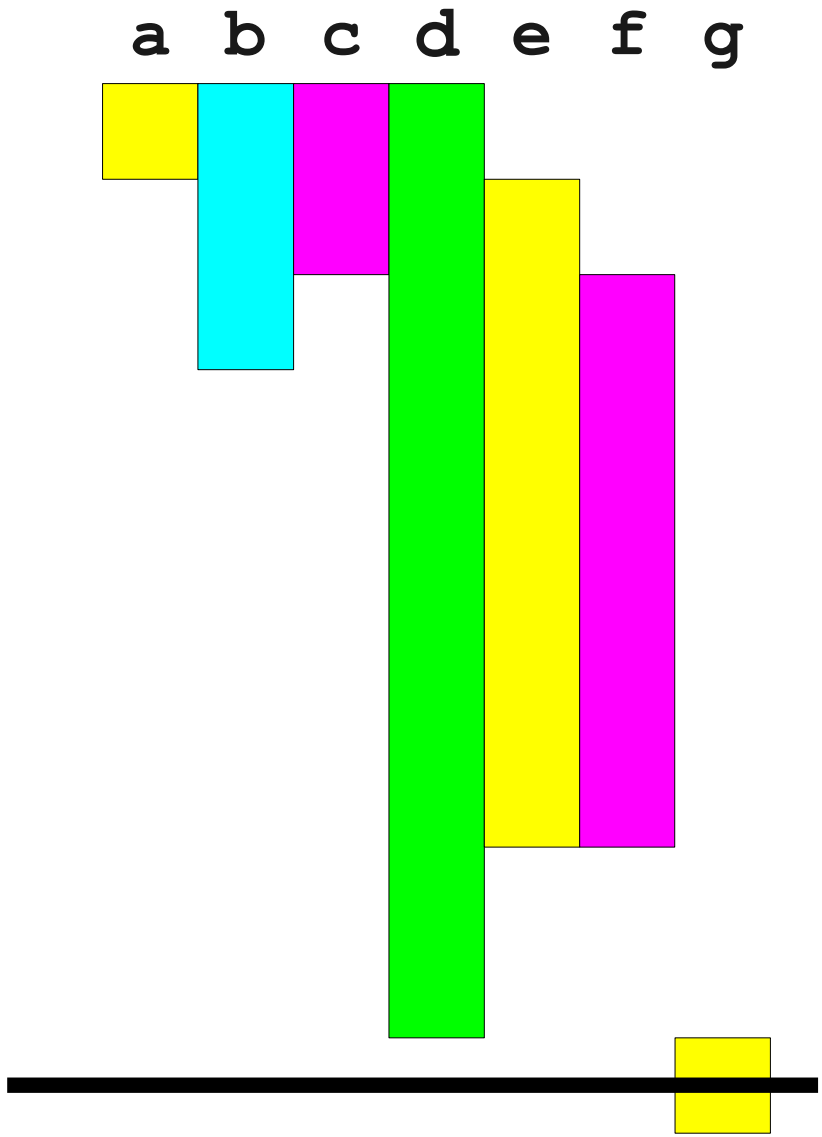
Register Allocation with Live Intervals



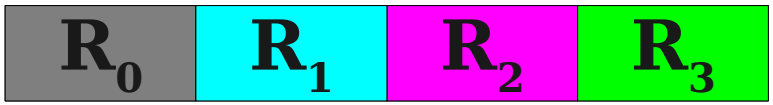
Free Registers



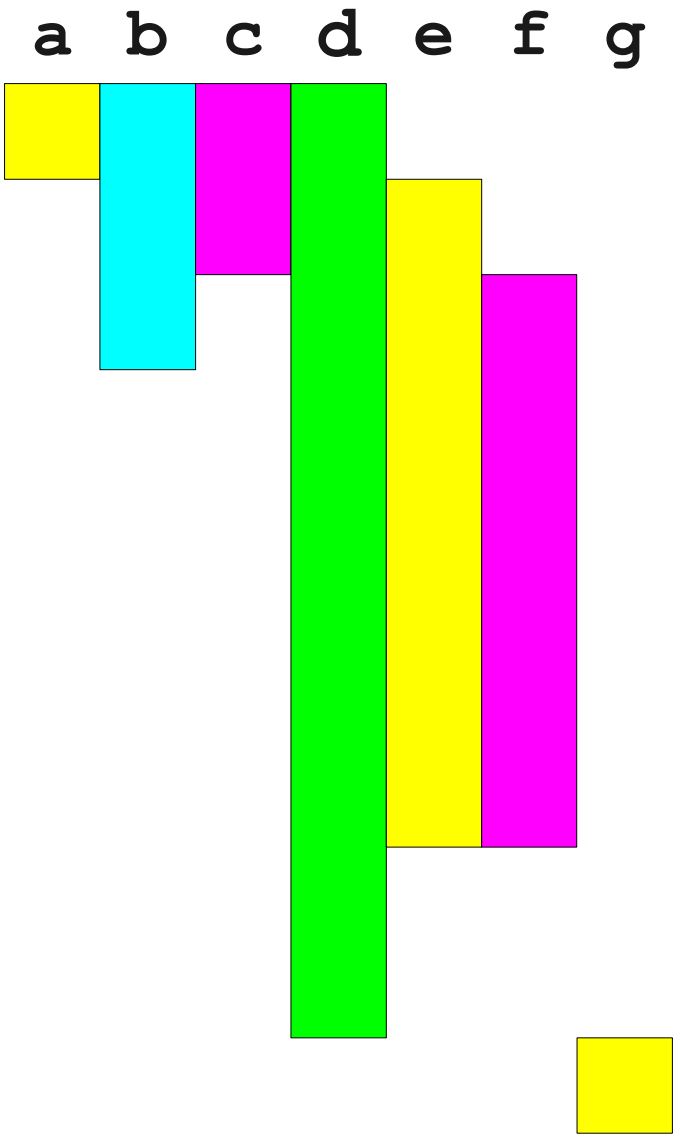
Register Allocation with Live Intervals



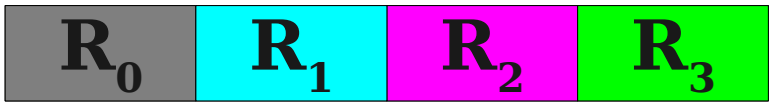
Free Registers



Register Allocation with Live Intervals

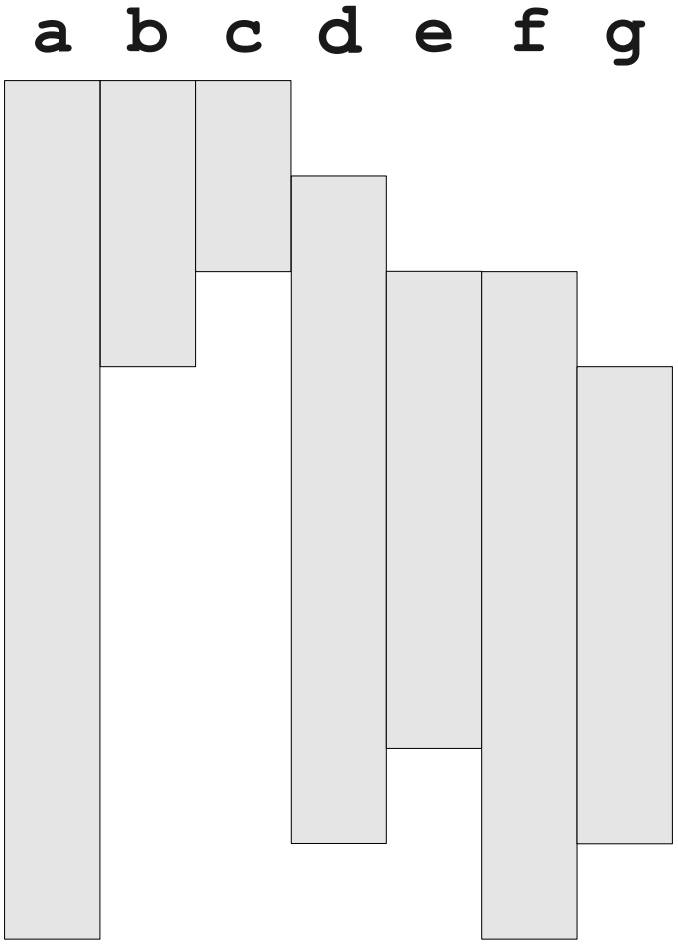


Free Registers

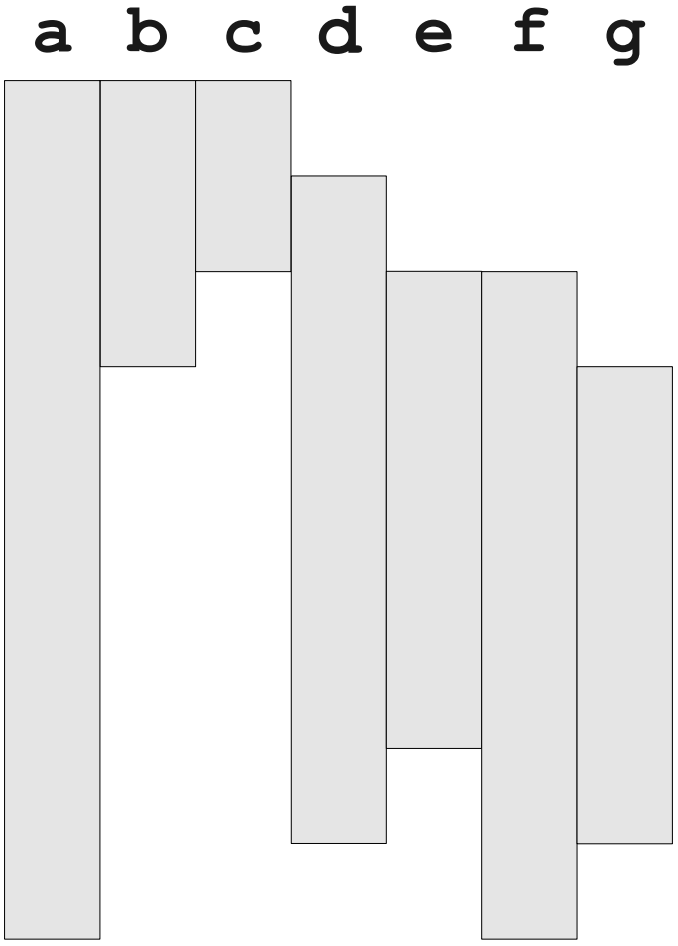


Another Example

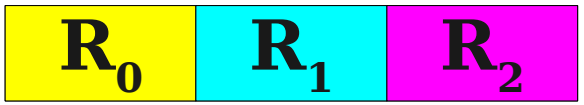
Another Example



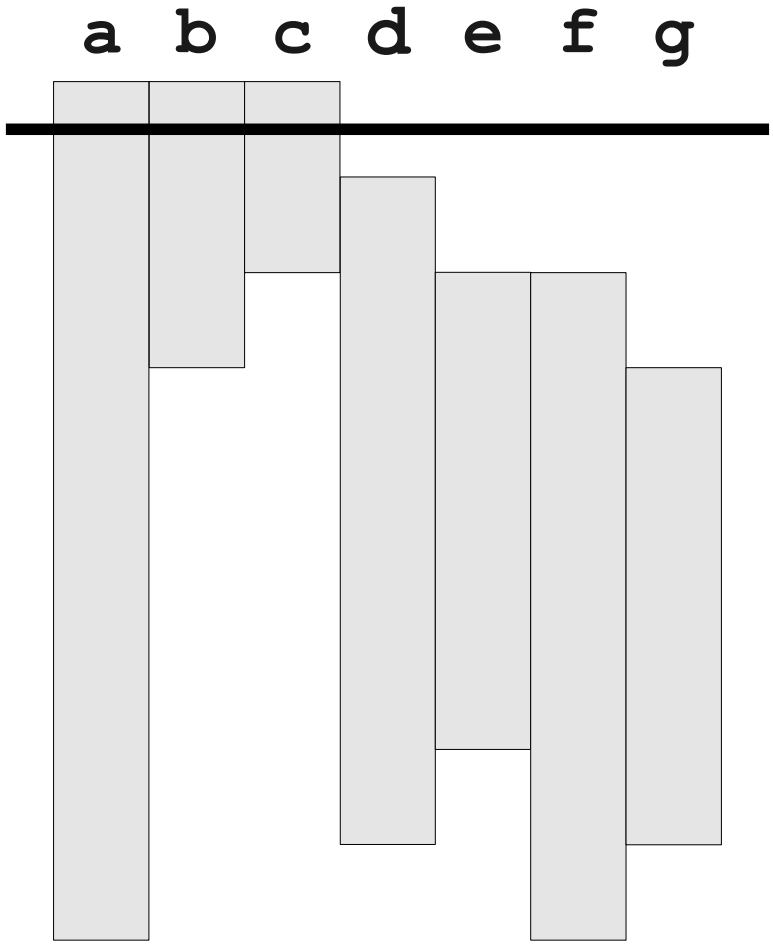
Another Example



Free Registers



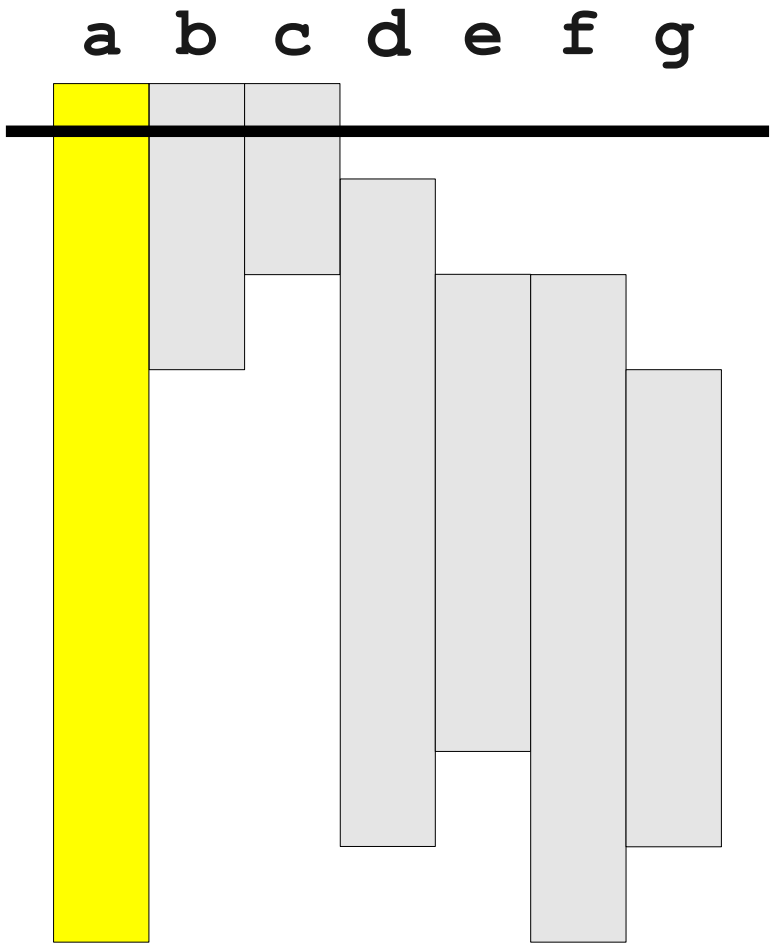
Another Example



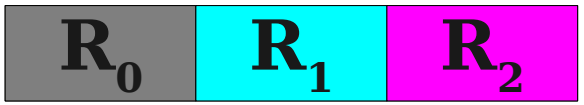
Free Registers

R_0	R_1	R_2

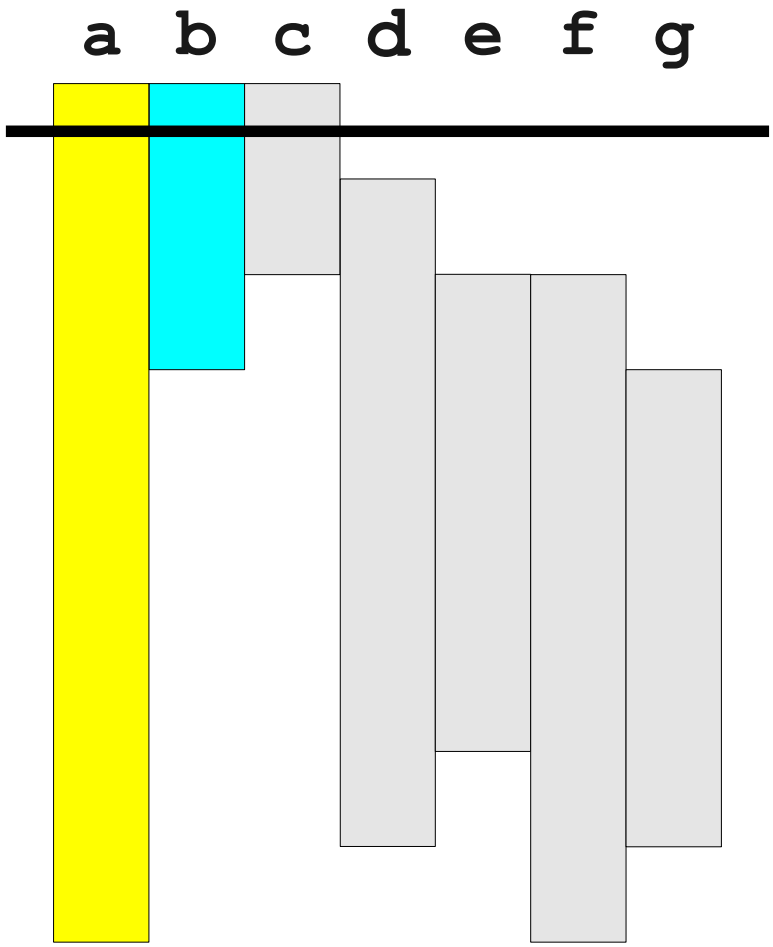
Another Example



Free Registers

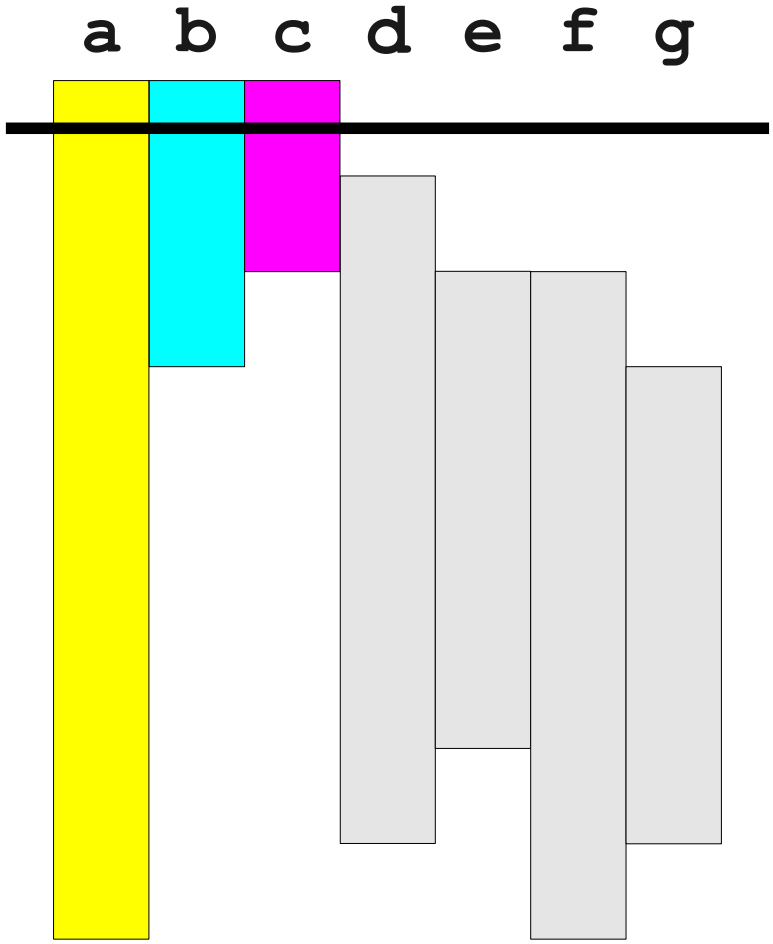


Another Example



Free Registers

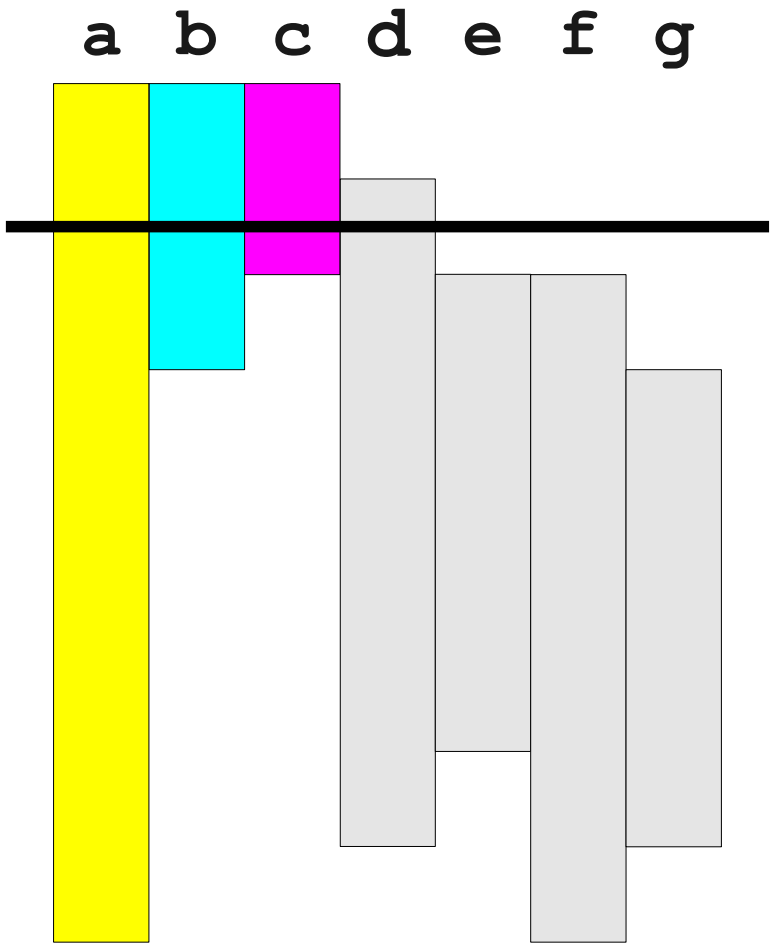
Another Example

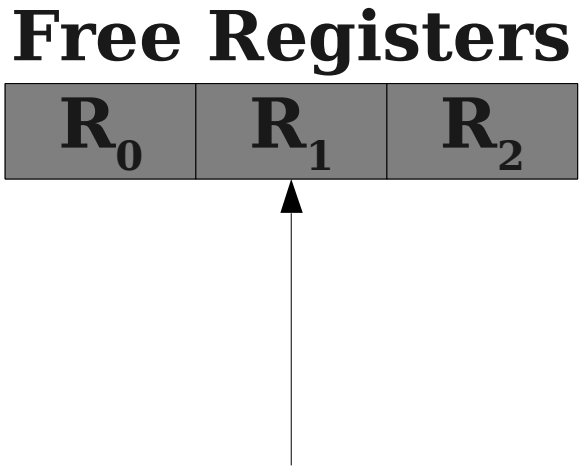


Free Registers

R_0	R_1	R_2

Another Example



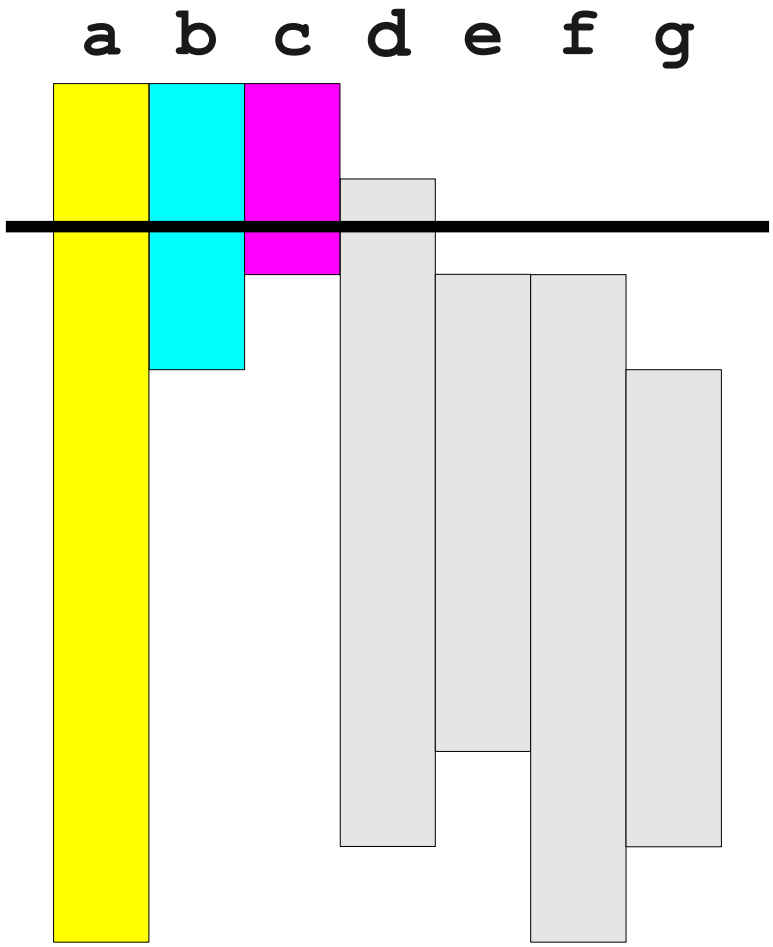


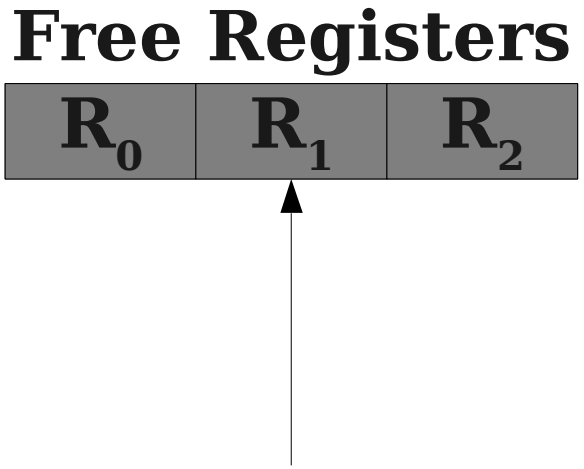
What do we do now?

Register Spilling

- If a register cannot be found for a variable v, we may need to **spill** a variable.
- When a variable is spilled, it is stored in memory rather than a register.
- When we need a register for the spilled variable:
  - Evict some existing register to memory.
  - Load the variable into the register.
  - When done, write the register back to memory and reload the register with its original value.
- Spilling is slow, but sometimes necessary.

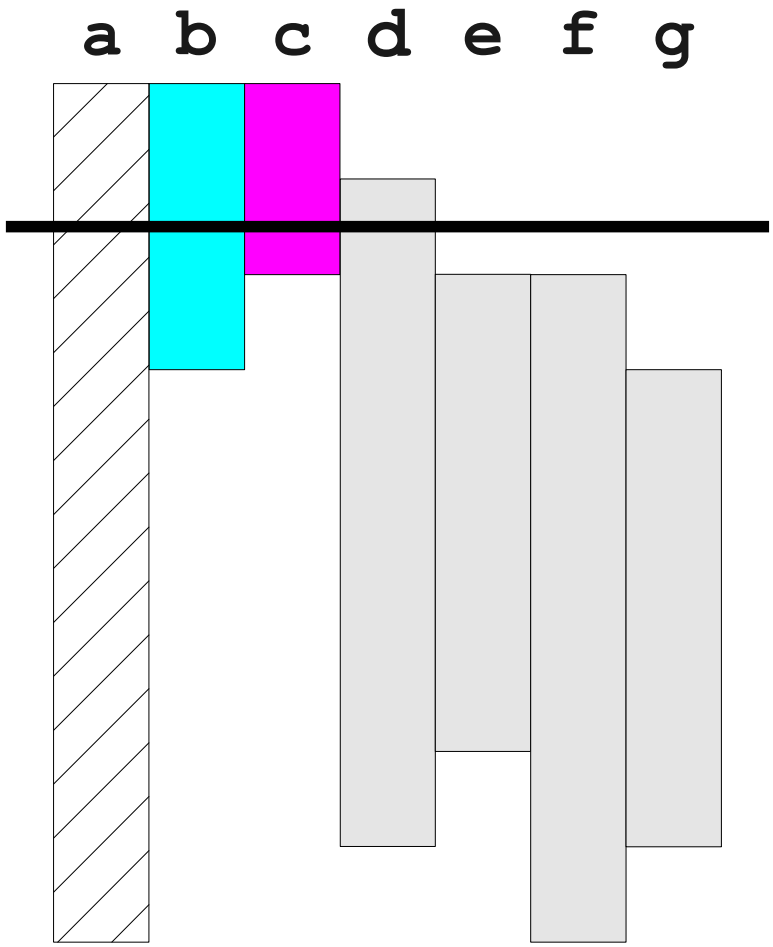
Another Example





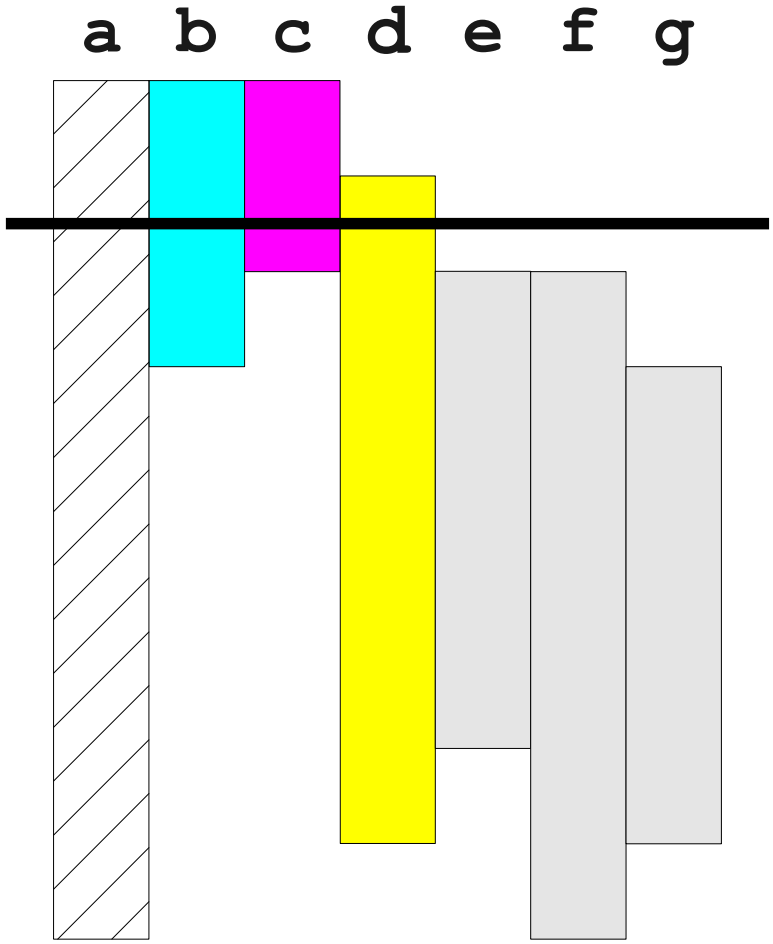
What do we do now?

Another Example



Free Registers

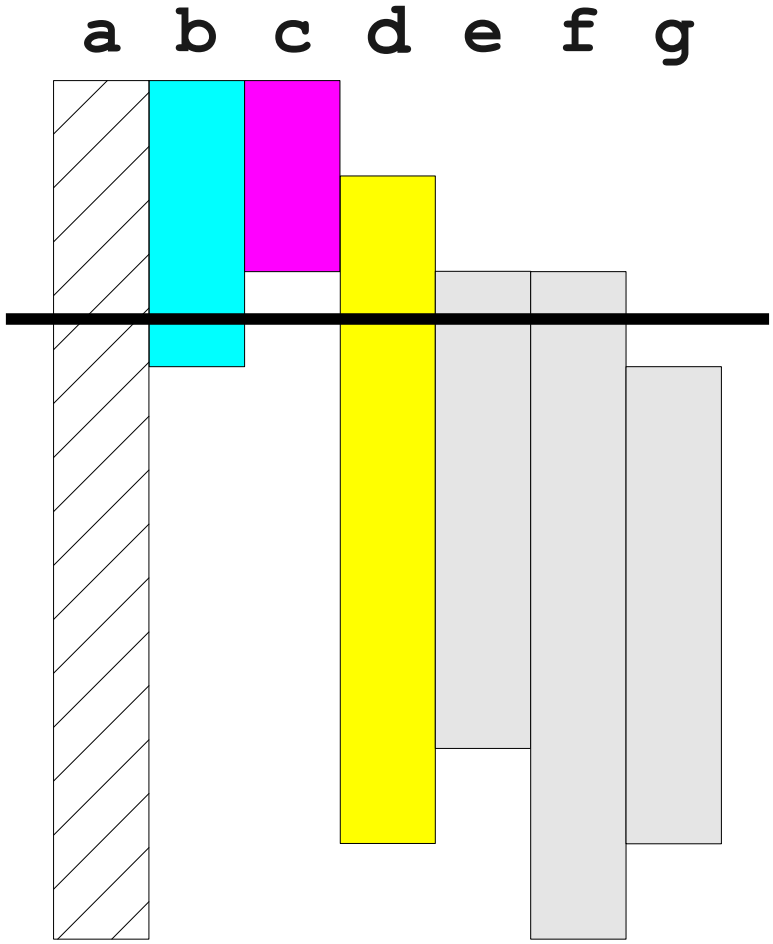
Another Example



Free Registers

R_0	R_1	R_2

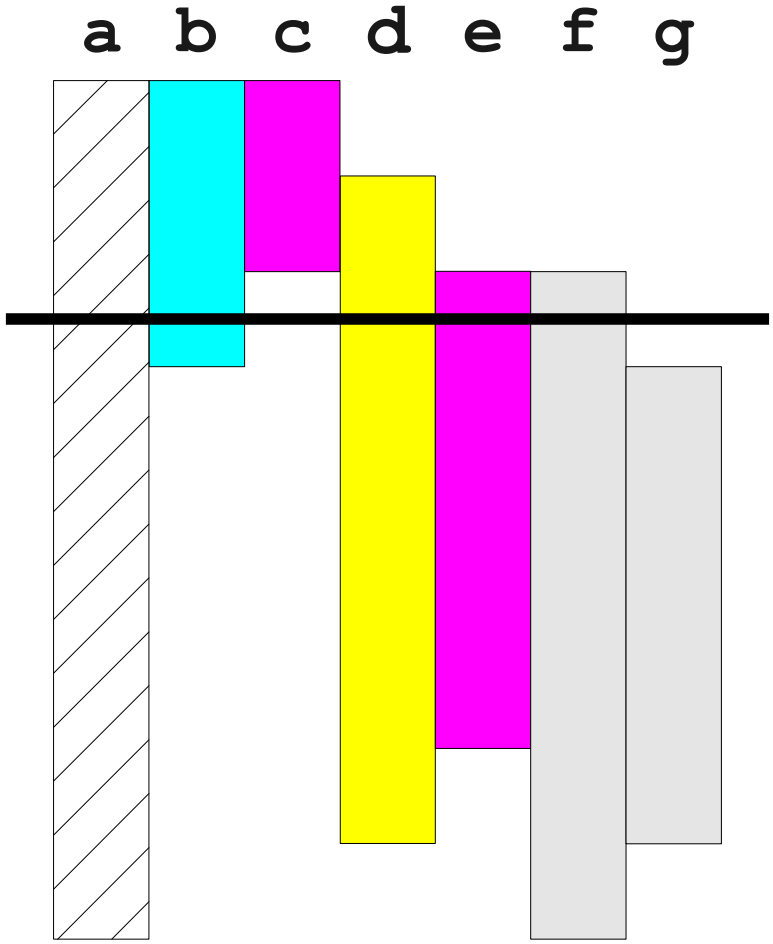
Another Example



Free Registers

R_0	R_1	R_2

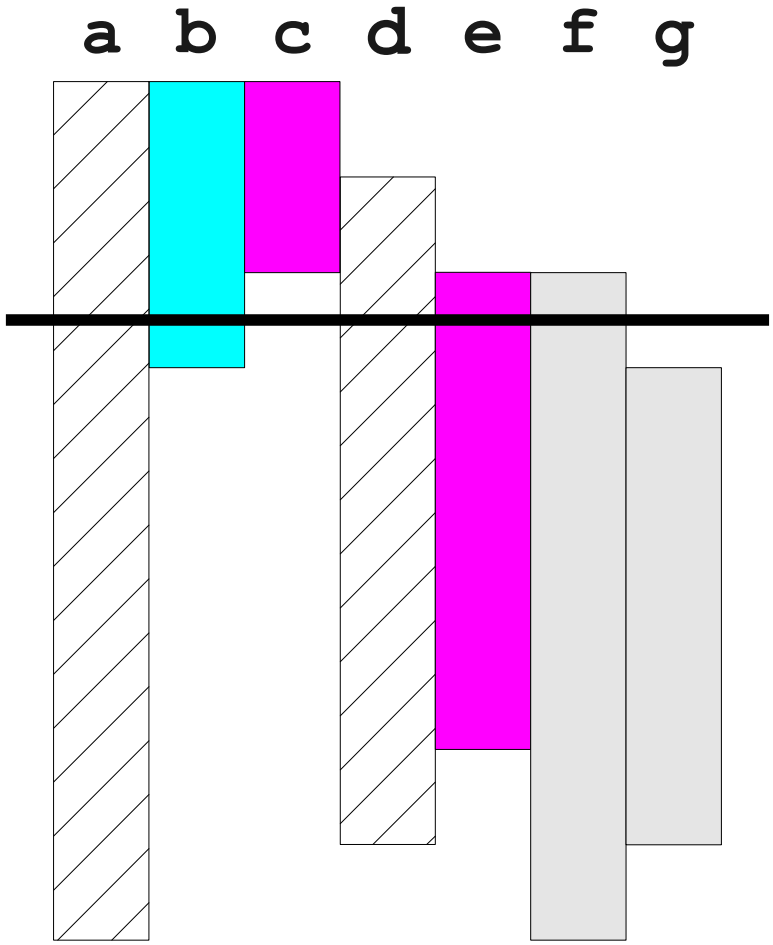
Another Example



Free Registers

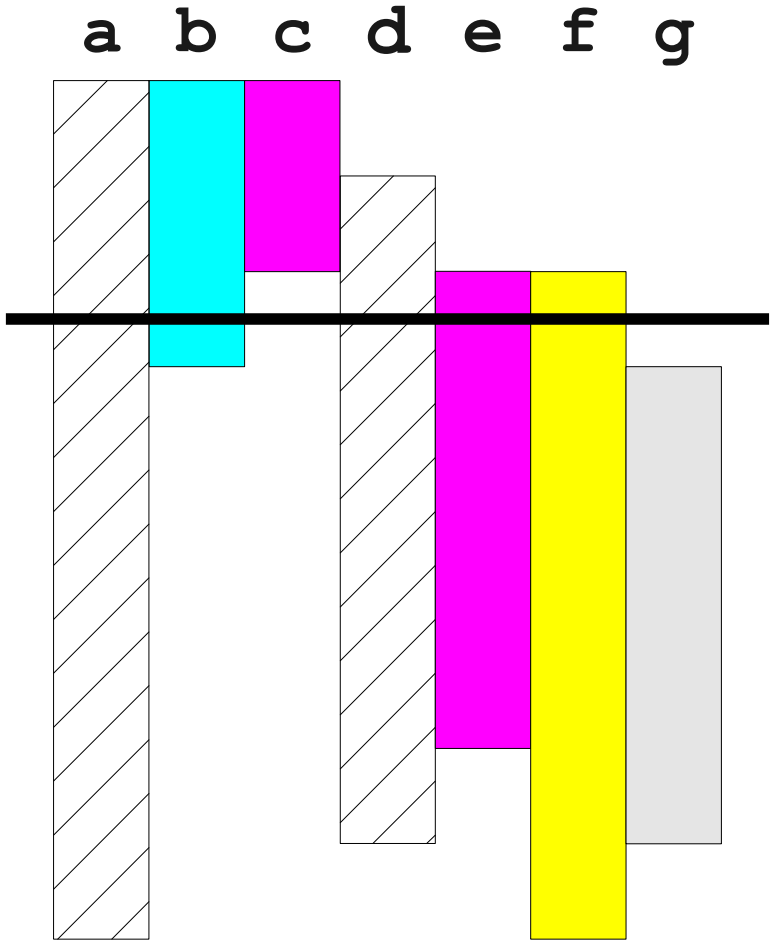
R_0	R_1	R_2

Another Example



Free Registers

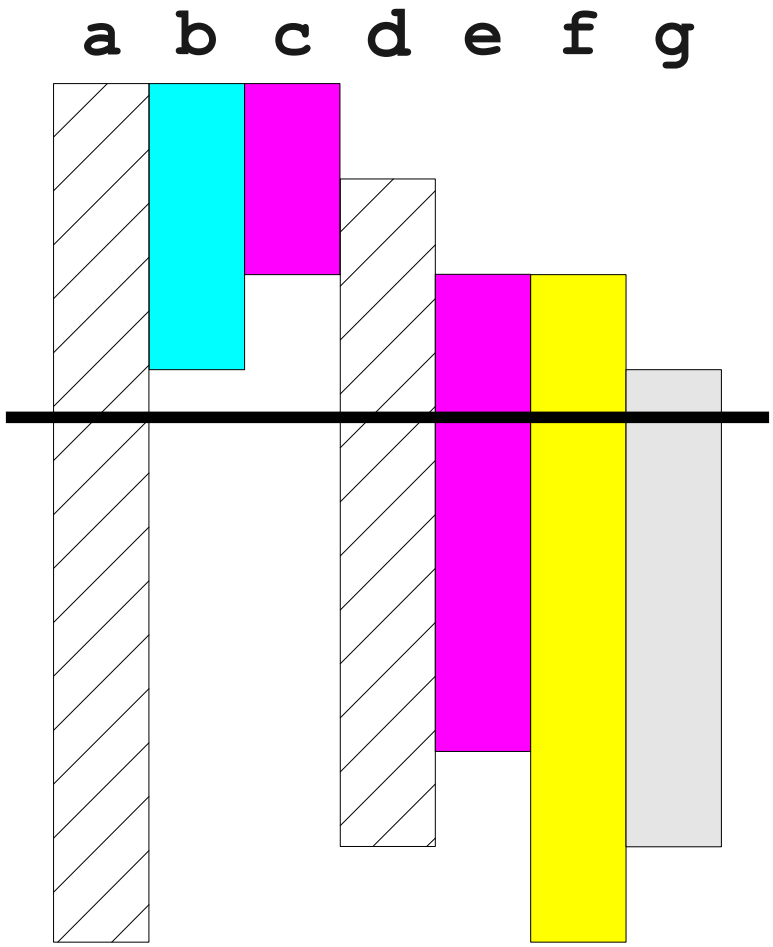
Another Example



Free Registers

R_0	R_1	R_2

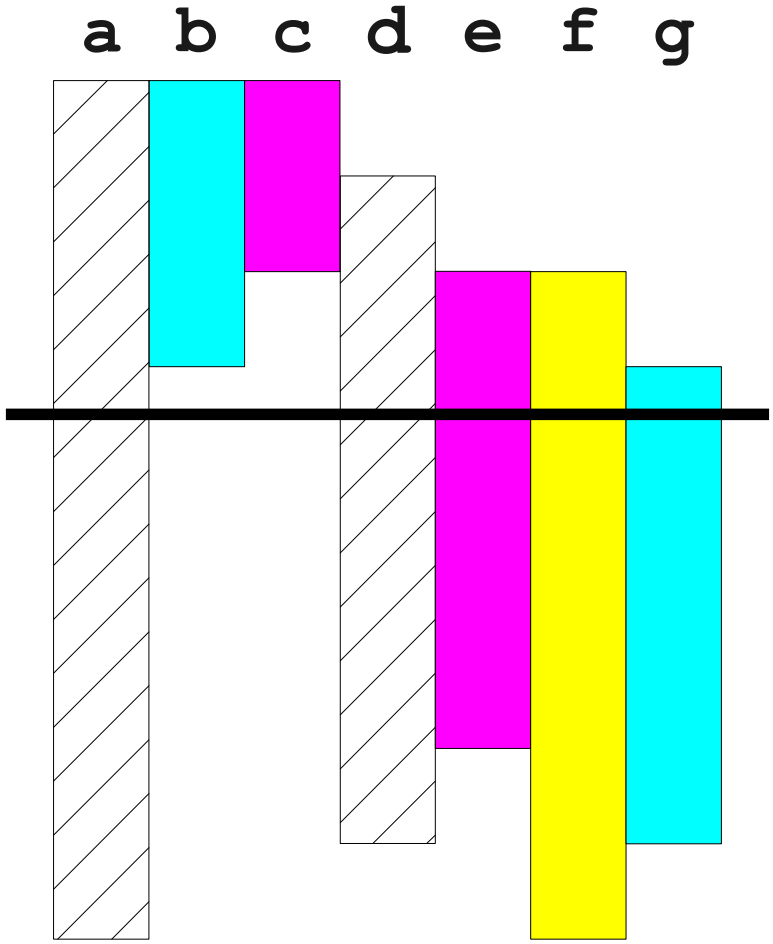
Another Example



Free Registers

R_0	R_1	R_2

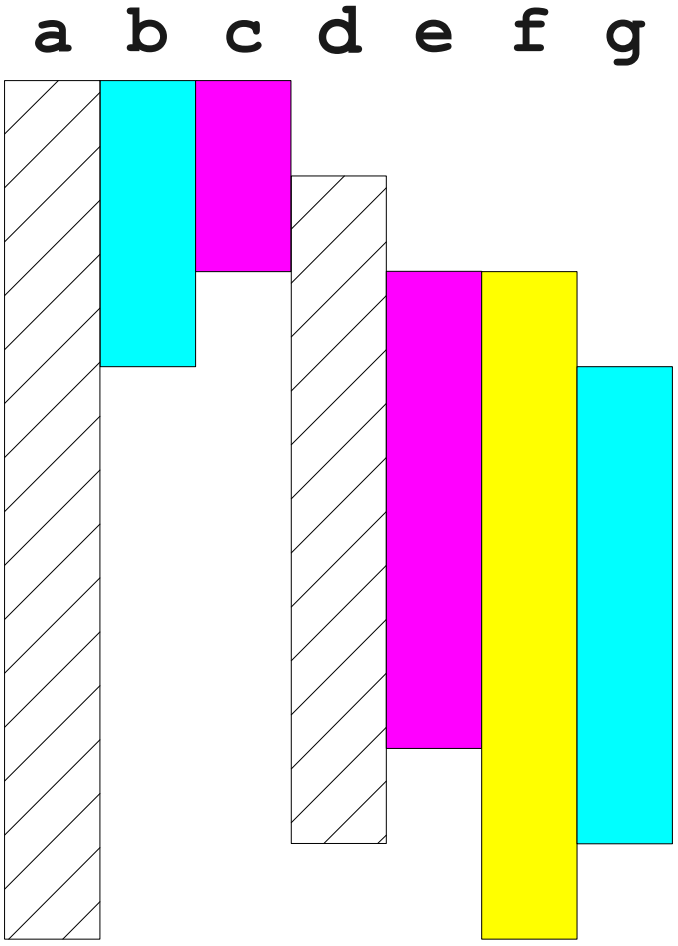
Another Example



Free Registers

R_0	R_1	R_2

Another Example



Free Registers

R_0	R_1	R_2

Linear Scan Register Allocation

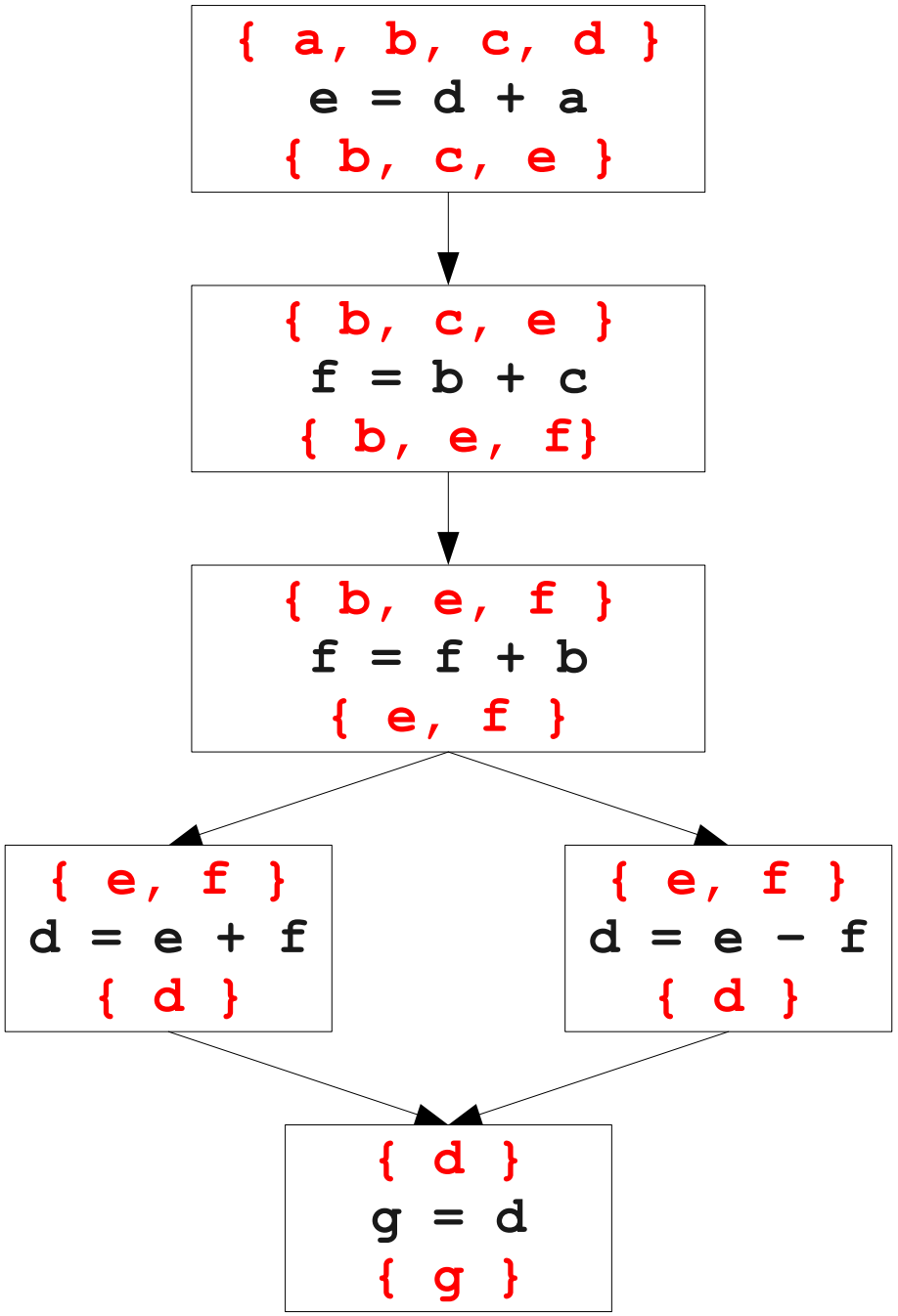
- This algorithm is called **linear scan register allocation** and is a comparatively new algorithm.
- Advantages:
  - Very efficient (after computing live intervals, runs in linear time)
  - Produces good code in many instances.
  - Allocation step works in one pass; can generate code during iteration.
  - Often used in JIT compilers like Java HotSpot.
- Disadvantages:
  - Imprecise due to use of live **intervals** rather than live **ranges**.
  - Other techniques known to be superior in many cases.

Second-Chance Bin Packing

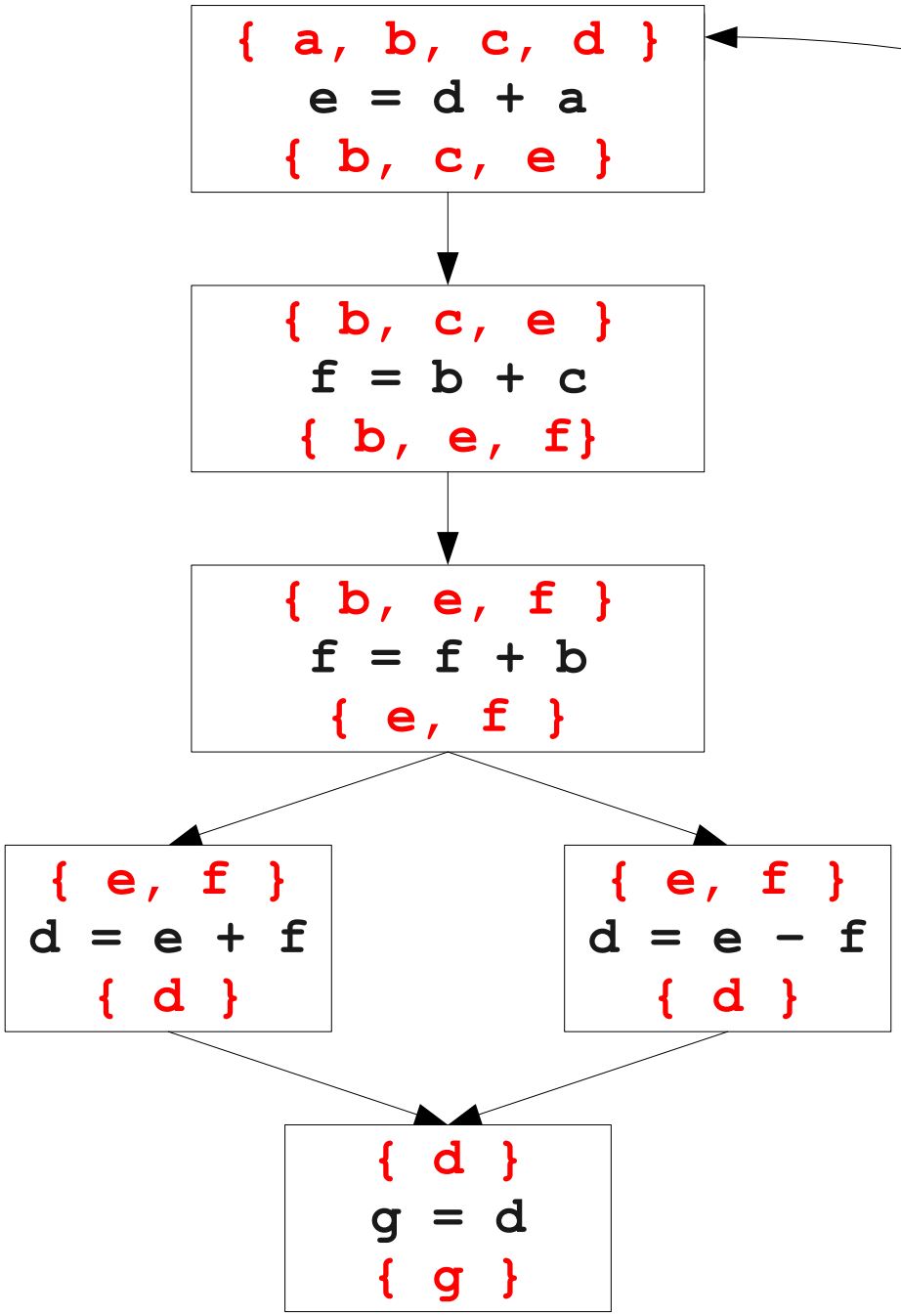
- A more aggressive version of linear-scan.
- Uses live **ranges** instead of live **intervals**.
- If a variable must be spilled, don't spill all uses of it.
  - A later live range might still fit into a register.
- Requires a final data-flow analysis to confirm variables are assigned consistent locations.
- See “Quality and Speed in Linear-scan Register Allocation” by Traub, Holloway, and Smith.

An Entirely Different Approach

An Entirely Different Approach

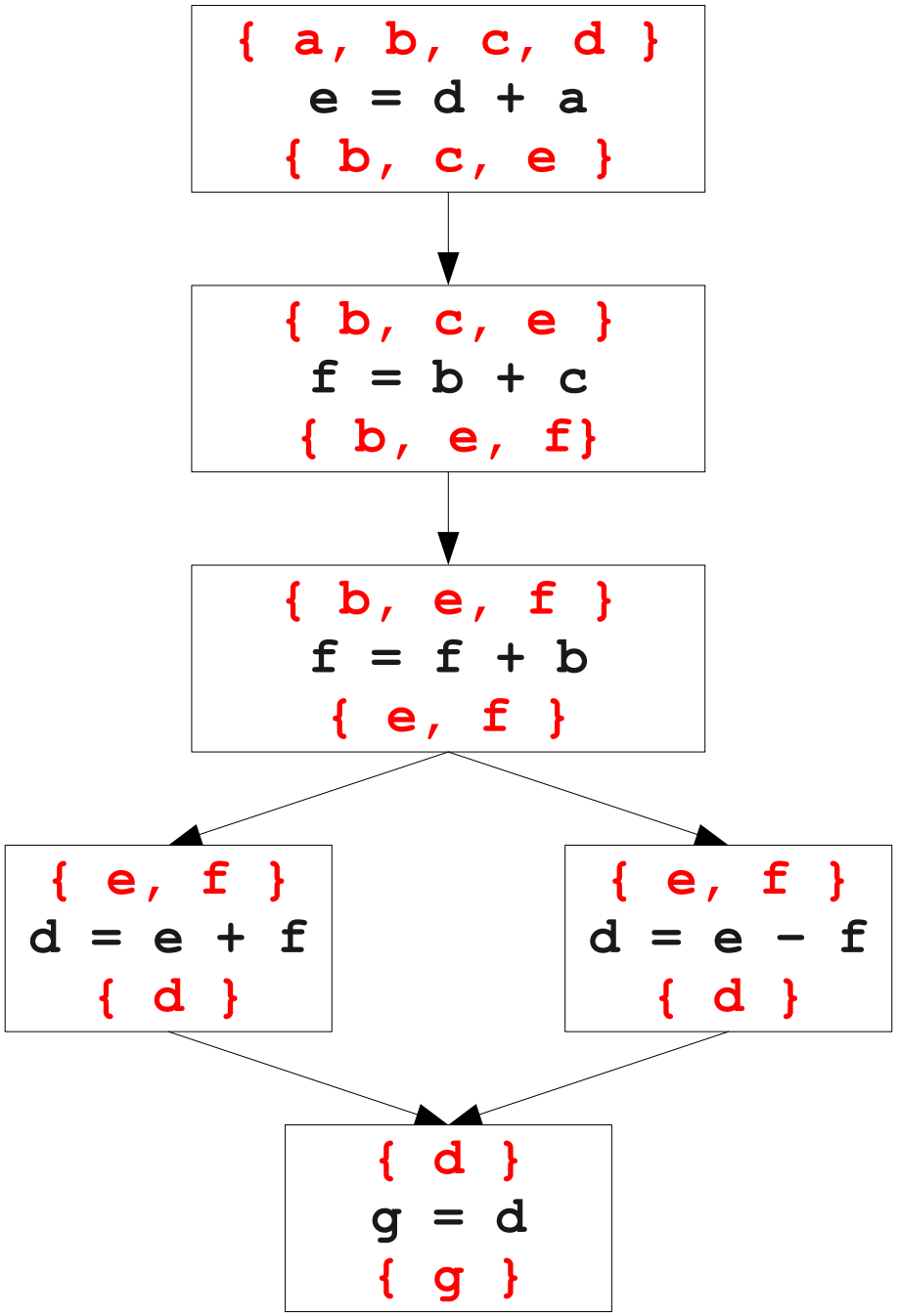


An Entirely Different Approach

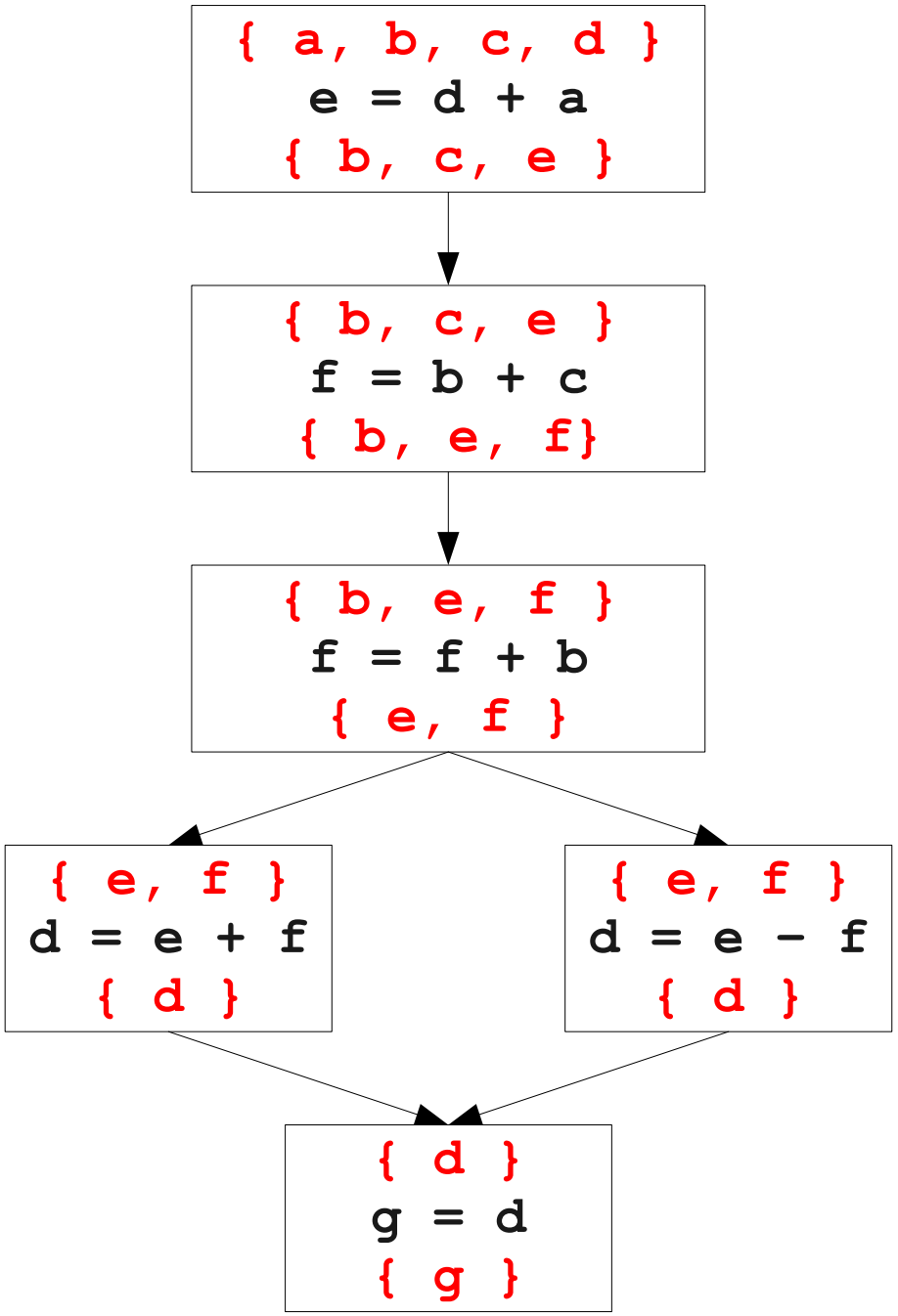


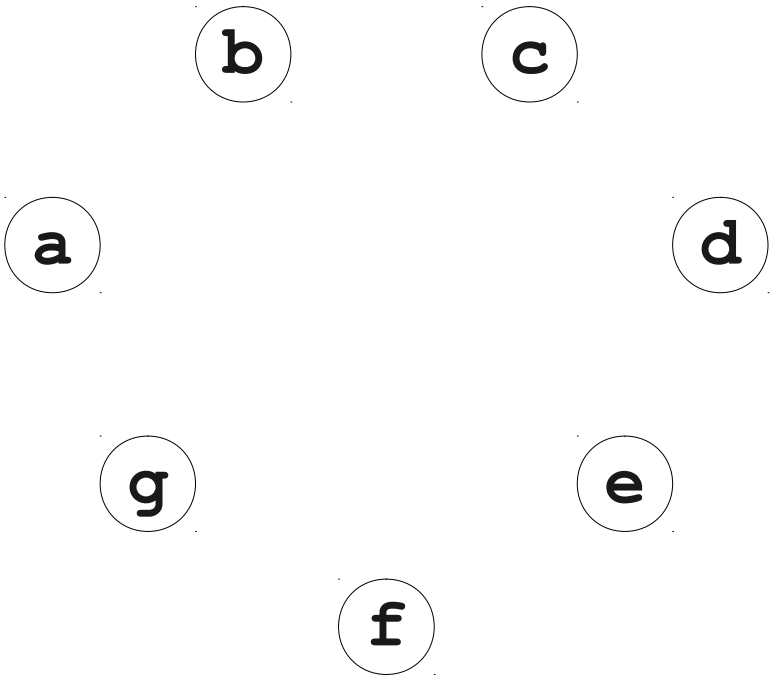
What can we infer from all these variables being live at this point?

An Entirely Different Approach

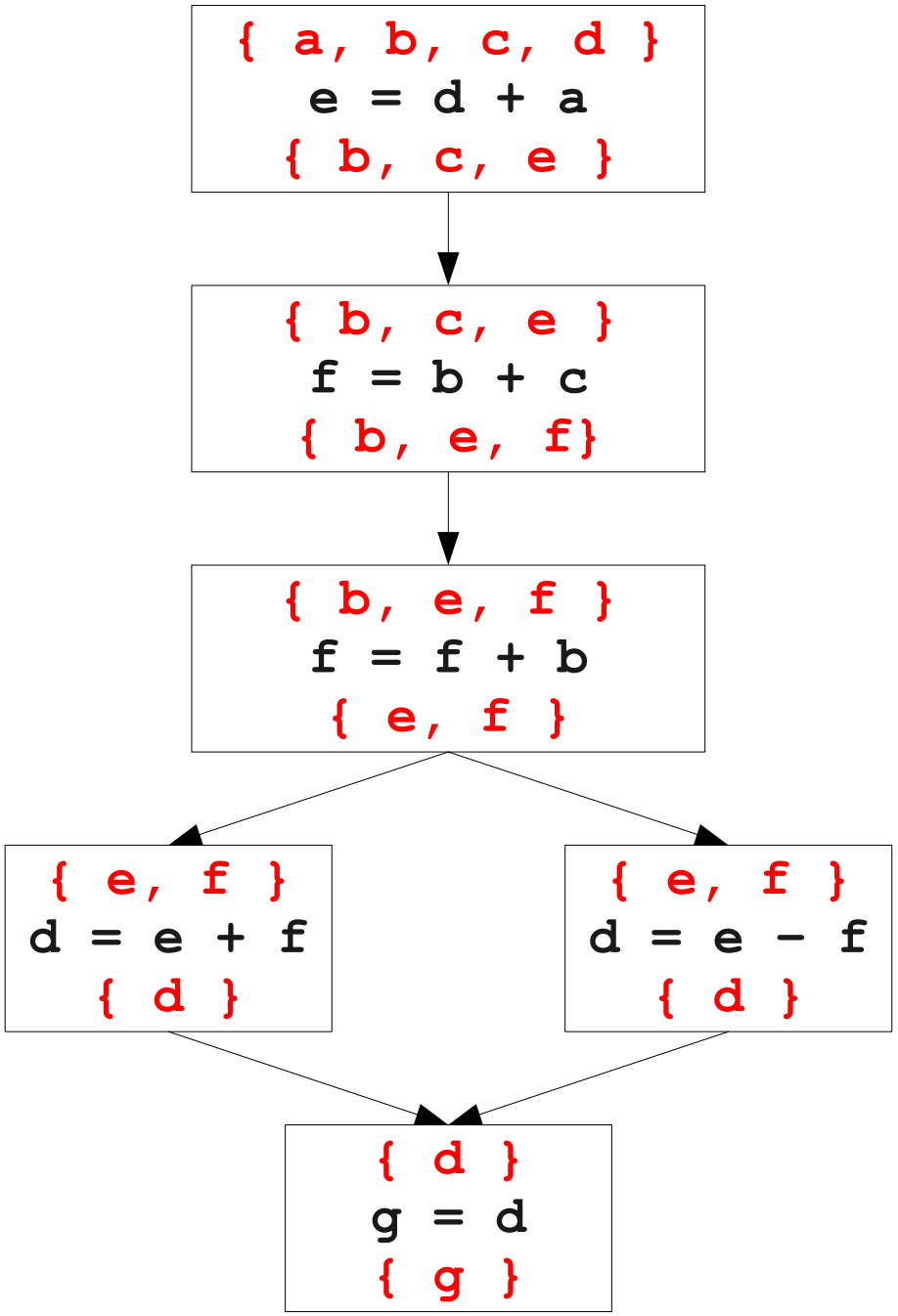


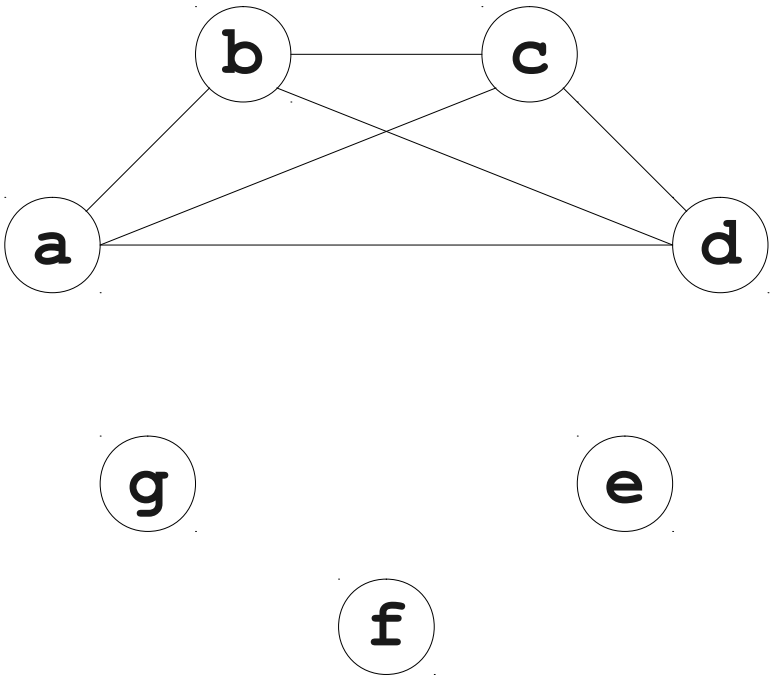
An Entirely Different Approach



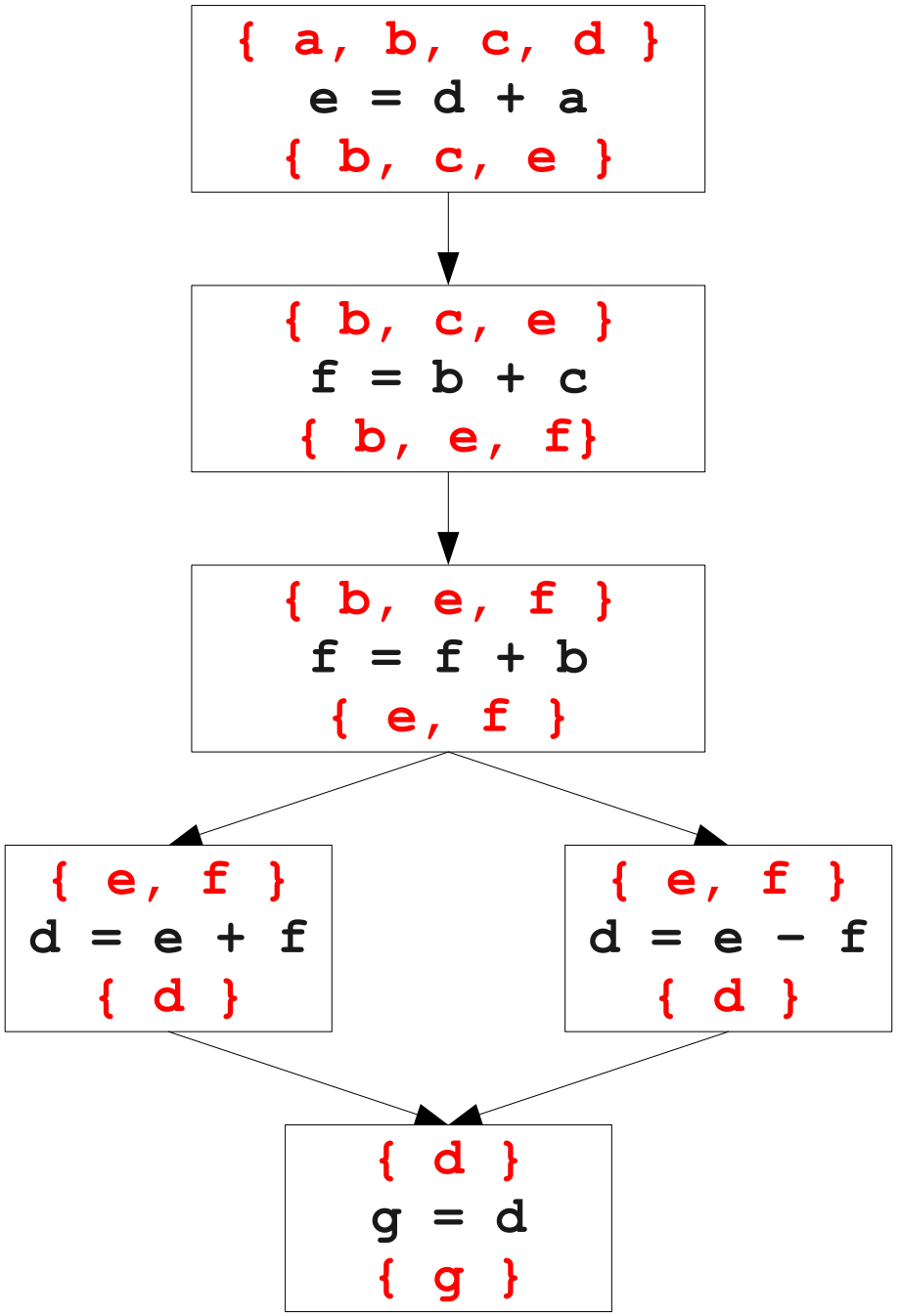


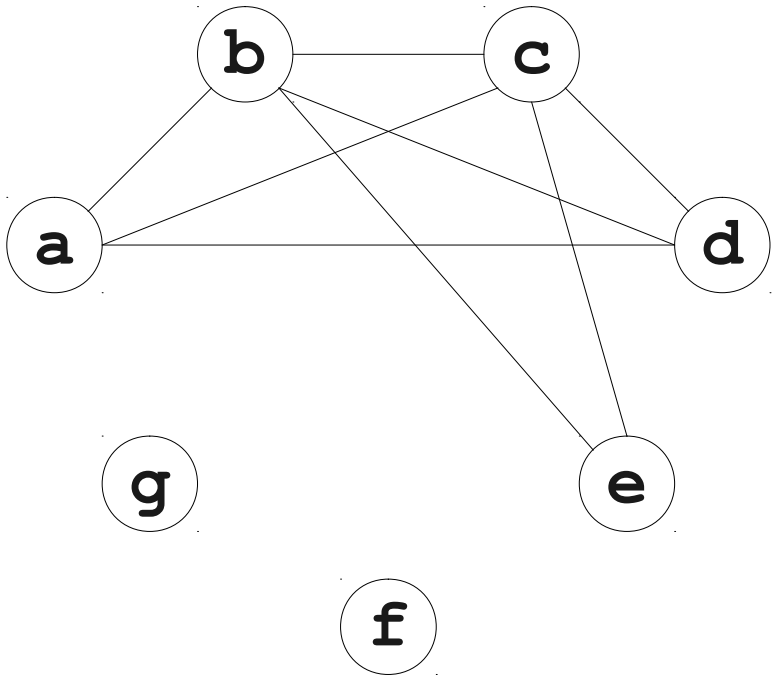
An Entirely Different Approach



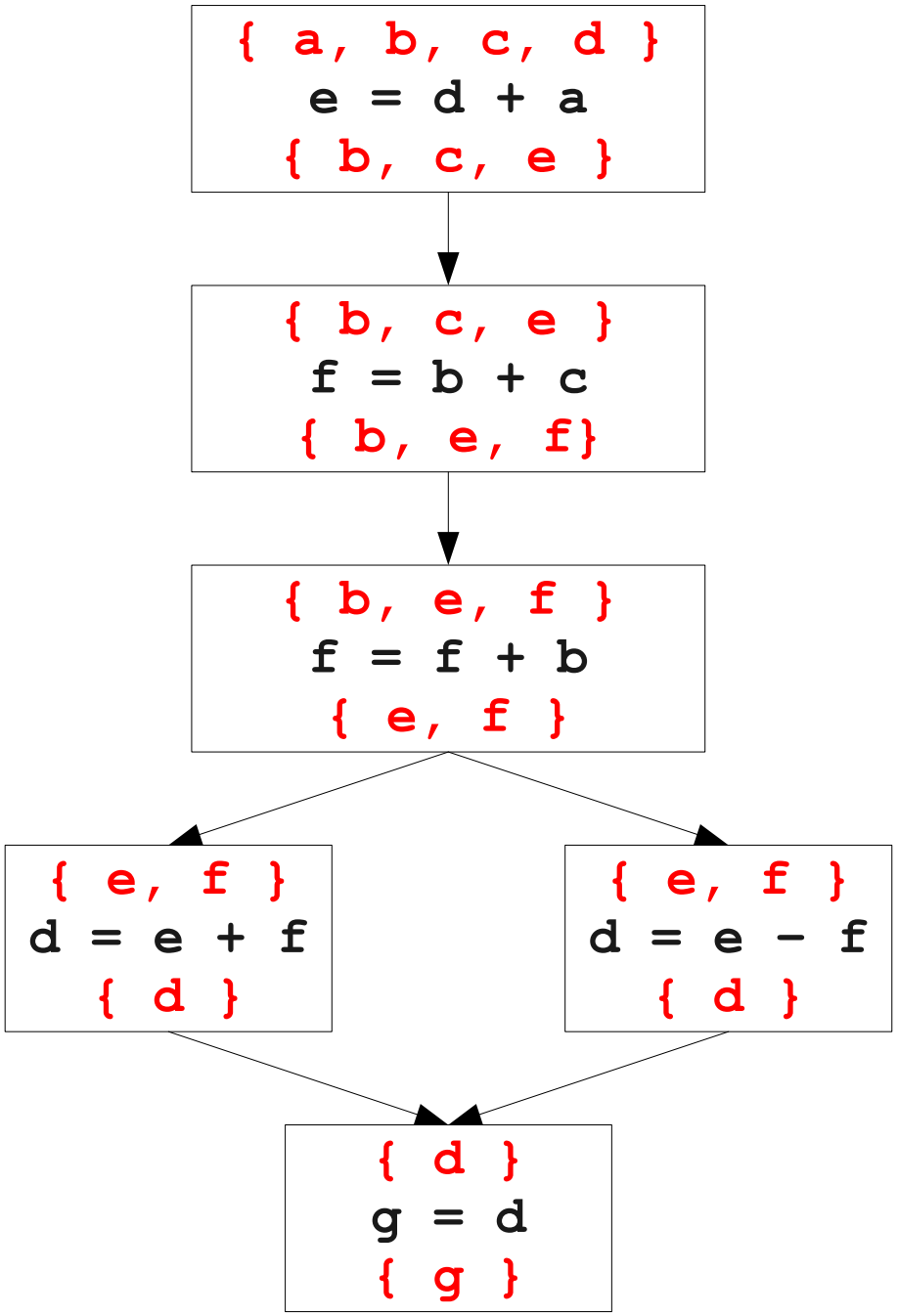


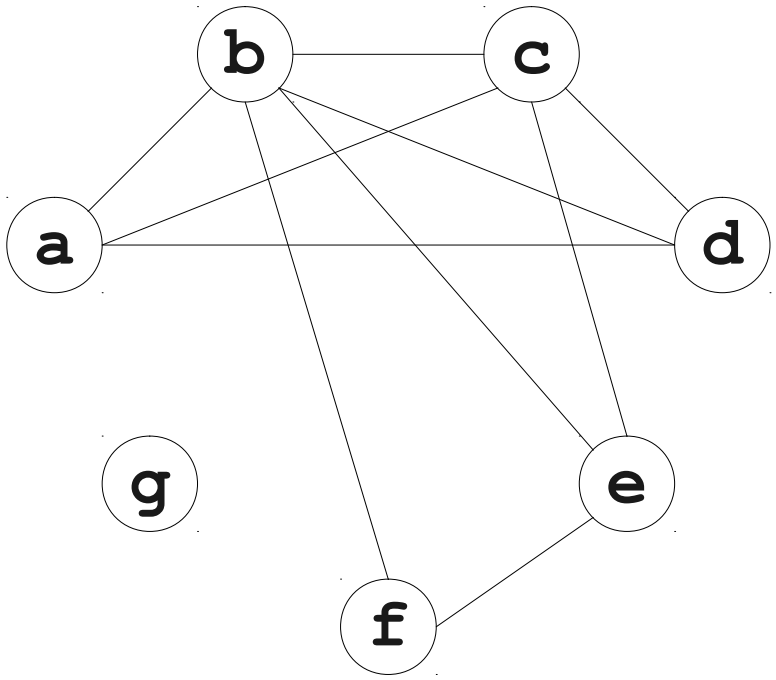
An Entirely Different Approach



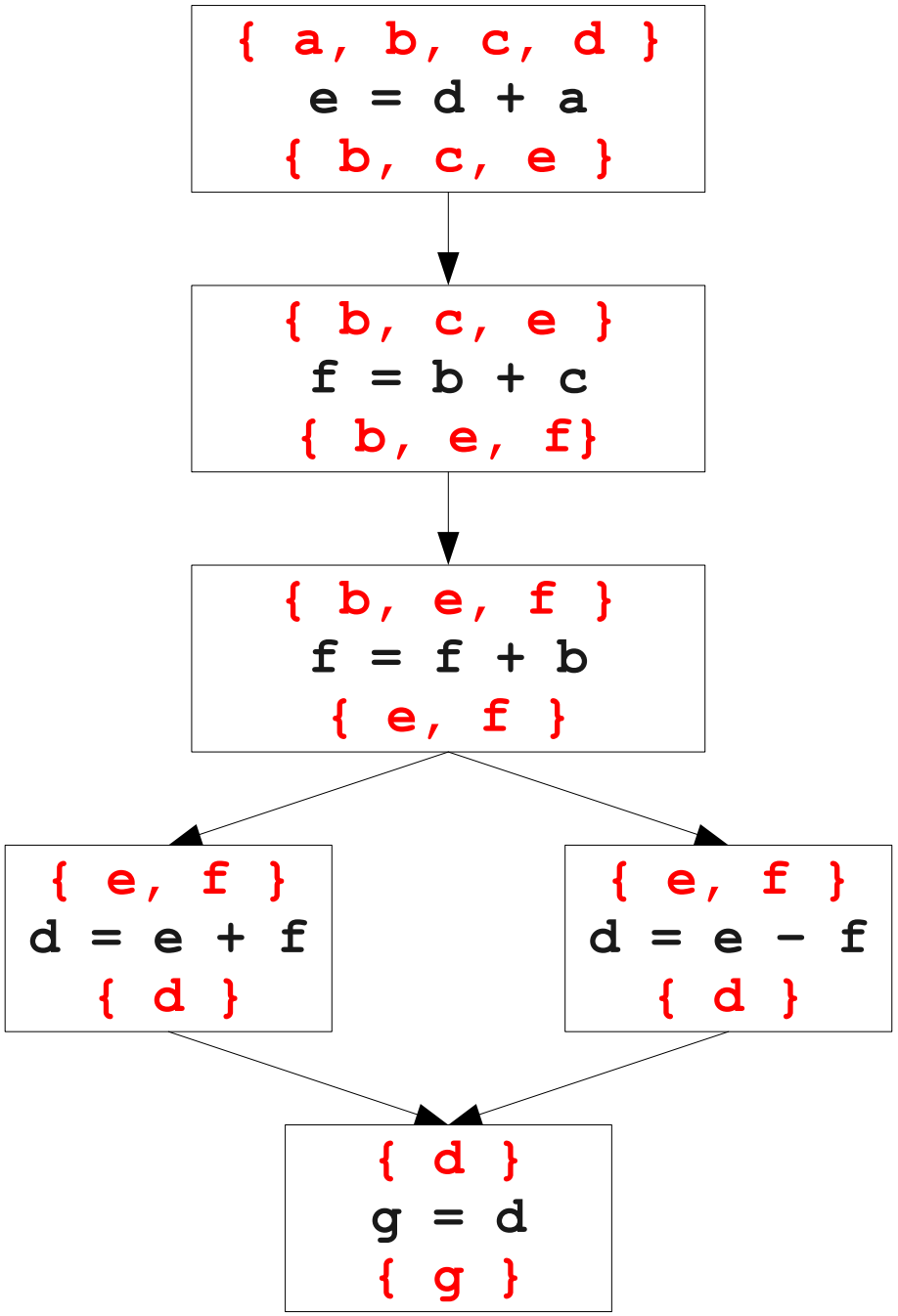


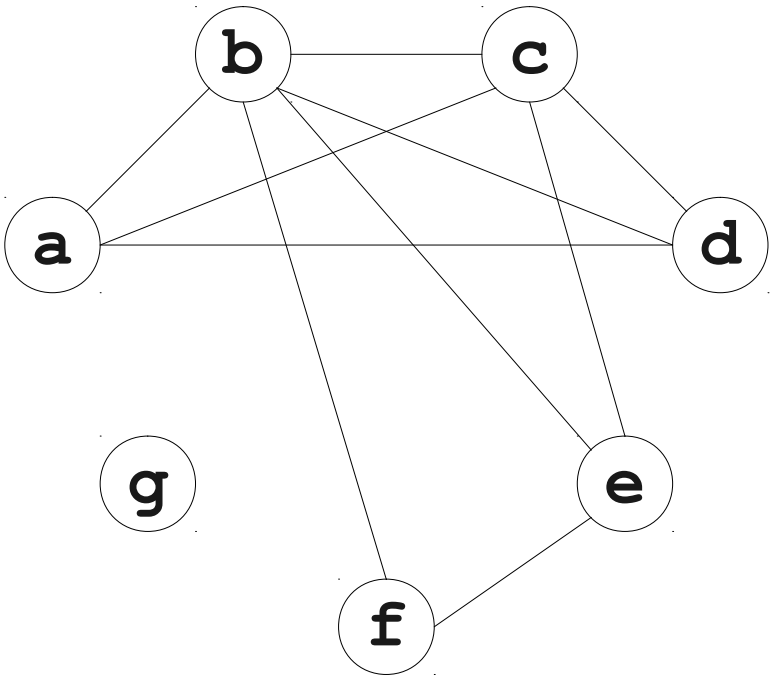
An Entirely Different Approach



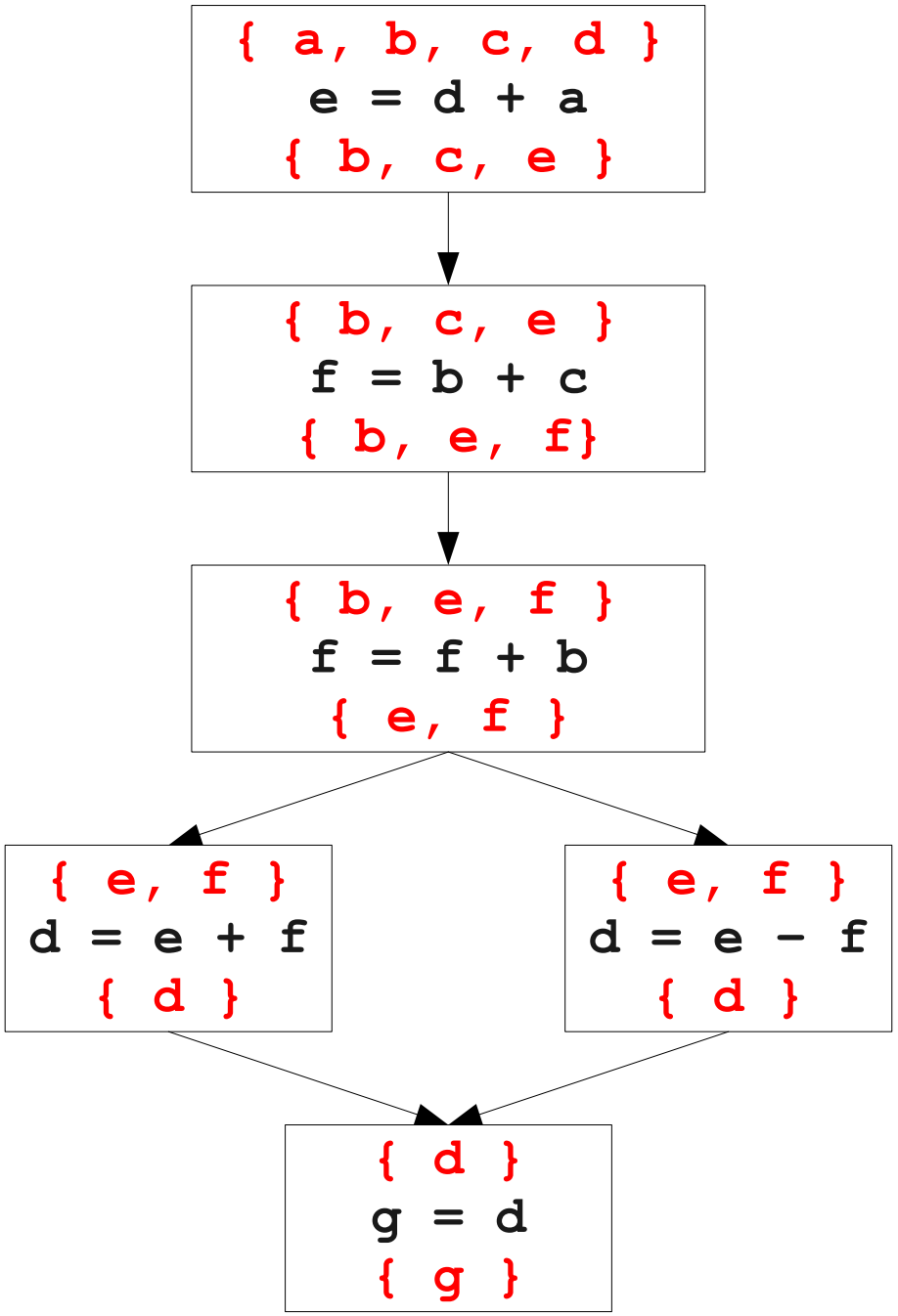


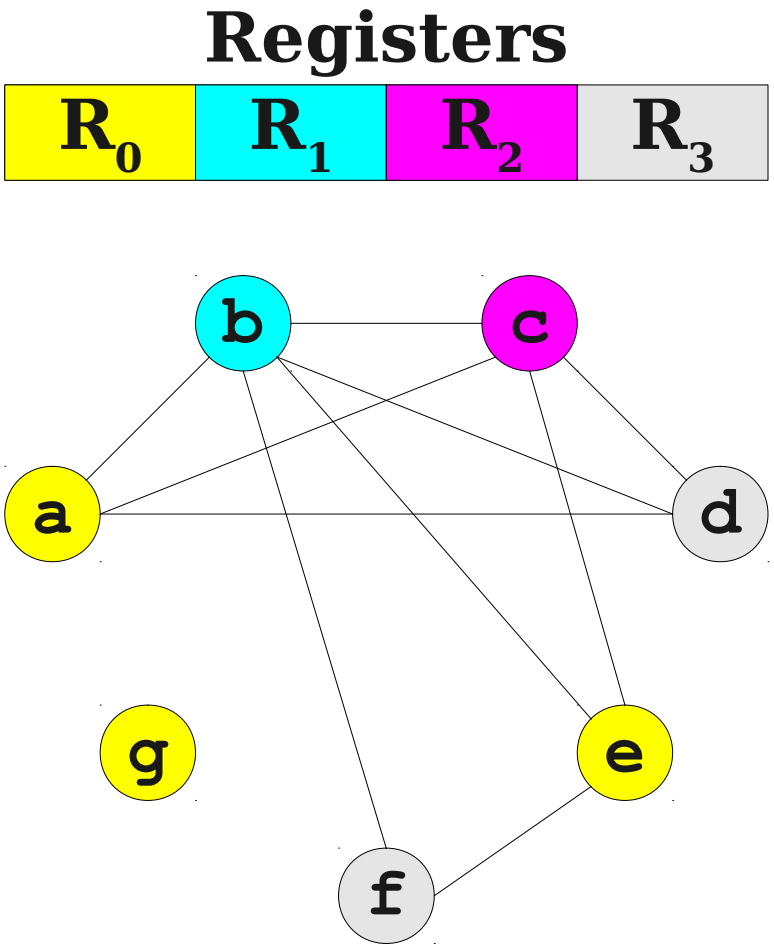
An Entirely Different Approach





An Entirely Different Approach





The Register Interference Graph

- The **register interference graph** (RIG) of a control-flow graph is an undirected graph where
  - Each node is a variable.
  - There is an edge between two variables that are live at the same program point.
- Perform register allocation by assigning each variable a different register from all of its neighbors.
- There's just one catch...

The One Catch

- This problem is equivalent to **graph-coloring**, which is **NP-hard** if there are at least three registers.
- No good polynomial-time algorithms (or even good approximations!) are known for this problem.
- We have to be content with a heuristic that is good enough for RIGs that arise in practice.

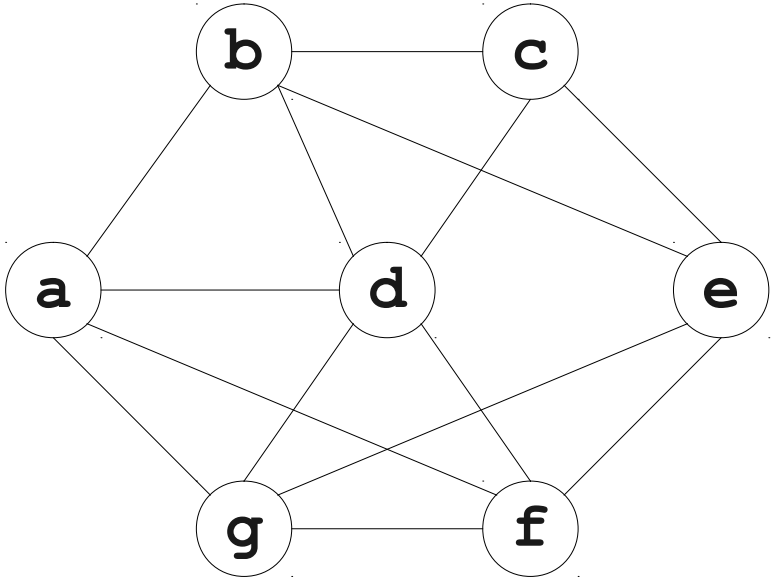
Battling **NP**-Hardness

Chaitin's Algorithm

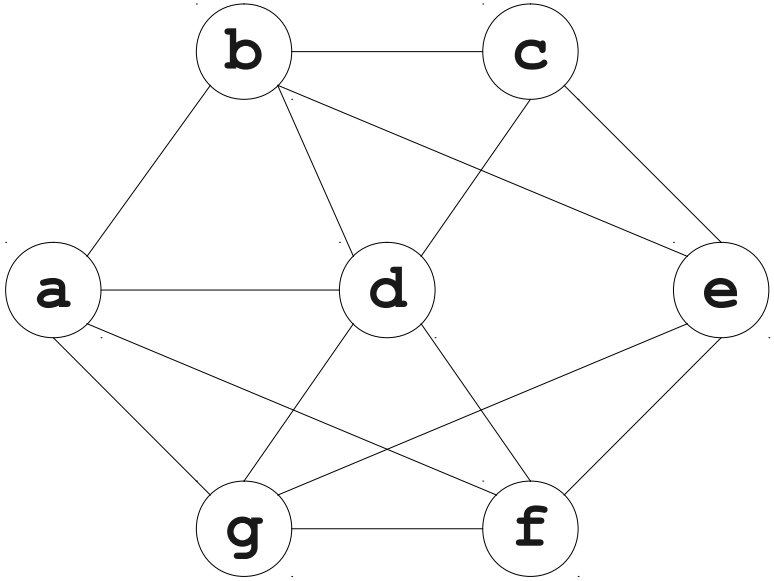
- Intuition:
  - Suppose we are trying to k-color a graph and find a node with fewer than k edges.
  - If we delete this node from the graph and color what remains, we can find a color for this node if we add it back in.
  - Reason: With fewer than k neighbors, some color must be left over.
- Algorithm:
  - Find a node with fewer than k outgoing edges.
  - Remove it from the graph.
  - Recursively color the rest of the graph.
  - Add the node back in.
  - Assign it a valid color.

Chaitin's Algorithm

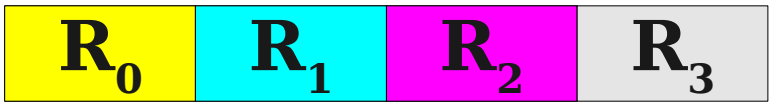
Chaitin's Algorithm



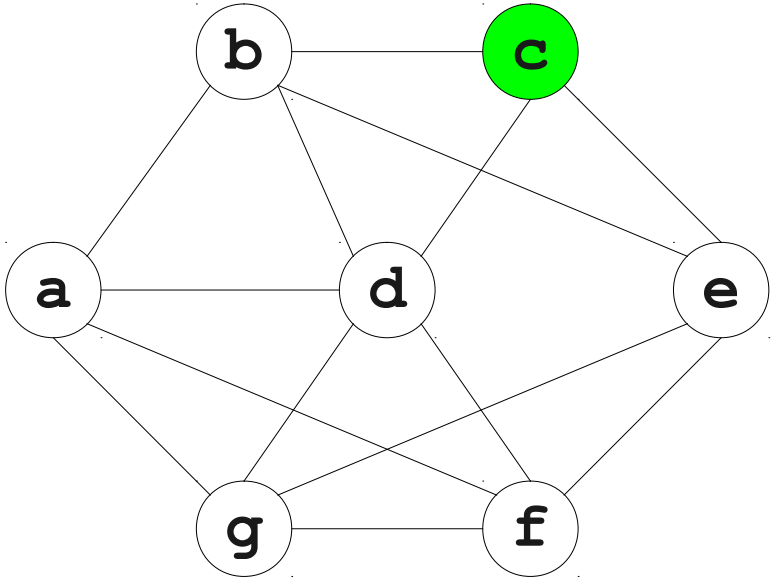
Chaitin's Algorithm



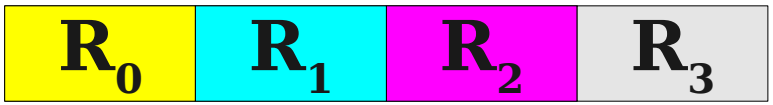
Registers



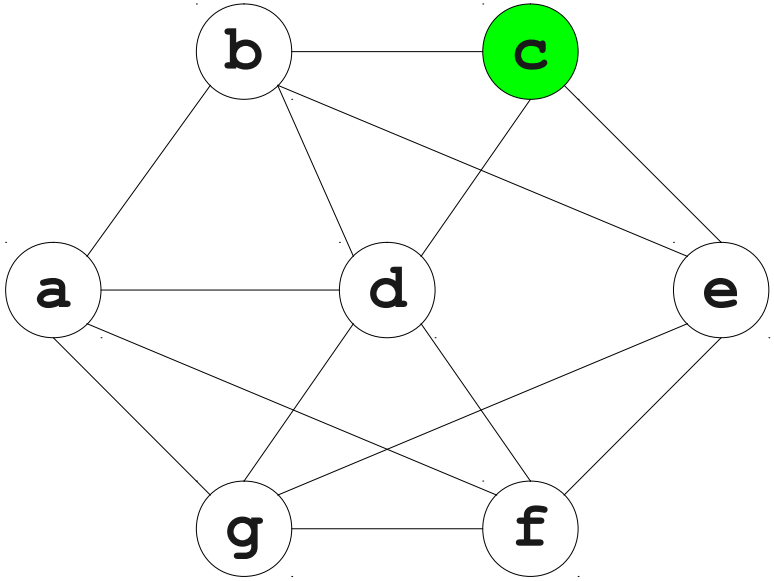
Chaitin's Algorithm



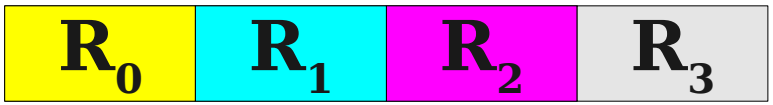
Registers



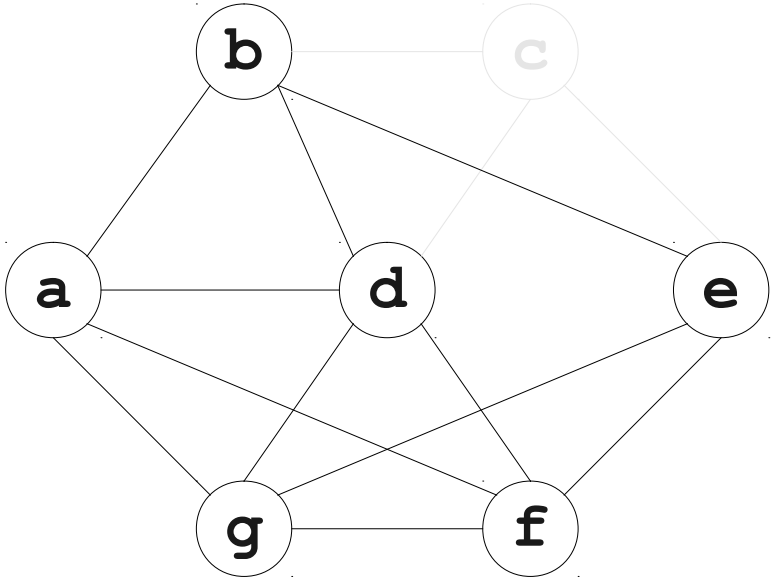
Chaitin's Algorithm



Registers



Chaitin's Algorithm

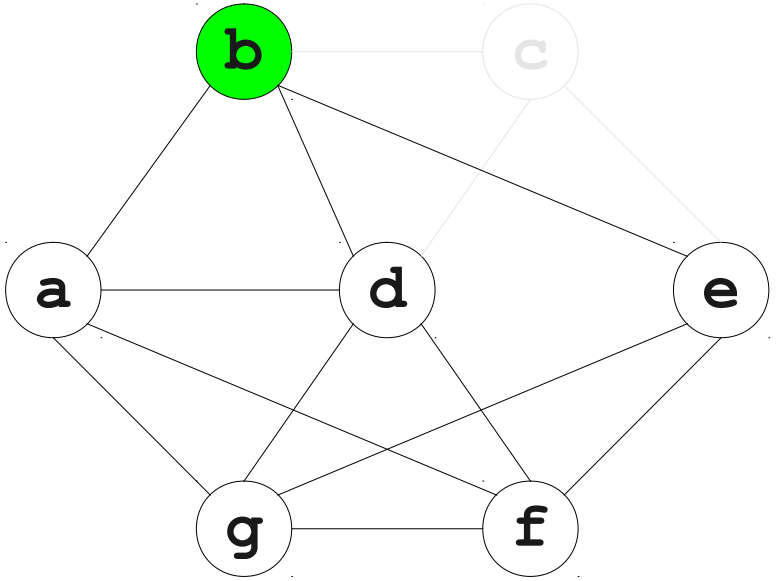


c

Registers

R_0	R_1	R_2	R_3

Chaitin's Algorithm

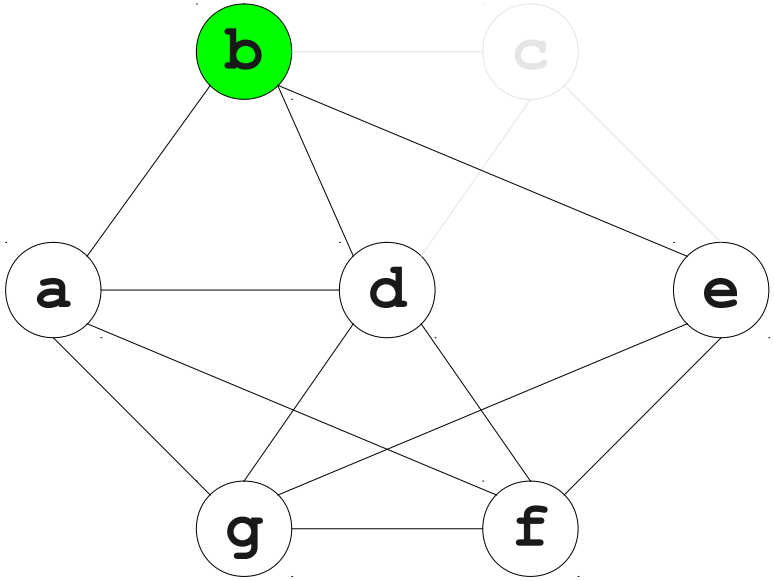


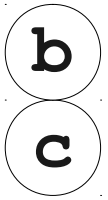
c

Registers

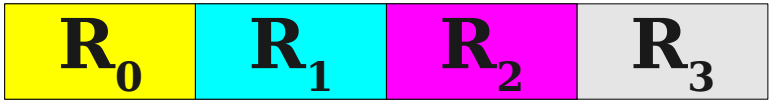
R_0	R_1	R_2	R_3

Chaitin's Algorithm

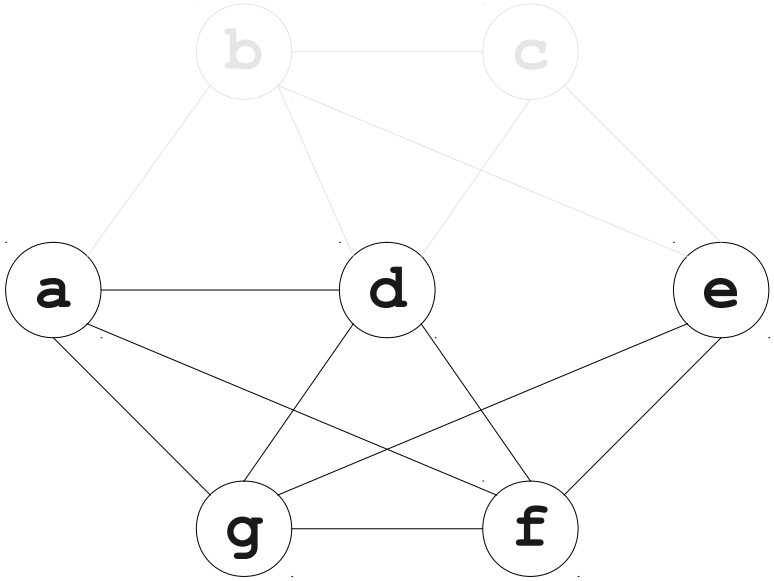


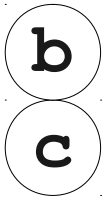


Registers

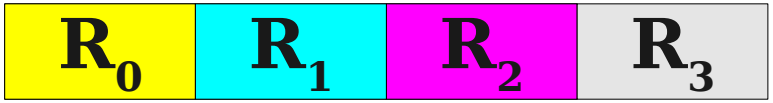


Chaitin's Algorithm

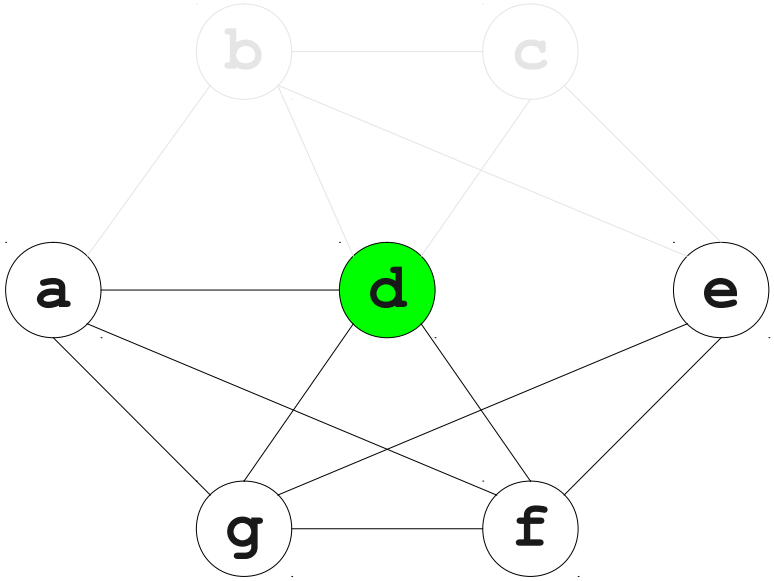


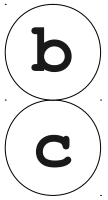


Registers

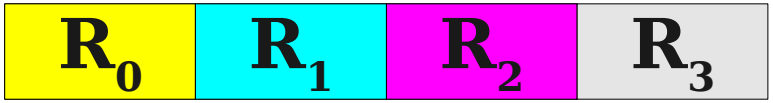


Chaitin's Algorithm

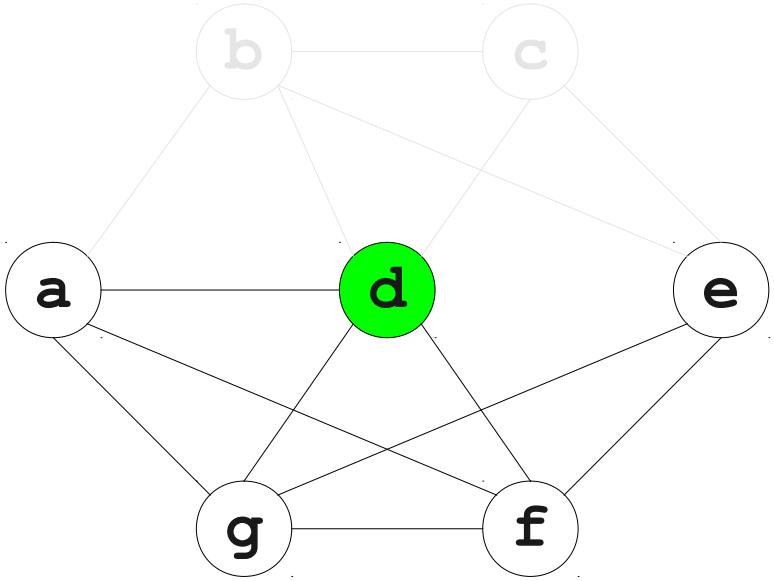


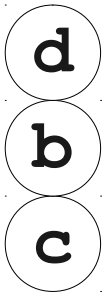


Registers

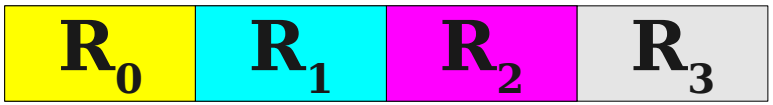


Chaitin's Algorithm

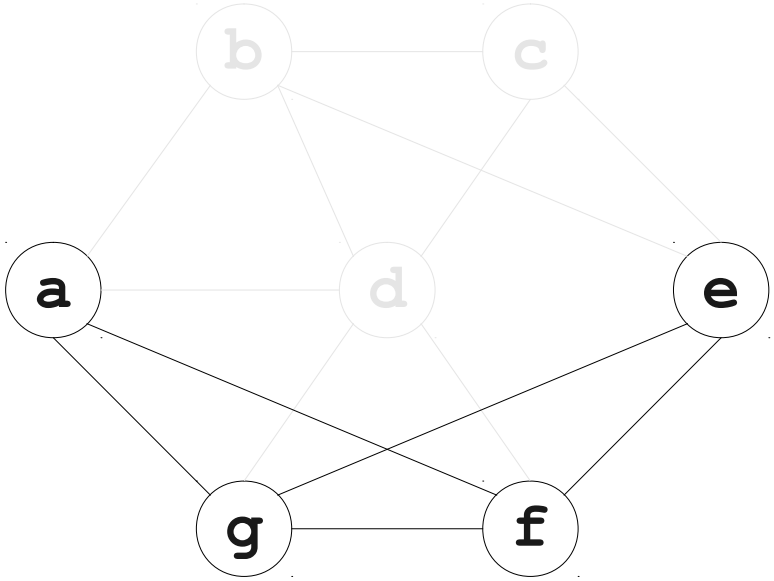


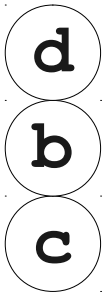


Registers

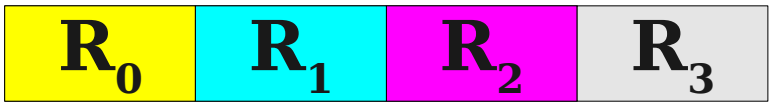


Chaitin's Algorithm

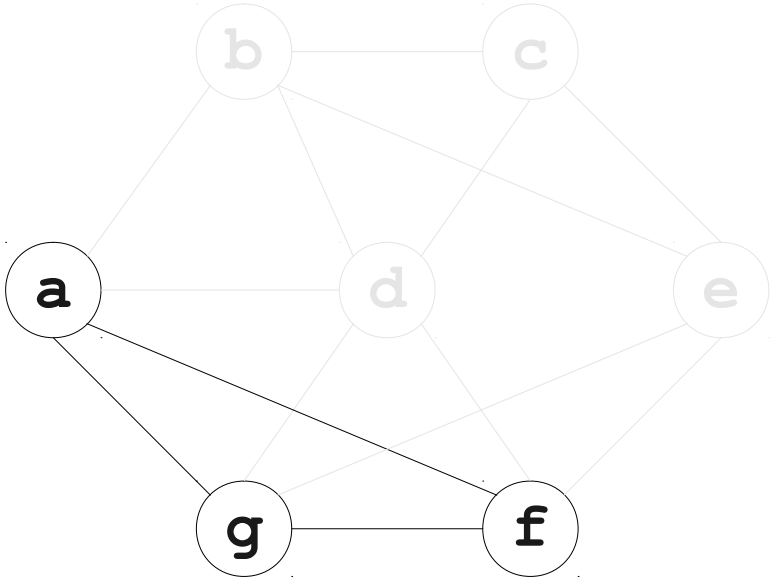


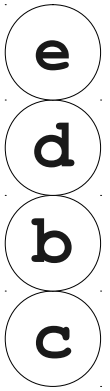


Registers

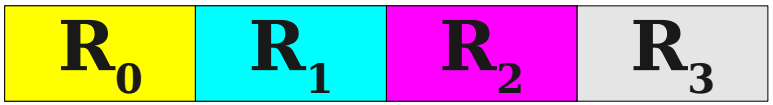


Chaitin's Algorithm

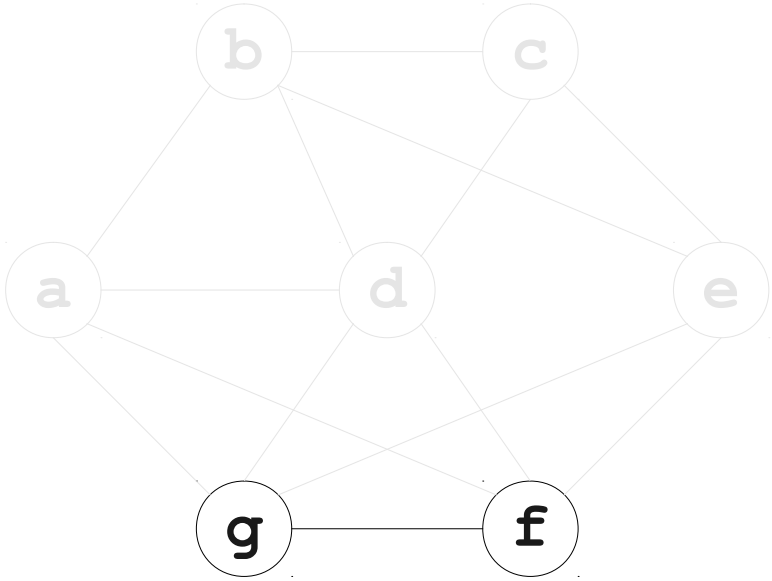




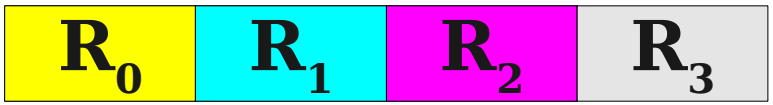
Registers



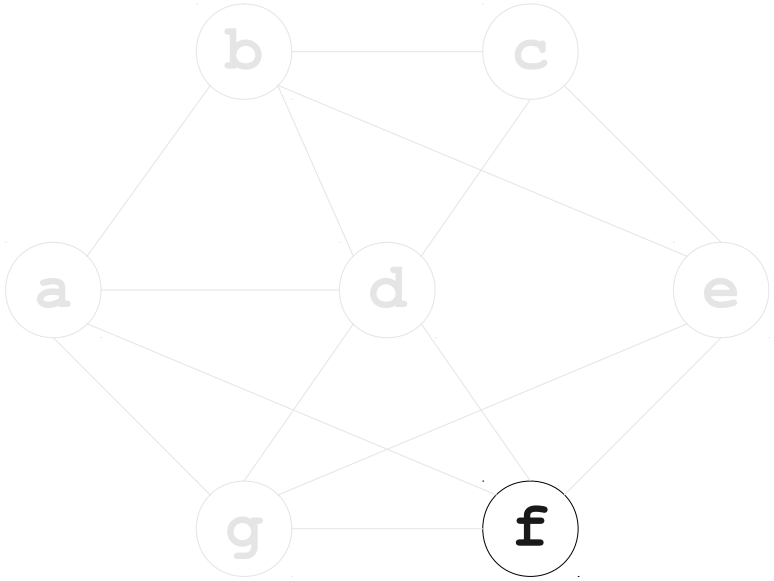
Chaitin's Algorithm

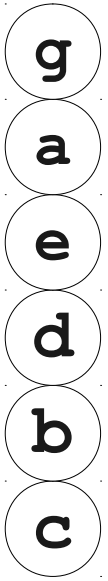


Registers

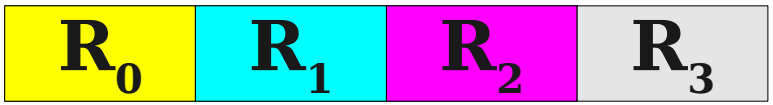


Chaitin's Algorithm

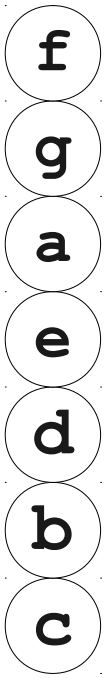




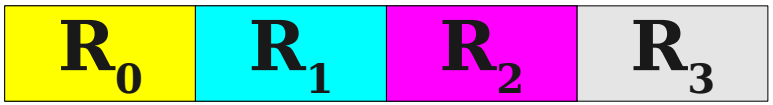
Registers



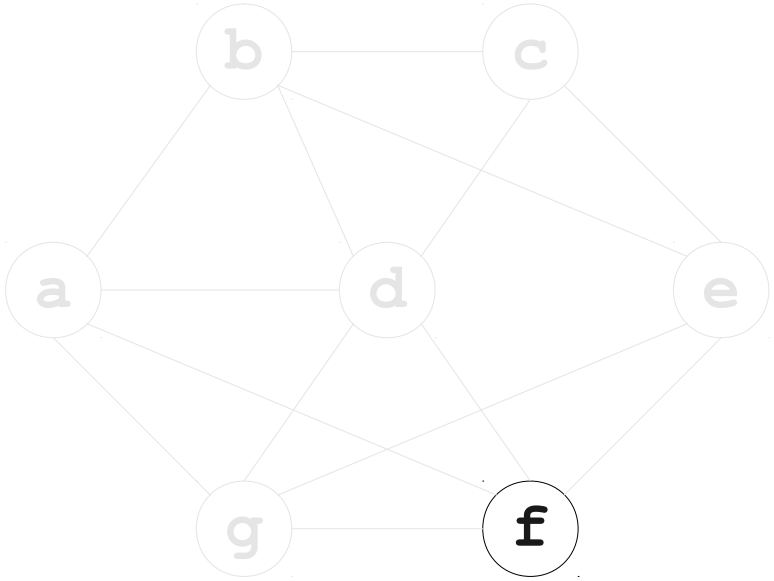
Chaitin's Algorithm

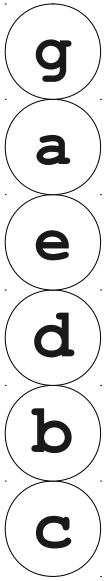


Registers

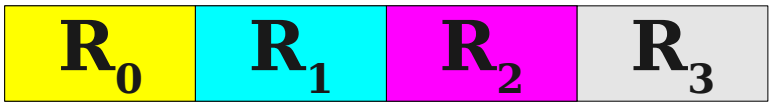


Chaitin's Algorithm

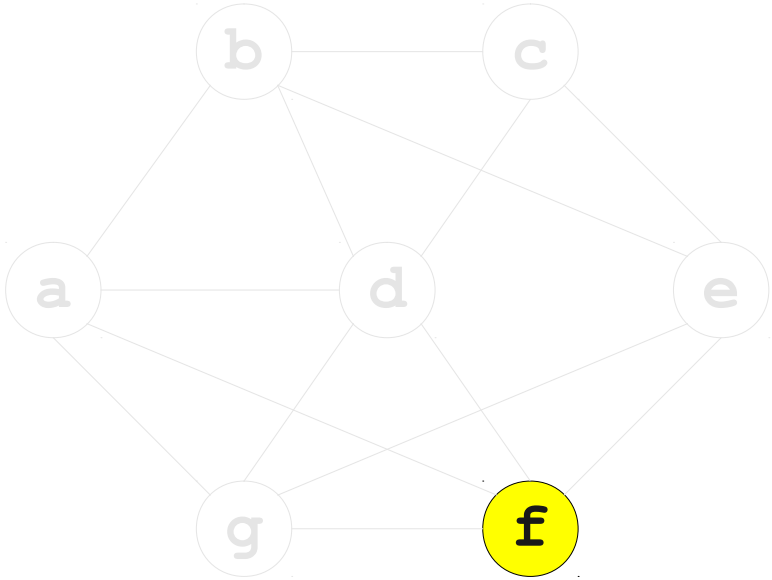


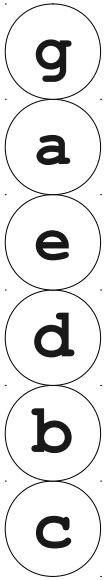


Registers

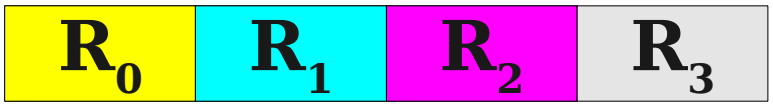


Chaitin's Algorithm

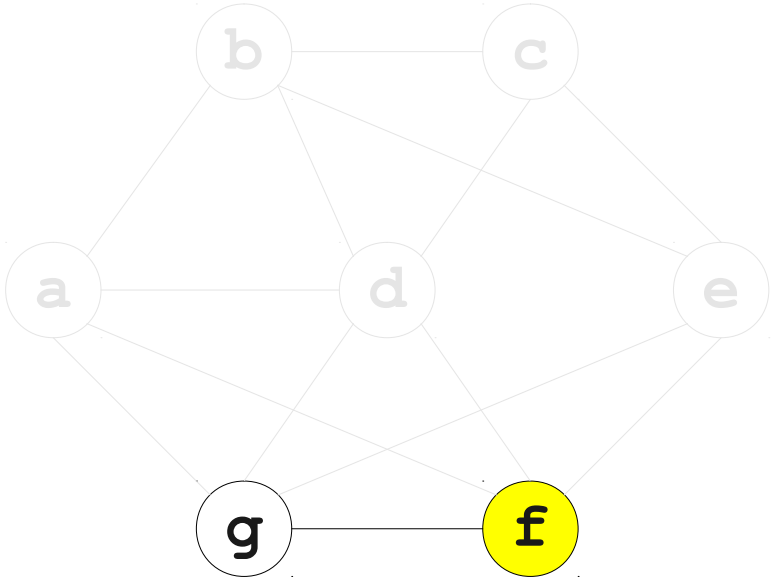




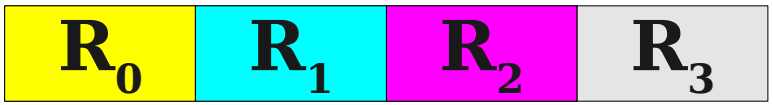
Registers



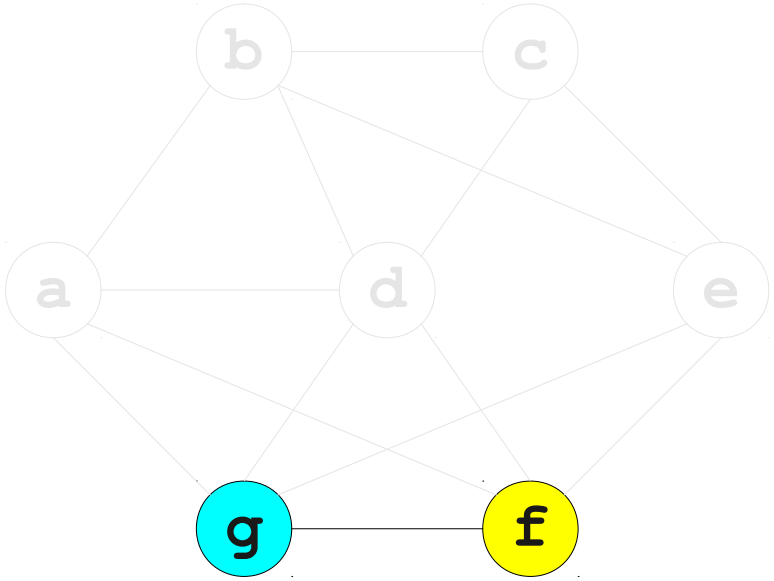
Chaitin's Algorithm

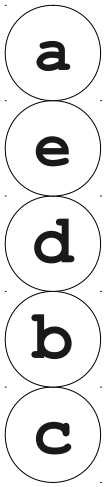


Registers

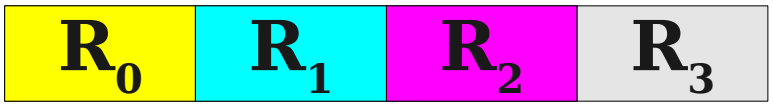


Chaitin's Algorithm

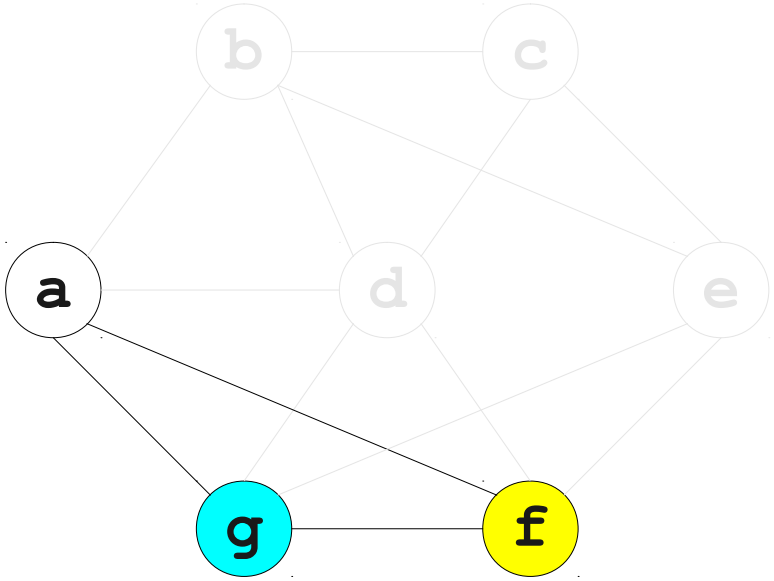


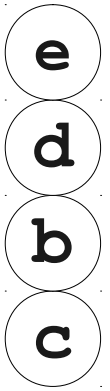


Registers

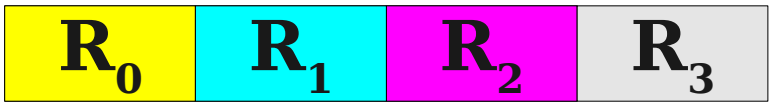


Chaitin's Algorithm

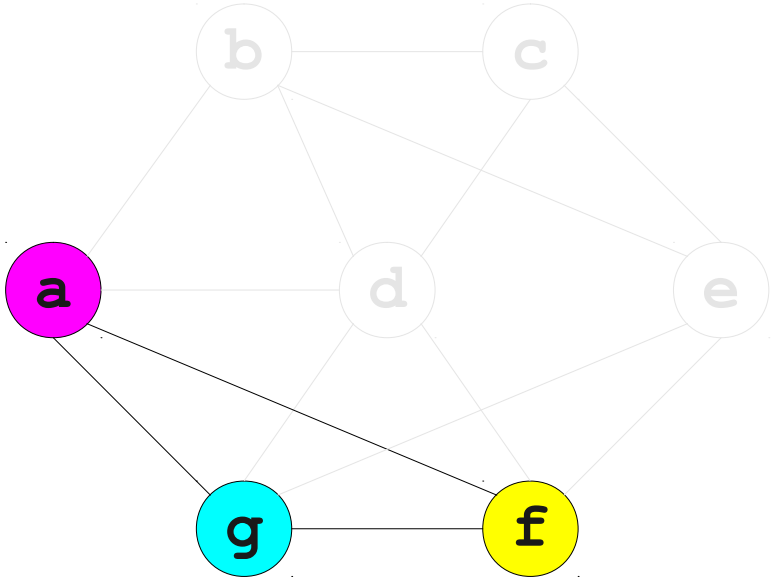


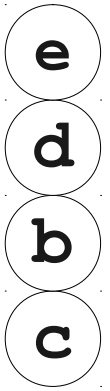


Registers

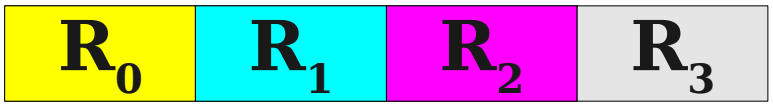


Chaitin's Algorithm

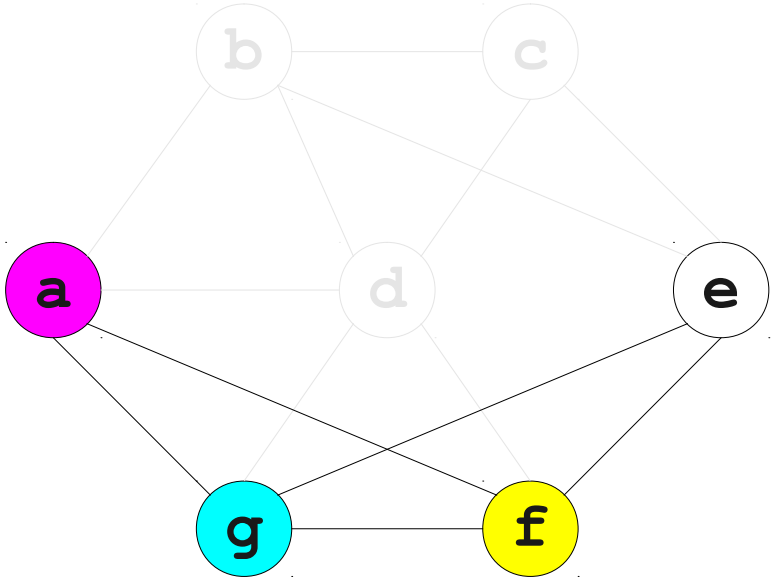


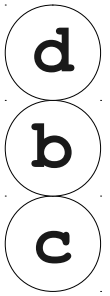


Registers

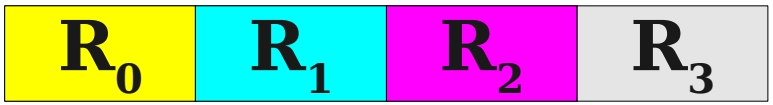


Chaitin's Algorithm

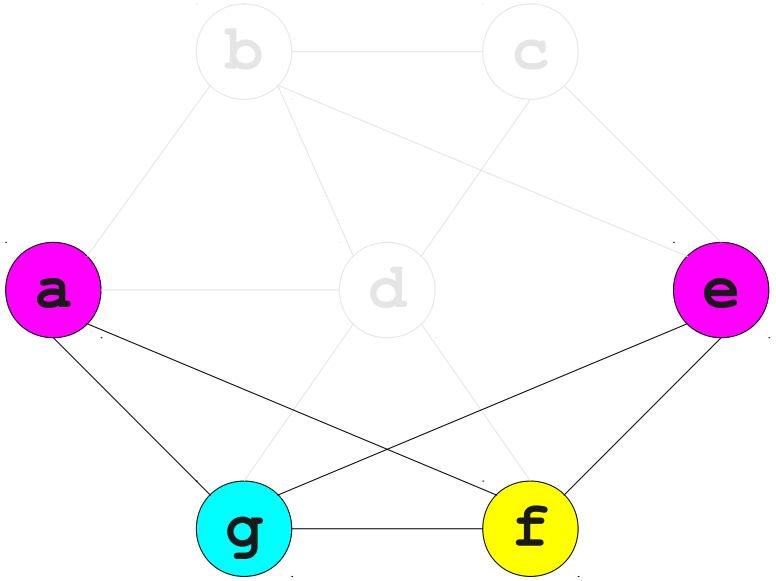


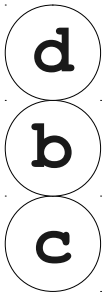


Registers

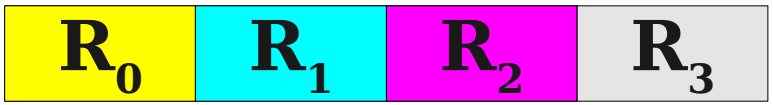


Chaitin's Algorithm

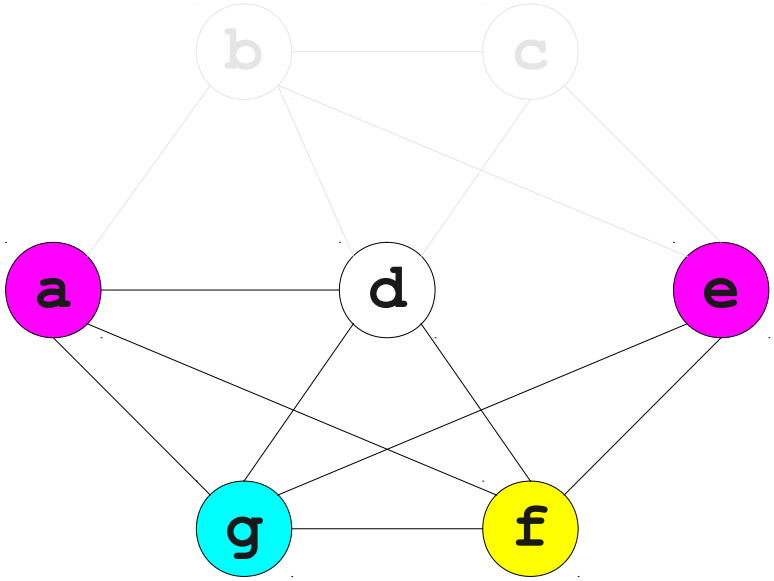


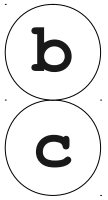


Registers

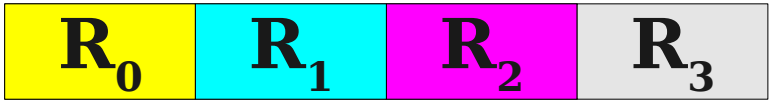


Chaitin's Algorithm

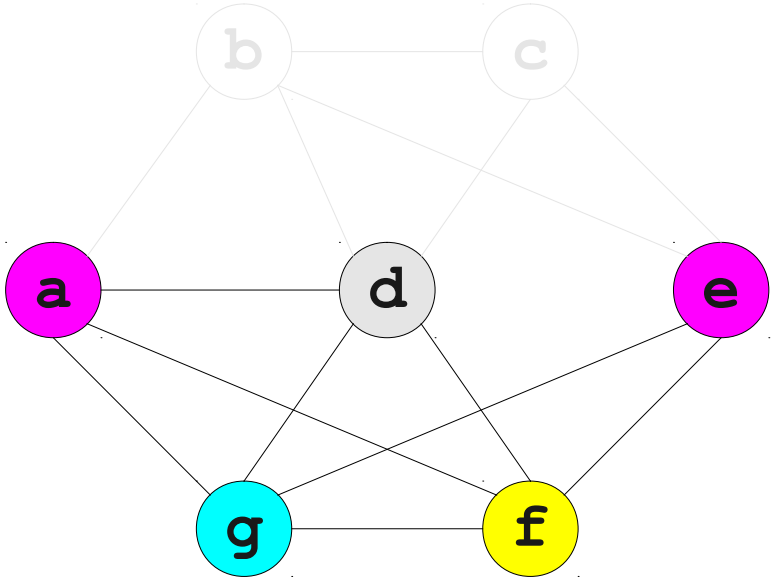


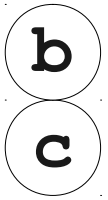


Registers

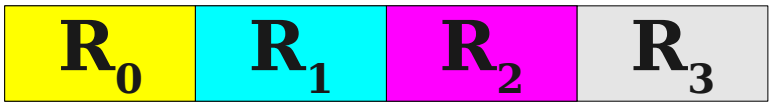


Chaitin's Algorithm

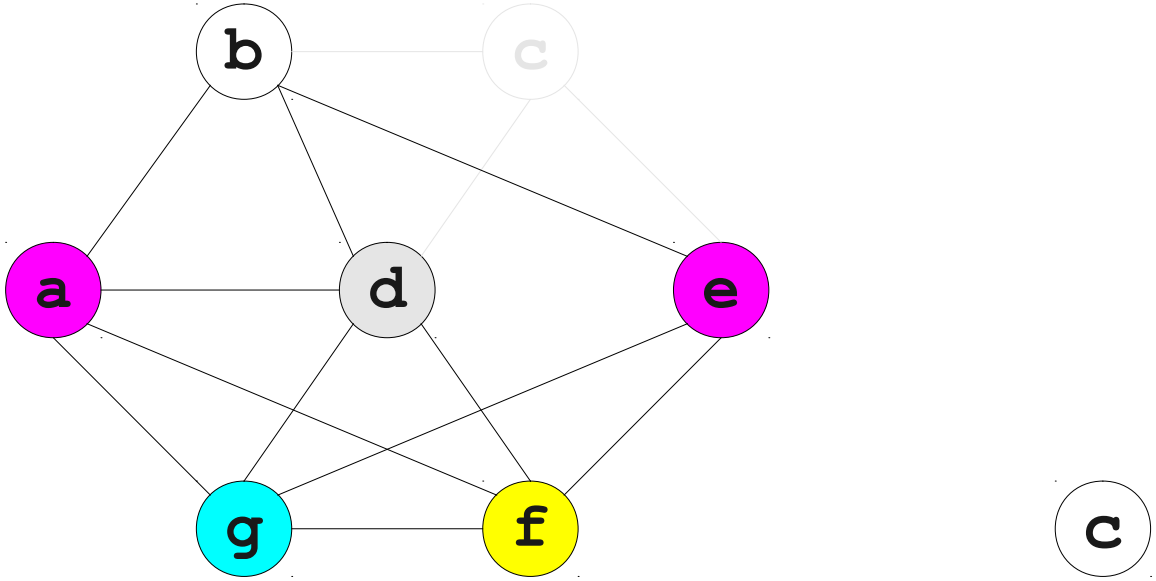




Registers



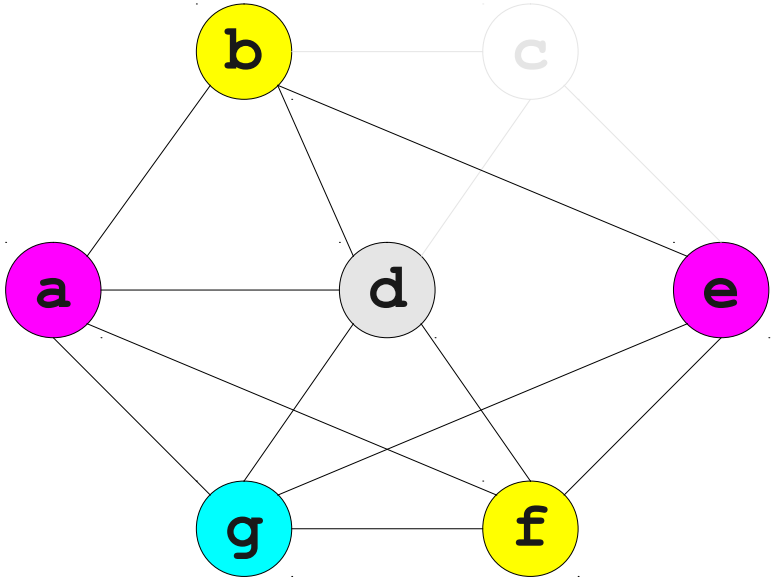
Chaitin's Algorithm



Registers

R_0	R_1	R_2	R_3

Chaitin's Algorithm

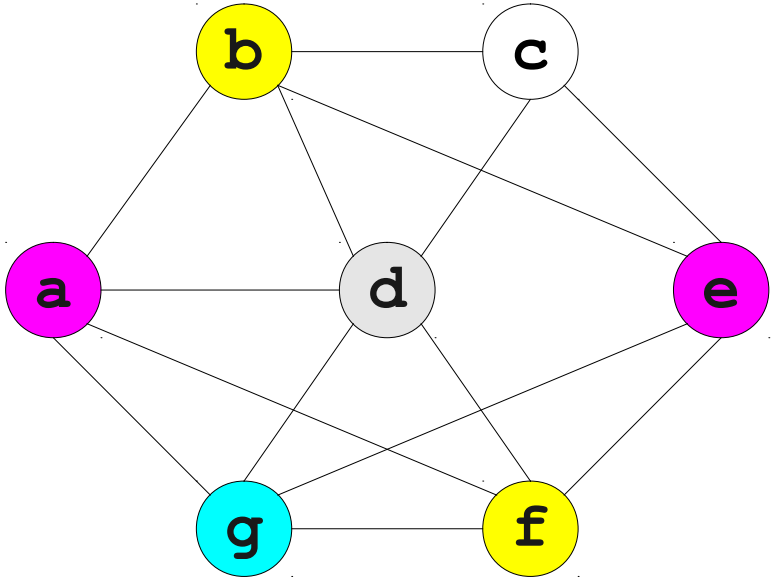


c

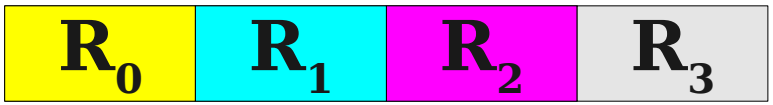
Registers

R_0	R_1	R_2	R_3

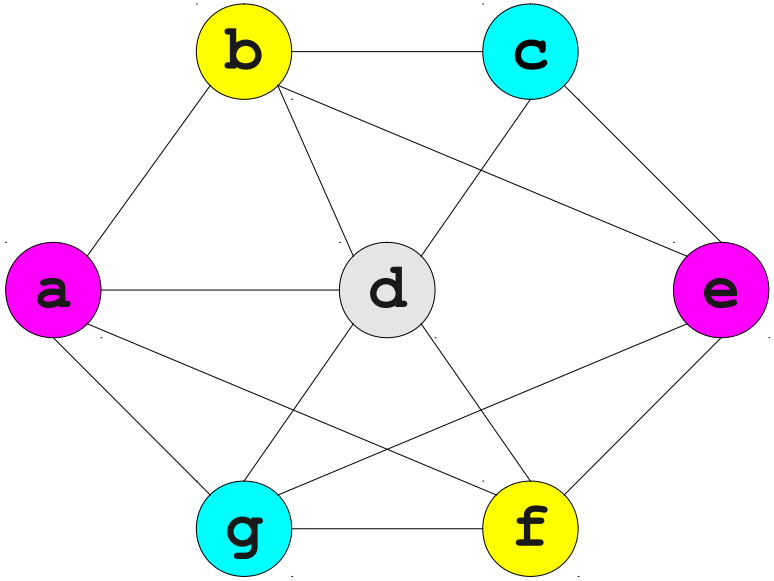
Chaitin's Algorithm



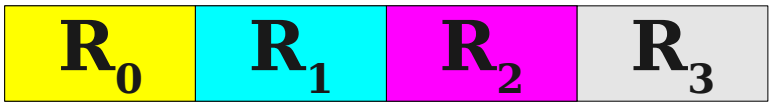
Registers



Chaitin's Algorithm



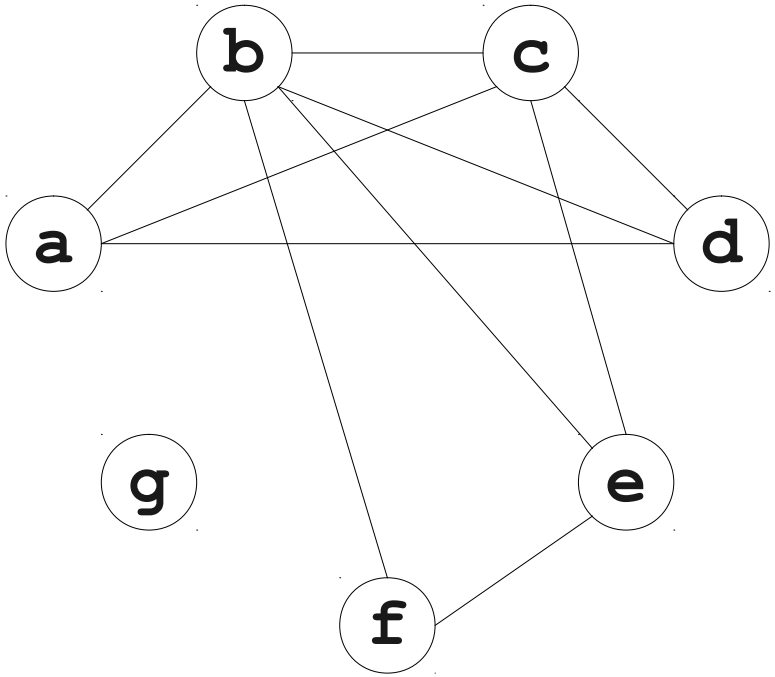
Registers



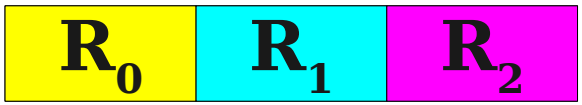
One Problem

- What if we can't find a node with fewer than k neighbors?
- Choose and remove an arbitrary node, marking it “troublesome.”
  - Use heuristics to choose which one.
- When adding node back in, it may be possible to find a valid color.
- Otherwise, we have to spill that node.

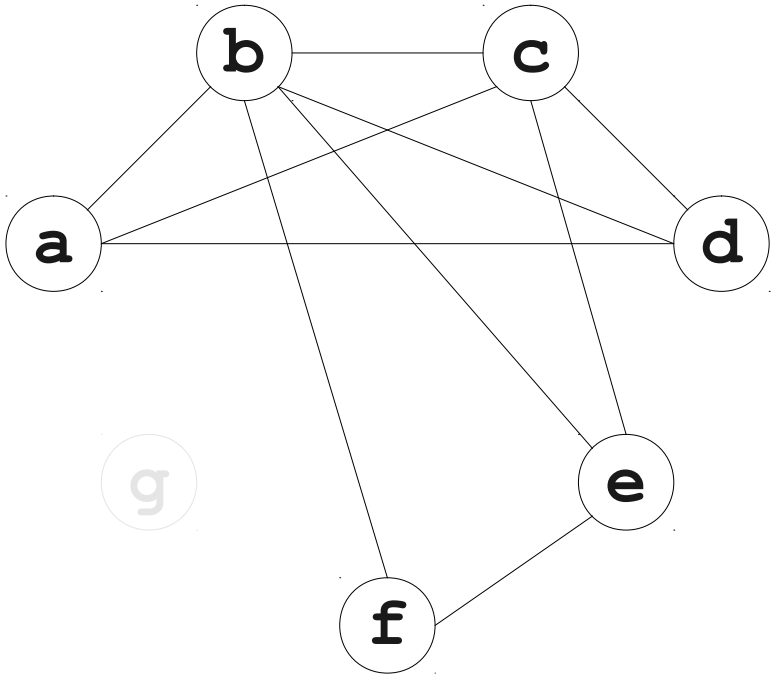
Chaitin's Algorithm Reloaded



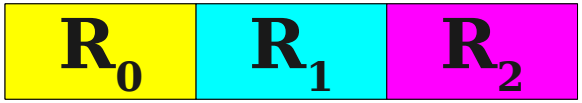
Registers



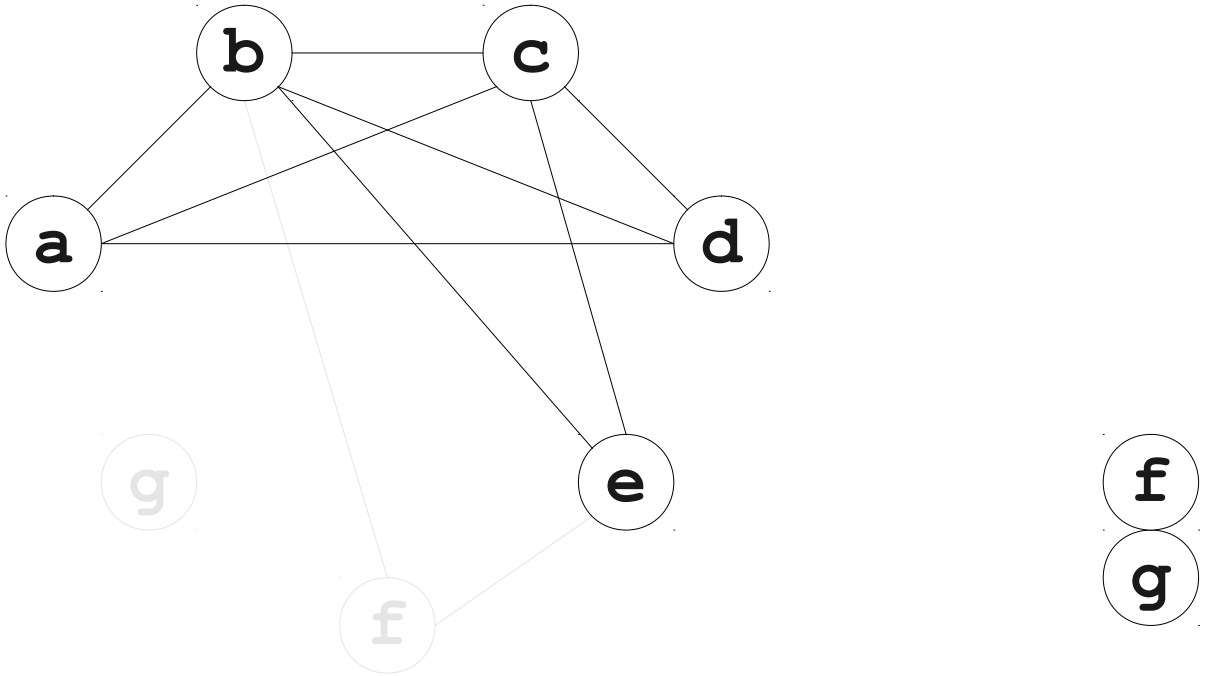
Chaitin's Algorithm Reloaded



Registers



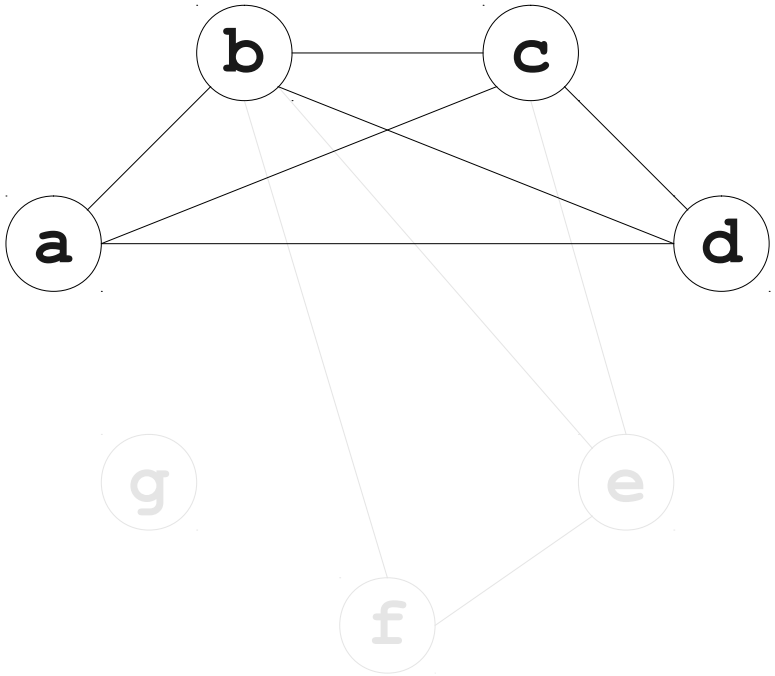
Chaitin's Algorithm Reloaded



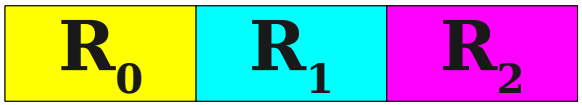
Registers

R_0	R_1	R_2

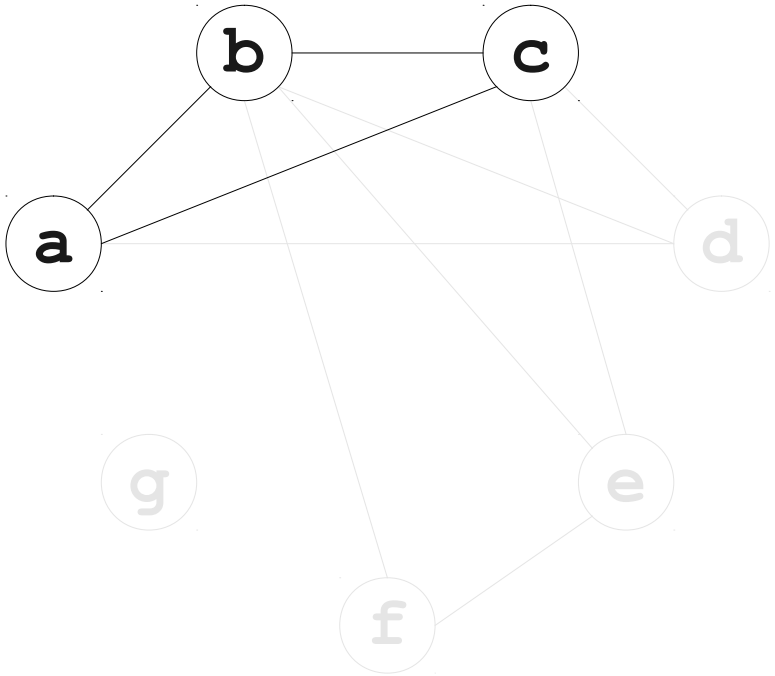
Chaitin's Algorithm Reloaded

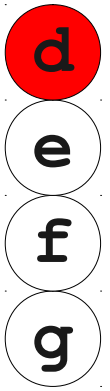


Registers

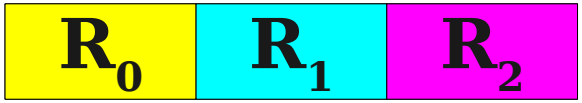


Chaitin's Algorithm Reloaded

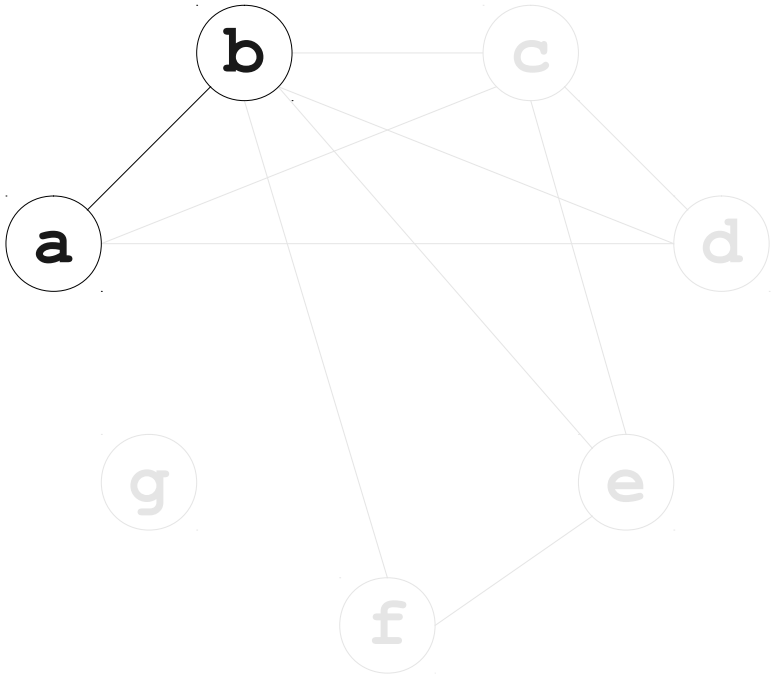


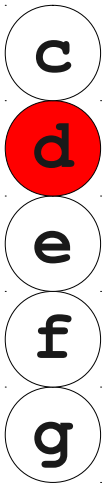


Registers

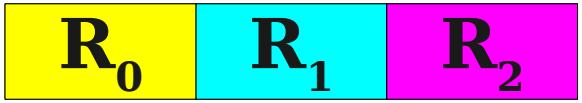


Chaitin's Algorithm Reloaded

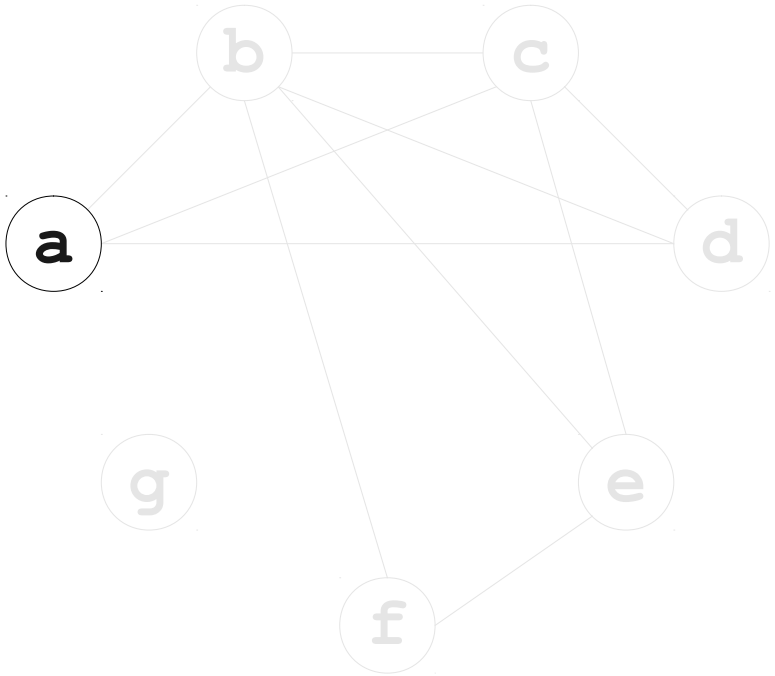


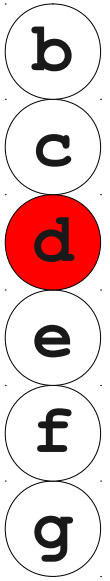


Registers

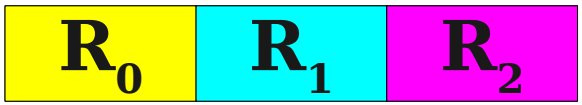


Chaitin's Algorithm Reloaded

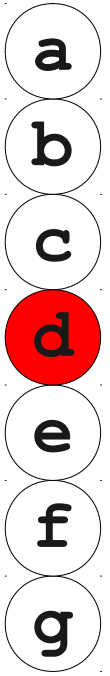




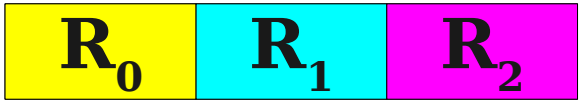
Registers



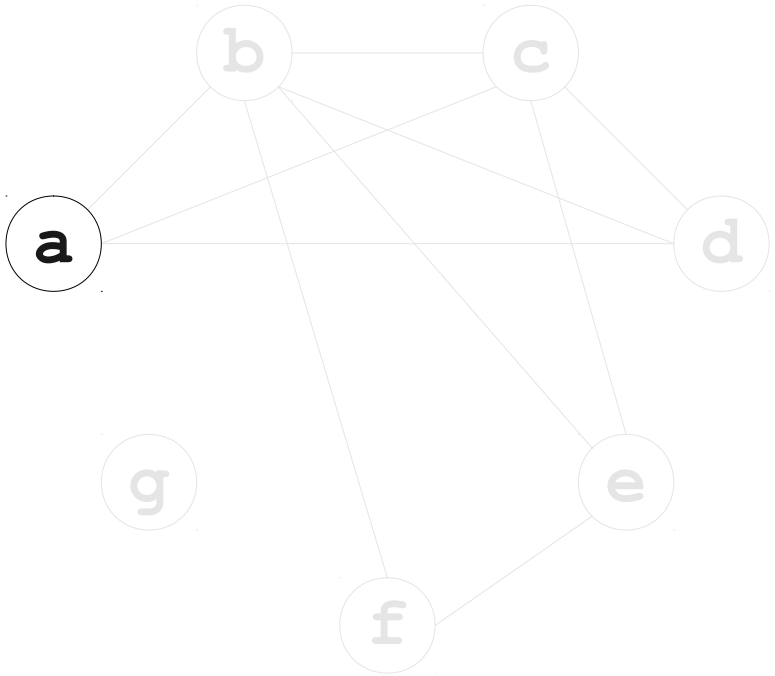
Chaitin's Algorithm Reloaded

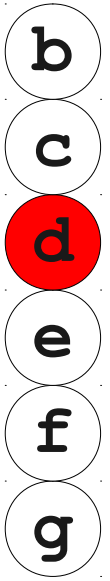


Registers

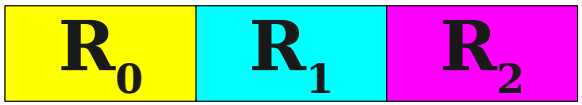


Chaitin's Algorithm Reloaded

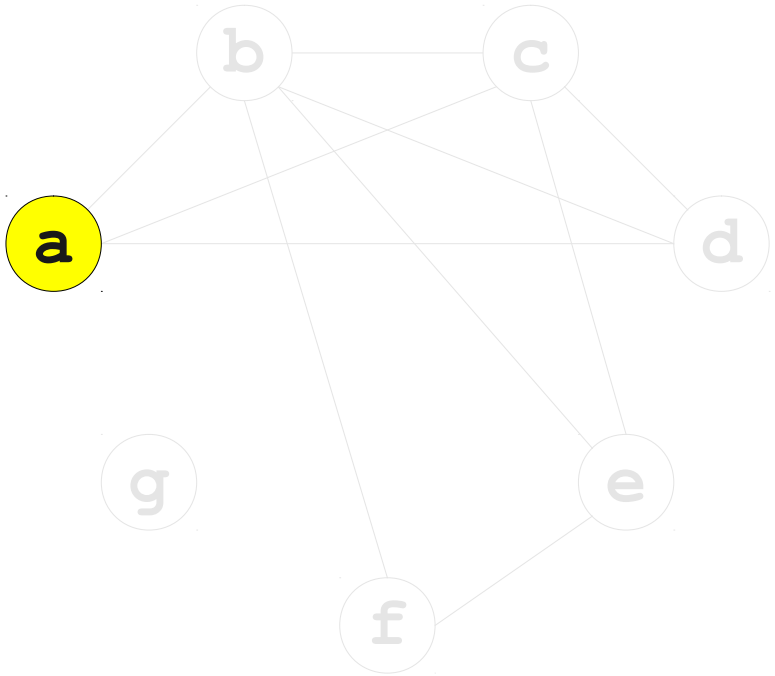


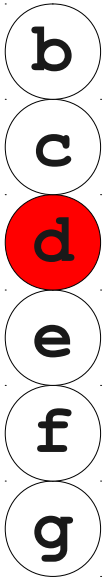


Registers

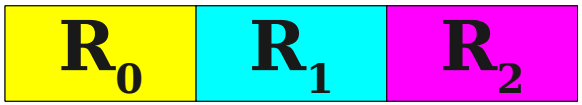


Chaitin's Algorithm Reloaded

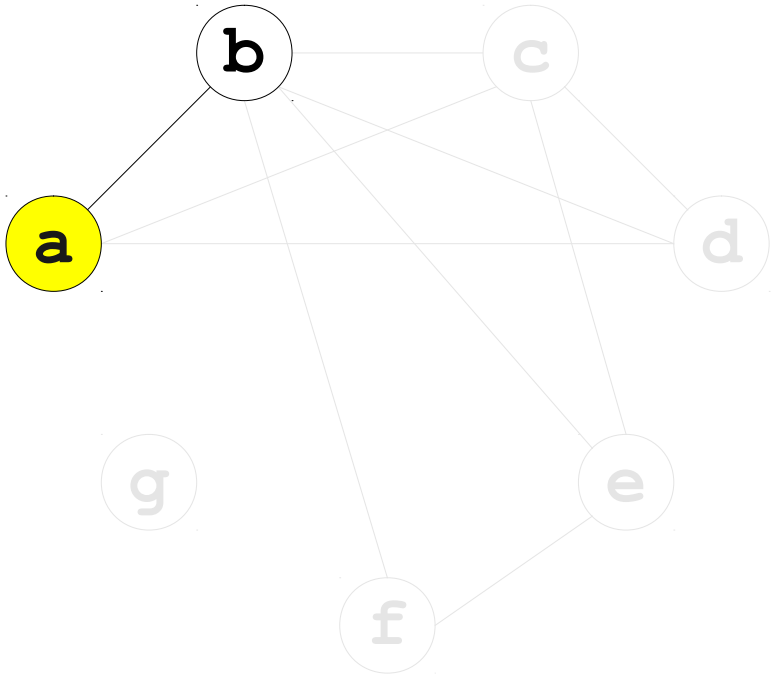


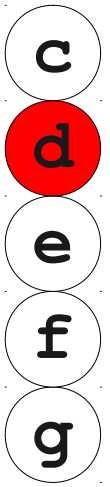


Registers

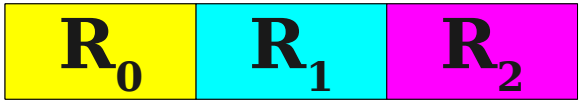


Chaitin's Algorithm Reloaded

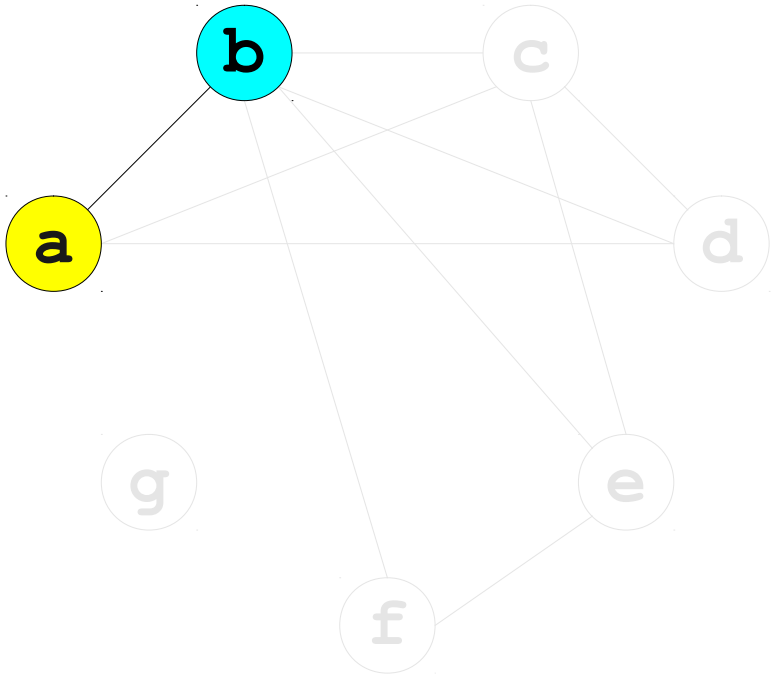


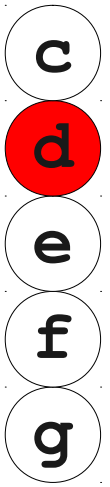


Registers

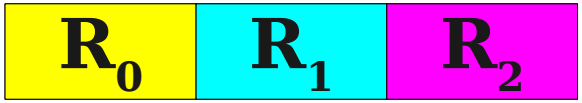


Chaitin's Algorithm Reloaded

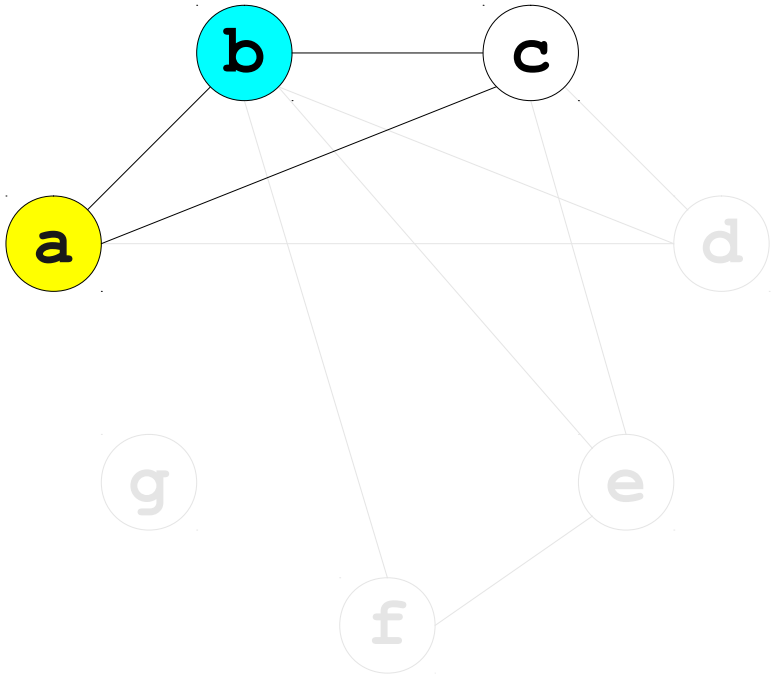


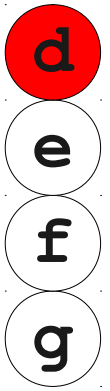


Registers

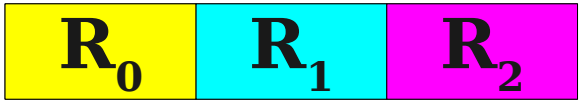


Chaitin's Algorithm Reloaded

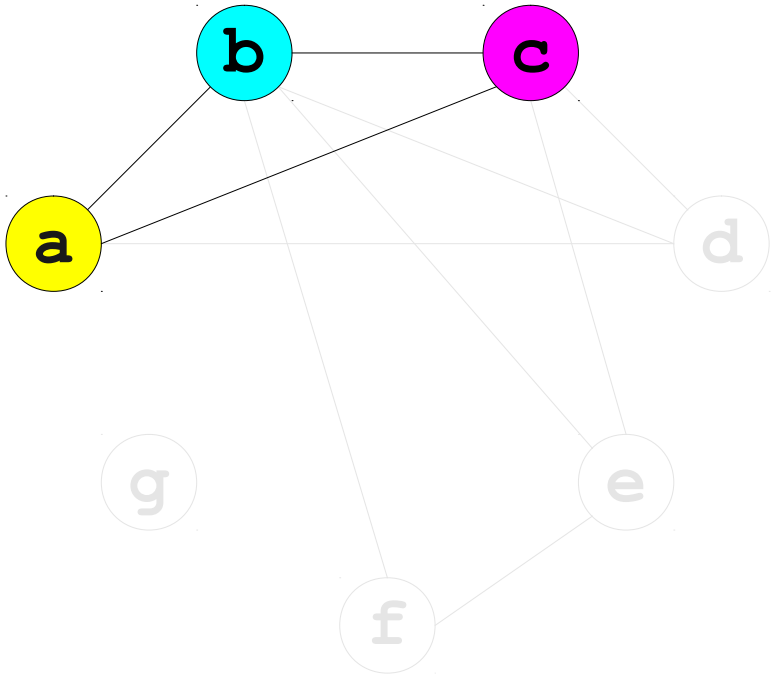


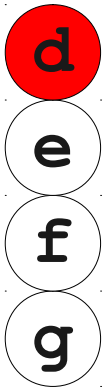


Registers

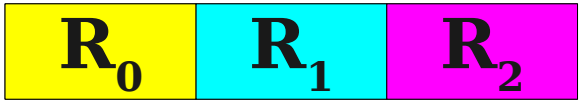


Chaitin's Algorithm Reloaded

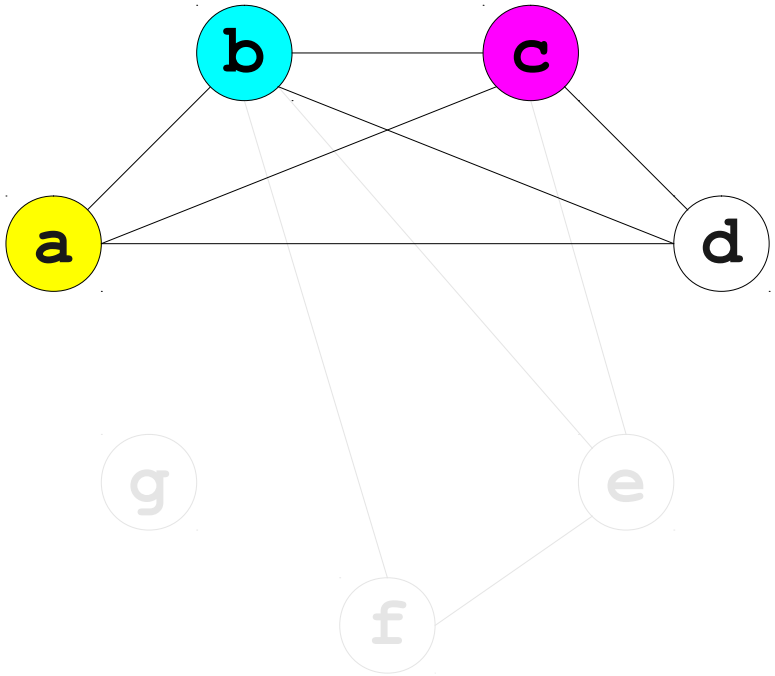




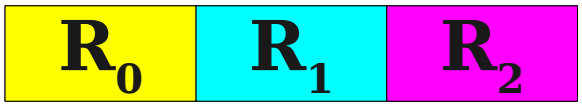
Registers



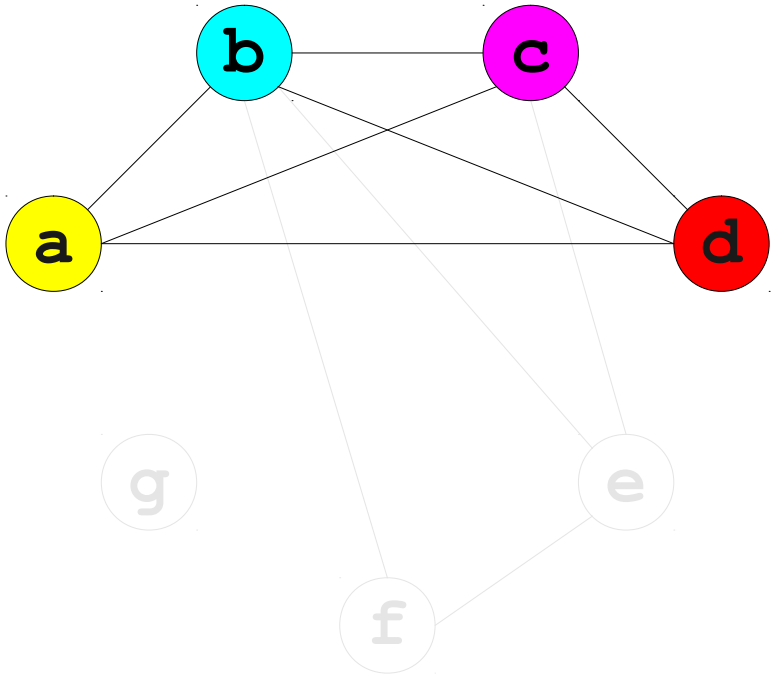
Chaitin's Algorithm Reloaded



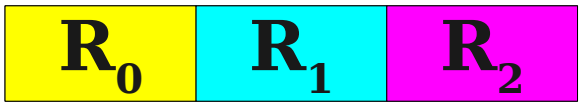
Registers



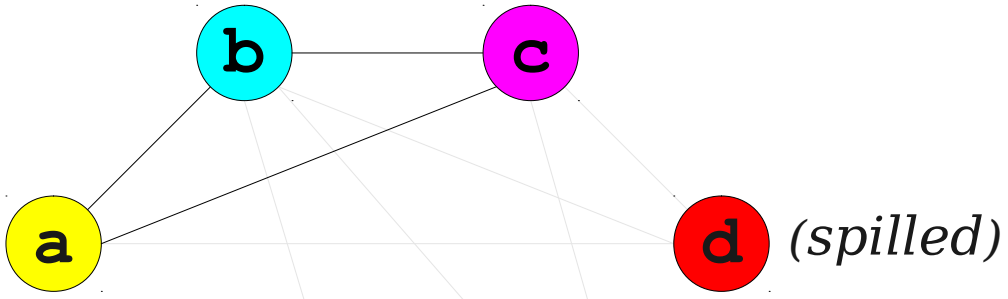
Chaitin's Algorithm Reloaded



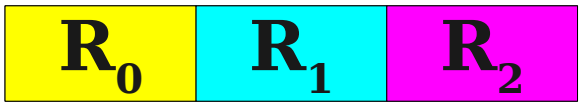
Registers



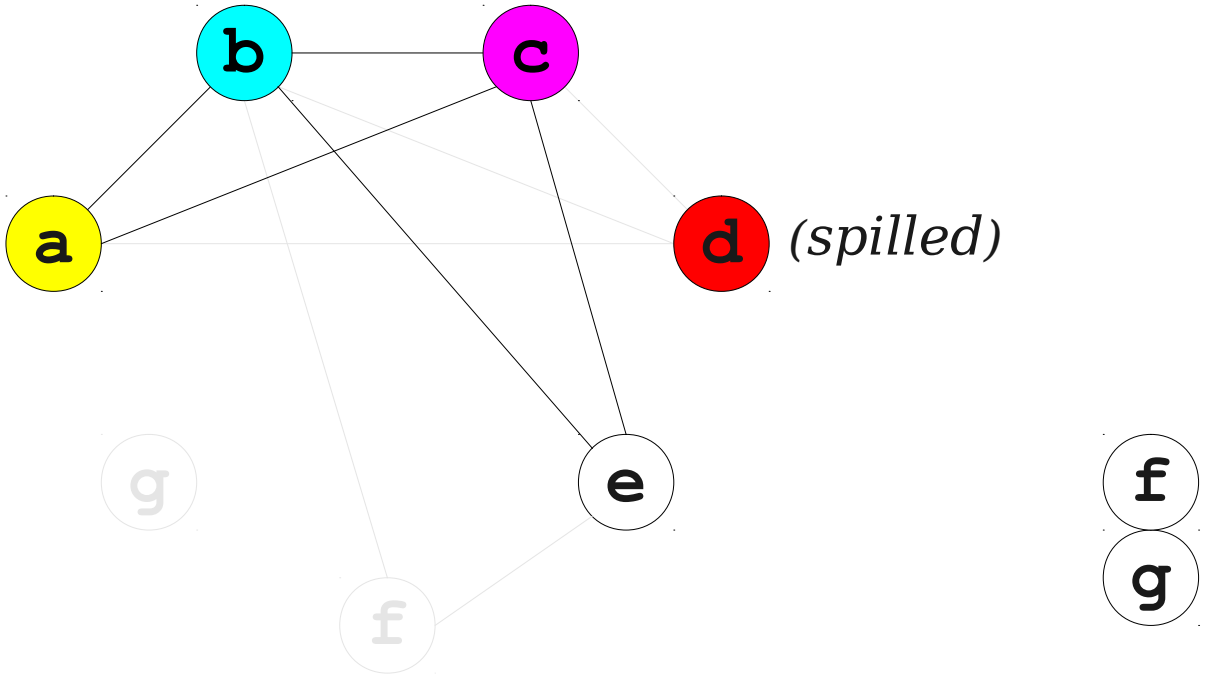
Chaitin's Algorithm Reloaded



Registers



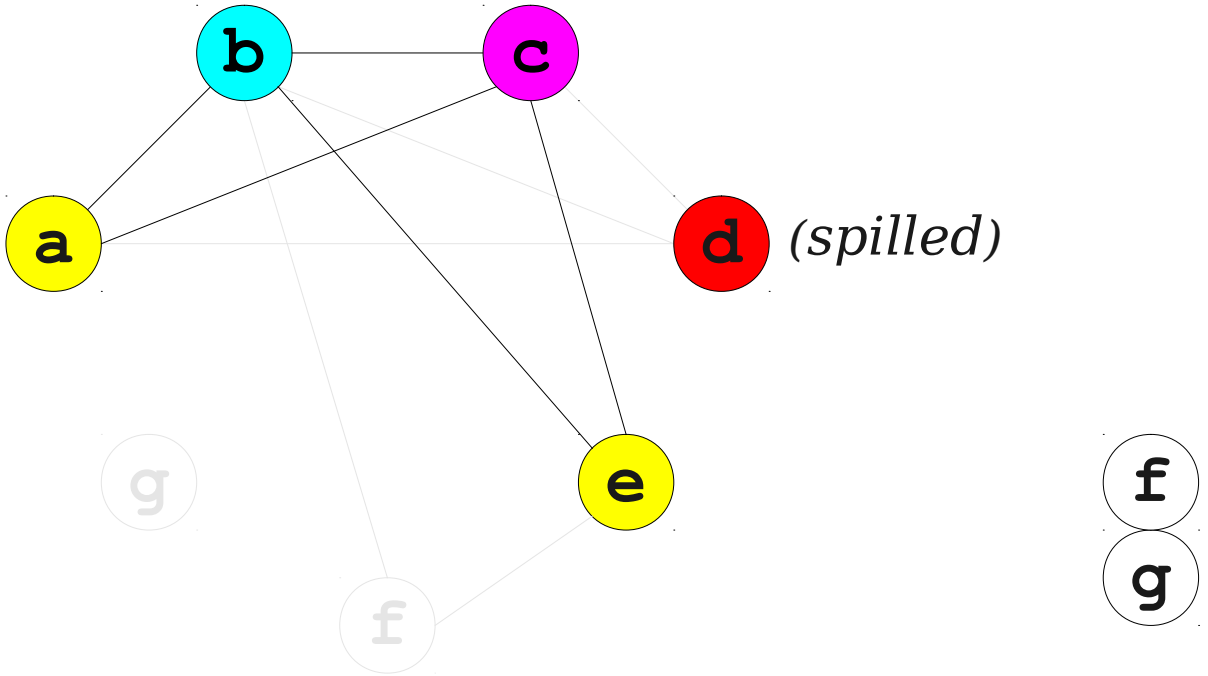
Chaitin's Algorithm Reloaded



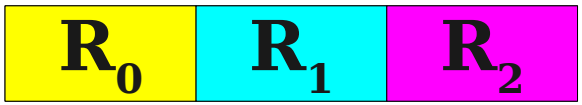
Registers

R_0	R_1	R_2

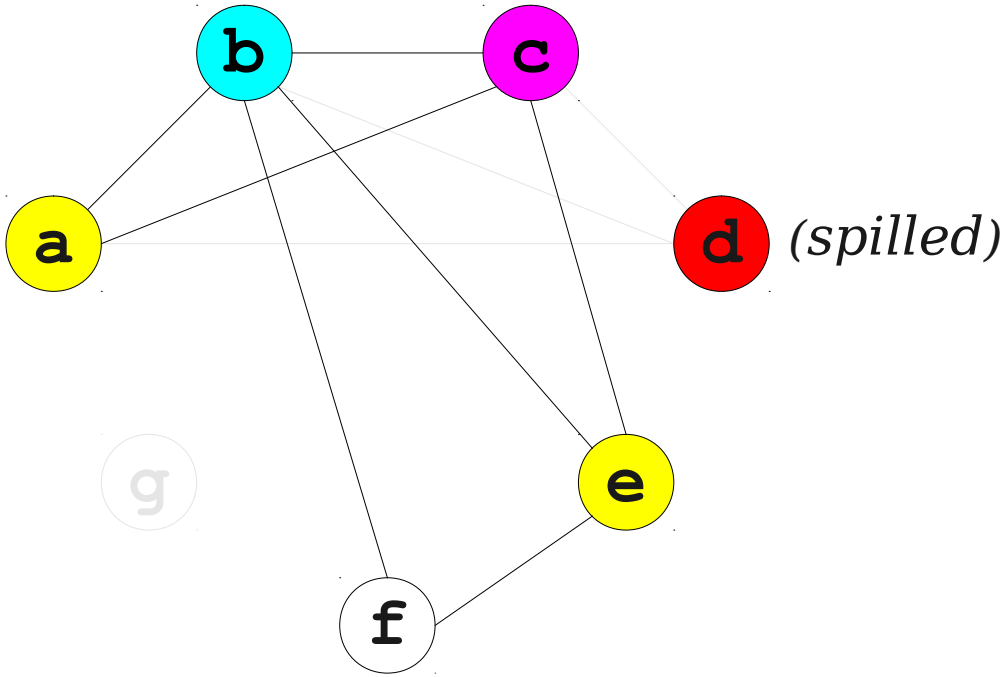
Chaitin's Algorithm Reloaded



Registers



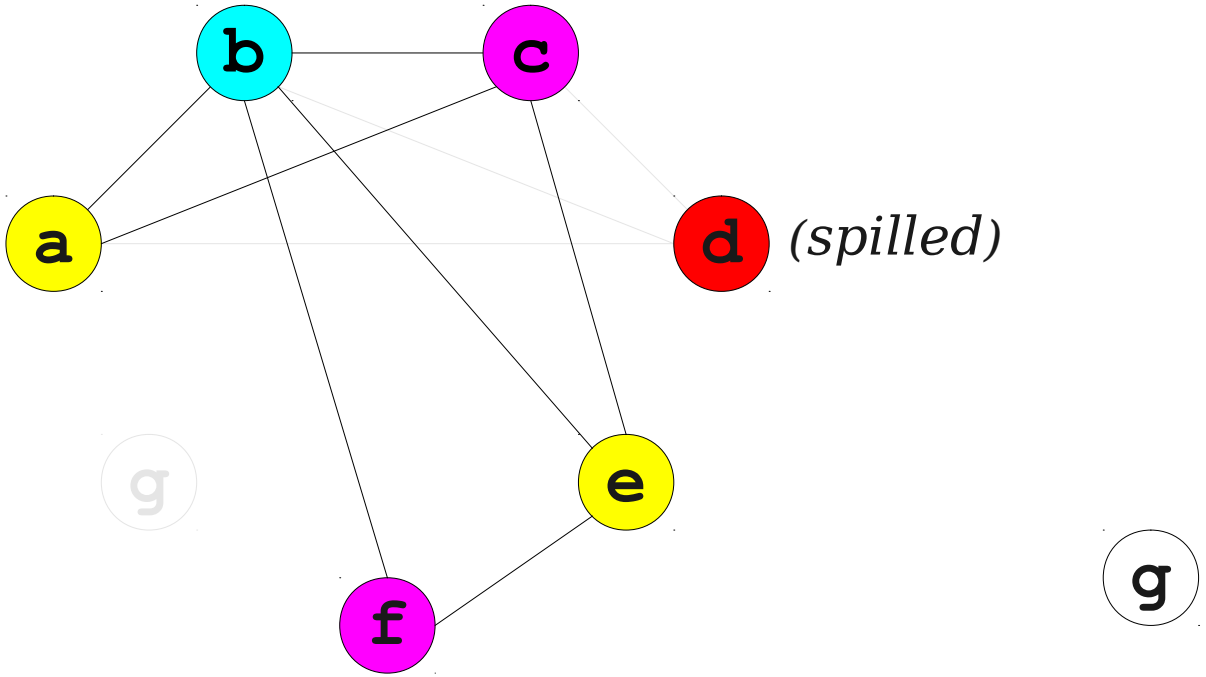
Chaitin's Algorithm Reloaded



Registers

R_0	R_1	R_2

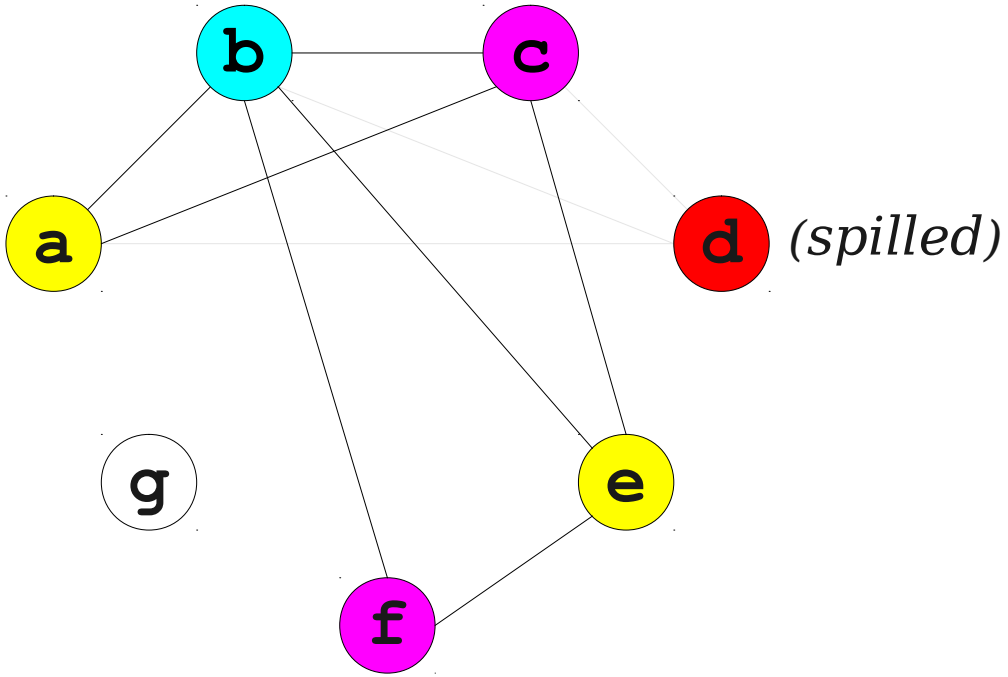
Chaitin's Algorithm Reloaded



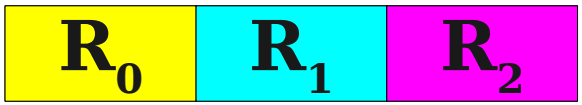
Registers

R_0	R_1	R_2

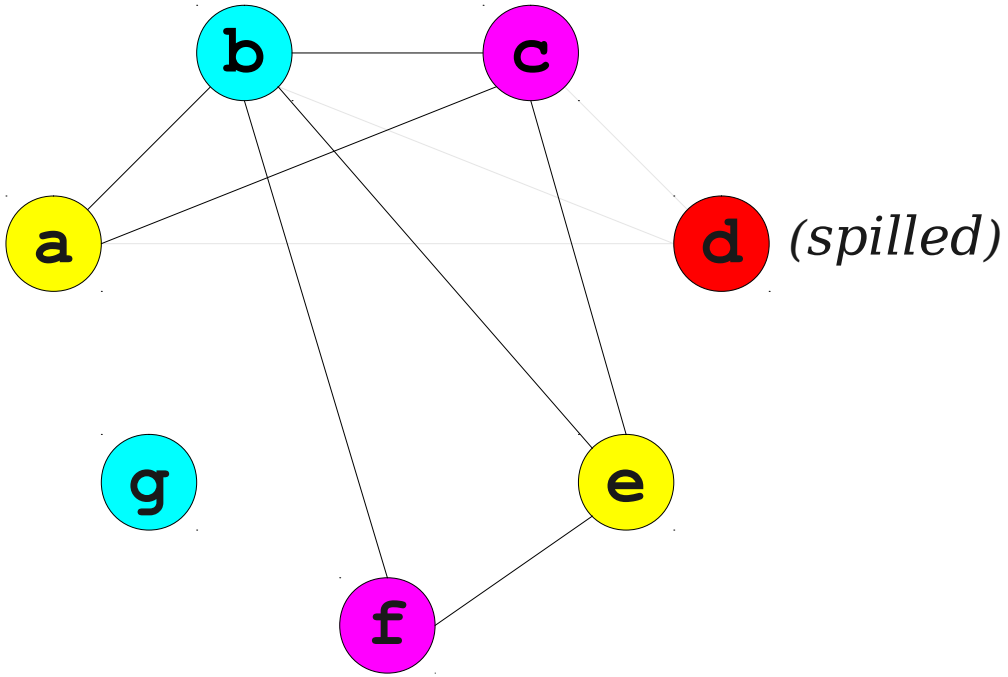
Chaitin's Algorithm Reloaded



Registers



Chaitin's Algorithm Reloaded

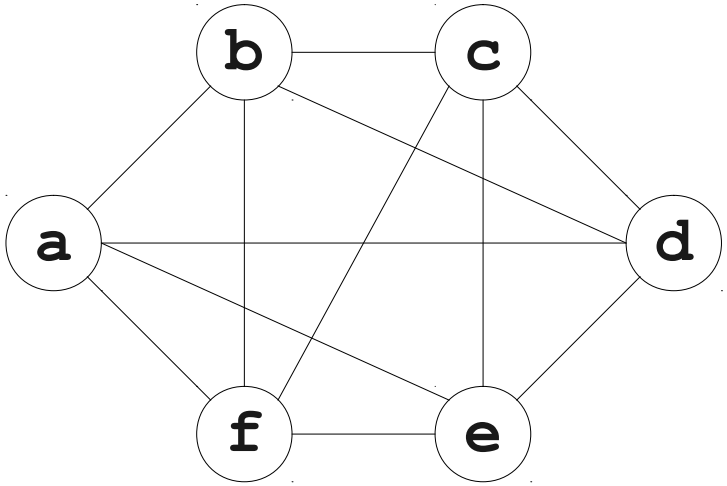


Registers

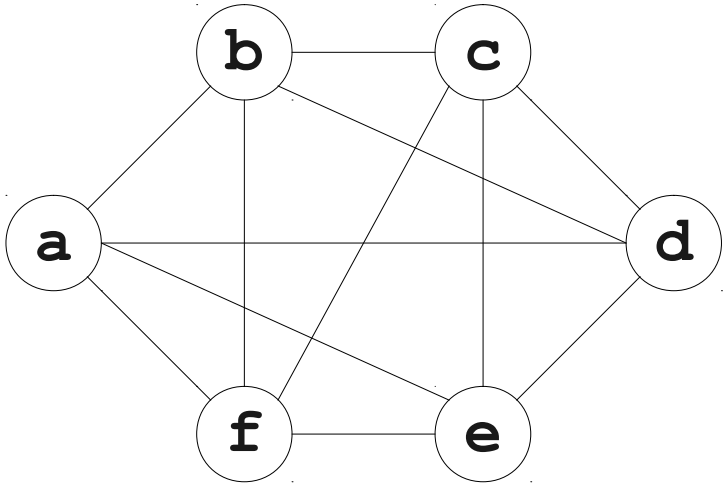
R_0	R_1	R_2

Another Example

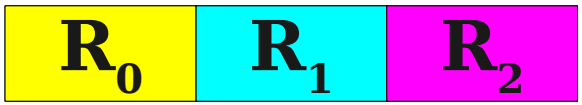
Another Example



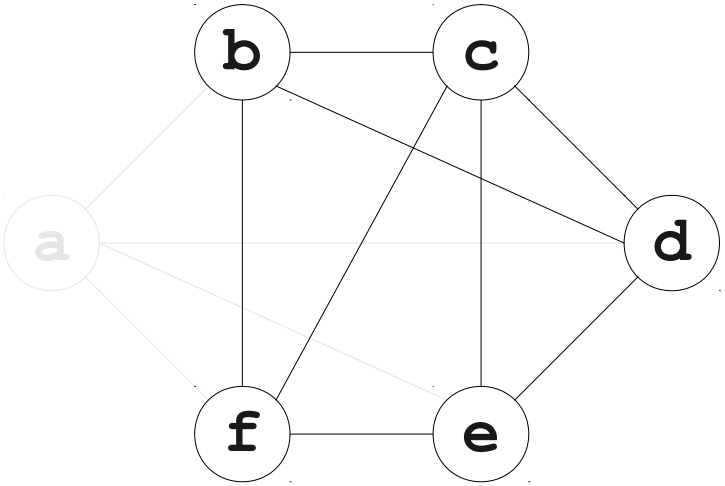
Another Example



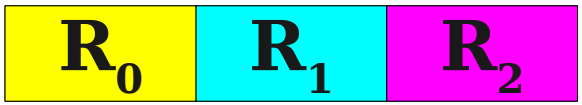
Registers



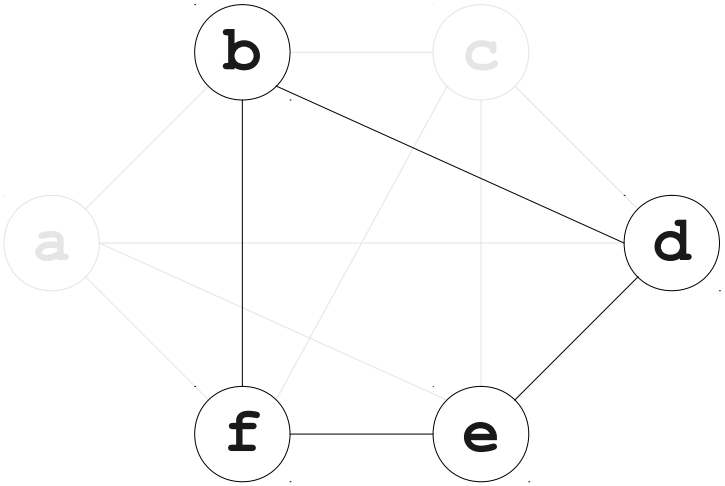
Another Example

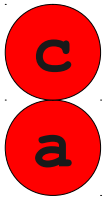


Registers

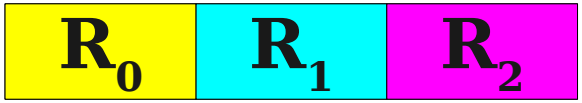


Another Example

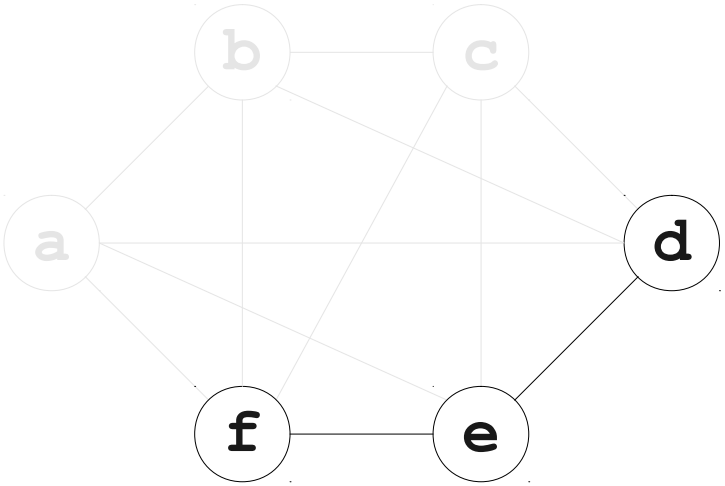


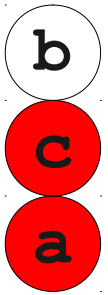


Registers

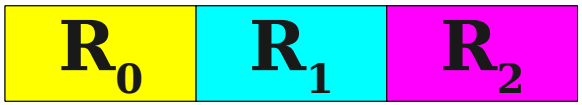


Another Example

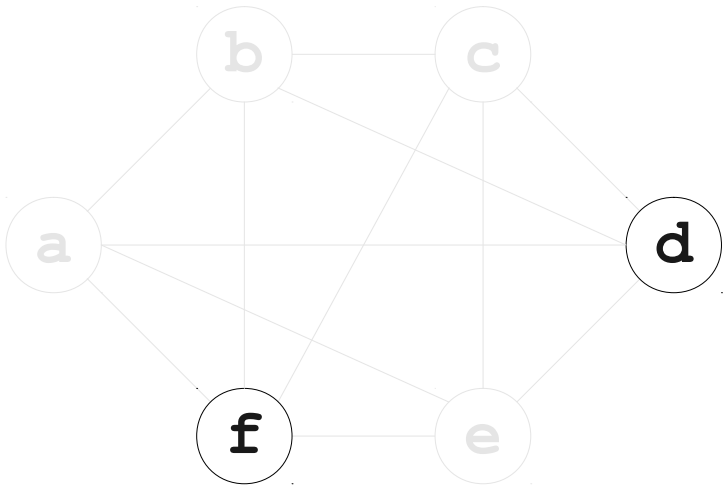


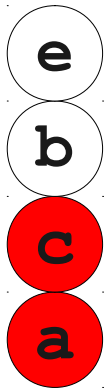


Registers

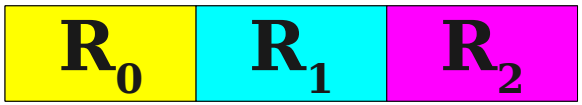


Another Example

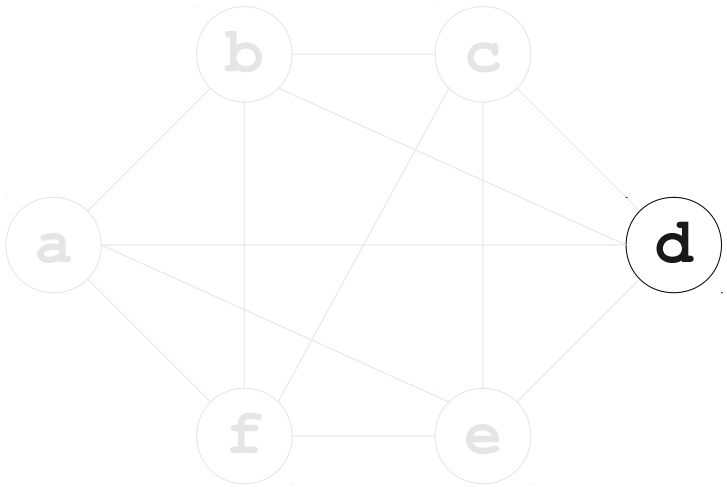


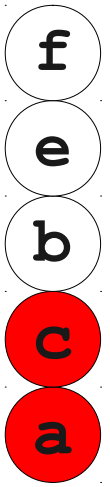


Registers

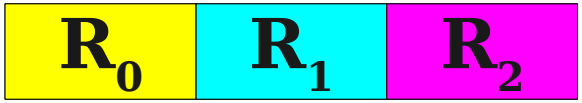


Another Example

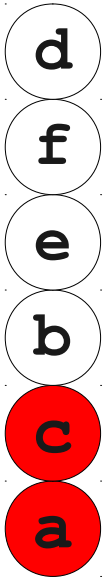




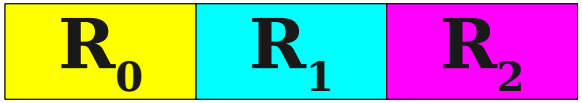
Registers



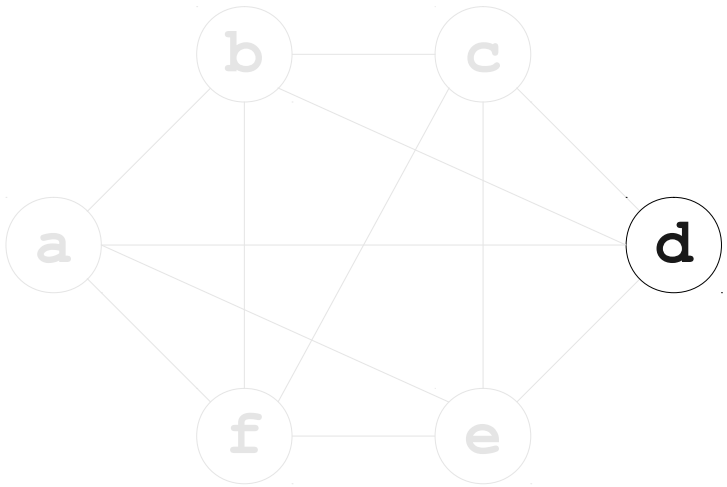
Another Example

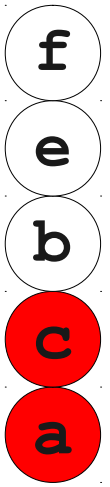


Registers

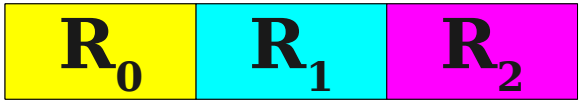


Another Example

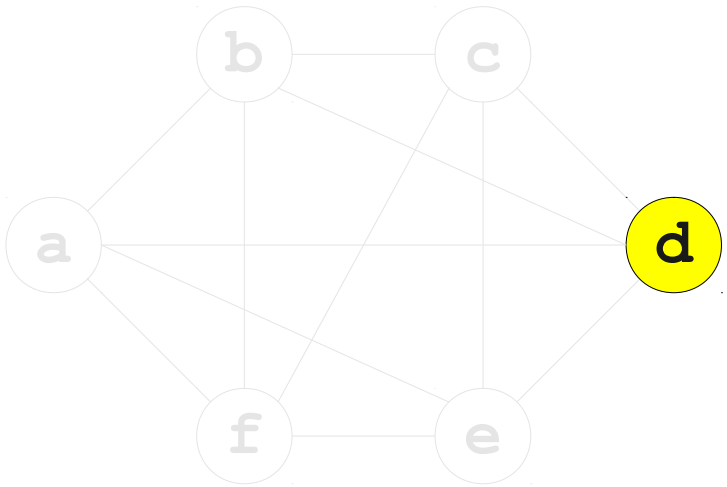


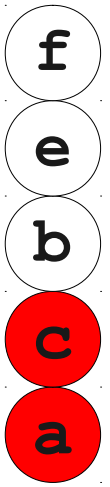


Registers

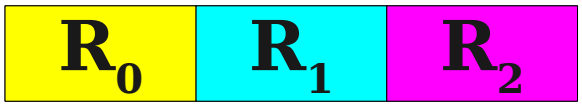


Another Example

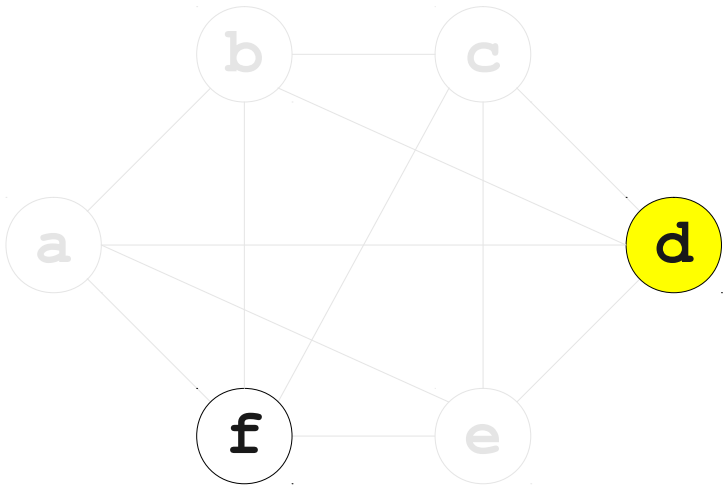


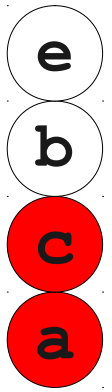


Registers

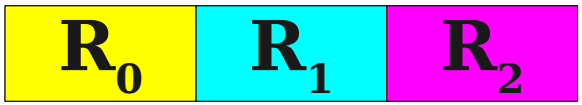


Another Example

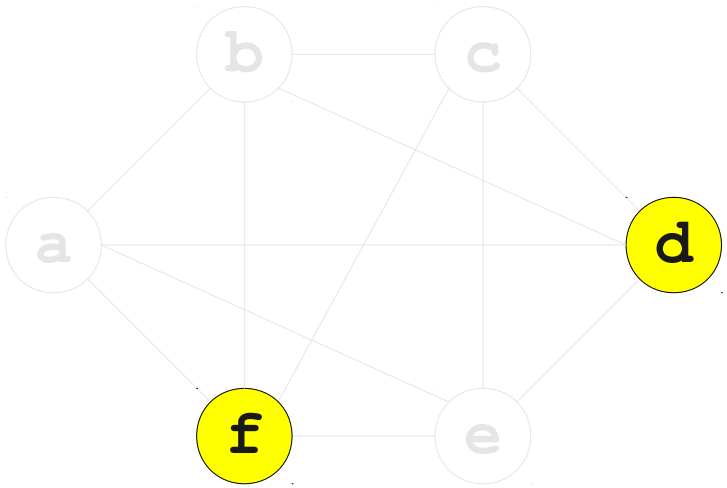


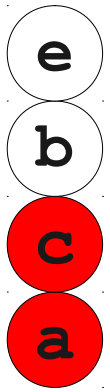


Registers

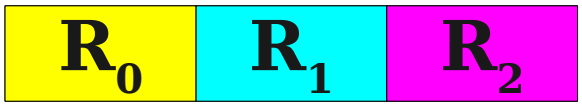


Another Example

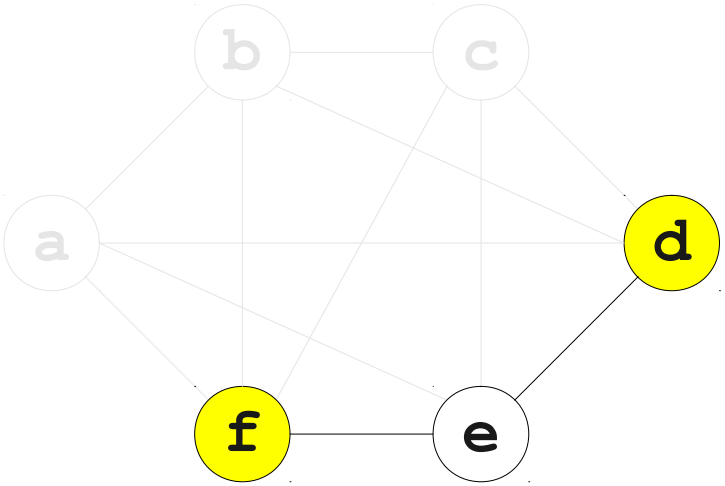


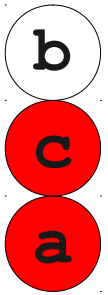


Registers

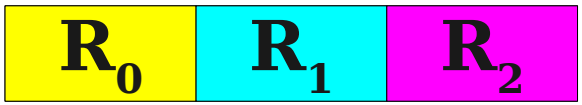


Another Example

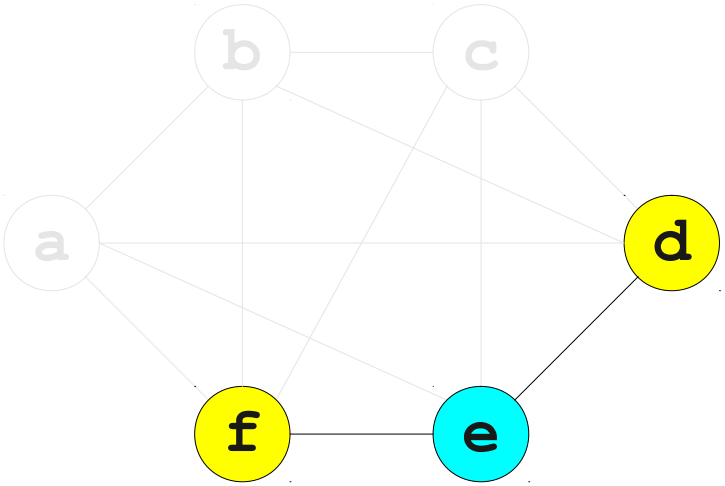


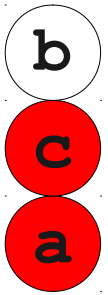


Registers

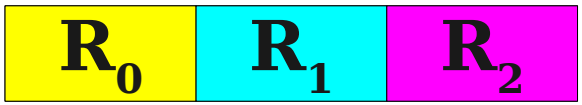


Another Example

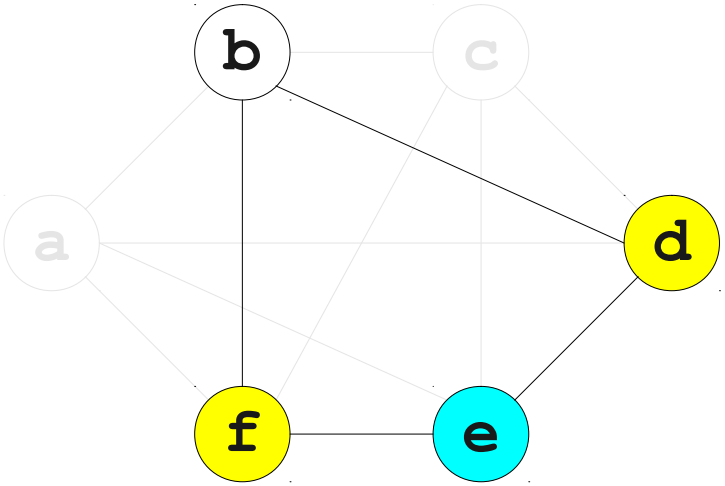


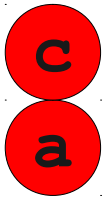


Registers

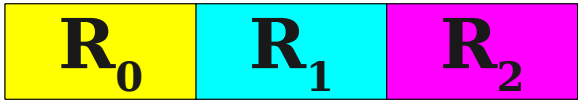


Another Example

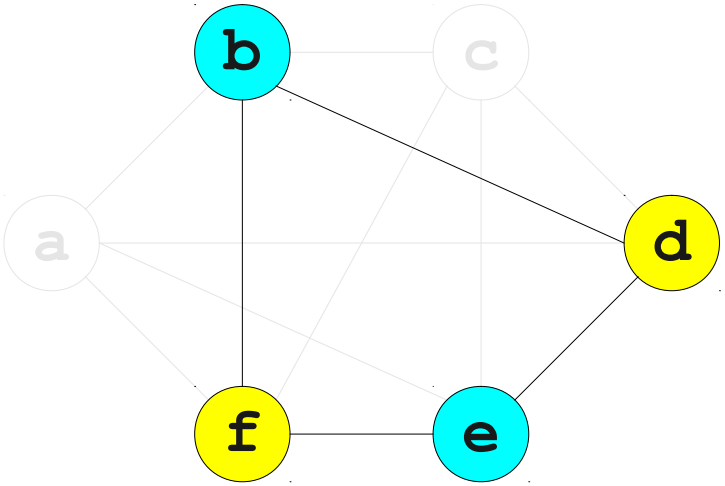


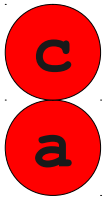


Registers

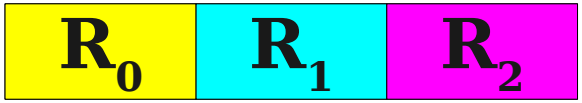


Another Example

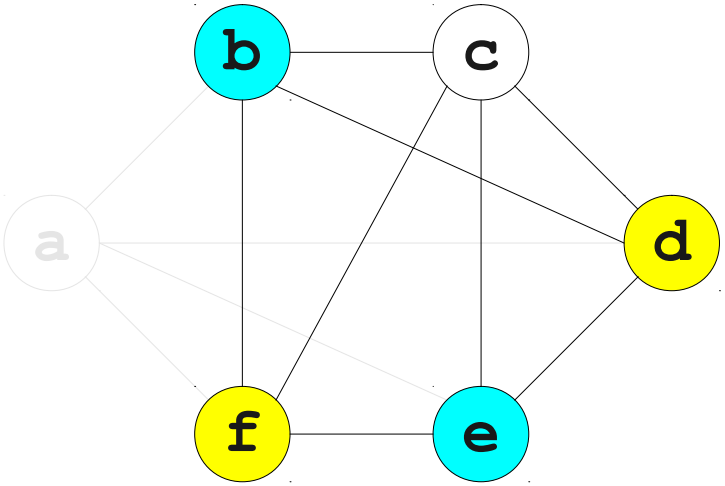




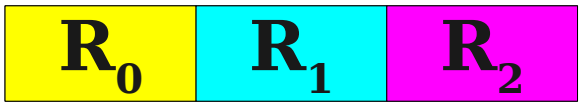
Registers



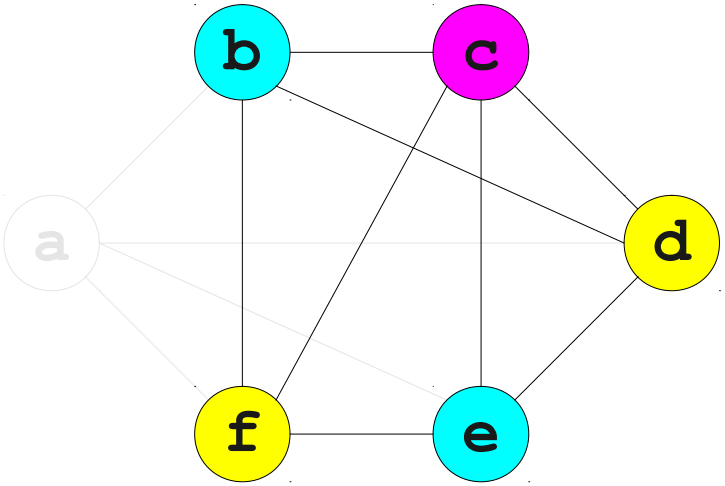
Another Example



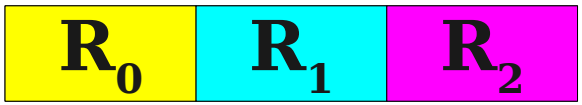
Registers



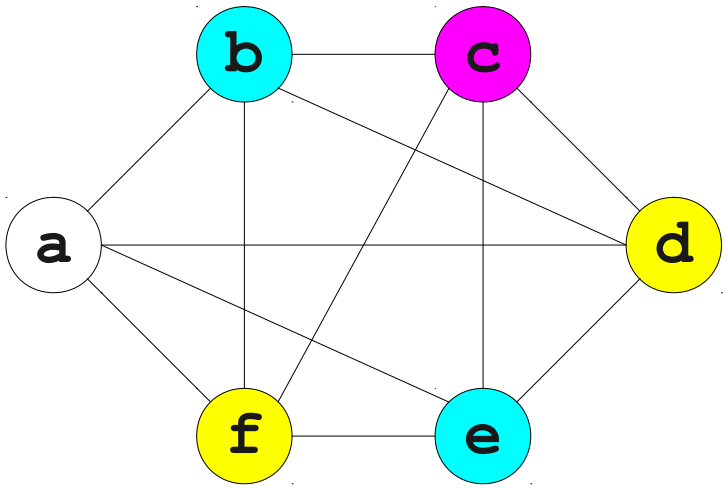
Another Example



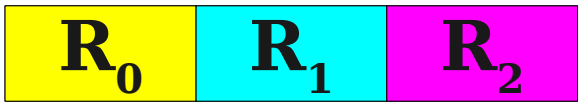
Registers



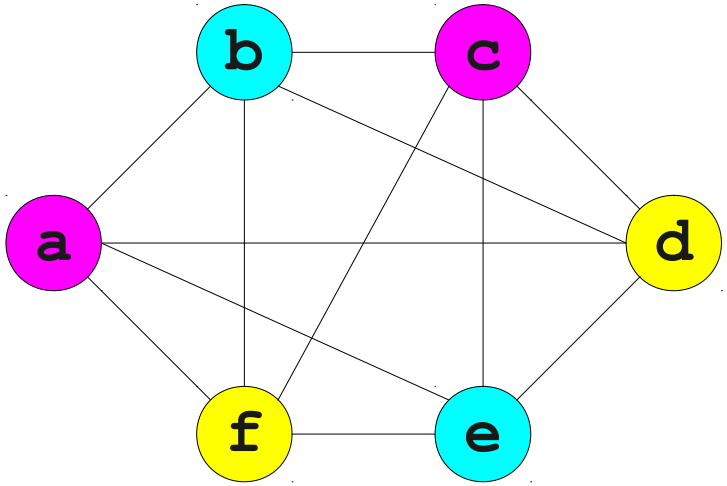
Another Example



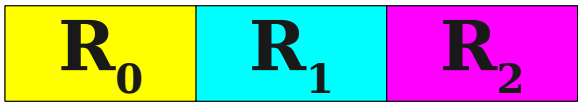
Registers



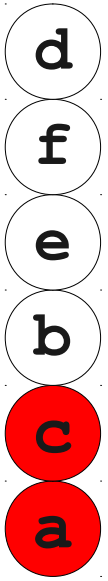
Another Example



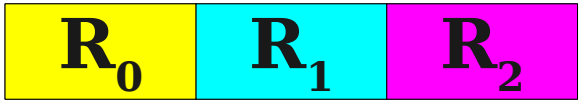
Registers



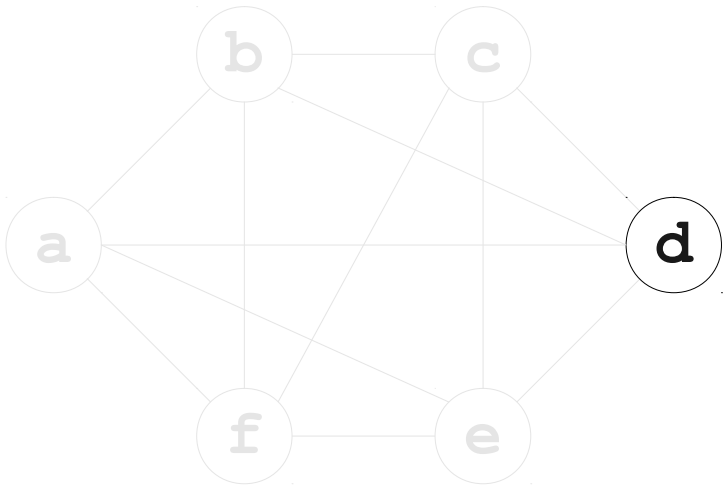
Another Example

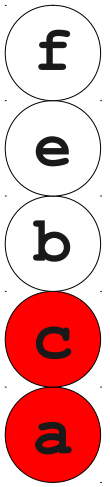


Registers

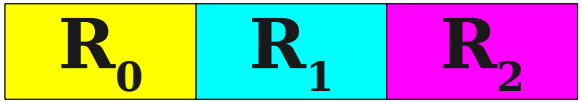


Another Example

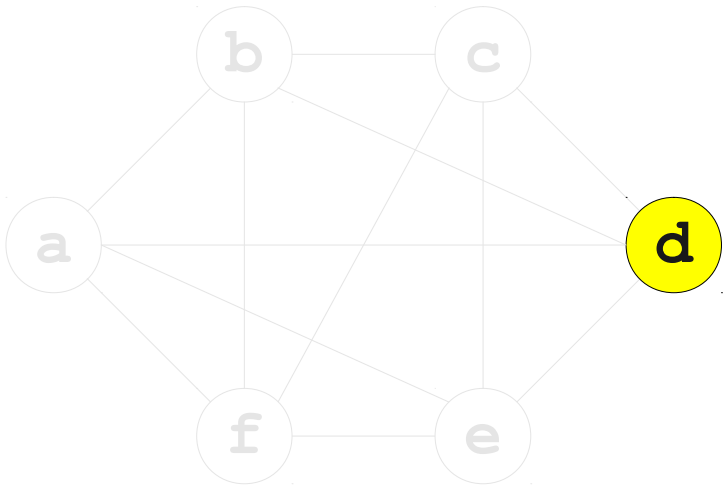


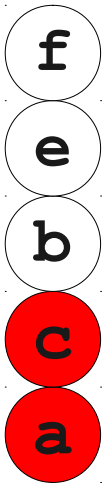


Registers

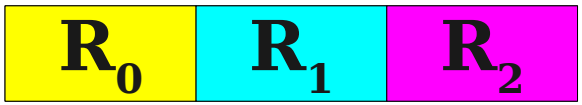


Another Example

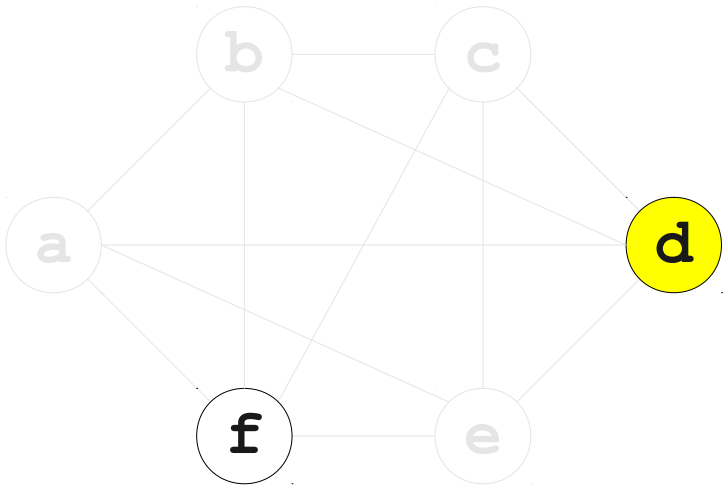


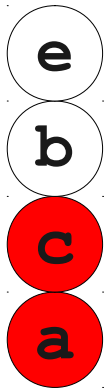


Registers

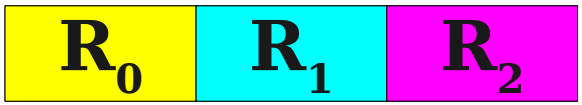


Another Example

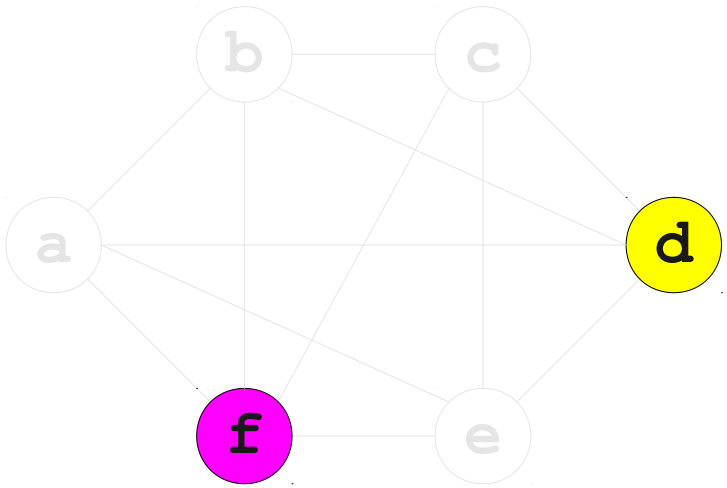


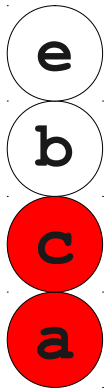


Registers

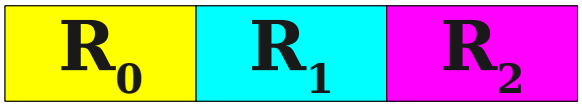


Another Example

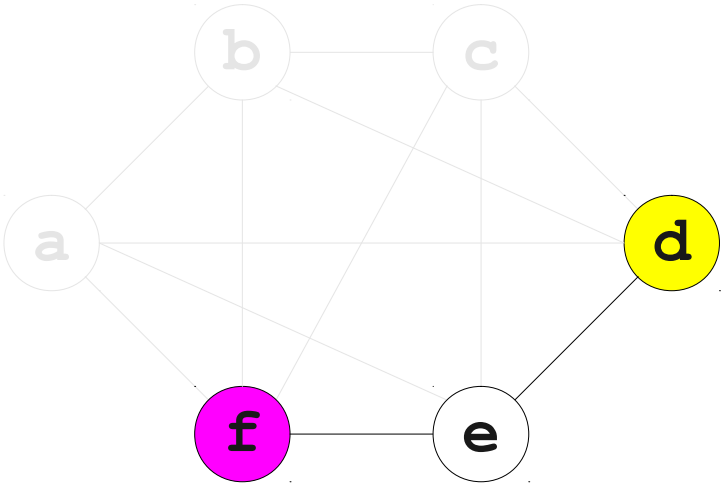


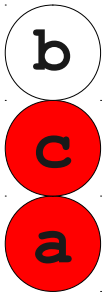


Registers

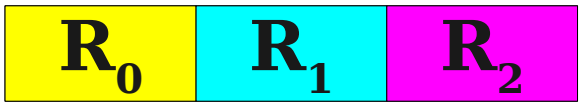


Another Example

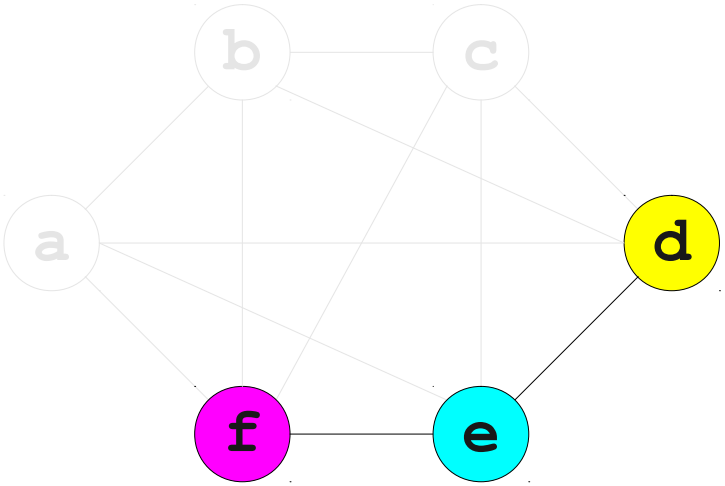


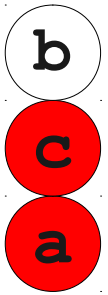


Registers

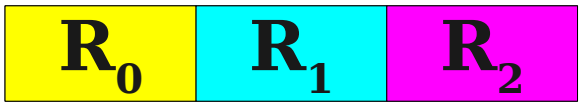


Another Example

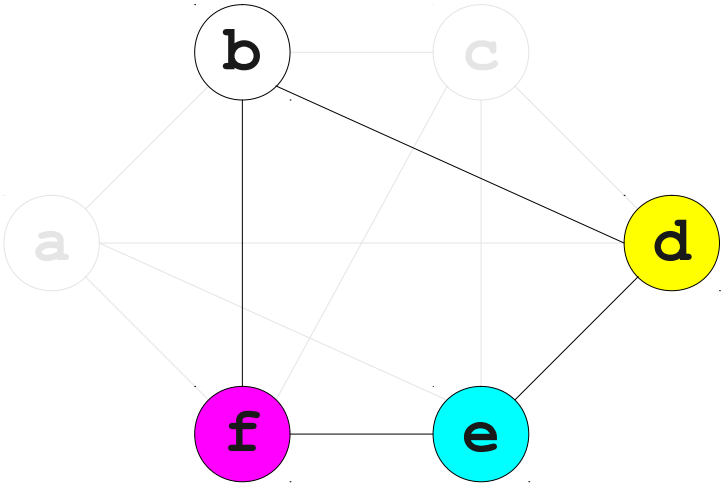


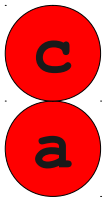


Registers

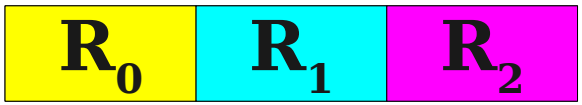


Another Example

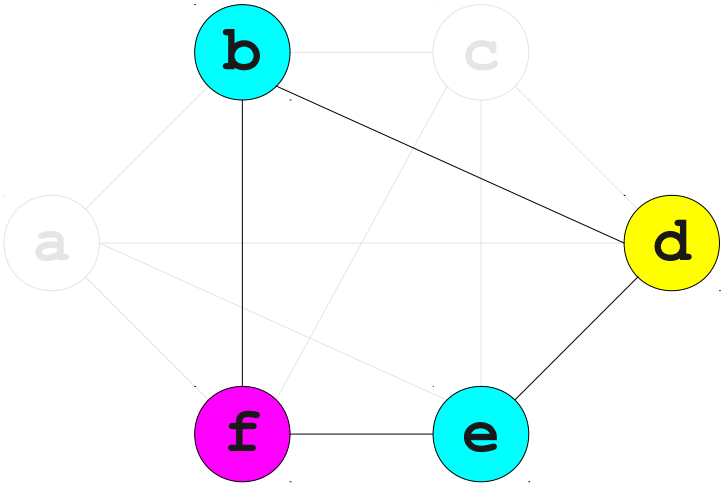


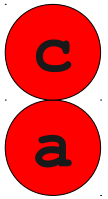


Registers

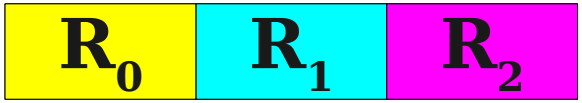


Another Example

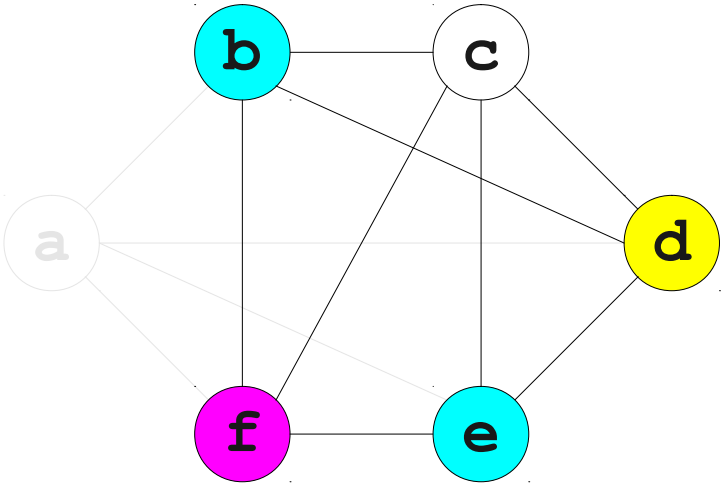




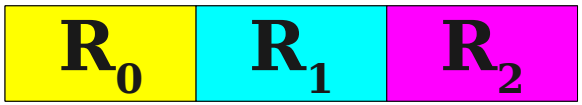
Registers



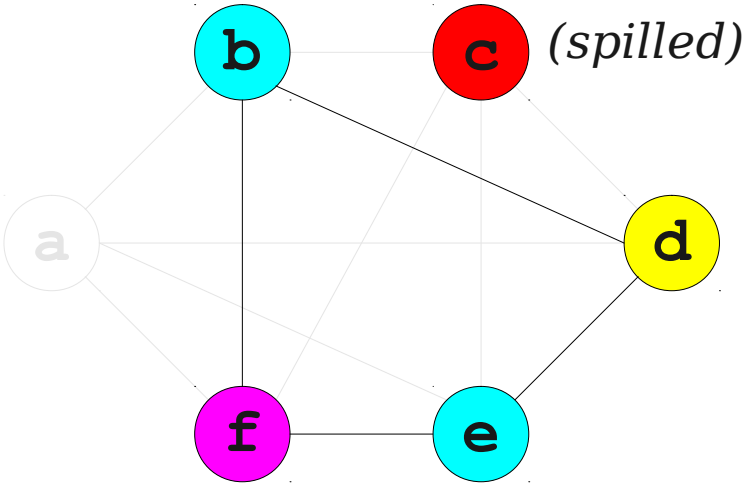
Another Example



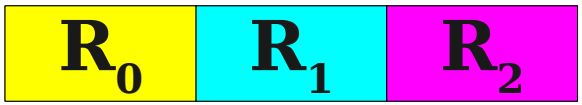
Registers



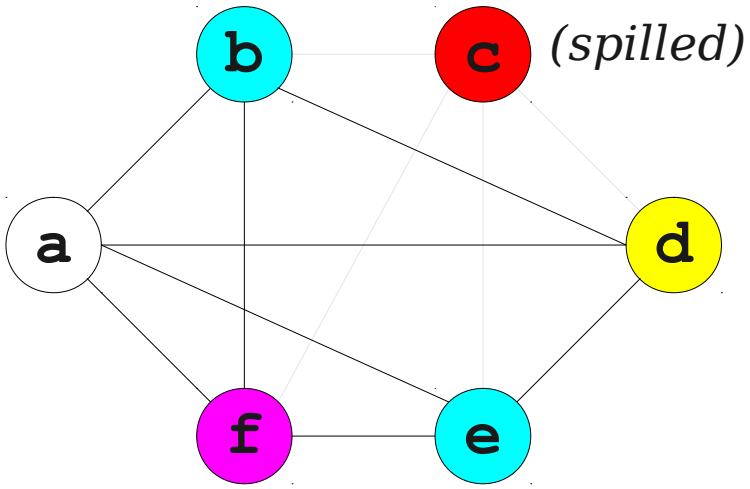
Another Example



Registers



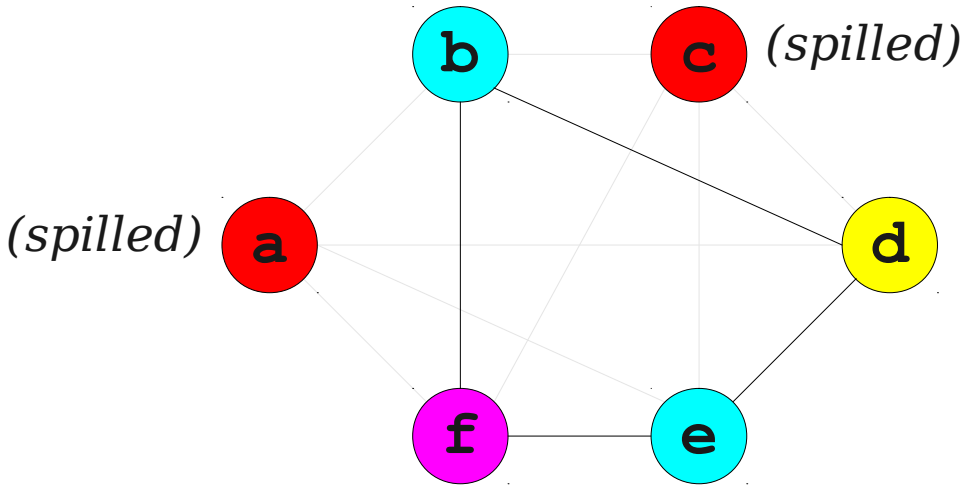
Another Example



Registers

R_0	R_1	R_2

Another Example



Registers

R_0	R_1	R_2

Chaitin's Algorithm

- Advantages:
  - For many control-flow graphs, finds an excellent assignment of variables to registers.
  - When distinguishing variables by use, produces a precise RIG.
  - Often used in production compilers like GCC.
- Disadvantages:
  - Core approach based on the NP-hard graph coloring problem.
  - Heuristic may produce pathologically worst-case assignments.

Improvements to the Algorithm

- Choose what to spill intelligently.
  - Use heuristics (least-commonly used, greatest improvement, etc.) to determine what to spill.
- Handle spilling intelligently.
  - When spilling a variable, recompute the RIG based on the spill and use a new coloring to find a register.

Summary of Register Allocation

- Critical step in all optimizing compilers.
- The **linear scan** algorithm uses **live intervals** to greedily assign variables to registers.
  - Often used in JIT compilers due to efficiency.
- **Chaitin's algorithm** uses the **register interference graph** (based on **live ranges**) and **graph coloring** to assign registers.
  - The basis for the technique used in GCC.

Chapter 9

Machine-independent Optimizations

Code Optimization

- High-level language constructs can introduce substantial run-time overhead if naively each construct is independently translated into machine code
- Code optimization or code improvement
  - Elimination of unnecessary instructions in object code
  - Replacement of one sequence of instructions by a faster sequence of instructions

Code Optimization

- Local Optimization
  - ➢Code improvement within a basic block
  - ➢Covered in chapter 8
- Global Optimization
  - ➢Code improvement across basic blocks
  - ➢Focus of this chapter

Global Optimizations

- Most global optimizations are based on data-flow analysis (DFA)
- DFA are algorithms that gather information about a program
- For each instruction in program, specifies some property that must hold every time that instruction is executed
- Example:
  - Constant-propagation analysis: checks if for each point in the program and for each variable used by the program, whether the variable has a unique constant value at that point
  - Liveliness analysis: checks for each point in the program, whether the value hold by a particular variable at that point is sure to be overwritten before it is read

Principle Sources of Optimization

- Compiler must preserve the semantics of the original program
- A compiler cannot understand enough about a program to replace with an efficient algorithm!
- A compiler knows only how to apply relatively low-level semantic transformations such that performing the same operation on the same values yields the same result.
- Examples -
  - Use algebraic identities such as replacing *i+0* with *i*
  - Remove redundant operations and codes

Causes of Redundancy

- There are many redundant operations in a typical program.
- Sometime redundancies are introduced at the source level by the programmer for convenience
  - Cut & paste
- More often the redundancy is a side effect of writing code in high level language such as an array access, *A[i][j]*
  - During compilation, such high-level data-structure accesses expands into a number of low-level arithmetic operations, such as the computation of the location of the *(i, j)*th element of a matrix *A*.

Example

Quick Sort(C)

```
void quicksort(int m, int n)
    /* recursively sorts a[m] through a[n] */
{
    int i, j;
    int v, x;
    if (n <= m) return;
    /* fragment begins here */
    i = m-1; j = n; v = a[n];
    while (1) {
        do i = i+1; while (a[i] < v);
        do j = j-1; while (a[j] > v);
        if (i >= j) break;
        x = a[i]; a[i] = a[j]; a[j] = x;
    }
    x = a[i]; a[i] = a[n]; a[n] = x;
    /* fragment ends here */
    quicksort(m,j); quicksort(i+1,n);
}
```

Quick Sort(TAC)

Basic Block

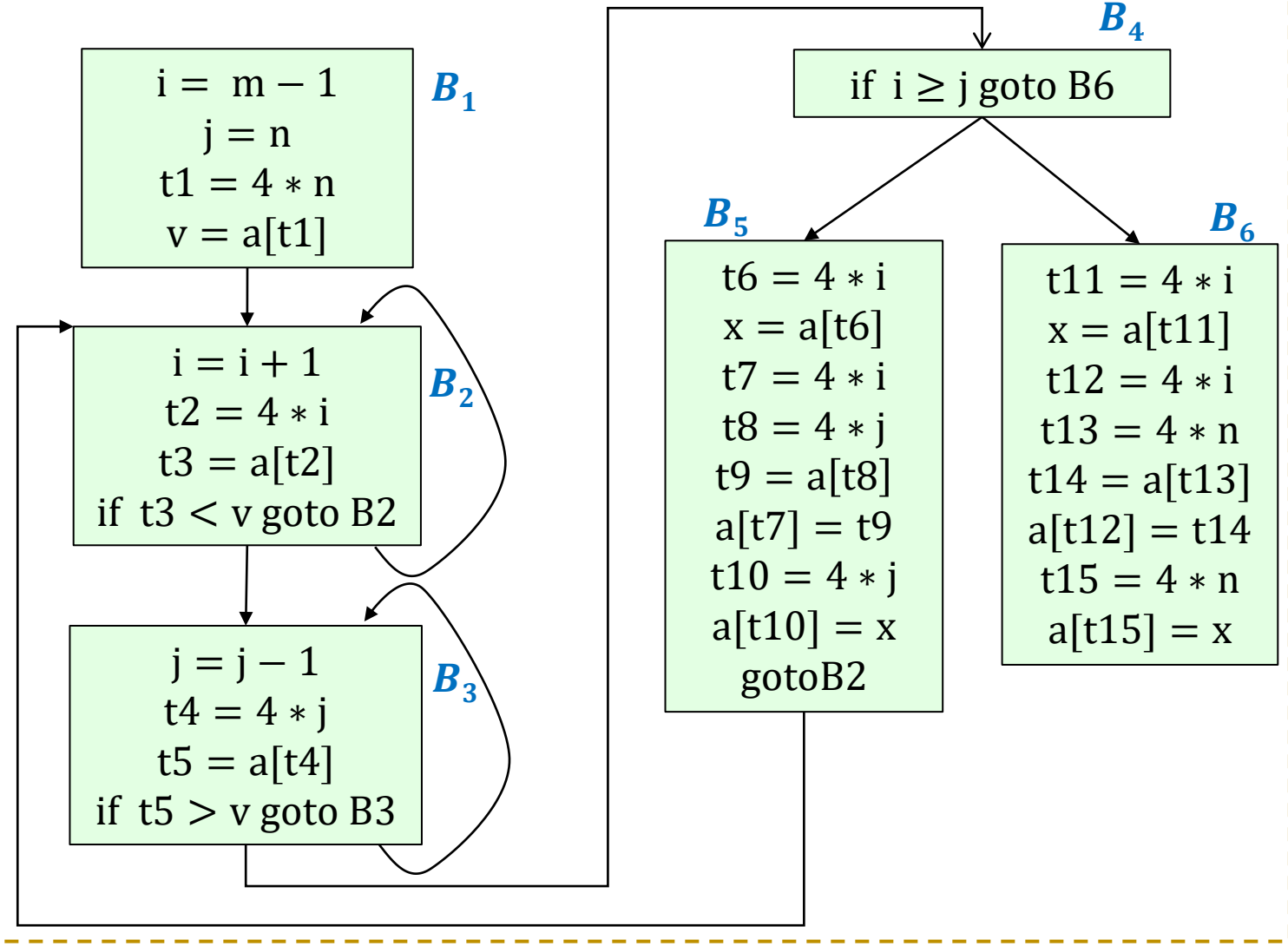
```
(1)   i = m-1
(2)   j = n
(3)   t1 = 4*n
(4)   v = a[t1]
(5)   i = i+1
(6)   t2 = 4*i
(7)   t3 = a[t2]
(8)   if t3<v goto (5)
(9)   j = j-1
(10)  t4 = 4*j
(11)  t5 = a[t4]
(12)  if t5>v goto (9)
(13)  if i>=j goto (23)
(14)  t6 = 4*i
(15)  x = a[t6]
(16)  t7 = 4*i
(17)  t8 = 4*j
(18)  t9 = a[t8]
(19)  a[t7] = t9
(20)  t10 = 4*j
(21)  a[t10] = x
(22)  goto (5)
(23)  t11 = 4*i
(24)  x  = a[t11]
(25)  t12 = 4*i
(26)  t13 = 4*n
(27)  t14 = a[t13]
(28)  a[t12] = t14
(29)  t15 = 4*n
(30)  a[t15] = x
```

Example

Quick Sort(C)

```
void quicksort(int m, int n)
    /* recursively sorts a[m] through a[n] */
{
    int i, j;
    int v, x;
    if (n <= m) return;
    /* fragment begins here */
    i = m-1; j = n; v = a[n];
    while (1) {
        do i = i+1; while (a[i] < v);
        do j = j-1; while (a[j] > v);
        if (i >= j) break;
        x = a[i]; a[i] = a[j]; a[j] = x;
    }
    x = a[i]; a[i] = a[n]; a[n] = x;
    /* fragment ends here */
    quicksort(m,j); quicksort(i+1,n);
}
```

Quick Sort(Flow Graph)

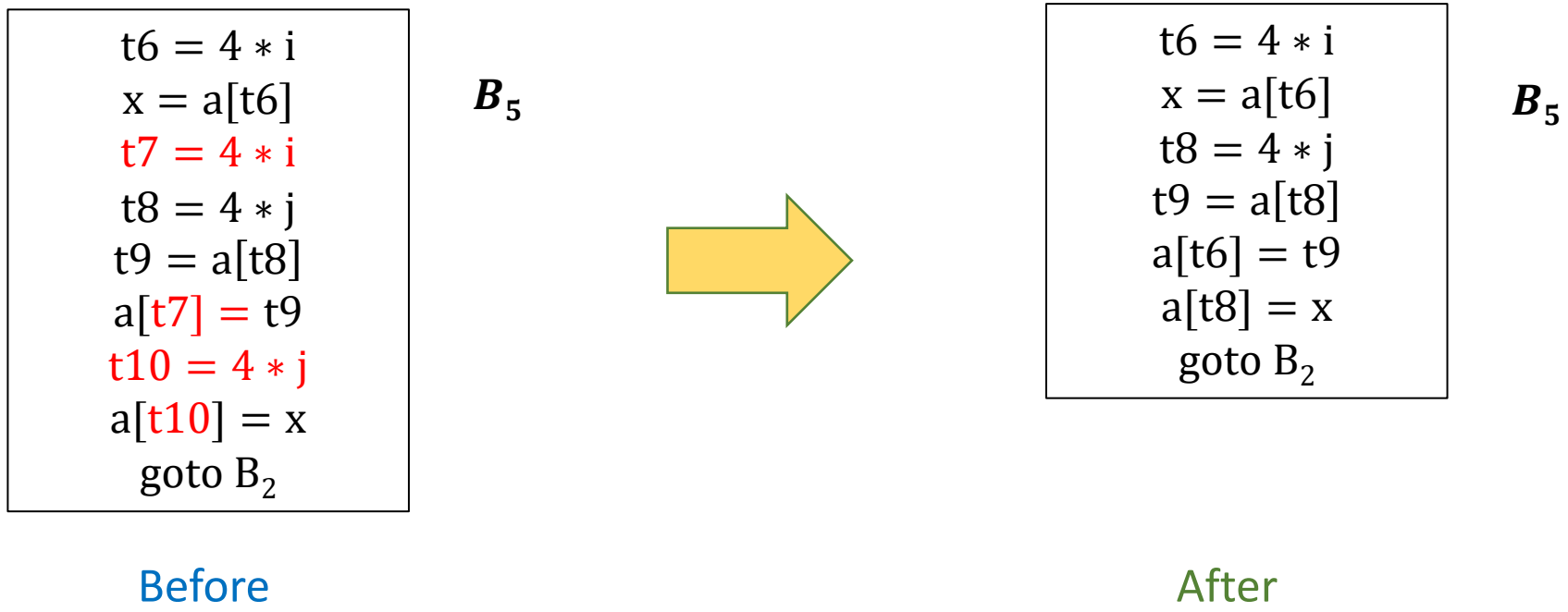


Semantic-Preserving Transformations

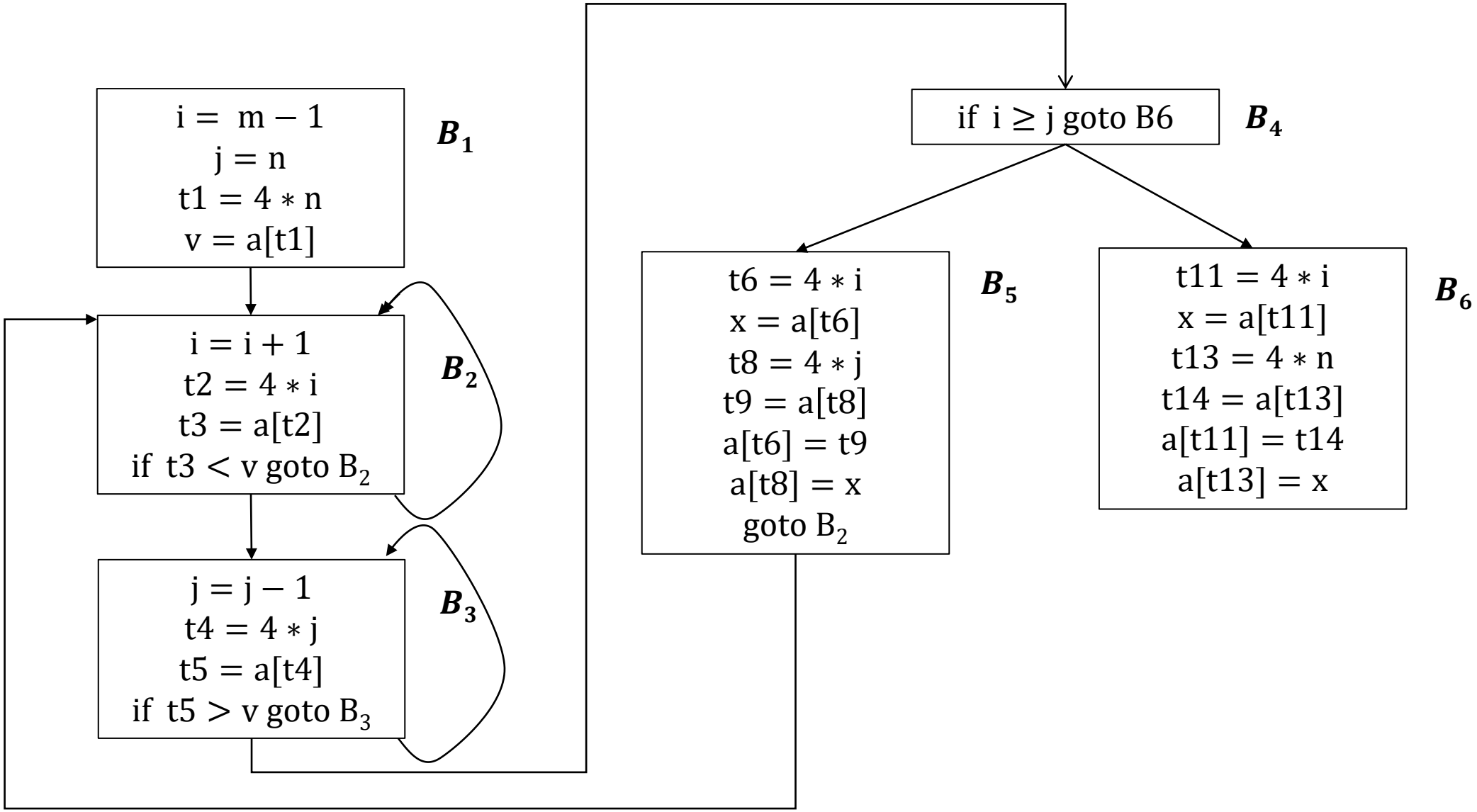
- Improve code preserving its semantics
- Common Examples:
  - Common subexpression elimination
  - Copy propagation
  - Constant folding
  - Dead-code elimination
  - Code motion
  - Induction variables and reduction in strength

Common-Subexpression Elimination

- *Local Common Subexpressions Elimination*

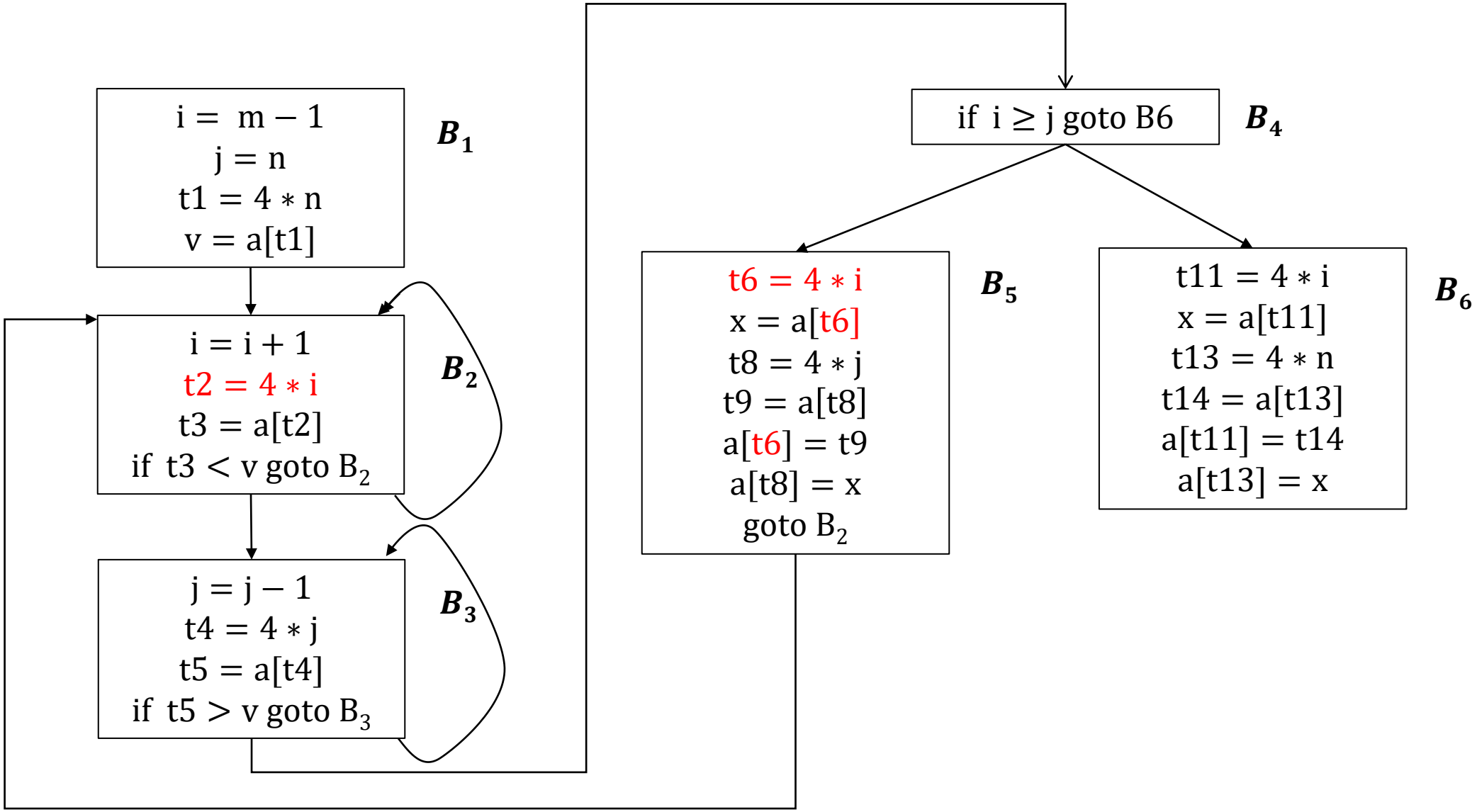


Global Common Subexpressions



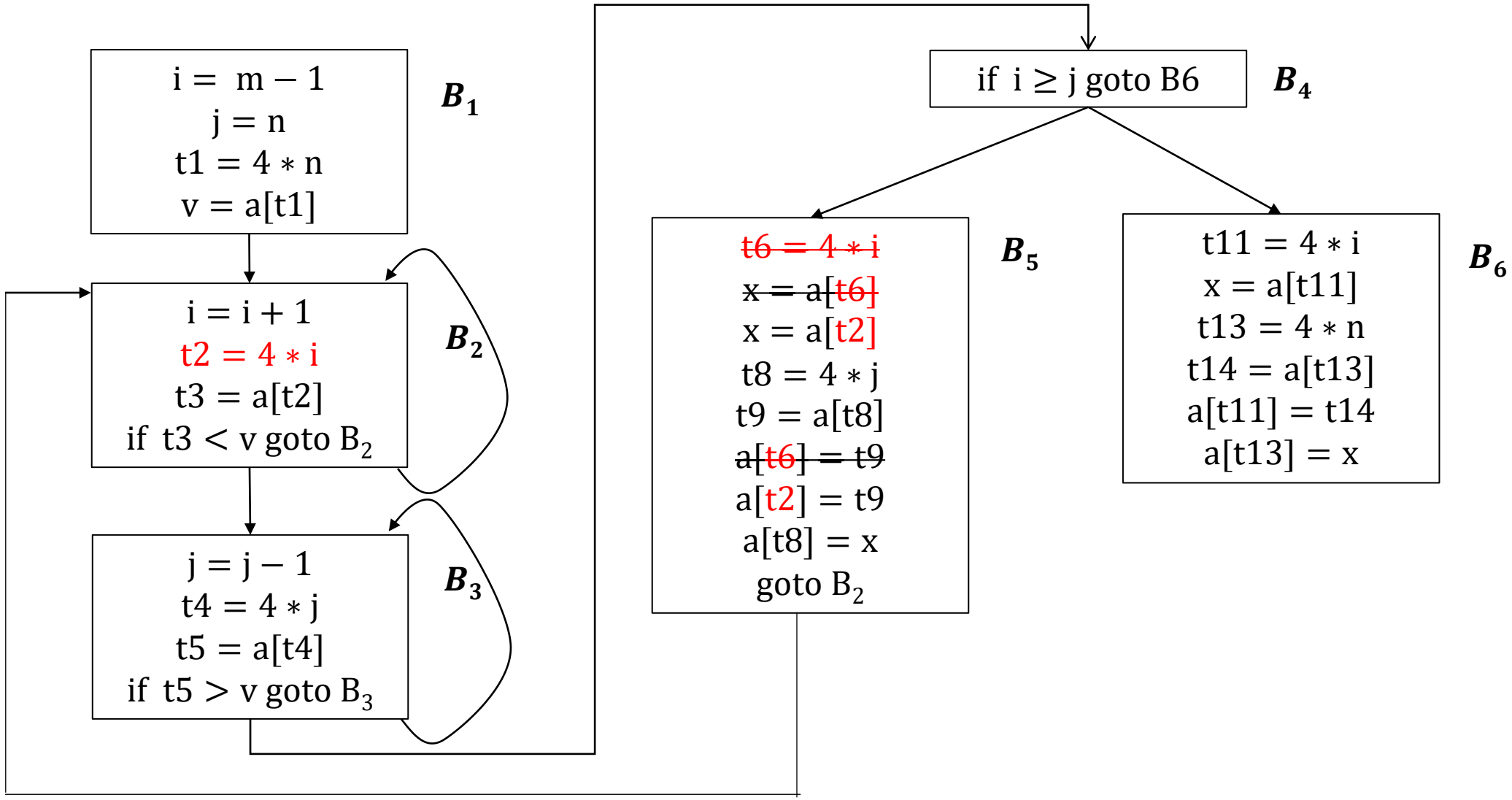
Global Common Subexpressions

i and t2 do not change when block B5 starts to execute

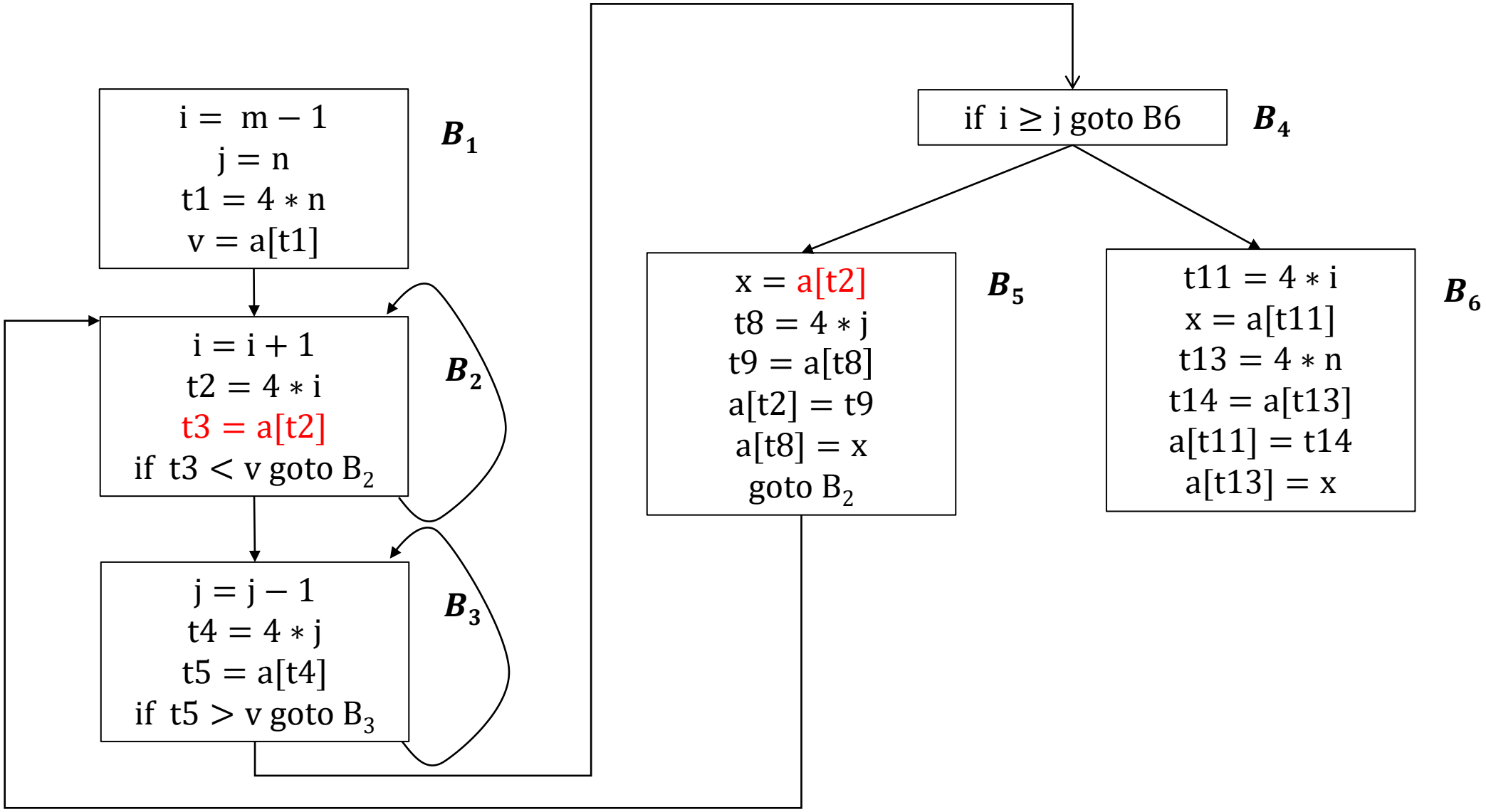


Global Common Subexpressions

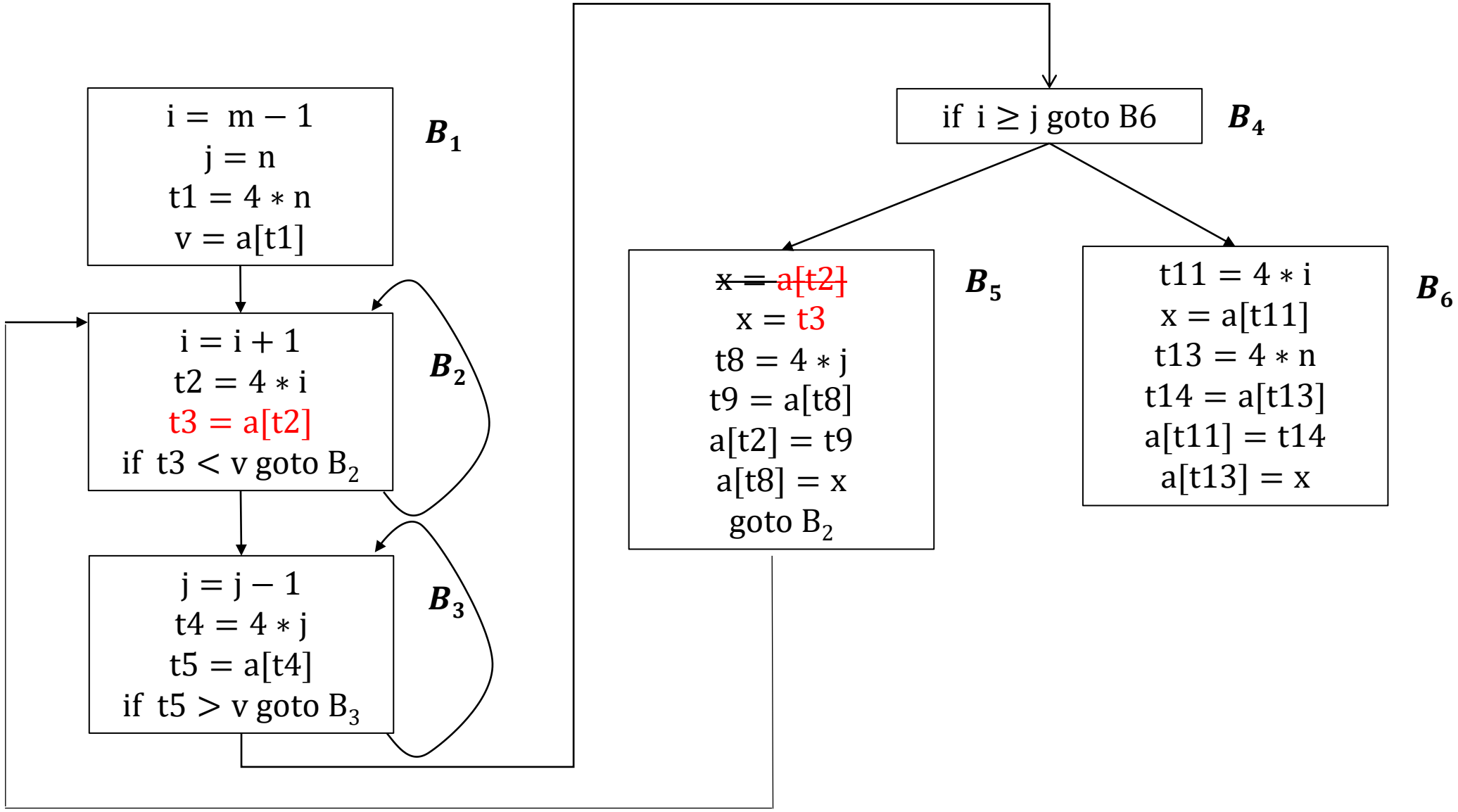
No need to calculate t6 and t6 is replaced with t2



Global Common Subexpressions

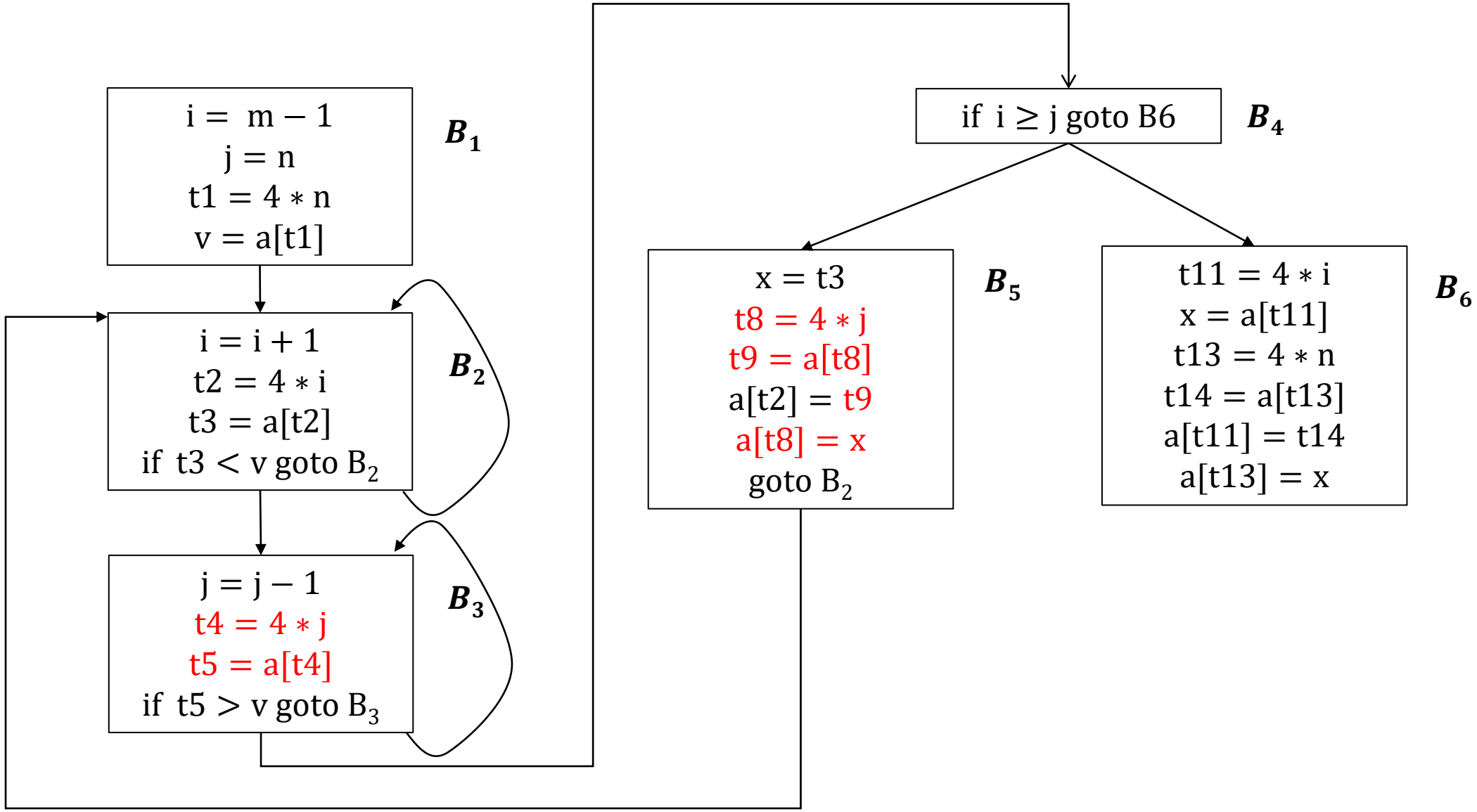


Global Common Subexpressions



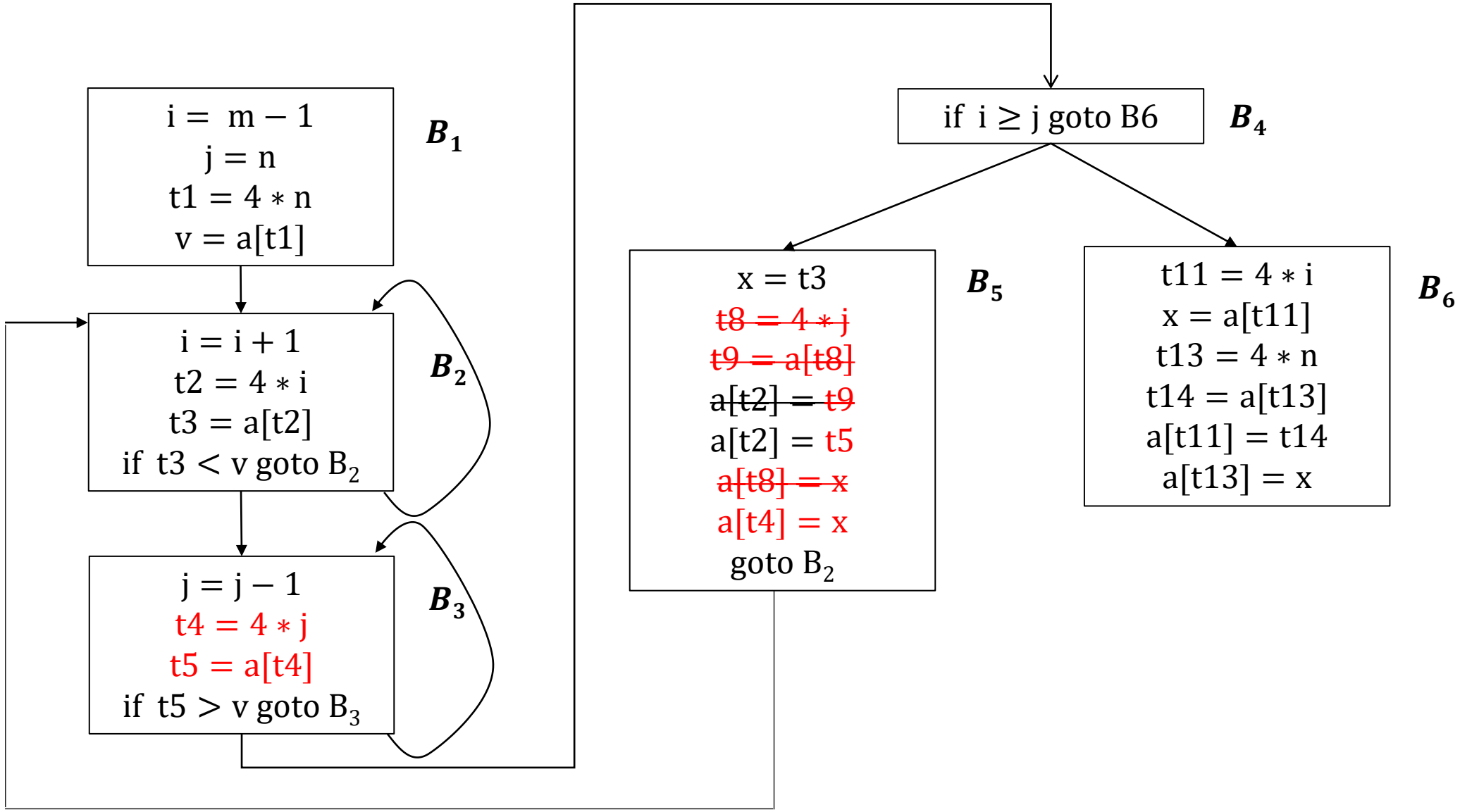
Global Common Subexpressions

j and t4 do not change when block B5 starts to execute

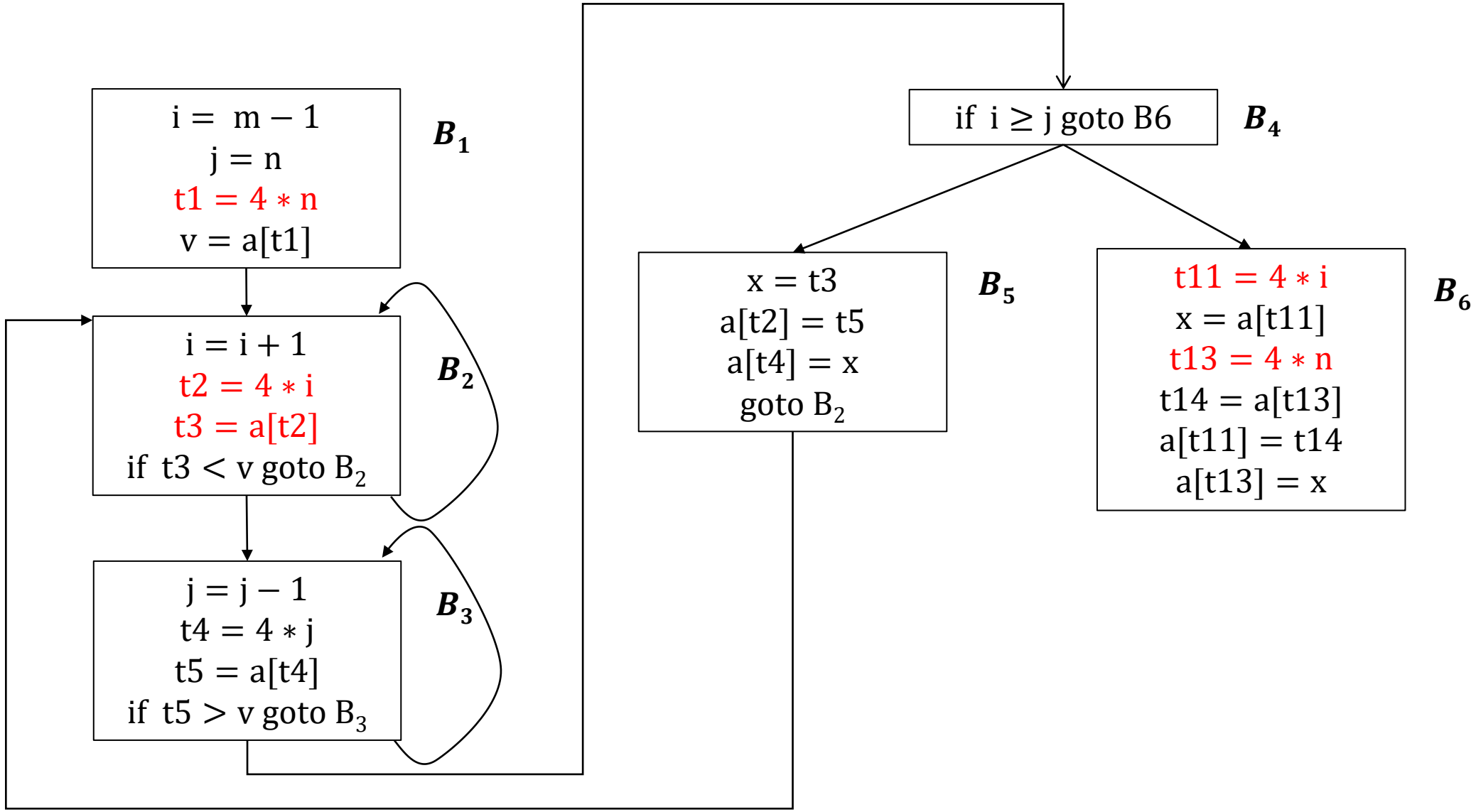


Global Common Subexpressions

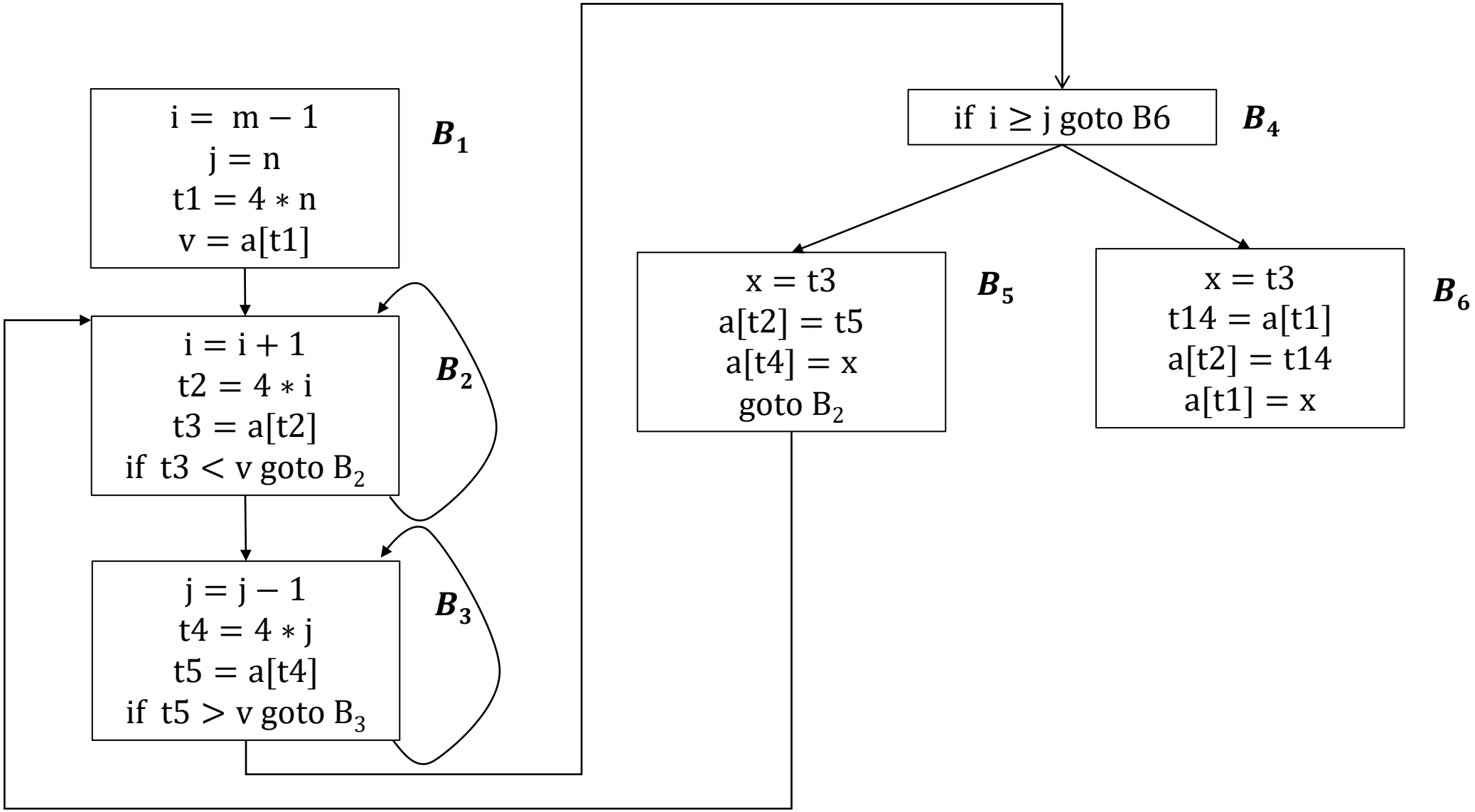
No need to calculate t8 and t8 is replaced with t4



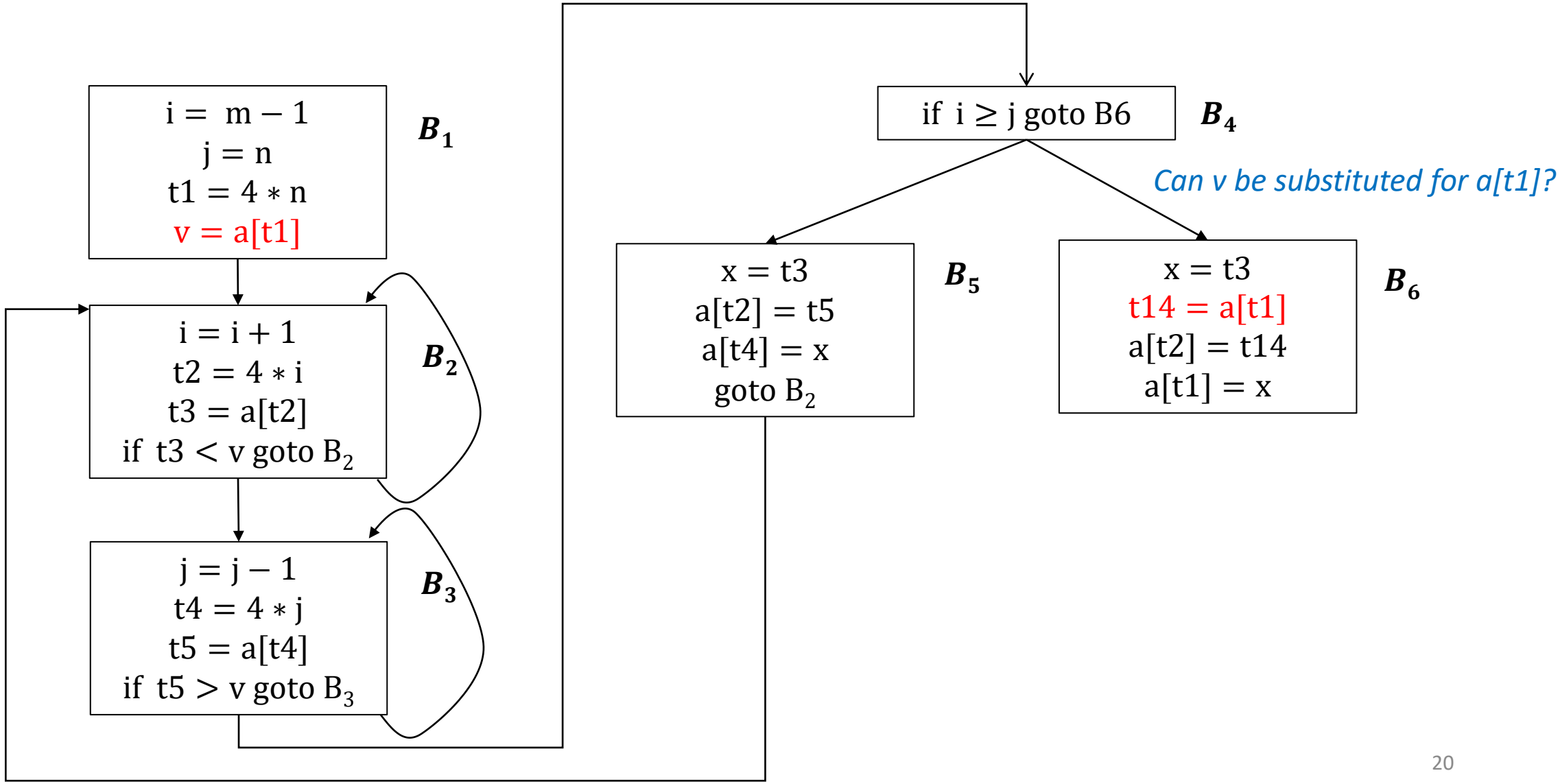
Global Common Subexpressions



Global Common Subexpressions

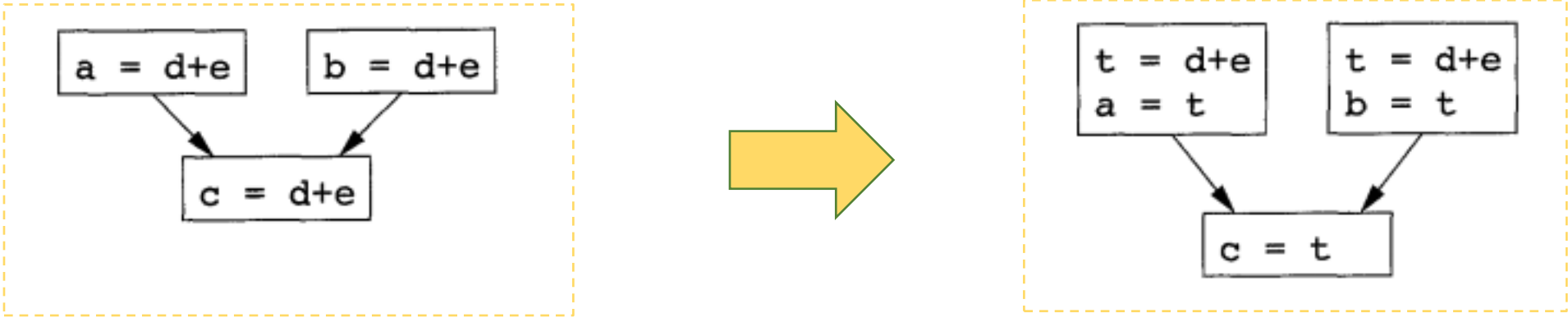


Global Common Subexpressions



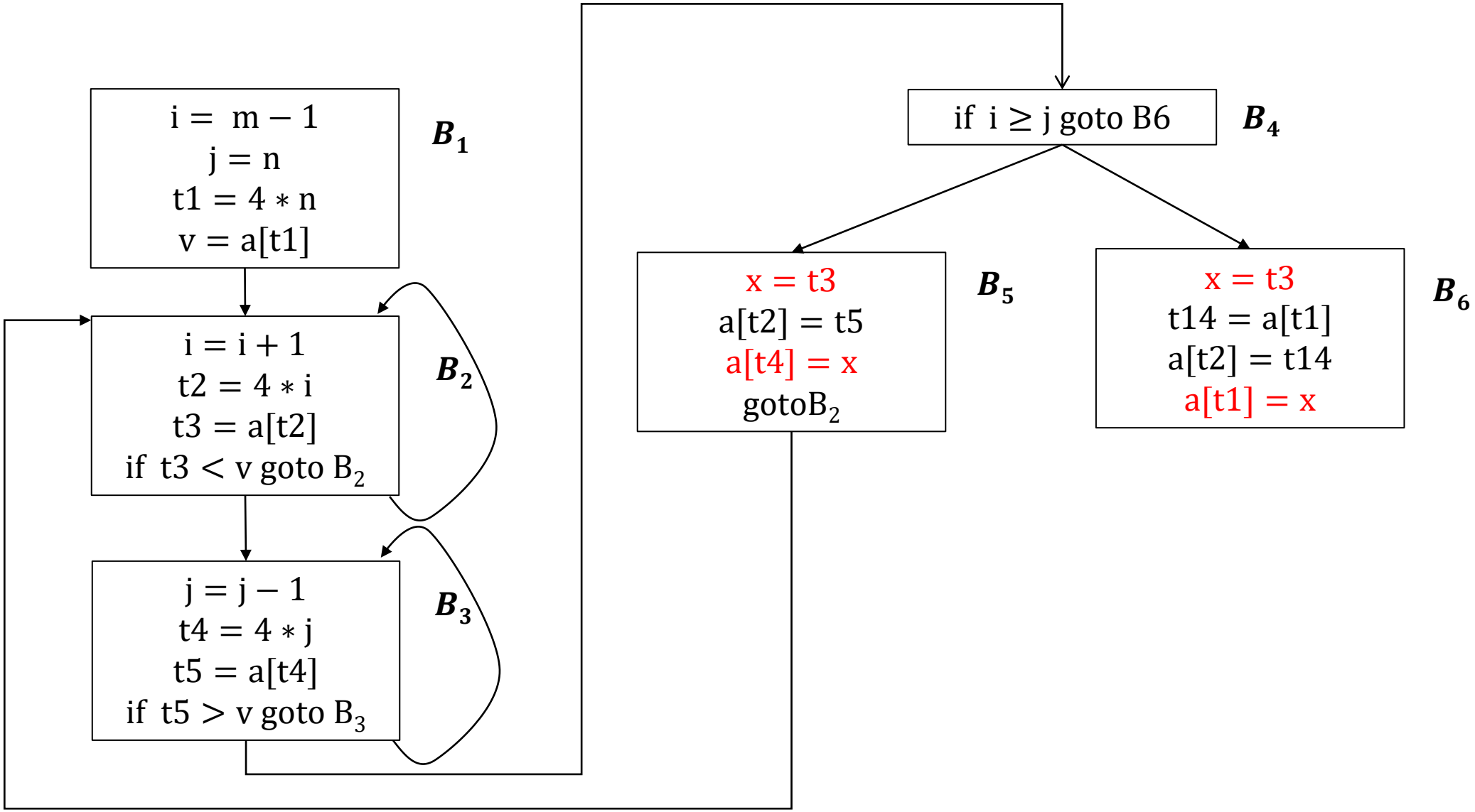
Copy Propagation

- Copies are introduced during common subexpression elimination

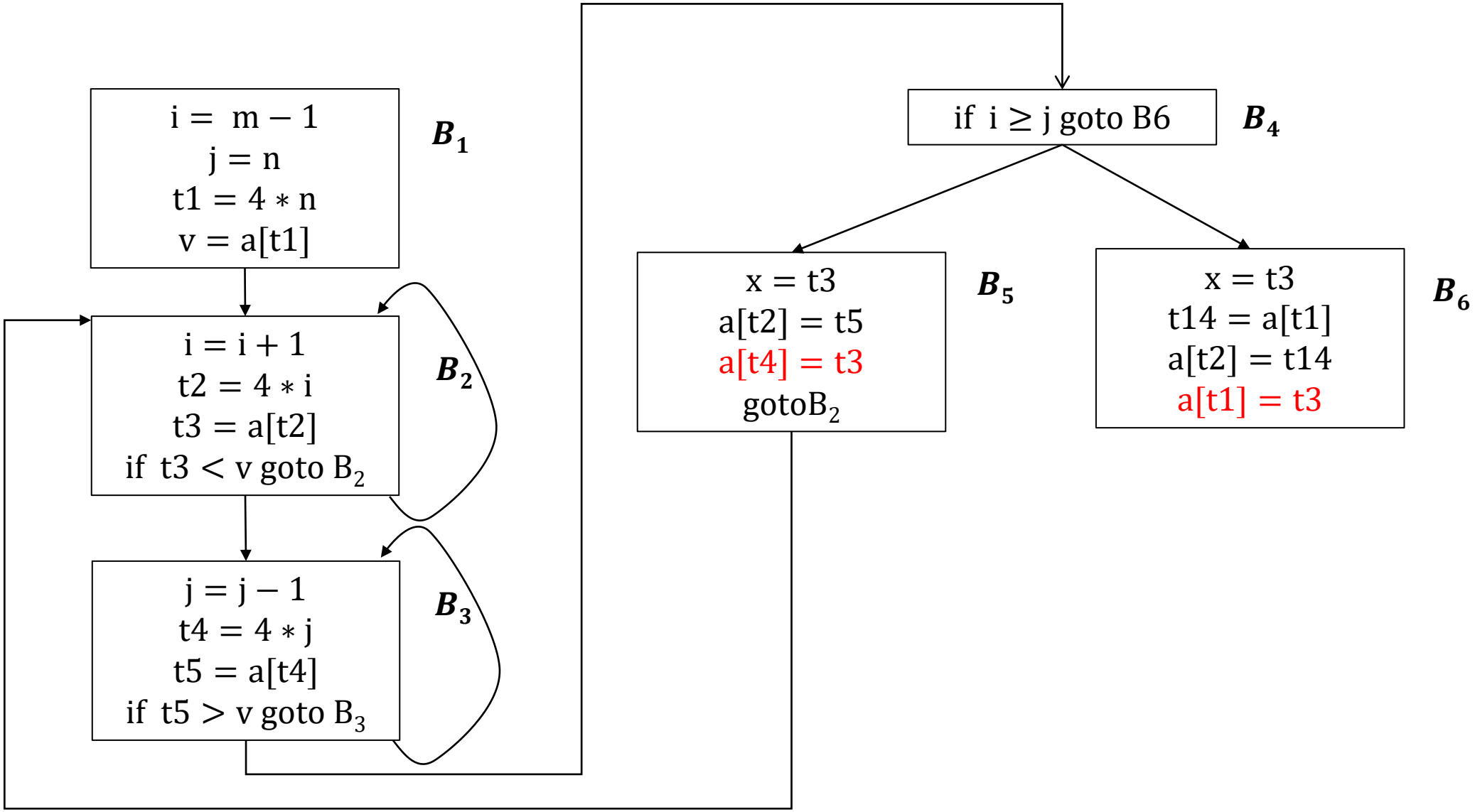


- The idea behind copy-propagation transformation is to use *t* for *c* whenever possible after the copy statement *c=t*

Copy Propagation



Copy Propagation



Dead Code Elimination

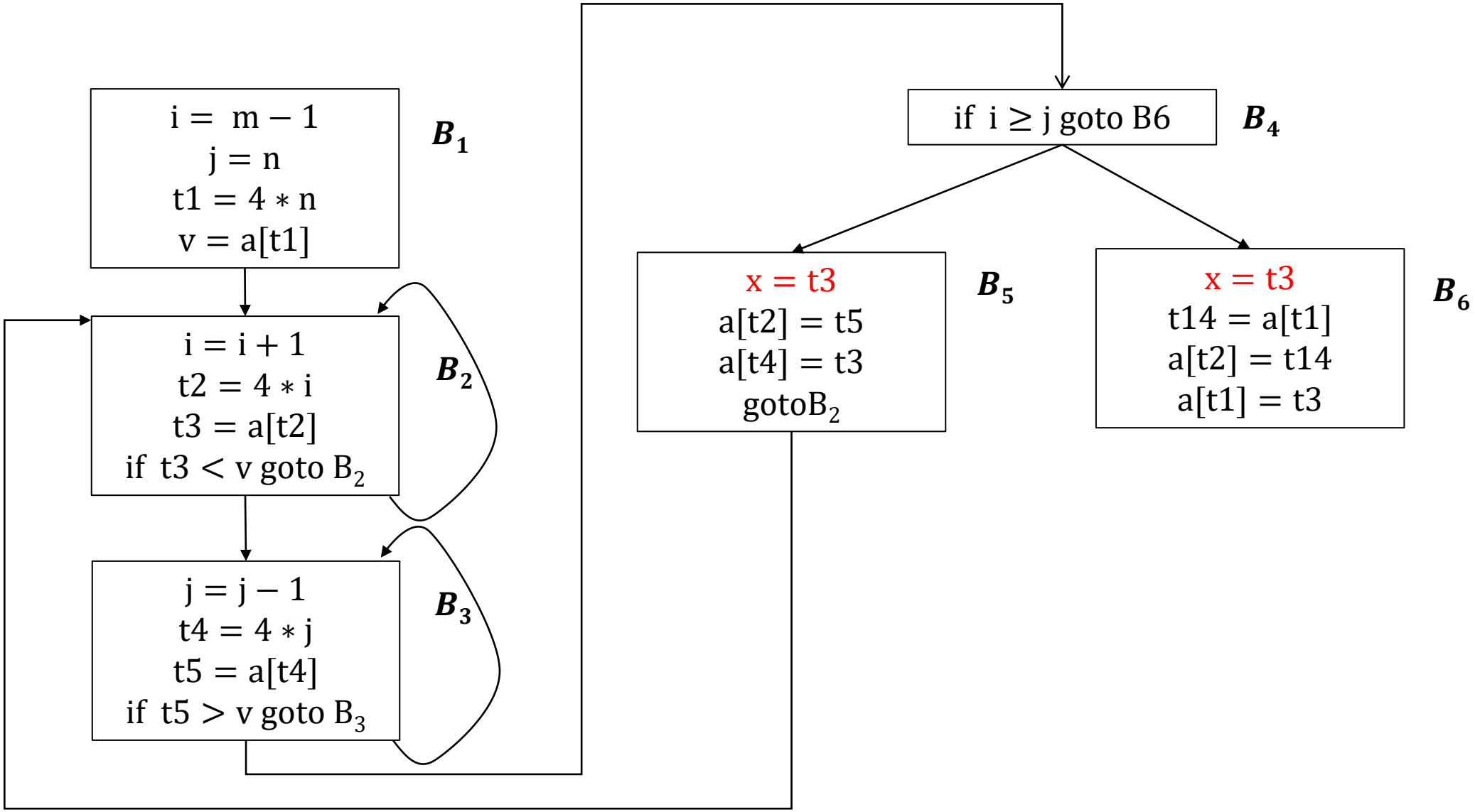
- *Dead Code*: Statements that computes values that never get used
- May result from various transformations, such as-
  - Constant folding: deducing at compile time that the value of an expression is a constant and using the constant instead
  - Copy propagation
- Example:

```
debug = false
… … …
if(debug) print…;
… … …
```

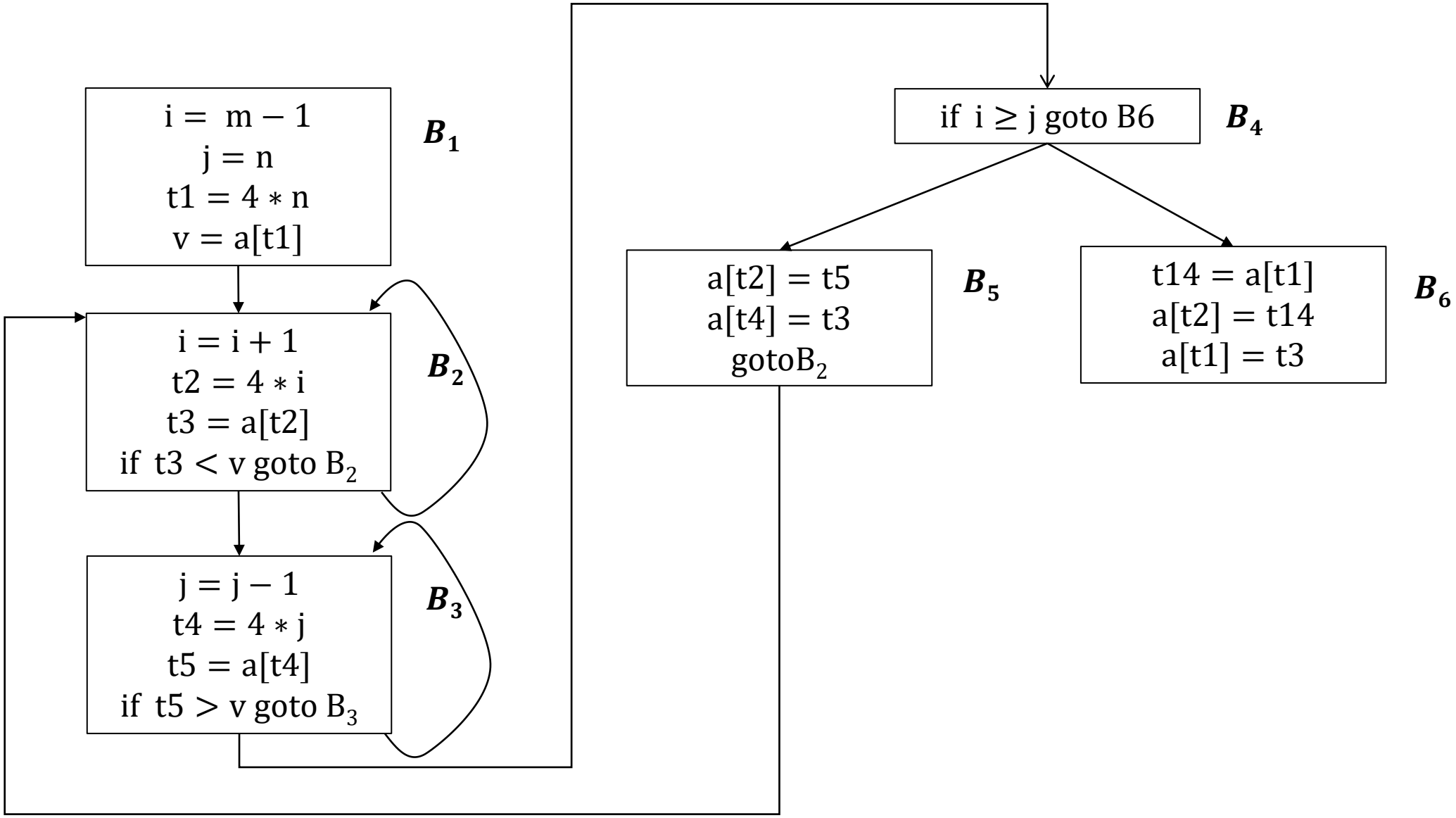
constant folding replaces debug with false

```
debug = false
… … …
if(false) print…;
… … …
```

Dead Code Elimination



Dead Code Elimination



Code Motion

- Reducing the amount of code in a loop may improve running time of a program significantly
  - Specially inner loops
- *Code Motion*
  - Moves loop-invariant computations out the loop and evaluates before the loop.
- Example:

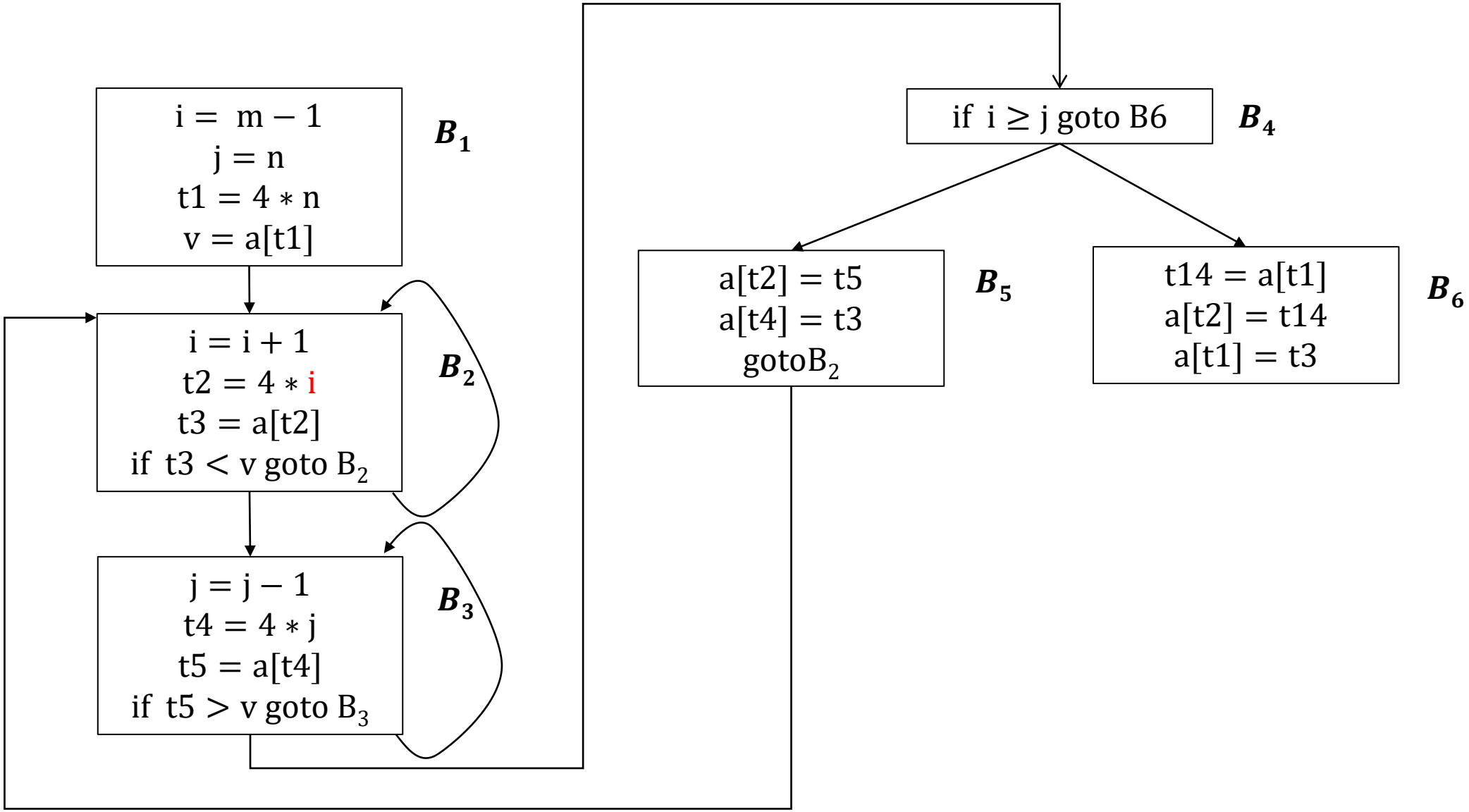
```
while (i <= limit-2) /* statement does not change limit */
```

```
t = limit-2
while (i <= t)  /* statement does not change limit or t */
```

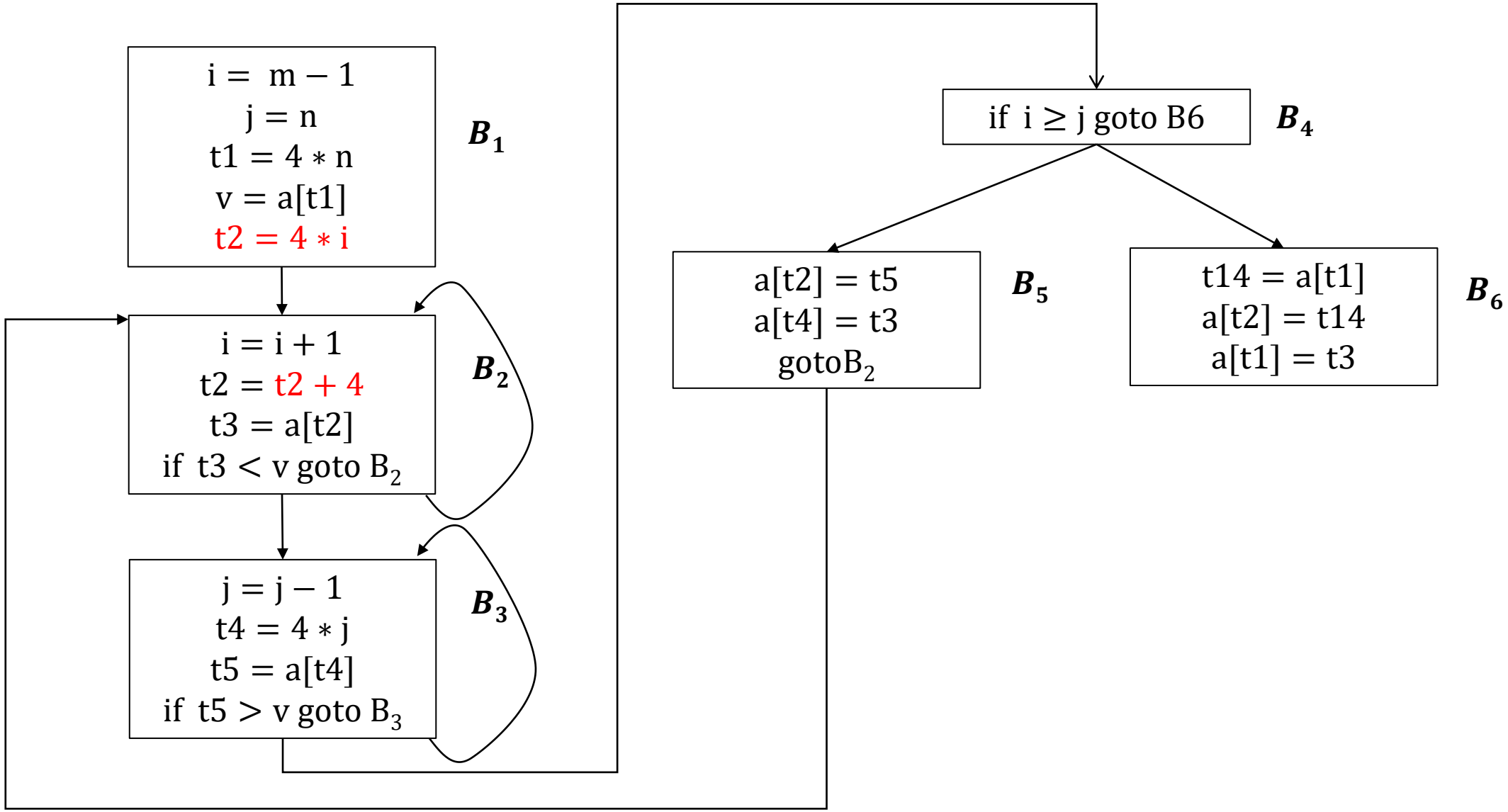
Induction Variables & Strength Reduction

- Induction Variables
  - A variable which is increased by a constant(+ve or –ve) *c*, every time the variable is assigned
- Strength Reduction
  - Replacing an expensive operation by a cheaper one
  - Multiplication by addition

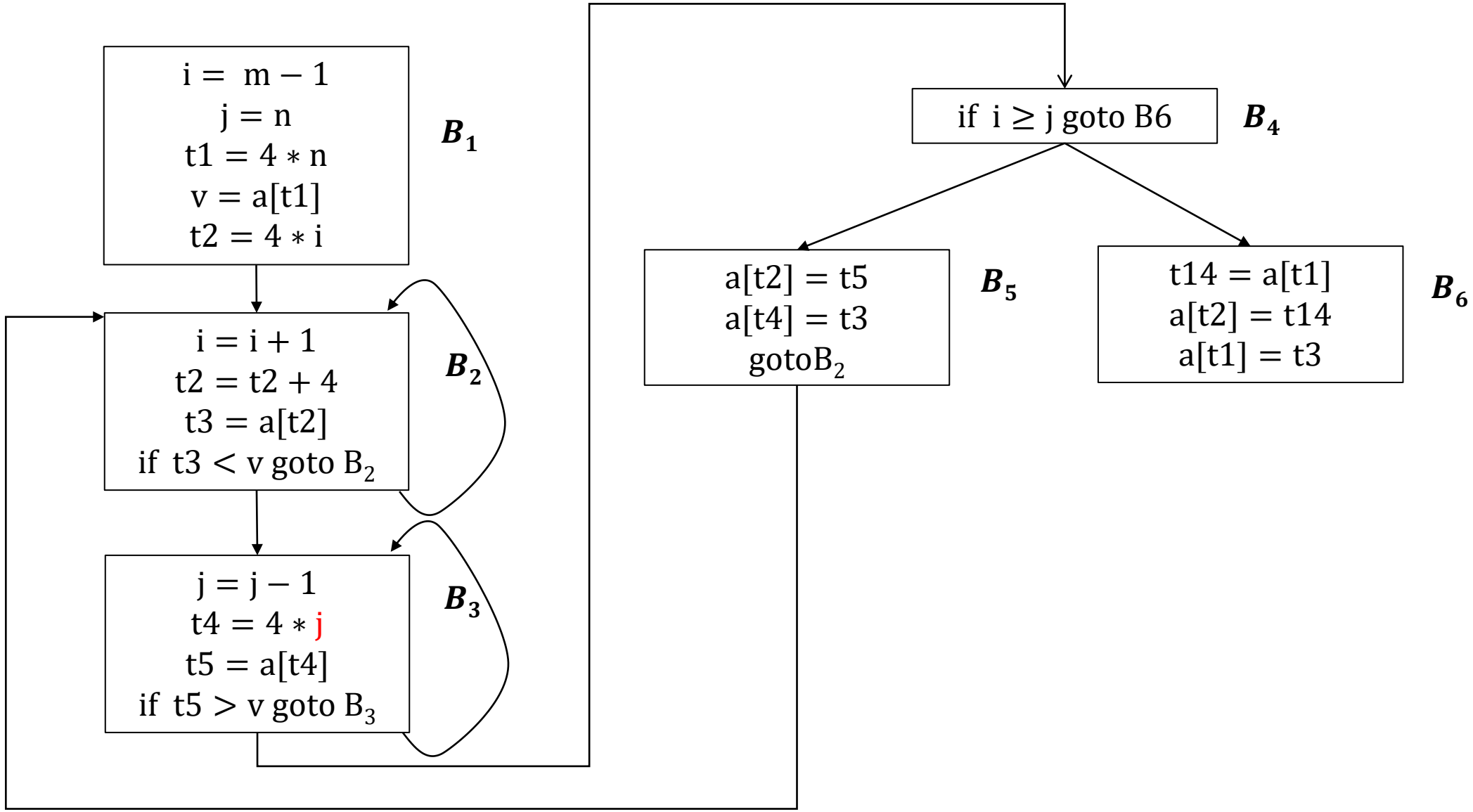
Induction Variables & Strength Reduction



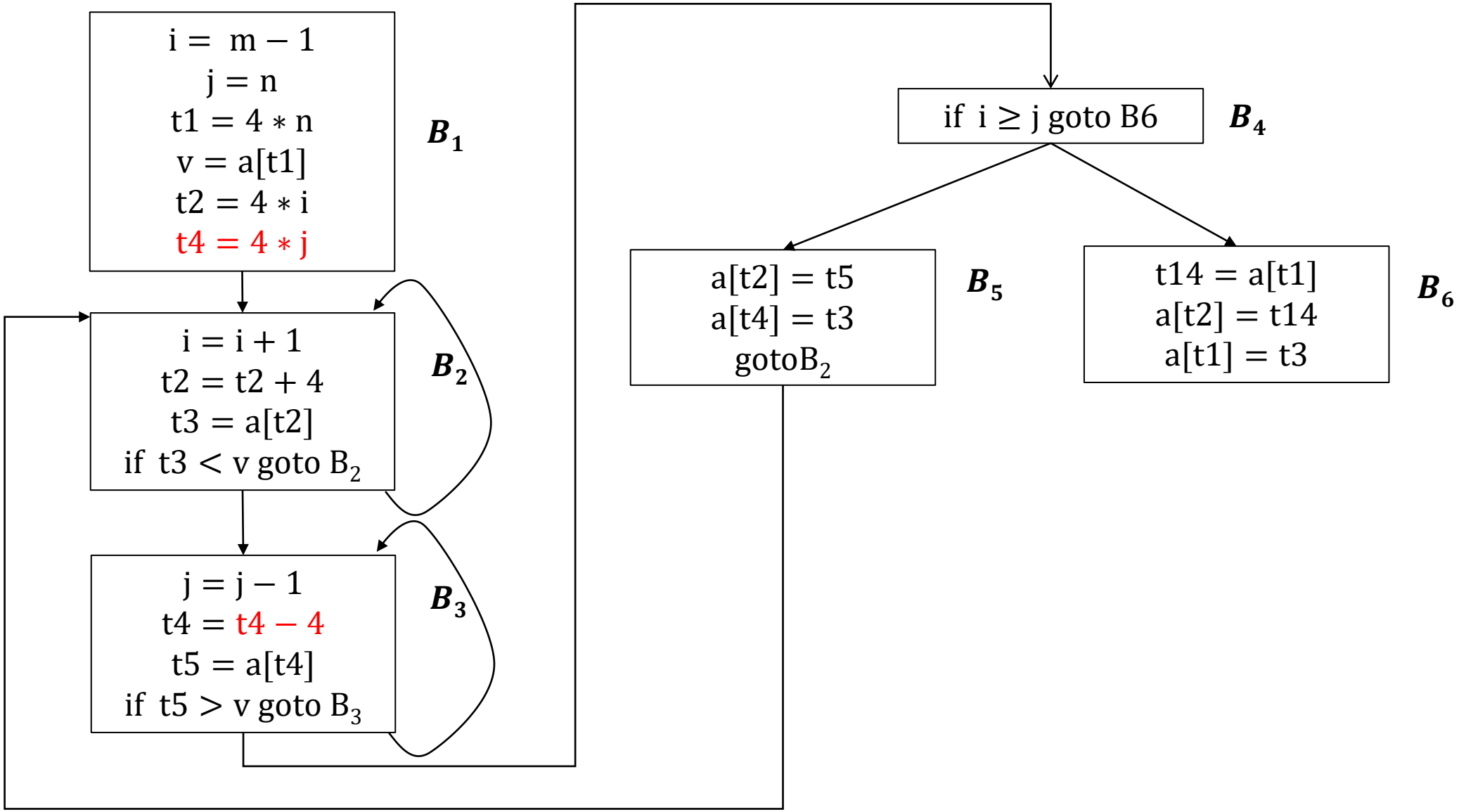
Induction Variables & Strength Reduction



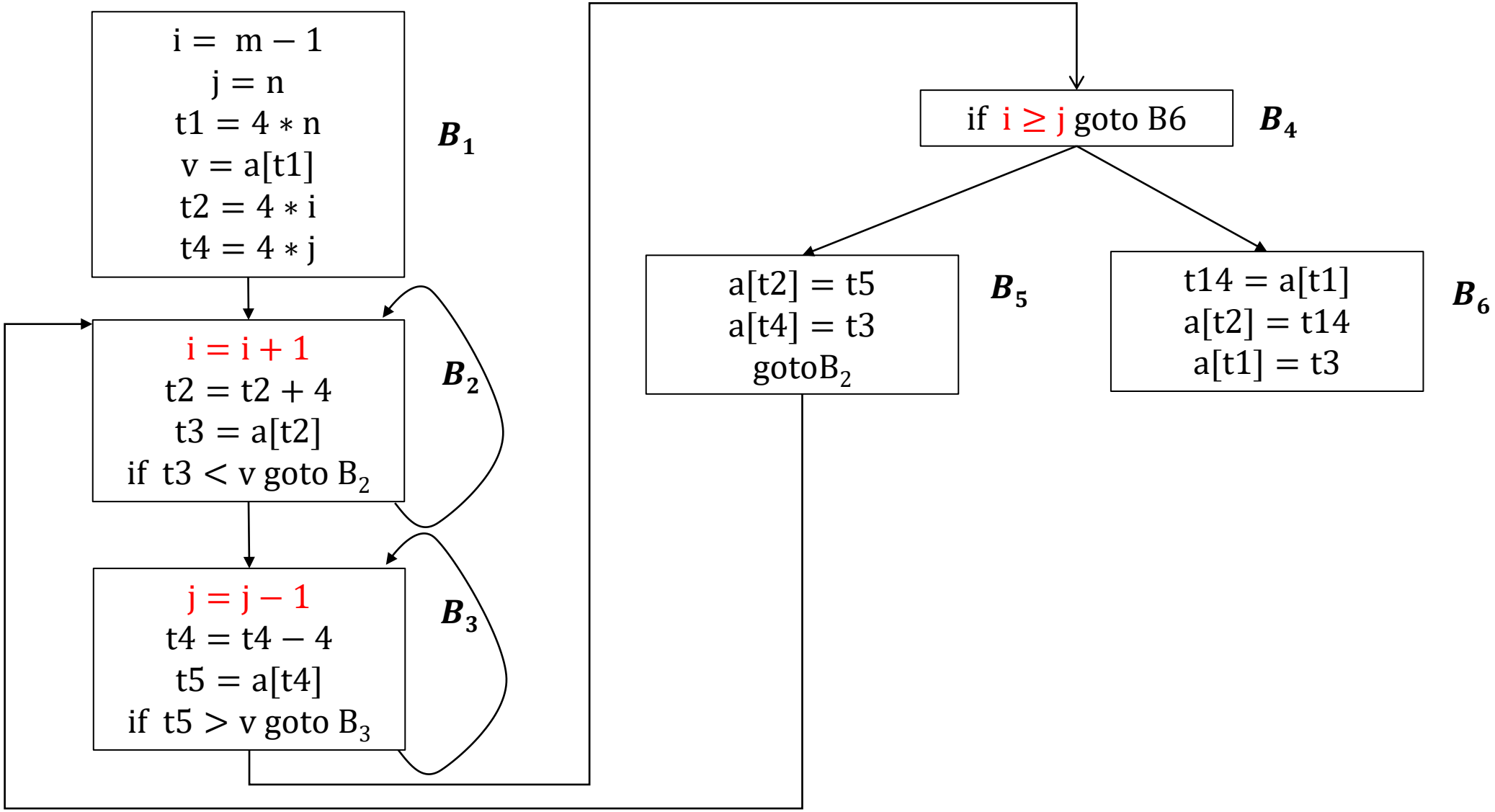
Induction Variables & Strength Reduction



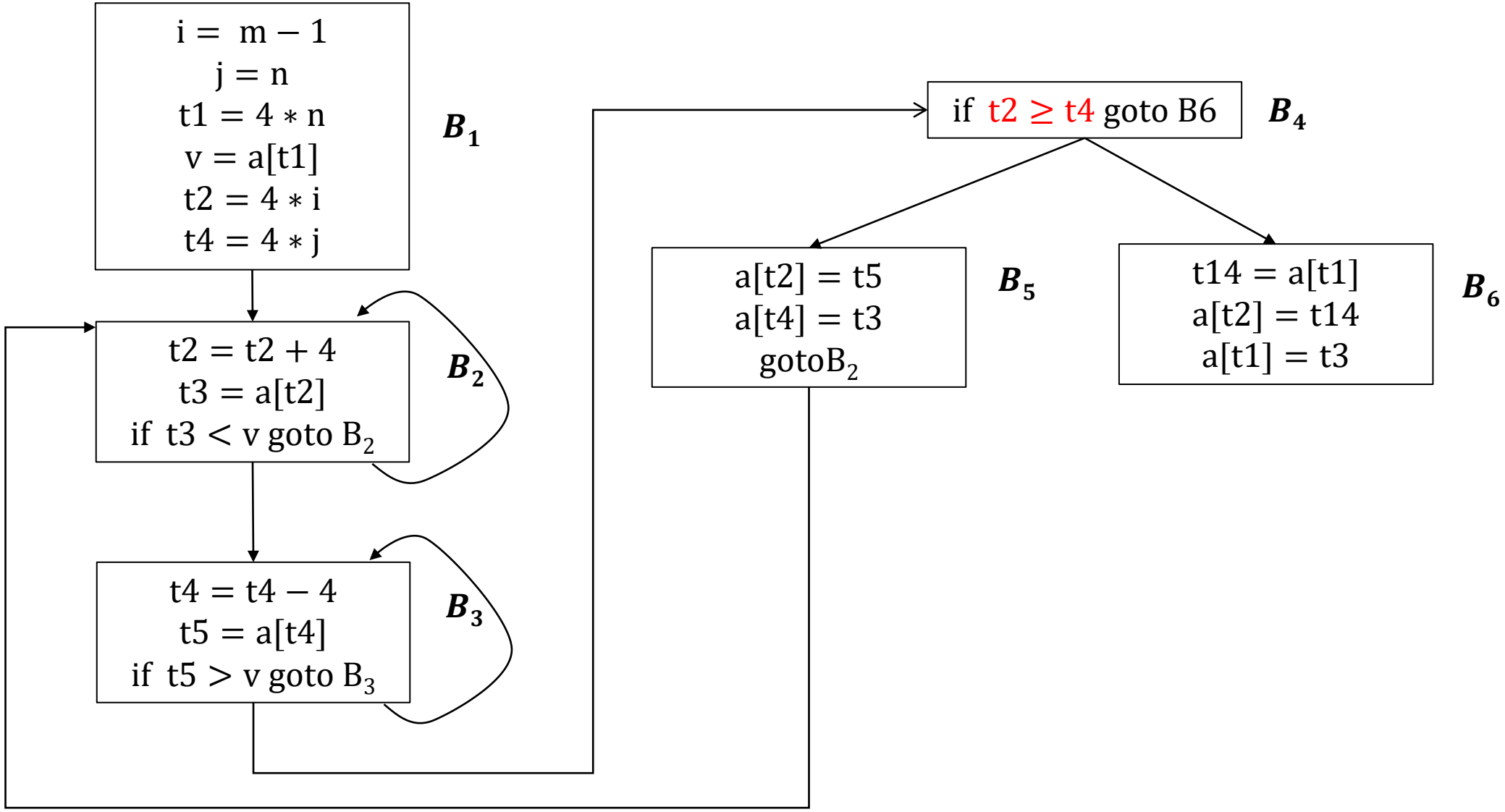
Induction Variables & Strength Reduction



Induction Variables & Strength Reduction



Induction Variables & Strength Reduction



Exercise

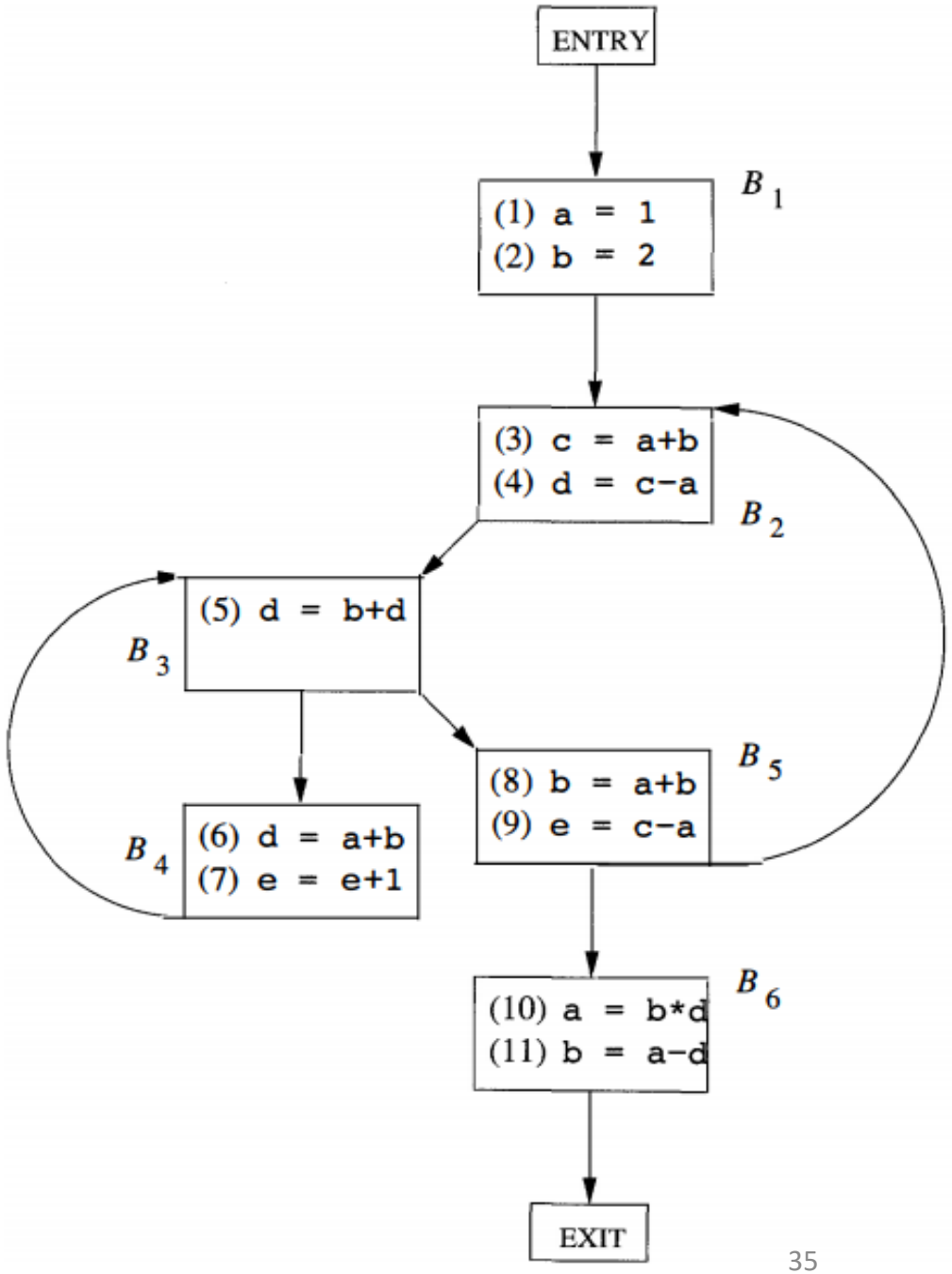
a) Identify the loops of the flow graph.

b) Statements (1) and (2) in B_1 are both copy statements, in which a and b are given constant values. For which uses of a and b can we perform copy propagation and replace these uses of variables by uses of a constant? Do so, wherever possible.

c) Identify any global common subexpressions for each loop.

d) Identify any induction variables for each loop. Be sure to take into account any constants introduced in (b).

e) Identify any loop-invariant computations for each loop.



Reference

- Compilers: Principles, Techniques, & Tools, by
  Alfred B. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman
  - Chapter 9
    - 9.1

Thank You!